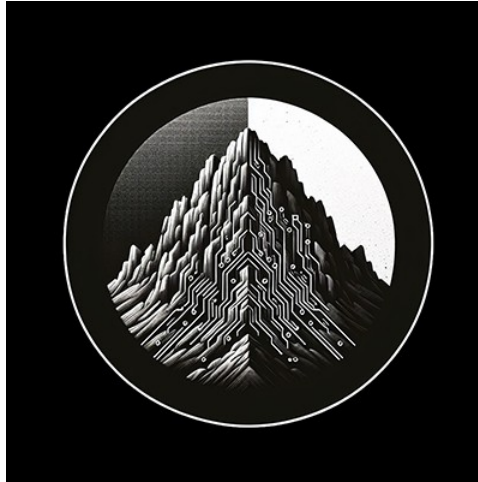




RTL Design Sherpa

RTL AMBA Monitor Subsystem Module Reference — Rev 0.90

July 2026

Table of Contents

1 AMBA4 Event Code Reference.....	63
1.1 AXI4 Event Codes.....	63
1.1.1 axi_error_code_t.....	63
1.1.2 axi_timeout_code_t.....	64
1.1.3 axi_completion_code_t.....	65
1.1.4 axi_threshold_code_t.....	65
1.1.5 axi_performance_code_t.....	66
1.1.6 axi_addr_match_code_t.....	67
1.1.7 axi_debug_code_t.....	67
1.2 APB4 Event Codes.....	68
1.2.1 apb_error_code_t.....	68
1.2.2 apb_timeout_code_t.....	69
1.2.3 apb_completion_code_t.....	69
1.2.4 apb_threshold_code_t.....	70
1.2.5 apb_performance_code_t.....	70
1.2.6 apb_debug_code_t.....	71
1.3 AXIS4 Event Codes.....	72
1.3.1 axis_error_code_t.....	72
1.3.2 axis_timeout_code_t.....	73
1.3.3 axis_completion_code_t.....	73
1.3.4 axis_credit_code_t.....	74
1.3.5 axis_channel_code_t.....	74

1.3.6 axis_stream_code_t.....	75
1.4 Unified Event Code Union.....	75
1.5 Related.....	76
1.6 Navigation.....	76
2 AMBA5 Extended Event Code Reference.....	76
2.1 AXI5 Extended Event Codes.....	76
2.1.1 axi5_atomic_code_t.....	77
2.1.2 axi5_trace_code_t.....	77
2.2 APB5 Extended Event Codes.....	78
2.2.1 apb5_wakeup_code_t.....	78
2.2.2 apb5_parity_code_t.....	79
2.2.3 apb5_user_code_t.....	79
2.3 AXIS5 Extended Event Codes.....	80
2.3.1 axis5_wakeup_code_t.....	80
2.3.2 axis5_parity_code_t.....	81
2.3.3 axis5_crc_code_t.....	81
2.4 Unified Event Code Union.....	82
2.5 Related.....	82
2.6 Navigation.....	82
3 ARB and CORE Event Code Reference.....	82
3.1 ARB Event Codes.....	83
3.1.1 arb_error_code_t.....	83
3.1.2 arb_timeout_code_t.....	83
3.1.3 arb_completion_code_t.....	84

3.1.4 arb_threshold_code_t.....	85
3.1.5 arb_performance_code_t.....	85
3.1.6 arb_debug_code_t.....	86
3.2 CORE Event Codes.....	87
3.2.1 core_error_code_t.....	87
3.2.2 core_timeout_code_t.....	88
3.2.3 core_completion_code_t.....	88
3.2.4 core_threshold_code_t.....	89
3.2.5 core_performance_code_t.....	90
3.2.6 core_debug_code_t.....	91
3.3 Unified Event Code Union (arb_core_event_code_t).....	91
3.4 Related.....	92
3.5 Navigation.....	92
4 Monitor Package Specification.....	92
4.1 Package Organization.....	93
4.2 The Monbus Record.....	93
4.2.1 SystemVerilog Type Aliases.....	95
4.3 Protocol Enum.....	95
4.4 Packet Type Enum.....	96
4.5 event_data Payload Conventions.....	97
4.6 Helper Functions.....	98
4.6.1 Construction Example.....	99
4.7 Side-Band Timestamp.....	99
4.8 Per-Protocol Event Code Packages.....	100

4.9 Backward Compatibility.....	101
4.10 Related Documentation.....	102
4.11 Navigation.....	102
5 APB5 Monitor.....	102
5.1 Overview.....	102
5.1.1 Key Features.....	102
5.2 Parameters.....	103
5.3 Ports.....	104
5.3.1 Clock and Reset.....	104
5.3.2 Command Interface Monitoring.....	105
5.3.3 Response Interface Monitoring.....	105
5.3.4 APB5 Extension Monitoring.....	106
5.3.5 Configuration Inputs.....	106
5.3.6 Address-Range Checker Configuration.....	107
5.3.7 Monitor Bus Output.....	108
5.3.8 Status Outputs.....	108
5.4 Monitor Packet Format.....	108
5.4.1 64-bit Packet Structure.....	108
5.4.2 Packet Types.....	109
5.4.3 APB5-Specific Event Codes.....	110
5.5 Event Priority.....	111
5.6 Wake-up Monitoring.....	112
5.6.1 Wake-up Timeout.....	112
5.7 Usage Example.....	112

5.8 Configuration Guidelines.....	114
5.8.1 Recommended Configurations.....	114
5.9 Design Notes.....	114
5.9.1 Transaction Table.....	114
5.9.2 Internal FIFOs.....	114
5.10 Related Documentation.....	115
5.11 Navigation.....	115
6 APB Monitor Address-Range Checker.....	115
6.1 Overview.....	115
6.1.1 Key Features.....	115
6.2 Module Purpose.....	116
6.3 Parameters.....	116
6.4 Port Groups.....	116
6.4.1 Clock and Reset.....	116
6.4.2 Timestamp.....	117
6.4.3 Snooped APB Command Stream.....	117
6.4.4 Range Configuration.....	117
6.4.5 Outgoing MonBus Packet.....	118
6.5 Functional Description.....	118
6.5.1 Combinational Range Hits.....	118
6.5.2 Pending Mask and Latched Snapshot.....	118
6.5.3 Packet Encoding.....	118
6.5.4 The is_read Carve-Out.....	119
6.5.5 Timestamp Side-Band.....	119

6.6 Usage Example.....	119
6.7 Design Notes.....	120
6.7.1 Mirror of the AXI Variant.....	120
6.7.2 Backpressure Safety.....	120
6.7.3 Address Field Width.....	120
6.7.4 Exact-Match Watchpoints.....	120
6.8 Related Modules.....	121
6.8.1 Used By.....	121
6.8.2 Uses.....	121
6.8.3 See Also.....	121
6.9 References.....	121
6.9.1 Source Code.....	121
6.9.2 Documentation.....	121
6.10 Navigation.....	121
7 apb_monitor.....	122
7.1 Overview.....	122
7.2 Module Declaration.....	122
7.3 Parameters.....	124
7.4 Ports.....	125
7.4.1 Clock and Reset.....	125
7.4.2 Command Interface Monitoring.....	125
7.4.3 Response Interface Monitoring.....	125
7.4.4 Configuration - Error Detection.....	126
7.4.5 Configuration - Performance Monitoring.....	126

7.4.6 Configuration - Debug.....	126
7.4.7 Configuration - Thresholds.....	127
7.4.8 Monitor Bus Interface.....	127
7.4.9 Status Outputs.....	127
7.5 Functional Description.....	128
7.5.1 Transaction Monitoring.....	128
7.5.2 Event Detection.....	128
7.5.3 Monitor Packet Format.....	129
7.6 Configuration Strategies.....	129
7.6.1 Functional Verification (Recommended).....	129
7.6.2 Performance Analysis.....	129
7.6.3 Debug Mode.....	130
7.7 Usage Example.....	130
7.8 Design Notes.....	132
7.8.1 Transaction Tracking.....	132
7.8.2 Packet Generation.....	132
7.8.3 Performance Considerations.....	132
7.9 Related Modules.....	132
7.10 References.....	132
7.11 Navigation.....	132
8 Monitor Bus Arbiter Common.....	133
8.1 Overview.....	133
8.2 Key Features.....	133
8.3 Module Purpose.....	133

8.4 Parameters.....	134
8.5 Port Groups.....	134
8.6 Usage in Monitor System.....	135
8.6.1 Internal Integration.....	135
8.7 Configuration Guidelines.....	136
8.8 Performance Characteristics.....	136
8.9 Verification Considerations.....	136
8.9.1 Test Coverage.....	136
8.10 Related Modules.....	136
8.11 See Also.....	136
8.12 Navigation.....	137
9 Round-Robin Arbiter with PWM and Monitor Bus.....	137
9.1 Overview.....	137
9.1.1 Key Features.....	137
9.2 Module Purpose.....	137
9.3 Parameters.....	138
9.3.1 Fixed Internal Configurations (Not User-Configurable).....	138
9.4 Port Groups.....	139
9.4.1 Clock and Reset.....	139
9.4.2 Arbiter Interface.....	139
9.4.3 PWM Configuration.....	139
9.4.4 Monitor Configuration.....	140
9.4.5 Monitor Bus Output.....	140
9.4.6 Debug Outputs.....	141

9.5 Functional Description.....	141
9.5.1 Round-Robin Arbiter.....	141
9.5.2 PWM Generator.....	141
9.5.3 Monitor Bus Common.....	141
9.5.4 Operation Flow.....	141
9.6 Usage Example.....	142
9.7 Design Notes.....	143
9.7.1 PWM Control Strategy.....	143
9.7.2 Fixed Configuration Benefits.....	143
9.7.3 Fairness in Round-Robin Mode.....	143
9.7.4 Monitor Bus Integration.....	144
9.8 Related Modules.....	144
9.8.1 Used By.....	144
9.8.2 Uses.....	144
9.8.3 Related Modules.....	144
9.9 References.....	144
9.9.1 Specifications.....	144
9.9.2 Source Code.....	144
9.10 Navigation.....	145
10 Weighted Round-Robin Arbiter with PWM and Monitor Bus.....	145
10.1 Overview.....	145
10.1.1 Key Features.....	145
10.2 Module Purpose.....	145
10.3 Parameters.....	146

10.3.1 Fixed Internal Configurations (Not User-Configurable).....	146
10.4 Port Groups.....	147
10.4.1 Clock and Reset.....	147
10.4.2 Arbiter Configuration and Interface.....	147
10.4.3 PWM Configuration.....	147
10.4.4 Monitor Configuration.....	148
10.4.5 Monitor Bus Output.....	149
10.4.6 Enhanced Debug Outputs.....	149
10.5 Functional Description.....	150
10.5.1 Weighted Round-Robin Arbiter.....	150
10.5.2 PWM Generator.....	150
10.5.3 Monitor Bus Common.....	150
10.5.4 Operation Flow.....	151
10.6 Usage Example.....	151
10.7 Design Notes.....	153
10.7.1 Weight Configuration.....	153
10.7.2 Fairness Deviation Threshold.....	153
10.7.3 Enhanced Debug Outputs.....	153
10.7.4 ACK Mode with Weighted Arbitration.....	154
10.8 Related Modules.....	154
10.8.1 Used By.....	154
10.8.2 Uses.....	154
10.8.3 Related Modules.....	154
10.9 References.....	154

10.9.1 Specifications.....	154
10.9.2 Source Code.....	154
10.10 Navigation.....	155
11 AXI4 Master Read Monitor (Clock-Gated).....	155
11.1 Overview.....	155
11.1.1 Key Differences from Base Module.....	155
11.2 When to Use Clock-Gated Variant.....	155
11.3 Clock Gating Parameters.....	156
11.3.1 Base Module Parameters.....	156
11.3.2 Parameter Relationships.....	157
11.4 Usage Examples.....	157
11.4.1 Example 1: Maximum Power Savings (Burst Traffic).....	157
11.4.2 Example 2: Balanced Performance and Power.....	157
11.4.3 Example 3: Clock Gating Disabled (Functional Verification).....	158
11.5 Performance Monitoring.....	158
11.6 Clock Gating Architecture.....	159
11.6.1 Gating Domains.....	159
11.6.2 Gating State Machine.....	159
11.6.3 Wake-Up Latency.....	160
11.7 Power Savings Analysis.....	160
11.7.1 Typical Power Reduction.....	160
11.7.2 Power Monitoring Signals.....	160
11.8 Verification Considerations.....	161
11.8.1 Clock Gating in Simulation.....	161

11.8.2 Power Analysis Verification.....	161
11.9 Synthesis Considerations.....	161
11.9.1 FPGA Implementations.....	161
11.9.2 ASIC Implementations.....	161
11.10 Related Modules.....	162
11.11 See Also.....	162
11.12 Navigation.....	162
12 AXI4 Master Read Monitor.....	162
12.1 Overview.....	162
12.1.1 Key Features.....	162
12.2 Parameters.....	164
12.2.1 AXI4 Master Parameters.....	164
12.2.2 Monitor Parameters.....	164
12.2.3 Synthesis-Cone Parameters.....	165
12.3 Monitor Backpressure (block_ready).....	166
12.4 Address-Range Checker.....	166
12.5 Performance Monitoring.....	167
12.5.1 The Measurement Window.....	167
12.5.2 Utilization Counters (R Data Channel).....	167
12.5.3 Throughput Counters.....	168
12.5.4 Performance Monitoring Ports.....	168
12.6 Port Groups.....	169
12.6.1 AXI4 Interfaces.....	169
12.6.2 Monitor Configuration.....	169

12.6.3 Monitor Bus Output.....	171
12.6.4 Status Outputs.....	171
12.7 Monitor Packet Format.....	172
12.8 Timing Diagrams.....	172
12.8.1 Scenario 1: Single-Beat Read Transaction.....	172
12.8.2 Scenario 2: AR Channel Handshake Detail.....	173
12.8.3 Scenario 3: R Response Channel Detail.....	173
12.8.4 Scenario 4: Complete AXI4 Read Transaction.....	174
12.8.5 Scenario 5: Monitor Bus Packet Generation.....	174
12.8.6 Scenario 6: Simple ARVALID Transaction.....	174
12.8.7 Scenario 7: Alternative Single-Beat Read.....	175
12.9 Configuration Strategies.....	175
12.9.1 Strategy 1: Functional Verification (Recommended).....	175
12.9.2 Strategy 2: Performance Analysis.....	176
12.9.3 Strategy 3: Debug Mode.....	176
12.10 Usage Example.....	177
12.10.1 Basic Integration.....	177
12.11 Design Notes.....	179
12.11.1 Filtering Hierarchy.....	179
12.11.2 Configuration Conflicts.....	180
12.11.3 Performance Considerations.....	180
12.11.4 Transaction Table.....	180
12.12 Related Modules.....	180
12.12.1 Companion Monitors.....	180

12.12.2 Base Modules.....	180
12.12.3 Used Components.....	180
12.13 References.....	181
12.13.1 Specifications.....	181
12.13.2 Source Code.....	181
12.13.3 Documentation.....	181
12.14 Navigation.....	181
13 AXI4 Master Write Monitor (Clock-Gated).....	181
13.1 Quick Reference.....	181
13.2 Summary.....	182
13.3 Common Parameters.....	182
13.4 Performance Monitoring.....	183
13.5 Quick Usage.....	183
13.6 Documentation.....	183
13.7 Navigation.....	184
14 AXI4 Master Write Monitor.....	184
14.1 Overview.....	184
14.1.1 Key Features.....	184
14.2 Parameters.....	185
14.2.1 AXI4 Master Parameters.....	185
14.2.2 Monitor Parameters.....	186
14.2.3 Synthesis-Cone Parameters.....	187
14.3 Monitor Backpressure (block_ready).....	188
14.4 Address-Range Checker.....	188

14.5 Performance Monitoring.....	189
14.5.1 The Measurement Window.....	189
14.5.2 Utilization Counters (W Data Channel).....	189
14.5.3 Throughput Counters.....	190
14.5.4 Performance Monitoring Ports.....	190
14.6 Port Groups.....	191
14.6.1 AXI4 Write Channels.....	191
14.6.2 Monitor Configuration.....	191
14.6.3 Monitor Bus Output.....	192
14.6.4 Status Outputs.....	192
14.7 Monitor Packet Format.....	193
14.8 Timing Diagrams.....	193
14.8.1 Scenario 1: Single-Beat Write Transaction.....	193
14.8.2 Scenario 2: Alternative Single-Beat Write.....	194
14.9 Configuration Strategies.....	194
14.9.1 Strategy 1: Functional Verification (Recommended).....	194
14.9.2 Strategy 2: Performance Analysis.....	194
14.9.3 Strategy 3: Debug Mode.....	195
14.10 Usage Example.....	195
14.10.1 Basic Integration.....	195
14.11 Design Notes.....	197
14.11.1 Write-Specific Monitoring.....	197
14.11.2 Performance Considerations.....	197
14.11.3 Buffer Depth Guidelines.....	197

14.12 Related Modules.....	197
14.12.1 Companion Monitors.....	197
14.12.2 Base Modules.....	197
14.12.3 Used Components.....	198
14.13 References.....	198
14.13.1 Specifications.....	198
14.13.2 Source Code.....	198
14.13.3 Documentation.....	198
14.14 Navigation.....	198
15 AXI4 Slave Read Monitor (Clock-Gated).....	198
15.1 Quick Reference.....	199
15.2 Summary.....	199
15.3 Common Parameters.....	199
15.4 Performance Monitoring.....	200
15.5 Quick Usage.....	200
15.6 Documentation.....	201
15.7 Navigation.....	201
16 AXI4 Slave Read Monitor.....	201
16.1 Overview.....	201
16.1.1 Key Features.....	201
16.2 Parameters.....	202
16.2.1 AXI4 Slave Parameters.....	202
16.2.2 Monitor Parameters.....	203
16.2.3 Synthesis-Cone Parameters.....	204

16.3 Monitor Backpressure (block_ready).....	205
16.4 Address-Range Checker.....	205
16.5 Performance Monitoring.....	206
16.5.1 The Measurement Window.....	206
16.5.2 Utilization Counters (R Data Channel).....	206
16.5.3 Throughput Counters.....	207
16.5.4 Performance Monitoring Ports.....	207
16.6 Port Groups.....	208
16.6.1 AXI4 Read Channels.....	208
16.6.2 Monitor Configuration.....	208
16.6.3 Monitor Bus Output.....	209
16.6.4 Status Outputs.....	209
16.7 Monitor Packet Format.....	210
16.8 Timing Diagrams.....	210
16.8.1 Scenario 1: Single-Beat Read (Slave View).....	210
16.8.2 Scenario 2: Alternative Single-Beat Read (Slave View).....	210
16.9 Configuration Strategies.....	211
16.9.1 Strategy 1: Functional Verification (Recommended).....	211
16.9.2 Strategy 2: Performance Analysis.....	211
16.10 Usage Example.....	211
16.10.1 Basic Integration with Memory.....	211
16.11 Design Notes.....	213
16.11.1 Slave-Side Monitoring.....	213
16.11.2 Performance Considerations.....	213

16.11.3 Buffer Depth Guidelines.....	213
16.12 Related Modules.....	214
16.12.1 Companion Monitors.....	214
16.12.2 Base Modules.....	214
16.12.3 Used Components.....	214
16.13 References.....	214
16.13.1 Specifications.....	214
16.13.2 Source Code.....	214
16.13.3 Documentation.....	214
16.14 Navigation.....	215
17 AXI4 Slave Write Monitor (Clock-Gated).....	215
17.1 Quick Reference.....	215
17.2 Summary.....	215
17.3 Common Parameters.....	215
17.4 Performance Monitoring.....	216
17.5 Quick Usage.....	217
17.6 Documentation.....	217
17.7 Navigation.....	217
18 AXI4 Slave Write Monitor.....	218
18.1 Overview.....	218
18.1.1 Key Features.....	218
18.2 Parameters.....	219
18.2.1 AXI4 Slave Parameters.....	219
18.2.2 Monitor Parameters.....	220

18.2.3 Synthesis-Cone Parameters.....	221
18.3 Monitor Backpressure (block_ready).....	221
18.4 Address-Range Checker.....	222
18.5 Performance Monitoring.....	222
18.5.1 The Measurement Window.....	223
18.5.2 Utilization Counters (W Data Channel).....	223
18.5.3 Throughput Counters.....	223
18.5.4 Performance Monitoring Ports.....	224
18.6 Port Groups.....	225
18.6.1 AXI4 Write Channels.....	225
18.6.2 Monitor Configuration.....	225
18.6.3 Monitor Bus Output.....	225
18.6.4 Status Outputs.....	226
18.7 Monitor Packet Format.....	226
18.8 Timing Diagrams.....	226
18.8.1 Scenario 1: Single-Beat Write (Slave View).....	227
18.8.2 Scenario 2: Alternative Single-Beat Write (Slave View).....	227
18.9 Configuration Strategies.....	227
18.9.1 Strategy 1: Functional Verification (Recommended).....	227
18.9.2 Strategy 2: Performance Analysis.....	228
18.10 Usage Example.....	228
18.10.1 Basic Integration with Memory.....	228
18.11 Design Notes.....	230
18.11.1 Slave-Side Monitoring.....	230

18.11.2 Performance Considerations.....	230
18.11.3 Buffer Depth Guidelines.....	230
18.11.4 Write Transaction Characteristics.....	231
18.12 Related Modules.....	231
18.12.1 Companion Monitors.....	231
18.12.2 Base Modules.....	231
18.12.3 Used Components.....	231
18.13 References.....	231
18.13.1 Specifications.....	231
18.13.2 Source Code.....	231
18.13.3 Documentation.....	232
18.14 Navigation.....	232
19 AXI5 Master Read with Monitor and Clock Gating.....	232
19.1 Overview.....	232
19.1.1 Key Features.....	232
19.2 Parameters.....	234
19.3 Ports.....	236
19.3.1 Clock and Reset.....	236
19.3.2 Clock Gating Configuration.....	236
19.3.3 FUB AXI5 Interface (Slave Side).....	236
19.3.4 Master AXI5 Interface (Output Side).....	236
19.3.5 Monitor Configuration.....	236
19.3.6 Monitor Bus Output.....	237
19.3.7 Status Outputs.....	237

19.4 Functionality.....	238
19.4.1 Combined Monitoring and Power Management.....	238
19.4.2 Clock Gating with Monitor Active.....	238
19.4.3 Ready Signal Control During Gating.....	238
19.5 Performance Monitoring.....	238
19.6 Timing Diagrams.....	239
19.6.1 Clock Gating with Monitor Activity.....	239
19.6.2 Wake-up with Immediate Monitoring.....	239
19.7 Usage Example.....	240
19.7.1 Comprehensive Debug + Power Optimization.....	240
19.8 Design Notes.....	242
19.8.1 When to Use This Module.....	242
19.8.2 Configuration Strategy Matrix.....	242
19.8.3 Monitor + Clock Gating Interaction.....	242
19.8.4 Verification Recommendations.....	243
19.9 Related Documentation.....	243
19.10 Navigation.....	244
20 AXI5 Master Read with Monitor.....	244
20.1 Overview.....	244
20.1.1 Key Features.....	244
20.2 Parameters.....	246
20.3 Monitor Backpressure (block_ready).....	249
20.4 Address-Range Checker.....	250
20.5 Ports.....	250

20.5.1 Clock and Reset.....	250
20.5.2 FUB AXI5 Interface (Slave Side).....	250
20.5.3 Master AXI5 Interface (Output Side).....	250
20.5.4 Monitor Configuration.....	251
20.5.5 AXI Protocol Filtering Configuration.....	252
20.5.6 Monitor Bus Output.....	252
20.5.7 Status Outputs.....	253
20.6 Performance Monitoring.....	253
20.6.1 The measurement window.....	253
20.6.2 Utilization buckets (R data channel).....	253
20.6.3 Throughput counters.....	254
20.6.4 Performance Monitoring Ports.....	254
20.7 Functionality.....	256
20.7.1 Monitor Bus Packet Format.....	256
20.7.2 Three-Level Filtering Hierarchy.....	256
20.7.3 Error Detection Events.....	257
20.7.4 Configuration Conflict Detection.....	257
20.8 Timing Diagrams.....	257
20.8.1 Monitored Read Transaction with Error.....	257
20.8.2 Timeout Detection.....	257
20.8.3 Performance Monitoring.....	258
20.9 Usage Example.....	258
20.9.1 Functional Verification Configuration.....	258
20.9.2 Performance Analysis Configuration.....	259

20.10 Design Notes.....	260
20.10.1 Configuration Best Practices.....	260
20.10.2 Transaction Table Sizing.....	260
20.10.3 Filtering Strategy.....	261
20.10.4 Monitor Bus Backpressure.....	261
20.11 Related Documentation.....	261
20.12 Navigation.....	261
21 AXI5 Master Write with Monitor and Clock Gating.....	261
21.1 Overview.....	262
21.1.1 Key Features.....	262
21.2 Parameters.....	263
21.3 Ports.....	266
21.3.1 Clock and Reset.....	266
21.3.2 Clock Gating Configuration.....	266
21.3.3 FUB AXI5 Interface (Slave Side).....	266
21.3.4 Master AXI5 Interface (Output Side).....	266
21.3.5 Monitor Configuration.....	266
21.3.6 Monitor Bus Output.....	267
21.3.7 Status Outputs.....	267
21.4 Functionality.....	268
21.4.1 Combined Write Monitoring and Power Management.....	268
21.4.2 Clock Gating with Write Monitor Active.....	268
21.4.3 Ready Signal Control During Gating.....	268
21.5 Performance Monitoring.....	268

21.6 Timing Diagrams.....	269
21.6.1 Clock Gating with Write Burst and Monitoring.....	269
21.6.2 Atomic Operation with Monitor + Clock Gating.....	269
21.7 Usage Example.....	270
21.7.1 Comprehensive Write Debug + Power Optimization.....	270
21.8 Design Notes.....	273
21.8.1 When to Use This Module.....	273
21.8.2 Write-Specific Configuration Strategy.....	273
21.8.3 Monitor + Clock Gating Interaction for Writes.....	273
21.8.4 Verification Recommendations for Write Path.....	274
21.8.5 Performance Analysis with Write Monitoring.....	274
21.9 Related Documentation.....	275
21.10 Navigation.....	275
22 AXI5 Master Write with Monitor.....	275
22.1 Overview.....	275
22.1.1 Key Features.....	276
22.2 Parameters.....	277
22.3 Monitor Backpressure (block_ready).....	280
22.4 Address-Range Checker.....	280
22.5 Ports.....	281
22.5.1 Clock and Reset.....	281
22.5.2 FUB AXI5 Interface (Slave Side).....	281
22.5.3 Master AXI5 Interface (Output Side).....	281
22.5.4 Monitor Configuration.....	281

22.5.5 Monitor Bus Output.....	282
22.5.6 Status Outputs.....	282
22.6 Performance Monitoring.....	282
22.6.1 The measurement window.....	283
22.6.2 Utilization buckets (W data channel).....	283
22.6.3 Throughput counters.....	283
22.6.4 Performance Monitoring Ports.....	284
22.7 Functionality.....	285
22.7.1 Monitor Bus Packet Format.....	285
22.7.2 Write-Specific Monitoring Events.....	285
22.7.3 Atomic Operation Monitoring.....	286
22.7.4 Three-Level Filtering Hierarchy.....	286
22.8 Timing Diagrams.....	286
22.8.1 Monitored Write Burst with Error.....	286
22.8.2 Atomic Operation Monitoring.....	286
22.8.3 Write Timeout Detection.....	286
22.9 Usage Example.....	286
22.9.1 Functional Verification Configuration.....	286
22.10 Design Notes.....	290
22.10.1 Write vs. Read Monitoring Differences.....	290
22.10.2 Write Transaction Table Management.....	290
22.10.3 Write Latency Measurement.....	290
22.10.4 Configuration Best Practices.....	290
22.11 Related Documentation.....	291

22.12 Navigation.....	291
23 AXI5 Slave Read Monitor with Clock Gating.....	291
23.1 Overview.....	291
23.1.1 Key Features.....	291
23.2 Parameters.....	292
23.3 Ports.....	295
23.3.1 Clock and Reset.....	295
23.3.2 Clock Gating Configuration.....	295
23.3.3 Slave AXI5 Interface.....	295
23.3.4 FUB Interface.....	295
23.3.5 Monitor Configuration.....	295
23.3.6 Monitor Bus Output.....	295
23.3.7 Status Outputs.....	296
23.4 Functionality.....	296
23.4.1 Clock Gating Behavior.....	296
23.5 Performance Monitoring.....	297
23.6 Combined Benefits.....	297
23.6.1 Power Optimization.....	297
23.6.2 Observability.....	297
23.7 Usage Example.....	298
23.8 Design Notes.....	299
23.8.1 When to Use This Module.....	299
23.8.2 Configuration Recommendations.....	300
23.8.3 Area and Timing Impact.....	300

23.9 Related Documentation.....	300
23.10 Navigation.....	300
24 AXI5 Slave Read Monitor.....	300
24.1 Overview.....	301
24.1.1 Key Features.....	301
24.2 Parameters.....	303
24.3 Monitor Backpressure (block_ready).....	305
24.4 Address-Range Checker.....	306
24.5 Ports.....	306
24.5.1 Clock and Reset.....	306
24.5.2 Slave AXI5 Interface.....	306
24.5.3 FUB Interface.....	306
24.5.4 Monitor Configuration.....	307
24.5.5 Monitor Bus Output.....	308
24.5.6 Status Outputs.....	309
24.6 Performance Monitoring.....	309
24.6.1 The measurement window.....	309
24.6.2 Utilization buckets (R data channel).....	310
24.6.3 Throughput counters.....	310
24.6.4 Performance Monitoring Ports.....	310
24.7 Functionality.....	312
24.7.1 Monitor Packet Format.....	312
24.7.2 Event Types.....	312
24.8 Configuration Guide.....	313

24.8.1 Common Configurations.....	313
24.8.2 Filter Masks.....	314
24.9 Usage Example.....	314
24.10 Design Notes.....	315
24.11 Related Documentation.....	316
24.12 Navigation.....	316
25 AXI5 Slave Write Monitor with Clock Gating.....	316
25.1 Overview.....	316
25.1.1 Key Features.....	316
25.2 Parameters.....	317
25.3 Ports.....	320
25.3.1 Clock and Reset.....	320
25.3.2 Clock Gating Configuration.....	320
25.3.3 Slave AXI5 Interface.....	320
25.3.4 FUB Interface.....	320
25.3.5 Monitor Configuration.....	320
25.3.6 Monitor Bus Output.....	320
25.3.7 Status Outputs.....	321
25.4 Functionality.....	321
25.4.1 Clock Gating Behavior.....	321
25.5 Performance Monitoring.....	322
25.6 Combined Benefits.....	322
25.6.1 Power Optimization.....	322
25.6.2 Observability.....	322

25.7 Usage Example.....	323
25.8 Design Notes.....	325
25.8.1 When to Use This Module.....	325
25.8.2 Configuration Recommendations.....	325
25.8.3 Area and Timing Impact.....	325
25.8.4 Write-Specific Considerations.....	326
25.9 Related Documentation.....	326
25.10 Navigation.....	326
26 AXI5 Slave Write Monitor.....	326
26.1 Overview.....	326
26.1.1 Key Features.....	327
26.2 Parameters.....	329
26.3 Monitor Backpressure (block_ready).....	331
26.4 Address-Range Checker.....	332
26.5 Ports.....	332
26.5.1 Clock and Reset.....	332
26.5.2 Slave AXI5 Interface.....	332
26.5.3 FUB Interface.....	333
26.5.4 Monitor Configuration.....	333
26.5.5 Monitor Bus Output.....	334
26.5.6 Status Outputs.....	335
26.6 Performance Monitoring.....	335
26.6.1 The measurement window.....	335
26.6.2 Utilization buckets (W data channel).....	336

26.6.3 Throughput counters.....	336
26.6.4 Performance Monitoring Ports.....	336
26.7 Functionality.....	338
26.7.1 Monitor Packet Format.....	338
26.7.2 Event Types.....	338
26.8 Configuration Guide.....	339
26.8.1 Common Configurations.....	339
26.8.2 Filter Masks.....	340
26.9 Usage Example.....	340
26.10 Design Notes.....	342
26.10.1 Write-Specific Monitoring.....	342
26.10.2 Best Practices.....	342
26.11 Related Documentation.....	342
26.12 Navigation.....	342
27 AXIL4 Master Read Monitor (Clock-Gated).....	343
27.1 Overview.....	343
27.1.1 Key Differences from Base Module.....	343
27.2 When to Use Clock-Gated Variant.....	343
27.3 Clock Gating Parameters.....	344
27.3.1 Parameter Relationships.....	344
27.4 Performance Monitoring.....	344
27.5 Usage Examples.....	345
27.5.1 Example 1: Maximum Power Savings (Burst Traffic).....	345
27.5.2 Example 2: Balanced Performance and Power.....	345

27.5.3 Example 3: Clock Gating Disabled (Functional Verification).....	346
27.6 Clock Gating Architecture.....	346
27.6.1 Gating Domains.....	346
27.6.2 Gating State Machine.....	347
27.6.3 Wake-Up Latency.....	347
27.7 Power Savings Analysis.....	347
27.7.1 Typical Power Reduction.....	347
27.7.2 Power Monitoring Signals.....	348
27.8 Verification Considerations.....	348
27.8.1 Clock Gating in Simulation.....	348
27.8.2 Power Analysis Verification.....	348
27.9 Synthesis Considerations.....	349
27.9.1 FPGA Implementations.....	349
27.9.2 ASIC Implementations.....	349
27.10 Related Modules.....	349
27.11 See Also.....	349
27.12 Navigation.....	349
28 AXIL4 Master Read with Monitoring.....	350
28.1 Overview.....	350
28.1.1 Key Features.....	350
28.2 Additional Parameters (Beyond Base Module).....	350
28.2.1 Synthesis-Cone Parameters.....	351
28.3 Performance Monitoring.....	352
28.3.1 The Measurement Window.....	352

28.3.2 Utilization Counters (R channel).....	352
28.3.3 Throughput Counters.....	353
28.4 Monitor Backpressure (block_ready).....	353
28.5 Address-Range Checker.....	354
28.6 Additional Ports (Beyond Base Module).....	354
28.6.1 Monitor Configuration.....	354
28.6.2 Performance Monitoring Ports.....	355
28.6.3 Filtering Masks (7 masks total).....	356
28.6.4 Monitor Bus Output.....	357
28.6.5 Status.....	357
28.7 Usage Example.....	357
28.8 Timing Diagrams.....	359
28.8.1 Scenario 1: Single-Beat Read Transaction.....	359
28.8.2 Scenario 2: Read Error (SLVERR).....	359
28.8.3 Scenario 3: Read with Backpressure.....	360
28.9 AXI4-Lite Simplifications.....	360
28.10 Related Documentation.....	360
28.10.1 Base Module.....	360
28.10.2 Monitor Infrastructure.....	360
28.10.3 Related Modules.....	360
28.11 Navigation.....	361
29 AXI4 Master Write Monitor (Clock-Gated).....	361
29.1 Quick Reference.....	361
29.2 Summary.....	361

29.3 Common Parameters.....	361
29.4 Performance Monitoring.....	362
29.5 Quick Usage.....	363
29.6 Documentation.....	363
29.7 Navigation.....	363
30 AXIL4 Master Write with Monitoring.....	363
30.1 Overview.....	364
30.1.1 Key Features.....	364
30.2 Additional Parameters.....	364
30.3 Performance Monitoring.....	364
30.3.1 The Measurement Window.....	364
30.3.2 Utilization Counters (W channel).....	365
30.3.3 Throughput Counters.....	365
30.3.4 Perfmon & Additional Control Ports.....	366
30.4 Monitor Backpressure (block_ready).....	366
30.5 Address-Range Checker.....	366
30.6 Additional Ports.....	367
30.7 Usage.....	367
30.8 Timing Diagrams.....	367
30.8.1 Scenario 1: Single-Beat Write Transaction.....	367
30.8.2 Scenario 2: Write Error (SLVERR).....	368
30.8.3 Scenario 3: Write with Backpressure.....	368
30.9 Related Modules.....	368
31 AXIL4 Slave Read Monitor (Clock-Gated).....	369

31.1 Quick Reference.....	369
31.2 Summary.....	369
31.3 Common Parameters.....	369
31.4 Performance Monitoring.....	370
31.5 Quick Usage.....	371
31.6 Documentation.....	371
31.7 Navigation.....	371
32 AXIL4 Slave Read with Monitoring.....	371
32.1 Overview.....	372
32.1.1 Key Features.....	372
32.2 Additional Parameters.....	372
32.3 Performance Monitoring.....	372
32.3.1 The Measurement Window.....	372
32.3.2 Utilization Counters (R channel).....	373
32.3.3 Throughput Counters.....	373
32.4 Monitor Backpressure (block_ready).....	374
32.5 Address-Range Checker.....	374
32.6 Additional Ports.....	374
32.7 Usage.....	375
32.8 Timing Diagrams.....	375
32.8.1 Scenario 1: Slave Read Transaction.....	375
32.8.2 Scenario 2: Slave Read Error (DECERR).....	376
32.8.3 Scenario 3: Slave Read with Wait States.....	376
32.9 Related Modules.....	376

33 AXIL4 Slave Write Monitor (Clock-Gated).....	377
33.1 Quick Reference.....	377
33.2 Summary.....	377
33.3 Common Parameters.....	377
33.4 Performance Monitoring.....	378
33.5 Quick Usage.....	378
33.6 Documentation.....	379
33.7 Navigation.....	379
34 AXIL4 Slave Write with Monitoring.....	379
34.1 Overview.....	379
34.1.1 Key Features.....	379
34.2 Additional Parameters.....	380
34.3 Performance Monitoring.....	380
34.3.1 The Measurement Window.....	380
34.3.2 Utilization Counters (W channel).....	381
34.3.3 Throughput Counters.....	381
34.4 Monitor Backpressure (block_ready).....	382
34.5 Address-Range Checker.....	382
34.6 Additional Ports.....	382
34.7 Usage.....	382
34.8 Timing Diagrams.....	383
34.8.1 Scenario 1: Slave Write Transaction.....	383
34.8.2 Scenario 2: Slave Write Error (DECERR).....	383
34.8.3 Scenario 3: Slave Write with Wait States.....	384

34.9 Related Modules.....	384
35 AXI Monitor Address-Range Checker.....	384
35.1 Overview.....	384
35.2 Key Features.....	385
35.3 Module Purpose.....	385
35.4 Parameters.....	385
35.5 Port Groups.....	386
35.5.1 Clock and Reset.....	386
35.5.2 Side-Band Timestamp.....	387
35.5.3 Command Interface (Snoop).....	387
35.5.4 Configuration Inputs.....	387
35.5.5 Monitor Bus Output.....	388
35.6 Internal Architecture.....	388
35.7 Event Encoding.....	389
35.8 Formal Properties.....	390
35.9 Configuration Examples.....	391
35.9.1 Example 1: Single Range Checker.....	391
35.9.2 Example 2: Multi-Range Checker.....	391
35.9.3 Example 3: Exact-Match Detector.....	392
35.10 Filtering Integration.....	392
35.11 Integration Notes.....	392
35.11.1 Instantiation Pattern.....	392
35.11.2 Synthesis Behavior.....	392
35.11.3 Downstream FIFO.....	392

35.12 Related Modules.....	393
35.13 See Also.....	393
35.14 Navigation.....	393
36 AXI Monitor Base.....	393
36.1 Overview.....	393
36.2 Key Features.....	394
36.3 Module Purpose.....	394
36.4 Parameters.....	395
36.4.1 Identity and Sizing.....	395
36.4.2 Master Switches.....	396
36.4.3 Synthesis-Cone Enables (ENABLE_*_LOGIC).....	396
36.5 Port Groups.....	397
36.5.1 Command Phase Interface (AW/AR).....	397
36.5.2 Data Channel Interface (R/W).....	397
36.5.3 Response Channel Interface (B).....	398
36.5.4 Configuration Interface.....	398
36.5.5 Address-Range Checker Interface.....	399
36.5.6 Side-Band Timestamp Input.....	400
36.5.7 Monitor Bus Output.....	400
36.5.8 Performance-Window Control Inputs.....	401
36.5.9 Performance-Window Status and Counter Outputs.....	401
36.6 Performance Monitoring.....	404
36.6.1 The Measurement Window.....	404
36.6.2 Utilization Buckets (data channel).....	405

36.6.3 Throughput Counters.....	405
36.7 Usage in Monitor System.....	406
36.7.1 Integration Example.....	406
36.8 Configuration Guidelines.....	407
36.8.1 Transaction Table Sizing.....	407
36.8.2 Timeout Configuration.....	407
36.9 Performance Characteristics.....	407
36.10 Verification Considerations.....	408
36.10.1 Key Test Scenarios.....	408
36.11 Related Modules.....	408
36.12 See Also.....	408
36.13 Navigation.....	409
37 AXI Monitor Filtered.....	409
37.1 Overview.....	409
37.2 Key Features.....	409
37.3 Module Purpose.....	410
37.4 Parameters.....	410
37.5 Port Groups.....	412
37.5.1 Observed AXI/AXIL Channels (inputs to the wrapped base).....	412
37.5.2 Monitor Bus Output (filtered).....	412
37.5.3 Reset and Synchronous Control.....	413
37.5.4 Filter Configuration.....	413
37.5.5 Configuration Status Output.....	414
37.5.6 Performance Window Control (Stage A of perfmon RFC).....	414

37.5.7 Performance Counters (Stage B of perfmon RFC).....	415
37.6 Usage in Monitor System.....	418
37.6.1 Internal Integration.....	418
37.7 Configuration Guidelines.....	418
37.7.1 Filter Strategy.....	418
37.8 Performance Characteristics.....	419
37.9 Verification Considerations.....	419
37.9.1 Key Test Scenarios.....	419
37.10 Related Modules.....	420
37.11 See Also.....	420
37.12 Navigation.....	420
38 AXI Monitor Reporter — Completion Cone.....	420
38.1 Overview.....	420
38.1.1 Key Features.....	420
38.2 Module Purpose.....	421
38.3 Parameters.....	421
38.4 Port Groups.....	422
38.4.1 Inputs (shared monitor state).....	422
38.4.2 Outputs (packet fields to the top reporter).....	422
38.5 Functional Description.....	423
38.5.1 Completion Scan.....	423
38.5.2 First-Match Selection.....	423
38.5.3 Emitted Fields.....	423
38.5.4 Ownership of State.....	423

38.6 Usage Example.....	423
38.7 Design Notes.....	424
38.7.1 Compile-Out Path.....	424
38.7.2 Combinational by Design.....	424
38.7.3 One Packet Per Cycle.....	424
38.8 Related Modules.....	424
38.8.1 Used By.....	424
38.8.2 Uses.....	424
38.8.3 See Also.....	425
38.9 References.....	425
38.9.1 Source Code.....	425
38.9.2 Documentation.....	425
38.10 Navigation.....	425
39 AXI Monitor Reporter — Debug State-Change Emitter.....	425
39.1 Overview.....	426
39.1.1 Key Features.....	426
39.2 Module Purpose.....	426
39.3 Parameters.....	427
39.4 Port Groups.....	427
39.4.1 Clock and Reset.....	427
39.4.2 Inputs (shared monitor state).....	427
39.4.3 Direct-Inject Handshake.....	427
39.4.4 Outputs (packet fields).....	428
39.5 Functional Description.....	428

39.5.1 Change Detection.....	428
39.5.2 Previous-State Flop.....	428
39.5.3 Packet Assembly.....	428
39.5.4 Direct-Inject and output_busy.....	429
39.6 Usage Example.....	429
39.7 Design Notes.....	430
39.7.1 pkt_taken Is Intentionally Unused.....	430
39.7.2 Sequential, Not Combinational.....	430
39.7.3 Compile-Out Path.....	430
39.7.4 State Tuple Encoding.....	430
39.8 Related Modules.....	430
39.8.1 Used By.....	430
39.8.2 Uses.....	430
39.8.3 See Also.....	430
39.9 References.....	431
39.9.1 Source Code.....	431
39.9.2 Documentation.....	431
39.10 Navigation.....	431
40 AXI Monitor Reporter — Error Cone.....	431
40.1 Overview.....	431
40.1.1 Key Features.....	432
40.2 Module Purpose.....	432
40.3 Parameters.....	432
40.4 Port Groups.....	433

40.4.1 Inputs (shared monitor state).....	433
40.4.2 Outputs (packet fields to the top reporter).....	433
40.5 Functional Description.....	434
40.5.1 Error / Orphan Scan.....	434
40.5.2 First-Match Selection.....	434
40.5.3 Emitted Fields.....	434
40.5.4 Ownership of State.....	435
40.6 Usage Example.....	435
40.7 Design Notes.....	435
40.7.1 Timeout De-Aliasing.....	435
40.7.2 Compile-Out Path.....	435
40.7.3 Raw Event Code Forwarding.....	436
40.8 Related Modules.....	436
40.8.1 Used By.....	436
40.8.2 Uses.....	436
40.8.3 See Also.....	436
40.9 References.....	436
40.9.1 Source Code.....	436
40.9.2 Documentation.....	436
40.10 Navigation.....	437
41 AXI Monitor Reporter (Dispatcher + 6 Sub-blocks).....	437
41.1 Overview.....	437
41.2 Key Features.....	437
41.3 Sub-blocks.....	438

41.4 Module Purpose.....	438
41.5 Parameters.....	439
41.6 Port Groups.....	440
41.7 Usage in Monitor System.....	441
41.7.1 Internal Integration.....	442
41.8 Configuration Guidelines.....	442
41.9 Performance Characteristics.....	442
41.10 Verification Considerations.....	442
41.10.1 Test Coverage.....	442
41.11 Related Modules.....	442
41.12 See Also.....	443
41.13 Navigation.....	443
42 AXI Monitor Reporter — Performance Packets.....	443
42.1 Overview.....	443
42.1.1 Key Features.....	443
42.2 Module Purpose.....	444
42.3 Parameters.....	444
42.4 Port Groups.....	444
42.4.1 Clock and Reset.....	444
42.4.2 Control / Status Inputs.....	445
42.4.3 Packet Output.....	445
42.4.4 Lifetime Counters (Status / Debug).....	446
42.5 Functional Description.....	446
42.5.1 Lifetime Counters.....	446

42.5.2 Cycle FSM.....	446
42.5.3 Output Multiplexer.....	447
42.5.4 Handshake.....	447
42.6 Usage Example.....	447
42.7 Design Notes.....	448
42.7.1 Never Enable Completion + Performance Packets Together.....	448
42.7.2 pkt_taken Is Currently Observational.....	448
42.7.3 Relationship to the Window-Bucket Perfmon.....	448
42.7.4 Counter Wrap.....	448
42.8 Related Modules.....	448
42.8.1 Used By.....	448
42.8.2 Uses.....	449
42.8.3 See Also.....	449
42.9 References.....	449
42.9.1 Source Code.....	449
42.9.2 Documentation.....	449
42.10 Navigation.....	449
43 AXI Monitor Reporter — Threshold Packets.....	450
43.1 Overview.....	450
43.1.1 Key Features.....	450
43.2 Module Purpose.....	450
43.3 Parameters.....	451
43.4 Port Groups.....	451
43.4.1 Clock and Reset.....	451

43.4.2 Transaction Table and Configuration.....	451
43.4.3 Handshake / Backpressure.....	452
43.4.4 Packet Output.....	452
43.5 Functional Description.....	453
43.5.1 Active-Count Detection.....	453
43.5.2 Per-Slot Latency Pipeline.....	453
43.5.3 Latency Event Selection.....	453
43.5.4 Edge-Sticky Flags.....	453
43.5.5 Output Multiplexer.....	454
43.6 Usage Example.....	454
43.7 Design Notes.....	454
43.7.1 Threshold Packets Bypass the FIFO.....	454
43.7.2 Active Count vs Latency Priority.....	455
43.7.3 Latency Pipeline Is the Area Cost.....	455
43.7.4 Read vs Write Latency.....	455
43.8 Related Modules.....	455
43.8.1 Used By.....	455
43.8.2 Uses.....	455
43.8.3 See Also.....	455
43.9 References.....	456
43.9.1 Source Code.....	456
43.9.2 Documentation.....	456
43.10 Navigation.....	456
44 AXI Monitor Reporter — Timeout Packets.....	456

44.1 Overview.....	456
44.1.1 Key Features.....	457
44.2 Module Purpose.....	457
44.3 Parameters.....	457
44.4 Port Groups.....	458
44.4.1 Inputs.....	458
44.4.2 Packet Output.....	458
44.5 Functional Description.....	459
44.5.1 Candidate Detection.....	459
44.5.2 Priority Encoding.....	459
44.5.3 Output Assignment.....	459
44.5.4 No Double-Reporting.....	459
44.6 Usage Example.....	459
44.7 Design Notes.....	460
44.7.1 Pure Combinational.....	460
44.7.2 Event Code Comes From the Slot.....	460
44.7.3 Shared Vectors Are Load-Bearing.....	460
44.8 Related Modules.....	460
44.8.1 Used By.....	460
44.8.2 Uses.....	461
44.8.3 See Also.....	461
44.9 References.....	461
44.9.1 Source Code.....	461
44.9.2 Documentation.....	461

44.10 Navigation.....	461
45 AXI Monitor Timeout.....	462
45.1 Overview.....	462
45.2 Key Features.....	462
45.3 Module Purpose.....	462
45.4 Parameters.....	463
45.5 Port Groups.....	463
45.6 Usage in Monitor System.....	465
45.6.1 Internal Integration.....	465
45.7 Configuration Guidelines.....	465
45.8 Performance Characteristics.....	465
45.9 Verification Considerations.....	465
45.9.1 Test Coverage.....	465
45.10 Related Modules.....	466
45.11 See Also.....	466
45.12 Navigation.....	466
46 AXI Monitor Timer.....	466
46.1 Overview.....	466
46.1.1 Key Features.....	466
46.2 Module Purpose.....	467
46.3 Parameters.....	467
46.4 Port Groups.....	468
46.4.1 Global Clock and Reset.....	468
46.4.2 Timer Configuration.....	468

46.4.3 Timer Outputs.....	468
46.5 Functional Description.....	468
46.5.1 Global Timestamp Counter.....	468
46.5.2 Frequency-Invariant Timer Tick.....	469
46.5.3 Counter Freq Invariant Integration.....	470
46.6 Integration with Monitor Architecture.....	470
46.6.1 Output Integration.....	471
46.6.2 Configuration Strategy.....	471
46.7 Usage Example.....	471
46.8 Design Notes.....	473
46.8.1 Timestamp Wraparound Handling.....	473
46.8.2 Frequency Selection Trade-offs.....	473
46.8.3 Dynamic Frequency Scaling.....	473
46.8.4 Zero Timestamp on Reset.....	474
46.8.5 Timer Tick Duty Cycle.....	474
46.8.6 Resource Utilization.....	474
46.8.7 Multiple Timer Instances.....	474
46.9 Related Modules.....	475
46.9.1 Used By.....	475
46.9.2 Uses.....	475
46.9.3 Critical Interfaces.....	475
46.9.4 Related Infrastructure.....	475
46.10 References.....	475
46.10.1 Specifications.....	475

46.10.2 Source Code.....	475
46.10.3 Configuration Guides.....	476
46.11 Navigation.....	476
47 AXI Monitor Transaction Manager.....	476
47.1 Overview.....	476
47.2 Key Features.....	476
47.3 Architecture.....	477
47.4 Parameters.....	478
47.5 Module Interface.....	479
47.6 Transaction Lifecycle.....	480
47.7 Synthesis Notes.....	481
47.8 Equivalence Reference.....	482
47.9 TRANS_MGR_VARIANT Knob.....	483
47.10 Performance Characteristics.....	483
47.11 Verification.....	483
47.12 Related Modules.....	484
47.13 See Also.....	484
47.14 Navigation.....	485
48 Monitor Bus Round-Robin Arbiter.....	485
48.1 Overview.....	485
48.1.1 Key Features.....	485
48.2 Module Purpose.....	485
48.3 Parameters.....	486
48.4 Port Groups.....	486

48.4.1 Clock and Reset.....	486
48.4.2 Block Arbitration.....	486
48.4.3 Monitor Bus Inputs (Array of CLIENTS).....	486
48.4.4 Monitor Bus Output (Aggregated).....	487
48.4.5 Debug/Status Outputs.....	487
48.5 Functional Description.....	488
48.5.1 Arbiter Request and Grant ACK Logic.....	488
48.5.2 Round-Robin Arbiter Instance.....	488
48.5.3 Client Ready Signal Generation.....	488
48.5.4 Output Multiplexer.....	488
48.5.5 Optional Skid Buffers.....	488
48.6 Usage Example.....	489
48.7 Design Notes.....	490
48.7.1 ACK Mode Operation.....	490
48.7.2 Skid Buffer Depth Selection.....	490
48.7.3 Zero-Latency Configuration.....	490
48.7.4 Assertions.....	490
48.8 Related Modules.....	491
48.8.1 Used By.....	491
48.8.2 Uses.....	491
48.8.3 Related Modules.....	491
48.9 References.....	491
48.9.1 Specifications.....	491
48.9.2 Source Code.....	491

48.10 Navigation.....	491
49 MonBus Group — AXI4 / AXI4.....	492
49.1 Overview.....	492
49.1.1 Key Features.....	492
49.2 Module Purpose.....	492
49.3 Parameters.....	493
49.4 Port Groups.....	494
49.4.1 Clock, Reset, Clear.....	494
49.4.2 MonBus Ingress and Timestamp.....	494
49.4.3 S-Side — AXI4 Slave Read (error record drain).....	494
49.4.4 M-Side — AXI4 Master Write (bulk trace capture).....	495
49.4.5 Status and Egress Configuration.....	496
49.5 Functional Description.....	497
49.5.1 S-Side Error Drain (AXI4 slave read).....	497
49.5.2 M-Side Bulk Capture (AXI4 master write).....	497
49.5.3 Shared Egress Configuration.....	498
49.6 Usage Example.....	498
49.7 Design Notes.....	499
49.7.1 Both Sides Are Full AXI4.....	499
49.7.2 AW Sideband Defaults Live in the Wrapper.....	499
49.7.3 Skid Depths Are Tunable.....	499
49.8 Related Modules.....	499
49.8.1 Used By.....	499
49.8.2 Uses.....	499

49.8.3 See Also.....	499
49.9 References.....	500
49.9.1 Source Code.....	500
49.9.2 Documentation.....	500
49.10 Navigation.....	500
50 MonBus Group — AXI4 / AXIL.....	500
50.1 Overview.....	500
50.1.1 Key Features.....	501
50.2 Module Purpose.....	501
50.3 Parameters.....	501
50.4 Port Groups.....	502
50.4.1 Clock, Reset, Clear.....	502
50.4.2 MonBus Ingress and Timestamp.....	503
50.4.3 S-Side — AXI4 Slave Read (error record drain).....	503
50.4.4 M-Side — AXIL Master Write (bulk trace capture).....	504
50.4.5 Status and Egress Configuration.....	504
50.5 Functional Description.....	505
50.5.1 S-Side Error Drain (AXI4 slave read).....	505
50.5.2 M-Side Bulk Capture (AXIL master write).....	505
50.5.3 Shared Egress Configuration.....	506
50.6 Usage Example.....	506
50.7 Design Notes.....	507
50.7.1 AXIL Forces Single-Beat Writes.....	507
50.7.2 Core Still Drives AXI4 FUB Fields.....	507

50.7.3 Read Side Is Unchanged.....	507
50.8 Related Modules.....	507
50.8.1 Used By.....	507
50.8.2 Uses.....	507
50.8.3 See Also.....	507
50.9 References.....	508
50.9.1 Source Code.....	508
50.9.2 Documentation.....	508
50.10 Navigation.....	508
51 Monitor Bus Group — AXIL Slave-Read / AXI4 Master-Write.....	508
51.1 Overview.....	508
51.1.1 Key Features.....	509
51.2 Module Purpose.....	509
51.3 Parameters.....	509
51.4 Port Groups.....	511
51.4.1 Clock, Reset, and CAM Clear.....	511
51.4.2 Monitor-Bus Input + Timestamp.....	511
51.4.3 AXIL Slave Read — Err-FIFO Drain.....	512
51.4.4 AXI4 Master Write — Burst Bulk Capture.....	512
51.4.5 Interrupt.....	514
51.4.6 Configuration.....	514
51.4.7 Per-Protocol Filter Masks.....	514
51.4.8 Status / Debug.....	516
51.5 Functional Description.....	516

51.5.1 S-Side Err-Drain Read Protocol.....	516
51.5.2 M-Side Bulk-Capture Protocol (AXI4 Burst).....	517
51.5.3 Interrupt.....	517
51.5.4 Shared Egress Packet Filter.....	517
51.5.5 Compression.....	517
51.6 Usage Example.....	518
51.7 Design Notes.....	519
51.7.1 When to Pick This Over AXIL/AXIL.....	519
51.7.2 AXI4 Leaf Default Tie-Offs.....	519
51.7.3 Why One Core + Four Wrappers.....	519
51.8 Related Modules.....	519
51.8.1 Used By.....	519
51.8.2 Uses.....	520
51.8.3 See Also.....	520
51.9 References.....	520
51.9.1 Source Code.....	520
51.9.2 Documentation.....	520
51.10 Navigation.....	521
52 Monitor Bus Group — AXIL Slave-Read / AXIL Master-Write.....	521
52.1 Overview.....	521
52.1.1 Key Features.....	521
52.2 Module Purpose.....	521
52.3 Parameters.....	522
52.4 Port Groups.....	523

52.4.1 Clock, Reset, and CAM Clear.....	523
52.4.2 Monitor-Bus Input + Timestamp.....	524
52.4.3 AXIL Slave Read — Err-FIFO Drain.....	524
52.4.4 AXIL Master Write — Bulk Capture.....	525
52.4.5 Interrupt.....	525
52.4.6 Configuration.....	526
52.4.7 Per-Protocol Filter Masks.....	526
52.4.8 Status / Debug.....	527
52.5 Functional Description.....	528
52.5.1 S-Side Err-Drain Read Protocol.....	528
52.5.2 M-Side Bulk-Capture Protocol.....	529
52.5.3 Interrupt.....	529
52.5.4 Shared Egress Packet Filter.....	529
52.5.5 Compression.....	529
52.6 Usage Example.....	529
52.7 Design Notes.....	530
52.7.1 Legacy Replacement.....	530
52.7.2 Why One Core + Four Wrappers.....	531
52.7.3 32-Bit Drain Serializer.....	531
52.8 Related Modules.....	531
52.8.1 Used By.....	531
52.8.2 Uses.....	531
52.8.3 See Also.....	531
52.9 References.....	532

52.9.1 Source Code.....	532
52.9.2 Documentation.....	532
52.10 Navigation.....	532
53 Monitor Bus LRU CAM.....	532
53.1 Overview.....	533
53.2 Key Features.....	533
53.3 Architecture.....	534
53.4 Top-level Interface.....	534
53.5 Storage Model: Position-Indexed LRU.....	535
53.6 Actions.....	536
53.7 Per-Slot Update Mechanics.....	537
53.8 Per-Entry Timestamp Storage.....	537
53.9 Match Logic.....	538
53.10 Configuration.....	538
53.11 Test.....	538
53.12 Related Modules.....	539
54 Monitor Bus LRU CAM (Pipelined).....	539
54.1 Overview.....	540
54.2 Top-level Interface.....	540
54.3 Forwarding (depth-1).....	542
54.4 Self-Derived Action.....	543
54.5 Synchronous Clear.....	543
54.6 Verification.....	544
54.7 Related Modules.....	544

55 Monitor Bus Compressor.....	544
55.1 Overview.....	544
55.2 Why Compress Monitor Traces?.....	545
55.3 Architecture.....	546
55.4 Top-level Interface.....	548
55.4.1 Synchronous CAM clear.....	549
55.5 Compression Techniques.....	549
55.5.1 1. Template extraction.....	549
55.5.2 2. Delta-encoded timestamp.....	550
55.5.3 3. Width-tiered Tier-1 formats.....	550
55.5.4 4. Differential payload encoding (Format C).....	551
55.5.5 5. Tier-0 RAW escape.....	551
55.6 Slot Bit Layouts.....	552
55.7 CAM Design.....	553
55.8 Encoder Decision Tree.....	554
55.9 Pipeline and Timing.....	555
55.9.1 Per-template delta_ts.....	555
55.9.2 Input skid.....	556
55.9.3 pblock (Nexys A7 build only).....	556
55.10 Verification Methodology.....	556
55.10.1 1. Unit-level (the compressor in isolation).....	556
55.10.2 2. Integration-level (compressor + AXIL writer + base/limit wrap).....	556
55.11 Statistics Counters.....	557
55.12 Validation Numbers (locked dataset).....	557

55.13 Related Modules.....	558
55.14 Test.....	559
56 MonBus Group Core.....	559
56.1 Overview.....	559
56.1.1 Key Features.....	559
56.2 Module Purpose.....	560
56.3 Parameters.....	560
56.4 Port Groups.....	561
56.4.1 Clock, Reset, and Clear.....	561
56.4.2 MonBus Ingress and Timestamp.....	561
56.4.3 Status / IRQ / Debug.....	562
56.4.4 Address Window and Flush Configuration.....	562
56.4.5 Per-Protocol Filter Masks.....	563
56.4.6 Compressor Statistics.....	565
56.4.7 AXI4-Shaped Master-Write FUB (bulk capture).....	565
56.4.8 AXI4-Shaped Slave-Read FUB (error drain).....	566
56.5 Functional Description.....	567
56.5.1 Free-Running Timestamp.....	567
56.5.2 Packet Filtering.....	567
56.5.3 Error FIFO and Slave-Read Drain.....	568
56.5.4 Write Path — Raw Expander vs Compressor.....	568
56.5.5 Master-Write Burst Writer.....	568
56.6 Usage Example.....	569
56.7 Design Notes.....	570

56.7.1 Hold <code>cfg_compress_en</code> Stable.....	570
56.7.2 AXIL Builds Use the Same Core.....	570
56.7.3 FIFO Granularity Difference.....	570
56.7.4 Timing-Motivated Structure.....	570
56.7.5 4KB and Window Compliance.....	570
56.8 Related Modules.....	571
56.8.1 Used By.....	571
56.8.2 Uses.....	571
56.8.3 See Also.....	571
56.9 References.....	571
56.9.1 Source Code.....	571
56.9.2 Documentation.....	571
56.10 Navigation.....	572
57 Monitor Bus Group Family.....	572
57.1 Overview.....	572
57.2 Architecture.....	573
57.3 Why one core + four wrappers?.....	573
57.4 Common Parameters.....	574
57.4.1 Runtime config.....	576
57.5 Beat Layout.....	577
57.5.1 Raw mode (<code>cfg_compress_en</code> = 0, or <code>USE_COMPRESSION</code> = 0 build)..	577
57.5.2 Compressed mode (<code>cfg_compress_en</code> = 1, requires <code>USE_COMPRESSION</code> = 1 build).....	577
57.5.3 Slave-read drain (both modes).....	578

57.6 Master-Write Behavior.....	578
57.6.1 Burst geometry: 3-stage pipeline + fresh-FIFO cap at commit.....	578
57.6.2 Address-window behavior: wrap, not saturate.....	580
57.6.3 mod-3 rounding (no /3 operator).....	580
57.6.4 Final port behavior.....	580
57.7 Per-Protocol Filter Configuration.....	580
57.8 Sub-modules Instantiated.....	581
57.8.1 Canonical filelist.....	582
57.9 Status / Debug Outputs.....	582
57.10 Test.....	583
57.11 Migration from monbus_axil_group.....	583
57.12 Related Modules.....	585
58 MonBus Half-Beat Packer.....	585
58.1 Overview.....	585
58.1.1 Key Features.....	585
58.2 Module Purpose.....	586
58.3 Parameters.....	586
58.4 Port Groups.....	586
58.4.1 Clock and Reset.....	586
58.4.2 Input — Beats from the Compressor.....	587
58.4.3 Output — Packed Beats to the Write FIFO.....	587
58.5 Functional Description.....	587
58.5.1 Beat Layout.....	587
58.5.2 Case Decode.....	587

58.5.3 Handshake and Ordering.....	588
58.5.4 Pending-Slot Register.....	588
58.5.5 Bit-Exactness.....	588
58.6 Usage Example.....	589
58.7 Design Notes.....	589
58.7.1 Requires the Compressor.....	589
58.7.2 Order Preservation Is Non-Negotiable.....	589
58.7.3 One-Slot Buffer Only.....	589
58.8 Related Modules.....	590
58.8.1 Used By.....	590
58.8.2 Uses.....	590
58.8.3 See Also.....	590
58.9 References.....	590
58.9.1 Source Code.....	590
58.9.2 Documentation.....	590
58.10 Navigation.....	590
59 Monitor Transaction CAM.....	591
59.1 Overview.....	591
59.2 Key Features.....	591
59.3 Architecture.....	592
59.4 Top-level Interface.....	593
59.5 Why Three Lookup Ports?.....	594
59.6 The “First-Match” Variant.....	594
59.7 3-Way Mutex Alloc Priority Encoder.....	595

59.8 Per-Slot Storage and Write Port.....	595
59.9 Caller Pattern (axi_monitor_trans_mgr).....	596
59.10 Configuration.....	597
59.11 Synthesis Notes.....	597
59.12 Test.....	598
59.13 Related Modules.....	599

1 AMBA4 Event Code Reference

This is the per-protocol event-code reference for the AMBA4 family — AXI4 / AXI4-Lite, APB4, and AXI4-Stream. The 8-bit `event_code` field of every monbus packet sourced by an AMBA4 monitor takes its value from one of the enums defined in `rtl/amba/includes/monitor_amba4_pkg.sv` and listed below. For the universal packet layout, `protocol / packet_type` enums, and helper functions see `monitor_package_spec.md`.

Each enum is 8 bits wide. Slots `8'h0–8'hE` are reserved for predefined codes (with `_RESERVED_*` placeholders absorbing unused indices) and slot `8'hF` is always the `*_USER_DEFINED` escape hatch.

1.1 AXI4 Event Codes

Nine categories cover the AXI4 / AXI4-Lite monitor surface: error, timeout, completion, threshold, performance, address-match, debug, and the two windowed-performance categories perf-window (`axi_perfwin_code_t`, `packet_type = PktTypePerfWin = 4'hD`) and perf-histogram (`axi_perfhist_select_t`, `packet_type = PktTypePerfHist = 4'hE`); see the RTL package for the full perf-window / perf-histogram value tables. The protocol field for every code in this section is `PROTOCOL_AXI (4'h0)`.

1.1.1 axi_error_code_t

Packet context: `packet_type = PktTypeError`, `protocol = PROTOCOL_AXI`

Value	Name	Description
8'h0	AXI_ERR_RESP_SLV ERR	Slave error response
8'h1	AXI_ERR_RESP_DEC ERR	Decode error response
8'h2	AXI_ERR_DATA_ORP	Data without command

Value	Name	Description
	HAN	
8'h3	AXI_ERR_RESP_ORP HAN	Response without transaction
8'h4	AXI_ERR_PROTOCOL	Protocol violation
8'h5	AXI_ERR_BURST_LENGTH	Invalid burst length
8'h6	AXI_ERR_BURST_SIZE	Invalid burst size
8'h7	AXI_ERR_BURST_TYPE	Invalid burst type
8'h8	AXI_ERR_ID_COLLISION	ID collision detected
8'h9	AXI_ERR_WRITE_BEFORE_ADDR	Write data before address
8'hA	AXI_ERR_RESP_BEFORE_DATA	Response before data complete
8'hB	AXI_ERR_LAST_MISSING	Missing LAST signal
8'hC	AXI_ERR_STROBE_ERROR	Write strobe error
8'hD	AXI_ERR_ADDR_RANGE	Address-range violation (from axi_monitor_addr_check)
8'hE	(reserved)	Reserved for future use
8'hF	AXI_ERR_USER_DEFINED	User-defined error

1.1.2 axi_timeout_code_t

Packet context: packet_type = PktTypeTimeout, protocol = PROTOCOL_AXI

Value	Name	Description
8'h0	AXI_TIMEOUT_CMD	Command/Address timeout
8'h1	AXI_TIMEOUT_DATA	Data timeout
8'h2	AXI_TIMEOUT_RESP	Response timeout
8'h3	AXI_TIMEOUT_HANDSHAKE	Handshake timeout
8'h4	AXI_TIMEOUT_BURST	Burst completion timeout

Value	Name	Description
8'h5	AXI_TIMEOUT_EXCLUSIVE	Exclusive access timeout
8'h6–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	AXI_TIMEOUT_USER_DEFINED	User-defined timeout

1.1.3 axi_completion_code_t

Packet context: packet_type = PktTypeCompletion, protocol = PROTOCOL_AXI

Value	Name	Description
8'h0	AXI_COMPL_TRANSACTION_COMPLETE	Transaction completed successfully
8'h1	AXI_COMPL_READ_COMPLETE	Read transaction complete
8'h2	AXI_COMPL_WRITE_COMPLETE	Write transaction complete
8'h3	AXI_COMPL_BURST_COMPLETE	Burst transaction complete
8'h4	AXI_COMPL_EXCLUSIVE_OK	Exclusive access completed
8'h5	AXI_COMPL_EXCLUSIVE_FAIL	Exclusive access failed
8'h6	AXI_COMPL_ATOMIC_OK	Atomic operation completed
8'h7	AXI_COMPL_ATOMIC_FAIL	Atomic operation failed
8'h8–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	AXI_COMPL_USER_DEFINED	User-defined completion

1.1.4 axi_threshold_code_t

Packet context: packet_type = PktTypeThreshold, protocol = PROTOCOL_AXI

Value	Name	Description
8'h0	AXI_THRESH_ACTIVE_COUNT	Active transaction count threshold
8'h1	AXI_THRESH_LATENCY	Latency threshold

Value	Name	Description
8'h2	AXI_THRESH_ERROR_RATE	Error rate threshold
8'h3	AXI_THRESH_THROUGHPUT	Throughput threshold
8'h4	AXI_THRESH_QUEUE_DEPTH	Queue depth threshold
8'h5	AXI_THRESH_BANDWIDTH	Bandwidth utilization threshold
8'h6	AXI_THRESH_OUTSTANDING	Outstanding transactions threshold
8'h7	AXI_THRESH_BURST_SIZE	Average burst size threshold
8'h8-8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	AXI_THRESH_USER_DEFINED	User-defined threshold

1.1.5 axi_performance_code_t

Packet context: packet_type = PktTypePerf, protocol = PROTOCOL_AXI

Value	Name	Description
8'h0	AXI_PERF_ADDR_LATENCY	Address phase latency
8'h1	AXI_PERF_DATA_LATENCY	Data phase latency
8'h2	AXI_PERF_RESP_LATENCY	Response phase latency
8'h3	AXI_PERF_TOTAL_LATENCY	Total transaction latency
8'h4	AXI_PERF_THROUGHPUT	Transaction throughput
8'h5	AXI_PERF_ERROR_RATE	Error rate
8'h6	AXI_PERF_ACTIVE_COUNT	Current active transaction count
8'h7	AXI_PERF_COMPLETED_COUNT	Total completed transaction count
8'h8	AXI_PERF_ERROR_COUNT	Total error transaction count
8'h9	AXI_PERF_BANDWIDTH_UTIL	Bandwidth utilization

Value	Name	Description
8'hA	AXI_PERF_QUEUE_DEPTH	Average queue depth
8'hB	AXI_PERF_BURST EFFICIENCY	Burst efficiency metric
8'hC–8'hD	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hE	AXI_PERF_READ_WRITE_RATIO	Read/Write transaction ratio
8'hF	AXI_PERF_USER_DEFINED	User-defined performance

1.1.6 axi_addr_match_code_t

Packet context: packet_type = PktTypeAddrMatch, protocol = PROTOCOL_AXI

Value	Name	Description
8'h0	AXI_ADDR_EXACT_MATCH	Exact address match
8'h1	AXI_ADDR_RANGE_MATCH	Address within range
8'h2	AXI_ADDR_MASK_MATCH	Address mask match
8'h3	AXI_ADDR_PATTERN_MATCH	Address pattern match
8'h4	AXI_ADDR_SEQUENTIAL	Sequential address access
8'h5	AXI_ADDR_STRIDE_MATCH	Stride pattern match
8'h6	AXI_ADDR_HOTSPOT	Address hotspot detected
8'h7	AXI_ADDR_CONFLICT	Address conflict detected
8'h8–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	AXI_ADDR_USER_DEFINED	User-defined address match

1.1.7 axi_debug_code_t

Packet context: packet_type = PktTypeDebug, protocol = PROTOCOL_AXI

Value	Name	Description
8'h0	AXI_DEBUG_STATE_CHANGE	AXI state machine change
8'h1	AXI_DEBUG_PIPELINE_STALL	Pipeline stall event
8'h2	AXI_DEBUG_BACKPRESSURE	Backpressure event
8'h3	AXI_DEBUG_OUTSTANDING	Outstanding transaction count
8'h4	AXI_DEBUG_REORDER_BUFFER	Reorder buffer status
8'h5	AXI_DEBUG_ID_ALLOCATION	Transaction ID allocation
8'h6	AXI_DEBUG_QOS_ESCALATION	QoS escalation event
8'h7	AXI_DEBUG_HANDSHAKE	Handshake timing event
8'h8	AXI_DEBUG_QUEUE_STATUS	Queue status change
8'h9	AXI_DEBUG_COUNTER	Counter snapshot
8'hA	AXI_DEBUG_FIFO_STATUS	FIFO status
8'hB–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	AXI_DEBUG_USER_DEFINED	User-defined debug

1.2 APB4 Event Codes

Six categories cover the APB4 monitor surface: error, timeout, completion, threshold, performance, and debug. The protocol field for every code in this section is `PROTOCOL_APB` (4'h2).

1.2.1 `apb_error_code_t`

Packet context: `packet_type = PktTypeError, protocol = PROTOCOL_APB`

Value	Name	Description
8'h0	APB_ERR_PSLVERR	Peripheral slave error
8'h1	APB_ERR_SETUP_VIOLATION	Setup phase protocol violation

Value	Name	Description
8'h2	APB_ERR_ACCESS_VIOLATION	Access phase protocol violation
8'h3	APB_ERR_STROBE_ERROR	Write strobe error
8'h4	APB_ERR_ADDR_DECODE	Address decode error
8'h5	APB_ERR_PROT_VIOLATION	Protection violation (PPROT)
8'h6	APB_ERR_ENABLE_ERROR	Enable phase error
8'h7	APB_ERR_READY_ERROR	PREADY protocol error
8'h8	APB_ERR_ADDR_RANGE	Address-range violation (from apb_monitor_addr_check)
8'h9–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	APB_ERR_USER_DEFINED	User-defined error

1.2.2 apb_timeout_code_t

Packet context: packet_type = PktTypeTimeout, protocol = PROTOCOL_APB

Value	Name	Description
8'h0	APB_TIMEOUT_SETUP	Setup phase timeout
8'h1	APB_TIMEOUT_ACCESS	Access phase timeout
8'h2	APB_TIMEOUT_ENABLE	Enable phase timeout (PREADY stuck)
8'h3	APB_TIMEOUT_PREADY_STUCK	PREADY stuck low
8'h4	APB_TIMEOUT_TRANSFER	Overall transfer timeout
8'h5–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	APB_TIMEOUT_USER_DEFINED	User-defined timeout

1.2.3 apb_completion_code_t

Packet context: packet_type = PktTypeCompletion, protocol = PROTOCOL_APB

Value	Name	Description
8'h0	APB_COMPL_TRANS_COMPLETE	Transaction completed
8'h1	APB_COMPL_READ_COMPLETE	Read transaction complete
8'h2	APB_COMPL_WRITE_COMPLETE	Write transaction complete
8'h3–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	APB_COMPL_USER_DEFINED	User-defined completion

1.2.4 apb_threshold_code_t

Packet context: packet_type = PktTypeThreshold, protocol = PROTOCOL_APB

Value	Name	Description
8'h0	APB_THRESH_LATENCY	Transaction latency threshold
8'h1	APB_THRESH_ERROR_RATE	Error rate threshold
8'h2	APB_THRESH_ACTIVE_COUNT	Active transaction count threshold
8'h3	APB_THRESH_THROUGHPUT	Throughput threshold
8'h4–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	APB_THRESH_USER_DEFINED	User-defined threshold

1.2.5 apb_performance_code_t

Packet context: packet_type = PktTypePerf, protocol = PROTOCOL_APB

Value	Name	Description
8'h0	APB_PERF_READ_LATENCY	Read transaction latency
8'h1	APB_PERF_WRITE_LATENCY	Write transaction latency
8'h2	APB_PERF_THROUGHPUT	Transaction

Value	Name	Description
		throughput
8'h3	APB_PERF_ERROR_RATE	Error rate
8'h4	APB_PERF_ACTIVE_COUNT	Active transaction count
8'h5	APB_PERF_COMPLETED_COUNT	Completed transaction count
8'h6–8'hE	(reserved)	<i>Reserved for future use</i>
8'hF	APB_PERF_USER_DEFINED	User-defined performance

1.2.6 apb_debug_code_t

Packet context: packet_type = PktTypeDebug, protocol = PROTOCOL_APB

Value	Name	Description
8'h0	APB_DEBUG_STATE_CHANGE	APB state changed
8'h1	APB_DEBUG_SETUP_PHASE	Setup phase event
8'h2	APB_DEBUG_ACCESS_PHASE	Access phase event
8'h3	APB_DEBUG_ENABLE_PHASE	Enable phase event
8'h4	APB_DEBUG_PSEL_TRACE	PSEL trace
8'h5	APB_DEBUG_PENABLE_TRACE	PENABLE trace
8'h6	APB_DEBUG_PREADY_TRACE	PREADY trace
8'h7	APB_DEBUG_PPROT_TRACE	PPROT trace
8'h8	APB_DEBUG_PSTRB_TRACE	PSTRB trace
8'h9–8'hE	(reserved)	<i>Reserved for future use</i>
8'hF	APB_DEBUG_USER_DEFINED	User-defined debug

1.3 AXIS4 Event Codes

Six categories cover the AXI4-Stream monitor surface: error, timeout, completion, credit, channel, and stream. The protocol field for every code in this section is `PROTOCOL_AXIS` (4'h1).

1.3.1 axis_error_code_t

Packet context: `packet_type = PktTypeError, protocol = PROTOCOL_AXIS`

Value	Name	Description
8'h0	AXIS_ERR_PROTOCOL	Protocol violation
8'h1	AXIS_ERR_READY_TIMING	TREADY timing violation
8'h2	AXIS_ERR_VALID_TIMING	TVALID timing violation
8'h3	AXIS_ERR_LAST_MISSING	Missing TLAST signal
8'h4	AXIS_ERR_LAST_ORPHAN	Orphaned TLAST signal
8'h5	AXIS_ERR_STRB_INVALID	Invalid TSTRB pattern
8'h6	AXIS_ERR_KEEP_INVALID	Invalid TKEEP pattern
8'h7	AXIS_ERR_DATA_ALIGNMENT	Data alignment error
8'h8	AXIS_ERR_ID_VIOLATION	TID violation
8'h9	AXIS_ERR_DEST_VIOLATION	TDEST violation
8'hA	AXIS_ERR_USER_VIOLATION	TUSER violation
8'hB–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	AXIS_ERR_USER_DEFINED	User-defined error

1.3.2 axis_timeout_code_t

Packet context: packet_type = PktTypeTimeout, protocol = PROTOCOL_AXIS

Value	Name	Description
8'h0	AXIS_TIMEOUT_HANDSHAKE	TVALID/TREADY handshake timeout
8'h1	AXIS_TIMEOUT_STREAM	Stream completion timeout
8'h2	AXIS_TIMEOUT_PACKET	Packet timeout (TLAST)
8'h3	AXIS_TIMEOUT_BACKPRESSURE	Backpressure timeout
8'h4	AXIS_TIMEOUT_BUFFER	Buffer drain timeout
8'h5	AXIS_TIMEOUT_STALL	Stream stall timeout
8'h6–8'hE	(reserved)	Reserved for future use
8'hF	AXIS_TIMEOUT_USER_DEFINED	User-defined timeout

1.3.3 axis_completion_code_t

Packet context: packet_type = PktTypeCompletion, protocol = PROTOCOL_AXIS

Value	Name	Description
8'h0	AXIS_COMPL_STREAM_END	Stream completed (TLAST)
8'h1	AXIS_COMPL_PACKET_SENT	Packet sent successfully
8'h2	AXIS_COMPL_TRANSFER	Data transfer completed
8'h3	AXIS_COMPL_BURST_END	Burst completed
8'h4	AXIS_COMPL_HANDSHAKE	Successful handshake
8'h5–8'hE	(reserved)	Reserved for future use
8'hF	AXIS_COMPL_USER_DEFINED	User-defined completion

1.3.4 axis_credit_code_t

Packet context: packet_type = PktTypeCredit, protocol = PROTOCOL_AXIS

Value	Name	Description
8'h0	AXIS_CREDIT_READY_ASSERT	TREADY asserted
8'h1	AXIS_CREDIT_READY_DEASSERT	TREADY deasserted
8'h2	AXIS_CREDIT_BUFFER_AVAILABLE	Buffer space available
8'h3	AXIS_CREDIT_BUFFER_FULL	Buffer full
8'h4	AXIS_CREDIT_FLOW_CONTROL	Flow control event
8'h5	AXIS_CREDIT_BACKPRESSURE	Backpressure applied
8'h6	AXIS_CREDIT_THROUGHPUT	Throughput event
8'h7	AXIS_CREDIT_TRANSFER EFFICIENCY	Transfer efficiency
8'h8–8'hE	(reserved)	Reserved for future use
8'hF	AXIS_CREDIT_USER_DEFINED	User-defined credit event

1.3.5 axis_channel_code_t

Packet context: packet_type = PktTypeChannel, protocol = PROTOCOL_AXIS

Value	Name	Description
8'h0	AXIS_CHAN_CONNECT	Channel connected
8'h1	AXIS_CHAN_DISCONNECT	Channel disconnected
8'h2	AXIS_CHAN_STALL	Channel stalled
8'h3	AXIS_CHAN_RESUME	Channel resumed
8'h4	AXIS_CHAN_CONGESTION	Channel congestion
8'h5	AXIS_CHAN_ID_CHANGE	TID change detected
8'h6	AXIS_CHAN_DEST_CHANGE	TDEST change detected

Value	Name	Description
8'h7	AXIS_CHAN_CONFIG_CHANGE	Channel configuration change
8'h8–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	AXIS_CHAN_USER_DEFINED	User-defined channel event

1.3.6 axis_stream_code_t

Packet context: packet_type = PktTypeStream, protocol = PROTOCOL_AXIS

Value	Name	Description
8'h0	AXIS_STREAM_START	Stream started
8'h1	AXIS_STREAM_END	Stream ended (TLAST)
8'h2	AXIS_STREAM_PAUSE	Stream paused
8'h3	AXIS_STREAM_RESUME	Stream resumed
8'h4	AXIS_STREAM_OVERFLOW	Stream buffer overflow
8'h5	AXIS_STREAM_UNDERFLOW	Stream buffer underflow
8'h6	AXIS_STREAM_TRANSFER	Active data transfer
8'h7	AXIS_STREAM_IDLE	Stream idle
8'h8–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	AXIS_STREAM_USER_DEFINED	User-defined stream event

1.4 Unified Event Code Union

monitor_amba4_pkg also exports a unified_event_code_t packed union that overlays every enum in this document onto a single 8-bit field, plus a raw view for direct 8-bit access. Helper functions create_axi*_event, create_apb*_event, and create_axis*_event (one per

enum) construct the union from a typed enum value — use these from any wrapper that needs to publish a protocol-tagged event_code without manual bit packing.

1.5 Related

- **monitor_package_spec.md** — Universal types, packet layout, helper functions.
 - **monitor_amba5_pkg.md** — AXI5 / APB5 / AXIS5 extended event codes.
 - **monitor_arbiter_pkg.md** — ARB and CORE event codes.
 - **monitor_system_whitepaper.md** — Design-surface view of the monitor system.
-

1.6 Navigation

- [← Back to Includes Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

2 AMBA5 Extended Event Code Reference

This is the per-protocol event-code reference for the AMBA5 family — AXI5, APB5, and AXIS5. The enums here **extend** the AMBA4 set with the new event categories introduced by the AMBA5 specifications: atomic operations and tracing on AXI5; wake-up signaling, parity, and user signals on APB5; wake-up, parity, and CRC on AXIS5. For the AMBA4 baseline see `monitor_amba4_pkg.md`; for the universal packet layout and helper functions see `monitor_package_spec.md`.

Source: `rtl/amba/includes/monitor_amba5_pkg.sv`. Each enum is 8 bits wide; slot 8'hF is the *_USER_DEFINED escape hatch. AMBA5 codes reuse the AMBA4 protocol values (PROTOCOL_AXI / PROTOCOL_APB / PROTOCOL_AXIS); the distinguishing field is the chosen event_code enum and the packet_type context the producer uses.

2.1 AXI5 Extended Event Codes

Two new categories on top of the AXI4 baseline: atomic-operation events and QoS / trace events.

2.1.1 axi5_atomic_code_t

Packet context: extends axi_completion_code_t semantics (packet_type = PktTypeCompletion, protocol = PROTOCOL_AXI) — used to report which AXI5 atomic primitive completed.

Value	Name	Description
8'h0	AXI5_ATOMIC_LOAD	Atomic load operation
8'h1	AXI5_ATOMIC_SWAP	Atomic swap operation
8'h2	AXI5_ATOMIC_COMPARE	Atomic compare operation
8'h3	AXI5_ATOMIC_ADD	Atomic add operation
8'h4	AXI5_ATOMIC_CLR	Atomic clear operation
8'h5	AXI5_ATOMIC_XOR	Atomic XOR operation
8'h6	AXI5_ATOMIC_SET	Atomic set operation
8'h7	AXI5_ATOMIC_SMAX	Atomic signed max
8'h8	AXI5_ATOMIC_SMIN	Atomic signed min
8'h9	AXI5_ATOMIC_UMAX	Atomic unsigned max
8'hA	AXI5_ATOMIC_UMIN	Atomic unsigned min
8'hB–8'hE	(reserved)	Reserved for future use
8'hF	AXI5_ATOMIC_USER_DEFINED	User-defined atomic

2.1.2 axi5_trace_code_t

Packet context: packet_type = PktTypeDebug (trace / QoS / poison / MPAM events), protocol = PROTOCOL_AXI.

Value	Name	Description
8'h0	AXI5_TRACE_START	Trace session start

Value	Name	Description
8'h1	AXI5_TRACE_END	Trace session end
8'h2	AXI5_TRACE_DATA	Trace data packet
8'h3	AXI5_QOS_ESCALATION	QoS level escalation
8'h4	AXI5_QOS_DEESCALATION	QoS level de-escalation
8'h5	AXI5_POISON_DETECTED	Poison bit detected
8'h6	AXI5_LOOP_DETECTED	Loop detection triggered
8'h7	AXI5_MPAM_EVENT	MPAM partition event
8'h8–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	AXI5_USER_DEFINED	User-defined

2.2 APB5 Extended Event Codes

Three new categories on top of the APB4 baseline: wake-up signaling (PWAKEUP), parity (PPARITY / PRDATAPARITY / PREADYPARITY / PSLVERRPARITY), and user signals (PUSER / PSUSER).

2.2.1 apb5_wakeup_code_t

Packet context: packet_type = PktTypeDebug (or producer-defined), protocol = PROTOCOL_APB — APB5 wake-up / sleep transitions.

Value	Name	Description
8'h0	APB5_WAKEUP_REQUEST	PWAKEUP asserted by slave
8'h1	APB5_WAKEUP_ACKNOWLEDGED	Wake-up acknowledged
8'h2	APB5_WAKEUP_TIMEOUT	Wake-up request timeout
8'h3	APB5_WAKEUP_REJECTED	Wake-up request rejected

Value	Name	Description
8'h4	APB5_SLEEP_REQUEST	Sleep mode request
8'h5	APB5_SLEEP_ENTERED	Entered sleep mode
8'h6–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	APB5_WAKEUP_USER_DEFINED	User-defined wake-up

2.2.2 apb5_parity_code_t

Packet context: packet_type = PktTypeError (parity errors) or PktTypeDebug (status), protocol = PROTOCOL_APB.

Value	Name	Description
8'h0	APB5_PARITY_PWDATA_ERROR	PWDATA parity error (PPARITY)
8'h1	APB5_PARITY_PRDATA_ERROR	PRDATA parity error (PRDATAPARITY)
8'h2	APB5_PARITY_PREADY_ERROR	PREADY parity error (PREADYPARITY)
8'h3	APB5_PARITY_PSLVERR_ERROR	PSLVERR parity error (PSLVERRPARITY)
8'h4	APB5_PARITY_CORRECTED	Parity error corrected
8'h5	APB5_PARITY_UNCORRECTED	Parity error uncorrectable
8'h6	APB5_PARITY_CHECK_DISABLED	Parity checking disabled
8'h7	APB5_PARITY_CHECK_ENABLED	Parity checking enabled
8'h8–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	APB5_PARITY_USER_DEFINED	User-defined parity

2.2.3 apb5_user_code_t

Packet context: packet_type = PktTypeDebug, protocol = PROTOCOL_APB — APB5 PUSER / PSUSER side-band signal events.

Value	Name	Description
8'h0	APB5_USER_PUSER_VALID	PUSER valid on LID

Value	Name	Description
		master
8'h1	APB5_USER_PSUSER_VALID	PSUSER valid on slave
8'h2	APB5_USER_SIGNAL_MISMATCH	User signal mismatch
8'h3–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	APB5_USER_USER_DEFINED	User-defined

2.3 AXIS5 Extended Event Codes

Three new categories on top of the AXIS4 baseline: wake-up (TWAKEUP), parity (TPARITY), and CRC (TCRC_ERROR).

2.3.1 axis5_wakeup_code_t

Packet context: packet_type = PktTypeDebug (or producer-defined), protocol = PROTOCOL_AXIS — AXIS5 wake-up / sleep transitions.

Value	Name	Description
8'h0	AXIS5_WAKEUP_REQUEST	TWAKEUP asserted
8'h1	AXIS5_WAKEUP_ACKNOWLEDGED	Wake-up acknowledged
8'h2	AXIS5_WAKEUP_TIMEOUT	Wake-up timeout
8'h3	AXIS5_WAKEUP_ACTIVE	Wake-up active, data pending
8'h4	AXIS5_SLEEP_ENTERING	Entering sleep mode
8'h5	AXIS5_SLEEP_EXITING	Exiting sleep mode
8'h6–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	AXIS5_WAKEUP_USER_DEFINED	User-defined wake-up

2.3.2 axis5_parity_code_t

Packet context: packet_type = PktTypeError (parity errors) or PktTypeDebug (status), protocol = PROTOCOL_AXIS.

Value	Name	Description
8'h0	AXIS5_PARITY_TDATA_ERROR	TDATA parity error (TPARITY)
8'h1	AXIS5_PARITY_CORRECTED	Parity error corrected
8'h2	AXIS5_PARITY_UNCORRECTED	Parity error uncorrectable
8'h3	AXIS5_PARITY_CHECK_DISABLED	Parity checking disabled
8'h4	AXIS5_PARITY_CHECK_ENABLED	Parity checking enabled
8'h5–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	AXIS5_PARITY_USER_DEFINED	User-defined parity

2.3.3 axis5_crc_code_t

Packet context: packet_type = PktTypeError (CRC failures) or PktTypeDebug (status), protocol = PROTOCOL_AXIS — optional CRC feature in AXIS5.

Value	Name	Description
8'h0	AXIS5_CRC_VALID	CRC check passed
8'h1	AXIS5_CRC_ERROR	CRC error detected (TCRC_ERROR)
8'h2	AXIS5_CRC_COMPUTED	CRC computed and sent
8'h3	AXIS5_CRC_DISABLED	CRC checking disabled
8'h4	AXIS5_CRC_ENABLED	CRC checking enabled
8'h5–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	AXIS5_CRC_USER_DEFINED	User-defined CRC

2.4 Unified Event Code Union

`monitor_amba5_pkg` exports an `amba5_event_code_t` packed union that overlays the eight AMBA5 enums above (plus a raw 8-bit view) onto a single field. Helper functions `create_axi5_atomic_event`, `create_axi5_trace_event`, `create_apb5_wakeup_event`, `create_apb5_parity_event`, `create_apb5_user_event`, `create_axis5_wakeup_event`, `create_axis5_parity_event`, and `create_axis5_crc_event` construct the union from a typed enum value.

2.5 Related

- `monitor_package_spec.md` — Universal types, packet layout, helper functions.
 - `monitor_amba4_pkg.md` — AXI4 / APB4 / AXIS4 baseline event codes.
 - `monitor_arbiter_pkg.md` — ARB and CORE event codes.
 - `monitor_system_whitepaper.md` — Design-surface view of the monitor system.
-

2.6 Navigation

- [← Back to Includes Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

3 ARB and CORE Event Code Reference

This is the per-protocol event-code reference for the two non-AMBA monbus protocols: **ARB** (arbiter monitoring — fairness, starvation, weighted round-robin, ACK protocol) and **CORE** (internal accelerator / DMA-engine monitoring — descriptor fetch, credit cycles, sub-engine state). Both sets follow the same packet layout as the AMBA monitors; only the `protocol` and `event_code` fields differ. For the universal packet layout and helper functions see `monitor_package_spec.md`.

Source: `rtl/amba/includes/monitor_arbiter_pkg.sv`. Each enum is 8 bits wide; slot 8'hF is the `*_USER_DEFINED` escape hatch. ARB codes carry `PROTOCOL_ARB` (4'h3); CORE codes carry `PROTOCOL_CORE` (4'h4).

3.1 ARB Event Codes

Six categories cover the arbiter monitor surface: error, timeout, completion, threshold, performance, and debug. The protocol field for every code in this section is `PROTOCOL_ARB` (4'h3).

3.1.1 `arb_error_code_t`

Packet context: `packet_type = PktTypeError, protocol = PROTOCOL_ARB`

Value	Name	Description
8'h0	<code>ARB_ERR_STARVATION</code>	Client request starvation
8'h1	<code>ARB_ERR_ACK_TIMEOUT</code>	Grant ACK timeout
8'h2	<code>ARB_ERR_PROTOCOL_VIOLATION</code>	ACK protocol violation
8'h3	<code>ARB_ERR_CREDIT_VIOLATION</code>	Credit system violation
8'h4	<code>ARB_ERR_FAIRNESS_VIOLATION</code>	Weighted fairness violation
8'h5	<code>ARB_ERR_WEIGHT_UNDERFLOW</code>	Weight credit underflow
8'h6	<code>ARB_ERR_CONCURRENT_GRANTS</code>	Multiple simultaneous grants
8'h7	<code>ARB_ERR_INVALID_GRANT_ID</code>	Invalid grant ID detected
8'h8	<code>ARB_ERR_ORPHAN_ACK</code>	ACK without pending grant
8'h9	<code>ARB_ERR_GRANT_OVERLAP</code>	Overlapping grant periods
8'hA	<code>ARB_ERR_MASK_ERROR</code>	Round-robin mask error
8'hB	<code>ARB_ERR_STATE_MACHINE</code>	FSM state error
8'hC	<code>ARB_ERR_CONFIGURATION</code>	Invalid configuration
8'hD–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	<code>ARB_ERR_USER_DEFINED</code>	User-defined error

3.1.2 `arb_timeout_code_t`

Packet context: `packet_type = PktTypeTimeout, protocol = PROTOCOL_ARB`

Value	Name	Description
8'h0	ARB_TIMEOUT_GRANT_ACK	Grant ACK timeout
8'h1	ARB_TIMEOUT_REQUEST_HOLD	Request held too long
8'h2	ARB_TIMEOUT_WEIGHT_UPDATE	Weight update timeout
8'h3	ARB_TIMEOUT_BLOCK_RELEASE	Block release timeout
8'h4	ARB_TIMEOUT_CREDIT_UPDATE	Credit update timeout
8'h5	ARB_TIMEOUT_STATE_CHANGE	State machine timeout
8'h6–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	ARB_TIMEOUT_USER_DEFINED	User-defined timeout

3.1.3 arb_completion_code_t

Packet context: packet_type = PktTypeCompletion, protocol = PROTOCOL_ARB

Value	Name	Description
8'h0	ARB_COMPL_GRANT_ISSUED	Grant successfully issued
8'h1	ARB_COMPL_ACK_RECEIVED	ACK successfully received
8'h2	ARB_COMPL_TRANSACTION	Complete transaction (grant+ack)
8'h3	ARB_COMPL_WEIGHT_UPDATE	Weight update completed
8'h4	ARB_COMPL_CREDIT_CYCLE	Credit cycle completed
8'h5	ARB_COMPL_FAIRNESS_PERIOD	Fairness analysis period
8'h6	ARB_COMPL_BLOCK_PERIOD	Block period completed
8'h7–8'hE	<i>(reserved)</i>	<i>Reserved for future use</i>
8'hF	ARB_COMPL_USER_DEFINED	User-defined completion

3.1.4 arb_threshold_code_t

Packet context: packet_type = PktTypeThreshold, protocol = PROTOCOL_ARB

Value	Name	Description
8'h0	ARB_THRESH_REQUEST_LATENCY	Request-to-grant latency threshold
8'h1	ARB_THRESH_ACK_LATENCY	Grant-to-ACK latency threshold
8'h2	ARB_THRESH_FAIRNESS_DEV	Fairness deviation threshold
8'h3	ARB_THRESH_ACTIVE_REQUESTS	Active request count threshold
8'h4	ARB_THRESH_GRANT_RATE	Grant rate threshold
8'h5	ARB_THRESH EFFICIENCY	Grant efficiency threshold
8'h6	ARB_THRESH_CREDIT_LOW	Low credit threshold
8'h7	ARB_THRESH_WEIGHT_IMBALANCE	Weight imbalance threshold
8'h8	ARB_THRESH_STARVATION_TIME	Starvation time threshold
8'h9–8'hE	(reserved)	Reserved for future use
8'hF	ARB_THRESH_USER_DEFINED	User-defined threshold

3.1.5 arb_performance_code_t

Packet context: packet_type = PktTypePerf, protocol = PROTOCOL_ARB

Value	Name	Description
8'h0	ARB_PERF_GRANT_ISSUED	Grant issued event
8'h1	ARB_PERF_ACK_RECEIVED	ACK received event
8'h2	ARB_PERF_GRANT EFFICIENCY	Grant completion efficiency
8'h3	ARB_PERF_FAIRNESS_METRIC	Fairness compliance metric
8'h4	ARB_PERF_THROUGHPUT	Arbitration throughput
8'h5	ARB_PERF_LATENCY	Average latency measurement

Value	Name	Description
	_AVG	
8'h6	ARB_PERF_WEIGHT_COMPLIANCE	Weight compliance metric
8'h7	ARB_PERF_CREDIT_UTILIZATION	Credit utilization efficiency
8'h8	ARB_PERF_CLIENT_ACTIVITY	Per-client activity metric
8'h9	ARB_PERF_STARVATION_COUNT	Starvation event count
8'hA	ARB_PERF_BLOCK EFFICIENCY	Block/unblock efficiency
8'hB–8'hE	(reserved)	Reserved for future use
8'hF	ARB_PERF_USER_DEFINED	User-defined performance

3.1.6 arb_debug_code_t

Packet context: packet_type = PktTypeDebug, protocol = PROTOCOL_ARB

Value	Name	Description
8'h0	ARB_DEBUG_STATE_CHANGE	Arbiter state machine change
8'h1	ARB_DEBUG_MASK_UPDATE	Round-robin mask update
8'h2	ARB_DEBUG_WEIGHT_CHANGE	Weight configuration change
8'h3	ARB_DEBUG_CREDIT_UPDATE	Credit level update
8'h4	ARB_DEBUG_CLIENT_MASK	Client enable/disable mask
8'h5	ARB_DEBUG_PRIORITY_CHANGE	Priority level change
8'h6	ARB_DEBUG_BLOCK_EVENT	Block/unblock event
8'h7	ARB_DEBUG_QUEUE_STATUS	Request queue status
8'h8	ARB_DEBUG_COUNTER_SNAPSHOT	Counter values snapshot
8'h9	ARB_DEBUG_FIFO_STATUS	FIFO status change
8'hA	ARB_DEBUG_FAIRNESS	Fairness tracking state

Value	Name	Description
	SS_STATE	
8'hB	ARB_DEBUG_ACK_STATE	ACK protocol state
8'hC–8'hE	(reserved)	Reserved for future use
8'hF	ARB_DEBUG_USER_DEFINED	User-defined debug

3.2 CORE Event Codes

Six categories cover the core/accelerator monitor surface: error, timeout, completion, threshold, performance, and debug. The protocol field for every code in this section is `PROTOCOL_CORE` (4'h4). These codes are used by custom DMA / accelerator subsystems (descriptor engines, control read/write engines, credit-based flow control).

3.2.1 core_error_code_t

Packet context: `packet_type = PktTypeError, protocol = PROTOCOL_CORE`

Value	Name	Description
8'h0	CORE_ERR_DESCRIPTOR_MALFORMED	Missing magic number (0x900dc0de)
8'h1	CORE_ERR_DESCRIPTOR_BAD_ADDR	Invalid descriptor address
8'h2	CORE_ERR_DATA_BAD_ADDR	Invalid data address (fetch or runtime)
8'h3	CORE_ERR_FLAG_COMPARE_MISMATCH	Flag mask/compare mismatch
8'h4	CORE_ERR_CREDIT_UNDERFLOW	Credit system violation
8'h5	CORE_ERR_STATE_MACHINE	Invalid FSM state transition
8'h6	CORE_ERR_DESCRIPTOR_ENGINE	Descriptor engine FSM error
8'h7	CORE_ERR_FLAG_ENGINE	Flag engine FSM error (legacy — pre-ctrlrd)
8'h8	CORE_ERR_CTRLWR_ENGINE	Control write engine FSM error
8'h9	CORE_ERR_DATA_ENGINE	Data engine error
8'hA	CORE_ERR_CHANNEL_INVALID	Invalid channel ID

Value	Name	Description
8'hB	CORE_ERR_CONTROL_VIOLATION	Control register violation
8'hC	CORE_ERR_OVERFLOW	Overflow
8'hD	CORE_ERR_CTRLRD_MAX_RETRIES	Control read max retries exceeded
8'hE	CORE_ERR_CTRLRD_ENGINE	Control read engine FSM error
8'hF	CORE_ERR_USER_DEFINED	User-defined error

3.2.2 core_timeout_code_t

Packet context: packet_type = PktTypeTimeout, protocol = PROTOCOL_CORE

Value	Name	Description
8'h0	CORE_TIMEOUT_DESCRIPTOR_FETCH	Descriptor fetch timeout
8'h1	CORE_TIMEOUT_CTRLRD_RETRY	Control read retry timeout
8'h2	CORE_TIMEOUT_CTRLWR_WRITE	Control write timeout
8'h3	CORE_TIMEOUT_DATA_TRANSFER	Data transfer timeout
8'h4	CORE_TIMEOUT_CREDIT_WAIT	Credit wait timeout
8'h5	CORE_TIMEOUT_CONTROL_WAIT	Control enable wait timeout
8'h6	CORE_TIMEOUT_ENGINE_RESPONSE	Sub-engine response timeout
8'h7	CORE_TIMEOUT_STATE_TRANSITION	FSM state transition timeout
8'h8–8'hE	(reserved)	Reserved for future use
8'hF	CORE_TIMEOUT_USER_DEFINED	User-defined timeout

3.2.3 core_completion_code_t

Packet context: packet_type = PktTypeCompletion, protocol = PROTOCOL_CORE

Value	Name	Description
8'h0	CORE_COMPL_DESCR	Descriptor successfully loaded

Value	Name	Description
	IPTOR_LOADED	
8'h1	CORE_COMPL_DESCR IPTOR_CHAIN	Descriptor chain completed
8'h2	CORE_COMPL_CTRLR D_COMPLETED	Control read completed (with match)
8'h3	CORE_COMPL_CTRLW R_COMPLETED	Control write completed
8'h4	CORE_COMPL_DATA_ TRANSFER	Data transfer completed
8'h5	CORE_COMPL_CREDI T_CYCLE	Credit cycle completed
8'h6	CORE_COMPL_CHANN EL_COMPLETE	Channel processing complete
8'h7	CORE_COMPL_ENGIN E_READY	Sub-engine ready
8'h8–8'hE	(reserved)	Reserved for future use
8'hF	CORE_COMPL_USER_ DEFINED	User-defined completion

3.2.4 core_threshold_code_t

Packet context: packet_type = PktTypeThreshold, protocol = PROTOCOL_CORE

Value	Name	Description
8'h0	CORE_THRESH_DESC RIPTOR_QUEUE	Descriptor queue depth threshold
8'h1	CORE_THRESH_CRED IT_LOW	Credit low threshold
8'h2	CORE_THRESH_FLAG _RETRY_COUNT	Flag retry count threshold
8'h3	CORE_THRESH_LATE NCY	Processing latency threshold
8'h4	CORE_THRESH_ERR0 R_RATE	Error rate threshold
8'h5	CORE_THRESH_THRO UGHPUT	Throughput threshold
8'h6	CORE_THRESH_ACTI VE_CHANNELS	Active channel count threshold
8'h7	CORE_THRESH_PROG RAM_LATENCY	Program write latency threshold
8'h8	CORE_THRESH_DATA	Data transfer rate threshold

Value	Name	Description
	_RATE	
8'h9–8'hE	(reserved)	<i>Reserved for future use</i>
8'hF	CORE_THRESH_USER_DEFINED	User-defined threshold

3.2.5 core_performance_code_t

Packet context: packet_type = PktTypePerf, protocol = PROTOCOL_CORE

Value	Name	Description
8'h0	CORE_PERF_END_OF_DATA	Stream continuation signal
8'h1	CORE_PERF_END_OF_STREAM	Stream termination signal
8'h2	CORE_PERF_ENTERING_IDLE	FSM returning to idle
8'h3	CORE_PERF_CREDIT_INCREMENTED	Credit added by software
8'h4	CORE_PERF_CREDIT_EXHAUSTED	Credit blocking execution
8'h5	CORE_PERF_STATE_TRANSITION	FSM state change
8'h6	CORE_PERF_DESCRIPTOR_ACTIVE	Data processing started
8'h7	CORE_PERF_CTRLRD_RETRY	Control read retry attempt
8'h8	CORE_PERF_CHANNEL_ENABLE	Channel enabled by software
8'h9	CORE_PERF_CHANNEL_DISABLE	Channel disabled by software
8'hA	CORE_PERF_CREDIT_UTILIZATION	Credit utilization metric
8'hB	CORE_PERF_PROCESSING_LATENCY	Total processing latency
8'hC	CORE_PERF_QUEUE_DEPTH	Current queue depth
8'hD–8'hE	(reserved)	<i>Reserved for future use</i>
8'hF	CORE_PERF_USER_DEFINED	User-defined performance

3.2.6 core_debug_code_t

Packet context: packet_type = PktTypeDebug, protocol = PROTOCOL_CORE

Value	Name	Description
8'h0	CORE_DEBUG_FSM_STATE_CHANGE	Descriptor FSM state change
8'h1	CORE_DEBUG_DESCRIPTOR_CONTENT	Descriptor content trace
8'h2	CORE_DEBUG_CTRLRD_ENGINE_STATE	Control read engine state trace
8'h3	CORE_DEBUG_CTRLWR_ENGINE_STATE	Control write engine state trace
8'h4	CORE_DEBUG_CREDIT_OPERATION	Credit system operation
8'h5	CORE_DEBUG_CONTROL_REGISTER	Control register access
8'h6	CORE_DEBUG_ENGINE_HANDSHAKE	Engine handshake trace
8'h7	CORE_DEBUG_QUEUE_STATUS	Queue status change
8'h8	CORE_DEBUG_COUNTER_SNAPSHOT	Counter values snapshot
8'h9	CORE_DEBUG_ADDRESS_TRACE	Address progression trace
8'hA	CORE_DEBUG_PAYLOAD_TRACE	Payload content trace
8'hB–8'hE	(reserved)	Reserved for future use
8'hF	CORE_DEBUG_USER_DEFINED	User-defined debug

3.3 Unified Event Code Union (arb_core_event_code_t)

monitor_arbiter_pkg exports a single arb_core_event_code_t packed union that overlays all twelve enums (six ARB + six CORE) plus a raw 8-bit view onto a single field. Use this when a wrapper carries event codes from either protocol on a shared bus — the protocol field of the monbus packet disambiguates which enum interpretation applies.

```
typedef union packed {
    arb_error_code_t      arb_error;
    arb_timeout_code_t    arb_timeout;
    arb_completion_code_t arb_completion;
    arb_threshold_code_t  arb_threshold;
```

```

arb_performance_code_t  arb_performance;
arb_debug_code_t        arb_debug;

core_error_code_t        core_error;
core_timeout_code_t      core_timeout;
core_completion_code_t   core_completion;
core_threshold_code_t    core_threshold;
core_performance_code_t  core_performance;
core_debug_code_t        core_debug;

logic [7:0]              raw;
} arb_core_event_code_t;

```

Helper constructors `create_arb_error_event`, `create_arb_timeout_event`, `create_arb_completion_event`, `create_arb_threshold_event`, `create_arb_performance_event`, and `create_arb_debug_event` build the union from a typed ARB enum value. (Equivalent CORE constructors are not exported by the package — code producing CORE codes assigns the union field directly.)

3.4 Related

- **monitor_package_spec.md** — Universal types, packet layout, helper functions.
 - **monitor_amba4_pkg.md** — AXI4 / APB4 / AXIS4 event codes.
 - **monitor_amba5_pkg.md** — AXI5 / APB5 / AXIS5 extended event codes.
 - **monitor_system_whitepaper.md** — Design-surface view of the monitor system.
-

3.5 Navigation

- [← Back to Includes Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

4 Monitor Package Specification

This is the canonical reference for the monitor-bus packet format and the universal types shared by every protocol monitor in the AMBA subsystem. For per-protocol event-code enums see the three sibling docs:

- `monitor_amba4_pkg.md` — AXI4 / APB4 / AXIS4 event codes
 - `monitor_amba5_pkg.md` — AXI5 / APB5 / AXIS5 extended codes
 - `monitor_arbiter_pkg.md` — ARB and CORE event codes
-

4.1 Package Organization

The RTL splits monitor type definitions across four SystemVerilog packages:

Package	File	Purpose
<code>monitor_common_pkg</code>	<code>rtl/amba/includes/monitor_common_pkg.sv</code>	Universal types: packet structure, timestamp, protocol enum, packet-type enum, helper functions. This doc.
<code>monitor_amba4_pkg</code>	<code>rtl/amba/includes/monitor_amba4_pkg.sv</code>	AXI4 / APB4 / AXIS4 event-code enums (error / timeout / completion / threshold / perf / addr-match / debug).
<code>monitor_amba5_pkg</code>	<code>rtl/amba/includes/monitor_amba5_pkg.sv</code>	AXI5 / APB5 / AXIS5 extended event-code enums (atomic, trace, wakeup, parity, user).
<code>monitor_arbiter_pkg</code>	<code>rtl/amba/includes/monitor_arbiter_pkg.sv</code>	ARB and CORE event codes for arbiter monitors and custom subsystems.

A wrapper `monitor_pkg` re-exports all four for back-compatibility — code that imports `monitor_pkg::*` continues to compile unchanged.

4.2 The Monbus Record

A monbus emission is two paired wires carried atomically through every level of the transport: a **128-bit packet** (`monbus_packet`) plus a **64-bit side-band timestamp** (`monbus_timestamp`). 192 bits total.

Wire	Bits	Width	Field	Owner	Notes
monbus_timestamp	[63:0]	64	timestamp	system	Free-running counter from monbus_axi_group, sampled at emission. Consumers treat it as an opaque ordering key.
monbus_packet	[127:124]	4	packet_type	enum (fixed)	See Packet Type Enum .
monbus_packet	[123:109]	15	reserved	(slack)	Currently emitted as zero.
monbus_packet	[108:105]	4	protocol	enum (fixed)	See Protocol Enum .
monbus_packet	[104:97]	8	event_code	enum (fixed)	Protocol-specific; see sibling docs.
monbus_packet	[96:88]	9	channel_id	designer	AXI ID or channel index.
monbus_packet	[87:72]	16	agent_id	designer	Designer-allocated agent ID.
monbus_packet	[71:64]	8	unit_id	designer	Designer-allocated unit ID.
monbus_packet	[63:0]	64	event_data	payload	Layout depends on (packet_type)

Wire	Bits	Width	Field	Owner	Notes
					e, event_code) — see event_data Payload Convention S.

4.2.1 SystemVerilog Type Aliases

```
localparam int MONBUS_PKT_WIDTH = 128;
localparam int MONBUS_TS_WIDTH = 64;
```

```
typedef logic [MONBUS_PKT_WIDTH-1:0] monitor_packet_t;
typedef logic [MONBUS_TS_WIDTH-1:0] monbus_timestamp_t;
```

The widths are localparams in `monitor_common_pkg`, **not** per-module parameters — there is exactly one packet format and one timestamp format across the codebase. Any consumer that declares a bus or FIFO holding a packet must reference these constants, not hard-code 128 / 64.

4.3 Protocol Enum

The 4-bit protocol field identifies which protocol family produced the event. The enum is defined in `monitor_common_pkg`:

Value	Name	Description
4'h0	PROTOCOL_AXI	AXI / AXI-Lite / AXI5
4'h1	PROTOCOL_AXIS	AXI4-Stream
4'h2	PROTOCOL_APB	Advanced Peripheral Bus (APB4 / APB5)
4'h3	PROTOCOL_ARB	Arbiter-specific events
4'h4	PROTOCOL_CORE	Core / custom subsystem events
4'h5–4'hF	(reserved)	Forward-compat slack — 16 protocols max.

```
typedef enum logic [3:0] {
    PROTOCOL_AXI    = 4'h0,
    PROTOCOL_AXIS   = 4'h1,
    PROTOCOL_APB    = 4'h2,
    PROTOCOL_ARB    = 4'h3,
    PROTOCOL_CORE   = 4'h4
} protocol_type_t;
```

4.4 Packet Type Enum

The 4-bit packet_type field classifies the event independent of protocol. The 16 codes are defined as localparams in monitor_common_pkg:

Value	Name	Description
4'h0	PktTypeError	Protocol violation, SLVERR/DECERR, orphan, range violation
4'h1	PktTypeCompletion	Transaction completed successfully
4'h2	PktTypeThreshold	Threshold crossed (latency, queue depth, etc.)
4'h3	PktTypeTimeout	Timeout detected on command / data / response phase
4'h4	PktTypePerf	Performance metric event
4'h5	PktTypeCredit	Credit status (AXIS)
4'h6	PktTypeChannel	Channel status (AXIS)
4'h7	PktTypeStream	Stream event (AXIS — start/end/abort)
4'h8	PktTypeAddrMatch	Address-match watchpoint (AXI)
4'h9	PktTypeAPB	APB-specific event
4'hA–4'hC	(reserved)	Forward-compat slack
4'hD	PktTypePerfWin	Windowed-performance

Value	Name	Description
		event (AXI perf window close)
4'hE	PktTypePerfHist	Latency-histogram bucket event (AXI)
4'hF	PktTypeDebug	Debug / trace event
localparam logic	[3:0] PktTypeError	= 4'h0;
localparam logic	[3:0] PktTypeCompletion	= 4'h1;
localparam logic	[3:0] PktTypeThreshold	= 4'h2;
localparam logic	[3:0] PktTypeTimeout	= 4'h3;
localparam logic	[3:0] PktTypePerf	= 4'h4;
localparam logic	[3:0] PktTypeCredit	= 4'h5;
localparam logic	[3:0] PktTypeChannel	= 4'h6;
localparam logic	[3:0] PktTypeStream	= 4'h7;
localparam logic	[3:0] PktTypeAddrMatch	= 4'h8;
localparam logic	[3:0] PktTypeAPB	= 4'h9;
localparam logic	[3:0] PktTypePerfWin	= 4'hD;
localparam logic	[3:0] PktTypePerfHist	= 4'hE;
localparam logic	[3:0] PktTypeDebug	= 4'hF;

4.5 event_data Payload Conventions

The 64-bit event_data field is interpreted in the context of (packet_type, event_code). There is no universal layout — each producer defines its own packing. The table below lists the most common shapes:

Producer	packet_type	event_code	event_data layout
axi_monitor_addr_check	Error	AXI_ERR_ADDR_RANGE (8'h0D)	[63:60] = range_index (4b), [59:0] = full matched address
apb_monitor_addr_check	Error	APB_ERR_ADDR_RANGE (8'h0D)	[63:60] = range_index (4b), [59] = is_read, [58:0] = address
axi_monitor_reporter	Error / Timeout / Completion	various	Full 64-bit address, or zero-extended ID / latency / counter
axi_monitor_reporter	Perf	AXI_PERF_*	Zero-extended counter value (completed/error counts, latencies)

Producer	packet_type	event_code	event_data layout
axi_monitor_reporter	Threshold	AXI_THRESH_ACTIVE_COUNT	Zero-extended active-transaction count
arbiter_monitor_common	Error / Perf	ARB_*	Arbiter-specific payload (winning client, weight, etc.)

Notable design rule: **direction is not embedded in event_data for AXI addr_check**. Each AXI monitor instance watches a single direction (AR or AW) set at build time via the IS_READ parameter; consumers recover direction from the emitting monitor's (unit_id, agent_id) tuple. APB addr_check is the exception — one APB monitor sees both reads and writes on the same channel, so it carries an explicit is_read flag.

4.6 Helper Functions

Field accessors and a packet builder are declared in monitor_common_pkg:

```

function automatic logic [3:0]      get_packet_type
(monitor_packet_t pkt); return pkt[127:124]; endfunction
function automatic logic [14:0]     get_reserved
(monitor_packet_t pkt); return pkt[123:109]; endfunction
function automatic protocol_type_t get_protocol_type
(monitor_packet_t pkt); return protocol_type_t'(pkt[108:105]);
endfunction
function automatic logic [7:0]      get_event_code
(monitor_packet_t pkt); return pkt[104:97]; endfunction
function automatic logic [8:0]      get_channel_id
(monitor_packet_t pkt); return pkt[96:88]; endfunction
function automatic logic [15:0]     get_agent_id
(monitor_packet_t pkt); return pkt[87:72]; endfunction
function automatic logic [7:0]      get_unit_id
(monitor_packet_t pkt); return pkt[71:64]; endfunction
function automatic logic [63:0]     get_event_data
(monitor_packet_t pkt); return pkt[63:0]; endfunction

function automatic monitor_packet_t create_monitor_packet(
    logic [3:0]      packet_type,
    protocol_type_t protocol,
    logic [7:0]      event_code,
    logic [8:0]      channel_id,
    logic [7:0]      unit_id,
    logic [15:0]     agent_id,
    logic [63:0]     event_data
);

```

```

    return {packet_type, 15'h0, protocol, event_code,
            channel_id, agent_id, unit_id, event_data};
endfunction

```

create_monitor_packet zeroes the reserved field; do not rely on its bits.

4.6.1 Construction Example

```

// AXI master-side wrapper emits a range-violation error packet
monitor_packet_t pkt = create_monitor_packet(
    .packet_type ( PktTypeError
                  ),
    .protocol    ( PROTOCOL_AXI
                  ),
    .event_code  ( AXI_ERR_ADDR_RANGE
                  ), // 8'h0D
    .channel_id  ( 9'(cmd_id)
                  ),
    .unit_id     ( UNIT_ID
                  ), // build-
time
    .agent_id    ( AGENT_ID
                  ), // build-
time
    .event_data  ( {range_index[3:0], 60'(cmd_addr)}
                  )
);

```

The wrapper drives pkt on monbus_packet and the sampled i_mon_time on monbus_timestamp in the same cycle.

4.7 Side-Band Timestamp

The timestamp lives outside the packet so the packet layout is fixed at 128 bits and create_monitor_packet doesn't need a timestamp argument.

```
typedef logic [MONBUS_TS_WIDTH-1:0] monbus_timestamp_t; // 64 bits

```

Stage	Wire	Notes
Source	i_mon_time	A free-running counter generated in monbus_axil_group and broadcast to every wrapper via the shared mon_time_w net.
Sampling	addr_pkt_timestamp / monbus_timestamp	Each producer (addr_check, reporter, debug) samples i_mon_time on the cycle its packet asserts valid.

Stage	Wire	Notes
Transport	monbus_arbiter	The arbiter carries (packet, timestamp) atomically through a 192-bit skid so consumers never see a mismatched pair.
Drain	monbus_axil_group	Emits a 3-beat record on m_mon_axil for bulk capture: beat 0 = {tag[3:0], source_ts[59:0]} (tag = 4'h0 raw; non-zero tags reserved for a future on-the-wire compression encoder), beat 1 = packet[127:64], beat 2 = packet[63:0]. Single-record reads via s_mon_axil use the same slice order for IRQ-driven consumption.

Consumers (host scripts, parsers, the bin/TBClasses/monbus decoder) treat the timestamp as an opaque ordering key — its semantics (cycle counter, microsecond, PTP time) are deployment-specific. See `monitor_system_whitepaper.md` §3 for timestamp-policy tradeoffs.

4.8 Per-Protocol Event Code Packages

Each event-code enum is 8 bits wide and lives in the protocol-specific sibling package. The pattern is:

```
event_code = enum value from
              monitor_{amba4, amba5, arbiter}_pkg::
{protocol}_{category}_code_t
```

Categories per protocol (each ~16-entry enum):

Package	Protocol	Categories
monitor_amba4_pkg	AXI4	error, timeout, completion, threshold, performance, addr_match, debug
monitor_amba4_pkg	APB4	error, timeout, completion, threshold, performance, debug
monitor_amba4_pkg	AXIS4	error, timeout, completion, credit, channel, stream
monitor_amba5_pkg	AXI5	atomic, trace
monitor_amba5_pkg	APB5	wakeup, parity, user
monitor_amba5_pkg	AXIS5	wakeup, parity
monitor_arbiter_pkg	ARB	error, timeout, completion, threshold, performance, debug
monitor_arbiter_pkg	CORE	error, timeout, completion, threshold, performance, debug

Full enum-value tables are in the sibling docs.

4.9 Backward Compatibility

The aggregating wrapper `monitor_pkg` re-exports every type from the four split packages, so existing code that does:

```
import monitor_pkg::*;
```

continues to compile. New code should import the specific package it needs:

```
// Universal types
import monitor_common_pkg::*;

// Plus the protocol you actually use:
import monitor_amba4_pkg::*;
```

This keeps namespace pollution down and makes the dependency graph explicit at the file level.

4.10 Related Documentation

- **monitor_system_whitepaper.md** — Design-surface view: identity allocation, timestamp policy, drain paths, aggregation topology.
 - **../monitor/axi_monitor_base.md** — Core monitor that emits packets.
 - **../monitor/axi_monitor_reporter.md** — Packet formatting logic (where `create_monitor_packet` is invoked).
 - **../monitor/axi_monitor_addr_check.md** — Address-range checker, canonical example of structured `event_data`.
 - **../monitor/monbus_arbiter.md** — 192-bit atomic packet+timestamp arbitration.
-

4.11 Navigation

- [← Back to Includes Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

5 APB5 Monitor

Module: `apb5_monitor.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

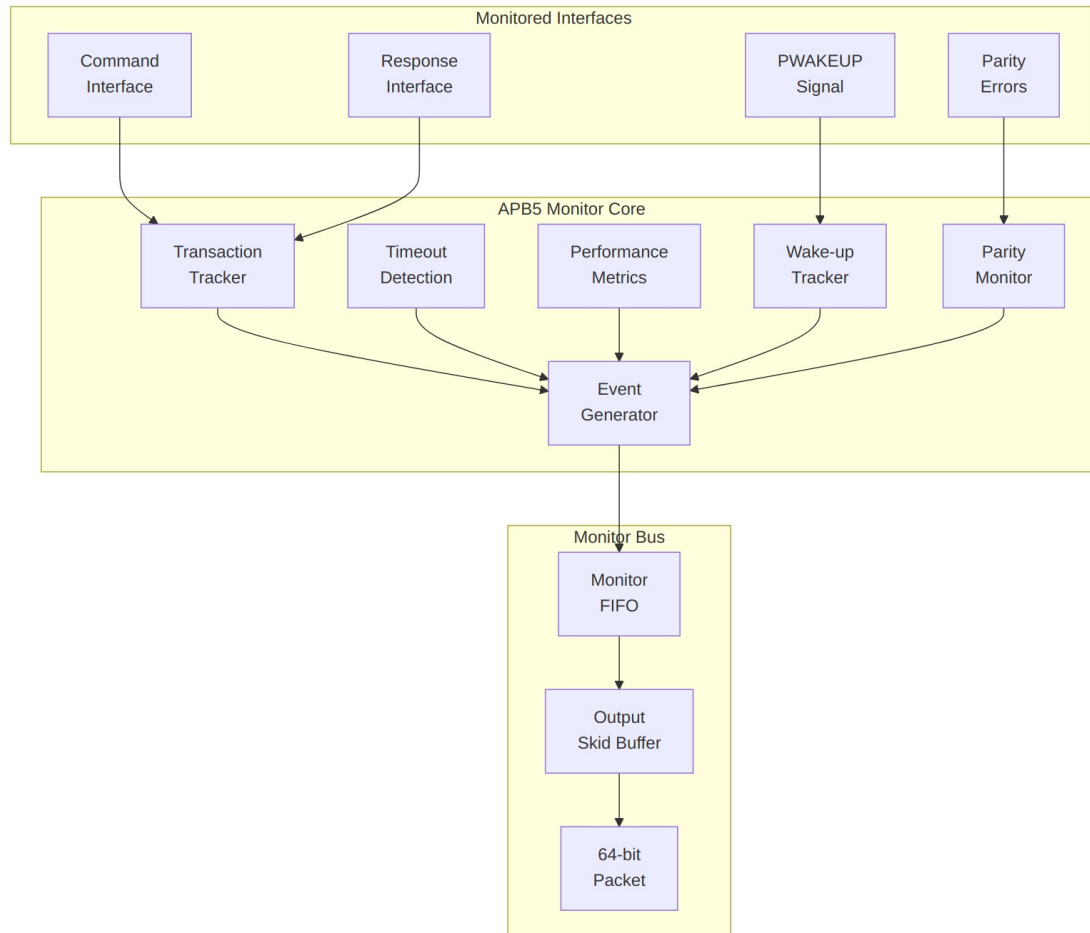
5.1 Overview

The APB5 Monitor provides comprehensive protocol monitoring for APB5 interfaces with support for all APB5 extensions. It tracks transactions, detects errors, monitors performance, and reports events through a standardized monitor bus interface.

5.1.1 Key Features

- Full APB5 protocol monitoring
- Wake-up event tracking (APB5 PWAKEUP)
- User signal tracking (PAUSER, PWUSER, PRUSER, PBUSER)
- Parity error detection (when enabled)
- Transaction timeout detection
- Performance latency measurement
- Protocol violation detection

- 64-bit monitor bus packet output (legacy format — no side-band timestamp)



Module Architecture

5.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
AUSER_WIDTH	int	4	Address user signal width
WUSER_WIDTH	int	4	Write user signal width

Parameter	Type	Default	Description
RUSER_WIDTH	int	4	Read user signal width
BUSER_WIDTH	int	4	Response user signal width
UNIT_ID	int	1	4-bit unit identifier
AGENT_ID	int	10	8-bit agent identifier
MAX_TRANSACTION_S	int	4	Maximum concurrent transactions
MONITOR_FIFO_DEPTH	int	8	Monitor packet FIFO depth
ENABLE_PARITY_MONITOR	bit	0	Enable parity monitoring
N_ADDR_RANGES	int	0	Number of apb_monitor_addr_check comparators. 0 = address-range checker not synthesized
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.

5.3 Ports

5.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	Monitor clock
aresetn	1	Input	Monitor reset (active low)

5.3.2 Command Interface Monitoring

Port	Width	Direction	Description
cmd_valid	1	Input	Command valid signal
cmd_ready	1	Input	Command ready signal
cmd_pwrite	1	Input	Write/read indicator
cmd_paddr	ADDR_WIDTH	Input	Command address
cmd_pwdata	DATA_WIDTH	Input	Command write data
cmd_pstrb	STRB_WIDTH	Input	Write byte strobes
cmd_pprot	3	Input	Protection attributes
cmd_pauser	AUSER_WIDTH	Input	Address user signal
cmd_pwuser	WUSER_WIDT H	Input	Write user signal

5.3.3 Response Interface Monitoring

Port	Width	Direction	Description
rsp_valid	1	Input	Response valid signal
rsp_ready	1	Input	Response ready signal
rsp_prdata	DATA_WIDTH	Input	Response read data
rsp_pslverr	1	Input	Slave error response
rsp_pruser	RUSER_WIDTH	Input	Read user signal
rsp_pbuser	BUSER_WIDTH	Input	Response user

Port	Width	Direction	Description
			signal

5.3.4 APB5 Extension Monitoring

Port	Width	Direction	Description
apb5_pwakeup	1	Input	APB5 wake-up signal
parity_error_wdata	1	Input	Write data parity error
parity_error_rdata	1	Input	Read data parity error
parity_error_ctl	1	Input	Control parity error

5.3.5 Configuration Inputs

Port	Width	Direction	Description
cfg_error_enable	1	Input	Enable error reporting
cfg_timeout_enable	1	Input	Enable timeout detection
cfg_protocol_enable	1	Input	Enable protocol checking
cfg_slverr_enable	1	Input	Enable SLVERR reporting
cfg_parity_enable	1	Input	Enable parity error reporting
cfg_wakeup_enable	1	Input	Enable wake-up event reporting
cfg_user_enable	1	Input	Enable user signal reporting
cfg_perf_enable	1	Input	Enable performance

Port	Width	Direction	Description
			reporting
cfg_latency_en able	1	Input	Enable latency threshold
cfg_cmd_timeo ut_cnt	16	Input	Command timeout threshold
cfg_rsp_timeou t_cnt	16	Input	Response timeout threshold
cfg_latency_thr eshold	32	Input	Latency threshold value
cfg_wakeup_ti meout_cnt	16	Input	Wake-up timeout threshold

5.3.6 Address-Range Checker Configuration

Active only when `N_ADDR_RANGES > 0`; otherwise these inputs are ignored and the `apb_monitor_addr_check` block is not synthesized. Range-violation packets are merged onto the monitor bus at lower priority than the event FIFO.

Port	Width	Direction	Description
cfg_addr_c heck_enabl e	1	Input	Master enable for the address-range checker
cfg_addr_r ange_enabl e	<code>N_ADDR_RA</code> <code>NGES</code>	Input	Per-range enable bit vector
cfg_addr_r ange_low	<code>N_ADDR_RA</code> <code>NGES</code> × <code>ADDR_WIDT</code> <code>H</code>	Input	Per-range low (inclusive) bounds
cfg_addr_r ange_high	<code>N_ADDR_RA</code> <code>NGES</code> × <code>ADDR_WIDT</code>	Input	Per-range high (inclusive) bounds

Port	Width	Direction	Description
	H		

5.3.7 Monitor Bus Output

The APB5 monitor emits the **64-bit legacy monitor-bus format**. There is **no** `monbus_timestamp` output and **no** `i_mon_time` input — timing lives in the packet’s own event-data field, populated from the internal free-running `r_timestamp` counter.

Port	Width	Direction	Description
<code>monbus_valid</code>	1	Output	Monitor packet valid
<code>monbus_ready</code>	1	Input	Monitor bus ready
<code>monbus_packet</code>	64	Output	64-bit monitor packet (see format below)

5.3.8 Status Outputs

Port	Width	Direction	Description
<code>active_count</code>	8	Output	Active transaction count
<code>error_count</code>	16	Output	Total error count
<code>transaction_count</code>	32	Output	Total transaction count
<code>wakeup_active</code>	1	Output	Wake-up currently active

5.4 Monitor Packet Format

5.4.1 64-bit Packet Structure

The APB5 monitor drives a single 64-bit `monbus_packet` (no side-band timestamp). Fields are packed as follows (see `w_fifo_pkt_data` in the RTL):

Bits [63:60] - Packet Type (4 bits, see table below)
 Bits [59:57] - Protocol (3 bits): fixed to `PROTOCOL_APB` (0x2)
 Bits [56:53] - Event Code (4 bits, APB-specific)
 Bits [52:47] - Channel ID (6 bits, always 0 for APB)
 Bits [46:43] - Unit ID (4 bits, from `UNIT_ID[3:0]`)
 Bits [42:35] - Agent ID (8 bits, from `AGENT_ID[7:0]`)
 Bits [34:3] - Event Data (32 bits – address, latency, wakeup timer, etc.)
 Bits [2:0] - Aux Data (3 bits – packet-specific side info)

5.4.2 Packet Types

Packet-type codes follow `monitor_common_pkg`. The APB5 monitor emits the subset marked below; other codes are defined for cross-protocol consistency.

Value	Type	Emitted by APB5 monitor	Description
0x0	Error	✓	SLVERR, protocol violations, parity errors
0x1	Completion	✓	Transaction completed without error
0x2	Threshold	—	Threshold-crossed events (not generated here)
0x3	Timeout	✓	Command / response / wake-up timeout
0x4	Performance	✓	Latency-threshold-exceeded metric
0x8	AddrMatch	—	Address-range violation (from <code>apb_monitor_addr_check</code> when <code>N_ADDR_RANGES > 0</code>)
0x9	APB	✓	APB-specific events (wake-up request/acknowledge)

Value	Type	Emitted by APB5 monitor	Description
0xF	Debug	—	Debug/trace events (not generated here)

Note: parity error events are emitted as **Error** packets (type 0x0) with an APB5 parity event code — not as a distinct packet type.

5.4.3 APB5-Specific Event Codes

Event codes live in per-category enums, so a given numeric code is disambiguated by the packet type it rides on. Wake-up codes ride the **APB** packet type (0x9); parity codes ride the **Error** packet type (0x0).

Event	Code	Packet Type	Description
APB5_WAKE UP_REQUEST	0x0	APB (0x9)	PWAKEUP rising edge
APB5_WAKE UP_ACKNOWLEDGED	0x1	APB (0x9)	PWAKEUP falling edge
APB5_PARITY Y_PWDATA_ERROR	0x0	Error (0x0)	Write data parity error
APB5_PARITY Y_PRDATA_ERROR	0x1	Error (0x0)	Read data parity error
APB5_PARITY Y_PREADY_ERROR	0x2	Error (0x0)	PREADY parity error

```
stateDiagram-v2
    [*] --> IDLE

    IDLE --> CMD_SENT : cmd_valid & cmd_ready
    CMD_SENT --> COMPLETE : rsp_valid & rsp_ready
    COMPLETE --> IDLE : (next cycle)

    state IDLE {
        note right of IDLE : No active transaction
```

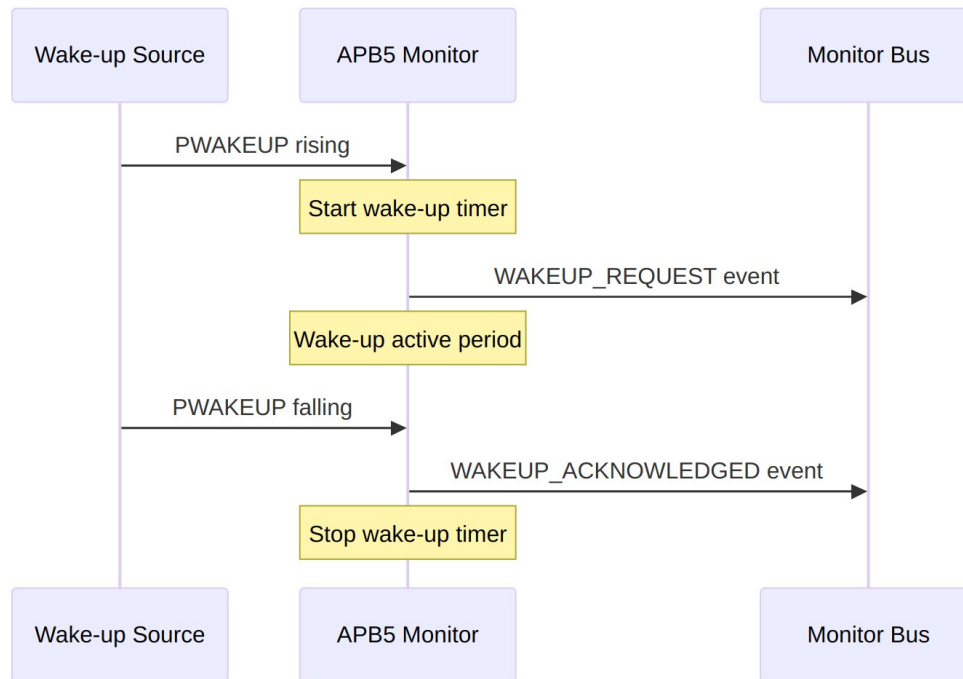
```
}  
state CMD_SENT {  
    note right of CMD_SENT : Waiting for response  
}  
state COMPLETE {  
    note right of COMPLETE : Transaction finished  
}
```

5.5 Event Priority

Events are generated with the following priority (highest first):

1. **Error Events** - Protocol violations, SLVERR
 2. **Parity Events** - Parity errors detected
 3. **Timeout Events** - Command/response timeouts
 4. **Wake-up Events** - PWAKEUP transitions
 5. **Performance Events** - Latency threshold exceeded
 6. **Completion Events** - Normal transaction completion
-

5.6 Wake-up Monitoring



Wake-up Detection

5.6.1 Wake-up Timeout

If PWAKEUP remains high longer than `cfg_wakeup_timeout_cnt`, a timeout event is generated.

5.7 Usage Example

```
apb5_monitor #(
    .ADDR_WIDTH      (32),
    .DATA_WIDTH      (32),
    .AUSER_WIDTH      (4),
    .WUSER_WIDTH      (4),
    .RUSER_WIDTH      (4),
    .BUSER_WIDTH      (4),
    .UNIT_ID          (1),
    .AGENT_ID          (10),
    .MAX_TRANSACTIONS (4),
    .MONITOR_FIFO_DEPTH (8),
    .ENABLE_PARITY_MON (0)
) u_apb5_monitor (
    .aclk              (apb_clk),
    .aresetn            (apb_rst_n),
```

```

// Command interface monitoring
.cmd_valid      (apb_cmd_valid),
.cmd_ready      (apb_cmd_ready),
.cmd_pwrite     (apb_cmd_pwrite),
.cmd_paddr      (apb_cmd_paddr),
.cmd_pwdata     (apb_cmd_pwdata),
.cmd_pstrb      (apb_cmd_pstrb),
.cmd_pprot      (apb_cmd_pprot),
.cmd_pauser     (apb_cmd_pauser),
.cmd_pwuser     (apb_cmd_pwuser),

// Response interface monitoring
.rsp_valid      (apb_rsp_valid),
.rsp_ready      (apb_rsp_ready),
.rsp_prdata     (apb_rsp_prdata),
.rsp_pslverr    (apb_rsp_pslverr),
.rsp_pruser     (apb_rsp_pruser),
.rsp_pbuser     (apb_rsp_pbuser),

// APB5 wake-up monitoring
.apb5_pwakeup   (apb_pwakeup),

// Parity error inputs
.parity_error_wdata (1'b0),
.parity_error_rdata (1'b0),
.parity_error_ctrl  (1'b0),

// Configuration
.cfg_error_enable   (1'b1),
.cfg_timeout_enable (1'b1),
.cfg_protocol_enable (1'b1),
.cfg_slverr_enable  (1'b1),
.cfg_parity_enable  (1'b0),
.cfg_wakeup_enable  (1'b1),
.cfg_user_enable    (1'b0),
.cfg_perf_enable    (1'b0),
.cfg_latency_enable (1'b0),
.cfg_cmd_timeout_cnt(16'd1000),
.cfg_rsp_timeout_cnt(16'd1000),
.cfg_latency_threshold(32'd100),
.cfg_wakeup_timeout_cnt(16'd500),

// Monitor bus output
.monbus_valid      (apb_mon_valid),
.monbus_ready      (apb_mon_ready),
.monbus_packet     (apb_mon_packet),

```

```

// Status
.active_count      (apb_active_count),
.error_count       (apb_error_count),
.transaction_count (apb_trans_count),
.wakeup_active     (apb_wakeup_active)
);

```

5.8 Configuration Guidelines

5.8.1 Recommended Configurations

Functional Debug:

```

.cfg_error_enable    (1'b1),
.cfg_timeout_enable  (1'b1),
.cfg_protocol_enable (1'b1),
.cfg_slvrr_enable    (1'b1),
.cfg_wakeup_enable   (1'b1),
.cfg_perf_enable     (1'b0) // Disable for lower traffic

```

Performance Analysis:

```

.cfg_error_enable    (1'b1),
.cfg_perf_enable     (1'b1),
.cfg_latency_enable  (1'b1),
.cfg_wakeup_enable   (1'b0) // Disable non-essential

```

5.9 Design Notes

5.9.1 Transaction Table

- Tracks up to MAX_TRANSACTIONS concurrent transactions
- APB typically has 1-4 outstanding (simple protocol)
- Table entries cleaned up after event reported

5.9.2 Internal FIFOs

- Monitor FIFO depth configurable via MONITOR_FIFO_DEPTH
 - Output skid buffer ensures no backpressure stalls
-

5.10 Related Documentation

- [APB5 Master](#) - APB5 master interface
 - [APB5 Slave](#) - APB5 slave interface
 - [Monitor Packet Format](#) - Standard packet format
-

5.11 Navigation

- [<- Back to APB5 Index](#)
- [<- Back to RTLamba Index](#)
- [<- Back to Main Documentation Index](#)

6 APB Monitor Address-Range Checker

Module: `apb_monitor_addr_check.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

6.1 Overview

The APB Monitor Address-Range Checker is a configurable N-range address-violation filter for the APB monitor pipeline. It is the APB mirror of `axi_monitor_addr_check`: it watches the `cmd_valid/cmd_ready` handshake the `apb_monitor` already snoops, and when an accepted command's `paddr` falls inside any of N configured `[low, high]` inclusive ranges it emits a `PktTypeError MonBus` packet with event code `APB_ERR_ADDR_RANGE (8'h08)`.

6.1.1 Key Features

- Up to N configurable inclusive `[low, high]` address ranges (default 4, up to 16 usable via the 4-bit range index)
- Per-range enable plus a global `cfg_addr_check_enable`
- Emits a standard MonBus error packet with the offending address, range index, and direction
- Preserves an `is_read` bit (APB has no separate AR/AW channels, so direction must be carried explicitly)
- Per-range pending mask latches hits and drains them one packet at a time under backpressure
- Side-band timestamp captured from the broadcast free-running counter on emission
- Exact-match support by setting a range's `low == high`

6.2 Module Purpose

APB peripherals frequently have reserved or protected address windows that software must never touch. This checker gives the APB monitor an inexpensive, fully-configurable way to flag such accesses in-band on the MonBus, without adding logic to the APB datapath itself. It snoops the same command handshake the monitor already observes, compares the accepted address against a set of programmable ranges, and reports a violation packet identifying which range was hit, the offending address, and whether the access was a read or a write.

Use Cases: - Flagging accesses to reserved/protected APB register windows - Security or safety guard-banding of peripheral address maps - Debug tripwires on unexpected address regions during bring-up - Exact-address watchpoints (set low == high)

Key Benefit: Programmable, per-range address-violation reporting on the MonBus that mirrors the AXI variant while carrying the APB-specific read/write direction bit.

6.3 Parameters

Parameter	Type	Default	Description
N_ADDR_RANGES	int	4	Number of configurable [low, high] ranges
ADDR_WIDTH	int	32	APB address width
UNIT_ID	logic [7:0]	8'h00	Unit id stamped into emitted packets
AGENT_ID	logic [15:0]	16'h0000	Agent id stamped into emitted packets
M	int	ADDR_WIDTH	Internal address-width alias

6.4 Port Groups

6.4.1 Clock and Reset

Port	Direction	Width	Description
clk	input	1	Clock
aresetn	input	1	Active-low

Port	Direction	Width	Description
			asynchronous reset

6.4.2 Timestamp

Port	Direction	Width	Description
i_mon_time	input	monbus_timestamp_t	Free-running counter broadcast by the monbus_group family; sampled on emission

6.4.3 Snooped APB Command Stream

Port	Direction	Width	Description
cmd_paddr	input	M	Command address (APB paddr)
cmd_pwrite	input	1	Command direction (1 = write, 0 = read)
cmd_valid	input	1	Command valid (from the monitor's snoop of the APB handshake)
cmd_ready	input	1	Command ready; valid && ready && enable marks an accepted command

6.4.4 Range Configuration

Port	Direction	Width	Description
cfg_addr_checker_enable	input	1	Global enable for the checker (also gates output valid)
cfg_addr_range_enable	input	N_ADDR_RANGES	Per-range enable mask
cfg_addr_range_low	input	N_ADDR_RANGES × M	Per-range inclusive lower bound
cfg_addr_range_high	input	N_ADDR_RANGES	Per-range inclusive upper bound

Port	Direction	Width	Description
ange_high		NGES × M	bound

6.4.5 Outgoing MonBus Packet

Port	Direction	Width	Description
addr_pkt_valid	output	1	Packet valid (a pending violation is ready to emit)
addr_pkt_ready	input	1	Downstream ready; valid && ready accepts the packet
addr_pkt_data	output	monitor_packet_t	Assembled MonBus error packet
addr_pkt_timestamp	output	monbus_timestamp_t	Timestamp side-band (i_mon_time)

6.5 Functional Description

6.5.1 Combinational Range Hits

A command “fires” when `cmd_valid && cmd_ready && cfg_addr_check_enable`. For each range `i`, a one-hot hit is asserted when the range is enabled, the command fires, and `cmd_paddr` lies within `[cfg_addr_range_low[i], cfg_addr_range_high[i]]` inclusive. Setting a range’s low and high equal yields exact-match (watchpoint) semantics.

6.5.2 Pending Mask and Latched Snapshot

Each range has a pending bit. On a hit, the range’s pending bit sets and its address and direction are latched (`r_lat_addr[i] = cmd_paddr`, `r_lat_is_read[i] = !cmd_pwrite`). This decouples detection from emission so a violation is not lost while the MonBus is backpressured. A first-match priority encoder over `r_pending` selects the range to emit (`emit_oh / emit_idx`); `addr_pkt_valid` asserts when any range is pending and the checker is enabled. When the packet is accepted (`addr_pkt_valid && addr_pkt_ready`), the emitted range’s pending bit clears. If a fresh hit and an accept collide on the same range in one cycle, the hit wins (the pending bit stays set), so a back-to-back violation is not dropped.

6.5.3 Packet Encoding

The emitted packet is the standard 128-bit MonBus format with a 64-bit `event_data` field, assembled via `create_monitor_packet`:

- **packet_type** = `PktTypeError (4'h0)`

- **protocol** = PROTOCOL_APB (4'h2)
- **event_code** = APB_ERR_ADDR_RANGE (8'h08)
- **channel_id** = 0 (APB has no ID concept)
- **unit_id** = UNIT_ID, **agent_id** = AGENT_ID
- **event_data[63:60]** = range index (4 bits → up to 16 ranges)
- **event_data[59]** = is_read (1 = read, 0 = write)
- **event_data[58:0]** = offending cmd_paddr (zero-padded if M < 59, or its low 59 bits if M >= 59)

6.5.4 The is_read Carve-Out

The AXI variant drops the direction bit because AR and AW are separate channels — direction is implied by which monitor (read vs write) emitted the packet. APB has no such split: the same monitor sees both directions, so this block preserves an explicit `is_read` bit (carved out of the 60-bit address slot) so consumers can disambiguate read from write.

6.5.5 Timestamp Side-Band

Exactly as in the AXI variant, `i_mon_time` (the free-running counter broadcast by the `monbus_group` family) is driven straight out on `addr_pkt_timestamp`, so the consuming group can stamp the violation with a bus-consistent time.

6.6 Usage Example

```
// Guard two reserved APB windows; report violations on the MonBus.
apb_monitor_addr_check #(
    .N_ADDR_RANGES    (4),
    .ADDR_WIDTH        (32),
    .UNIT_ID           (8'h10),
    .AGENT_ID          (16'h0001)
) u_apb_addr_check (
    .clk                (pclk),
    .aresetn            (presetn),
    .i_mon_time         (mon_time),           // from the monbus
group

    // Snooped APB command handshake (from apb_monitor)
    .cmd_paddr          (apb_paddr),
    .cmd_pwrite         (apb_pwrite),
    .cmd_valid          (cmd_valid),
    .cmd_ready          (cmd_ready),

    // Range config (range 0 = a window, range 1 = an exact
```

```

watchpoint)
    .cfg_addr_check_enable    (1'b1),
    .cfg_addr_range_enable   (4'b0011),
    .cfg_addr_range_low      ('{32'h0000_1000, 32'h0000_2000, 32'h0,
32'h0}),
    .cfg_addr_range_high     ('{32'h0000_1FFF, 32'h0000_2000, 32'h0,
32'h0}),

    // MonBus output (connect to the group's error-drain path)
    .addr_pkt_valid          (addr_pkt_valid),
    .addr_pkt_ready          (addr_pkt_ready),
    .addr_pkt_data           (addr_pkt_data),
    .addr_pkt_timestamp      (addr_pkt_timestamp)
);

```

6.7 Design Notes

6.7.1 Mirror of the AXI Variant

The block is a deliberate mirror of `axi_monitor_addr_check` so the two share timestamp handling, the pending-mask/priority-encoder emission structure, and the range-index/address packet layout. The only intentional divergence is the preserved `is_read` bit.

6.7.2 Backpressure Safety

Detection latches into `r_pending` and emission drains one range per accepted packet, so violations survive downstream backpressure. The hit-versus-accept collision rule keeps the pending bit set when a new hit lands on a range being drained the same cycle.

6.7.3 Address Field Width

For `ADDR_WIDTH >= 59` the low 59 bits of the address are carried; for narrower buses the address is zero-extended into the 59-bit payload slot. The upper four `event_data` bits are always the range index.

6.7.4 Exact-Match Watchpoints

Program `cfg_addr_range_low[i] == cfg_addr_range_high[i]` to turn a range into a single-address watchpoint.

6.8 Related Modules

6.8.1 Used By

- `apb_monitor` pipeline (address-range violation reporting)

6.8.2 Uses

- **`monitor_common_pkg`** - `PktTypeError`, `PROTOCOL_APB`, `monbus_timestamp_t`, `create_monitor_packet`
- **`monitor_amba4_pkg`** - `APB_ERR_ADDR_RANGE` event code
- **`reset_defs.svh`** - `ALWAYS_FF_RST` / `RST_ASSERTED` reset macros

6.8.3 See Also

- **`axi_monitor_addr_check.sv`** - AXI-side equivalent (no `is_read` bit)
 - **`apb_monitor.sv`** - The APB monitor this checker plugs into
-

6.9 References

6.9.1 Source Code

- RTL: `rtl/amba/monitor/apb_monitor_addr_check.sv`

6.9.2 Documentation

- Packet format:
`docs/markdown/RTLAmba/includes/monitor_package_spec.md`
 - Architecture: `docs/markdown/RTLAmba/shared/README.md`
 - Design Guide: `rtl/amba/PRD.md`
-

Last Updated: 2026-07-15

6.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

7 apb_monitor

An Advanced Peripheral Bus (APB) transaction monitor that provides comprehensive error detection, performance tracking, and debug capabilities through a standardized monitor bus interface.

7.1 Overview

The `apb_monitor` module monitors APB command/response interfaces to detect protocol violations, track performance metrics, and report events via the 128-bit monitor bus protocol (paired with a 64-bit side-band timestamp). It attaches to APB master `cmd/rsp` interfaces (timing-convenient proxies for APB signals) and generates standardized event packets for error detection, latency analysis, and transaction debugging.

7.2 Module Declaration

```
module apb_monitor
  import monitor_pkg::*;
#(
  parameter int ADDR_WIDTH           = 32,
  parameter int DATA_WIDTH          = 32,
  parameter logic [7:0] UNIT_ID      = 8'h01,      // 8-bit Unit ID
  parameter logic [15:0] AGENT_ID    = 16'h000A,    // 16-bit Agent ID
  parameter int MAX_TRANSACTIONS     = 4,          // APB is typically
single outstanding
  parameter int MONITOR_FIFO_DEPTH   = 8,          // Monitor packet FIFO
depth

  // Short params
  parameter int AW                    = ADDR_WIDTH,
  parameter int DW                    = DATA_WIDTH,
  parameter int SW                    = DW/8
)
(
  // Clock and Reset (aclk domain - matches cmd/rsp interfaces)
  input logic aclk,
  input logic aresetn,

  // Command Interface Monitoring (aclk domain)
  input logic cmd_valid,
  input logic cmd_ready,
  input logic cmd_pwrite,
  input logic [AW-1:0] cmd_paddr,
  input logic [DW-1:0] cmd_pwdata,
  input logic [SW-1:0] cmd_pstrb,
  input logic [2:0] cmd_pprot,
```

```

// Response Interface Monitoring (aclk domain)
input logic      rsp_valid,
input logic      rsp_ready,
input logic [DW-1:0]  rsp_prdata,
input logic      rsp_pslverr,

// Configuration - Error Detection
input logic      cfg_error_enable,          //
Enable error event packets
input logic      cfg_timeout_enable,        //
Enable timeout event packets
input logic      cfg_protocol_enable,       //
Enable protocol violation detection
input logic      cfg_slverr_enable,         //
Enable slave error detection

// Configuration - Performance Monitoring
input logic      cfg_perf_enable,           //
Enable performance packets
input logic      cfg_latency_enable,        //
Enable latency tracking
input logic      cfg_throughput_enable,     //
Enable throughput tracking

// Configuration - Debug
input logic      cfg_debug_enable,          //
Enable debug packets
input logic      cfg_trans_debug_enable,    //
Enable transaction debug
input logic [3:0]  cfg_debug_level,         // Debug
verbosity level

// Configuration - Thresholds and Timeouts
input logic [15:0]  cfg_cmd_timeout_cnt,    //
Command timeout (cycles)
input logic [15:0]  cfg_rsp_timeout_cnt,    //
Response timeout (cycles)
input logic [31:0]  cfg_latency_threshold,  //
Latency threshold (cycles)
input logic [15:0]  cfg_throughput_threshold, //
Throughput threshold

// Consolidated 128-bit event packet interface (monitor bus) + 64-
bit side-band timestamp
output logic      monbus_valid,             //
Monitor bus valid

```

```

    input logic                                monbus_ready,          //
    Monitor bus ready
    output logic [127:0]                       monbus_packet,        //
    Consolidated monitor packet (monitor_packet_t)
    output logic [63:0]                       monbus_timestamp,      // Side-
    band timestamp (monbus_timestamp_t)
    input logic [63:0]                         i_mon_time,           // Free-
    running counter from monbus_axil_group

    // Status outputs
    output logic [7:0]                         active_count,         //
    Number of active transactions
    output logic [15:0]                       error_count,          // Total
    error count
    output logic [31:0]                       transaction_count      // Total
    transaction count
);

```

7.3 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
UNIT_ID	logic [7:0]	8'h01	8-bit Unit identifier for monitor packets
AGENT_ID	logic [15:0]	16'h000A	16-bit Agent identifier for monitor packets
MAX_TRANSACTION S	int	4	Maximum concurrent transactions (APB typically 1-4)
MONITOR_FIFO_DEP TH	int	8	Internal FIFO depth for monitor packets
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.

7.4 Ports

7.4.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	Monitor clock (matches cmd/rsp domain)
aresetn	1	Input	Active-low asynchronous reset

7.4.2 Command Interface Monitoring

Port	Width	Direction	Description
cmd_valid	1	Input	Command valid signal
cmd_ready	1	Input	Command ready signal
cmd_pwrite	1	Input	Write/read indicator (1=write, 0=read)
cmd_paddr	AW	Input	Command address
cmd_pwdata	DW	Input	Write data
cmd_pstrb	SW	Input	Write strobe
cmd_pprot	3	Input	Protection attributes

7.4.3 Response Interface Monitoring

Port	Width	Direction	Description
rsp_valid	1	Input	Response valid signal
rsp_ready	1	Input	Response ready signal

Port	Width	Direction	Description
rsp_prdata	DW	Input	Read data
rsp_pslverr	1	Input	Slave error indicator

7.4.4 Configuration - Error Detection

Port	Width	Direction	Description
cfg_error_enable	1	Input	Enable error event packet generation
cfg_timeout_enable	1	Input	Enable timeout detection
cfg_protocol_enable	1	Input	Enable protocol violation detection
cfg_slverr_enable	1	Input	Enable slave error reporting

7.4.5 Configuration - Performance Monitoring

Port	Width	Direction	Description
cfg_perf_enable	1	Input	Enable performance packet generation
cfg_latency_enable	1	Input	Enable latency measurement
cfg_throughput_enable	1	Input	Enable throughput tracking

7.4.6 Configuration - Debug

Port	Width	Direction	Description
cfg_debug_enable	1	Input	Enable debug packet generation
cfg_transaction_level_debug_enable	1	Input	Enable transaction-level debugging

Port	Width	Direction	Description
cfg_debug_level	4	Input	Debug verbosity level (0-15)

7.4.7 Configuration - Thresholds

Port	Width	Direction	Description
cfg_cmd_timeout_cnt	16	Input	Command timeout threshold (clock cycles)
cfg_rsp_timeout_cnt	16	Input	Response timeout threshold (clock cycles)
cfg_latency_threshold	32	Input	Latency threshold for alerts (clock cycles)
cfg_throughput_threshold	16	Input	Throughput threshold for alerts

7.4.8 Monitor Bus Interface

Port	Width	Direction	Description
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor packet ready (backpressure)
monbus_packet	128	Output	Consolidated monitor_packet_t (see format below)
monbus_timestamp	64	Output	monbus_timestamp_t paired atomically with monbus_packet
i_mon_time	64	Input	Free-running counter from monbus_axil_group, sampled at packet emission

7.4.9 Status Outputs

Port	Width	Direction	Description
active_count	8	Output	Number of

Port	Width	Direction	Description
			currently active transactions
error_count	16	Output	Cumulative error count
transaction_count	32	Output	Total transaction count

7.5 Functional Description

7.5.1 Transaction Monitoring

The APB monitor tracks transactions through the command/response pipeline:

1. **Command Phase:** Monitors cmd_valid/cmd_ready handshake
 - Captures address, write/read type, data, strobe
 - Starts latency counter
 - Detects protocol violations
2. **Response Phase:** Monitors rsp_valid/rsp_ready handshake
 - Captures response data and error status
 - Calculates transaction latency
 - Matches response to command

7.5.2 Event Detection

The monitor generates standardized 128-bit packets (paired with 64-bit side-band timestamps) for:

Error Events (when cfg_error_enable = 1): - SLVERR responses (when cfg_slverr_enable = 1) - Protocol violations (when cfg_protocol_enable = 1) - Timeout conditions (when cfg_timeout_enable = 1)

Performance Events (when cfg_perf_enable = 1): - Latency threshold violations - Throughput degradation - Transaction statistics

Debug Events (when cfg_debug_enable = 1): - Transaction start/completion - State transitions - Internal status changes

7.5.3 Monitor Packet Format

The 128-bit monbus_packet (paired with the 64-bit monbus_timestamp side-band signal) follows the standardized APB monitor bus format. The layout is identical across protocols:

Bits [127:124] - Packet Type:
0x0 = ERROR Error events (SLVERR, protocol violations)
0x1 = COMPL Completion events (transaction finished)
0x2 = THRESH Threshold events
0x3 = TIMEOUT Timeout events
0x4 = PERF Performance metrics
0x8 = ADDR_MATCH Address match events
0x9 = APB APB-specific events
0xF = DEBUG Debug events
Bits [123:109] - Reserved (15 bits, forward-compat slack)
Bits [108:105] - Protocol (4 bits): 0x0=AXI, 0x1=AXIS, 0x2=APB, 0x3=ARB, 0x4=CORE
Bits [104:97] - Event Code (8 bits, protocol-specific)
Bits [96:88] - Channel ID (9 bits)
Bits [87:72] - Agent ID (16 bits, from AGENT_ID parameter)
Bits [71:64] - Unit ID (8 bits, from UNIT_ID parameter)
Bits [63:0] - Event Data (64 bits – full address, latency, etc.)

7.6 Configuration Strategies

7.6.1 Functional Verification (Recommended)

```
.cfg_error_enable(1'b1),           // Catch all errors
.cfg_timeout_enable(1'b1),         // Detect hangs
.cfg_protocol_enable(1'b1),        // Catch violations
.cfg_slvrr_enable(1'b1),           // Report slave errors
.cfg_perf_enable(1'b0),            // Disable (reduces packet traffic)
.cfg_debug_enable(1'b0),           // Only if deep debugging needed
.cfg_cmd_timeout_cnt(16'd1000),    // 1000 cycle timeout
.cfg_rsp_timeout_cnt(16'd1000)
```

7.6.2 Performance Analysis

```
.cfg_error_enable(1'b1),           // Still catch errors
.cfg_timeout_enable(1'b0),         // Disable
.cfg_protocol_enable(1'b0),        // Disable
.cfg_slvrr_enable(1'b1),           // Keep error reporting
.cfg_perf_enable(1'b1),            // Enable performance tracking
.cfg_latency_enable(1'b1),         // Track latencies
.cfg_throughput_enable(1'b1),      // Track throughput
.cfg_latency_threshold(32'd100)    // Alert on >100 cycle latency
```

7.6.3 Debug Mode

```
.cfg_error_enable(1'b1),  
.cfg_debug_enable(1'b1),           // Enable debug packets  
.cfg_trans_debug_enable(1'b1),     // Transaction-level debug  
.cfg_debug_level(4'd2),            // Medium verbosity  
.cfg_perf_enable(1'b0)              // Reduce traffic
```

7.7 Usage Example

```
// APB Master with integrated monitor  
apb_master #(  
    .ADDR_WIDTH(32),  
    .DATA_WIDTH(32)  
) u_apb_master (  
    .pclk(pclk),  
    .presetn(presetn),  
    // APB master interface  
    .m_apb_PSEL(apb_psel),  
    .m_apb_PENABLE(apb_penable),  
    .m_apb_PADDR(apb_paddr),  
    .m_apb_PWRITE(apb_pwrite),  
    .m_apb_PWDATA(apb_pwdata),  
    .m_apb_PREADY(apb_pready),  
    .m_apb_PRDATA(apb_prdata),  
    .m_apb_PSLVERR(apb_pslverr),  
    // Command/Response interfaces  
    .cmd_valid(cmd_valid),  
    .cmd_ready(cmd_ready),  
    .cmd_pwrite(cmd_pwrite),  
    .cmd_paddr(cmd_paddr),  
    .cmd_pwdata(cmd_pwdata),  
    .rsp_valid(rsp_valid),  
    .rsp_ready(rsp_ready),  
    .rsp_prdata(rsp_prdata),  
    .rsp_pslverr(rsp_pslverr)  
);  
  
// APB Monitor attached to cmd/rsp interfaces  
apb_monitor #(  
    .ADDR_WIDTH(32),  
    .DATA_WIDTH(32),  
    .UNIT_ID(1),  
    .AGENT_ID(10),  
    .MAX_TRANSACTIONS(4)  
) u_apb_monitor (  
    .acclk(pclk),  
    .aresetn(presetn),  
    // Monitor cmd/rsp interfaces
```

```

.cmd_valid(cmd_valid),
.cmd_ready(cmd_ready),
.cmd_pwrite(cmd_pwrite),
.cmd_paddr(cmd_paddr),
.cmd_pwdata(cmd_pwdata),
.cmd_pstrb(cmd_pstrb),
.cmd_pprot(cmd_pprot),
.rsp_valid(rsp_valid),
.rsp_ready(rsp_ready),
.rsp_prdata(rsp_prdata),
.rsp_pslverr(rsp_pslverr),
// Configuration
.cfg_error_enable(1'b1),
.cfg_timeout_enable(1'b1),
.cfg_protocol_enable(1'b1),
.cfg_slverr_enable(1'b1),
.cfg_perf_enable(1'b0),
.cfg_debug_enable(1'b0),
.cfg_cmd_timeout_cnt(16'd1000),
.cfg_rsp_timeout_cnt(16'd1000),
// Monitor bus
.monbus_valid(mon_valid),
.monbus_ready(mon_ready),
.monbus_packet(mon_packet),
// Status
.active_count(mon_active),
.error_count(mon_errors),
.transaction_count(mon_trans_cnt)
);

// Downstream FIFO for monitor packets
gaxi_fifo_sync #(
    .DATA_WIDTH(64),
    .DEPTH(128)
) u_mon_fifo (
    .i_clk(pclk),
    .i_rst_n(presetn),
    .i_valid(mon_valid),
    .i_data(mon_packet),
    .o_ready(mon_ready),
    // Connect to system monitor consumer
    .o_valid(sys_mon_valid),
    .o_data(sys_mon_data),
    .i_ready(sys_mon_ready)
);

```

7.8 Design Notes

7.8.1 Transaction Tracking

- APB is typically single-outstanding, so MAX_TRANSACTIONS=4 is usually sufficient
- Monitor tracks cmd→rsp matching to detect orphaned responses
- Separate timeout counters for command and response phases

7.8.2 Packet Generation

- Internal FIFO buffers monitor packets (depth = MONITOR_FIFO_DEPTH)
- Backpressure on monbus_ready propagates to packet generation
- FIFO full condition prevents packet loss

7.8.3 Performance Considerations

- Disable unused packet types to reduce traffic
- Performance mode (cfg_perf_enable) can generate high packet rates
- Debug mode should only be used for targeted debugging

7.9 Related Modules

- `apb_master.sv` - APB master with cmd/rsp interfaces
- `apb_slave.sv` - APB slave with cmd/rsp interfaces
- `monitor_pkg.sv` - Monitor packet definitions and utilities
- `gaxi_fifo_sync.sv` - Recommended for monitor packet buffering

7.10 References

- **Monitor Packet Specification:** [monitor_package_spec.md](#)
 - **APB Protocol:** AMBA APB Protocol Specification v2.0
 - **Verification Guide:** See testbench in `val/amba/test_apb_monitor.py`
-

7.11 Navigation

- [← Back to APB Index](#) (if exists, otherwise remove this line)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

8 Monitor Bus Arbiter Common

Module: `arbiter_monbus_common.sv` **Location:** `rtl/amba/shared/` **Category:** Core Infrastructure **Status:** ✓ Production Ready

8.1 Overview

The `arbiter_monbus_common` module provides Base infrastructure for monitor bus arbitration.

This is a **shared infrastructure module** used internally by AXI/AXIL monitors. It is not typically instantiated directly by users but is critical for understanding the monitor architecture.

8.2 Key Features

- ✓ **Shared arbitration logic for multiple monitor sources:** Shared arbitration logic for multiple monitor sources
 - ✓ **Fair access to monitor bus bandwidth:** Fair access to monitor bus bandwidth
 - ✓ **Packet multiplexing with source identification:** Packet multiplexing with source identification
 - ✓ **Backpressure handling and flow control:** Backpressure handling and flow control
 - ✓ **Configurable arbitration policy support:** Configurable arbitration policy support
 - ✓ **Grant tracking and fairness enforcement:** Grant tracking and fairness enforcement
-

8.3 Module Purpose

The `arbiter_monbus_common` module is the core building block for:

1. **Fair Arbitration:** Prevents starvation with round-robin or weighted policies
2. **Source Multiplexing:** Combines multiple monitor streams
3. **Backpressure Management:** Handles flow control from downstream
4. **Grant Tracking:** Maintains fairness across arbitration cycles

5. Scalable Design: Supports configurable number of sources

8.4 Parameters

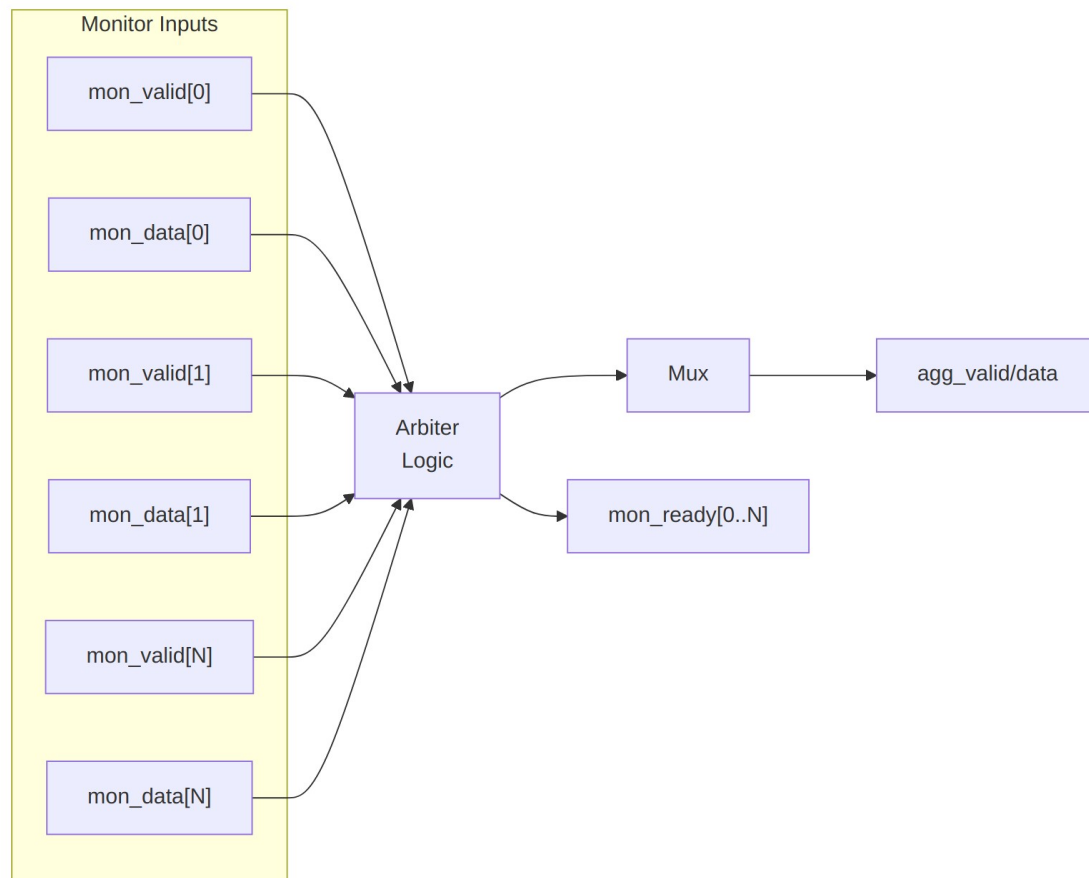
Parameter	Type	Default	Description
CLIENTS	int	4	Number of monitor sources to arbitrate
WAIT_GNT_ACK	int	0	1 = require a grant-ACK handshake
WEIGHTED_MODE	int	0	1 = weighted round-robin, 0 = plain round-robin
MON_AGENT_ID	logic [15:0]	16'h0010	Monitor agent identifier (16-bit)
MON_UNIT_ID	logic [7:0]	8'h00	Monitor unit identifier (8-bit)
MON_FIFO_DEPTH	int	8	Monitor packet FIFO depth
MON_FIFO_ALMOST_MARGIN	int	1	Almost-full margin for the monitor FIFO
FAIRNESS_REPORT_CYCLES	int	256	Window (cycles) for fairness reporting
MIN_GRANTS_FOR_FAIRNESS	int	100	Minimum grants before a fairness report is valid
DEFAULT_ACK_TIMEOUT	int	64	Default grant-ACK timeout (cycles)

$N(\lceil \log_2(\text{CLIENTS}) \rceil)$, `MON_FIFO_COUNT_WIDTH`, and the weight-vector widths are derived localparams, not user-settable parameters.

8.5 Port Groups

See RTL source: `rtl/amba/monitor/arbiter_monbus_common.sv` for complete port listing.

Key interface groups: - Clock and reset - Input signals from monitored interface - Configuration signals - Output signals to downstream logic



Architecture

Arbitration policies supported: - Round-robin (fair) - Weighted round-robin (QoS) - Priority-based (critical first)

8.6 Usage in Monitor System

This module is used by:

- **arbiter_rr_pwm_monbus**
- **arbiter_wrr_pwm_monbus**

8.6.1 Internal Integration

This module is instantiated automatically within higher-level monitor modules. Users configure behavior through top-level monitor parameters.

8.7 Configuration Guidelines

See individual monitor documentation for configuration examples.

Configuration is typically handled at the top-level monitor instantiation.

8.8 Performance Characteristics

Metric	Value	Notes
Latency	1-2 cycles	Typical processing delay
Throughput	1 operation/cycle	Maximum rate
Resource Usage	Varies	Depends on configuration

8.9 Verification Considerations

8.9.1 Test Coverage

- Functional correctness of core logic
- Boundary conditions (min/max values)
- Error handling and recovery
- Interface protocol compliance

See: `val/amba/test_arbiter_monbus_common.py` for verification tests

8.10 Related Modules

- [axi_monitor_reporter](#)
-

8.11 See Also

- **Monitor Architecture:** `docs/markdown/RTLAmba/overview.md`
- **Monitor Configuration Guide:** [Monitor Base Configuration](#)

- **Packet Format Specification:**
docs/markdown/RTLamba/includes/monitor_package_spec.md
-

8.12 Navigation

- [← Back to Shared Infrastructure Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

9 Round-Robin Arbiter with PWM and Monitor Bus

Module: arbiter_rr_pwm_monbus.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

9.1 Overview

The Round-Robin Arbiter with PWM and Monitor Bus combines fair round-robin arbitration with PWM-based flow control and comprehensive silicon debug monitoring. This standardized module uses fixed internal configurations for consistency while exposing only essential user-configurable parameters.

9.1.1 Key Features

- Fair round-robin arbitration across N clients
 - PWM-based arbiter blocking for periodic control
 - Standardized 16-bit PWM resolution
 - Comprehensive monitoring via arbiter_monbus_common
 - Optional ACK protocol support
 - Fixed internal sizing for consistency (16-entry FIFO, 256-cycle fairness reporting)
 - Configurable client count (1-64)
 - Real-time performance and fairness tracking
-

9.2 Module Purpose

This module provides a complete arbitration solution for multi-master systems where equal priority access is required. The integrated PWM generator enables precise timing control for

periodic arbitration windows, while the monitor bus integration provides real-time visibility into arbitration behavior, request latencies, and potential issues like starvation or protocol violations.

The standardized internal configurations (PWM width, FIFO depth, fairness intervals) ensure consistent behavior across all instantiations, simplifying integration and reducing parameter proliferation.

9.3 Parameters

Parameter	Type	Default	Description
CLIENTS	int	4	Number of arbitration clients (1-64)
WAIT_GNT_ACK	int	0	ACK protocol enable (0=disable, 1=enable)
MON_AGENT_ID	logic [15:0]	16'h0010	Monitor agent identifier (16-bit per monitor bus protocol)
MON_UNIT_ID	logic [7:0]	8'h00	Monitor unit identifier (8-bit per monitor bus protocol)
USE_MONITOR	bit	1	Synthesis-time monitor enable. When 0, omits monbus telemetry block (arbiter and PWM remain intact). When 1, full monitoring.

9.3.1 Fixed Internal Configurations (Not User-Configurable)

Configuration	Value	Rationale
PWM_WIDTH	16 bits	Adequate resolution for most use cases
MON_FIFO_DEPTH	16 entries	Optimal for monitoring scenarios
MON_FIFO_ALMOST_MARGIN	2 entries	Safety margin for FIFO
FAIRNESS_REPORT_CYCLES	256 cycles	Power-of-2 efficient interval

Configuration	Value	Rationale
MIN_GRANTS_FOR_FAIRNESS	64 grants	Statistical significance threshold

9.4 Port Groups

9.4.1 Clock and Reset

Port	Direction	Width	Description
clk	input	1	Clock signal
rst_n	input	1	Active-low asynchronous reset

9.4.2 Arbiter Interface

Port	Direction	Width	Description
request	input	CLIENTS	Client request signals
grant_valid	output	1	Grant is valid this cycle
grant	output	CLIENTS	One-hot grant vector
grant_id	output	$\$clog2(CLIENTS)$	Binary encoded grant ID
grant_ack	input	CLIENTS	Grant acknowledge signals (ACK mode only)

9.4.3 PWM Configuration

Port	Direction	Width	Description
cfg_pwm_sync_rst_n	input	1	PWM synchronous reset (active-low)
cfg_pwm_start	input	1	Start PWM generation
cfg_pwm_duty	input	16	Duty cycle value (0 to period-1)
cfg_pwm_period	input	16	Period value (1 to 65535)
cfg_pwm_reps	input	16	Number of periods to

Port	Direction	Width	Description
epeat_count			generate (0=infinite)
cfg_pwm_status_done	output	1	PWM sequence complete status
pwm_out	output	1	PWM output (controls arbiter blocking)

9.4.4 Monitor Configuration

Port	Direction	Width	Description
cfg_monitor_enable	input	1	Global monitor enable
cfg_monitor_packet_type_enable	input	16	Packet type enable mask
cfg_monitor_latency	input	16	Latency threshold (cycles)
cfg_monitor_starvation	input	16	Starvation threshold (cycles)
cfg_monitor_fairness	input	16	Fairness deviation threshold (percent)
cfg_monitor_active	input	16	Active request count threshold
cfg_monitor_period	input	8	Sample period for metrics

9.4.5 Monitor Bus Output

Port	Direction	Width	Description
monbus_valid	output	1	Monitor bus packet valid
monbus_ready	input	1	Monitor bus ready (from downstream)
monbus_packet	output	128	monitor_packet_event

Port	Direction	Width	Description
			packet

9.4.6 Debug Outputs

Port	Direction	Width	Description
debug_fifo_count	output	$\lceil \log_2(17) \rceil$	Monitor FIFO fill level (0-16)
debug_packet_count	output	16	Total packets generated

9.5 Functional Description

The module integrates three key components:

9.5.1 Round-Robin Arbiter

Provides fair arbitration using the `arbiter_round_robin` module: - Equal priority for all clients - Sequential rotation through active requests - Optional ACK protocol (`WAIT_GNT_ACK=1`) - `Block_arb` input for external flow control

9.5.2 PWM Generator

Controls arbiter availability through periodic blocking: - 16-bit resolution (65536 discrete levels) - Configurable duty cycle and period - Repeat count for finite sequences - PWM output directly drives arbiter `block_arb` input

Duty Cycle Calculation: - `pwm_out = 1` (arbiter blocked) for `cfg_pwm_duty` cycles - `pwm_out = 0` (arbiter active) for `cfg_pwm_period - cfg_pwm_duty` cycles

Example: 25% arbiter availability:

```
cfg_pwm_period = 16'd100;    // 100-cycle period
cfg_pwm_duty   = 16'd75;    // Block for 75 cycles
// Result: arbiter active for 25 cycles, blocked for 75 cycles
```

9.5.3 Monitor Bus Common

Comprehensive monitoring via `arbiter_monbus_common`: - Default client weights = 1 (equal priority) - Tracks grant distribution, latencies, starvation - Fixed internal configurations ensure consistency - Event packets include request latency, starvation detection, active request count

9.5.4 Operation Flow

1. Clients assert request signals

2. PWM output modulates arbiter availability (pwm_out -> block_arb)
 3. When arbiter active (pwm_out=0), round-robin arbiter grants one request
 4. Monitor tracks arbitration events (grants, timeouts, fairness)
 5. Event packets generated to monitor bus output
 6. If ACK mode enabled, arbiter waits for grant_ack before next grant
-

9.6 Usage Example

```
// 8-client round-robin arbiter with 30% availability window
arbiter_rr_pwm_monbus #(
    .CLIENTS      (8),
    .WAIT_GNT_ACK (0),           // No ACK protocol
    .MON_AGENT_ID (16'h0015),
    .MON_UNIT_ID  (8'h03)
) u_rr_arb (
    .clk           (clk),
    .rst_n         (rst_n),

    // Arbiter interface
    .request        (client_requests),
    .grant_valid    (grant_valid),
    .grant          (grant_onehot),
    .grant_id       (grant_binary),
    .grant_ack      (8'h00),    // Not used (ACK disabled)

    // PWM configuration - 30% availability
    .cfg_pwm_sync_rst_n (1'b1),
    .cfg_pwm_start      (pwm_start),
    .cfg_pwm_duty       (16'd70),    // Block for 70 cycles
    .cfg_pwm_period     (16'd100),   // Out of 100 cycles total
    .cfg_pwm_repeat_count (16'd0),   // Infinite repeat
    .cfg_pwm_sts_done   (pwm_done),
    .pwm_out            (pwm_signal),

    // Monitor configuration
    .cfg_mon_enable     (1'b1),
    .cfg_mon_pkt_type_enable (16'h000F), // Error, timeout,
completion, threshold
    .cfg_mon_latency    (16'd50),    // 50-cycle latency warning
    .cfg_mon_starvation (16'd200),    // 200-cycle starvation
threshold
    .cfg_mon_fairness   (16'd15),    // 15% fairness deviation
    .cfg_mon_active     (16'd7),     // Warn if 7+ active
requests

```

```

        .cfg_mon_period            (8'd128),

        // Monitor bus output
        .monbus_valid              (mon_valid),
        .monbus_ready              (mon_ready),
        .monbus_packet             (mon_packet),

        // Debug
        .debug_fifo_count          (mon_fifo_level),
        .debug_packet_count        (mon_pkt_count)
    );

    // Start PWM generation
    initial begin
        pwm_start = 1'b0;
        @(posedge clk);
        pwm_start = 1'b1;
        @(posedge clk);
        pwm_start = 1'b0;
    end

```

9.7 Design Notes

9.7.1 PWM Control Strategy

The PWM output directly controls arbiter blocking: - **pwm_out = 1**: Arbiter blocked (no grants issued) - **pwm_out = 0**: Arbiter active (can issue grants)

This allows precise timing control for: - Time-division multiple access (TDMA) scheduling - Bandwidth reservation windows - Periodic maintenance operations - Energy-efficient arbitration (idle periods)

9.7.2 Fixed Configuration Benefits

The standardized internal configurations provide: - **Consistency**: All instances behave identically - **Predictability**: Known FIFO depths, fairness intervals - **Simplified Integration**: Fewer parameters to configure - **Verified Operation**: Tested configurations

If different internal sizes are needed, create a new variant rather than parameterizing.

9.7.3 Fairness in Round-Robin Mode

All clients have equal priority (weight = 1 in monitoring logic). Fairness deviation events generated if any client receives significantly more or fewer grants than expected over the FAIRNESS_REPORT_CYCLES window (256 cycles).

Expected: Each client receives $(1/\text{CLIENTS}) * 100\%$ of grants Threshold: `cfg_mon_fairness` parameter (percent)

9.7.4 Monitor Bus Integration

See `arbiter_monbus_common.md` for complete monitoring details. Key points: - Uses `PROTOCOL_ARB` (3'b011) event encoding - Fixed 16-entry FIFO prevents event loss - Per-client latency and starvation tracking - Grant efficiency monitoring

9.8 Related Modules

9.8.1 Used By

- System arbitration hierarchies
- Multi-master interconnects
- TDMA scheduling systems

9.8.2 Uses

- `arbiter_round_robin.sv` (core RR arbiter)
- `pwm.sv` (PWM generator)
- `arbiter_monbus_common.sv` (monitoring infrastructure)

9.8.3 Related Modules

- `arbiter_wrr_pwm_monbus.sv` (weighted variant)
 - `monbus_arbiter.sv` (aggregates monitor bus streams)
-

9.9 References

9.9.1 Specifications

- Internal: `rtl/amba/PRD.md` (AMBA subsystem requirements)
- Internal: `docs/markdown/RTLAmba/arbiter_monbus_common.md` (monitoring details)

9.9.2 Source Code

- RTL: `rtl/amba/monitor/arbiter_rr_pwm_monbus.sv`
 - Tests: `val/amba/test_arbiter_rr_pwm_monbus.py`
-

9.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLamba Index](#)

10 Weighted Round-Robin Arbiter with PWM and Monitor Bus

Module: `arbiter_wrr_pwm_monbus.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

10.1 Overview

The Weighted Round-Robin Arbiter with PWM and Monitor Bus provides priority-based arbitration with configurable client weights, PWM flow control, and comprehensive silicon debug monitoring. This module extends round-robin arbitration with weight-based priority levels while maintaining the same monitoring infrastructure and standardized internal configurations.

10.1.1 Key Features

- Weighted round-robin arbitration with configurable priorities
 - Per-client weight thresholds (1 to MAX_LEVELS)
 - PWM-based arbiter blocking for periodic control
 - Standardized 16-bit PWM resolution
 - Comprehensive fairness deviation monitoring
 - Optional ACK protocol support
 - Fixed internal sizing (16-entry FIFO, 256-cycle fairness reporting)
 - Enhanced debug outputs for priority tracking
-

10.2 Module Purpose

This module enables priority-based multi-master arbitration where different clients require different service levels. Higher-weight clients receive proportionally more grants while lower-weight clients still receive guaranteed service, preventing starvation. The integrated monitoring infrastructure tracks whether actual grant distribution matches configured weights and reports fairness violations.

The PWM integration allows temporal control of arbiter availability, enabling TDMA windows or bandwidth reservation combined with priority-based selection within each active window.

10.3 Parameters

Parameter	Type	Default	Description
MAX_LEVELS	int	16	Maximum weight levels per client (1-64)
CLIENTS	int	4	Number of arbitration clients (1-64)
WAIT_GNT_ACK	int	0	ACK protocol enable (0=disable, 1=enable)
MON_AGENT_ID	logic [15:0]	16'h0010	Monitor agent identifier (16-bit per monitor bus protocol)
MON_UNIT_ID	logic [7:0]	8'h00	Monitor unit identifier (8-bit per monitor bus protocol)
USE_MONITOR	bit	1	Synthesis-time monitor enable. When 0, omits monbus telemetry block (arbiter and PWM remain intact). When 1, full monitoring.

10.3.1 Fixed Internal Configurations (Not User-Configurable)

Configuration	Value	Rationale
PWM_WIDTH	16 bits	Adequate resolution for most use cases
MON_FIFO_DEPTH	16 entries	Optimal for monitoring scenarios
MON_FIFO_ALMOST_MARGIN	2 entries	Safety margin for FIFO
FAIRNESS_REPORT_CYCLES	256 cycles	Power-of-2 efficient interval
MIN_GRANTS_FOR_FAIRNESS	64 grants	Statistical significance

Configuration	Value	Rationale
		threshold

10.4 Port Groups

10.4.1 Clock and Reset

Port	Direction	Width	Description
clk	input	1	Clock signal
rst_n	input	1	Active-low asynchronous reset

10.4.2 Arbiter Configuration and Interface

Port	Direction	Width	Description
cfg_arb_m ax_thresh	input	CLIENTS * \$clog2(MAX_ LEVELS)	Per-client weight thresholds
request	input	CLIENTS	Client request signals
grant_valid	output	1	Grant is valid this cycle
grant	output	CLIENTS	One-hot grant vector
grant_id	output	\$clog2(CLIENTS)	Binary encoded grant ID
grant_ack	input	CLIENTS	Grant acknowledge signals (ACK mode only)

10.4.3 PWM Configuration

Port	Direction	Width	Description
cfg_pwm_sync_ rst_n	input	1	PWM synchronous reset (active-low)
cfg_pwm_start	input	1	Start PWM generation
cfg_pwm_duty	input	16	Duty cycle

Port	Direction	Width	Description
			value (0 to period-1)
cfg_pwm_period	input	16	Period value (1 to 65535)
cfg_pwm_repeat_count	input	16	Number of periods (0=infinite)
cfg_pwm_statusdone	output	1	PWM sequence complete status
pwm_out	output	1	PWM output (controls arbiter blocking)

10.4.4 Monitor Configuration

Port	Direction	Width	Description
cfg_monitor_enable	input	1	Global monitor enable
cfg_monitor_packet_type_enable	input	16	Packet type enable mask
cfg_monitor_latency_threshold	input	16	Latency threshold (cycles)
cfg_monitor_starvation_threshold	input	16	Starvation threshold (cycles)
cfg_monitor_fairness_threshold	input	16	Fairness deviation threshold (percent)
cfg_monitor_active_threshold	input	16	Active request count threshold

Port	Direction	Width	Description
cfg_mon_ack_timeout_thresh	input	16	ACK timeout threshold (cycles)
cfg_mon_efficiency_thresh	input	16	Grant efficiency threshold (percent)
cfg_mon_sample_period	input	8	Sample period for metrics

10.4.5 Monitor Bus Output

Port	Direction	Width	Description
monbus_valid	output	1	Monitor bus packet valid
monbus_ready	input	1	Monitor bus ready (from downstream)
monbus_packet	output	128	monitor_packet_t event packet

10.4.6 Enhanced Debug Outputs

Port	Direction	Width	Description
debug_fifo_count	output	$\lceil \log_2(17) \rceil$	Monitor FIFO fill level
debug_packet_count	output	16	Total packets generated
debug_ack_timeout	output	CLIENTS	Per-client ACK timeout status
debug_protocol_violations	output	16	Protocol violation count
debug_grant_efficiency	output	16	Grant efficiency percentage

Port	Direction	Width	Description
debug_client_starvation	output	CLIENTS	Per-client starvation flags
debug_fairness_deviation	output	16	Maximum fairness deviation
debug_monitor_state	output	3	Monitor internal state

10.5 Functional Description

The module integrates four key components:

10.5.1 Weighted Round-Robin Arbiter

Provides priority-based arbitration using `arbiter_round_robin_weighted`: - Per-client weight configuration via `cfg_arb_max_thresh` - Higher weight = higher priority - Fair rotation within same weight level - Prevents starvation of low-priority clients - Optional ACK protocol support

Weight Encoding: `cfg_arb_max_thresh` is a concatenated vector:

```
cfg_arb_max_thresh = {weight[CLIENTS-1], ..., weight[1], weight[0]}
```

Where each weight is $\lceil \log_2(\text{MAX_LEVELS}) \rceil$ bits wide.

Arbitration Algorithm: 1. Identify all active requests 2. Select highest weight among active requests 3. Round-robin among clients with that weight 4. Issue grant and update rotation pointer

10.5.2 PWM Generator

Controls arbiter availability (same as RR variant): - 16-bit resolution - Configurable duty cycle and period - PWM output directly drives `block_arb`

10.5.3 Monitor Bus Common

Comprehensive monitoring with enhanced fairness tracking: - **WEIGHTED_MODE = 1** enables weight-aware fairness analysis - Compares actual vs expected grant distribution based on configured weights - Detects fairness violations when distribution deviates from weight ratios

Fairness Calculation Example:

4 clients with weights [8, 4, 2, 1] (total weight = 15)

Expected distribution:

Client 0: $(8/15) * 100 = 53.3\%$

Client 1: $(4/15) * 100 = 26.7\%$
 Client 2: $(2/15) * 100 = 13.3\%$
 Client 3: $(1/15) * 100 = 6.7\%$

Actual distribution measured over 256-cycle window.
 Deviation = $|\text{actual}\% - \text{expected}\%|$ for each client.
 Event generated if $\text{max deviation} > \text{cfg_mon_fairness_thresh}$.

10.5.4 Operation Flow

1. Configure per-client weights via `cfg_arb_max_thresh`
 2. Clients assert request signals
 3. PWM output modulates arbiter availability
 4. When active, weighted arbiter grants highest-priority request
 5. Monitor tracks grants and calculates fairness metrics
 6. Fairness violations reported if distribution deviates from weights
-

10.6 Usage Example

```
// 4-client weighted arbiter with priorities [8, 4, 2, 1]
localparam int MAX_LEVELS = 16;
localparam int CLIENTS = 4;
localparam int WEIGHT_WIDTH = $clog2(MAX_LEVELS); // 4 bits

// Configure client weights
logic [WEIGHT_WIDTH-1:0] client_weights [CLIENTS];
assign client_weights[0] = 4'd8; // Highest priority
assign client_weights[1] = 4'd4;
assign client_weights[2] = 4'd2;
assign client_weights[3] = 4'd1; // Lowest priority

wire [CLIENTS*WEIGHT_WIDTH-1:0] cfg_weights = {
    client_weights[3], client_weights[2],
    client_weights[1], client_weights[0]
};

arbiter_wrr_pwm_monbus #(
    .MAX_LEVELS (MAX_LEVELS),
    .CLIENTS (CLIENTS),
    .WAIT_GNT_ACK (1), // Enable ACK protocol
    .MON_AGENT_ID (16'h0018),
    .MON_UNIT_ID (8'h04)
) u_wrr_arb (
    .clk (clk),
```

```

.rst_n                (rst_n),

// Arbiter configuration
.cfg_arb_max_thresh   (cfg_weights),
.request              (client_requests),
.grant_valid          (grant_valid),
.grant                (grant_onehot),
.grant_id             (grant_binary),
.grant_ack            (grant_ack),

// PWM configuration - 40% availability
.cfg_pwm_sync_rst_n   (1'b1),
.cfg_pwm_start        (pwm_start),
.cfg_pwm_duty         (16'd60),      // Block for 60 cycles
.cfg_pwm_period       (16'd100),     // Out of 100 total
.cfg_pwm_repeat_count (16'd0),       // Infinite
.cfg_pwm_sts_done     (pwm_done),
.pwm_out              (pwm_signal),

// Monitor configuration
.cfg_mon_enable        (1'b1),
.cfg_mon_pkt_type_enable (16'h003F),
.cfg_mon_latency_thresh (16'd100),
.cfg_mon_starvation_thresh (16'd500),
.cfg_mon_fairness_thresh (16'd10),   // 10% deviation threshold
.cfg_mon_active_thresh (16'd4),
.cfg_mon_ack_timeout_thresh (16'd200),
.cfg_mon_efficiency_thresh (16'd75),
.cfg_mon_sample_period (8'd100),

// Monitor bus output
.monbus_valid          (mon_valid),
.monbus_ready          (mon_ready),
.monbus_packet         (mon_packet),

// Enhanced debug outputs
.debug_fifo_count      (mon_fifo_level),
.debug_packet_count    (mon_pkt_count),
.debug_ack_timeout     (mon_ack_timeouts),
.debug_protocol_violations (mon_violations),
.debug_grant_efficiency (mon_efficiency),
.debug_client_starvation (mon_starvation),
.debug_fairness_deviation (mon_fairness_dev),
.debug_monitor_state   (mon_state)
);

```

10.7 Design Notes

10.7.1 Weight Configuration

The `cfg_arb_max_thresh` parameter requires careful configuration:

Valid Range: Each client weight must be 1 to (MAX_LEVELS-1) **Zero Weights:** Zero-weight clients are effectively disabled **Dynamic Updates:** Weights can be changed runtime, but may cause temporary fairness violations during transition

Example Calculation (4 clients, MAX_LEVELS=16):

```
localparam WEIGHT_WIDTH = 4; // $clog2(16) = 4
logic [WEIGHT_WIDTH-1:0] w0 = 4'd12; // 75% of max
logic [WEIGHT_WIDTH-1:0] w1 = 4'd6; // 37.5% of max
logic [WEIGHT_WIDTH-1:0] w2 = 4'd3; // 18.75% of max
logic [WEIGHT_WIDTH-1:0] w3 = 4'd1; // 6.25% of max

assign cfg_arb_max_thresh = {w3, w2, w1, w0}; // LSB is client 0
```

10.7.2 Fairness Deviation Threshold

The `cfg_mon_fairness_thresh` parameter should account for statistical variation:

Recommended Values: - 5% for strict fairness enforcement - 10% for typical systems (allows reasonable variation) - 20% for relaxed monitoring

Factors Affecting Deviation: - Short measurement windows (MIN_GRANTS_FOR_FAIRNESS) - Bursty request patterns - Weight ratio extremes (e.g., 16:1 vs 2:1) - PWM blocking periods

10.7.3 Enhanced Debug Outputs

This module exposes complete debug outputs from `arbiter_monbus_common`:

- **debug_client_starvation:** Bit vector showing starved clients
 - Use to identify clients not receiving service
 - Indicates potential weight configuration issues
- **debug_fairness_deviation:** Maximum deviation percentage
 - Real-time fairness metric without consuming monitor bus bandwidth
 - Connect to system debug registers for runtime visibility
- **debug_grant_efficiency:** Grant completion ratio
 - Low efficiency may indicate ACK timeout issues or downstream congestion

10.7.4 ACK Mode with Weighted Arbitration

When `WAIT_GNT_ACK=1`, the arbiter waits for `grant_ack` before issuing next grant. This can affect fairness:

Impact on Fairness: - Slow-ACK clients may receive fewer grants than their weight warrants - ACK timeout tracking helps identify problematic clients - Consider increasing `cfg_mon_ack_timeout_thresh` for clients with known slower response

10.8 Related Modules

10.8.1 Used By

- Priority-based interconnect arbiters
- QoS-aware memory controllers
- Multi-tier scheduling hierarchies

10.8.2 Uses

- `arbiter_round_robin_weighted.sv` (core WRR arbiter)
- `pwm.sv` (PWM generator)
- `arbiter_monbus_common.sv` (monitoring with `WEIGHTED_MODE=1`)

10.8.3 Related Modules

- `arbiter_rr_pwm_monbus.sv` (equal-priority variant)
 - `monbus_arbiter.sv` (aggregates monitor bus streams)
-

10.9 References

10.9.1 Specifications

- Internal: `rtl/amba/PRD.md` (AMBA subsystem requirements)
- Internal: `docs/markdown/RTLAmba/arbiter_monbus_common.md` (monitoring details)

10.9.2 Source Code

- RTL: `rtl/amba/monitor/arbiter_wrr_pwm_monbus.sv`
 - Tests: `val/amba/test_arbiter_wrr_pwm_monbus.py`
-

10.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

11 AXI4 Master Read Monitor (Clock-Gated)

Module: `axi4_master_rd_mon_cg.sv` **Base Module:** [axi4_master_rd_mon](#) **Location:** `rtl/amba/axi4/` **Status:** ✓ Production Ready

11.1 Overview

The `axi4_master_rd_mon_cg` module is a clock-gated variant of [axi4_master_rd_mon](#) that adds comprehensive power optimization capabilities through activity-based clock gating.

11.1.1 Key Differences from Base Module

- ✓ **Activity-Based Clock Gating:** Automatically gates clocks when subsystems are idle
- ✓ **Configurable Policies:** Fine-grained control over what gets gated and when
- ✓ **Power Monitoring:** Built-in statistics for clock gating effectiveness
- ✓ **Independent Gating Domains:** Separate control for different functional blocks
- ✓ **Zero Functional Impact:** Maintains 100% functional equivalence with base module

All other functionality is identical to the base module. See [axi4_master_rd_mon.md](#) for complete functional specification.

11.2 When to Use Clock-Gated Variant

Use `axi4_master_rd_mon_cg` when: - Power consumption is a critical concern - Design has periods of inactivity (burst traffic patterns) - FPGA/ASIC has integrated clock gating support - Meeting power budgets for battery-operated systems

Use base module (`axi4_master_rd_mon`) when: - Maximum performance with no power constraints - Continuous high-activity traffic - Simpler design with fewer configuration parameters - Minimizing gate count is priority

11.3 Clock Gating Parameters

In addition to all parameters from [axi4_master_rd_mon](#), this module adds:

Parameter	Type	Default	Description
ENABLE_CLOCK_GATING	bit	1	Master enable for clock gating (0=disable all gating)
CG_IDLE_CYCLES	int	8	Number of idle cycles before asserting clock gate
CG_GATE_MONITOR	bit	1	Enable clock gating for monitor logic
CG_GATE_REPORTER	bit	1	Enable clock gating for packet reporter
CG_GATE_TIMERS	bit	1	Enable clock gating for timeout timers

11.3.1 Base Module Parameters

All parameters from [axi4_master_rd_mon](#) are supported, including:

Parameter	Type	Default	Description
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
N_ADDR_RANGES	int	0	Number of address-range comparators (forwarded to base module).

All base-module ports are forwarded unchanged, including the `cam_clear` control input (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]) - and the full performance-monitoring interface (see [Performance Monitoring](#) below). The six `ENABLE_*_LOGIC` synthesis-cone parameters and the `cfg_compl_enable / cfg_threshold_enable / cfg_debug_enable` control enables are also passed straight through.

11.3.2 Parameter Relationships

- **ENABLE_CLOCK_GATING = 0:** Disables all clock gating, module behaves identically to base
 - **CG_IDLE_CYCLES:** Higher values = more power savings but slower wake-up from idle
 - **Individual CG_GATE_* signals:** Allow fine-grained control over which subsystems are gated
-

11.4 Usage Examples

11.4.1 Example 1: Maximum Power Savings (Burst Traffic)

```
axi4_master_rd_mon_cg #(
    // Base parameters (see axi4_master_rd_mon.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating - aggressive power savings
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(4),           // Gate quickly after 4 idle cycles
    .CG_GATE_MONITOR(1),         // Gate monitor logic
    .CG_GATE_REPORTER(1),        // Gate reporter logic
    .CG_GATE_TIMERS(1)           // Gate timers (if no timeouts enabled)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... connect signals same as base module
);
```

11.4.2 Example 2: Balanced Performance and Power

```
axi4_master_rd_mon_cg #(
    // Base parameters
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),
```

```

        // Clock gating - balanced approach
        .ENABLE_CLOCK_GATING(1),
        .CG_IDLE_CYCLES(16),      // Wait longer before gating (faster
wake-up)
        .CG_GATE_MONITOR(1),      // Gate monitor logic
        .CG_GATE_REPORTER(0),     // Don't gate reporter (always ready)
        .CG_GATE_TIMERS(1)        // Gate timers when not needed
    ) u_cg (
        .aclk(clk),
        .aresetn(rst_n),
        // ... connect signals same as base module
    );

```

11.4.3 Example 3: Clock Gating Disabled (Functional Verification)

```

axi4_master_rd_mon_cg #(
    // Base parameters
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating - DISABLED for verification
    .ENABLE_CLOCK_GATING(0) // Disable all gating
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... connect signals same as base module
);

```

Note: With `ENABLE_CLOCK_GATING=0`, this module is functionally identical to the base module.

11.5 Performance Monitoring

The clock-gated wrapper exposes the full perfmon interface of the base module and **forwards every port unchanged** to the inner `axi4_master_rd_mon`. The measurement-window state machine, the four R-channel utilization buckets (productive / back-pressure / starvation / idle), and the beat/byte/burst throughput counters behave exactly as documented in the base module — see [Performance Monitoring in axi4_master_rd_mon](#) for the full narrative and per-bit semantics.

Forwarded perfmon ports (identical width and direction to the base module):

- **Inputs:** `cfg_perf_enable`, `cfg_start_event_sel` (3), `cfg_end_event_sel` (3), `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`

- **Outputs:** window_active, window_cycles (32), perf_prod_cycles (32), perf_bp_cycles (32), perf_starv_cycles (32), perf_idle_cycles (32), perf_beat_count (32), perf_byte_count (64), perf_burst_count (32)

Clock gating never suppresses these paths: while a measurement window is open the monitor stays awake, so window cycle accounting remains exact regardless of CG_IDLE_CYCLES.

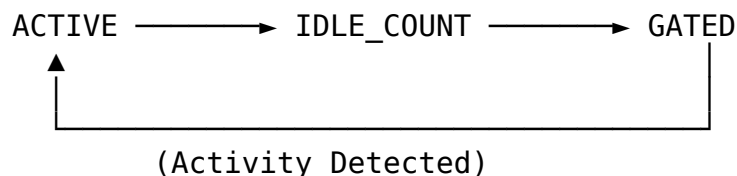
11.6 Clock Gating Architecture

11.6.1 Gating Domains

The module implements independent clock gating for these functional blocks:

1. **Monitor Logic Domain**
 - Transaction tracking and state machines
 - Gated when: No active transactions for CG_IDLE_CYCLES
 - Controlled by: CG_GATE_MONITOR
2. **Reporter Domain**
 - Monitor packet generation and formatting
 - Gated when: No packets to report for CG_IDLE_CYCLES
 - Controlled by: CG_GATE_REPORTER
3. **Timer Domain**
 - Timeout detection and performance counters
 - Gated when: All timeout enables are 0
 - Controlled by: CG_GATE_TIMERS

11.6.2 Gating State Machine



States:

- **ACTIVE:** Clocks enabled, monitoring activity
- **IDLE_COUNT:** Counting CG_IDLE_CYCLES before gating
- **GATED:** Clocks disabled, waiting for activity

11.6.3 Wake-Up Latency

Configuration	Wake-Up Time	Use Case
CG_IDLE_CYCLES=4	~4 clock cycles	Low-latency, frequent bursts
CG_IDLE_CYCLES=8	~8 clock cycles	Balanced (default)
CG_IDLE_CYCLES=16	~16 clock cycles	Maximum power savings, infrequent traffic

11.7 Power Savings Analysis

11.7.1 Typical Power Reduction

Based on representative workloads:

Traffic Pattern	Clock Gating Enabled	Power Savings
10% Utilization	Aggressive (CG_IDLE_CYCLES=4)	60-70%
25% Utilization	Balanced (CG_IDLE_CYCLES=8)	45-55%
50% Utilization	Conservative (CG_IDLE_CYCLES=16)	25-35%
90% Utilization	Any configuration	5-10%

Note: Actual savings depend on FPGA/ASIC technology, tool implementation, and traffic patterns.

11.7.2 Power Monitoring Signals

The module provides these status signals for power analysis:

Signal	Width	Description
cg_monitor_gated	1	Monitor domain clock is gated
cg_reporter_gated	1	Reporter domain clock is gated
cg_timers_gated	1	Timer domain clock is gated

11.8 Verification Considerations

11.8.1 Clock Gating in Simulation

Recommendation: Disable clock gating during functional verification:

```
// Testbench instantiation
axi4_master_rd_mon_cg #(
    .ENABLE_CLOCK_GATING(0) // Disable for faster simulation
) dut (
    // ... connections
);
```

Rationale: - Simpler waveforms (no clock gating events) - Faster simulation (no gating overhead) - Easier debug (no timing dependencies)

11.8.2 Power Analysis Verification

For power-specific verification:

1. **Enable clock gating** with realistic parameters
 2. **Monitor gating signals** (cg_*_gated) to verify expected behavior
 3. **Vary traffic patterns** to test gating effectiveness
 4. **Check wake-up timing** meets system requirements
-

11.9 Synthesis Considerations

11.9.1 FPGA Implementations

Xilinx: - Use ENABLE_CLOCK_GATING=1 with BUFGCE primitives - Tool will infer clock enables automatically - Verify with post-synthesis power analysis

Intel (Altera): - Use ENABLE_CLOCK_GATING=1 with ALTCLKCTRL - May need vendor-specific clock gating primitives - Check power reports for gating effectiveness

Lattice: - Basic clock gating supported - May require manual instantiation of clock enables - Verify functionality in timing simulation

11.9.2 ASIC Implementations

- Work with foundry to select appropriate clock gating cells
- Integrated Clock Gating (ICG) cells provide best results
- Consider hold-time implications of clock gating

- Verify power intent with UPF (Unified Power Format)
-

11.10 Related Modules

- [axi4_master_rd_mon](#) - Base module (non-clock-gated)
 - [axi_monitor_base](#) - Core monitoring infrastructure
 - [axi_monitor_filtered](#) - Filtering capabilities
-

11.11 See Also

- **Power Optimization Guide:** docs/POWER_OPTIMIZATION_GUIDE.md
 - **Clock Gating Best Practices:** docs/CLOCK_GATING_GUIDE.md
 - **AMBA Subsystem Overview:** docs/markdown/RTLAmba/overview.md
-

11.12 Navigation

- [← Back to Base Module](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

12 AXI4 Master Read Monitor

Module: axi4_master_rd_mon.sv **Location:** rtl/amba/axi4/ **Status:** ✓ Production Ready

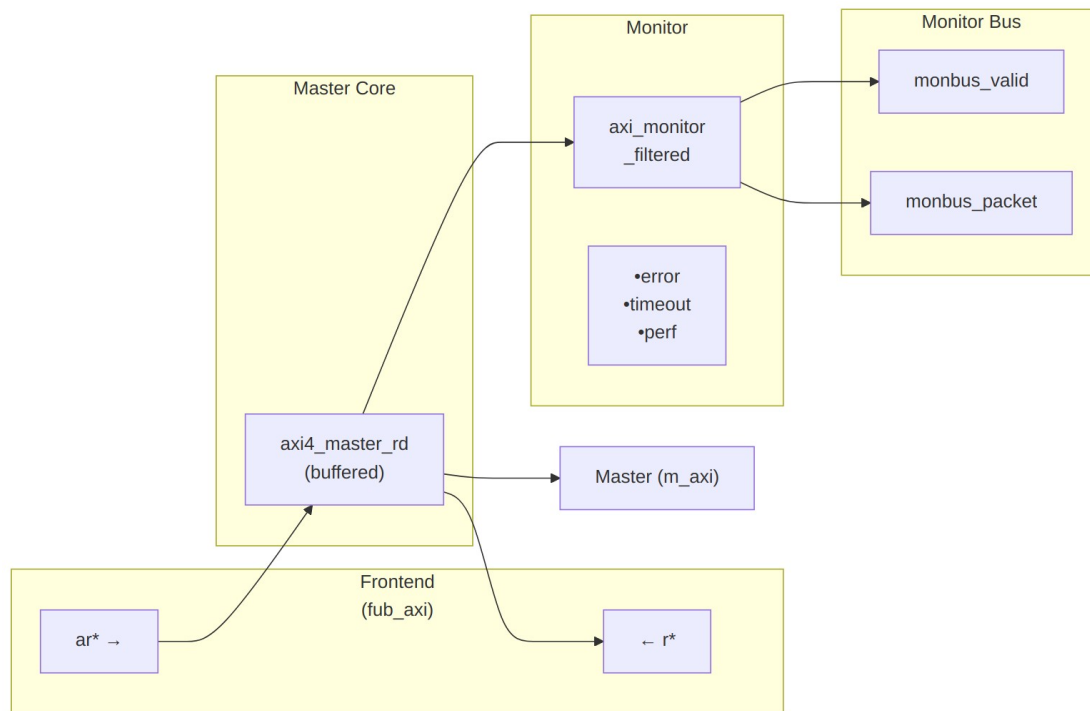
12.1 Overview

The AXI4 Master Read Monitor module combines a functional AXI4 master read interface with comprehensive transaction monitoring and filtering capabilities. This module is essential for verification environments, providing real-time protocol checking, error detection, performance metrics, and configurable packet filtering.

12.1.1 Key Features

- ✓ **Integrated Monitoring:** Combines axi4_master_rd with axi_monitor_filtered
- ✓ **3-Level Filtering:** Packet type masks, error routing, individual event masking

- ✓ **Error Detection:** Protocol violations, SLVERR, DECERR, orphan transactions
 - ✓ **Timeout Monitoring:** Configurable timeout detection for stuck transactions
 - ✓ **Performance Metrics:** Latency tracking, transaction counting, throughput analysis
 - ✓ **Monitor Bus Output:** 128-bit packets paired with 64-bit side-band timestamps
 - ✓ **Configuration Validation:** Detects conflicting configuration settings
 - ✓ **Clock Gating Support:** Busy signal for power management
-



Module Architecture

The module instantiates two sub-modules: 1. **axi4_master_rd** - Core AXI4 functionality with buffering 2. **axi_monitor_filtered** - Transaction monitoring with 3-level filtering

12.2 Parameters

12.2.1 AXI4 Master Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel skid buffer depth
SKID_DEPTH_R	int	4	R channel skid buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width

12.2.2 Monitor Parameters

Parameter	Type	Default	Description
UNIT_ID	int	1	4-bit unit identifier in monitor packets
AGENT_ID	int	10	8-bit agent identifier in monitor packets
MAX_TRANSACTIONS	int	16	Maximum concurrent outstanding transactions
ENABLE_FILTERING	bit	1	Enable packet filtering (0=pass all packets)
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor

Parameter	Type	Default	Description
N_ADDR_RANGES	int	0	functionality. Number of address-range comparators. 0 = checker omitted (zero area). >0 = N independent [low, high] ranges; hits emit PktTypeError + AXI_ERR_ADDR_RANGE on monbus.

12.2.3 Synthesis-Cone Parameters

Each detection cone can be compiled out to save area. The classic cones default on; the debug cone defaults off. Setting a bit to 0 drops the corresponding cone at synthesis via a generate-if inside `axi_monitor_reporter` — the area saving is real.

Parameter	Type	Default	Effect when 0
ENABLE_ERROR_LOGIC	bit	1	Drop the error-detection cone
ENABLE_TIMEOUT_LOGIC	bit	1	Drop the timeout cone and the <code>axi_monitor_timeout</code> instance
ENABLE_COMPL_LOGIC	bit	1	Drop the completion cone
ENABLE_THRESHOLD_LOGIC	bit	1	Drop the threshold cone
ENABLE_PERF_LOGIC	bit	1	Drop the perfmon window + counters
ENABLE_DEBUG_LOGIC	bit	0	Drop the debug (trace) cone — the 6th reporter sub-block (off by default)

Internally the wrapper hardwires two master switches on its `axi_monitor_filtered` instance: `ENABLE_PERF_PACKETS = 1` (perf datapath present, so `ENABLE_PERF_LOGIC` alone gates the window/counters) and `ENABLE_DEBUG_MODULE = 0` (debug tracking module omitted). These are not top-level parameters of the wrapper.

The transaction CAM is always pipelined.

12.3 Monitor Backpressure (block_ready)

The monitor exposes a `block_ready` signal that goes low when its internal FIFO is saturated and cannot accept a new in-flight transaction. The wrapper ANDs `block_ready` into the upstream-facing `fub_axi_arready` so a saturated monitor stalls new transactions on the wire instead of dropping events.

- **Where the stall lands:** the upstream `fub_axi_arready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`:** `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.

This replaces a previous bug where `block_ready` was left unconnected and a full monitor FIFO would silently lose events.

12.4 Address-Range Checker

The wrapper can be parameterized with `N_ADDR_RANGES > 0` to instantiate an N-comparator address-range checker that watches every accepted AR handshake and emits a `PktTypeError` monbus packet (event code `AXI_ERR_ADDR_RANGE = 8'h0D`) when an address falls inside any of the configured `[low, high]` inclusive ranges.

Config inputs (active only when `N_ADDR_RANGES > 0`): - `cfg_addr_check_enable` — master on/off for the checker. - `cfg_addr_range_enable[N-1:0]` — per-range enable bit. - `cfg_addr_range_low[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive low bound for each range. - `cfg_addr_range_high[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive high bound for each range.

Event encoding (within the standard 128-bit `monitor_packet_t`, `event_data` field): - `packet_type = PktTypeError (4'h0)` - `protocol = AXI (3'b000)` - `event_code = AXI_ERR_ADDR_RANGE (8'h0D)` - `event_data[63:60] = range_index` (4 bits; supports up to 16 ranges) - `event_data[59:0] = full matched address` (up to 60 bits, zero-padded if narrower)

Exact match: set `cfg_addr_range_low[i] == cfg_addr_range_high[i]`.

Filtering: the existing `cfg_axi_error_mask[13]` bit masks this event code; set it high to suppress range packets without disabling other errors. No new mask wiring needed.

Per-range coalescing: if a range hits again before its packet has been emitted, the latched address is overwritten (latest hit wins). One packet per cycle drains the pending mask via a lowest-index priority encoder. Distinct ranges never lose events; under sustained per-range bursts, only the latest address per range is reported per emission cycle.

12.5 Performance Monitoring

The wrapper hardwires `ENABLE_PERF_PACKETS = 1` on its inner `axi_monitor_filtered`, so the perfmon datapath is present whenever `ENABLE_PERF_LOGIC = 1` (the default). It instantiates a **measurement-window state machine** plus a bank of R-data-channel utilization counters. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows, so the host can read a completed window's totals.

12.5.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**. The event sources are selected by `cfg_start_event_sel / cfg_end_event_sel` (3-bit; e.g. `3'b010` selects the `cfg_perf_enable` edge) and can also be fired directly by the `cfg_start_trigger / cfg_end_trigger` pulses (from an engine or CSR). `cfg_window_force_close` is a software override that closes the window immediately. While the window is open:

- `window_active` is high.
- `window_cycles[31:0]` free-runs, counting every clock elapsed inside the window.

Sample the counters on the cycle `window_active` falls to 0 (drive `cfg_end_trigger`, or wait for the configured end event).

12.5.2 Utilization Counters (R Data Channel)

Every cycle inside the window is classified by the R channel's `rvalid / rready` into exactly one of four buckets:

Output	Width	Condition	Meaning
<code>perf_prod_cycles</code>	32	<code>rvalid && rready</code>	productive beat transferred
<code>perf_bp_cycles</code>	32	<code>rvalid && !rready</code>	back-pressure (data offered, sink not ready)
<code>perf_starv_cycles</code>	32	<code>!rvalid && rready</code>	starvation (sink ready, no data)
<code>perf_idle_cycles</code>	32	<code>!rvalid && !rready</code>	idle

The four buckets sum to `window_cycles`, so `utilization = perf_prod_cycles / window_cycles`.

12.5.3 Throughput Counters

Output	Width	Meaning
perf_beat_count	32	R data beats transferred (= perf_prod_cycles, 1 beat/cycle)
perf_byte_count	64	bytes transferred = beats × (1 << ARSIZE), using the ARSIZE captured at the most recent AR address phase
perf_burst_count	32	AR address-phase handshakes

The integrator computes average burst length as $\text{perf_beat_count} / \text{perf_burst_count}$.

12.5.4 Performance Monitoring Ports

Port	Direction	Width	Description
cfg_perf_enable	Input	1	Enable performance-metric packet generation
cfg_start_event_sel	Input	3	Window start event source select
cfg_end_event_sel	Input	3	Window end event source select
cfg_start_trigger	Input	1	Pulse: open the measurement window
cfg_end_trigger	Input	1	Pulse: close the measurement window
cfg_window_force_close	Input	1	Software override: force the window closed
window_active	Output	1	High while a measurement window is open
window_cycles	Output	32	Cycles elapsed in the current window
perf_prod_cycles	Output	32	rvalid && rready cycles

Port	Direction	Width	Description
perf_bp_cycles	Output	32	rvalid && !rready cycles (back-pressure)
perf_starv_cycles	Output	32	!rvalid && rready cycles (starvation)
perf_idle_cycles	Output	32	!rvalid && !rready cycles
perf_beat_count	Output	32	R data beats transferred
perf_byte_count	Output	64	bytes transferred
perf_burst_count	Output	32	AR address-phase handshakes

When `USE_MONITOR = 0`, every perfmon output is tied to 0 and the window never opens.

12.6 Port Groups

12.6.1 AXI4 Interfaces

****Frontend Interface (fub_axi_*)**** - AR channel inputs: arid, araddr, arlen, arsize, arburst, arlock, arcache, arprot, arqos, arregion, aruser, arvalid - AR channel output: arready - R channel outputs: rid, rdata, rresp, rl原因, ruser, rvalid - R channel input: rready

****Master Interface (m_axi_*)**** - AR channel outputs: arid, araddr, arlen, arsize, arburst, arlock, arcache, arprot, arqos, arregion, aruser, arvalid - AR channel input: arready - R channel inputs: rid, rdata, rresp, rl原因, ruser, rvalid - R channel output: rready

12.6.2 Monitor Configuration

Basic Configuration:

Port	Direction	Width	Description
cfg_monitor_enable	Input	1	Master enable for all monitoring
cam_clear	Input	1	Synchronous clear of the monitor transaction CAM (driven from the harness)

Port	Direction	Width	Description
			clear control bit, e.g. CTRL[4])
cfg_error_enable	Input	1	Enable error packet generation
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance metrics
cfg_completion_enable	Input	1	Enable transaction-completion packets
cfg_threshold_enable	Input	1	Enable threshold-crossed packets
cfg_debug_enable	Input	1	Enable debug/trace packets (gates the debug cone — the 6th reporter sub-block)
cfg_timeout_cycles	Input	16	Timeout threshold in clock cycles
cfg_latency_threshold	Input	32	Latency threshold for alerts

The inner monitor's `cfg_debug_level` (tied to 0), `cfg_debug_mask` (0) and `cfg_active_trans_threshold` (8) are fixed inside the wrapper and are **not** top-level ports on this module. `cfg_debug_enable` is the only debug-cone control exposed here.

Filtering Configuration (3 Levels):

Level 1 - Packet Type Masks:

Port	Direction	Width	Description
cfg_axi_packet_mask	Input	16	Drop mask for entire packet types
cfg_axi_error_select	Input	16	Error routing select (future use)

Level 2 & 3 - Individual Event Masks:

Port	Direction	Width	Description
cfg_axi_error_mask	Input	16	Mask specific error events
cfg_axi_timeout_mask	Input	16	Mask specific timeout events
cfg_axi_completion_mask	Input	16	Mask specific completion events
cfg_axi_threshold_mask	Input	16	Mask specific threshold events
cfg_axi_performance_mask	Input	16	Mask specific performance events
cfg_axi_address_match_mask	Input	16	Mask specific address match events
cfg_axi_debug_mask	Input	16	Mask specific debug events

12.6.3 Monitor Bus Output

Port	Direction	Width	Description
monbus_valid	Output	1	Monitor packet valid
monbus_ready	Input	1	Downstream ready to accept packet
monbus_packet	Output	128	monitor_packet_t (see format below)
monbus_timestamp	Output	64	monbus_timestamp_t paired atomically with monbus_packet
i_mon_time	Input	64	Free-running counter from monbus_axil_group, sampled at packet emission

12.6.4 Status Outputs

Port	Direction	Width	Description
busy	Output	1	Indicates active transactions (for clock

Port	Direction	Width	Description
			gating)
active_transactions	Output	8	Current number of outstanding transactions
error_count	Output	16	Cumulative error count
transaction_count	Output	32	Total transaction count
cfg_conflict_error	Output	1	Configuration conflict detected

12.7 Monitor Packet Format

The 128-bit monbus_packet (paired with the 64-bit monbus_timestamp side-band signal) follows the standardized AMBA monitor bus format:

Bits [127:124] - Packet Type:

- 0x0 = ERROR Error events (SLVERR, DECERR, protocol violations)
- 0x1 = COMPL Completion events (transaction finished)
- 0x2 = THRESH Threshold events
- 0x3 = TIMEOUT Timeout events
- 0x4 = PERF Performance metrics
- 0x8 = ADDR_MATCH Address match events
- 0x9 = APB APB-specific events
- 0xF = DEBUG Debug events

Bits [123:109] - Reserved (15 bits, forward-compat slack)

Bits [108:105] - Protocol (4 bits): 0x0=AXI, 0x1=AXIS, 0x2=APB, 0x3=ARB, 0x4=CORE

Bits [104:97] - Event Code (8 bits, protocol-specific)

Bits [96:88] - Channel ID (9 bits – AXI ID or channel index)

Bits [87:72] - Agent ID (16 bits, from AGENT_ID parameter)

Bits [71:64] - Unit ID (8 bits, from UNIT_ID parameter)

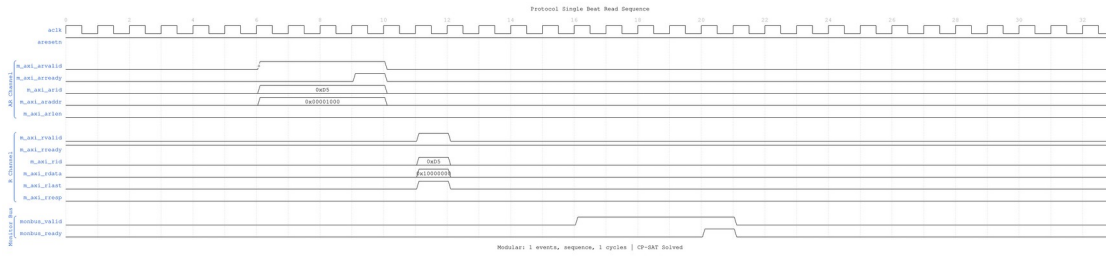
Bits [63:0] - Event Data (64 bits – full address, latency, etc.)

12.8 Timing Diagrams

The following waveforms show AXI4 master read monitor behavior across various scenarios:

12.8.1 Scenario 1: Single-Beat Read Transaction

Complete AXI4 read transaction with AR handshake and R response:



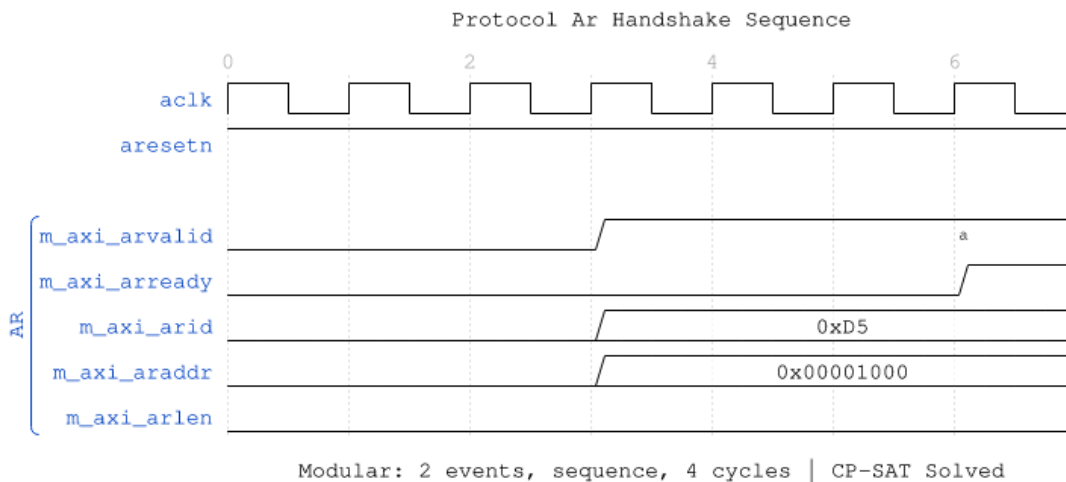
Single Beat Read

WaveJSON: [single_beat_read_001.json](#)

Key Observations: - AR channel handshake (ARVALID/ARREADY) - R channel response (RVALID/RREADY/RLAST) - Monitor bus packet generation - Transaction tracking and completion

12.8.2 Scenario 2: AR Channel Handshake Detail

Detailed view of address read channel handshake:



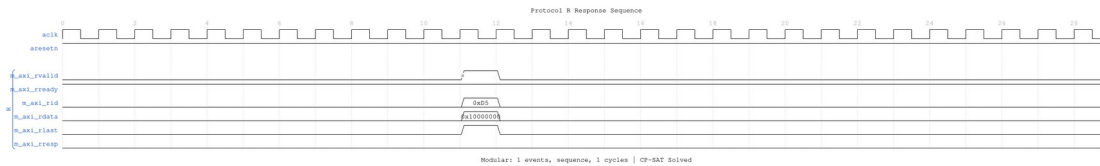
AR Handshake

WaveJSON: [ar_handshake_001.json](#)

Key Observations: - ARVALID assertion timing - ARREADY response from slave - Address and control signal stability - Handshake protocol compliance

12.8.3 Scenario 3: R Response Channel Detail

Read data response channel behavior:



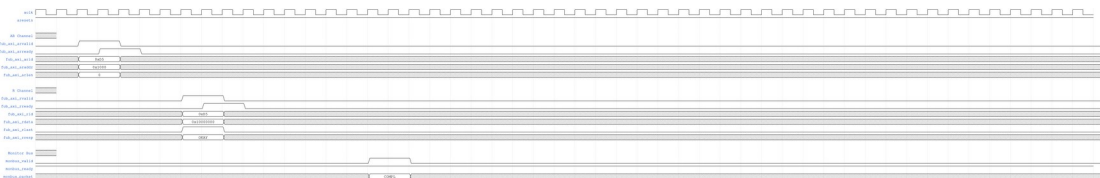
R Response

WaveJSON: [r_response_001.json](#)

Key Observations: - RVALID/RREADY handshake - RLAST assertion for transaction completion - RRESP status (OKAY/SLVERR/DECERR) - Read data capture

12.8.4 Scenario 4: Complete AXI4 Read Transaction

End-to-end AXI4 read transaction flow:



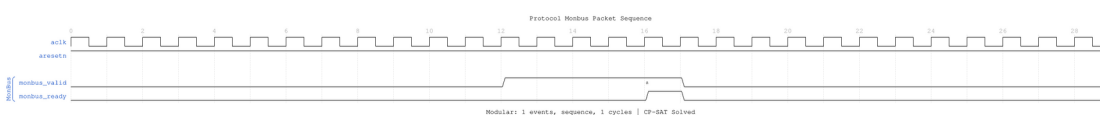
AXI4 Read Transaction

WaveJSON: [axi4_read_transaction_001.json](#)

Key Observations: - Complete AR → R channel flow - Transaction ID tracking - Burst length and size handling - Monitor packet correlation

12.8.5 Scenario 5: Monitor Bus Packet Generation

Monitor bus output packet format and timing:



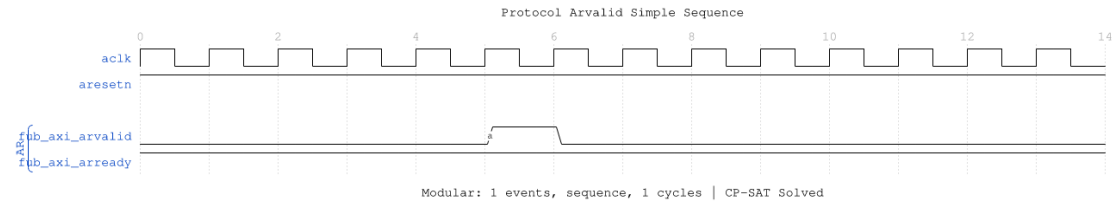
MonBus Packet

WaveJSON: [monbus_packet_001.json](#)

Key Observations: - Monitor bus valid/ready handshake - 64-bit packet format - Packet type encoding - Event data payload

12.8.6 Scenario 6: Simple ARVALID Transaction

Simplified view of ARVALID-based transaction:



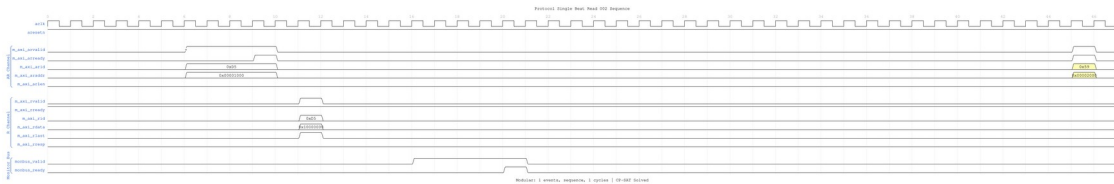
ARVALID Simple

WaveJSON: [arvalid_simple_001.json](#)

Key Observations: - Minimal handshake sequence - Single-beat read operation - Fast transaction completion

12.8.7 Scenario 7: Alternative Single-Beat Read

Variant single-beat read with different timing:



Single Beat Read Alt

WaveJSON: [single_beat_read_002_001.json](#)

Key Observations: - Different backpressure pattern - Ready signal de-assertion effects - Transaction latency variation

12.9 Configuration Strategies

12.9.1 Strategy 1: Functional Verification (Recommended)

Goal: Catch protocol errors and track completion

// Enable configuration

```
.cfg_monitor_enable    (1'b1),
.cfg_error_enable      (1'b1),           // Errors
.cfg_timeout_enable    (1'b1),           // Timeouts
.cfg_perf_enable       (1'b0),           // Disable (reduces traffic)
```

// Filtering - pass error and timeout, drop others

```
.cfg_axi_pkt_mask      (16'b1111_1111_0000_0011), // Pass ERROR,
TIMEOUT only
```

```
.cfg_axi_err_select      (16'h0000),
.cfg_axi_error_mask      (16'h0000), // Pass all errors
.cfg_axi_timeout_mask    (16'h0000), // Pass all timeouts
.cfg_axi_compl_mask      (16'hFFFF), // Drop completions
.cfg_axi_perf_mask       (16'hFFFF), // Drop performance
.cfg_axi_debug_mask      (16'hFFFF), // Drop debug
```

// Timeouts

```
.cfg_timeout_cycles      (16'd1000),
.cfg_latency_threshold    (32'd500)
```

12.9.2 Strategy 2: Performance Analysis

Goal: Collect performance metrics, suppress completions

// Enable configuration

```
.cfg_monitor_enable      (1'b1),
.cfg_error_enable        (1'b1), // Still catch errors
.cfg_timeout_enable      (1'b0), // Disable timeouts
.cfg_perf_enable         (1'b1), // Enable performance
```

// Filtering - pass error and performance only

```
.cfg_axi_pkt_mask        (16'b1111_1110_0000_0001), // Pass ERROR,
PERF only
```

```
.cfg_axi_err_select      (16'h0000),
.cfg_axi_error_mask      (16'h0000), // Pass all errors
.cfg_axi_perf_mask       (16'h0000), // Pass all performance
.cfg_axi_compl_mask      (16'hFFFF), // Drop completions
.cfg_axi_timeout_mask    (16'hFFFF), // Drop timeouts
.cfg_axi_debug_mask      (16'hFFFF), // Drop debug
```

12.9.3 Strategy 3: Debug Mode

Goal: Maximum visibility, expect low traffic

// Enable everything

```
.cfg_monitor_enable      (1'b1),
.cfg_error_enable        (1'b1),
.cfg_timeout_enable      (1'b1),
.cfg_perf_enable         (1'b1),
```

// Filtering - pass all packets

```
.cfg_axi_pkt_mask        (16'h0000), // Pass all packet types
.cfg_axi_err_select      (16'h0000),
.cfg_axi_error_mask      (16'h0000),
.cfg_axi_timeout_mask    (16'h0000),
.cfg_axi_compl_mask      (16'h0000), // Pass completions
.cfg_axi_thresh_mask     (16'h0000),
.cfg_axi_perf_mask       (16'h0000),
```

```
.cfg_axi_addr_mask      (16'h0000),
.cfg_axi_debug_mask     (16'h0000)
```

⚠ WARNING: Never enable all packet types in high-throughput scenarios - monitor bus will saturate!

12.10 Usage Example

12.10.1 Basic Integration

// Instantiate AXI4 master read monitor

```
axi4_master_rd_mon #(
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R       (4),
    .AXI_ID_WIDTH       (4),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),
    .AXI_USER_WIDTH     (1),
    .UNIT_ID            (1),
    .AGENT_ID           (10),
    .MAX_TRANSACTIONS   (16),
    .ENABLE_FILTERING   (1)
) u_master_rd_mon (
    .aclk                (axi_aclk),
    .aresetn             (axi_aresetn),
```

// Frontend interface

```
.fub_axi_arid          (read_arid),
.fub_axi_araddr        (read_araddr),
.fub_axi_arlen         (read_arlen),
.fub_axi_arsize        (read_arsize),
.fub_axi_arburst       (read_arburst),
.fub_axi_arlock        (1'b0),
.fub_axi_arcache       (4'b0010),
.fub_axi_arprot        (3'b000),
.fub_axi_arqos         (4'b0000),
.fub_axi_arregion      (4'b0000),
.fub_axi_aruser        (1'b0),
.fub_axi_arvalid       (read_arvalid),
.fub_axi_arready       (read_arready),

.fub_axi_rid           (read_rid),
.fub_axi_rdata         (read_rdata),
.fub_axi_rresp         (read_rresp),
.fub_axi_rlast         (read_rlast),
.fub_axi_ruser         (read_ruser),
```

```

.fub_axi_rvalid      (read_rvalid),
.fub_axi_rready      (read_rready),

// Master interface (to interconnect)
.m_axi_arid          (m_axi_arid),
.m_axi_araddr        (m_axi_araddr),
.m_axi_arlen         (m_axi_arlen),
.m_axi_arsize        (m_axi_arsize),
.m_axi_arburst       (m_axi_arburst),
.m_axi_arlock        (m_axi_arlock),
.m_axi_arcache       (m_axi_arcache),
.m_axi_arprot        (m_axi_arprot),
.m_axi_arqos         (m_axi_arqos),
.m_axi_arregion      (m_axi_arregion),
.m_axi_aruser        (m_axi_aruser),
.m_axi_arvalid       (m_axi_arvalid),
.m_axi_arready       (m_axi_arready),

.m_axi_rid           (m_axi_rid),
.m_axi_rdata         (m_axi_rdata),
.m_axi_rresp         (m_axi_rresp),
.m_axi_rlast         (m_axi_rlast),
.m_axi_ruser         (m_axi_ruser),
.m_axi_rvalid        (m_axi_rvalid),
.m_axi_rready        (m_axi_rready),

// Monitor configuration (Strategy 1 - Functional)
.cfg_monitor_enable  (1'b1),
.cfg_error_enable    (1'b1),
.cfg_timeout_enable  (1'b1),
.cfg_perf_enable     (1'b0),
.cfg_timeout_cycles  (16'd1000),
.cfg_latency_threshold (32'd500),

.cfg_axi_pkt_mask    (16'b1111_1111_0000_0011),
.cfg_axi_err_select  (16'h0000),
.cfg_axi_error_mask  (16'h0000),
.cfg_axi_timeout_mask (16'h0000),
.cfg_axi_compl_mask  (16'hFFFF),
.cfg_axi_thresh_mask (16'hFFFF),
.cfg_axi_perf_mask   (16'hFFFF),
.cfg_axi_addr_mask   (16'hFFFF),
.cfg_axi_debug_mask  (16'hFFFF),

// Monitor bus output
.monbus_valid        (mon_valid),
.monbus_ready        (mon_ready),

```

```

        .monbus_packet          (mon_packet),

// Status
        .busy                   (rd_busy),
        .active_transactions    (rd_active),
        .error_count            (rd_errors),
        .transaction_count      (rd_count),
        .cfg_conflict_error     (cfg_conflict)
);

// Downstream FIFO for monitor packets
gaxi_fifo_sync #(
    .DATA_WIDTH(64),
    .DEPTH(256)
) u_mon_fifo (
    .i_clk          (axi_aclk),
    .i_rst_n        (axi_aresetn),
    .i_valid         (mon_valid),
    .i_data          (mon_packet),
    .o_ready         (mon_ready),
    .o_valid         (fifo_valid),
    .o_data          (fifo_data),
    .i_ready         (consumer_ready)
);

```

12.11 Design Notes

12.11.1 Filtering Hierarchy

The 3-level filtering provides flexible packet control:

1. **Level 1 (cfg_axi_pkt_mask):** Coarse filtering by packet type
 - Bit per packet type (ERROR, TIMEOUT, COMPL, etc.)
 - 1 = drop, 0 = pass to next level
2. **Level 2 (cfg_axi_err_select):** Future error routing
 - Reserved for directing errors to alternate paths
3. ****Level 3 (cfg_axi_*_mask):**** Fine-grained event filtering
 - Individual event masks within each packet type
 - Allows selecting specific events while dropping others

12.11.2 Configuration Conflicts

The module detects conflicting configurations: - Monitoring disabled but packet types enabled - Timeout enabled with zero timeout_cycles - Filter masks incompatible with enable signals

When detected, `cfg_conflict_error` asserts.

12.11.3 Performance Considerations

Monitor Bus Bandwidth: - Each transaction can generate multiple packets - Completion packets: 1 per transaction - Performance packets: 1 per transaction - Error packets: Variable (0-N per transaction)

Recommended Packet Budget: - Functional mode: 2-3 packets per transaction (ERROR + TIMEOUT) - Performance mode: 2 packets per transaction (ERROR + PERF) - Debug mode: 5-10 packets per transaction (all types)

12.11.4 Transaction Table

Monitors up to MAX_TRANSACTIONS concurrent transactions: - Tracks ARID, address, latency, status - Generates packets on completion, timeout, or error - Proper cleanup via event_reported feedback (fixed in v1.1)

12.12 Related Modules

12.12.1 Companion Monitors

- **axi4_master_wr_mon** - AXI4 master write with monitoring
- **axi4_slave_rd_mon** - AXI4 slave read with monitoring
- **axi4_slave_wr_mon** - AXI4 slave write with monitoring

12.12.2 Base Modules

- **axi4_master_rd** - Functional AXI4 master read (without monitoring)
- **axi_monitor_filtered** - Monitoring engine with filtering (shared/)

12.12.3 Used Components

- **gaxi_skid_buffer** - Elastic buffering
 - **axi_monitor_base** - Core monitoring logic (shared/)
 - **axi_monitor_trans_mgr** - Transaction tracking (shared/)
-

12.13 References

12.13.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)
- Monitor Bus Packet Format: [monitor_package_spec.md](#)

12.13.2 Source Code

- RTL: rtl/amba/monitor/axi4_master_rd_mon.sv
- Tests: val/amba/test_axi4_master_rd_mon.py
- Framework: bin/TBClasses/components/axi4/

12.13.3 Documentation

- Configuration Guide: [AXI Monitor Base](#)
 - Architecture: [RTLAmba Overview](#)
 - AXI4 Index: [axi4/README.md](#)
-

Last Updated: 2026-07-18

12.14 Navigation

- [← Back to AXI4 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

13 AXI4 Master Write Monitor (Clock-Gated)

Module: axi4_master_wr_mon_cg.sv **Base Module:** [axi4_master_wr_mon](#) **Location:** rtl/amba/axi4/ **Status:** ✓ Production Ready

13.1 Quick Reference

This is the **clock-gated variant** of [axi4_master_wr_mon](#).

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

13.2 Summary

The `axi4_master_wr_mon_cg` module adds power optimization to `axi4_master_wr_mon` through activity-based clock gating:

- ✓ **Same Functionality:** 100% equivalent to base module
 - ✓ **Power Savings:** 25-70% depending on traffic utilization
 - ✓ **Configurable:** Idle threshold, gating domains, enable/disable
 - ✓ **Zero Overhead When Disabled:** `ENABLE_CLOCK_GATING=0` → identical to base
-

13.3 Common Parameters

In addition to all [axi4_master_wr_mon](#) parameters (including `USE_MONITOR`):

Parameter	Default	Description
<code>ENABLE_CLOCK_GATING</code>	1	Master enable (0=disable, identical to base)
<code>CG_IDLE_CYCLES</code>	8	Cycles before clock gating activates
<code>CG_GATE_*</code>	1	Domain-specific gating enables
<code>USE_MONITOR</code>	1	Synthesis-time monitor enable (forwarded to inner monitor).
<code>N_ADDR_RANGES</code>	0	Number of address-range comparators (forwarded to base module).

All base-module ports are forwarded unchanged, including the `cam_clear` control input (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. `CTRL[4]`) - and the full performance-monitoring interface (see [Performance Monitoring](#) below). The six `ENABLE_*_LOGIC` synthesis-cone parameters and the `cfg_compl_enable` / `cfg_threshold_enable` / `cfg_debug_enable` control enables are also passed straight through.

13.4 Performance Monitoring

The clock-gated wrapper exposes the full perfmon interface of the base module and **forwards every port unchanged** to the inner `axi4_master_wr_mon`. The measurement-window state machine, the four W-channel utilization buckets (productive / back-pressure / starvation / idle), and the beat/byte/burst throughput counters behave exactly as documented in the base module — see [Performance Monitoring in axi4_master_wr_mon](#) for the full narrative and per-bit semantics.

Forwarded perfmon ports (identical width and direction to the base module):

- **Inputs:** `cfg_perf_enable`, `cfg_start_event_sel` (3), `cfg_end_event_sel` (3), `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Outputs:** `window_active`, `window_cycles` (32), `perf_prod_cycles` (32), `perf_bp_cycles` (32), `perf_starv_cycles` (32), `perf_idle_cycles` (32), `perf_beat_count` (32), `perf_byte_count` (64), `perf_burst_count` (32)

Clock gating never suppresses these paths: while a measurement window is open the monitor stays awake, so window cycle accounting remains exact regardless of `CG_IDLE_CYCLES`.

13.5 Quick Usage

```
axi4_master_wr_mon_cg #(
    // Base module parameters (see axi4_master_wr_mon.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating (see CG guide for details)
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... all other ports same as axi4_master_wr_mon
);
```

13.6 Documentation

- **Base Module Functionality:** [axi4_master_wr_mon.md](#)

- **Clock Gating Guide:** [clock_gated_variants.md](#)
 - **Detailed CG Examples:**
 - [axi4_master_rd_mon_cg.md](#) (AXI4 monitor)
 - [axil4_master_rd_mon_cg.md](#) (AXIL4 monitor)
 - [apb_slave_cg.md](#) (APB interface)
-

13.7 Navigation

- [← Back to AXI4 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

14 AXI4 Master Write Monitor

Module: `axi4_master_wr_mon.sv` **Location:** `rtl/amba/axi4/` **Status:** ✓ Production Ready

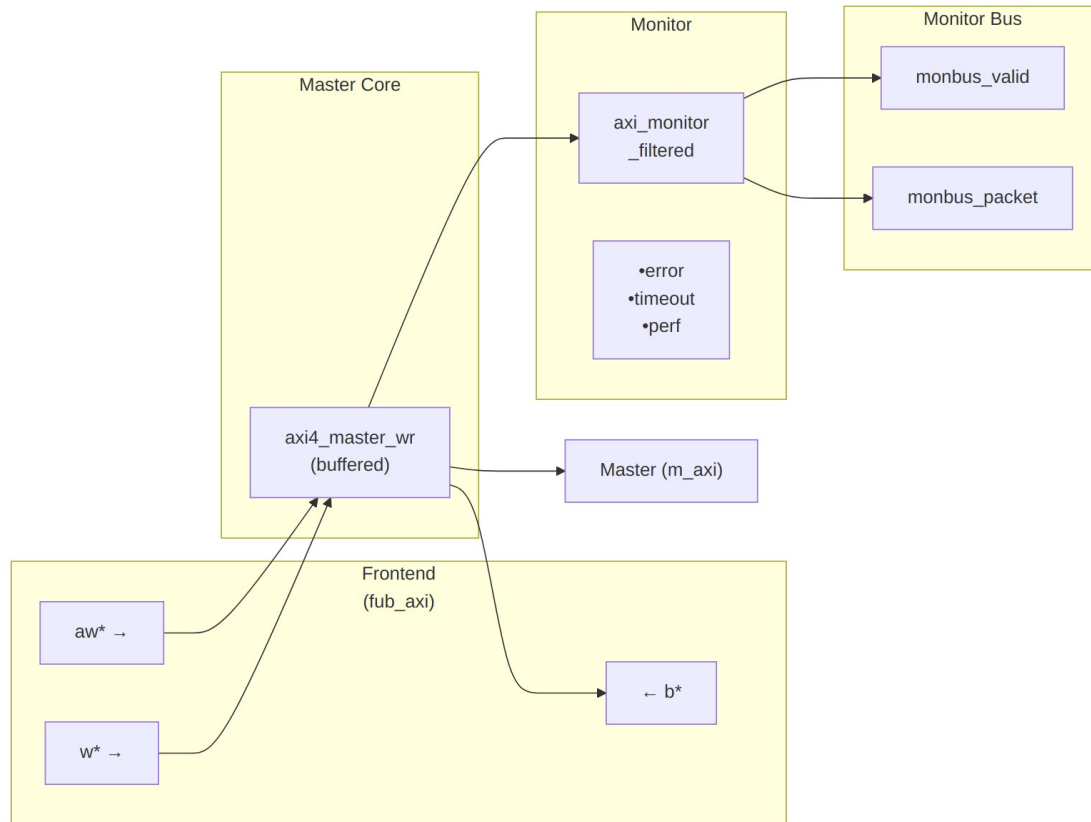
14.1 Overview

The AXI4 Master Write Monitor module combines a functional AXI4 master write interface with comprehensive transaction monitoring and filtering capabilities. This module is essential for verification environments, providing real-time protocol checking, error detection, performance metrics, and configurable packet filtering for write transactions.

14.1.1 Key Features

- ✓ **Integrated Monitoring:** Combines `axi4_master_wr` with `axi_monitor_filtered`
- ✓ **3-Level Filtering:** Packet type masks, error routing, individual event masking
- ✓ **Error Detection:** Protocol violations, SLVERR, DECERR, orphan transactions
- ✓ **Timeout Monitoring:** Configurable timeout detection for stuck transactions
- ✓ **Performance Metrics:** Latency tracking, transaction counting, throughput analysis
- ✓ **Monitor Bus Output:** 128-bit packets paired with 64-bit side-band timestamps

- ✓ **Configuration Validation:** Detects conflicting configuration settings
- ✓ **Clock Gating Support:** Busy signal for power management



Module Architecture

The module instantiates two sub-modules: 1. **axi4_master_wr** - Core AXI4 write functionality with buffering 2. **axi_monitor_filtered** - Transaction monitoring with 3-level filtering

14.2 Parameters

14.2.1 AXI4 Master Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel skid buffer depth
SKID_DEPTH_W	int	4	W channel

Parameter	Type	Default	Description
			skid buffer depth
SKID_DEPTH_B	int	2	B channel skid buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width

14.2.2 Monitor Parameters

Parameter	Type	Default	Description
UNIT_ID	int	1	4-bit unit identifier in monitor packets
AGENT_ID	int	11	8-bit agent identifier in monitor packets
MAX_TRANSACTIONS	int	16	Maximum concurrent outstanding transactions
ENABLE_FILTERING	bit	1	Enable packet filtering (0=pass all packets)
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
N_ADDR_RANGES	int	0	Number of address-range comparators. 0 = checker omitted (zero

Parameter	Type	Default	Description
			area). >0 = N independent [low, high] ranges; hits emit PktTypeError + AXI_ERR_ADDR_RANGE on monbus.

14.2.3 Synthesis-Cone Parameters

Each detection cone can be compiled out to save area. The classic cones default on; the debug cone defaults off. Setting a bit to 0 drops the corresponding cone at synthesis via a generate-if inside `axi_monitor_reporter` — the area saving is real.

Parameter	Type	Default	Effect when 0
ENABLE_ERROR_LOGIC	bit	1	Drop the error-detection cone
ENABLE_TIMEOUT_LOGIC	bit	1	Drop the timeout cone and the <code>axi_monitor_timeout</code> instance
ENABLE_COMPL_LOGIC	bit	1	Drop the completion cone
ENABLE_THRESHOLD_LOGIC	bit	1	Drop the threshold cone
ENABLE_PERF_LOGIC	bit	1	Drop the perfmon window + counters
ENABLE_DEBUG_LOGIC	bit	0	Drop the debug (trace) cone — the 6th reporter sub-block (off by default)

Internally the wrapper hardwires two master switches on its `axi_monitor_filtered` instance: `ENABLE_PERF_PACKETS = 1` (perf datapath present, so `ENABLE_PERF_LOGIC` alone gates the window/counters) and `ENABLE_DEBUG_MODULE = 0` (debug tracking module omitted). These are not top-level parameters of the wrapper.

The transaction CAM is always pipelined.

14.3 Monitor Backpressure (block_ready)

The monitor exposes a `block_ready` signal that goes low when its internal FIFO is saturated and cannot accept a new in-flight transaction. The wrapper ANDs `block_ready` into the upstream-facing `fub_axi_awready` so a saturated monitor stalls new transactions on the wire instead of dropping events.

- **Where the stall lands:** the upstream `fub_axi_awready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`:** `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.

This replaces a previous bug where `block_ready` was left unconnected and a full monitor FIFO would silently lose events.

14.4 Address-Range Checker

The wrapper can be parameterized with `N_ADDR_RANGES > 0` to instantiate an N-comparator address-range checker that watches every accepted AW handshake and emits a `PktTypeError` monbus packet (event code `AXI_ERR_ADDR_RANGE = 8'h0D`) when an address falls inside any of the configured `[low, high]` inclusive ranges.

Config inputs (active only when `N_ADDR_RANGES > 0`): - `cfg_addr_check_enable` — master on/off for the checker. - `cfg_addr_range_enable[N-1:0]` — per-range enable bit. - `cfg_addr_range_low[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive low bound for each range. - `cfg_addr_range_high[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive high bound for each range.

Event encoding (within the standard 128-bit `monitor_packet_t`, `event_data` field): - `packet_type = PktTypeError (4'h0)` - `protocol = AXI (3'b000)` - `event_code = AXI_ERR_ADDR_RANGE (8'h0D)` - `event_data[63:60] = range_index` (4 bits; supports up to 16 ranges) - `event_data[59:0]` = full matched address (up to 60 bits, zero-padded if narrower)

Exact match: set `cfg_addr_range_low[i] == cfg_addr_range_high[i]`.

Filtering: the existing `cfg_axi_error_mask[13]` bit masks this event code; set it high to suppress range packets without disabling other errors. No new mask wiring needed.

Per-range coalescing: if a range hits again before its packet has been emitted, the latched address is overwritten (latest hit wins). One packet per cycle drains the pending mask via a lowest-index priority encoder. Distinct ranges never lose events; under sustained per-range bursts, only the latest address per range is reported per emission cycle.

14.5 Performance Monitoring

The wrapper hardwires `ENABLE_PERF_PACKETS = 1` on its inner `axi_monitor_filtered`, so the perfmon datapath is present whenever `ENABLE_PERF_LOGIC = 1` (the default). It instantiates a **measurement-window state machine** plus a bank of W-data-channel utilization counters. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows, so the host can read a completed window's totals.

14.5.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**. The event sources are selected by `cfg_start_event_sel / cfg_end_event_sel` (3-bit; e.g. `3'b010` selects the `cfg_perf_enable` edge) and can also be fired directly by the `cfg_start_trigger / cfg_end_trigger` pulses (from an engine or CSR). `cfg_window_force_close` is a software override that closes the window immediately. While the window is open:

- `window_active` is high.
- `window_cycles[31:0]` free-runs, counting every clock elapsed inside the window.

Sample the counters on the cycle `window_active` falls to 0 (drive `cfg_end_trigger`, or wait for the configured end event).

14.5.2 Utilization Counters (W Data Channel)

Every cycle inside the window is classified by the W channel's `wvalid / wready` into exactly one of four buckets:

Output	Width	Condition	Meaning
<code>perf_prod_cycles</code>	32	<code>wvalid && wready</code>	productive beat transferred
<code>perf_bp_cycles</code>	32	<code>wvalid && !wready</code>	back-pressure (data offered, sink not ready)
<code>perf_starv_cycles</code>	32	<code>!wvalid && wready</code>	starvation (sink ready, no data)
<code>perf_idle_cycles</code>	32	<code>!wvalid && !wready</code>	idle

The four buckets sum to `window_cycles`, so `utilization = perf_prod_cycles / window_cycles`.

14.5.3 Throughput Counters

Output	Width	Meaning
perf_beat_count	32	W data beats transferred (= perf_prod_cycles, 1 beat/cycle)
perf_byte_count	64	bytes transferred = beats × (1 << AWSIZE), using the AWSIZE captured at the most recent AW address phase
perf_burst_count	32	AW address-phase handshakes

The integrator computes average burst length as $\text{perf_beat_count} / \text{perf_burst_count}$.

14.5.4 Performance Monitoring Ports

Port	Direction	Width	Description
cfg_perf_enable	Input	1	Enable performance-metric packet generation
cfg_start_event_sel	Input	3	Window start event source select
cfg_end_event_sel	Input	3	Window end event source select
cfg_start_trigger	Input	1	Pulse: open the measurement window
cfg_end_trigger	Input	1	Pulse: close the measurement window
cfg_window_force_close	Input	1	Software override: force the window closed
window_active	Output	1	High while a measurement window is open
window_cycles	Output	32	Cycles elapsed in the current window
perf_prod	Output	32	wvalid && wready cycles

Port	Direction	Width	Description
<code>_cycles</code>			
<code>perf_bp_cycles</code>	Output	32	wvalid && !wready cycles (back-pressure)
<code>perf_starv_cycles</code>	Output	32	!wvalid && wready cycles (starvation)
<code>perf_idle_cycles</code>	Output	32	!wvalid && !wready cycles
<code>perf_beat_count</code>	Output	32	W data beats transferred
<code>perf_byte_count</code>	Output	64	bytes transferred
<code>perf_burst_count</code>	Output	32	AW address-phase handshakes

When `USE_MONITOR = 0`, every perfmon output is tied to 0 and the window never opens.

14.6 Port Groups

14.6.1 AXI4 Write Channels

****Frontend Interface (fub_axi_*):**** - AW channel: `awid`, `awaddr`, `awlen`, `awsize`, `awburst`, `awlock`, `awcache`, `awprot`, `awqos`, `awregion`, `awuser`, `awvalid`, `awready` - W channel: `wdata`, `wstrb`, `wlast`, `wuser`, `wvalid`, `wready` - B channel: `bid`, `bresp`, `buser`, `bvalid`, `bready`

****Master Interface (m_axi_*):**** - Same signals as frontend, mirrored direction

14.6.2 Monitor Configuration

Configuration ports are identical to [axi4_master_rd_mon](#): - Basic enables: `cfg_monitor_enable`, `cfg_error_enable`, `cfg_timeout_enable`, `cfg_perf_enable`, `cfg_compl_enable` (completion packets), `cfg_threshold_enable` (threshold-crossed packets), `cfg_debug_enable` (debug/trace cone — the 6th reporter sub-block) - Clear: `cam_clear` (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]) - Thresholds: `cfg_timeout_cycles`, `cfg_latency_threshold` - Filtering: 7 mask signals (`cfg_axi_*_mask`) - Performance window control: `cfg_start_event_sel`, `cfg_end_event_sel`, `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close` (see [Performance Monitoring](#))

The inner monitor's `cfg_debug_level` (tied to 0), `cfg_debug_mask` (0) and `cfg_active_trans_threshold` (8) are fixed inside the wrapper and are **not** top-level ports on this module.

14.6.3 Monitor Bus Output

Port	Direction	Width	Description
<code>monbus_valid</code>	Output	1	Monitor packet valid
<code>monbus_ready</code>	Input	1	Downstream ready to accept packet
<code>monbus_packet</code>	Output	128	<code>monitor_packet_t</code> (see format below)
<code>monbus_timestamp</code>	Output	64	<code>monbus_timestamp_t</code> paired atomically with <code>monbus_packet</code>
<code>i_mon_time</code>	Input	64	Free-running counter from <code>monbus_axil_group</code> , sampled at packet emission

14.6.4 Status Outputs

Port	Direction	Width	Description
<code>busy</code>	Output	1	Indicates active transactions (for clock gating)
<code>active_transactions</code>	Output	8	Current number of outstanding transactions
<code>error_count</code>	Output	16	Cumulative error count
<code>transaction_count</code>	Output	32	Total transaction count
<code>cfg_conflict_error</code>	Output	1	Configuration conflict detected

14.7 Monitor Packet Format

The 128-bit monbus_packet (paired with the 64-bit monbus_timestamp side-band signal) follows the standardized AMBA monitor bus format. Identical format as [axi4_master_rd_mon](#):

Bits [127:124] - Packet Type:

0x0 = ERROR	Error events (SLVERR, DECERR, protocol violations)
0x1 = COMPL	Completion events (transaction finished)
0x2 = THRESH	Threshold events
0x3 = TIMEOUT	Timeout events
0x4 = PERF	Performance metrics
0x8 = ADDR_MATCH	Address match events
0x9 = APB	APB-specific events
0xF = DEBUG	Debug events

Bits [123:109] - Reserved (15 bits, forward-compat slack)

Bits [108:105] - Protocol (4 bits): 0x0=AXI, 0x1=AXIS, 0x2=APB, 0x3=ARB, 0x4=CORE

Bits [104:97] - Event Code (8 bits, protocol-specific)

Bits [96:88] - Channel ID (9 bits – AXI ID or channel index)

Bits [87:72] - Agent ID (16 bits, from AGENT_ID parameter)

Bits [71:64] - Unit ID (8 bits, from UNIT_ID parameter)

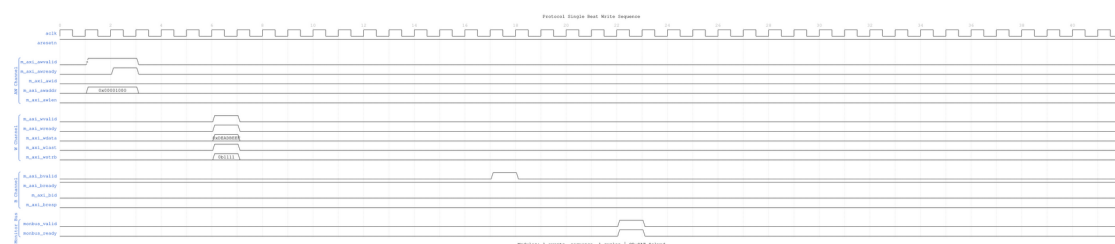
Bits [63:0] - Event Data (64 bits – full address, latency, etc.)

14.8 Timing Diagrams

The following waveforms show AXI4 master write monitor behavior:

14.8.1 Scenario 1: Single-Beat Write Transaction

Complete AXI4 write transaction showing AW, W, and B channels:



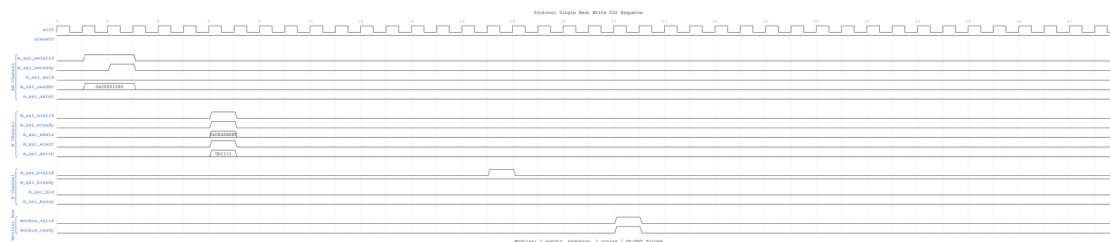
Single Beat Write

WaveJSON: [single_beat_write_001.json](#)

Key Observations: - AW channel handshake (AWVALID/AWREADY) - W channel data (WVALID/WREADY/WLAST/WSTRB) - B channel response (BVALID/BREADY/BRESP) - Monitor bus packet generation - Three-channel write protocol coordination

14.8.2 Scenario 2: Alternative Single-Beat Write

Variant single-beat write with different backpressure pattern:



Single Beat Write Alt

WaveJSON: [single_beat_write_002_001.json](#)

Key Observations: - Different ready signal timing - Channel interleaving effects - Write strobe (WSTRB) handling - Response latency variation - Monitor packet correlation with transaction ID

14.9 Configuration Strategies

14.9.1 Strategy 1: Functional Verification (Recommended)

Goal: Catch write errors and track completion

```
// Enable configuration
.cfg_monitor_enable      (1'b1),
.cfg_error_enable        (1'b1),          // Catch SLVERR/DECERR
.cfg_timeout_enable      (1'b1),          // Detect stuck writes
.cfg_perf_enable         (1'b0),          // Disable (reduces traffic)

// Filtering - pass error and timeout only
.cfg_axi_pkt_mask        (16'b1111_1111_0000_0011),
.cfg_axi_error_mask      (16'h0000),      // Pass all errors
.cfg_axi_timeout_mask    (16'h0000),      // Pass all timeouts
.cfg_axi_compl_mask      (16'hFFFF),      // Drop completions
.cfg_axi_perf_mask       (16'hFFFF),      // Drop performance

// Timeouts
.cfg_timeout_cycles       (16'd1000),
.cfg_latency_threshold    (32'd500)
```

14.9.2 Strategy 2: Performance Analysis

Goal: Collect write performance metrics

```
// Enable configuration
.cfg_monitor_enable      (1'b1),
.cfg_error_enable        (1'b1),      // Still catch errors
.cfg_timeout_enable      (1'b0),      // Disable timeouts
.cfg_perf_enable         (1'b1),      // Enable performance

// Filtering - pass error and performance only
.cfg_axi_pkt_mask        (16'b1111_1110_0000_0001),
.cfg_axi_error_mask      (16'h0000),  // Pass all errors
.cfg_axi_perf_mask       (16'h0000),  // Pass all performance
.cfg_axi_compl_mask      (16'hFFFF),  // Drop completions
.cfg_axi_timeout_mask    (16'hFFFF)  // Drop timeouts
```

14.9.3 Strategy 3: Debug Mode

Goal: Maximum visibility for write transactions

```
// Enable everything
.cfg_monitor_enable      (1'b1),
.cfg_error_enable        (1'b1),
.cfg_timeout_enable      (1'b1),
.cfg_perf_enable         (1'b1),

// Filtering - pass all packets
.cfg_axi_pkt_mask        (16'h0000),  // Pass all
// All individual masks set to 16'h0000
```

⚠ WARNING: Never enable all packet types in high-throughput write scenarios!

14.10 Usage Example

14.10.1 Basic Integration

```
axi4_master_wr_mon #(
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W       (4),
    .SKID_DEPTH_B       (2),
    .AXI_ID_WIDTH       (4),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),
    .AXI_USER_WIDTH     (1),
    .UNIT_ID            (1),
    .AGENT_ID           (11),
    .MAX_TRANSACTIONS   (16),
    .ENABLE_FILTERING   (1)
) u_master_wr_mon (
```

```

.aclk                (axi_aclk),
.aresetn             (axi_aresetn),

// Frontend interface (from write initiator)
.fub_axi_awid        (write_awid),
.fub_axi_awaddr      (write_awaddr),
// ... rest of AW/W/B signals

// Master interface (to interconnect)
.m_axi_awid          (m_axi_awid),
.m_axi_awaddr        (m_axi_awaddr),
// ... rest of AW/W/B signals

// Monitor configuration (Strategy 1 - Functional)
.cfg_monitor_enable   (1'b1),
.cfg_error_enable     (1'b1),
.cfg_timeout_enable   (1'b1),
.cfg_perf_enable      (1'b0),
.cfg_timeout_cycles   (16'd1000),
.cfg_latency_threshold (32'd500),

.cfg_axi_pkt_mask     (16'b1111_1111_0000_0011),
.cfg_axi_error_mask   (16'h0000),
.cfg_axi_timeout_mask (16'h0000),
.cfg_axi_compl_mask   (16'hFFFF),
// ... rest of mask signals

// Monitor bus output
.monbus_valid         (mon_valid),
.monbus_ready         (mon_ready),
.monbus_packet        (mon_packet),

// Status
.busy                 (wr_busy),
.active_transactions  (wr_active),
.error_count          (wr_errors),
.transaction_count    (wr_count),
.cfg_conflict_error   (cfg_conflict)
);

```

14.11 Design Notes

14.11.1 Write-Specific Monitoring

Write Response Tracking: - Monitors AW channel for write address capture - Tracks W channel beats (WLAST detection) - Correlates B channel responses with AWID - Detects mismatched burst lengths (W beats vs AWLEN+1)

Common Write Errors Detected: - **SLVERR:** Slave error response (decode failure, access violation) - **DECERR:** Decode error (address not mapped) - **Orphan transactions:** AWID without matching BID - **Timeout:** Write address or response stuck beyond threshold - **Protocol violations:** Invalid WSTRB, missing WLAST

14.11.2 Performance Considerations

Write Transaction Bandwidth: - AW channel: 1 transaction/cycle (when buffer not full) - W channel: 1 beat/cycle sustained (burst pipelining) - B channel: 1 response/cycle (sparse compared to W beats)

Monitor Packet Budget (Write): - Typical: 2-3 packets per write transaction - Burst writes: More efficient packet/beat ratio - Single writes: Higher packet overhead

14.11.3 Buffer Depth Guidelines

Same as [axi4_master_wr](#): - **SKID_DEPTH_AW:** 2 (default) - sufficient for most systems - **SKID_DEPTH_W:** 4 (default) - accommodates moderate bursts - **SKID_DEPTH_B:** 2 (default) - responses are single-beat

Increase depths for high-latency or high-throughput scenarios.

14.12 Related Modules

14.12.1 Companion Monitors

- [axi4_master_rd_mon](#) - AXI4 master read with monitoring
- [axi4_slave_rd_mon](#) - AXI4 slave read with monitoring
- [axi4_slave_wr_mon](#) - AXI4 slave write with monitoring

14.12.2 Base Modules

- [axi4_master_wr](#) - Functional AXI4 master write (without monitoring)
- [axi_monitor_filtered](#) - Monitoring engine with filtering (shared/)

14.12.3 Used Components

- [gaxi_skid_buffer](#) - Elastic buffering
 - [axi_monitor_base](#) - Core monitoring logic (shared/)
 - [axi_monitor_trans_mgr](#) - Transaction tracking (shared/)
-

14.13 References

14.13.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)
- Monitor Bus Packet Format: [monitor_package_spec.md](#)

14.13.2 Source Code

- RTL: `rtl/amba/monitor/axi4_master_wr_mon.sv`
- Tests: `val/amba/test_axi4_master_wr_mon.py`
- Framework: `bin/TBClasses/components/axi4/`

14.13.3 Documentation

- Configuration Guide: [AXI Monitor Base](#)
 - Architecture: [RTLAmba Overview](#)
 - AXI4 Index: [README.md](#)
-

Last Updated: 2026-07-18

14.14 Navigation

- [← Back to AXI4 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

15 AXI4 Slave Read Monitor (Clock-Gated)

Module: `axi4_slave_rd_mon_cg.sv` **Base Module:** [axi4_slave_rd_mon](#) **Location:** `rtl/amba/axi4/` **Status:** ✓ Production Ready

15.1 Quick Reference

This is the **clock-gated variant** of [axi4_slave_rd_mon](#).

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ [Clock-Gated Variants Guide](#)

15.2 Summary

The `axi4_slave_rd_mon_cg` module adds power optimization to `axi4_slave_rd_mon` through activity-based clock gating:

- ✓ **Same Functionality:** 100% equivalent to base module
 - ✓ **Power Savings:** 25-70% depending on traffic utilization
 - ✓ **Configurable:** Idle threshold, gating domains, enable/disable
 - ✓ **Zero Overhead When Disabled:** `ENABLE_CLOCK_GATING=0` → identical to base
-

15.3 Common Parameters

In addition to all [axi4_slave_rd_mon](#) parameters (including `USE_MONITOR`):

Parameter	Default	Description
<code>ENABLE_CLOCK_GATING</code>	1	Master enable (0=disable, identical to base)
<code>CG_IDLE_CYCLES</code>	8	Cycles before clock gating activates
<code>CG_GATE_*</code>	1	Domain-specific gating enables
<code>USE_MONITOR</code>	1	Synthesis-time monitor enable (forwarded to inner monitor).
<code>N_ADDR_RANGES</code>	0	Number of address-range comparators (forwarded to base module).

All base-module ports are forwarded unchanged, including the full performance-monitoring interface (see [Performance Monitoring](#) below). The six `ENABLE_*_LOGIC` synthesis-cone parameters (`ENABLE_ERROR_LOGIC`, `ENABLE_TIMEOUT_LOGIC`, `ENABLE_COMPL_LOGIC`, `ENABLE_THRESHOLD_LOGIC`, `ENABLE_PERF_LOGIC`, `ENABLE_DEBUG_LOGIC`) and the `cfg_compl_enable` / `cfg_threshold_enable` / `cfg_debug_enable` control enables are also passed straight through.

15.4 Performance Monitoring

The clock-gated wrapper exposes the full perfmon interface of the base module and **forwards every port unchanged** to the inner `axi4_slave_rd_mon`. The measurement-window state machine, the four R-channel utilization buckets (productive / back-pressure / starvation / idle), and the beat/byte/burst throughput counters behave exactly as documented in the base module — see [Performance Monitoring in axi4_slave_rd_mon](#) for the full narrative and per-bit semantics.

Forwarded perfmon ports (identical width and direction to the base module):

- **Inputs:** `cfg_perf_enable`, `cfg_start_event_sel` (3), `cfg_end_event_sel` (3), `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Outputs:** `window_active`, `window_cycles` (32), `perf_prod_cycles` (32), `perf_bp_cycles` (32), `perf_starv_cycles` (32), `perf_idle_cycles` (32), `perf_beat_count` (32), `perf_byte_count` (64), `perf_burst_count` (32)

The `perf_burst_count` output tracks AR (read address) handshakes. Clock gating never suppresses these paths: while a measurement window is open the monitor stays awake, so window cycle accounting remains exact regardless of `CG_IDLE_CYCLES`.

15.5 Quick Usage

```
axi4_slave_rd_mon_cg #(
    // Base module parameters (see axi4_slave_rd_mon.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating (see CG guide for details)
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
```

```
// ... all other ports same as axi4_slave_rd_mon  
);
```

15.6 Documentation

- **Base Module Functionality:** [axi4_slave_rd_mon.md](#)
 - **Clock Gating Guide:** [clock_gated_variants.md](#)
 - **Detailed CG Examples:**
 - [axi4_master_rd_mon_cg.md](#) (AXI4 monitor)
 - [axil4_master_rd_mon_cg.md](#) (AXIL4 monitor)
 - [apb_slave_cg.md](#) (APB interface)
-

15.7 Navigation

- [← Back to AXI4 Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

16 AXI4 Slave Read Monitor

Module: `axi4_slave_rd_mon.sv` **Location:** `rtl/amba/axi4/` **Status:** ✓ Production Ready

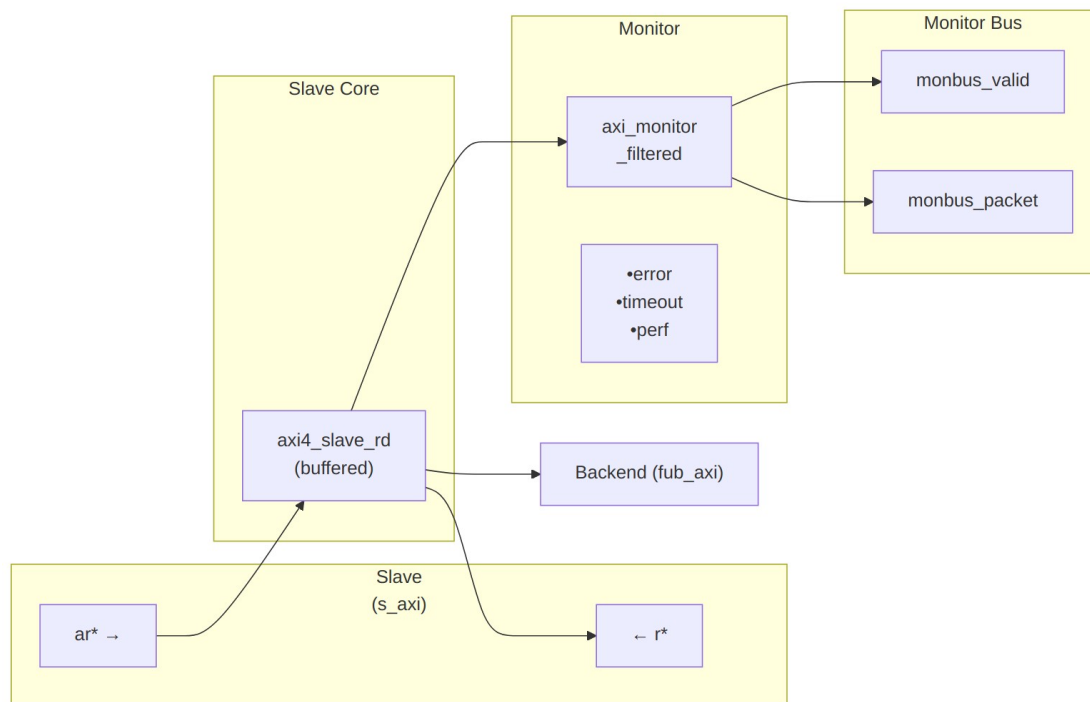
16.1 Overview

The AXI4 Slave Read Monitor module combines a functional AXI4 slave read interface with comprehensive transaction monitoring and filtering capabilities. This module is essential for verification environments, providing real-time protocol checking, error detection, performance metrics, and configurable packet filtering for slave-side read transactions.

16.1.1 Key Features

- ✓ **Integrated Monitoring:** Combines `axi4_slave_rd` with `axi_monitor_filtered`
- ✓ **3-Level Filtering:** Packet type masks, error routing, individual event masking
- ✓ **Error Detection:** Protocol violations, SLVERR, DECERR, orphan transactions

- ✓ **Timeout Monitoring:** Configurable timeout detection for stuck transactions
 - ✓ **Performance Metrics:** Latency tracking, transaction counting, throughput analysis
 - ✓ **Monitor Bus Output:** 128-bit packets paired with 64-bit side-band timestamps
 - ✓ **Configuration Validation:** Detects conflicting configuration settings
 - ✓ **Clock Gating Support:** Busy signal for power management
-



Module Architecture

The module instantiates two sub-modules: 1. **axi4_slave_rd** - Core AXI4 slave read functionality with buffering 2. **axi_monitor_filtered** - Transaction monitoring with 3-level filtering

16.2 Parameters

16.2.1 AXI4 Slave Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel

Parameter	Type	Default	Description
			skid buffer depth
SKID_DEPTH_R	int	4	R channel skid buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width

16.2.2 Monitor Parameters

Parameter	Type	Default	Description
UNIT_ID	int	2	4-bit unit identifier in monitor packets
AGENT_ID	int	20	8-bit agent identifier in monitor packets
MAX_TRANSACTIONS	int	16	Maximum concurrent outstanding transactions
ENABLE_FILTERING	bit	1	Enable packet filtering (0=pass all packets)
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
N_ADDR_RANGES	int	0	Number of address-range comparators. 0 = checker omitted (zero

Parameter	Type	Default	Description
			area). >0 = N independent [low, high] ranges; hits emit PktTypeError + AXI_ERR_ADDR_RANGE on monbus.

16.2.3 Synthesis-Cone Parameters

Each detection cone can be compiled out to save area. The classic cones default on; the debug cone defaults off. Setting a bit to 0 drops the corresponding cone at synthesis via a generate-if inside `axi_monitor_reporter` — the area saving is real.

Parameter	Type	Default	Effect when 0
ENABLE_ERROR_LOGIC	bit	1	Drop the error-detection cone
ENABLE_TIMEOUT_LOGIC	bit	1	Drop the timeout cone and the <code>axi_monitor_timeout</code> instance
ENABLE_COMPL_LOGIC	bit	1	Drop the completion cone
ENABLE_THRESHOLD_LOGIC	bit	1	Drop the threshold cone
ENABLE_PERF_LOGIC	bit	1	Drop the perfmon window + counters
ENABLE_DEBUG_LOGIC	bit	0	Drop the debug (trace) cone — the 6th reporter sub-block (off by default)

Internally the wrapper hardwires two master switches on its `axi_monitor_filtered` instance: `ENABLE_PERF_PACKETS = 1` (perf datapath present, so `ENABLE_PERF_LOGIC` alone gates the window/counters) and `ENABLE_DEBUG_MODULE = 0` (debug tracking module omitted). These are not top-level parameters of the wrapper.

The transaction CAM is always pipelined.

16.3 Monitor Backpressure (block_ready)

The monitor exposes a `block_ready` signal that goes low when its internal FIFO is saturated and cannot accept a new in-flight transaction. The wrapper ANDs `block_ready` into the upstream-facing `s_axi_arready` so a saturated monitor stalls new transactions on the wire instead of dropping events.

- **Where the stall lands:** the upstream `s_axi_arready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`:** `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.

This replaces a previous bug where `block_ready` was left unconnected and a full monitor FIFO would silently lose events.

16.4 Address-Range Checker

The wrapper can be parameterized with `N_ADDR_RANGES > 0` to instantiate an N-comparator address-range checker that watches every accepted AR handshake and emits a `PktTypeError` monbus packet (event code `AXI_ERR_ADDR_RANGE = 8'h0D`) when an address falls inside any of the configured `[low, high]` inclusive ranges.

Config inputs (active only when `N_ADDR_RANGES > 0`): - `cfg_addr_check_enable` — master on/off for the checker. - `cfg_addr_range_enable[N-1:0]` — per-range enable bit. - `cfg_addr_range_low[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive low bound for each range. - `cfg_addr_range_high[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive high bound for each range.

Event encoding (within the standard 128-bit `monitor_packet_t`, `event_data` field): - `packet_type = PktTypeError (4'h0)` - `protocol = AXI (3'b000)` - `event_code = AXI_ERR_ADDR_RANGE (8'h0D)` - `event_data[63:60] = range_index` (4 bits; supports up to 16 ranges) - `event_data[59:0]` = full matched address (up to 60 bits, zero-padded if narrower)

Exact match: set `cfg_addr_range_low[i] == cfg_addr_range_high[i]`.

Filtering: the existing `cfg_axi_error_mask[13]` bit masks this event code; set it high to suppress range packets without disabling other errors. No new mask wiring needed.

Per-range coalescing: if a range hits again before its packet has been emitted, the latched address is overwritten (latest hit wins). One packet per cycle drains the pending mask via a lowest-index priority encoder. Distinct ranges never lose events; under sustained per-range bursts, only the latest address per range is reported per emission cycle.

16.5 Performance Monitoring

The wrapper hardwires `ENABLE_PERF_PACKETS = 1` on its inner `axi_monitor_filtered`, so the perfmon datapath is present whenever `ENABLE_PERF_LOGIC = 1` (the default). It instantiates a **measurement-window state machine** plus a bank of R-data-channel utilization counters. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows, so the host can read a completed window's totals.

16.5.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**. The event sources are selected by `cfg_start_event_sel / cfg_end_event_sel` (3-bit; e.g. `3'b010` selects the `cfg_perf_enable` edge) and can also be fired directly by the `cfg_start_trigger / cfg_end_trigger` pulses (from an engine or CSR). `cfg_window_force_close` is a software override that closes the window immediately. While the window is open:

- `window_active` is high.
- `window_cycles[31:0]` free-runs, counting every clock elapsed inside the window.

Sample the counters on the cycle `window_active` falls to 0 (drive `cfg_end_trigger`, or wait for the configured end event).

16.5.2 Utilization Counters (R Data Channel)

Every cycle inside the window is classified by the R channel's `rvalid / rready` into exactly one of four buckets:

Output	Width	Condition	Meaning
<code>perf_prod_cycles</code>	32	<code>rvalid && rready</code>	productive beat transferred
<code>perf_bp_cycles</code>	32	<code>rvalid && !rready</code>	back-pressure (data offered, sink not ready)
<code>perf_starv_cycles</code>	32	<code>!rvalid && rready</code>	starvation (sink ready, no data)
<code>perf_idle_cycles</code>	32	<code>!rvalid && !rready</code>	idle

The four buckets sum to `window_cycles`, so `utilization = perf_prod_cycles / window_cycles`.

16.5.3 Throughput Counters

Output	Width	Meaning
perf_beat_count	32	R data beats transferred (= perf_prod_cycles, 1 beat/cycle)
perf_byte_count	64	bytes transferred = beats × (1 << ARSIZE), using the ARSIZE captured at the most recent AR address phase
perf_burst_count	32	AR address-phase handshakes

The integrator computes average burst length as $\text{perf_beat_count} / \text{perf_burst_count}$.

16.5.4 Performance Monitoring Ports

Port	Direction	Width	Description
cfg_perf_enable	Input	1	Enable performance-metric packet generation
cfg_start_event_sel	Input	3	Window start event source select
cfg_end_event_sel	Input	3	Window end event source select
cfg_start_trigger	Input	1	Pulse: open the measurement window
cfg_end_trigger	Input	1	Pulse: close the measurement window
cfg_window_force_close	Input	1	Software override: force the window closed
window_active	Output	1	High while a measurement window is open
window_cycles	Output	32	Cycles elapsed in the current window
perf_prod_cycles	Output	32	rvalid && rready cycles

Port	Direction	Width	Description
perf_bp_cycles	Output	32	rvalid && !rready cycles (back-pressure)
perf_starv_cycles	Output	32	!rvalid && rready cycles (starvation)
perf_idle_cycles	Output	32	!rvalid && !rready cycles
perf_beat_count	Output	32	R data beats transferred
perf_byte_count	Output	64	bytes transferred
perf_burst_count	Output	32	AR address-phase handshakes

When USE_MONITOR = 0, every perfmon output is tied to 0 and the window never opens.

16.6 Port Groups

16.6.1 AXI4 Read Channels

****Slave Interface (s_axi_*):**** - AR channel: arid, araddr, arlen, arsize, arbust, arlock, arcache, arprot, arqos, arregion, aruser, arvalid, arready - R channel: rid, rdata, rresp, rlack, ruser, rvalid, rready

****Backend Interface (fub_axi_*):**** - Same signals as slave, mirrored direction (to memory/backend)

16.6.2 Monitor Configuration

Configuration ports are identical to other AXI4 monitors: - Basic enables: cfg_monitor_enable, cfg_error_enable, cfg_timeout_enable, cfg_perf_enable, cfg_compl_enable (completion packets), cfg_threshold_enable (threshold-crossed packets), cfg_debug_enable (debug/trace cone — the 6th reporter sub-block) - Clear: cam_clear (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]) - Thresholds: cfg_timeout_cycles, cfg_latency_threshold - Filtering: 7 mask signals (cfg_axi_*_mask) - Performance window control: cfg_start_event_sel, cfg_end_event_sel, cfg_start_trigger, cfg_end_trigger, cfg_window_force_close (see [Performance Monitoring](#))

The inner monitor's `cfg_debug_level` (tied to 0), `cfg_debug_mask` (0) and `cfg_active_trans_threshold` (8) are fixed inside the wrapper and are **not** top-level ports on this module.

16.6.3 Monitor Bus Output

Port	Direction	Width	Description
<code>monbus_valid</code>	Output	1	Monitor packet valid
<code>monbus_ready</code>	Input	1	Downstream ready to accept packet
<code>monbus_packet</code>	Output	128	<code>monitor_packet_t</code> (see format below)
<code>monbus_timestamp</code>	Output	64	<code>monbus_timestamp_t</code> paired atomically with <code>monbus_packet</code>
<code>i_mon_time</code>	Input	64	Free-running counter from <code>monbus_axil_group</code> , sampled at packet emission

16.6.4 Status Outputs

Port	Direction	Width	Description
<code>busy</code>	Output	1	Indicates active transactions (for clock gating)
<code>active_transactions</code>	Output	8	Current number of outstanding transactions
<code>error_count</code>	Output	16	Cumulative error count
<code>transaction_count</code>	Output	32	Total transaction count
<code>cfg_conflict_error</code>	Output	1	Configuration conflict detected

Identical 128-bit format (with 64-bit side-band timestamp) as other AXI4 monitors. See [axi4_master_rd_mon](#) for complete specification.

The following waveforms show AXI4 slave read monitor behavior from the slave perspective:

AXI4 read transaction from slave interface perspective:



WaveJSON: [single_beat_read_001.json](#)

Key Observations: - Slave-side AR channel (s_axi_ar) - *Slave-side R channel response (s_axi_r)* - Monitor packet generation from slave perspective - Slave RRESP status monitoring - Transaction tracking in slave context

Variant read transaction with different timing from slave:



WaveJSON: [single beat read 002_001.json](#)

Key Observations: - Slave ready signal behavior - ARREADY backpressure effects - RVALID generation timing - Slave latency monitoring - Error detection from slave side

16.9 Configuration Strategies

16.9.1 Strategy 1: Functional Verification (Recommended)

Goal: Catch slave-side read errors

```
// Enable configuration
.cfg_monitor_enable    (1'b1),
.cfg_error_enable      (1'b1),      // Detect SLVERR/DECERR
.cfg_timeout_enable    (1'b1),      // Detect backend timeouts
.cfg_perf_enable       (1'b0),      // Disable (reduces traffic)

// Filtering - pass error and timeout only
.cfg_axi_pkt_mask      (16'b1111_1111_0000_0011),
.cfg_axi_error_mask    (16'h0000),  // Pass all errors
.cfg_axi_timeout_mask  (16'h0000),  // Pass all timeouts
.cfg_axi_compl_mask    (16'hFFFF),  // Drop completions
.cfg_axi_perf_mask     (16'hFFFF),  // Drop performance

// Timeouts
.cfg_timeout_cycles    (16'd1000),  // Backend response timeout
.cfg_latency_threshold (32'd500)
```

16.9.2 Strategy 2: Performance Analysis

Goal: Analyze slave read performance

```
// Enable configuration
.cfg_monitor_enable    (1'b1),
.cfg_error_enable      (1'b1),
.cfg_timeout_enable    (1'b0),
.cfg_perf_enable       (1'b1),      // Enable performance metrics

// Filtering - pass error and performance only
.cfg_axi_pkt_mask      (16'b1111_1110_0000_0001),
.cfg_axi_error_mask    (16'h0000),  // Pass all errors
.cfg_axi_perf_mask     (16'h0000),  // Pass all performance
.cfg_axi_compl_mask    (16'hFFFF),  // Drop completions
```

16.10 Usage Example

16.10.1 Basic Integration with Memory

```
axi4_slave_rd_mon #(
    .SKID_DEPTH_AR    (2),
    .SKID_DEPTH_R     (4),
```

```

.AXI_ID_WIDTH           (4),
.AXI_ADDR_WIDTH         (32),
.AXI_DATA_WIDTH         (64),
.AXI_USER_WIDTH         (1),
.UNIT_ID                (2),
.AGENT_ID               (20),
.MAX_TRANSACTIONS       (16),
.ENABLE_FILTERING       (1)
) u_slave_rd_mon (
    .aclk                 (axi_aclk),
    .aresetn              (axi_aresetn),

    // Slave interface (from interconnect)
    .s_axi_arid            (s_axi_arid),
    .s_axi_araddr          (s_axi_araddr),
    // ... rest of AR/R signals

    // Backend interface (to memory controller)
    .fub_axi_arid          (mem_arid),
    .fub_axi_araddr        (mem_araddr),
    // ... rest of AR/R signals

    // Monitor configuration
    .cfg_monitor_enable    (1'b1),
    .cfg_error_enable      (1'b1),
    .cfg_timeout_enable    (1'b1),
    .cfg_perf_enable        (1'b0),
    .cfg_timeout_cycles     (16'd2000), // Higher for memory latency
    .cfg_latency_threshold (32'd1000),

    .cfg_axi_pkt_mask       (16'b1111_1111_0000_0011),
    .cfg_axi_error_mask     (16'h0000),
    .cfg_axi_timeout_mask   (16'h0000),
    .cfg_axi_compl_mask     (16'hFFFF),
    // ... rest of mask signals

    // Monitor bus output
    .monbus_valid           (mon_valid),
    .monbus_ready           (mon_ready),
    .monbus_packet          (mon_packet),

    // Status
    .busy                   (rd_slave_busy),
    .active_transactions    (rd_active),
    .error_count            (rd_errors),
    .transaction_count      (rd_count),
    .cfg_conflict_error      (cfg_conflict)

```



```
);

// Memory controller backend
memory_controller u_mem (
    .axi_aclk      (axi_aclk),
    .axi_aresetn   (axi_aresetn),
    // Connect to fub_axi_* signals
    .axi_arid       (mem_arid),
    .axi_araddr     (mem_araddr),
    // ...
);
```

16.11 Design Notes

16.11.1 Slave-Side Monitoring

Key Differences from Master Monitors: - Monitors from slave perspective (interconnect → backend) - Tracks backend response latency - Detects backend timeout scenarios - Default UNIT_ID=2, AGENT_ID=20 (distinguishes from master)

Slave Read Sequence: 1. AR channel: Master request arrives at slave 2. Monitor captures: Address, ID, burst parameters 3. AR forwarded to backend (memory/logic) 4. R channel: Backend returns data 5. Monitor tracks: Response latency, RLAST, RRESP 6. R forwarded back to master via interconnect

Common Slave Errors Detected: - **Backend timeout:** AR accepted but R never returned - **SLVERR:** Slave error (access violation, parity error) - **DECERR:** Decode error (shouldn't occur at slave, but detected) - **Burst mismatch:** R beats don't match ARLEN+1 - **ID corruption:** RID doesn't match tracked ARID

16.11.2 Performance Considerations

Backend Latency Monitoring: - Tracks AR → R first beat latency - Configurable timeout threshold - Separate from master-side latency

Typical Timeout Values: - SRAM backend: 100-500 cycles - DDR controller: 1000-5000 cycles - PCIe/external: 10000+ cycles

16.11.3 Buffer Depth Guidelines

Same as [axi4_slave_rd](#): - **SKID_DEPTH_AR:** 2 (default) - handles interconnect backpressure - **SKID_DEPTH_R:** 4 (default) - buffers backend burst data - Increase for high-latency backends or large bursts

16.12 Related Modules

16.12.1 Companion Monitors

- [axi4_master_rd_mon](#) - AXI4 master read with monitoring
- [axi4_master_wr_mon](#) - AXI4 master write with monitoring
- [axi4_slave_wr_mon](#) - AXI4 slave write with monitoring

16.12.2 Base Modules

- [axi4_slave_rd](#) - Functional AXI4 slave read (without monitoring)
- [axi_monitor_filtered](#) - Monitoring engine with filtering (shared/)

16.12.3 Used Components

- [gaxi_skid_buffer](#) - Elastic buffering
 - [axi_monitor_base](#) - Core monitoring logic (shared/)
 - [axi_monitor_trans_mgr](#) - Transaction tracking (shared/)
-

16.13 References

16.13.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)
- Monitor Bus Packet Format: [monitor_package_spec.md](#)

16.13.2 Source Code

- RTL: `rtl/amba/monitor/axi4_slave_rd_mon.sv`
- Tests: `val/amba/test_axi4_slave_rd_mon.py`
- Framework: `bin/TBClasses/components/axi4/`

16.13.3 Documentation

- Configuration Guide: [AXI Monitor Base](#)
 - Architecture: [RTLAmba Overview](#)
 - AXI4 Index: [README.md](#)
-

Last Updated: 2026-07-18

16.14 Navigation

- [← Back to AXI4 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

17 AXI4 Slave Write Monitor (Clock-Gated)

Module: axi4_slave_wr_mon_cg.sv **Base Module:** [axi4_slave_wr_mon](#) **Location:** rtl/amba/axi4/ **Status:** ✓ Production Ready

17.1 Quick Reference

This is the **clock-gated variant** of [axi4_slave_wr_mon](#).

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ [Clock-Gated Variants Guide](#)

17.2 Summary

The axi4_slave_wr_mon_cg module adds power optimization to axi4_slave_wr_mon through activity-based clock gating:

- ✓ **Same Functionality:** 100% equivalent to base module
 - ✓ **Power Savings:** 25-70% depending on traffic utilization
 - ✓ **Configurable:** Idle threshold, gating domains, enable/disable
 - ✓ **Zero Overhead When Disabled:** ENABLE_CLOCK_GATING=0 → identical to base
-

17.3 Common Parameters

In addition to all [axi4_slave_wr_mon](#) parameters (including USE_MONITOR):

Parameter	Default	Description
ENABLE_CLOCK_GATING	1	Master enable (0=disable,

Parameter	Default	Description
CG_IDLE_CYCLES	8	identical to base) Cycles before clock gating activates
CG_GATE_*	1	Domain-specific gating enables
USE_MONITOR	1	Synthesis-time monitor enable (forwarded to inner monitor).
N_ADDR_RANGES	0	Number of address-range comparators (forwarded to base module).

All base-module ports are forwarded unchanged, including the full performance-monitoring interface (see [Performance Monitoring](#) below). The six ENABLE_*_LOGIC synthesis-cone parameters (ENABLE_ERROR_LOGIC, ENABLE_TIMEOUT_LOGIC, ENABLE_COMPL_LOGIC, ENABLE_THRESHOLD_LOGIC, ENABLE_PERF_LOGIC, ENABLE_DEBUG_LOGIC) and the cfg_compl_enable / cfg_threshold_enable / cfg_debug_enable control enables are also passed straight through.

17.4 Performance Monitoring

The clock-gated wrapper exposes the full perfmon interface of the base module and **forwards every port unchanged** to the inner axi4_slave_wr_mon. The measurement-window state machine, the four W-channel utilization buckets (productive / back-pressure / starvation / idle), and the beat/byte/burst throughput counters behave exactly as documented in the base module — see [Performance Monitoring in axi4_slave_wr_mon](#) for the full narrative and per-bit semantics.

Forwarded perfmon ports (identical width and direction to the base module):

- **Inputs:** cfg_perf_enable, cfg_start_event_sel (3), cfg_end_event_sel (3), cfg_start_trigger, cfg_end_trigger, cfg_window_force_close
- **Outputs:** window_active, window_cycles (32), perf_prod_cycles (32), perf_bp_cycles (32), perf_starv_cycles (32), perf_idle_cycles (32), perf_beat_count (32), perf_byte_count (64), perf_burst_count (32)

The `perf_burst_count` output tracks AW (write address) handshakes. Clock gating never suppresses these paths: while a measurement window is open the monitor stays awake, so window cycle accounting remains exact regardless of `CG_IDLE_CYCLES`.

17.5 Quick Usage

```
axi4_slave_wr_mon_cg #(
    // Base module parameters (see axi4_slave_wr_mon.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating (see CG guide for details)
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... all other ports same as axi4_slave_wr_mon
);
```

17.6 Documentation

- **Base Module Functionality:** [axi4_slave_wr_mon.md](#)
 - **Clock Gating Guide:** [clock_gated_variants.md](#)
 - **Detailed CG Examples:**
 - [axi4_master_rd_mon_cg.md](#) (AXI4 monitor)
 - [axil4_master_rd_mon_cg.md](#) (AXIL4 monitor)
 - [apb_slave_cg.md](#) (APB interface)
-

17.7 Navigation

- [← Back to AXI4 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

18 AXI4 Slave Write Monitor

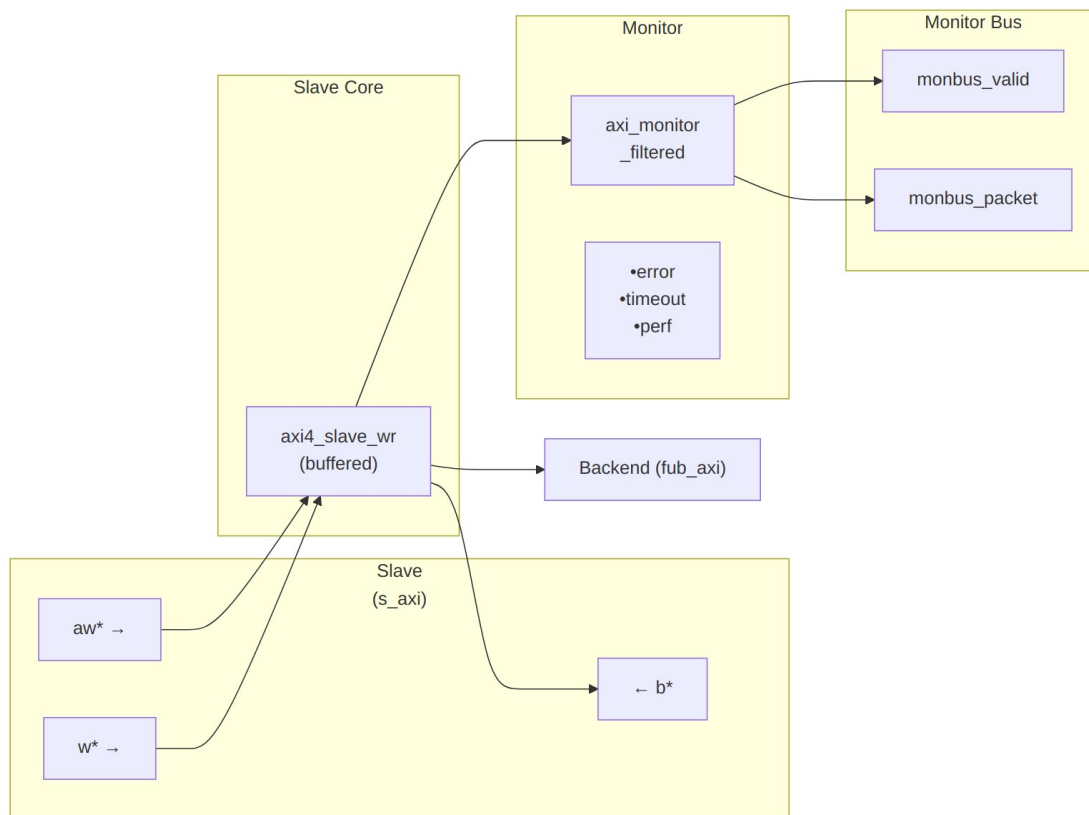
Module: axi4_slave_wr_mon.sv **Location:** rtl/amba/axi4/ **Status:** ✓ Production Ready

18.1 Overview

The AXI4 Slave Write Monitor module combines a functional AXI4 slave write interface with comprehensive transaction monitoring and filtering capabilities. This module is essential for verification environments, providing real-time protocol checking, error detection, performance metrics, and configurable packet filtering for slave-side write transactions.

18.1.1 Key Features

- ✓ **Integrated Monitoring:** Combines axi4_slave_wr with axi_monitor_filtered
 - ✓ **3-Level Filtering:** Packet type masks, error routing, individual event masking
 - ✓ **Error Detection:** Protocol violations, SLVERR, DECERR, orphan transactions
 - ✓ **Timeout Monitoring:** Configurable timeout detection for stuck transactions
 - ✓ **Performance Metrics:** Latency tracking, transaction counting, throughput analysis
 - ✓ **Monitor Bus Output:** 128-bit packets paired with 64-bit side-band timestamps
 - ✓ **Configuration Validation:** Detects conflicting configuration settings
 - ✓ **Clock Gating Support:** Busy signal for power management
-



Module Architecture

The module instantiates two sub-modules: 1. **axi4_slave_wr** - Core AXI4 slave write functionality with buffering 2. **axi_monitor_filtered** - Transaction monitoring with 3-level filtering

18.2 Parameters

18.2.1 AXI4 Slave Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel skid buffer depth
SKID_DEPTH_W	int	4	W channel skid buffer depth
SKID_DEPTH_B	int	2	B channel skid

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	buffer depth Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width

18.2.2 Monitor Parameters

Parameter	Type	Default	Description
UNIT_ID	int	2	4-bit unit identifier in monitor packets
AGENT_ID	int	21	8-bit agent identifier in monitor packets
MAX_TRANSACTIONS	int	16	Maximum concurrent outstanding transactions
ENABLE_FILTERING	bit	1	Enable packet filtering (0=pass all packets)
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
N_ADDR_RANGES	int	0	Number of address-range comparators. 0 = checker omitted (zero area). >0 = N independent [low, high] ranges; hits emit PktTypeError +

Parameter	Type	Default	Description
			AXI_ERR_ADDR_RANGE on monbus.

18.2.3 Synthesis-Cone Parameters

Each detection cone can be compiled out to save area. The classic cones default on; the debug cone defaults off. Setting a bit to 0 drops the corresponding cone at synthesis via a generate-if inside `axi_monitor_reporter` — the area saving is real.

Parameter	Type	Default	Effect when 0
ENABLE_ERROR_LOGIC	bit	1	Drop the error-detection cone
ENABLE_TIMEOUT_LOGIC	bit	1	Drop the timeout cone and the <code>axi_monitor_timeout</code> instance
ENABLE_COMPL_LOGIC	bit	1	Drop the completion cone
ENABLE_THRESHOLD_LOGIC	bit	1	Drop the threshold cone
ENABLE_PERF_LOGIC	bit	1	Drop the perfmon window + counters
ENABLE_DEBUG_LOGIC	bit	0	Drop the debug (trace) cone — the 6th reporter sub-block (off by default)

Internally the wrapper hardwires two master switches on its `axi_monitor_filtered` instance: `ENABLE_PERF_PACKETS = 1` (perf datapath present, so `ENABLE_PERF_LOGIC` alone gates the window/counters) and `ENABLE_DEBUG_MODULE = 0` (debug tracking module omitted). These are not top-level parameters of the wrapper.

The transaction CAM is always pipelined.

18.3 Monitor Backpressure (`block_ready`)

The monitor exposes a `block_ready` signal that goes low when its internal FIFO is saturated and cannot accept a new in-flight transaction. The wrapper ANDs `block_ready` into the upstream-facing `s_axi_awready` so a saturated monitor stalls new transactions on the wire instead of dropping events.

- **Where the stall lands:** the upstream `s_axi_awready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`:** `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.

This replaces a previous bug where `block_ready` was left unconnected and a full monitor FIFO would silently lose events.

18.4 Address-Range Checker

The wrapper can be parameterized with `N_ADDR_RANGES > 0` to instantiate an N-comparator address-range checker that watches every accepted AW handshake and emits a `PktTypeError` monbus packet (event code `AXI_ERR_ADDR_RANGE = 8'h0D`) when an address falls inside any of the configured `[low, high]` inclusive ranges.

Config inputs (active only when `N_ADDR_RANGES > 0`): - `cfg_addr_check_enable` — master on/off for the checker. - `cfg_addr_range_enable[N-1:0]` — per-range enable bit. - `cfg_addr_range_low[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive low bound for each range. - `cfg_addr_range_high[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive high bound for each range.

Event encoding (within the standard 128-bit `monitor_packet_t`, `event_data` field): - `packet_type = PktTypeError (4'h0)` - `protocol = AXI (3'b000)` - `event_code = AXI_ERR_ADDR_RANGE (8'h0D)` - `event_data[63:60] = range_index` (4 bits; supports up to 16 ranges) - `event_data[59:0] = full matched address` (up to 60 bits, zero-padded if narrower)

Exact match: set `cfg_addr_range_low[i] == cfg_addr_range_high[i]`.

Filtering: the existing `cfg_axi_error_mask[13]` bit masks this event code; set it high to suppress range packets without disabling other errors. No new mask wiring needed.

Per-range coalescing: if a range hits again before its packet has been emitted, the latched address is overwritten (latest hit wins). One packet per cycle drains the pending mask via a lowest-index priority encoder. Distinct ranges never lose events; under sustained per-range bursts, only the latest address per range is reported per emission cycle.

18.5 Performance Monitoring

The wrapper hardwires `ENABLE_PERF_PACKETS = 1` on its inner `axi_monitor_filtered`, so the perfmon datapath is present whenever `ENABLE_PERF_LOGIC = 1` (the default). It instantiates a **measurement-window state machine** plus a bank of W-data-channel utilization counters. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows, so the host can read a completed window's totals.

18.5.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**. The event sources are selected by `cfg_start_event_sel` / `cfg_end_event_sel` (3-bit; e.g. 3'b010 selects the `cfg_perf_enable` edge) and can also be fired directly by the `cfg_start_trigger` / `cfg_end_trigger` pulses (from an engine or CSR). `cfg_window_force_close` is a software override that closes the window immediately. While the window is open:

- `window_active` is high.
- `window_cycles[31:0]` free-runs, counting every clock elapsed inside the window.

Sample the counters on the cycle `window_active` falls to 0 (drive `cfg_end_trigger`, or wait for the configured end event).

18.5.2 Utilization Counters (W Data Channel)

Every cycle inside the window is classified by the W channel's `wvalid` / `wready` into exactly one of four buckets:

Output	Width	Condition	Meaning
<code>perf_prod_cycles</code>	32	<code>wvalid && wready</code>	productive beat transferred
<code>perf_bp_cycles</code>	32	<code>wvalid && !wready</code>	back-pressure (data offered, sink not ready)
<code>perf_starv_cycles</code>	32	<code>!wvalid && wready</code>	starvation (sink ready, no data)
<code>perf_idle_cycles</code>	32	<code>!wvalid && !wready</code>	idle

The four buckets sum to `window_cycles`, so `utilization = perf_prod_cycles / window_cycles`.

18.5.3 Throughput Counters

Output	Width	Meaning
<code>perf_beat_count</code>	32	W data beats transferred (= <code>perf_prod_cycles</code> , 1 beat/cycle)
<code>perf_byte_count</code>	64	bytes transferred = beats × (1 << AWSIZE), using the AWSIZE captured at the

Output	Width	Meaning
perf_burst_count	32	most recent AW address phase AW address-phase handshakes

The integrator computes average burst length as $\text{perf_beat_count} / \text{perf_burst_count}$.

18.5.4 Performance Monitoring Ports

Port	Direction	Width	Description
cfg_perf_enable	Input	1	Enable performance-metric packet generation
cfg_start_event_sel	Input	3	Window start event source select
cfg_end_event_sel	Input	3	Window end event source select
cfg_start_trigger	Input	1	Pulse: open the measurement window
cfg_end_trigger	Input	1	Pulse: close the measurement window
cfg_window_force_close	Input	1	Software override: force the window closed
window_active	Output	1	High while a measurement window is open
window_cycles	Output	32	Cycles elapsed in the current window
perf_prod_cycles	Output	32	wvalid && wready cycles
perf_bp_cycles	Output	32	wvalid && !wready cycles (back-pressure)
perf_starv_cycles	Output	32	!wvalid && wready cycles (starvation)
perf_idle_cycles	Output	32	!wvalid && !wready cycles

Port	Direction	Width	Description
perf_beat_count	Output	32	W data beats transferred
perf_byte_count	Output	64	bytes transferred
perf_burst_count	Output	32	AW address-phase handshakes

When USE_MONITOR = 0, every perfmon output is tied to 0 and the window never opens.

18.6 Port Groups

18.6.1 AXI4 Write Channels

****Slave Interface (s_axi_*):**** - AW channel: awid, awaddr, awlen, awsize, awburst, awlock, awcache, awprot, awqos, awregion, awuser, awvalid, awready - W channel: wdata, wstrb, wlast, wuser, wvalid, wready - B channel: bid, bresp, buser, bvalid, bready

****Backend Interface (fub_axi_*):**** - Same signals as slave, mirrored direction (to memory/backend)

18.6.2 Monitor Configuration

Configuration ports are identical to other AXI4 monitors: - Basic enables: cfg_monitor_enable, cfg_error_enable, cfg_timeout_enable, cfg_perf_enable, cfg_compl_enable (completion packets), cfg_threshold_enable (threshold-crossed packets), cfg_debug_enable (debug/trace cone — the 6th reporter sub-block) - Clear: cam_clear (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]) - Thresholds: cfg_timeout_cycles, cfg_latency_threshold - Filtering: 7 mask signals (cfg_axi_*_mask) - Performance window control: cfg_start_event_sel, cfg_end_event_sel, cfg_start_trigger, cfg_end_trigger, cfg_window_force_close (see [Performance Monitoring](#))

The inner monitor's cfg_debug_level (tied to 0), cfg_debug_mask (0) and cfg_active_trans_threshold (8) are fixed inside the wrapper and are **not** top-level ports on this module.

18.6.3 Monitor Bus Output

Port	Direction	Width	Description
monbus_valid	Output	1	Monitor packet valid

Port	Direction	Width	Description
monbus_ready	Input	1	Downstream ready to accept packet
monbus_packet	Output	128	monitor_packet_t (see format below)
monbus_timestamp	Output	64	monbus_timestamp_t paired atomically with monbus_packet
i_mon_time	Input	64	Free-running counter from monbus_axil_group, sampled at packet emission

18.6.4 Status Outputs

Port	Direction	Width	Description
busy	Output	1	Indicates active transactions (for clock gating)
active_transactions	Output	8	Current number of outstanding transactions
error_count	Output	16	Cumulative error count
transaction_count	Output	32	Total transaction count
cfg_conflict_error	Output	1	Configuration conflict detected

18.7 Monitor Packet Format

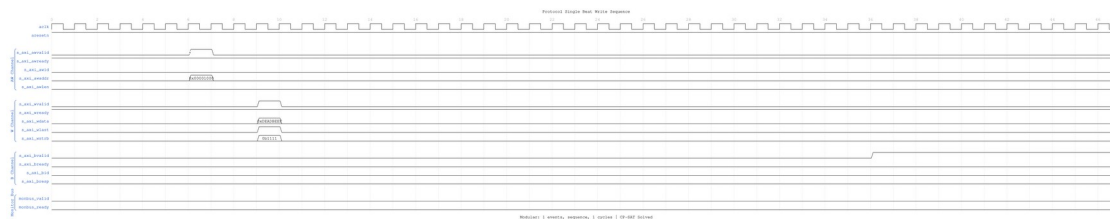
Identical 128-bit format (with 64-bit side-band timestamp) as other AXI4 monitors. See [axi4_master_rd_mon](#) for complete specification.

18.8 Timing Diagrams

The following waveforms show AXI4 slave write monitor behavior from the slave perspective:

18.8.1 Scenario 1: Single-Beat Write (Slave View)

AXI4 write transaction from slave interface perspective:



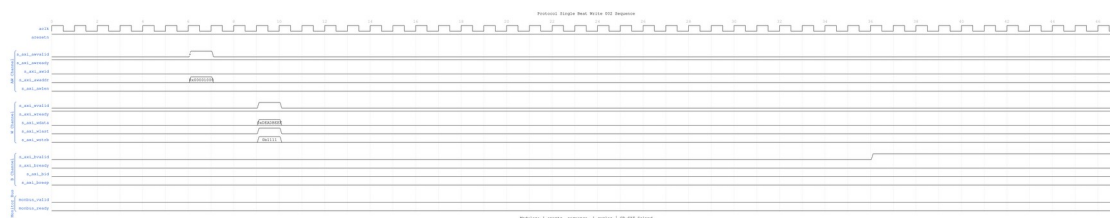
Single Beat Write Slave

WaveJSON: [single_beat_write_001.json](#)

Key Observations: - Slave-side AW channel (s_axi_aw) - *Slave-side W channel data (s_axi_w)* - Slave-side B channel response (s_axi_b*) - Monitor packet generation from slave perspective - Slave BRESP status monitoring - Transaction tracking in slave context

18.8.2 Scenario 2: Alternative Single-Beat Write (Slave View)

Variant write transaction with different timing from slave:



Single Beat Write Slave Alt

WaveJSON: [single_beat_write_002_001.json](#)

Key Observations: - Slave ready signal behavior - AWREADY backpressure effects - WREADY flow control - BVALID generation timing - Slave latency monitoring - Error detection from slave side

18.9 Configuration Strategies

18.9.1 Strategy 1: Functional Verification (Recommended)

Goal: Catch slave-side write errors

```
// Enable configuration
.cfg_monitor_enable      (1'b1),
.cfg_error_enable        (1'b1),           // Detect SLVERR/DECERR from
```

```

backend
.cfg_timeout_enable      (1'b1),      // Detect backend timeouts
.cfg_perf_enable         (1'b0),      // Disable (reduces traffic)

// Filtering - pass error and timeout only
.cfg_axi_pkt_mask        (16'b1111_1111_0000_0011),
.cfg_axi_error_mask      (16'h0000),  // Pass all errors
.cfg_axi_timeout_mask    (16'h0000),  // Pass all timeouts
.cfg_axi_compl_mask      (16'hFFFF),  // Drop completions
.cfg_axi_perf_mask       (16'hFFFF),  // Drop performance

// Timeouts
.cfg_timeout_cycles      (16'd2000),  // Backend write response timeout
.cfg_latency_threshold   (32'd1000)

```

18.9.2 Strategy 2: Performance Analysis

Goal: Analyze slave write performance

```

// Enable configuration
.cfg_monitor_enable      (1'b1),
.cfg_error_enable        (1'b1),
.cfg_timeout_enable      (1'b0),
.cfg_perf_enable         (1'b1),      // Enable performance metrics

// Filtering - pass error and performance only
.cfg_axi_pkt_mask        (16'b1111_1110_0000_0001),
.cfg_axi_error_mask      (16'h0000),  // Pass all errors
.cfg_axi_perf_mask       (16'h0000),  // Pass all performance
.cfg_axi_compl_mask      (16'hFFFF),  // Drop completions

```

18.10 Usage Example

18.10.1 Basic Integration with Memory

```

axi4_slave_wr_mon #(
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W        (4),
    .SKID_DEPTH_B        (2),
    .AXI_ID_WIDTH        (4),
    .AXI_ADDR_WIDTH      (32),
    .AXI_DATA_WIDTH      (64),
    .AXI_USER_WIDTH      (1),
    .UNIT_ID             (2),
    .AGENT_ID            (21),
    .MAX_TRANSACTIONS    (16),

```



```

        .ENABLE_FILTERING    (1)
    ) u_slave_wr_mon (
        .aclk                (axi_aclk),
        .aresetn             (axi_aresetn),

        // Slave interface (from interconnect)
        .s_axi_awid          (s_axi_awid),
        .s_axi_awaddr        (s_axi_awaddr),
        // ... rest of AW/W/B signals

        // Backend interface (to memory controller)
        .fub_axi_awid        (mem_awid),
        .fub_axi_awaddr      (mem_awaddr),
        // ... rest of AW/W/B signals

        // Monitor configuration
        .cfg_monitor_enable   (1'b1),
        .cfg_error_enable     (1'b1),
        .cfg_timeout_enable   (1'b1),
        .cfg_perf_enable      (1'b0),
        .cfg_timeout_cycles   (16'd2000), // Higher for memory latency
        .cfg_latency_threshold (32'd1000),

        .cfg_axi_pkt_mask     (16'b1111_1111_0000_0011),
        .cfg_axi_error_mask   (16'h0000),
        .cfg_axi_timeout_mask (16'h0000),
        .cfg_axi_compl_mask   (16'hFFFF),
        // ... rest of mask signals

        // Monitor bus output
        .monbus_valid         (mon_valid),
        .monbus_ready         (mon_ready),
        .monbus_packet        (mon_packet),

        // Status
        .busy                 (wr_slave_busy),
        .active_transactions  (wr_active),
        .error_count          (wr_errors),
        .transaction_count    (wr_count),
        .cfg_conflict_error    (cfg_conflict)
    );

    // Memory controller backend
    memory_controller u_mem (
        .axi_aclk            (axi_aclk),
        .axi_aresetn         (axi_aresetn),

```

```

// Connect to fub_axi_* signals
.axi_awid      (mem_awid),
.axi_awaddr    (mem_awaddr),
// ...
);

```

18.11 Design Notes

18.11.1 Slave-Side Monitoring

Key Differences from Master Monitors: - Monitors from slave perspective (interconnect → backend) - Tracks backend write response latency - Detects backend timeout scenarios - Default UNIT_ID=2, AGENT_ID=21 (distinguishes from master)

Slave Write Sequence: 1. AW channel: Master write address arrives at slave 2. Monitor captures: Address, ID, burst parameters 3. AW forwarded to backend (memory/logic) 4. W channel: Master sends data beats 5. Monitor tracks: Write beat count, WLAST 6. B channel: Backend returns write response 7. Monitor tracks: Response latency, BRESP 8. B forwarded back to master via interconnect

Common Slave Errors Detected: - **Backend timeout:** AW accepted but B never returned - **Write data timeout:** AW accepted but W never completed - **SLVERR:** Slave error (access violation, parity error, write protection) - **DECERR:** Decode error (shouldn't occur at slave, but detected) - **Burst mismatch:** W beats don't match AWLEN+1 - **ID corruption:** BID doesn't match tracked AWID - **WSTRB protocol violations:** Invalid byte lane enables

18.11.2 Performance Considerations

Backend Latency Monitoring: - Tracks AW → B response latency - Includes W channel completion time - Configurable timeout threshold - Separate from master-side latency

Typical Timeout Values: - SRAM backend: 100-500 cycles - DDR controller: 1000-5000 cycles - Flash/EEPROM: 10000+ cycles (write operations) - PCIe/external: 10000+ cycles

Write-Specific Timing: - AW-W ordering flexible in AXI4 - Backend may wait for WLAST before responding - B response latency includes W channel completion

18.11.3 Buffer Depth Guidelines

Same as [axi4_slave_wr](#): - **SKID_DEPTH_AW:** 2 (default) - handles interconnect backpressure - **SKID_DEPTH_W:** 4 (default) - buffers burst write data - **SKID_DEPTH_B:** 2 (default) - write responses are single-beat - Increase for high-latency backends or large bursts

18.11.4 Write Transaction Characteristics

Burst Write Monitoring: - Monitors AW channel for address capture - Tracks W channel beats until WLAST - Correlates B channel response with AWID - Validates burst length (W beats == AWLEN+1) - Checks WSTRB validity per beat

Performance Impact: - Write bursts more efficient than single writes - Backend may buffer/pipeline write data - B response can occur before W completion (with reordering) - Monitor packets generated per transaction, not per beat

18.12 Related Modules

18.12.1 Companion Monitors

- [axi4_master_rd_mon](#) - AXI4 master read with monitoring
- [axi4_master_wr_mon](#) - AXI4 master write with monitoring
- [axi4_slave_rd_mon](#) - AXI4 slave read with monitoring

18.12.2 Base Modules

- [axi4_slave_wr](#) - Functional AXI4 slave write (without monitoring)
- [axi_monitor_filtered](#) - Monitoring engine with filtering (shared/)

18.12.3 Used Components

- [gaxi_skid_buffer](#) - Elastic buffering
 - [axi_monitor_base](#) - Core monitoring logic (shared/)
 - [axi_monitor_trans_mgr](#) - Transaction tracking (shared/)
-

18.13 References

18.13.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)
- Monitor Bus Packet Format: [monitor_package_spec.md](#)

18.13.2 Source Code

- RTL: rtl/amba/monitor/axi4_slave_wr_mon.sv
- Tests: val/amba/test_axi4_slave_wr_mon.py
- Framework: bin/TBClasses/components/axi4/

18.13.3 Documentation

- Configuration Guide: [AXI Monitor Base](#)
 - Architecture: [RTLamba Overview](#)
 - AXI4 Index: [README.md](#)
-

Last Updated: 2026-07-18

18.14 Navigation

- [← Back to AXI4 Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

19 AXI5 Master Read with Monitor and Clock Gating

Module: `axi5_master_rd_mon_cg.sv` **Location:** `rtl/amba/axi5/` **Status:** Production Ready

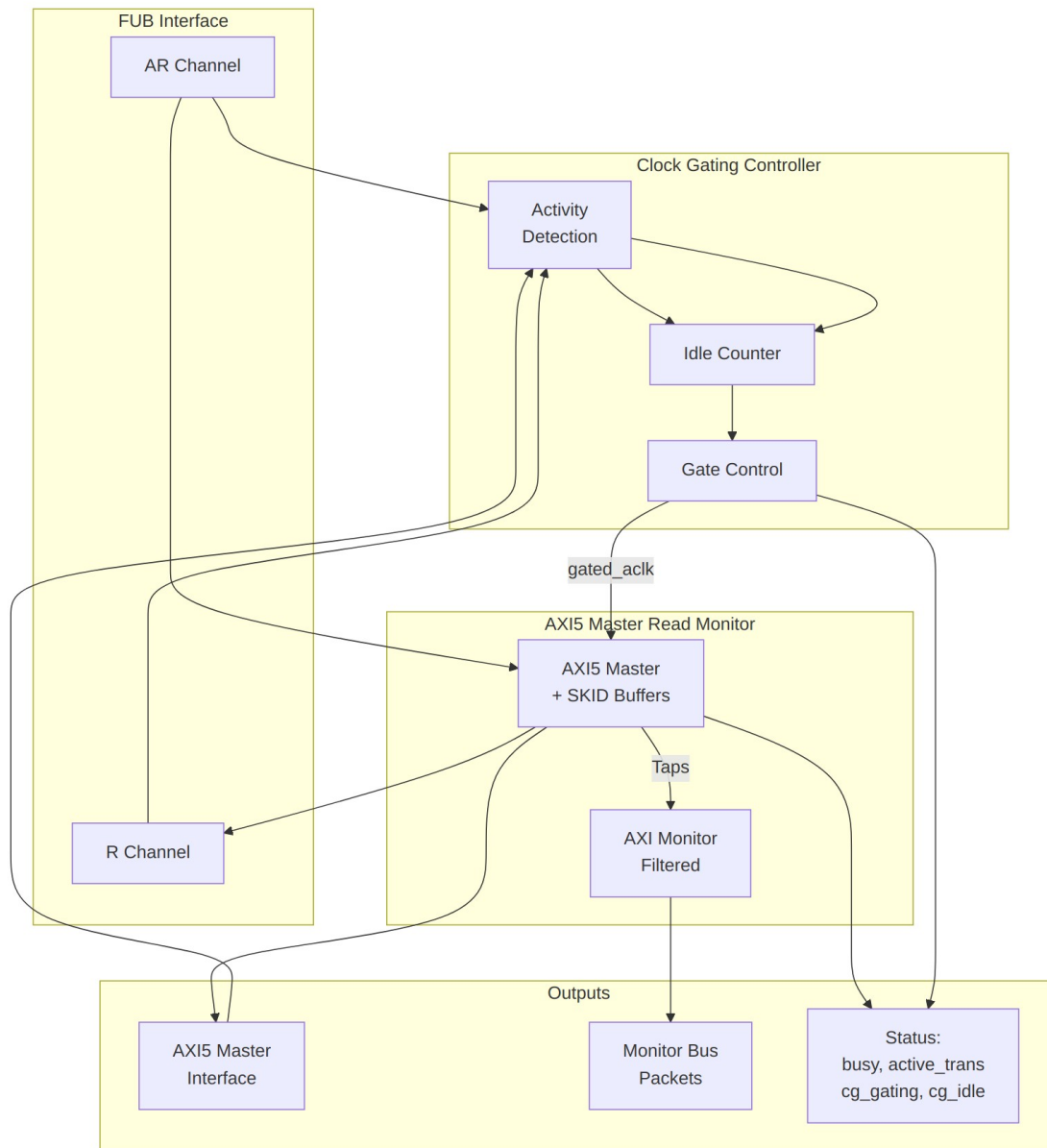
19.1 Overview

The AXI5 Master Read with Monitor and Clock Gating module combines `axi5_master_rd_mon` (AXI5 master with integrated monitoring) with intelligent clock gating for power optimization. This provides the ultimate combination: comprehensive transaction monitoring, error detection, and automatic power management.

19.1.1 Key Features

- Full AMBA AXI5 protocol compliance
- **All AXI5 extensions:** NSAID, TRACE, MPAM, MECID, UNIQUE, CHUNKING, MTE, POISON
- **Integrated AXI monitor** with 3-level filtering hierarchy
- **Error detection:** Protocol violations, SLVERR, DECERR
- **Timeout monitoring:** Stuck transactions, stalled channels
- **Performance metrics:** Latency, throughput, outstanding transactions
- **MonBus output:** Standardized 128-bit monitor packet format paired with 64-bit side-band timestamp
- **Automatic clock gating** based on activity detection

- **Configurable idle threshold** before clock gating activates
 - **Power savings** during idle periods
 - **Transparent operation** - no protocol changes
 - **Dual status outputs:** Monitor status + clock gating status
-



Module Architecture

19.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel SKID buffer depth
SKID_DEPTH_R	int	4	R channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width (PartID + PMG)
AXI_MECID_WIDTH	int	16	Memory encryption context ID width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
AXI_CHUNKNUM_WIDTH	int	4	Chunk number width
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption context
ENABLE_UNIQUE	bit	1	Enable unique ID indicator
ENABLE_CHUNKING	bit	1	Enable data chunking
ENABLE_MTE	bit	1	Enable Memory Tagging Extension

Parameter	Type	Default	Description
ENABLE_POISON	bit	1	Enable poison indicator
UNIT_ID	int	1	Monitor unit identifier
AGENT_ID	int	10	Monitor agent identifier
MAX_TRANSACTION S	int	16	Transaction table size
ENABLE_FILTERING	bit	1	Enable 3-level packet filtering
ADD_PIPELINE_STAGE	bit	0	Add pipeline stage in monitor
USE_MONITOR	bit	1	Synthesis-time monitor enable (forwarded to inner monitor).
N_ADDR_RANGES	int	0	Number of address-range comparators (forwarded to base module).
ENABLE_ERROR_LOGIC	bit	1	Synthesis-cone enable for error detection (forwarded to base module)
ENABLE_TIMEOUT_LOGIC	bit	1	Synthesis-cone enable for timeout detection (forwarded to base module)
ENABLE_COMPL_LOGIC	bit	1	Synthesis-cone enable for completion tracking (forwarded to base module)
ENABLE_THRESHOLD_LOGIC	bit	1	Synthesis-cone enable for threshold detection (forwarded to base module)
ENABLE_PERF_LOGIC	bit	1	Synthesis-cone enable for the perfmon window

Parameter	Type	Default	Description
ENABLE_DEBUG_LO GIC	bit	0	+ counters (forwarded to base module) Synthesis-cone enable for the debug/trace cone (forwarded to base module)
CG_IDLE_COUNT_W IDTH	int	4	Width of idle counter (max 2^N-1 cycles)

19.3 Ports

19.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock (ungated)
aresetn	1	Input	AXI active-low reset

19.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Clock gating enable (1=enable, 0=always active)
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating activates

19.3.3 FUB AXI5 Interface (Slave Side)

Same as axi5_master_rd_mon - see [AXI5 Master Read Monitor](#).

19.3.4 Master AXI5 Interface (Output Side)

Same as axi5_master_rd_mon - see [AXI5 Master Read Monitor](#).

19.3.5 Monitor Configuration

Same as axi5_master_rd_mon - see [AXI5 Master Read Monitor](#).

19.3.6 Monitor Bus Output

Port	Width	Direction	Description
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor packet ready (backpressure)
monbus_packet	128	Output	monitor_packet_t (128-bit format)
monbus_timestamp	64	Output	monbus_timestamp_t paired atomically with monbus_packet
i_mon_time	64	Input	Free-running counter from monbus_axil_group, sampled at packet emission

19.3.7 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Core busy indicator
active_transactions	8	Output	Number of outstanding transactions
error_count	16	Output	Cumulative error count (placeholder)
transaction_count	32	Output	Total transaction count (placeholder)
cfg_conflict_error	1	Output	Configuration conflict detected
cg_gating	1	Output	Clock gating active (1=gated, 0=running)
cg_idle	1	Output	Interface idle (1=no activity)

19.4 Functionality

19.4.1 Combined Monitoring and Power Management

This module provides the best of both worlds:

Monitoring (from axi5_master_rd_mon): - Real-time transaction monitoring - Error and timeout detection - Performance metrics collection - Configurable packet filtering - MonBus output for system integration

Power Management (from axi5_master_rd_cg): - Automatic clock gating during idle periods - Configurable idle threshold - Activity-based wake-up - Status outputs for power monitoring

19.4.2 Clock Gating with Monitor Active

The clock gating logic considers monitor activity:

```
user_valid = fub_axi_arvalid || fub_axi_rready || int_busy;
axi_valid = m_axi_arvalid || m_axi_rvalid;
```

```
// Clock remains active if:
// - User interface active (AR or R channels)
// - AXI interface active
// - Internal busy (transactions in flight)
// - Monitor has pending packets (implicit in int_busy)
```

The monitor continues operating during clock gating transitions, ensuring no events are lost.

19.4.3 Ready Signal Control During Gating

When clock gating is active, ready signals are forced low to prevent new transactions:

```
fub_axi_arready = cg_gating ? 1'b0 : int_arready;
m_axi_rready = cg_gating ? 1'b0 : int_rready;
```

This ensures protocol compliance during power management.

19.5 Performance Monitoring

The clock-gated wrapper exposes the full perfmon interface of the base module and **forwards every port unchanged** to the inner axi5_master_rd_mon. The measurement-window state machine, the four R-channel utilization buckets (productive / back-pressure / starvation / idle), and the beat/byte/burst throughput counters behave exactly as documented in the base module — see [Performance Monitoring in axi5_master_rd_mon](#) for the full narrative and per-bit semantics.

Forwarded perfmon ports (identical width and direction to the base module):

- **Inputs:** `cfg_perf_enable`, `cfg_start_event_sel` (3), `cfg_end_event_sel` (3), `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Outputs:** `window_active`, `window_cycles` (32), `perf_prod_cycles` (32), `perf_bp_cycles` (32), `perf_starv_cycles` (32), `perf_idle_cycles` (32), `perf_beat_count` (32), `perf_byte_count` (64), `perf_burst_count` (32)

The `perf_burst_count` output tracks AR (read address) handshakes. Clock gating never suppresses these paths: while a measurement window is open the monitor stays awake, so window cycle accounting remains exact regardless of the idle-count setting.

Alongside perfmon, the wrapper forwards the `cfg_compl_enable`, `cfg_threshold_enable`, and `cfg_debug_enable` control inputs, plus the six `ENABLE_*_LOGIC` synthesis-cone parameters (see [Parameters](#)), straight through to the base monitor.

19.6 Timing Diagrams

19.6.1 Clock Gating with Monitor Activity

TODO: Wavedrom timing diagram showing:

- `ACLK` (ungated)
- `GATED_ACLK` (gated clock)
- AXI transaction sequence
- Monitor packet generation
- Idle period detection
- `cg_gating` activation
- Monitor quiescent before clock stops

19.6.2 Wake-up with Immediate Monitoring

TODO: Wavedrom timing diagram showing:

- `ACLK` (ungated)
 - `GATED_ACLK` resuming
 - `ARVALID` assertion (wake trigger)
 - `cg_gating` deactivation
 - AXI transaction proceeds
 - Monitor packets generated immediately
-

19.7 Usage Example

19.7.1 Comprehensive Debug + Power Optimization

```
axi5_master_rd_mon_cg #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH     (64),
    .AXI_USER_WIDTH     (4),
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R       (4),
    // Enable all AXI5 features
    .ENABLE_NSaid       (1),
    .ENABLE_TRACE       (1),
    .ENABLE_MPAM        (1),
    .ENABLE_MECID       (1),
    .ENABLE_UNIQUE      (1),
    .ENABLE_CHUNKING    (1),
    .ENABLE_MTE         (1),
    .ENABLE_POISON      (1),
    // Monitor configuration
    .UNIT_ID            (1),
    .AGENT_ID           (10),
    .MAX_TRANSACTIONS   (16),
    .ENABLE_FILTERING   (1),
    // Clock gating configuration
    .CG_IDLE_COUNT_WIDTH (4)
) u_axi5_master_rd_mon_cg (
    .aclk                (axi_clk),
    .aresetn              (axi_rst_n),

    // Clock gating configuration
    .cfg_cg_enable        (power_save_enable),
    .cfg_cg_idle_count    (4'd3), // Gate after 4 idle cycles

    // FUB and Master interfaces
    // ... (connect AXI5 signals)

    // Monitor configuration - FUNCTIONAL DEBUG MODE
    .cfg_monitor_enable   (1'b1), // Completions
    .cfg_error_enable     (1'b1), // Errors
    .cfg_timeout_enable   (1'b1), // Timeouts
    .cfg_perf_enable      (1'b0), // DISABLED
    .cfg_timeout_cycles   (16'd1000),
    .cfg_latency_threshold (32'd500),

    // Filtering configuration
```

```

.cfg_axi_pkt_mask    (16'h0007),    // ERROR|COMPL|TIMEOUT
.cfg_axi_err_select (16'h0001),
.cfg_axi_error_mask (16'hFFFF),
.cfg_axi_timeout_mask (16'hFFFF),
.cfg_axi_compl_mask (16'hFFFF),

// Monitor bus
.monbus_valid        (mon_valid),
.monbus_ready        (mon_ready),
.monbus_packet       (mon_pkt),

// Status outputs
.busy                (master_busy),
.active_transactions (active_trans),
.cfg_conflict_error  (cfg_error),
.cg_gating           (clock_gated),
.cg_idle             (interface_idle)
);

// Monitor packet consumer with power-aware handling
always_ff @(posedge axi_clk or negedge axi_rst_n) begin
    if (!axi_rst_n) begin
        mon_pkt_count <= '0;
        power_cycles_saved <= '0;
    end else begin
        // Count monitor packets
        if (mon_valid && mon_ready)
            mon_pkt_count <= mon_pkt_count + 1;

        // Track power savings
        if (clock_gated)
            power_cycles_saved <= power_cycles_saved + 1;
    end
end

// Alert on configuration conflicts
assert property (@(posedge axi_clk) disable iff (!axi_rst_n)
    !cfg_error
) else $error("Monitor configuration conflict detected!");

// Downstream FIFO for monitor packets
gaxi_fifo_sync #(
    .DATA_WIDTH (64),
    .DEPTH      (256)
) u_mon_fifo (
    .i_clk      (axi_clk),
    .i_rst_n    (axi_rst_n),

```

```

        .i_valid      (mon_valid),
        .i_data       (mon_pkt),
        .o_ready      (mon_ready),
        .o_valid      (fifo_valid),
        .o_data       (fifo_pkt),
        .i_ready      (consumer_ready)
    );

```

19.8 Design Notes

19.8.1 When to Use This Module

Ideal for: - Battery-powered or power-sensitive systems - Bursty traffic patterns with significant idle periods - Systems requiring comprehensive debug visibility - Production systems needing runtime monitoring + power optimization

Consider alternatives if: - Continuous, high-throughput traffic (clock gating ineffective) - Ultra-low latency requirements (avoid gating overhead) - Area-constrained designs (monitor + CG adds ~10% area)

19.8.2 Configuration Strategy Matrix

Scenario	Monitor Config	Clock Gating Config	Expected Benefit
Development/Debug	ERROR + COMPL + TIMEOUT	Disabled	Full visibility, no wake latency
Performance Tuning	ERROR + PERF	Disabled	Metrics without power interference
Power Testing	ERROR only	Aggressive (count=0)	Maximum power savings
Production	ERROR + TIMEOUT	Balanced (count=3)	Error detection + power savings

19.8.3 Monitor + Clock Gating Interaction

Key Considerations:

- Monitor packets generated before gating:**
 - Monitor processes all events before idle state
 - MonBus packets flushed before clock stops
 - No events lost during gating transition

2. **Wake latency includes monitor initialization:**
 - 1-2 cycles for clock ungating
 - Monitor ready immediately (no reset needed)
 - First transaction monitored correctly
3. **Power savings account for monitor overhead:**
 - Monitor adds ~10% dynamic power when active
 - Clock gating saves ~80-90% when idle
 - Net savings: 70-80% during idle periods

19.8.4 Verification Recommendations

1. **Test monitor + CG independently first:**

```
// Phase 1: Monitor only (CG disabled)  
.cfg_cg_enable (1'b0)  
// Verify all monitor features  
  
// Phase 2: CG only (monitor minimal)  
.cfg_monitor_enable (1'b0)  
.cfg_error_enable   (1'b0)  
.cfg_cg_enable      (1'b1)  
// Verify clock gating behavior  
  
// Phase 3: Combined  
// Both enabled, verify interaction
```

2. **Check monitor packet integrity across gating:**
 - No packets lost during gating transitions
 - No duplicate packets
 - Timestamps/latencies accurate
 3. **Measure actual power savings:**
 - Instrument with power counters
 - Compare vs. always-on baseline
 - Verify efficiency in target traffic patterns
-

19.9 Related Documentation

- [AXI5 Master Read](#) - Base module
- [AXI5 Master Read CG](#) - Clock gating only

- [AXI5 Master Read Monitor](#) - Monitor only
 - [AXI5 Master Write Monitor CG](#) - Write variant
 - [AXI Monitor Configuration Guide](#) - Monitor setup
 - [AMBA Clock Gate Controller](#) - Clock gating details
-

19.10 Navigation

- [← Back to AXI5 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

20 AXI5 Master Read with Monitor

Module: `axi5_master_rd_mon.sv` **Location:** `rtl/amba/axi5/` **Status:** Production Ready

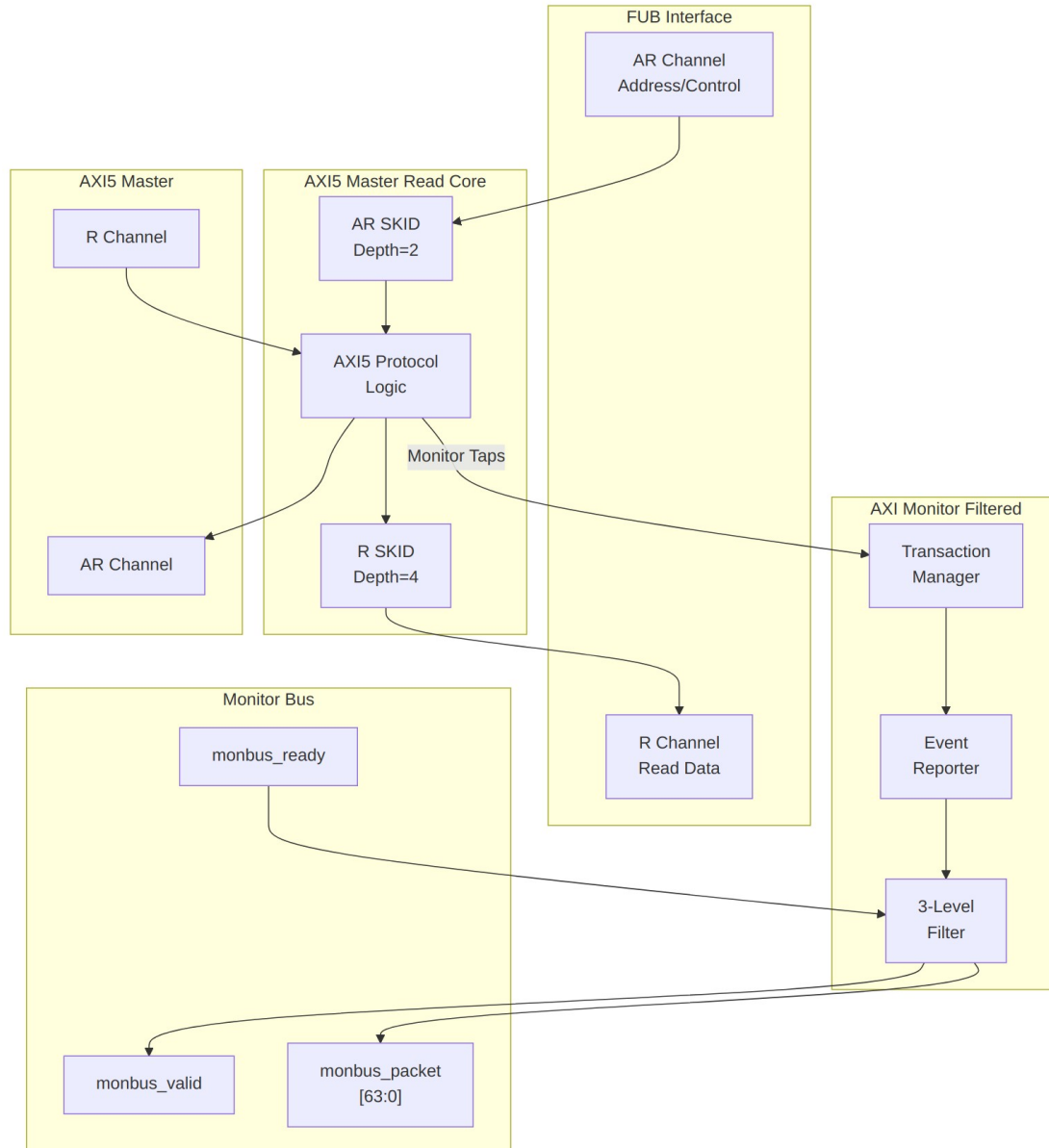
20.1 Overview

The AXI5 Master Read with Monitor module combines the standard `axi5_master_rd` core with an integrated `axi_monitor_filtered` for comprehensive transaction monitoring and error detection. This module provides real-time visibility into AXI5 read operations with configurable packet filtering.

20.1.1 Key Features

- Full AMBA AXI5 protocol compliance (wraps `axi5_master_rd`)
- **ARNSAID:** Non-secure access identifier for security domains
- **ARTRACE:** Trace signal for debug and performance monitoring
- **ARMPAM:** Memory Partitioning and Monitoring (PartID + PMG)
- **ARMECID:** Memory Encryption Context ID for secure memory
- **ARUNIQUE:** Unique ID indicator for cache operations
- **ARCHUNKEN:** Read data chunking enable for partial data transfers
- **ARTAGOP:** Memory tag operation (MTE - Memory Tagging Extension)
- **RTRACE:** Read data trace signal
- **RPOISON:** Data poison indicator for corrupted data detection
- **RCHUNKV/RCHUNKNUM/RCHUNKSTRB:** Chunking control signals
- **RTAG/RTAGMATCH:** Memory tags and tag match response (MTE)

- **Integrated AXI monitor** with 3-level filtering hierarchy
 - **Error detection:** Protocol violations, SLVERR, DECERR
 - **Timeout monitoring:** Stuck transactions, stalled channels
 - **Performance metrics:** Latency, throughput, outstanding transactions
 - **MonBus output:** Standardized 128-bit monitor packet format paired with 64-bit side-band timestamp
 - **Configuration validation:** Detects filter conflicts
-



Module Architecture

20.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel SKID buffer depth
SKID_DEPTH_R	int	4	R channel SKID buffer

Parameter	Type	Default	Description
			depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_WSTRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width (PartID + PMG)
AXI_MECID_WIDTH	int	16	Memory encryption context ID width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
AXI_CHUNKNUM_WIDTH	int	4	Chunk number width
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption context
ENABLE_UNIQUE	bit	1	Enable unique ID indicator
ENABLE_CHUNKING	bit	1	Enable data chunking
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
UNIT_ID	int	1	Monitor unit identifier

Parameter	Type	Default	Description
AGENT_ID	int	10	Monitor agent identifier
MAX_TRANSACTION	int	16	Transaction table size
ENABLE_FILTERING	bit	1	Enable 3-level packet filtering
ADD_PIPELINE_STAGE	bit	0	Add pipeline stage in monitor (latency vs. timing)
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
N_ADDR_RANGES	int	0	Number of address-range comparators. 0 = checker omitted (zero area). >0 = N independent [low, high] ranges; hits emit PktTypeError + AXI_ERR_ADDR_RANGE on monbus.
ENABLE_ERROR_LOGIC	bit	1	Compile-in the error-detection cone (0 drops it for area).
ENABLE_TIMEOUT_LOGIC	bit	1	Compile-in the timeout cone and the axi_monitor_timeout instance.
ENABLE_COMPLETION_LOGIC	bit	1	Compile-in the completion cone.
ENABLE_THRESHOLD_LOGIC	bit	1	Compile-in the threshold cone.

Parameter	Type	Default	Description
ENABLE_PERF_LOG IC	bit	1	Compile-in the perfmon measurement window and utilization/throughput counters.
ENABLE_DEBUG_LOG IC	bit	0	Compile-in the debug cone (off by default).

Synthesis-cone note: the six ENABLE_*_LOGIC parameters gate each detection cone at synthesis via generate-if, so unused logic drops to zero area (classic cones default on, debug off). Inside the wrapper's axi_monitor_filtered instance the perf/debug master switches are fixed (ENABLE_PERF_PACKETS = 1, ENABLE_DEBUG_MODULE = 0). The former CAM_PIPELINE / TRANS_CAM_PIPELINE parameters were removed — the transaction CAM is now always pipelined.

20.3 Monitor Backpressure (block_ready)

The monitor exposes a block_ready signal that goes low when its internal FIFO is saturated and cannot accept a new in-flight transaction. The wrapper ANDs block_ready into the upstream-facing fub_axi_arready so a saturated monitor stalls new transactions on the wire instead of dropping events.

- **Where the stall lands:** the upstream fub_axi_arready is forced low until the monitor drains.
- **When USE_MONITOR=0:** block_ready is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.
- **For axi5 slave variants** (only applies to axi5_slave_rd_mon): the monitor watches the FUB-side handshake, so there is a SKID_DEPTH_AR cycle lag between block_ready going low and new events ceasing. MAX_TRANSACTIONS should be sized to cover this margin.

This replaces a previous bug where block_ready was left unconnected and a full monitor FIFO would silently lose events.

20.4 Address-Range Checker

The wrapper can be parameterized with `N_ADDR_RANGES > 0` to instantiate an N-comparator address-range checker that watches every accepted AR handshake and emits a `PktTypeError` monbus packet (event code `AXI_ERR_ADDR_RANGE = 8'h0D`) when an address falls inside any of the configured `[low, high]` inclusive ranges.

Config inputs (active only when `N_ADDR_RANGES > 0`): - `cfg_addr_check_enable` — master on/off for the checker. - `cfg_addr_range_enable[N-1:0]` — per-range enable bit. - `cfg_addr_range_low[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive low bound for each range. - `cfg_addr_range_high[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive high bound for each range.

Event encoding (within the standard 128-bit `monitor_packet_t`, `event_data` field): - `packet_type = PktTypeError (4'h0)` - `protocol = AXI (3'b000)` - `event_code = AXI_ERR_ADDR_RANGE (8'h0D)` - `event_data[63:60] = range_index (4 bits; supports up to 16 ranges)` - `event_data[59:0] = full matched address (up to 60 bits, zero-padded if narrower)`

Exact match: set `cfg_addr_range_low[i] == cfg_addr_range_high[i]`.

Filtering: the existing `cfg_axi_error_mask[13]` bit masks this event code; set it high to suppress range packets without disabling other errors. No new mask wiring needed.

Per-range coalescing: if a range hits again before its packet has been emitted, the latched address is overwritten (latest hit wins). One packet per cycle drains the pending mask via a lowest-index priority encoder. Distinct ranges never lose events; under sustained per-range bursts, only the latest address per range is reported per emission cycle.

20.5 Ports

20.5.1 Clock and Reset

Port	Width	Direction	Description
<code>aclk</code>	1	Input	AXI clock
<code>aresetn</code>	1	Input	AXI active-low reset

20.5.2 FUB AXI5 Interface (Slave Side)

Same as `axi5_master_rd` - see [AXI5 Master Read](#) for complete port listing.

20.5.3 Master AXI5 Interface (Output Side)

Same as `axi5_master_rd` - see [AXI5 Master Read](#) for complete port listing.

20.5.4 Monitor Configuration

Port	Width	Direction	Description
cfg_monitor_enable	1	Input	Enable completion packet generation
cam_clear	1	Input	Synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4])
cfg_error_enable	1	Input	Enable error packet generation
cfg_timeout_enable	1	Input	Enable timeout packet generation
cfg_performance_enable	1	Input	Enable performance packet generation (see Performance Monitoring)
cfg_completion_enable	1	Input	Enable transaction-completion packets
cfg_threshold_enable	1	Input	Enable threshold-crossed packets
cfg_debug_enable	1	Input	Enable debug/trace packets (gates the debug cone — the 6th reporter sub-block)
cfg_timeout_cycles	16	Input	Timeout threshold (clock cycles)
cfg_latency_threshold	32	Input	Latency threshold for performance alerts

Detection-cone enables: `cfg_completion_enable`, `cfg_threshold_enable`, and `cfg_debug_enable` turn on the completion, threshold, and debug reporter sub-blocks respectively. `cfg_debug_enable` gates the **debug cone** — the 6th reporter sub-block (`axi_monitor_reporter_debug`). In this wrapper the debug module's `cfg_debug_level` (4), `cfg_debug_mask` (16), and `cfg_active_trans_threshold` (16) inputs are tied to their defaults (4'h0 / 16'h0 / 16'd8) and are not exposed as wrapper ports.

20.5.5 AXI Protocol Filtering Configuration

Port	Width	Direction	Description
cfg_axi_pkt_mask	16	Input	Packet type filter (Level 1)
cfg_axi_err_select	16	Input	Error routing configuration (Level 2)
cfg_axi_err_or_mask	16	Input	Error event filter (Level 3)
cfg_axi_timeout_mask	16	Input	Timeout event filter (Level 3)
cfg_axi_completion_mask	16	Input	Completion event filter (Level 3)
cfg_axi_threshold_mask	16	Input	Threshold event filter (Level 3)
cfg_axi_performance_mask	16	Input	Performance event filter (Level 3)
cfg_axi_address_mask	16	Input	Address event filter (Level 3)
cfg_axi_debug_mask	16	Input	Debug event filter (Level 3)

20.5.6 Monitor Bus Output

Port	Width	Direction	Description
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor packet ready (backpressure)
monbus_packet	128	Output	monitor_packet_t (see format below)
monbus_timestamp	64	Output	monbus_timestamp_t paired atomically with monbus_packet
i_mon_time	64	Input	Free-running counter from monbus_axil_group, sampled at packet

Port	Width	Direction	Description
			emission

20.5.7 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Core busy indicator
active_transactions	8	Output	Number of outstanding transactions
error_count	16	Output	Cumulative error count (placeholder)
transaction_count	32	Output	Total transaction count (placeholder)
cfg_conflict_error	1	Output	Configuration conflict detected

20.6 Performance Monitoring

When performance tracking is compiled in (`ENABLE_PERF_LOGIC = 1`, the wrapper default, with `ENABLE_PERF_PACKETS` fixed to 1 inside the monitor instance) and `cfg_perf_enable` is asserted at runtime, the monitor runs a **measurement-window state machine** plus a bank of data-channel utilization counters. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows so the host can read a completed window's totals.

20.6.1 The measurement window

A window is opened by a **start event** and closed by an **end event**. The event sources are selected by `cfg_start_event_sel` / `cfg_end_event_sel` (3-bit; e.g. 3'b010 selects the `cfg_perf_enable` edge) and can also be fired directly by the `cfg_start_trigger` / `cfg_end_trigger` pulses (from an engine or CSR write). `cfg_window_force_close` is a software override that closes the window immediately. While the window is open:

- `window_active` is high.
- `window_cycles` free-runs, counting every clock elapsed inside the window.

20.6.2 Utilization buckets (R data channel)

Every cycle inside the window is classified by the **R** data channel's valid/ready handshake into exactly one of four buckets:

Output	Width	Condition	Meaning
perf_prod_cycles	32	rvalid && rready	productive beat transferred
perf_bp_cycles	32	rvalid && !rready	back-pressure (data offered, sink not ready)
perf_starv_cycles	32	!rvalid && rready	starvation (sink ready, no data)
perf_idle_cycles	32	!rvalid && !rready	idle

The four buckets sum to window_cycles, so utilization = perf_prod_cycles / window_cycles.

20.6.3 Throughput counters

Output	Width	Meaning
perf_beat_count	32	R data beats transferred (= perf_prod_cycles, 1 beat/cycle)
perf_byte_count	64	bytes transferred = beats x (1 << latched ARSIZE), using the ARSIZE captured at the most recent AR address phase
perf_burst_count	32	AR address-phase handshakes

Average burst length is perf_beat_count / perf_burst_count.

20.6.4 Performance Monitoring Ports

Port	Width	Direction	Description
cfg_perf_enable	1	Input	Enable performance-metric packet generation (also listed under Monitor Configuration)
cfg_start_event_sel	3	Input	Window start event source select
cfg_end_event_sel	3	Input	Window end event source

Port	Width	Direction	Description
ent_sel			select
cfg_start_trigger	1	Input	Pulse: open the measurement window
cfg_end_trigger	1	Input	Pulse: close the measurement window
cfg_window_force_close	1	Input	Software override: force the window closed
window_active	1	Output	High while a measurement window is open
window_cycles	32	Output	Cycles elapsed in the current window
perf_prod_cycles	32	Output	rvalid && rready cycles
perf_bp_cycles	32	Output	rvalid && !rready cycles (back-pressure)
perf_starv_cycles	32	Output	!rvalid && rready cycles (starvation)
perf_idle_cycles	32	Output	!rvalid && !rready cycles
perf_beat_count	32	Output	R data beats transferred
perf_byte_count	64	Output	bytes transferred
perf_burst_count	32	Output	AR address-phase handshakes

When `USE_MONITOR = 0` (or `ENABLE_PERF_LOGIC = 0`) all perfmon outputs are tied to 0.

20.7 Functionality

20.7.1 Monitor Bus Packet Format

The 128-bit monbus_packet (paired with the 64-bit monbus_timestamp side-band signal) follows the standardized AMBA monitor bus format:

Bits [127:124] - Packet Type:

0x0 = ERROR	Error events (SLVERR, DECERR, protocol violations)
0x1 = COMPL	Completion events (transaction finished)
0x2 = THRESH	Threshold events
0x3 = TIMEOUT	Timeout events
0x4 = PERF	Performance metrics
0x8 = ADDR_MATCH	Address match events
0x9 = APB	APB-specific events
0xF = DEBUG	Debug events

Bits [123:109] - Reserved (15 bits, forward-compat slack)

Bits [108:105] - Protocol (4 bits): 0x0=AXI, 0x1=AXIS, 0x2=APB, 0x3=ARB, 0x4=CORE

Bits [104:97] - Event Code (8 bits, protocol-specific)

Bits [96:88] - Channel ID (9 bits – AXI ID or channel index)

Bits [87:72] - Agent ID (16 bits, from AGENT_ID parameter)

Bits [71:64] - Unit ID (8 bits, from UNIT_ID parameter)

Bits [63:0] - Event Data (64 bits – full address, latency, etc.)

20.7.2 Three-Level Filtering Hierarchy

Level 1: Packet Type Mask (cfg_axi_pkt_mask)

cfg_axi_pkt_mask[0] = 1 → Enable ERROR packets
cfg_axi_pkt_mask[1] = 1 → Enable COMPL packets
cfg_axi_pkt_mask[2] = 1 → Enable TIMEOUT packets
cfg_axi_pkt_mask[3] = 1 → Enable THRESH packets
cfg_axi_pkt_mask[4] = 1 → Enable PERF packets
cfg_axi_pkt_mask[5] = 1 → Enable ADDR packets
cfg_axi_pkt_mask[6] = 1 → Enable DEBUG packets

Level 2: Error Routing (cfg_axi_err_select)

Determines whether errors generate ERROR packets or COMPL packets with error status.

****Level 3: Event Masks (cfg_axi_*_mask)****

Fine-grained control over specific events within each packet type: - cfg_axi_error_mask: SLVERR, DECERR, orphan detection, etc. - cfg_axi_timeout_mask: AR timeout, R timeout, response timeout - cfg_axi_compl_mask: Normal completion, error completion - cfg_axi_thresh_mask: Outstanding transaction count, latency threshold - cfg_axi_perf_mask: Average latency, peak bandwidth, utilization - cfg_axi_addr_mask: Address range tracking - cfg_axi_debug_mask: Internal state, debug events

20.7.3 Error Detection Events

The monitor detects and reports:

Protocol Errors: - AR handshake violations - R handshake violations - ID width mismatches - Burst length violations - Unaligned addresses

Response Errors: - SLVERR (slave error response) - DECERR (decode error response) - Orphaned read data (no matching AR) - ID mismatch (RID != ARID)

Timeout Errors: - AR channel stall (no ARREADY) - R channel stall (no RVALID) - Transaction timeout (AR to RLAST)

Threshold Violations: - Outstanding transaction count > threshold - Transaction latency > cfg_latency_threshold

20.7.4 Configuration Conflict Detection

The cfg_conflict_error output flags invalid configurations:

```
// Example conflict: COMPL and PERF both enabled
if (cfg_monitor_enable && cfg_perf_enable)
    cfg_conflict_error = 1; // Packet congestion risk!
```

Recommended: Enable only ONE high-traffic packet type at a time.

20.8 Timing Diagrams

20.8.1 Monitored Read Transaction with Error

TODO: Wavedrom timing diagram showing:

- ACLK
- AR channel: ARID, ARADDR, ARVALID, ARREADY
- R channel: RID, RDATA, RRESP (SLVERR), RVALID, RREADY
- Monitor bus: monbus_valid, monbus_packet showing ERROR packet
- Event sequence: AR → R error → ERROR packet generated

20.8.2 Timeout Detection

TODO: Wavedrom timing diagram showing:

- ACLK
- AR channel: ARVALID asserted, ARREADY stuck low
- Timeout counter incrementing
- cfg_timeout_cycles threshold
- Monitor bus: monbus_valid, monbus_packet showing TIMEOUT packet

20.8.3 Performance Monitoring

TOD0: Wavedrom timing diagram showing:

- Multiple read transactions
 - Latency measurement (AR to RLAST)
 - Monitor bus: PERF packets with latency data
 - Threshold comparison with `cfg_latency_threshold`
-

20.9 Usage Example

20.9.1 Functional Verification Configuration

```
axi5_master_rd_mon #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH    (64),
    .AXI_USER_WIDTH    (4),
    .SKID_DEPTH_AR     (2),
    .SKID_DEPTH_R      (4),
    // Enable AXI5 features
    .ENABLE_NSAID      (1),
    .ENABLE_TRACE      (1),
    .ENABLE_MPAM       (1),
    .ENABLE_MECID      (1),
    .ENABLE_UNIQUE     (1),
    .ENABLE_CHUNKING   (1),
    .ENABLE_MTE        (1),
    .ENABLE_POISON     (1),
    // Monitor configuration
    .UNIT_ID           (1),
    .AGENT_ID          (10),
    .MAX_TRANSACTIONS  (16),
    .ENABLE_FILTERING  (1)
) u_axi5_master_rd_mon (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // FUB and Master interfaces
    // ... (connect AXI5 signals)

    // Monitor configuration - FUNCTIONAL DEBUG MODE
    .cfg_monitor_enable (1'b1),           // Enable completions
    .cfg_error_enable   (1'b1),           // Enable errors
    .cfg_timeout_enable (1'b1),           // Enable timeouts
    .cfg_perf_enable    (1'b0),           // DISABLE (high traffic)
    .cfg_timeout_cycles (16'd1000),       // 1000 cycle timeout
```

```

.cfg_latency_threshold (32'd500), // 500 cycle threshold

// Level 1: Enable ERROR, COMPL, TIMEOUT packets
.cfg_axi_pkt_mask (16'h0007), // [2:0] = ERROR|COMPL|TIMEOUT

// Level 2: Route errors to ERROR packets
.cfg_axi_err_select (16'h0001), // Generate ERROR packets

// Level 3: Enable all error events
.cfg_axi_error_mask (16'hFFFF), // All errors
.cfg_axi_timeout_mask (16'hFFFF), // All timeouts
.cfg_axi_compl_mask (16'hFFFF), // All completions

// Monitor bus output
.monbus_valid (mon_valid),
.monbus_ready (mon_ready),
.monbus_packet (mon_pkt),

// Status
.busy (master_busy),
.active_transactions (active_trans),
.cfg_conflict_error (cfg_error)
);

// Downstream FIFO for monitor packets
gaxi_fifo_sync #(
    .DATA_WIDTH (64),
    .DEPTH (256)
) u_mon_fifo (
    .i_clk (axi_clk),
    .i_rst_n (axi_rst_n),
    .i_valid (mon_valid),
    .i_data (mon_pkt),
    .o_ready (mon_ready),
    .o_valid (fifo_valid),
    .o_data (fifo_pkt),
    .i_ready (consumer_ready)
);

```

20.9.2 Performance Analysis Configuration

```

// Monitor configuration - PERFORMANCE MODE
.cfg_monitor_enable (1'b0), // DISABLE completions
.cfg_error_enable (1'b1), // Keep errors enabled
.cfg_timeout_enable (1'b0), // DISABLE timeouts
.cfg_perf_enable (1'b1), // ENABLE performance
.cfg_timeout_cycles (16'd1000),

```

```
.cfg_latency_threshold (32'd500),

// Level 1: Enable ERROR and PERF packets only
.cfg_axi_pkt_mask    (16'h0011),    // ERROR | PERF

// Level 3: Enable performance metrics
.cfg_axi_error_mask (16'hFFFF),    // All errors
.cfg_axi_perf_mask  (16'hFFFF),    // All perf metrics
```

20.10 Design Notes

20.10.1 Configuration Best Practices

CRITICAL: Never enable COMPL + PERF simultaneously!

High packet volume can overwhelm monitor bus and cause backpressure.

Recommended Configurations:

Use Case	Enabled Packet Types	Notes
Functional debug	ERROR + COMPL + TIMEOUT	Catches bugs, monitors correctness
Performance tuning	ERROR + PERF	Latency, throughput metrics
Protocol compliance	ERROR + TIMEOUT	Minimal overhead
Deep debug	ERROR + DEBUG	Internal state visibility

20.10.2 Transaction Table Sizing

MAX_TRANSACTION must accommodate maximum outstanding reads:

AXI Configuration	Recommended MAX_TRANSACTION
Single outstanding	4

AXI Configuration	Recommended MAX_TRANSACTIONS
Moderate pipelining	16
Deep pipelining	32-64
High-performance	128+

20.10.3 Filtering Strategy

Start broad, then narrow: 1. Enable all packet types initially 2. Identify high-traffic events 3. Mask low-value events with Level 3 filters 4. Separate configurations for different test phases

20.10.4 Monitor Bus Backpressure

The monitor respects monbus_ready backpressure: - Packets buffered internally when monbus_ready = 0 - Oldest packets dropped if buffer full - **Always add downstream FIFO** for robustness

20.11 Related Documentation

- [AXI5 Master Read](#) - Base module without monitoring
- [AXI5 Master Read CG](#) - With clock gating only
- [AXI5 Master Read Monitor CG](#) - Monitor + clock gating
- [AXI Monitor Filtered](#) - Monitor core specification
- [Monitor Package Spec](#) - Packet format details
- [AXI Monitor Configuration Guide](#) - Complete configuration reference

20.12 Navigation

- [← Back to AXI5 Index](#)
- [← Back to RTLAmBa Index](#)
- [← Back to Main Documentation Index](#)

21 AXI5 Master Write with Monitor and Clock Gating

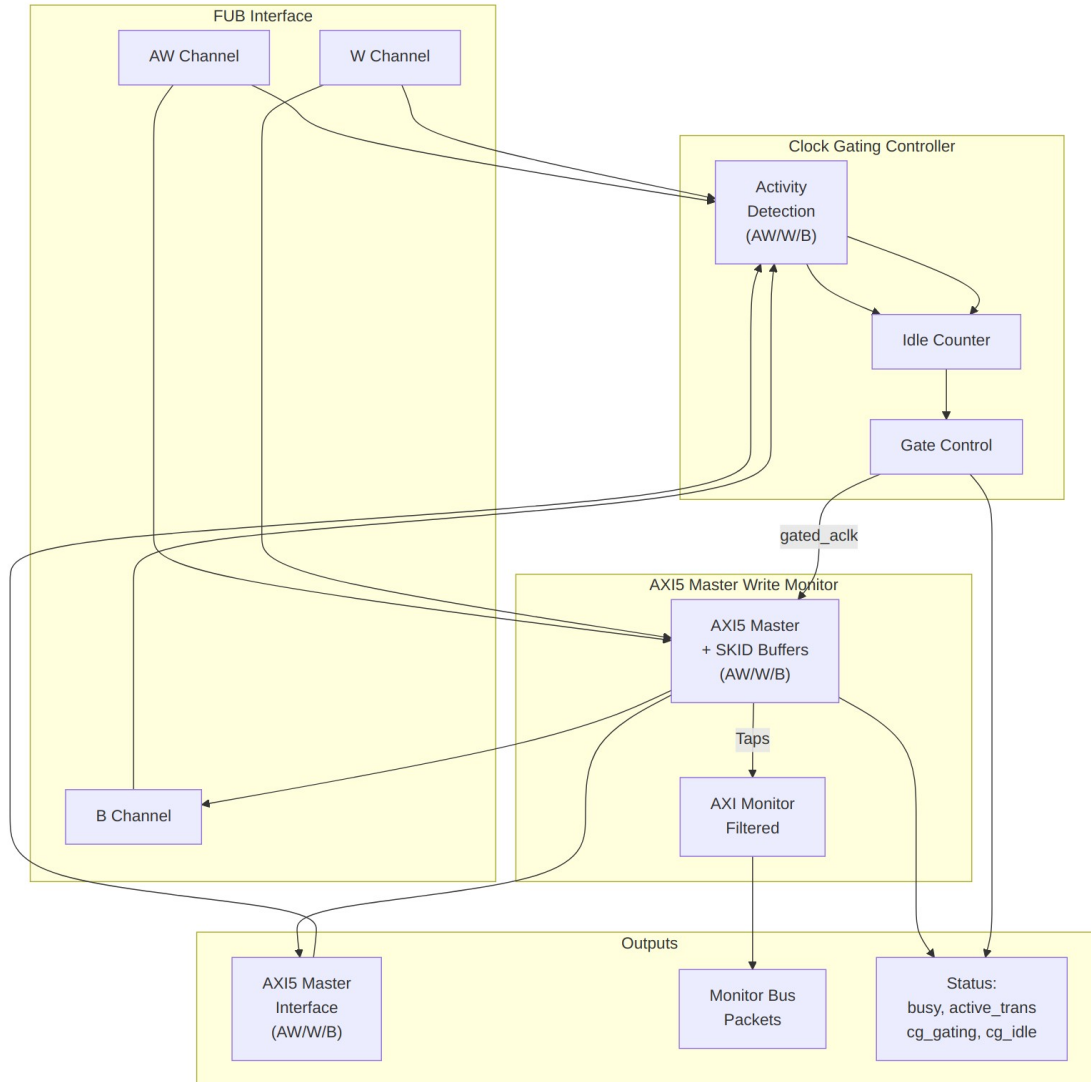
Module: axi5_master_wr_mon_cg.sv **Location:** rtl/amba/axi5/ **Status:** Production Ready

21.1 Overview

The AXI5 Master Write with Monitor and Clock Gating module combines `axi5_master_wr_mon` (AXI5 write master with integrated monitoring) with intelligent clock gating for power optimization. This provides comprehensive write transaction monitoring, error detection, and automatic power management.

21.1.1 Key Features

- Full AMBA AXI5 protocol compliance
 - **All AXI5 write extensions:** ATOP, NSAIID, TRACE, MPAM, MECID, UNIQUE, MTE, POISON
 - **Integrated AXI monitor** with 3-level filtering hierarchy
 - **Error detection:** Protocol violations, SLVERR, DECERR
 - **Timeout monitoring:** Stuck transactions, stalled channels
 - **Performance metrics:** Latency, throughput, outstanding transactions
 - **MonBus output:** Standardized 128-bit monitor packet format paired with 64-bit side-band timestamp
 - **Automatic clock gating** based on write activity detection
 - **Configurable idle threshold** before clock gating activates
 - **Power savings** during idle periods
 - **Transparent operation** - no protocol changes
 - **Dual status outputs:** Monitor status + clock gating status
-



Module Architecture

21.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel SKID buffer depth
SKID_DEPTH_W	int	4	W channel SKID buffer depth
SKID_DEPTH_B	int	2	B channel SKID buffer

Parameter	Type	Default	Description
			depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_ATOP_WIDTH	int	6	Atomic operation width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width (PartID + PMG)
AXI_MECID_WIDTH	int	16	Memory encryption context ID width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
ENABLE_ATOMIC	bit	1	Enable atomic operations
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption context
ENABLE_UNIQUE	bit	1	Enable unique ID indicator
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
UNIT_ID	int	1	Monitor unit identifier
AGENT_ID	int	11	Monitor agent identifier (default 11 for write)

Parameter	Type	Default	Description
MAX_TRANSACTION_S	int	16	Transaction table size
ENABLE_FILTERING	bit	1	Enable 3-level packet filtering
ADD_PIPELINE_STAGE	bit	0	Add pipeline stage in monitor
USE_MONITOR	bit	1	Synthesis-time monitor enable (forwarded to inner monitor).
N_ADDR_RANGES	int	0	Number of address-range comparators (forwarded to base module).
ENABLE_ERROR_LOGIC	bit	1	Synthesis-cone enable for error detection (forwarded to base module)
ENABLE_TIMEOUT_LOGIC	bit	1	Synthesis-cone enable for timeout detection (forwarded to base module)
ENABLE_COMPL_LOGIC	bit	1	Synthesis-cone enable for completion tracking (forwarded to base module)
ENABLE_THRESHOLD_LOGIC	bit	1	Synthesis-cone enable for threshold detection (forwarded to base module)
ENABLE_PERF_LOGIC	bit	1	Synthesis-cone enable for the perfmon window + counters (forwarded to base module)
ENABLE_DEBUG_LOG	bit	0	Synthesis-cone enable

Parameter	Type	Default	Description
GIC			for the debug/trace cone (forwarded to base module)
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (max 2^N-1 cycles)

21.3 Ports

21.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock (ungated)
aresetn	1	Input	AXI active-low reset

21.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Clock gating enable (1=enable, 0=always active)
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating activates

21.3.3 FUB AXI5 Interface (Slave Side)

Same as axi5_master_wr_mon - see [AXI5 Master Write Monitor](#).

21.3.4 Master AXI5 Interface (Output Side)

Same as axi5_master_wr_mon - see [AXI5 Master Write Monitor](#).

21.3.5 Monitor Configuration

Same as axi5_master_wr_mon - see [AXI5 Master Write Monitor](#).

21.3.6 Monitor Bus Output

Port	Width	Direction	Description
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor packet ready (backpressure)
monbus_packet	128	Output	monitor_packet_t (128-bit format)
monbus_timestamp	64	Output	monbus_timestamp_t paired atomically with monbus_packet
i_mon_time	64	Input	Free-running counter from monbus_axil_group, sampled at packet emission

21.3.7 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Core busy indicator
active_transactions	8	Output	Number of outstanding write transactions
error_count	16	Output	Cumulative error count (placeholder)
transaction_count	32	Output	Total transaction count (placeholder)
cfg_conflict_error	1	Output	Configuration conflict detected
cg_gating	1	Output	Clock gating active (1=gated, 0=running)
cg_idle	1	Output	Interface idle (1=no activity)

21.4 Functionality

21.4.1 Combined Write Monitoring and Power Management

This module provides the ultimate combination for AXI5 write paths:

Monitoring (from axi5_master_wr_mon): - Real-time write transaction monitoring (AW/W/B) - Error and timeout detection - Performance metrics collection - Atomic operation tracking - Configurable packet filtering - MonBus output for system integration

Power Management (from axi5_master_wr_cg): - Automatic clock gating during write idle periods - Configurable idle threshold - Activity-based wake-up - Status outputs for power monitoring

21.4.2 Clock Gating with Write Monitor Active

The clock gating logic considers all three write channels plus monitor activity:

```
user_valid = fub_axi_awvalid || fub_axi_wvalid || fub_axi_bready ||
int_busy;
axi_valid = m_axi_awvalid || m_axi_wvalid || m_axi_bvalid;
```

```
// Clock remains active if:
// - AW channel active (address phase)
// - W channel active (data phase)
// - B channel active (response phase)
// - Internal busy (transactions in flight)
// - Monitor has pending packets (implicit in int_busy)
```

The monitor continues operating during clock gating transitions, ensuring no write events are lost.

21.4.3 Ready Signal Control During Gating

When clock gating is active, all write channel ready signals are forced low:

```
fub_axi_awready = cg_gating ? 1'b0 : int_awready;
fub_axi_wready = cg_gating ? 1'b0 : int_wready;
m_axi_bready = cg_gating ? 1'b0 : int_bready;
```

This ensures protocol compliance during power management across all write phases.

21.5 Performance Monitoring

The clock-gated wrapper exposes the full perfmon interface of the base module and **forwards every port unchanged** to the inner axi5_master_wr_mon. The measurement-window state

machine, the four W-channel utilization buckets (productive / back-pressure / starvation / idle), and the beat/byte/burst throughput counters behave exactly as documented in the base module — see [Performance Monitoring in axi5_master_wr_mon](#) for the full narrative and per-bit semantics.

Forwarded perfmon ports (identical width and direction to the base module):

- **Inputs:** `cfg_perf_enable`, `cfg_start_event_sel` (3), `cfg_end_event_sel` (3), `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Outputs:** `window_active`, `window_cycles` (32), `perf_prod_cycles` (32), `perf_bp_cycles` (32), `perf_starv_cycles` (32), `perf_idle_cycles` (32), `perf_beat_count` (32), `perf_byte_count` (64), `perf_burst_count` (32)

The `perf_burst_count` output tracks AW (write address) handshakes. Clock gating never suppresses these paths: while a measurement window is open the monitor stays awake, so window cycle accounting remains exact regardless of the idle-count setting.

Alongside perfmon, the wrapper forwards the `cfg_compl_enable`, `cfg_threshold_enable`, and `cfg_debug_enable` control inputs, plus the six `ENABLE_*_LOGIC` synthesis-cone parameters (see [Parameters](#)), straight through to the base monitor.

21.6 Timing Diagrams

21.6.1 Clock Gating with Write Burst and Monitoring

TODO: Wavedrom timing diagram showing:

- `ACLK` (ungated)
- `GATED_ACLK` (gated clock)
- AW channel: `AWVALID`, `AWREADY`
- W channel: `WDATA`, `WLAST`, `WVALID`, `WREADY`
- B channel: `BVALID`, `BREADY`
- Monitor packets generated
- Idle period detection after B phase
- `cg_gating` activation
- Monitor quiescent before clock stops

21.6.2 Atomic Operation with Monitor + Clock Gating

TODO: Wavedrom timing diagram showing:

- `AWATOP` encoding
- Atomic write sequence
- Monitor packet for atomic operation
- B response with `BTAGMATCH`
- Clock gating after atomic completion

21.7 Usage Example

21.7.1 Comprehensive Write Debug + Power Optimization

```
axi5_master_wr_mon_cg #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH    (64),
    .AXI_USER_WIDTH    (4),
    .SKID_DEPTH_AW     (2),
    .SKID_DEPTH_W      (4),
    .SKID_DEPTH_B      (2),
    // Enable all AXI5 write features
    .ENABLE_ATOMIC     (1),
    .ENABLE_NSAID      (1),
    .ENABLE_TRACE      (1),
    .ENABLE_MPAM        (1),
    .ENABLE_MECID      (1),
    .ENABLE_UNIQUE     (1),
    .ENABLE_MTE         (1),
    .ENABLE_POISON     (1),
    // Monitor configuration
    .UNIT_ID           (1),
    .AGENT_ID          (11), // Write agent
    .MAX_TRANSACTIONS  (16),
    .ENABLE_FILTERING  (1),
    // Clock gating configuration
    .CG_IDLE_COUNT_WIDTH (4)
) u_axi5_master_wr_mon_cg (
    .aclk              (axi_clk),
    .aresetn           (axi_rst_n),

    // Clock gating configuration
    .cfg_cg_enable     (power_save_enable),
    .cfg_cg_idle_count (4'd3), // Gate after 4 idle cycles

    // FUB and Master interfaces
    // ... (connect all AXI5 signals - AW, W, B channels)

    // Monitor configuration - FUNCTIONAL DEBUG MODE
    .cfg_monitor_enable (1'b1), // Completions
    .cfg_error_enable   (1'b1), // Errors
    .cfg_timeout_enable  (1'b1), // Timeouts
    .cfg_perf_enable     (1'b0), // DISABLED
    .cfg_timeout_cycles (16'd1000),
```

```

.cfg_latency_threshold (32'd800), // Higher for writes

// Filtering configuration
.cfg_axi_pkt_mask (16'h0007), // ERROR|COMPL|TIMEOUT
.cfg_axi_err_select (16'h0001),
.cfg_axi_error_mask (16'hFFFF),
.cfg_axi_timeout_mask (16'hFFFF),
.cfg_axi_compl_mask (16'hFFFF),

// Monitor bus
.monbus_valid (mon_valid),
.monbus_ready (mon_ready),
.monbus_packet (mon_pkt),

// Status outputs
.busy (master_busy),
.active_transactions (active_trans),
.cfg_conflict_error (cfg_error),
.cg_gating (clock_gated),
.cg_idle (interface_idle)
);

// Monitor packet consumer with write-specific handling
logic [31:0] write_count, write_errors, write_timeouts;
logic [63:0] total_write_latency;

always_ff @(posedge axi_clk or negedge axi_rst_n) begin
    if (!axi_rst_n) begin
        write_count <= '0;
        write_errors <= '0;
        write_timeouts <= '0;
        total_write_latency <= '0;
        power_cycles_saved <= '0;
    end else begin
        // Process monitor packets (128-bit monitor_packet_t)
        if (mon_valid && mon_ready) begin
            case (mon_pkt[127:124]) // Packet type
                4'h0: begin // ERROR
                    write_errors <= write_errors + 1;
                    $display("Write Error: EvtCode=%h, ChanID=%h,
EvtData=%h",
                        mon_pkt[104:97], mon_pkt[96:88],
mon_pkt[63:0]);
                end
                4'h1: begin // COMPL
                    write_count <= write_count + 1;
                    total_write_latency <= total_write_latency +

```

```

mon_pkt[63:0];
    end
    4'h2: begin // THRESH (was TIMEOUT in old layout)
        // see monitor_common_pkg for current packet-type
mapping
    end
    4'h3: begin // TIMEOUT
        write_timeouts <= write_timeouts + 1;
        $display("Write Timeout: EvtCode=%h, ChanID=%h",
            mon_pkt[104:97], mon_pkt[96:88]);
    end
endcase
end

    // Track power savings
    if (clock_gated)
        power_cycles_saved <= power_cycles_saved + 1;
    end
end

// Calculate average write latency
logic [31:0] avg_write_latency;
assign avg_write_latency = (write_count > 0) ?
    (total_write_latency / write_count) : 32'd0;

// Alert on configuration conflicts
assert property (@(posedge axi_clk) disable iff (!axi_rst_n)
    !cfg_error
) else $error("Write monitor configuration conflict detected!");

// Downstream FIFO for monitor packets
gaxi_fifo_sync #(
    .DATA_WIDTH (64),
    .DEPTH      (256)
) u_mon_fifo (
    .i_clk      (axi_clk),
    .i_rst_n    (axi_rst_n),
    .i_valid    (mon_valid),
    .i_data     (mon_pkt),
    .o_ready    (mon_ready),
    .o_valid    (fifo_valid),
    .o_data     (fifo_pkt),
    .i_ready    (consumer_ready)
);

```

21.8 Design Notes

21.8.1 When to Use This Module

Ideal for: - Battery-powered systems with write-intensive workloads - Systems with bursty write traffic and significant idle periods - Designs requiring comprehensive write path visibility - Production systems needing runtime monitoring + power optimization - Write-heavy cache implementations - DMA engines with intermittent activity

Consider alternatives if: - Continuous streaming writes (clock gating ineffective) - Ultra-low write latency requirements (avoid gating overhead) - Area-constrained designs (monitor + CG adds ~12% area) - Simple write-through caches (simpler monitoring sufficient)

21.8.2 Write-Specific Configuration Strategy

Write Path Characteristics:

Aspect	Typical Value	Impact on Configuration
Write latency	10-100 cycles	Set higher cfg_latency_threshold
Burst length	4-16 beats	Longer monitoring windows
Response delay	5-50 cycles	Longer idle count before gating
Atomic operations	20-200 cycles	Extended timeout thresholds

Recommended Configuration:

```
// For write path (vs. read path adjustments)
.cfg_cg_idle_count      (4'd5),      // Longer than read (B latency)
.cfg_timeout_cycles     (16'd2000), // 2x read timeout
.cfg_latency_threshold  (32'd800)    // Higher variance than reads
```

21.8.3 Monitor + Clock Gating Interaction for Writes

Key Write-Specific Considerations:

1. Three-channel coordination:

- Monitor tracks AW, W, B phases independently
- All three must be idle before gating
- Longer idle detection time than read (2 channels)

2. Write buffering impact:

- Posted writes may delay B responses
- Clock gating may activate between W and B
- Monitor ensures B responses still tracked after wake

3. Atomic operation handling:

- Atomic ops have read+write phases
- Longer latency prevents premature gating
- Monitor tracks full atomic sequence

4. Power savings accounting:

- Writes typically 60-70% traffic vs. reads (system dependent)
- Write-specific gating saves 15-30% total power
- Combined with read path CG: 40-60% total savings

21.8.4 Verification Recommendations for Write Path

1. Test write burst + gating:

```
// Phase 1: Short bursts with gating between
for (int i = 0; i < 100; i++) begin
    write_burst(addr, len=4);
    wait_idle_cycles(10); // Allow gating
    check_monitor_packets();
end
```

2. Test atomic operations:

```
// Verify atomic ops don't get interrupted by gating
atomic_compare_swap(addr, old_val, new_val);
check_no_premature_gating();
verify_atomic_completion_monitored();
```

3. Test write errors with gating:

```
// Trigger SLVERR, verify monitored correctly
write_to_protected_address();
wait_error_packet();
check_gating_after_error_handled();
```

21.8.5 Performance Analysis with Write Monitoring

The monitor provides valuable write path metrics:

```
// Extract write performance from monitor packets
always_comb begin
    write_bandwidth = (write_count * data_width) / elapsed_cycles;
```

```

        write_efficiency = (active_write_cycles) / total_cycles;
        avg_outstanding = total_outstanding_samples / sample_count;
end

// Compare with/without clock gating
$display("Write Perf: BW=%d GB/s, Efficiency=%d%%, Power saved=%d%%",
        write_bandwidth, write_efficiency,
        (power_cycles_saved * 100) / total_cycles);

```

21.9 Related Documentation

- [AXI5 Master Write](#) - Base module
 - [AXI5 Master Write CG](#) - Clock gating only
 - [AXI5 Master Write Monitor](#) - Monitor only
 - [AXI5 Master Read Monitor CG](#) - Read variant
 - [AXI Monitor Configuration Guide](#) - Monitor setup
 - [AMBA Clock Gate Controller](#) - Clock gating details
-

21.10 Navigation

- [← Back to AXI5 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

22 AXI5 Master Write with Monitor

Module: axi5_master_wr_mon.sv **Location:** rtl/amba/axi5/ **Status:** Production Ready

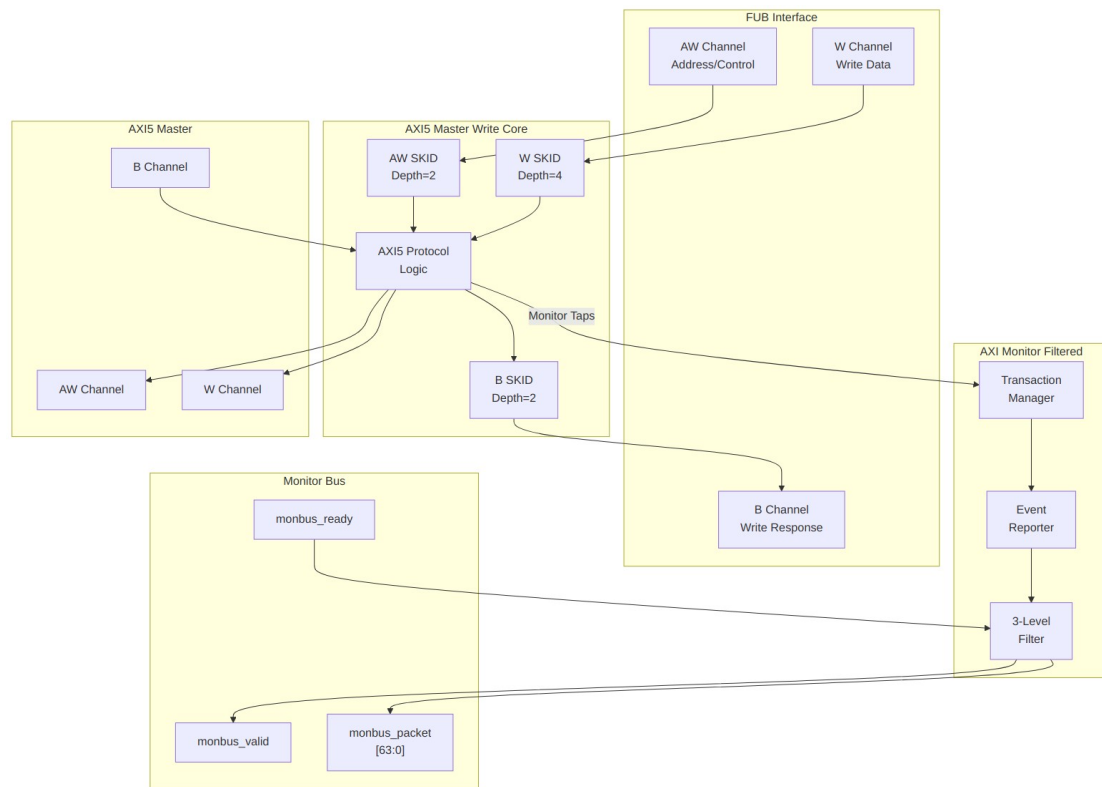
22.1 Overview

The AXI5 Master Write with Monitor module combines the standard axi5_master_wr core with an integrated axi_monitor_filtered for comprehensive write transaction monitoring and error detection. This module provides real-time visibility into AXI5 write operations with configurable packet filtering.

22.1.1 Key Features

- Full AMBA AXI5 protocol compliance (wraps axi5_master_wr)

- **AWATOP:** Atomic operation support (compare-and-swap, atomic operations)
 - **AWNSAID:** Non-secure access identifier for security domains
 - **AWTRACE:** Trace signal for debug and performance monitoring
 - **AWMPAM:** Memory Partitioning and Monitoring (PartID + PMG)
 - **AWMECID:** Memory Encryption Context ID for secure memory
 - **AWUNIQUE:** Unique ID indicator for cache operations
 - **AWTAGOP:** Memory tag operation (MTE - Memory Tagging Extension)
 - **AWTAG:** Memory tags on address channel
 - **WPOISON:** Write data poison indicator for corrupted data detection
 - **WTAG/WTAGUPDATE:** Memory tags and tag update control (MTE)
 - **BTRACE/BTAG/BTAGMATCH:** Response trace and tag signals
 - **Integrated AXI monitor** with 3-level filtering hierarchy
 - **Error detection:** Protocol violations, SLVERR, DECERR
 - **Timeout monitoring:** Stuck transactions, stalled channels
 - **Performance metrics:** Latency, throughput, outstanding transactions
 - **MonBus output:** Standardized 128-bit monitor packet format paired with 64-bit side-band timestamp
 - **Configuration validation:** Detects filter conflicts
-



Module Architecture

22.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel SKID buffer depth
SKID_DEPTH_W	int	4	W channel SKID buffer depth
SKID_DEPTH_B	int	2	B channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_WSTRB_WIDTH	int	DATA_WIDTH/8	Write strobe width

Parameter	Type	Default	Description
			(calculated)
AXI_ATOP_WIDTH	int	6	Atomic operation width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width (PartID + PMG)
AXI_MECID_WIDTH	int	16	Memory encryption context ID width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
ENABLE_ATOMIC	bit	1	Enable atomic operations
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption context
ENABLE_UNIQUE	bit	1	Enable unique ID indicator
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
UNIT_ID	int	1	Monitor unit identifier
AGENT_ID	int	11	Monitor agent identifier (default 11 for write)
MAX_TRANSACTION	int	16	Transaction table size
ENABLE_FILTERING	bit	1	Enable 3-level packet filtering

Parameter	Type	Default	Description
ADD_PIPELINE_STAGE	bit	0	Add pipeline stage in monitor
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
N_ADDR_RANGES	int	0	Number of address-range comparators. 0 = checker omitted (zero area). >0 = N independent [low, high] ranges; hits emit PktTypeError + AXI_ERR_ADDR_RANGE on monbus.
ENABLE_ERROR_LOGIC	bit	1	Compile-in the error-detection cone (0 drops it for area).
ENABLE_TIMEOUT_LOGIC	bit	1	Compile-in the timeout cone and the axi_monitor_timeout instance.
ENABLE_COMPL_LOGIC	bit	1	Compile-in the completion cone.
ENABLE_THRESHOLD_LOGIC	bit	1	Compile-in the threshold cone.
ENABLE_PERF_LOGIC	bit	1	Compile-in the perfmon measurement window and utilization/throughput counters.
ENABLE_DEBUG_LOGIC	bit	0	Compile-in the debug cone (off by default).

Synthesis-cone note: the six `ENABLE_*_LOGIC` parameters gate each detection cone at synthesis via generate-if, so unused logic drops to zero area (classic cones default on, debug off). Inside the wrapper's `axi_monitor_filtered` instance the perf/debug master switches are fixed (`ENABLE_PERF_PACKETS = 1`, `ENABLE_DEBUG_MODULE = 0`). The former `CAM_PIPELINE / TRANS_CAM_PIPELINE` parameters were removed — the transaction CAM is now always pipelined.

22.3 Monitor Backpressure (`block_ready`)

The monitor exposes a `block_ready` signal that goes low when its internal FIFO is saturated and cannot accept a new in-flight transaction. The wrapper ANDs `block_ready` into the upstream-facing `fub_axi_awready` so a saturated monitor stalls new transactions on the wire instead of dropping events.

- **Where the stall lands:** the upstream `fub_axi_awready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`:** `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.
- **For axi5 slave variants** (only applies to `axi5_slave_wr_mon`): the monitor watches the FUB-side handshake, so there is a `SKID_DEPTH_AW` cycle lag between `block_ready` going low and new events ceasing. `MAX_TRANSACTIONS` should be sized to cover this margin.

This replaces a previous bug where `block_ready` was left unconnected and a full monitor FIFO would silently lose events.

22.4 Address-Range Checker

The wrapper can be parameterized with `N_ADDR_RANGES > 0` to instantiate an N-comparator address-range checker that watches every accepted AW handshake and emits a `PktTypeError` monbus packet (event code `AXI_ERR_ADDR_RANGE = 8'h0D`) when an address falls inside any of the configured [`low`, `high`] inclusive ranges.

Config inputs (active only when `N_ADDR_RANGES > 0`): - `cfg_addr_check_enable` — master on/off for the checker. - `cfg_addr_range_enable[N-1:0]` — per-range enable bit. - `cfg_addr_range_low[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive low bound for each range. - `cfg_addr_range_high[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive high bound for each range.

Event encoding (within the standard 128-bit `monitor_packet_t`, `event_data` field): -
`packet_type` = `PktTypeError` (4'h0) - `protocol` = `AXI` (3'b000) - `event_code` =
`AXI_ERR_ADDR_RANGE` (8'h0D) - `event_data[63:60]` = `range_index` (4 bits; supports up to 16
ranges) - `event_data[59:0]` = full matched address (up to 60 bits, zero-padded if narrower)

Exact match: set `cfg_addr_range_low[i] == cfg_addr_range_high[i]`.

Filtering: the existing `cfg_axi_error_mask[13]` bit masks this event code; set it high to
suppress range packets without disabling other errors. No new mask wiring needed.

Per-range coalescing: if a range hits again before its packet has been emitted, the latched address
is overwritten (latest hit wins). One packet per cycle drains the pending mask via a lowest-index
priority encoder. Distinct ranges never lose events; under sustained per-range bursts, only the
latest address per range is reported per emission cycle.

22.5 Ports

22.5.1 Clock and Reset

Port	Width	Direction	Description
<code>aclk</code>	1	Input	AXI clock
<code>aresetn</code>	1	Input	AXI active-low reset

22.5.2 FUB AXI5 Interface (Slave Side)

Same as `axi5_master_wr` - see [AXI5 Master Write](#) for complete port listing.

22.5.3 Master AXI5 Interface (Output Side)

Same as `axi5_master_wr` - see [AXI5 Master Write](#) for complete port listing.

22.5.4 Monitor Configuration

Same as `axi5_master_rd_mon` - see [AXI5 Master Read Monitor](#) for complete configuration
port listing. This includes the `cam_clear` control input (1, Input) - synchronous clear of the
monitor transaction CAM (driven from the harness clear control bit, e.g. `CTRL[4]`).

The configuration set also includes the detection-cone enables `cfg_compl_enable`,
`cfg_threshold_enable`, and `cfg_debug_enable` (each 1, Input) which turn on the completion,
threshold, and debug reporter sub-blocks. `cfg_debug_enable` gates the **debug cone** — the 6th
reporter sub-block (`axi_monitor_reporter_debug`). In this wrapper the debug module's
`cfg_debug_level` (4), `cfg_debug_mask` (16), and `cfg_active_trans_threshold` (16) inputs
are tied to their defaults (4'h0 / 16'h0 / 16'd8) and are not exposed as wrapper ports.

22.5.5 Monitor Bus Output

Port	Width	Direction	Description
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor packet ready (backpressure)
monbus_packet	128	Output	monitor_packet_t (see format below)
monbus_timestamp	64	Output	monbus_timestamp_t paired atomically with monbus_packet
i_mon_time	64	Input	Free-running counter from monbus_axil_group, sampled at packet emission

22.5.6 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Core busy indicator
active_transactions	8	Output	Number of outstanding write transactions
error_count	16	Output	Cumulative error count (placeholder)
transaction_count	32	Output	Total transaction count (placeholder)
cfg_conflict_error	1	Output	Configuration conflict detected

22.6 Performance Monitoring

When performance tracking is compiled in (`ENABLE_PERF_LOGIC = 1`, the wrapper default, with `ENABLE_PERF_PACKETS` fixed to 1 inside the monitor instance) and `cfg_perf_enable` is asserted at runtime, the monitor runs a **measurement-window state machine** plus a bank of data-channel utilization counters. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows so the host can read a completed window's totals.

22.6.1 The measurement window

A window is opened by a **start event** and closed by an **end event**. The event sources are selected by `cfg_start_event_sel` / `cfg_end_event_sel` (3-bit; e.g. 3'b010 selects the `cfg_perf_enable` edge) and can also be fired directly by the `cfg_start_trigger` / `cfg_end_trigger` pulses (from an engine or CSR write). `cfg_window_force_close` is a software override that closes the window immediately. While the window is open:

- `window_active` is high.
- `window_cycles` free-runs, counting every clock elapsed inside the window.

22.6.2 Utilization buckets (W data channel)

Every cycle inside the window is classified by the **W** data channel's valid/ready handshake into exactly one of four buckets:

Output	Width	Condition	Meaning
<code>perf_prod_cycles</code>	32	<code>wvalid && wready</code>	productive beat transferred
<code>perf_bp_cycles</code>	32	<code>wvalid && !wready</code>	back-pressure (data offered, sink not ready)
<code>perf_starv_cycles</code>	32	<code>!wvalid && wready</code>	starvation (sink ready, no data)
<code>perf_idle_cycles</code>	32	<code>!wvalid && !wready</code>	idle

The four buckets sum to `window_cycles`, so $\text{utilization} = \text{perf_prod_cycles} / \text{window_cycles}$.

22.6.3 Throughput counters

Output	Width	Meaning
<code>perf_beat_count</code>	32	W data beats transferred (= <code>perf_prod_cycles</code> , 1 beat/cycle)
<code>perf_byte_count</code>	64	bytes transferred = beats x (1 << latched <code>AWSIZE</code>), using the <code>AWSIZE</code> captured at the most recent AW address phase
<code>perf_burst_count</code>	32	AW address-phase

Output	Width	Meaning
		handshakes

Average burst length is `perf_beat_count / perf_burst_count`. (Transaction completion is still tracked on the B channel; the throughput counters measure the W data phase.)

22.6.4 Performance Monitoring Ports

Port	Width	Direction	Description
<code>cfg_perf_enable</code>	1	Input	Enable performance-metric packet generation (also listed under Monitor Configuration)
<code>cfg_start_event_sel</code>	3	Input	Window start event source select
<code>cfg_end_event_sel</code>	3	Input	Window end event source select
<code>cfg_start_trigger</code>	1	Input	Pulse: open the measurement window
<code>cfg_end_trigger</code>	1	Input	Pulse: close the measurement window
<code>cfg_window_force_close</code>	1	Input	Software override: force the window closed
<code>window_active</code>	1	Output	High while a measurement window is open
<code>window_cycles</code>	32	Output	Cycles elapsed in the current window
<code>perf_prod_cycles</code>	32	Output	<code>wvalid && wready</code> cycles
<code>perf_bp_cycles</code>	32	Output	<code>wvalid && !wready</code> cycles (back-pressure)
<code>perf_starv_cycles</code>	32	Output	<code>!wvalid && wready</code> cycles (starvation)
<code>perf_idle_cycles</code>	32	Output	<code>!wvalid && !wready</code> cycles

Port	Width	Direction	Description
perf_beat_count	32	Output	W data beats transferred
perf_byte_count	64	Output	bytes transferred
perf_burst_count	32	Output	AW address-phase handshakes

When USE_MONITOR = 0 (or ENABLE_PERF_LOGIC = 0) all perfmon outputs are tied to 0.

22.7 Functionality

22.7.1 Monitor Bus Packet Format

Same 128-bit standardized format (with 64-bit side-band timestamp) as read monitor - see [AXI5 Master Read Monitor](#).

22.7.2 Write-Specific Monitoring Events

Write Transaction Tracking: The monitor correlates three channels: 1. **AW:** Address and control (AWID, AWADDR, AWLEN, etc.) 2. **W:** Write data beats (WDATA, WSTRB, WLAST) 3. **B:** Write response (BID, BRESP)

Event Timeline:

1. AW handshake → Transaction started
2. W beats → Data transfer monitored
3. WLAST → Write data complete
4. B response → Transaction complete

Error Detection Events:

Protocol Errors: - AW handshake violations - W handshake violations - Missing WLAST - ID width mismatches - Burst length violations - Unaligned addresses - Strobe violations

Response Errors: - SLVERR (slave error response) - DECERR (decode error response) - Orphaned write response (no matching AW) - ID mismatch (BID != AWID)

Timeout Errors: - AW channel stall (no AWREADY) - W channel stall (no WREADY) - B channel stall (no BVALID) - Transaction timeout (AW to B)

Data Integrity: - WPOISON asserted (corrupted data) - Tag mismatch (MTE) - Atomic operation failures

Threshold Violations: - Outstanding transaction count > threshold - Transaction latency > `cfg_latency_threshold`

22.7.3 Atomic Operation Monitoring

When `ENABLE_ATOMIC=1`, the monitor tracks: - Atomic operation type (from `AWATOP`) - Atomic sequence (read-modify-write) - Success/failure (from `BRESP`) - Tag validation (`BTAGMATCH`)

22.7.4 Three-Level Filtering Hierarchy

Same as read monitor - see [AXI5 Master Read Monitor](#).

22.8 Timing Diagrams

22.8.1 Monitored Write Burst with Error

TOD0: Wavedrom timing diagram showing:

- `ACLK`
- AW channel: `AWID`, `AWADDR`, `AWLEN`, `AWVALID`, `AWREADY`
- W channel: `WDATA`, `WSTRB`, `WLAST`, `WVALID`, `WREADY`
- B channel: `BID`, `BRESP` (`SLVERR`), `BVALID`, `BREADY`
- Monitor bus: `monbus_valid`, `monbus_packet` showing `ERROR` packet

22.8.2 Atomic Operation Monitoring

TOD0: Wavedrom timing diagram showing:

- `AWATOP` encoding (atomic type)
- AW/W channels
- B response with `BTAG`
- Monitor packets for atomic sequence

22.8.3 Write Timeout Detection

TOD0: Wavedrom timing diagram showing:

- AW channel: `AWVALID` asserted, `AWREADY` stuck low
 - Timeout counter incrementing
 - `cfg_timeout_cycles` threshold
 - Monitor bus: `TIMEOUT` packet generated
-

22.9 Usage Example

22.9.1 Functional Verification Configuration

```
axi5_master_wr_mon #(
    .AXI_ID_WIDTH      (8),
```

```

.AXI_ADDR_WIDTH      (32),
.AXI_DATA_WIDTH      (64),
.AXI_USER_WIDTH      (4),
.SKID_DEPTH_AW       (2),
.SKID_DEPTH_W        (4),
.SKID_DEPTH_B        (2),
// Enable AXI5 features
.ENABLE_ATOMIC       (1),
.ENABLE_NSaid        (1),
.ENABLE_TRACE        (1),
.ENABLE_MPAM         (1),
.ENABLE_MECID        (1),
.ENABLE_UNIQUE       (1),
.ENABLE_MTE          (1),
.ENABLE_POISON       (1),
// Monitor configuration
.UNIT_ID             (1),
.AGENT_ID             (11), // Write agent
.MAX_TRANSACTIONS    (16),
.ENABLE_FILTERING    (1)
) u_axi5_master_wr_mon (
    .aclk              (axi_clk),
    .aresetn           (axi_rst_n),

    // FUB interface (slave side)
    .fub_axi_awid      (fub_awid),
    .fub_axi_awaddr    (fub_awaddr),
    .fub_axi_awlen     (fub_awlen),
    .fub_axi_awsz      (fub_awsz),
    .fub_axi_awburst   (fub_awburst),
    .fub_axi_awlock    (fub_awlock),
    .fub_axi_awcache   (fub_awcache),
    .fub_axi_awprot    (fub_awprot),
    .fub_axi_awqos     (fub_awqos),
    .fub_axi_awuser    (fub_awuser),
    .fub_axi_awvalid   (fub_awvalid),
    .fub_axi_awready   (fub_awready),

    // AXI5 AW extensions
    .fub_axi_awatop    (fub_atop),
    .fub_axi_awnsaid   (fub_awnsaid),
    .fub_axi_awtrace   (fub_awtrace),
    .fub_axi_awmpam    (fub_awmpam),
    .fub_axi_awmecid   (fub_awmecid),
    .fub_axi_awunique  (fub_awunique),
    .fub_axi_awtagop   (fub_awtagop),
    .fub_axi_awtag     (fub_awtag),

```

```

// W channel
.fub_axi_wdata      (fub_wdata),
.fub_axi_wstrb      (fub_wstrb),
.fub_axi_wlast      (fub_wlast),
.fub_axi_wuser      (fub_wuser),
.fub_axi_wvalid     (fub_wvalid),
.fub_axi_wready     (fub_wready),

// AXI5 W extensions
.fub_axi_wpoison    (fub_wpoison),
.fub_axi_wtag       (fub_wtag),
.fub_axi_wtagupdate (fub_wtagupdate),

// B channel
.fub_axi_bid        (fub_bid),
.fub_axi_bresp      (fub_bresp),
.fub_axi_buser      (fub_buser),
.fub_axi_bvalid     (fub_bvalid),
.fub_axi_bready     (fub_bready),

// AXI5 B extensions
.fub_axi_btrace     (fub_btrace),
.fub_axi_btag       (fub_btag),
.fub_axi_btagmatch  (fub_btagmatch),

// Master interface (output side)
// ... (connect all m_axi_* signals)

// Monitor configuration - FUNCTIONAL DEBUG MODE
.cfg_monitor_enable (1'b1),           // Enable completions
.cfg_error_enable   (1'b1),           // Enable errors
.cfg_timeout_enable (1'b1),           // Enable timeouts
.cfg_perf_enable    (1'b0),           // DISABLE (high traffic)
.cfg_timeout_cycles (16'd1000),       // 1000 cycle timeout
.cfg_latency_threshold (32'd500),     // 500 cycle threshold

// Level 1: Enable ERROR, COMPL, TIMEOUT packets
.cfg_axi_pkt_mask   (16'h0007),       // [2:0] = ERROR|COMPL|TIMEOUT

// Level 2: Route errors to ERROR packets
.cfg_axi_err_select (16'h0001),

// Level 3: Enable all error events
.cfg_axi_error_mask (16'hFFFF),
.cfg_axi_timeout_mask (16'hFFFF),
.cfg_axi_compl_mask (16'hFFFF),

```

```

// Monitor bus output
.monbus_valid      (mon_valid),
.monbus_ready      (mon_ready),
.monbus_packet     (mon_pkt),

// Status
.busy              (master_busy),
.active_transactions (active_trans),
.cfg_conflict_error (cfg_error)
);

// Downstream FIFO for monitor packets
gaxi_fifo_sync #(
    .DATA_WIDTH (64),
    .DEPTH      (256)
) u_mon_fifo (
    .i_clk      (axi_clk),
    .i_rst_n    (axi_rst_n),
    .i_valid     (mon_valid),
    .i_data      (mon_pkt),
    .o_ready     (mon_ready),
    .o_valid     (fifo_valid),
    .o_data      (fifo_pkt),
    .i_ready     (consumer_ready)
);

// Monitor packet decoder for write events
always_ff @(posedge axi_clk) begin
    if (fifo_valid && consumer_ready) begin
        case (fifo_pkt[63:60]) // Packet type
            4'h0: begin // ERROR
                $display("Write Error: Type=%h, ID=%h, Addr=%h",
                    fifo_pkt[56:53], fifo_pkt[52:47], fifo_pkt[34:0]);
            end
            4'h1: begin // COMPL
                $display("Write Complete: ID=%h, Latency=%d",
                    fifo_pkt[52:47], fifo_pkt[34:0]);
            end
            4'h2: begin // TIMEOUT
                $display("Write Timeout: Channel=%h, ID=%h",
                    fifo_pkt[56:53], fifo_pkt[52:47]);
            end
        endcase
    end
end
end

```

22.10 Design Notes

22.10.1 Write vs. Read Monitoring Differences

Aspect	Read Monitor	Write Monitor
Channels monitored	2 (AR, R)	3 (AW, W, B)
Transaction ID	ARID → RID	AWID → BID
Data direction	Slave → Master	Master → Slave
Typical latency	Lower	Higher (write buffers)
Common errors	SLVERR on read data	SLVERR on write response
Atomic ops	Load atomics	Store/RMW atomics

22.10.2 Write Transaction Table Management

The transaction table tracks: - **AW phase**: Allocate entry, store address/control - **W phase**: Count beats, verify WLAST matches AWLEN - **B phase**: Match BID to AWID, record response, deallocate entry

Outstanding write tracking:

Active transactions = (AW issued) - (B received)

22.10.3 Write Latency Measurement

Start: AW handshake (AWVALID && AWREADY) **End**: B handshake (BVALID && BREADY)

Latency: Clock cycles from start to end

Typical write latencies: - Single-beat write: 5-20 cycles - Burst write: 10-100 cycles - Atomic operation: 20-200 cycles (includes read phase)

22.10.4 Configuration Best Practices

Same as read monitor - see [AXI5 Master Read Monitor](#).

Write-specific tip: Writes often have higher latency variance than reads due to write buffering and posted writes. Set `cfg_latency_threshold` higher for write monitors.

22.11 Related Documentation

- [AXI5 Master Write](#) - Base module without monitoring
 - [AXI5 Master Write CG](#) - With clock gating only
 - [AXI5 Master Write Monitor CG](#) - Monitor + clock gating
 - [AXI5 Master Read Monitor](#) - Read variant
 - [AXI Monitor Filtered](#) - Monitor core specification
 - [Monitor Package Spec](#) - Packet format details
 - [AXI Monitor Configuration Guide](#) - Complete configuration reference
-

22.12 Navigation

- [← Back to AXI5 Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

23 AXI5 Slave Read Monitor with Clock Gating

Module: axi5_slave_rd_mon_cg.sv **Location:** rtl/amba/axi5/ **Status:** Production Ready

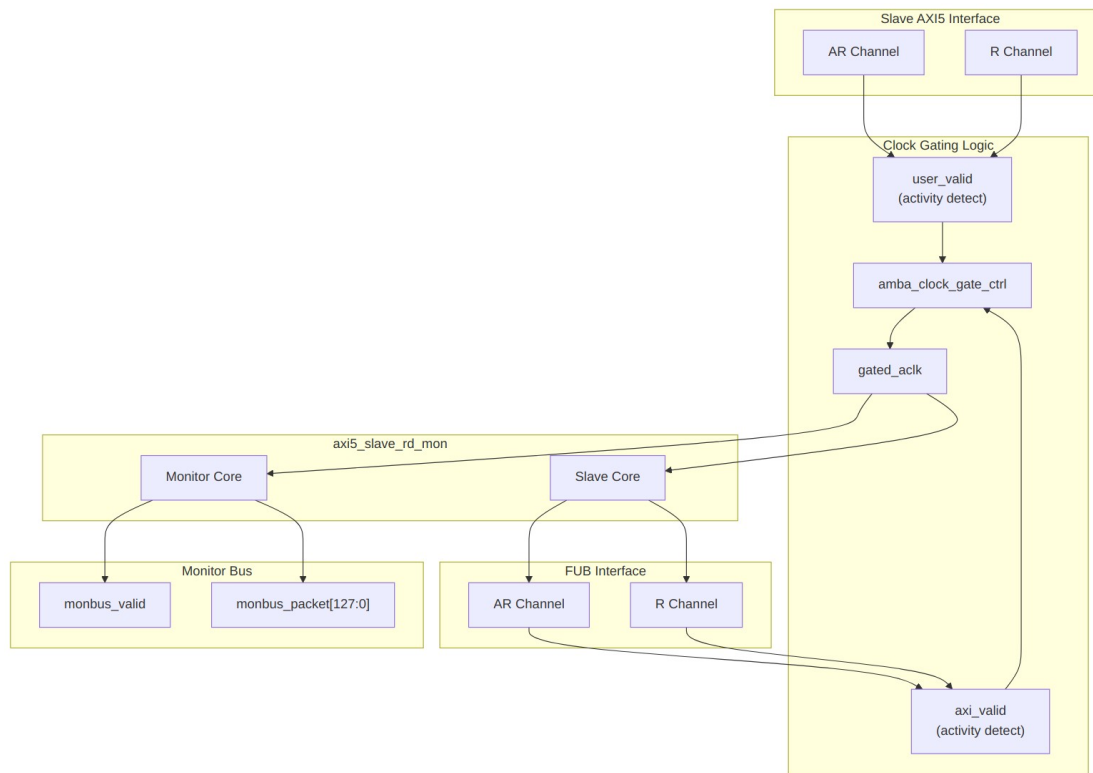
23.1 Overview

The AXI5 Slave Read Monitor with Clock Gating module combines integrated transaction monitoring with automatic clock gating for power optimization. It wraps axi5_slave_rd_mon with clock gating logic.

23.1.1 Key Features

- Full AMBA AXI5 slave read protocol compliance
- **Integrated filtered monitoring** - real-time transaction visibility
- **Integrated clock gating** for dynamic power reduction
- All AXI5 extensions supported (NSAID, TRACE, MPAM, MECID, UNIQUE, CHUNKING, MTE, POISON)
- Configurable idle count before gating
- Transaction tracking with error detection
- Performance metrics and filtering
- Transparent gating - no protocol changes

- Gating status outputs for system monitoring



Module Architecture

23.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel SKID buffer depth
SKID_DEPTH_R	int	4	R channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_NSAID_WIDTH	int	4	Non-secure access ID

Parameter	Type	Default	Description
			width
AXI_MPAM_WIDTH	int	11	MPAM width
AXI_MECID_WIDTH	int	16	Memory encryption context width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
AXI_CHUNKNUM_WIDTH	int	4	Chunk number width
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption
ENABLE_UNIQUE	bit	1	Enable unique ID indicator
ENABLE_CHUNKING	bit	1	Enable data chunking
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
UNIT_ID	int	1	Monitoring unit identifier
AGENT_ID	int	12	Agent identifier
MAX_TRANSACTION_SIZE	int	16	Transaction table size
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add pipeline stage for timing
USE_MONITOR	bit	1	Synthesis-time monitor enable (forwarded to

Parameter	Type	Default	Description
N_ADDR_RANGES	int	0	inner monitor). Number of address-range comparators (forwarded to base module).
ENABLE_ERROR_LO GIC	bit	1	Synthesis-cone enable for error detection (forwarded to base module)
ENABLE_TIMEOUT_ LOGIC	bit	1	Synthesis-cone enable for timeout detection (forwarded to base module)
ENABLE_COMPL_LO GIC	bit	1	Synthesis-cone enable for completion tracking (forwarded to base module)
ENABLE_THRESHOL D_LOGIC	bit	1	Synthesis-cone enable for threshold detection (forwarded to base module)
ENABLE_PERF_LOGI C	bit	1	Synthesis-cone enable for the perfmon window + counters (forwarded to base module)
ENABLE_DEBUG_LO GIC	bit	0	Synthesis-cone enable for the debug/trace cone (forwarded to base module)
CG_IDLE_COUNT_WI DTH	int	4	Clock gating idle counter width

23.3 Ports

23.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock (ungated)
aresetn	1	Input	AXI active-low reset

23.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

23.3.3 Slave AXI5 Interface

Same as axi5_slave_rd - see [AXI5 Slave Read](#) for complete port list.

23.3.4 FUB Interface

Same as axi5_slave_rd - see [AXI5 Slave Read](#) for complete port list.

23.3.5 Monitor Configuration

Same as axi5_slave_rd_mon - see [AXI5 Slave Read Monitor](#) for complete list.

23.3.6 Monitor Bus Output

Port	Width	Direction	Description
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor packet ready
monbus_packet	128	Output	monitor_packet_t (see format below)
monbus_timestamp	64	Output	monbus_timestamp_t paired atomically with monbus_packet

Port	Width	Direction	Description
i_mon_time	64	Input	Free-running counter from monbus_axil_group, sampled at packet emission

23.3.7 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy indicator
active_transactions	8	Output	Current outstanding transactions
error_count	16	Output	Total errors detected
transaction_count	32	Output	Total transactions completed
cfg_conflict_error	1	Output	Configuration conflict detected
cg_gating	1	Output	Clock is currently gated
cg_idle	1	Output	Module is idle

23.4 Functionality

23.4.1 Clock Gating Behavior

Activity Detection: - **user_valid:** Asserted when slave interface has activity (arvalid, rready, or internal busy) - **axi_valid:** Asserted when FUB interface has activity (arvalid, rvalid)

Key Points: - Clock gating disabled when `cfg_cg_enable = 0` - Ready signals forced to 0 when gated (prevents new transactions) - Gating only occurs after configured idle period - Any activity immediately ungates the clock - Monitor continues to track transactions even when gated

23.5 Performance Monitoring

The clock-gated wrapper exposes the full perfmon interface of the base module and **forwards every port unchanged** to the inner `axi5_slave_rd_mon`. The measurement-window state machine, the four R-channel utilization buckets (productive / back-pressure / starvation / idle), and the beat/byte/burst throughput counters behave exactly as documented in the base module — see [Performance Monitoring in axi5_slave_rd_mon](#) for the full narrative and per-bit semantics.

Forwarded perfmon ports (identical width and direction to the base module):

- **Inputs:** `cfg_perf_enable`, `cfg_start_event_sel` (3), `cfg_end_event_sel` (3), `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Outputs:** `window_active`, `window_cycles` (32), `perf_prod_cycles` (32), `perf_bp_cycles` (32), `perf_starv_cycles` (32), `perf_idle_cycles` (32), `perf_beat_count` (32), `perf_byte_count` (64), `perf_burst_count` (32)

The `perf_burst_count` output tracks AR (read address) handshakes. Clock gating never suppresses these paths: while a measurement window is open the monitor stays awake, so window cycle accounting remains exact regardless of the `idle_count` setting.

Alongside perfmon, the wrapper forwards the `cfg_compl_enable`, `cfg_threshold_enable`, and `cfg_debug_enable` control inputs, plus the six `ENABLE_*_LOGIC` synthesis-cone parameters (see [Parameters](#)), straight through to the base monitor.

23.6 Combined Benefits

23.6.1 Power Optimization

Power Savings Estimation: - Base slave logic: ~40% of total power - Monitor logic: ~60% of total power - With gating at 50% duty cycle: ~50% dynamic power savings - Actual savings depend on traffic pattern and `idle_count` setting

23.6.2 Observability

Monitoring Capabilities: - Error detection (SLVERR, timeout, orphan) - Performance tracking (latency, throughput) - Transaction completion tracking - Protocol violation detection - All monitoring continues when ungated

23.7 Usage Example

```
axi5_slave_rd_mon_cg #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH     (64),
    .UNIT_ID           (1),
    .AGENT_ID          (12),
    .MAX_TRANSACTIONS   (16),
    .CG_IDLE_COUNT_WIDTH(4),
    .ENABLE_FILTERING   (1),
    .ENABLE_NSAID       (1),
    .ENABLE_TRACE       (1),
    .ENABLE_MPAM        (1),
    .ENABLE_MECID       (1),
    .ENABLE_UNIQUE      (1),
    .ENABLE_CHUNKING    (1),
    .ENABLE_MTE         (1),
    .ENABLE_POISON      (1)
) u_axi5_slave_rd_mon_cg (
    .aclk                (axi_clk),
    .aresetn             (axi_rst_n),

    // Clock gating config
    .cfg_cg_enable       (1'b1),           // Enable gating
    .cfg_cg_idle_count   (4'd3),           // Gate after 8 idle cycles

    // Slave interface (from external master)
    .s_axi_arid          (s_axi_arid),
    .s_axi_araddr        (s_axi_araddr),
    // ... (connect all slave AR/R signals)

    // FUB interface (to backend)
    .fub_axi_arid        (mem_arid),
    .fub_axi_araddr      (mem_araddr),
    // ... (connect to memory controller)

    // Monitor configuration
    .cfg_monitor_enable  (1'b1),
    .cfg_error_enable    (1'b1),
    .cfg_timeout_enable  (1'b1),
    .cfg_perf_enable     (1'b0),
    .cfg_timeout_cycles  (16'd1000),
    .cfg_latency_threshold (32'd500),
    .cfg_axi_pkt_mask    (16'h0007),

    // Monitor bus
```

```

        .monbus_valid      (mon_valid),
        .monbus_ready     (mon_ready),
        .monbus_packet    (mon_packet),

        // Status
        .busy              (slave_rd_busy),
        .active_transactions(active_txns),
        .error_count       (total_errors),
        .transaction_count (total_txns),
        .cfg_conflict_error (cfg_conflict),

        // Clock gating status
        .cg_gating         (slave_rd_gating),
        .cg_idle           (slave_rd_idle)
    );

    // Power management integration
    assign system_power_save = slave_rd_gating &&
                               slave_wr_gating;

    // Monitor packet handling
    gaxi_fifo_sync #(.DATA_WIDTH(64), .DEPTH(256)) u_mon_fifo (
        .i_clk      (axi_clk),
        .i_rst_n    (axi_rst_n),
        .i_valid     (mon_valid),
        .i_data      (mon_packet),
        .o_ready     (mon_ready),
        .o_valid     (fifo_valid),
        .o_data      (fifo_data),
        .i_ready     (consumer_ready)
    );

```

23.8 Design Notes

23.8.1 When to Use This Module

Ideal for: - Power-constrained systems needing visibility - Debug/validation builds with power budgets - Systems with sporadic transaction patterns - SoC integration requiring both monitoring and power optimization

Avoid when: - Interface is continuously active (minimal gating opportunities) - Monitoring overhead unacceptable (use non-monitored version) - Deterministic latency critical (use high idle count or disable gating)

23.8.2 Configuration Recommendations

Low-Power Debug Mode:

```
.cfg_cg_enable      (1'b1),  
.cfg_cg_idle_count  (4'd1),    // Aggressive gating  
.cfg_monitor_enable (1'b1),    // Full monitoring  
.cfg_error_enable   (1'b1),  
.cfg_perf_enable    (1'b0)    // Reduce packet traffic
```

Performance Analysis Mode:

```
.cfg_cg_enable      (1'b1),  
.cfg_cg_idle_count  (4'd4),    // Conservative gating  
.cfg_monitor_enable (1'b0),    // Disable completions  
.cfg_perf_enable    (1'b1)    // Enable performance metrics
```

23.8.3 Area and Timing Impact

- **Area:** ~5-8% increase over non-monitored slave (monitoring + clock gating)
 - **Timing:** Clock gating adds <50ps, monitoring is off critical path
 - **Power:** Net savings depends on activity factor (typically 30-50% at 50% duty cycle)
-

23.9 Related Documentation

- [AXI5 Slave Read](#) - Non-monitored, non-gated version
 - [AXI5 Slave Read CG](#) - Clock-gated without monitoring
 - [AXI5 Slave Read Monitor](#) - Monitored without clock gating
 - [AXI5 Slave Write Monitor CG](#) - Write variant
 - [AMBA Clock Gate Control](#) - Clock gating controller
-

23.10 Navigation

- [← Back to AXI5 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

24 AXI5 Slave Read Monitor

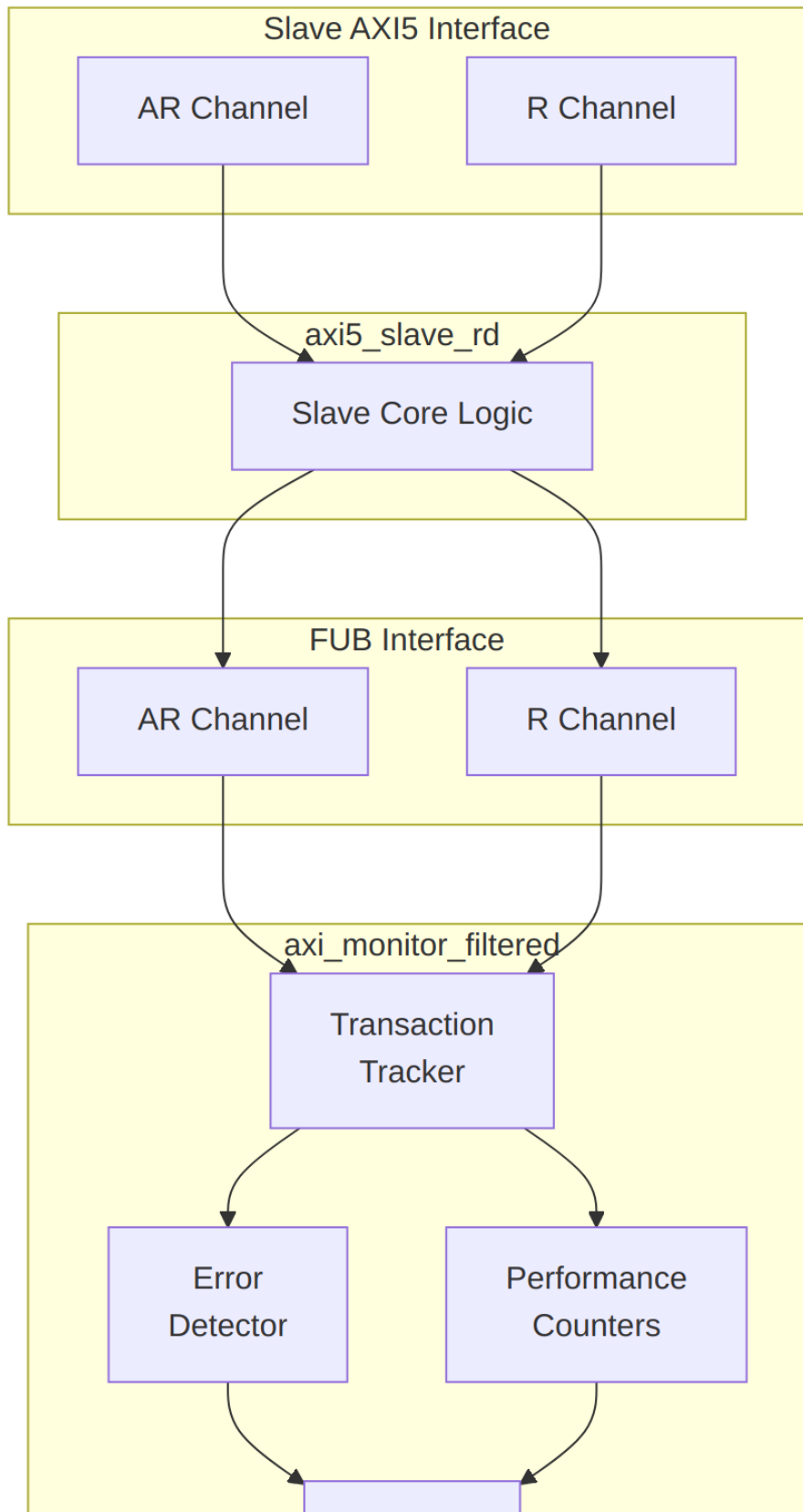
Module: axi5_slave_rd_mon.sv **Location:** rtl/amba/axi5/ **Status:** Production Ready

24.1 Overview

The AXI5 Slave Read Monitor module combines the `axi5_slave_rd` interface with integrated transaction monitoring. It provides real-time visibility into slave read operations with configurable packet filtering and error detection.

24.1.1 Key Features

- Full AMBA AXI5 slave read protocol compliance
 - **Integrated filtered monitoring** - no external monitor needed
 - All AXI5 extensions supported (NSAID, TRACE, MPAM, MECID, UNIQUE, CHUNKING, MTE, POISON)
 - Transaction tracking with configurable table size
 - Error detection (SLVERR, timeout, orphan transactions)
 - Performance metrics (latency, throughput)
 - Configurable packet filtering to reduce bandwidth
 - 128-bit monitor bus packet output paired with 64-bit side-band timestamp
 - Active transaction count tracking
-



24.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel SKID buffer depth
SKID_DEPTH_R	int	4	R channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width
AXI_MECID_WIDTH	int	16	Memory encryption context width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
AXI_CHUNKNUM_WIDTH	int	4	Chunk number width
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption
ENABLE_UNIQUE	bit	1	Enable unique ID indicator
ENABLE_CHUNKING	bit	1	Enable data chunking

Parameter	Type	Default	Description
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
UNIT_ID	int	1	Monitoring unit identifier
AGENT_ID	int	12	Agent identifier
MAX_TRANSACTION S	int	16	Transaction table size
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add pipeline stage for timing
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
N_ADDR_RANGES	int	0	Number of address-range comparators. 0 = checker omitted (zero area). >0 = N independent [low, high] ranges; hits emit PktTypeError + AXI_ERR_ADDR_RANGE on monbus.
ENABLE_ERROR_LO GIC	bit	1	Compile-in the error-detection cone (0 drops it for area).
ENABLE_TIMEOUT_ LOGIC	bit	1	Compile-in the timeout cone and the axi_monitor_timeout instance.
ENABLE_COMPL_LO	bit	1	Compile-in the

Parameter	Type	Default	Description
GIC			completion cone.
ENABLE_THRESHOL D_LOGIC	bit	1	Compile-in the threshold cone.
ENABLE_PERF_LOGI C	bit	1	Compile-in the perfmon measurement window and utilization/throughput counters.
ENABLE_DEBUG_LO GIC	bit	0	Compile-in the debug cone (off by default).

Synthesis-cone note: the six ENABLE_*_LOGIC parameters gate each detection cone at synthesis via generate-if, so unused logic drops to zero area (classic cones default on, debug off). Inside the wrapper's axi_monitor_filtered instance the perf/debug master switches are fixed (ENABLE_PERF_PACKETS = 1, ENABLE_DEBUG_MODULE = 0). The former CAM_PIPELINE / TRANS_CAM_PIPELINE parameters were removed — the transaction CAM is now always pipelined.

24.3 Monitor Backpressure (block_ready)

The monitor exposes a block_ready signal that goes low when its internal FIFO is saturated and cannot accept a new in-flight transaction. The wrapper ANDs block_ready into the upstream-facing s_axi_arready so a saturated monitor stalls new transactions on the wire instead of dropping events.

- **Where the stall lands:** the upstream s_axi_arready is forced low until the monitor drains.
- **When USE_MONITOR=0:** block_ready is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.
- **For axi5 slave variants:** the monitor watches the FUB-side handshake, so there is a SKID_DEPTH_AR cycle lag between block_ready going low and new events ceasing. MAX_TRANSACTIONS should be sized to cover this margin.

This replaces a previous bug where block_ready was left unconnected and a full monitor FIFO would silently lose events.

24.4 Address-Range Checker

The wrapper can be parameterized with `N_ADDR_RANGES > 0` to instantiate an N-comparator address-range checker that watches every accepted AR handshake and emits a `PktTypeError` monbus packet (event code `AXI_ERR_ADDR_RANGE = 8'h0D`) when an address falls inside any of the configured `[low, high]` inclusive ranges.

Config inputs (active only when `N_ADDR_RANGES > 0`): - `cfg_addr_check_enable` — master on/off for the checker. - `cfg_addr_range_enable[N-1:0]` — per-range enable bit. - `cfg_addr_range_low[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive low bound for each range. - `cfg_addr_range_high[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive high bound for each range.

Event encoding (within the standard 128-bit `monitor_packet_t`, `event_data` field): - `packet_type = PktTypeError (4'h0)` - `protocol = AXI (3'b000)` - `event_code = AXI_ERR_ADDR_RANGE (8'h0D)` - `event_data[63:60] = range_index (4 bits; supports up to 16 ranges)` - `event_data[59:0] = full matched address (up to 60 bits, zero-padded if narrower)`

Exact match: set `cfg_addr_range_low[i] == cfg_addr_range_high[i]`.

Filtering: the existing `cfg_axi_error_mask[13]` bit masks this event code; set it high to suppress range packets without disabling other errors. No new mask wiring needed.

Per-range coalescing: if a range hits again before its packet has been emitted, the latched address is overwritten (latest hit wins). One packet per cycle drains the pending mask via a lowest-index priority encoder. Distinct ranges never lose events; under sustained per-range bursts, only the latest address per range is reported per emission cycle.

24.5 Ports

24.5.1 Clock and Reset

Port	Width	Direction	Description
<code>aclk</code>	1	Input	AXI clock
<code>aresetn</code>	1	Input	AXI active-low reset

24.5.2 Slave AXI5 Interface

Same as `axi5_slave_rd` - see [AXI5 Slave Read](#) for complete port list.

24.5.3 FUB Interface

Same as `axi5_slave_rd` - see [AXI5 Slave Read](#) for complete port list.

24.5.4 Monitor Configuration

Port	Width	Direction	Description
cfg_monitor_enable	1	Input	Enable completion packets
cam_clear	1	Input	Synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4])
cfg_error_enable	1	Input	Enable error packets
cfg_timeout_enable	1	Input	Enable timeout packets
cfg_performance_enable	1	Input	Enable performance packets (see Performance Monitoring)
cfg_completion_enable	1	Input	Enable transaction-completion packets
cfg_threshold_enable	1	Input	Enable threshold-crossed packets
cfg_debug_enable	1	Input	Enable debug/trace packets (gates the debug cone — the 6th reporter sub-block)
cfg_timeout_cycles	16	Input	Timeout threshold (cycles)
cfg_latency_threshold	32	Input	High latency threshold (cycles)
cfg_axi_pkt_mask	16	Input	Packet type filter mask
cfg_axi_err_select	16	Input	Error selection mask
cfg_axi_err_or_mask	16	Input	Error event filter

Port	Width	Direction	Description
cfg_axi_timeout_mask	16	Input	Timeout event filter
cfg_axi_completion_mask	16	Input	Completion event filter
cfg_axi_threshold_mask	16	Input	Threshold event filter
cfg_axi_performance_mask	16	Input	Performance event filter
cfg_axi_address_mask	16	Input	Address event filter
cfg_axi_debug_mask	16	Input	Debug event filter

Detection-cone enables: `cfg_completion_enable`, `cfg_threshold_enable`, and `cfg_debug_enable` turn on the completion, threshold, and debug reporter sub-blocks respectively. `cfg_debug_enable` gates the **debug cone** — the 6th reporter sub-block (`axi_monitor_reporter_debug`). In this wrapper the debug module's `cfg_debug_level` (4), `cfg_debug_mask` (16), and `cfg_active_trans_threshold` (16) inputs are tied to their defaults (4'h0 / 16'h0 / 16'd8) and are not exposed as wrapper ports.

24.5.5 Monitor Bus Output

Port	Width	Direction	Description
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor packet ready
monbus_packet	128	Output	<code>monitor_packet_t</code> (see format below)
monbus_timestamp	64	Output	<code>monbus_timestamp_t</code> paired atomically with <code>monbus_packet</code>
i_mon_time	64	Input	Free-running counter from <code>monbus_axil_group</code> , sampled at packet

Port	Width	Direction	Description
			emission

24.5.6 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy indicator
active_transactions	8	Output	Current outstanding transactions
error_count	16	Output	Total errors detected
transaction_count	32	Output	Total transactions completed
cfg_conflict_error	1	Output	Configuration conflict detected

24.6 Performance Monitoring

When performance tracking is compiled in (`ENABLE_PERF_LOGIC = 1`, the wrapper default, with `ENABLE_PERF_PACKETS` fixed to 1 inside the monitor instance) and `cfg_perf_enable` is asserted at runtime, the monitor runs a **measurement-window state machine** plus a bank of data-channel utilization counters. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows so the host can read a completed window's totals.

24.6.1 The measurement window

A window is opened by a **start event** and closed by an **end event**. The event sources are selected by `cfg_start_event_sel` / `cfg_end_event_sel` (3-bit; e.g. 3'b010 selects the `cfg_perf_enable` edge) and can also be fired directly by the `cfg_start_trigger` / `cfg_end_trigger` pulses (from an engine or CSR write). `cfg_window_force_close` is a software override that closes the window immediately. While the window is open:

- `window_active` is high.
- `window_cycles` free-runs, counting every clock elapsed inside the window.

24.6.2 Utilization buckets (R data channel)

Every cycle inside the window is classified by the **R** data channel's valid/ready handshake into exactly one of four buckets:

Output	Width	Condition	Meaning
perf_prod_cycles	32	rvalid && rready	productive beat transferred
perf_bp_cycles	32	rvalid && !rready	back-pressure (data offered, sink not ready)
perf_starv_cycles	32	!rvalid && rready	starvation (sink ready, no data)
perf_idle_cycles	32	!rvalid && !rready	idle

The four buckets sum to window_cycles, so utilization = perf_prod_cycles / window_cycles.

24.6.3 Throughput counters

Output	Width	Meaning
perf_beat_count	32	R data beats transferred (= perf_prod_cycles, 1 beat/cycle)
perf_byte_count	64	bytes transferred = beats x (1 << latched ARSIZE), using the ARSIZE captured at the most recent AR address phase
perf_burst_count	32	AR address-phase handshakes

Average burst length is perf_beat_count / perf_burst_count.

24.6.4 Performance Monitoring Ports

Port	Width	Direction	Description
cfg_perf_enable	1	Input	Enable performance-metric packet generation (also listed under Monitor

Port	Width	Direction	Description
			Configuration)
cfg_start_event_sel	3	Input	Window start event source select
cfg_end_event_sel	3	Input	Window end event source select
cfg_start_trigger	1	Input	Pulse: open the measurement window
cfg_end_trigger	1	Input	Pulse: close the measurement window
cfg_window_force_close	1	Input	Software override: force the window closed
window_active	1	Output	High while a measurement window is open
window_cycles	32	Output	Cycles elapsed in the current window
perf_prod_cycles	32	Output	rvalid && rready cycles
perf_bp_cycles	32	Output	rvalid && !rready cycles (back-pressure)
perf_starv_cycles	32	Output	!rvalid && rready cycles (starvation)
perf_idle_cycles	32	Output	!rvalid && !rready cycles
perf_beat_count	32	Output	R data beats transferred
perf_byte_count	64	Output	bytes transferred
perf_burst_count	32	Output	AR address-phase handshakes

When `USE_MONITOR = 0` (or `ENABLE_PERF_LOGIC = 0`) all perfmon outputs are tied to 0.

24.7 Functionality

24.7.1 Monitor Packet Format

The 128-bit monbus_packet (paired with the 64-bit monbus_timestamp side-band signal) follows the standardized AMBA monitor bus format:

Bits [127:124] - Packet Type:

- 0x0 = ERROR Error events (SLVERR, DECERR, protocol violations)
- 0x1 = COMPL Completion events (transaction finished)
- 0x2 = THRESH Threshold events
- 0x3 = TIMEOUT Timeout events
- 0x4 = PERF Performance metrics
- 0x8 = ADDR_MATCH Address match events
- 0x9 = APB APB-specific events
- 0xF = DEBUG Debug events

Bits [123:109] - Reserved (15 bits, forward-compat slack)

Bits [108:105] - Protocol (4 bits): 0x0=AXI, 0x1=AXIS, 0x2=APB, 0x3=ARB, 0x4=CORE

Bits [104:97] - Event Code (8 bits, protocol-specific)

Bits [96:88] - Channel ID (9 bits – AXI ID or channel index)

Bits [87:72] - Agent ID (16 bits, from AGENT_ID parameter)

Bits [71:64] - Unit ID (8 bits, from UNIT_ID parameter)

Bits [63:0] - Event Data (64 bits – full address, latency, etc.)

24.7.2 Event Types

24.7.2.1 Error Packets (Type=0)

Event Code	Description	Event Data
0x1	SLVERR response	Transaction ID, address[18:0]
0x2	DECERR response	Transaction ID, address[18:0]
0x3	Orphan data	Transaction ID
0x4	Protocol violation	Violation code
0x5	Poison detected	Transaction ID, beat number
0x6	Tag mismatch	Transaction ID, expected/actual tags

24.7.2.2 Completion Packets (Type=1)

Event Code	Description	Event Data
0x0	Read completion	Transaction ID, burst length, latency

24.7.2.3 Timeout Packets (Type=2)

Event Code	Description	Event Data
0x1	AR channel timeout	Transaction ID, cycles elapsed
0x2	R channel timeout	Transaction ID, cycles elapsed

24.7.2.4 Performance Packets (Type=4)

Event Code	Description	Event Data
0x1	High latency	Transaction ID, latency (cycles)
0x2	Bandwidth sample	Bytes transferred, time window
0x3	Outstanding count	Max outstanding, average

24.8 Configuration Guide

24.8.1 Common Configurations

24.8.1.1 Functional Verification Mode

```
.cfg_monitor_enable    (1'b1), // Track completions
.cfg_error_enable      (1'b1), // Catch errors
.cfg_timeout_enable    (1'b1), // Detect stalls
.cfg_perf_enable       (1'b0), // Disable (reduces traffic)
.cfg_timeout_cycles    (16'd1000),
.cfg_latency_threshold (32'd500)
```

24.8.1.2 Performance Analysis Mode

```
.cfg_monitor_enable    (1'b0), // Disable completions
.cfg_error_enable      (1'b1), // Still catch errors
.cfg_timeout_enable    (1'b0), // Disable
.cfg_perf_enable       (1'b1), // Enable performance metrics
.cfg_latency_threshold (32'd100)
```

24.8.1.3 Debug Mode

```
.cfg_monitor_enable    (1'b1), // Track all transactions
.cfg_error_enable      (1'b1), // All errors
.cfg_timeout_enable    (1'b1), // All timeouts
.cfg_perf_enable       (1'b0), // Disable perf (too much data)
.cfg_axi_pkt_mask      (16'hFFFF) // All packet types
```

24.8.2 Filter Masks

cfg_axi_pkt_mask bits: - [0]: Error packets - [1]: Completion packets - [2]: Timeout packets - [3]: Threshold packets - [4]: Performance packets - [15:5]: Reserved

Example: `cfg_axi_pkt_mask = 16'h0007` enables error, completion, and timeout packets only.

24.9 Usage Example

```
axi5_slave_rd_mon #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH     (64),
    .UNIT_ID           (1),
    .AGENT_ID          (12),
    .MAX_TRANSACTIONS  (16),
    .ENABLE_FILTERING   (1),
    .ENABLE_NSAID       (1),
    .ENABLE_TRACE       (1),
    .ENABLE_MPAM        (1),
    .ENABLE_MECID       (1),
    .ENABLE_UNIQUE      (1),
    .ENABLE_CHUNKING    (1),
    .ENABLE_MTE         (1),
    .ENABLE_POISON      (1)
) u_axi5_slave_rd_mon (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // Slave interface (from external master)
    .s_axi_arid         (s_axi_arid),
    .s_axi_araddr       (s_axi_araddr),
    // ... (connect all slave AR/R signals)

    // FUB interface (to backend)
    .fub_axi_arid       (mem_arid),
    .fub_axi_araddr     (mem_araddr),
    // ... (connect to memory controller)
```

```

// Monitor configuration
.cfg_monitor_enable (1'b1),
.cfg_error_enable   (1'b1),
.cfg_timeout_enable (1'b1),
.cfg_perf_enable    (1'b0),
.cfg_timeout_cycles (16'd1000),
.cfg_latency_threshold (32'd500),
.cfg_axi_pkt_mask   (16'h0007),

// Monitor bus (connect to FIFO or consumer)
.monbus_valid      (mon_valid),
.monbus_ready      (mon_ready),
.monbus_packet     (mon_packet),

// Status
.busy              (slave_rd_busy),
.active_transactions(active_txns),
.error_count       (total_errors),
.transaction_count (total_txns),
.cfg_conflict_error(cfg_conflict)
);

// Downstream packet handling
gaxi_fifo_sync #(.DATA_WIDTH(64), .DEPTH(256)) u_mon_fifo (
.i_clk      (axi_clk),
.i_rst_n    (axi_rst_n),
.i_valid    (mon_valid),
.i_data     (mon_packet),
.o_ready    (mon_ready),
.o_valid    (fifo_valid),
.o_data     (fifo_data),
.i_ready    (consumer_ready)
);

```

24.10 Design Notes

- Monitor packets provide real-time transaction visibility without external logic
- Filtering reduces monitor bus bandwidth - critical for high-throughput systems
- Transaction table size (MAX_TRANSACTIONS) must accommodate peak outstanding transactions

- Performance packets can generate high traffic - use sparingly or with filtering
 - UNIT_ID and AGENT_ID identify this monitor in multi-agent systems
 - Error count saturates at max value (does not wrap)
-

24.11 Related Documentation

- [AXI5 Slave Read](#) - Non-monitored version
 - [AXI5 Slave Read Monitor CG](#) - Clock-gated variant
 - [AXI5 Slave Write Monitor](#) - Write monitor
 - [AXI Monitor Filtered](#) - Monitor core
 - [Monitor Package Spec](#) - Packet format details
-

24.12 Navigation

- [← Back to AXI5 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

25 AXI5 Slave Write Monitor with Clock Gating

Module: axi5_slave_wr_mon_cg.sv **Location:** rtl/amba/axi5/ **Status:** Production Ready

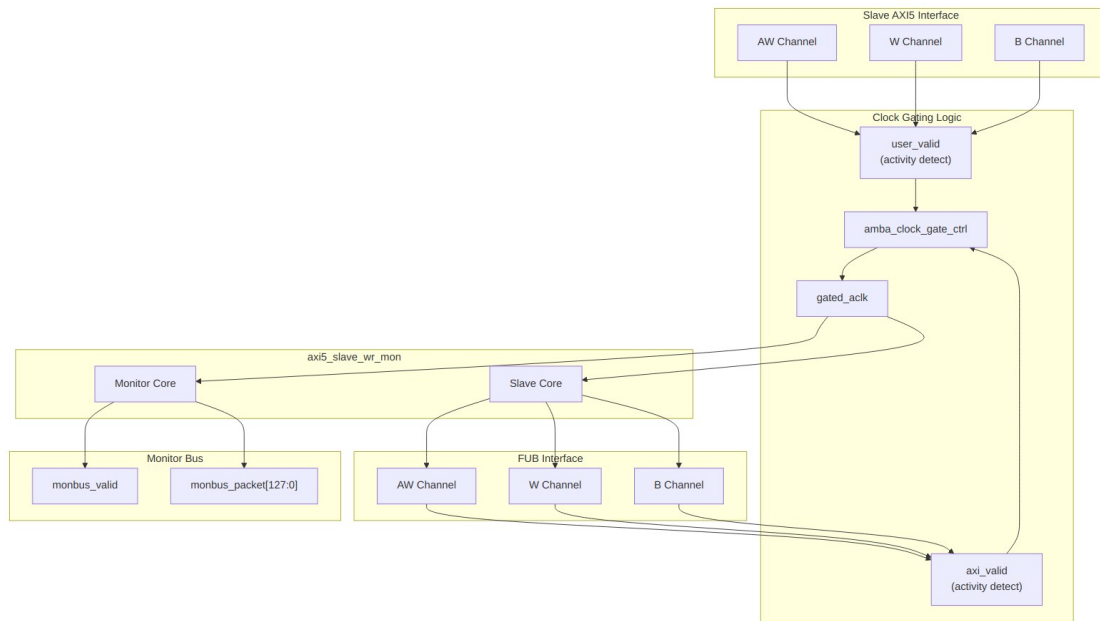
25.1 Overview

The AXI5 Slave Write Monitor with Clock Gating module combines integrated transaction monitoring with automatic clock gating for power optimization. It wraps axi5_slave_wr_mon with clock gating logic.

25.1.1 Key Features

- Full AMBA AXI5 slave write protocol compliance
- **Integrated filtered monitoring** - real-time transaction visibility
- **Integrated clock gating** for dynamic power reduction
- All AXI5 extensions supported (ATOMIC, NSAIID, TRACE, MPAM, MECID, UNIQUE, MTE, POISON)

- Configurable idle count before gating
- Transaction tracking with error detection
- Performance metrics and filtering
- Transparent gating - no protocol changes
- Gating status outputs for system monitoring



Module Architecture

25.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel SKID buffer depth
SKID_DEPTH_W	int	4	W channel SKID buffer depth
SKID_DEPTH_B	int	2	B channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width

Parameter	Type	Default	Description
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_ATOP_WIDTH	int	6	Atomic operation width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width
AXI_MECID_WIDTH	int	16	Memory encryption context width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
ENABLE_ATOMIC	bit	1	Enable atomic operations
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption
ENABLE_UNIQUE	bit	1	Enable unique ID indicator
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
UNIT_ID	int	1	Monitoring unit identifier
AGENT_ID	int	13	Agent identifier
MAX_TRANSACTION S	int	16	Transaction table size
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STA	bit	0	Add pipeline stage for

Parameter	Type	Default	Description
GE			timing
USE_MONITOR	bit	1	Synthesis-time monitor enable (forwarded to inner monitor).
N_ADDR_RANGES	int	0	Number of address-range comparators (forwarded to base module).
ENABLE_ERROR_LO GIC	bit	1	Synthesis-cone enable for error detection (forwarded to base module)
ENABLE_TIMEOUT_ LOGIC	bit	1	Synthesis-cone enable for timeout detection (forwarded to base module)
ENABLE_COMPL_LO GIC	bit	1	Synthesis-cone enable for completion tracking (forwarded to base module)
ENABLE_THRESHOL D_LOGIC	bit	1	Synthesis-cone enable for threshold detection (forwarded to base module)
ENABLE_PERF_LOGI C	bit	1	Synthesis-cone enable for the perfmon window + counters (forwarded to base module)
ENABLE_DEBUG_LO GIC	bit	0	Synthesis-cone enable for the debug/trace cone (forwarded to base module)
CG_IDLE_COUNT_WI DTH	int	4	Clock gating idle counter width

25.3 Ports

25.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock (ungated)
aresetn	1	Input	AXI active-low reset

25.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

25.3.3 Slave AXI5 Interface

Same as axi5_slave_wr - see [AXI5 Slave Write](#) for complete port list.

25.3.4 FUB Interface

Same as axi5_slave_wr - see [AXI5 Slave Write](#) for complete port list.

25.3.5 Monitor Configuration

Same as axi5_slave_wr_mon - see [AXI5 Slave Write Monitor](#) for complete list.

25.3.6 Monitor Bus Output

Port	Width	Direction	Description
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor packet ready
monbus_packet	128	Output	monitor_packet_t (see format below)
monbus_timestamp	64	Output	monbus_timestamp_t paired atomically with monbus_packet

Port	Width	Direction	Description
i_mon_time	64	Input	Free-running counter from monbus_axil_group, sampled at packet emission

25.3.7 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy indicator
active_transactions	8	Output	Current outstanding transactions
error_count	16	Output	Total errors detected
transaction_count	32	Output	Total transactions completed
cfg_conflict_error	1	Output	Configuration conflict detected
cg_gating	1	Output	Clock is currently gated
cg_idle	1	Output	Module is idle

25.4 Functionality

25.4.1 Clock Gating Behavior

Activity Detection: - **user_valid:** Asserted when slave interface has activity (awvalid, wvalid, bready, or internal busy) - **axi_valid:** Asserted when FUB interface has activity (awvalid, wvalid, bvalid)

Key Points: - Clock gating disabled when `cfg_cg_enable = 0` - Ready signals (awready, wready) forced to 0 when gated (prevents new transactions) - bready forced to 0 when gated (prevents accepting responses) - Gating only occurs after configured idle period - Any activity immediately ungates the clock - Monitor continues to track transactions even when gated

25.5 Performance Monitoring

The clock-gated wrapper exposes the full perfmon interface of the base module and **forwards every port unchanged** to the inner `axi5_slave_wr_mon`. The measurement-window state machine, the four W-channel utilization buckets (productive / back-pressure / starvation / idle), and the beat/byte/burst throughput counters behave exactly as documented in the base module — see [Performance Monitoring in axi5_slave_wr_mon](#) for the full narrative and per-bit semantics.

Forwarded perfmon ports (identical width and direction to the base module):

- **Inputs:** `cfg_perf_enable`, `cfg_start_event_sel` (3), `cfg_end_event_sel` (3), `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Outputs:** `window_active`, `window_cycles` (32), `perf_prod_cycles` (32), `perf_bp_cycles` (32), `perf_starv_cycles` (32), `perf_idle_cycles` (32), `perf_beat_count` (32), `perf_byte_count` (64), `perf_burst_count` (32)

The `perf_burst_count` output tracks AW (write address) handshakes. Clock gating never suppresses these paths: while a measurement window is open the monitor stays awake, so window cycle accounting remains exact regardless of the `idle_count` setting.

Alongside perfmon, the wrapper forwards the `cfg_compl_enable`, `cfg_threshold_enable`, and `cfg_debug_enable` control inputs, plus the six `ENABLE_*_LOGIC` synthesis-cone parameters (see [Parameters](#)), straight through to the base monitor.

25.6 Combined Benefits

25.6.1 Power Optimization

Power Savings Estimation: - Base slave logic: ~40% of total power - Monitor logic: ~60% of total power - With gating at 50% duty cycle: ~50% dynamic power savings - Actual savings depend on traffic pattern and `idle_count` setting

Write-Specific Power Considerations: - Burst writes keep module active longer - W channel SKID buffering consumes additional power - Poison/tag checking adds monitoring overhead - Atomic operations may extend active periods

25.6.2 Observability

Monitoring Capabilities: - Error detection (SLVERR, timeout, orphan, poison) - Performance tracking (latency, throughput) - Transaction completion tracking - Protocol violation detection -

25.7 Usage Example

```
axi5_slave_wr_mon_cg #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH    (64),
    .UNIT_ID           (1),
    .AGENT_ID          (13),
    .MAX_TRANSACTIONS  (16),
    .CG_IDLE_COUNT_WIDTH(4),
    .ENABLE_FILTERING   (1),
    .ENABLE_ATOMIC      (1),
    .ENABLE_NSAID       (1),
    .ENABLE_TRACE       (1),
    .ENABLE_MPAM        (1),
    .ENABLE_MECID       (1),
    .ENABLE_UNIQUE      (1),
    .ENABLE_MTE         (1),
    .ENABLE_POISON      (1)
) u_axi5_slave_wr_mon_cg (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // Clock gating config
    .cfg_cg_enable      (1'b1),           // Enable gating
    .cfg_cg_idle_count  (4'd3),           // Gate after 8 idle cycles

    // Slave interface (from external master)
    .s_axi_awid         (s_axi_awid),
    .s_axi_awaddr       (s_axi_awaddr),
    // ... (connect all slave AW/W/B signals)

    // FUB interface (to backend)
    .fub_axi_awid       (mem_awid),
    .fub_axi_awaddr     (mem_awaddr),
    // ... (connect to memory controller)

    // Monitor configuration
    .cfg_monitor_enable (1'b1),
    .cfg_error_enable   (1'b1),
    .cfg_timeout_enable (1'b1),
    .cfg_perf_enable    (1'b0),
```

```

.cfg_timeout_cycles (16'd1000),
.cfg_latency_threshold (32'd500),
.cfg_axi_pkt_mask    (16'h0007),

// Monitor bus
.monbus_valid        (mon_valid),
.monbus_ready        (mon_ready),
.monbus_packet        (mon_packet),

// Status
.busy                (slave_wr_busy),
.active_transactions (active_txns),
.error_count          (total_errors),
.transaction_count    (total_txns),
.cfg_conflict_error   (cfg_conflict),

// Clock gating status
.cg_gating            (slave_wr_gating),
.cg_idle              (slave_wr_idle)
);

// System power management integration
assign system_power_save = slave_wr_gating &&
                             slave_rd_gating;

// Error tracking
always @(posedge axi_clk or negedge axi_rst_n) begin
    if (!axi_rst_n) begin
        critical_errors <= 0;
    end else if (mon_valid && mon_ready) begin
        // Decode packet type
        case (mon_packet[63:60])
            4'd0: begin // Error packet
                if (mon_packet[56:53] == 4'd5) // Poison detected
                    critical_errors <= critical_errors + 1;
            end
        endcase
    end
end

// Monitor packet handling
gaxi_fifo_sync #(.DATA_WIDTH(64), .DEPTH(256)) u_mon_fifo (
    .i_clk        (axi_clk),
    .i_rst_n      (axi_rst_n),
    .i_valid       (mon_valid),
    .i_data        (mon_packet),
    .o_ready       (mon_ready),

```



```

        .o_valid      (fifo_valid),
        .o_data       (fifo_data),
        .i_ready      (consumer_ready)
    );

```

25.8 Design Notes

25.8.1 When to Use This Module

Ideal for: - Power-constrained systems needing visibility into write operations - Debug/validation builds with power budgets - Systems with sporadic write transaction patterns - SoC integration requiring both monitoring and power optimization - Data integrity validation (poison detection)

Avoid when: - Interface is continuously active (minimal gating opportunities) - Monitoring overhead unacceptable (use non-monitored version) - Deterministic latency critical (use high idle count or disable gating) - Write bursts dominate traffic (reduces gating efficiency)

25.8.2 Configuration Recommendations

Low-Power Debug Mode:

```

.cfg_cg_enable      (1'b1),
.cfg_cg_idle_count  (4'd1),    // Aggressive gating
.cfg_monitor_enable (1'b1),    // Full monitoring
.cfg_error_enable   (1'b1),
.cfg_perf_enable    (1'b0)    // Reduce packet traffic

```

Performance Analysis Mode:

```

.cfg_cg_enable      (1'b1),
.cfg_cg_idle_count  (4'd4),    // Conservative gating
.cfg_monitor_enable (1'b0),    // Disable completions
.cfg_perf_enable    (1'b1)    // Enable performance metrics

```

Data Integrity Validation:

```

.cfg_cg_enable      (1'b1),
.cfg_cg_idle_count  (4'd2),    // Balanced
.cfg_error_enable   (1'b1),    // Catch all errors
.cfg_monitor_enable (1'b1),    // Track completions
.ENABLE_POISON      (1),       // Enable poison detection
.ENABLE_MTE          (1)       // Enable tag checking

```

25.8.3 Area and Timing Impact

- **Area:** ~5-8% increase over non-monitored slave (monitoring + clock gating)
- **Timing:** Clock gating adds <50ps, monitoring is off critical path

- **Power:** Net savings depends on activity factor (typically 30-50% at 50% duty cycle)

25.8.4 Write-Specific Considerations

Burst Handling: - Monitor tracks AW/W synchronization - Detects missing WLAST errors - Burst latency includes all beats - Gating should not occur mid-burst

Atomic Operations: - Monitor tracks AWATOP field - Atomic transaction latency measured end-to-end - Consider higher idle_count for atomic-heavy workloads

Poison Detection: - Generates error packet when WPOISON asserted - Critical for data integrity in safety-critical systems - Monitor overhead minimal (single bit check per beat)

25.9 Related Documentation

- [AXI5 Slave Write](#) - Non-monitored, non-gated version
 - [AXI5 Slave Write CG](#) - Clock-gated without monitoring
 - [AXI5 Slave Write Monitor](#) - Monitored without clock gating
 - [AXI5 Slave Read Monitor CG](#) - Read variant
 - [AMBA Clock Gate Control](#) - Clock gating controller
-

25.10 Navigation

- [← Back to AXI5 Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

26 AXI5 Slave Write Monitor

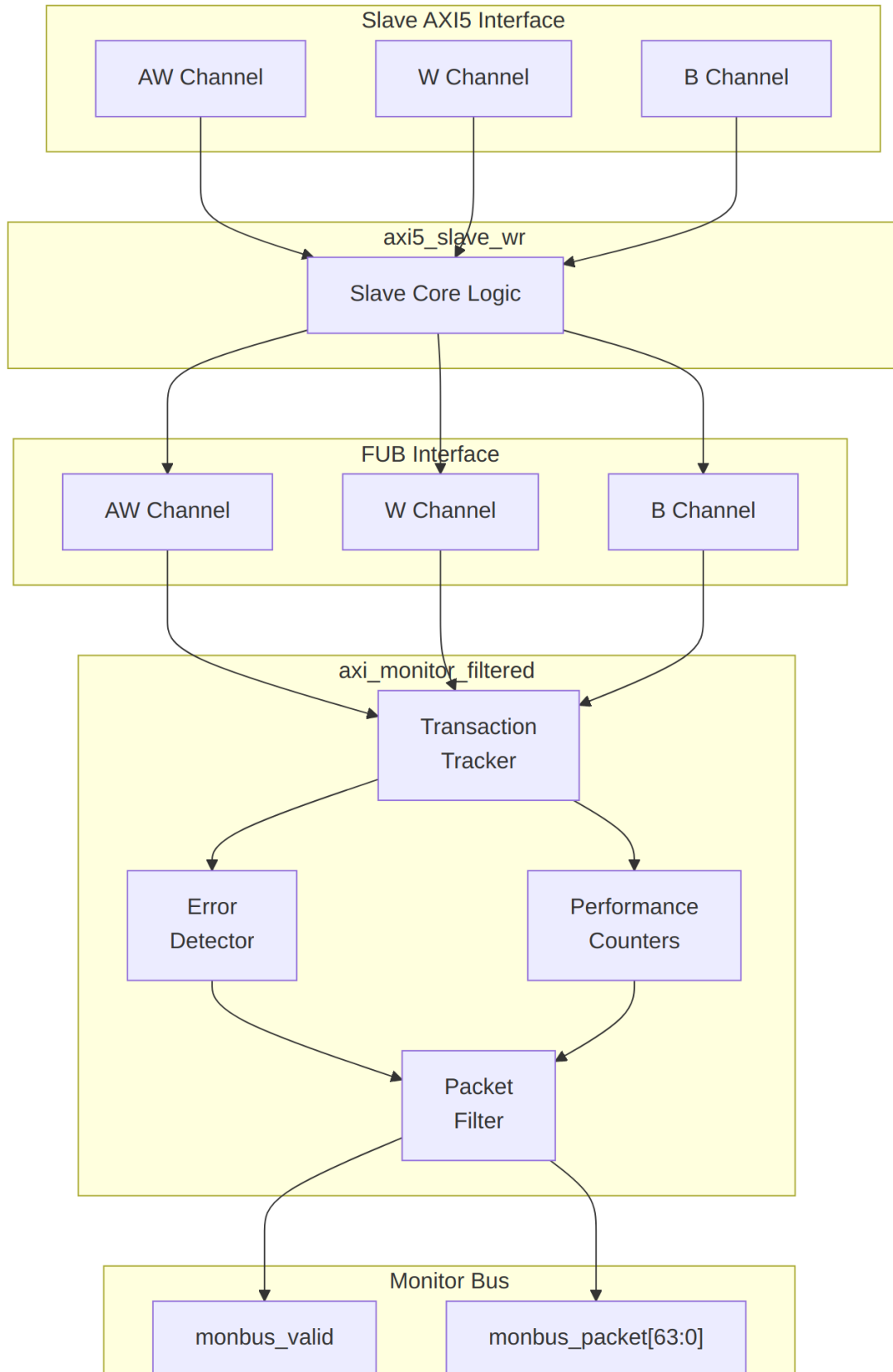
Module: axi5_slave_wr_mon.sv **Location:** rtl/amba/axi5/ **Status:** Production Ready

26.1 Overview

The AXI5 Slave Write Monitor module combines the axi5_slave_wr interface with integrated transaction monitoring. It provides real-time visibility into slave write operations with configurable packet filtering and error detection.

26.1.1 Key Features

- Full AMBA AXI5 slave write protocol compliance
 - **Integrated filtered monitoring** - no external monitor needed
 - All AXI5 extensions supported (ATOMIC, NSAID, TRACE, MPAM, MECID, UNIQUE, MTE, POISON)
 - Transaction tracking with configurable table size
 - Error detection (SLVERR, timeout, orphan transactions, poison data)
 - Performance metrics (latency, throughput)
 - Configurable packet filtering to reduce bandwidth
 - 128-bit monitor bus packet output paired with 64-bit side-band timestamp
 - Active transaction count tracking
-



26.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel SKID buffer depth
SKID_DEPTH_W	int	4	W channel SKID buffer depth
SKID_DEPTH_B	int	2	B channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_ATOP_WIDTH	int	6	Atomic operation width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width
AXI_MECID_WIDTH	int	16	Memory encryption context width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
ENABLE_ATOMIC	bit	1	Enable atomic operations
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption

Parameter	Type	Default	Description
ENABLE_UNIQUE	bit	1	Enable unique ID indicator
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
UNIT_ID	int	1	Monitoring unit identifier
AGENT_ID	int	13	Agent identifier
MAX_TRANSACTION_S	int	16	Transaction table size
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add pipeline stage for timing
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
N_ADDR_RANGES	int	0	Number of address-range comparators. 0 = checker omitted (zero area). >0 = N independent [low, high] ranges; hits emit PktTypeError + AXI_ERR_ADDR_RANGE on monbus.
ENABLE_ERROR_LOGIC	bit	1	Compile-in the error-detection cone (0 drops it for area).
ENABLE_TIMEOUT_LOGIC	bit	1	Compile-in the timeout cone and the axi_monitor_timeout

Parameter	Type	Default	Description
			instance.
ENABLE_COMPL_LO GIC	bit	1	Compile-in the completion cone.
ENABLE_THRESHOL D_LOGIC	bit	1	Compile-in the threshold cone.
ENABLE_PERF_LOGI C	bit	1	Compile-in the perfmon measurement window and utilization/throughput counters.
ENABLE_DEBUG_LO GIC	bit	0	Compile-in the debug cone (off by default).

Synthesis-cone note: the six ENABLE_*_LOGIC parameters gate each detection cone at synthesis via generate-if, so unused logic drops to zero area (classic cones default on, debug off). Inside the wrapper's axi_monitor_filtered instance the perf/debug master switches are fixed (ENABLE_PERF_PACKETS = 1, ENABLE_DEBUG_MODULE = 0). The former CAM_PIPELINE / TRANS_CAM_PIPELINE parameters were removed — the transaction CAM is now always pipelined.

26.3 Monitor Backpressure (block_ready)

The monitor exposes a block_ready signal that goes low when its internal FIFO is saturated and cannot accept a new in-flight transaction. The wrapper ANDs block_ready into the upstream-facing s_axi_awready so a saturated monitor stalls new transactions on the wire instead of dropping events.

- **Where the stall lands:** the upstream s_axi_awready is forced low until the monitor drains.
- **When USE_MONITOR=0:** block_ready is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.
- **For axi5 slave variants:** the monitor watches the FUB-side handshake, so there is a SKID_DEPTH_AW cycle lag between block_ready going low and new events ceasing. MAX_TRANSACTIONS should be sized to cover this margin.

This replaces a previous bug where `block_ready` was left unconnected and a full monitor FIFO would silently lose events.

26.4 Address-Range Checker

The wrapper can be parameterized with `N_ADDR_RANGES > 0` to instantiate an N-comparator address-range checker that watches every accepted AW handshake and emits a `PktTypeError` monbus packet (event code `AXI_ERR_ADDR_RANGE = 8'h0D`) when an address falls inside any of the configured `[low, high]` inclusive ranges.

Config inputs (active only when `N_ADDR_RANGES > 0`): - `cfg_addr_check_enable` — master on/off for the checker. - `cfg_addr_range_enable[N-1:0]` — per-range enable bit. - `cfg_addr_range_low[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive low bound for each range. - `cfg_addr_range_high[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive high bound for each range.

Event encoding (within the standard 128-bit `monitor_packet_t`, `event_data` field): - `packet_type = PktTypeError (4'h0)` - `protocol = AXI (3'b000)` - `event_code = AXI_ERR_ADDR_RANGE (8'h0D)` - `event_data[63:60] = range_index` (4 bits; supports up to 16 ranges) - `event_data[59:0]` = full matched address (up to 60 bits, zero-padded if narrower)

Exact match: set `cfg_addr_range_low[i] == cfg_addr_range_high[i]`.

Filtering: the existing `cfg_axi_error_mask[13]` bit masks this event code; set it high to suppress range packets without disabling other errors. No new mask wiring needed.

Per-range coalescing: if a range hits again before its packet has been emitted, the latched address is overwritten (latest hit wins). One packet per cycle drains the pending mask via a lowest-index priority encoder. Distinct ranges never lose events; under sustained per-range bursts, only the latest address per range is reported per emission cycle.

26.5 Ports

26.5.1 Clock and Reset

Port	Width	Direction	Description
<code>aclk</code>	1	Input	AXI clock
<code>aresetn</code>	1	Input	AXI active-low reset

26.5.2 Slave AXI5 Interface

Same as `axi5_slave_wr` - see [AXI5 Slave Write](#) for complete port list.

26.5.3 FUB Interface

Same as axi5_slave_wr - see [AXI5 Slave Write](#) for complete port list.

26.5.4 Monitor Configuration

Port	Width	Direction	Description
cfg_monitor_enable	1	Input	Enable completion packets
cam_clear	1	Input	Synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4])
cfg_error_enable	1	Input	Enable error packets
cfg_timeout_enable	1	Input	Enable timeout packets
cfg_performance_enable	1	Input	Enable performance packets (see Performance Monitoring)
cfg_completion_enable	1	Input	Enable transaction-completion packets
cfg_threshold_enable	1	Input	Enable threshold-crossed packets
cfg_debug_enable	1	Input	Enable debug/trace packets (gates the debug cone — the 6th reporter sub-block)
cfg_timeout_cycles	16	Input	Timeout threshold (cycles)
cfg_latency_threshold	32	Input	High latency threshold (cycles)
cfg_axi_pkt_mask	16	Input	Packet type filter mask
cfg_axi_err_select	16	Input	Error selection mask

Port	Width	Direction	Description
cfg_axi_err or_mask	16	Input	Error event filter
cfg_axi_tim eout_mask	16	Input	Timeout event filter
cfg_axi_co mpl_mask	16	Input	Completion event filter
cfg_axi_thr esh_mask	16	Input	Threshold event filter
cfg_axi_per f_mask	16	Input	Performance event filter
cfg_axi_ad dr_mask	16	Input	Address event filter
cfg_axi_de bug_mask	16	Input	Debug event filter

Detection-cone enables: `cfg_compl_enable`, `cfg_threshold_enable`, and `cfg_debug_enable` turn on the completion, threshold, and debug reporter sub-blocks respectively. `cfg_debug_enable` gates the **debug cone** — the 6th reporter sub-block (`axi_monitor_reporter_debug`). In this wrapper the debug module's `cfg_debug_level` (4), `cfg_debug_mask` (16), and `cfg_active_trans_threshold` (16) inputs are tied to their defaults (4'h0 / 16'h0 / 16'd8) and are not exposed as wrapper ports.

26.5.5 Monitor Bus Output

Port	Width	Direction	Description
monbus_v alid	1	Output	Monitor packet valid
monbus_re ady	1	Input	Monitor packet ready
monbus_p acket	128	Output	<code>monitor_packet_t</code> (see format below)
monbus_ti mestamp	64	Output	<code>monbus_timestamp_t</code> paired atomically with <code>monbus_packet</code>
i_mon_tim	64	Input	Free-running counter

Port	Width	Direction	Description
e			from monbus_axil_group, sampled at packet emission

26.5.6 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy indicator
active_transactions	8	Output	Current outstanding transactions
error_count	16	Output	Total errors detected
transaction_count	32	Output	Total transactions completed
cfg_conflict_error	1	Output	Configuration conflict detected

26.6 Performance Monitoring

When performance tracking is compiled in (`ENABLE_PERF_LOGIC = 1`, the wrapper default, with `ENABLE_PERF_PACKETS` fixed to 1 inside the monitor instance) and `cfg_perf_enable` is asserted at runtime, the monitor runs a **measurement-window state machine** plus a bank of data-channel utilization counters. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows so the host can read a completed window's totals.

26.6.1 The measurement window

A window is opened by a **start event** and closed by an **end event**. The event sources are selected by `cfg_start_event_sel` / `cfg_end_event_sel` (3-bit; e.g. 3'b010 selects the `cfg_perf_enable` edge) and can also be fired directly by the `cfg_start_trigger` / `cfg_end_trigger` pulses (from an engine or CSR write). `cfg_window_force_close` is a software override that closes the window immediately. While the window is open:

- `window_active` is high.

- `window_cycles` free-runs, counting every clock elapsed inside the window.

26.6.2 Utilization buckets (W data channel)

Every cycle inside the window is classified by the **W** data channel's valid/ready handshake into exactly one of four buckets:

Output	Width	Condition	Meaning
<code>perf_prod_cycles</code>	32	<code>wvalid && wready</code>	productive beat transferred
<code>perf_bp_cycles</code>	32	<code>wvalid && !wready</code>	back-pressure (data offered, sink not ready)
<code>perf_starv_cycles</code>	32	<code>!wvalid && wready</code>	starvation (sink ready, no data)
<code>perf_idle_cycles</code>	32	<code>!wvalid && !wready</code>	idle

The four buckets sum to `window_cycles`, so $\text{utilization} = \text{perf_prod_cycles} / \text{window_cycles}$.

26.6.3 Throughput counters

Output	Width	Meaning
<code>perf_beat_count</code>	32	W data beats transferred (= <code>perf_prod_cycles</code> , 1 beat/cycle)
<code>perf_byte_count</code>	64	bytes transferred = beats x (1 << latched <code>AWSIZE</code>), using the <code>AWSIZE</code> captured at the most recent AW address phase
<code>perf_burst_count</code>	32	AW address-phase handshakes

Average burst length is $\text{perf_beat_count} / \text{perf_burst_count}$. (Transaction completion is still tracked on the B channel; the throughput counters measure the W data phase.)

26.6.4 Performance Monitoring Ports

Port	Width	Direction	Description
<code>cfg_perf_e</code>	1	Input	Enable performance-

Port	Width	Direction	Description
nable			metric packet generation (also listed under Monitor Configuration)
cfg_start_event_sel	3	Input	Window start event source select
cfg_end_event_sel	3	Input	Window end event source select
cfg_start_trigger	1	Input	Pulse: open the measurement window
cfg_end_trigger	1	Input	Pulse: close the measurement window
cfg_window_force_close	1	Input	Software override: force the window closed
window_active	1	Output	High while a measurement window is open
window_cycles	32	Output	Cycles elapsed in the current window
perf_prod_cycles	32	Output	wvalid && wready cycles
perf_bp_cycles	32	Output	wvalid && !wready cycles (back-pressure)
perf_starv_cycles	32	Output	!wvalid && wready cycles (starvation)
perf_idle_cycles	32	Output	!wvalid && !wready cycles
perf_beat_count	32	Output	W data beats transferred
perf_byte_count	64	Output	bytes transferred
perf_burst_count	32	Output	AW address-phase handshakes

When USE_MONITOR = 0 (or ENABLE_PERF_LOGIC = 0) all perfmon outputs are tied to 0.

26.7 Functionality

26.7.1 Monitor Packet Format

The 128-bit monbus_packet (paired with the 64-bit monbus_timestamp side-band signal) follows the standardized AMBA monitor bus format:

Bits [127:124] - Packet Type:
0x0 = ERROR Error events (SLVERR, DECERR, protocol violations)
0x1 = COMPL Completion events (transaction finished)
0x2 = THRESH Threshold events
0x3 = TIMEOUT Timeout events
0x4 = PERF Performance metrics
0x8 = ADDR_MATCH Address match events
0x9 = APB APB-specific events
0xF = DEBUG Debug events
Bits [123:109] - Reserved (15 bits, forward-compat slack)
Bits [108:105] - Protocol (4 bits): 0x0=AXI, 0x1=AXIS, 0x2=APB, 0x3=ARB, 0x4=CORE
Bits [104:97] - Event Code (8 bits, protocol-specific)
Bits [96:88] - Channel ID (9 bits – AXI ID or channel index)
Bits [87:72] - Agent ID (16 bits, from AGENT_ID parameter)
Bits [71:64] - Unit ID (8 bits, from UNIT_ID parameter)
Bits [63:0] - Event Data (64 bits – full address, latency, etc.)

26.7.2 Event Types

26.7.2.1 Error Packets (Type=0)

Event Code	Description	Event Data
0x1	SLVERR response	Transaction ID, address[18:0]
0x2	DECERR response	Transaction ID, address[18:0]
0x3	Orphan data	Transaction ID, beat count
0x4	Protocol violation	Violation code
0x5	Poison detected	Transaction ID, beat number
0x6	Tag mismatch	Transaction ID,

Event Code	Description	Event Data
0x7	Missing WLAST	expected/actual tags Transaction ID, beat count

26.7.2.2 Completion Packets (Type=1)

Event Code	Description	Event Data
0x0	Write completion	Transaction ID, burst length, latency

26.7.2.3 Timeout Packets (Type=2)

Event Code	Description	Event Data
0x1	AW channel timeout	Transaction ID, cycles elapsed
0x2	W channel timeout	Transaction ID, cycles elapsed
0x3	B channel timeout	Transaction ID, cycles elapsed

26.7.2.4 Performance Packets (Type=4)

Event Code	Description	Event Data
0x1	High latency	Transaction ID, latency (cycles)
0x2	Bandwidth sample	Bytes transferred, time window
0x3	Outstanding count	Max outstanding, average

26.8 Configuration Guide

26.8.1 Common Configurations

26.8.1.1 Functional Verification Mode

```
.cfg_monitor_enable    (1'b1), // Track completions
.cfg_error_enable      (1'b1), // Catch errors
.cfg_timeout_enable    (1'b1), // Detect stalls
.cfg_perf_enable       (1'b0), // Disable (reduces traffic)
```

```
.cfg_timeout_cycles      (16'd1000),
.cfg_latency_threshold   (32'd500)
```

26.8.1.2 Performance Analysis Mode

```
.cfg_monitor_enable      (1'b0), // Disable completions
.cfg_error_enable        (1'b1), // Still catch errors
.cfg_timeout_enable      (1'b0), // Disable
.cfg_perf_enable         (1'b1), // Enable performance metrics
.cfg_latency_threshold   (32'd100)
```

26.8.1.3 Debug Mode

```
.cfg_monitor_enable      (1'b1), // Track all transactions
.cfg_error_enable        (1'b1), // All errors
.cfg_timeout_enable      (1'b1), // All timeouts
.cfg_perf_enable         (1'b0), // Disable perf (too much data)
.cfg_axi_pkt_mask        (16'hFFFF) // All packet types
```

26.8.2 Filter Masks

cfg_axi_pkt_mask bits: - [0]: Error packets - [1]: Completion packets - [2]: Timeout packets - [3]: Threshold packets - [4]: Performance packets - [15:5]: Reserved

Example: `cfg_axi_pkt_mask = 16'h0007` enables error, completion, and timeout packets only.

26.9 Usage Example

```
axi5_slave_wr_mon #(
    .AXI_ID_WIDTH          (8),
    .AXI_ADDR_WIDTH        (32),
    .AXI_DATA_WIDTH        (64),
    .UNIT_ID               (1),
    .AGENT_ID              (13),
    .MAX_TRANSACTIONS       (16),
    .ENABLE_FILTERING       (1),
    .ENABLE_ATOMIC         (1),
    .ENABLE_NSaid          (1),
    .ENABLE_TRACE          (1),
    .ENABLE_MPAM            (1),
    .ENABLE_MECID          (1),
    .ENABLE_UNIQUE         (1),
    .ENABLE_MTE            (1),
    .ENABLE_POISON         (1)
) u_axi5_slave_wr_mon (
    .aclk                   (axi_clk),
    .aresetn                (axi_rst_n),
```



```

// Slave interface (from external master)
.s_axi_awid      (s_axi_awid),
.s_axi_awaddr    (s_axi_awaddr),
// ... (connect all slave AW/W/B signals)

// FUB interface (to backend)
.fub_axi_awid     (mem_awid),
.fub_axi_awaddr   (mem_awaddr),
// ... (connect to memory controller)

// Monitor configuration
.cfg_monitor_enable (1'b1),
.cfg_error_enable   (1'b1),
.cfg_timeout_enable (1'b1),
.cfg_perf_enable    (1'b0),
.cfg_timeout_cycles (16'd1000),
.cfg_latency_threshold (32'd500),
.cfg_axi_pkt_mask   (16'h0007),

// Monitor bus (connect to FIFO or consumer)
.monbus_valid      (mon_valid),
.monbus_ready      (mon_ready),
.monbus_packet     (mon_packet),

// Status
.busy              (slave_wr_busy),
.active_transactions(active_txns),
.error_count       (total_errors),
.transaction_count (total_txns),
.cfg_conflict_error (cfg_conflict)
);

// Downstream packet handling
gaxi_fifo_sync #(.DATA_WIDTH(64), .DEPTH(256)) u_mon_fifo (
.i_clk      (axi_clk),
.i_rst_n    (axi_rst_n),
.i_valid     (mon_valid),
.i_data      (mon_packet),
.o_ready     (mon_ready),
.o_valid     (fifo_valid),
.o_data      (fifo_data),
.i_ready     (consumer_ready)
);

```

26.10 Design Notes

26.10.1 Write-Specific Monitoring

Poison Detection: - WPOISON indicator tracked per beat - Generates error packet when poison detected - Critical for data integrity validation

Burst Completions: - Tracks AWLEN to verify complete bursts - Detects missing WLAST errors - Monitors AW/W channel synchronization

Atomic Operations: - Monitors AWATOP field when ENABLE_ATOMIC=1 - Tracks atomic transaction latency - Detects atomic operation failures

26.10.2 Best Practices

- Monitor packets provide real-time transaction visibility without external logic
 - Filtering reduces monitor bus bandwidth - critical for high-throughput systems
 - Transaction table size (MAX_TRANSACTIONS) must accommodate peak outstanding transactions
 - Performance packets can generate high traffic - use sparingly or with filtering
 - UNIT_ID and AGENT_ID identify this monitor in multi-agent systems
 - Error count saturates at max value (does not wrap)
-

26.11 Related Documentation

- [AXI5 Slave Write](#) - Non-monitored version
 - [AXI5 Slave Write Monitor CG](#) - Clock-gated variant
 - [AXI5 Slave Read Monitor](#) - Read monitor
 - [AXI Monitor Filtered](#) - Monitor core
 - [Monitor Package Spec](#) - Packet format details
-

26.12 Navigation

- [← Back to AXI5 Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

27 AXIL4 Master Read Monitor (Clock-Gated)

Module: `axil4_master_rd_mon_cg.sv` **Base Module:** [axil4_master_rd_mon](#) **Location:** `rtl/amba/axil4/` **Status:** ✓ Production Ready

27.1 Overview

The `axil4_master_rd_mon_cg` module is a clock-gated variant of [axil4_master_rd_mon](#) that adds comprehensive power optimization capabilities through activity-based clock gating.

27.1.1 Key Differences from Base Module

- ✓ **Activity-Based Clock Gating:** Automatically gates clocks when subsystems are idle
- ✓ **Configurable Policies:** Fine-grained control over what gets gated and when
- ✓ **Power Monitoring:** Built-in statistics for clock gating effectiveness
- ✓ **Independent Gating Domains:** Separate control for different functional blocks
- ✓ **Zero Functional Impact:** Maintains 100% functional equivalence with base module

All other functionality is identical to the base module. See [axil4_master_rd_mon.md](#) for complete functional specification.

27.2 When to Use Clock-Gated Variant

Use `axil4_master_rd_mon_cg` when: - Power consumption is a critical concern - Design has periods of inactivity (burst traffic patterns) - FPGA/ASIC has integrated clock gating support - Meeting power budgets for battery-operated systems

Use base module (`axil4_master_rd_mon`) when: - Maximum performance with no power constraints - Continuous high-activity traffic - Simpler design with fewer configuration parameters - Minimizing gate count is priority

27.3 Clock Gating Parameters

In addition to all parameters from [axil4_master_rd_mon](#) (including USE_MONITOR and N_ADDR_RANGES), this module adds:

Parameter	Type	Default	Description
ENABLE_CLOCK_GATING	bit	1	Master enable for clock gating (0=disable all gating)
CG_IDLE_CYCLES	int	8	Number of idle cycles before asserting clock gate
CG_GATE_MONITOR	bit	1	Enable clock gating for monitor logic
CG_GATE_REPORTER	bit	1	Enable clock gating for packet reporter
CG_GATE_TIMERS	bit	1	Enable clock gating for timeout timers

All base-module ports are forwarded unchanged, including the cam_clear control input (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]).

27.3.1 Parameter Relationships

- **ENABLE_CLOCK_GATING = 0:** Disables all clock gating, module behaves identically to base
- **CG_IDLE_CYCLES:** Higher values = more power savings but slower wake-up from idle
- **Individual CG_GATE_* signals:** Allow fine-grained control over which subsystems are gated

27.4 Performance Monitoring

The clock-gated wrapper forwards the base module's full performance-monitoring interface to axi_monitor_base **unchanged** — clock gating neither adds, removes, nor retimes any perfmon port. They behave exactly as documented for [axil4_master_rd_mon](#):

- **Config inputs:** `cfg_perf_enable`, `cfg_start_event_sel`, `cfg_end_event_sel`, `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Status / counters:** `window_active`, `window_cycles`, `perf_prod_cycles`, `perf_bp_cycles`, `perf_starv_cycles`, `perf_idle_cycles`, `perf_beat_count`, `perf_byte_count`, `perf_burst_count`

The completion/threshold/debug enables (`cfg_compl_enable`, `cfg_threshold_enable`, `cfg_debug_enable`) and the synthesis-cone parameters (`ENABLE_ERROR_LOGIC`, `ENABLE_TIMEOUT_LOGIC`, `ENABLE_COMPL_LOGIC`, `ENABLE_THRESHOLD_LOGIC`, `ENABLE_PERF_LOGIC`, `ENABLE_DEBUG_LOGIC`) are likewise forwarded unchanged. The utilization buckets watch the **R** (read-data) channel; for AXI4-Lite each transaction is a single data beat, so `perf_burst_count` counts AR handshakes = transactions.

27.5 Usage Examples

27.5.1 Example 1: Maximum Power Savings (Burst Traffic)

```
axil4_master_rd_mon_cg #(
    // Base parameters (see axil4_master_rd_mon.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating - aggressive power savings
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(4),           // Gate quickly after 4 idle cycles
    .CG_GATE_MONITOR(1),         // Gate monitor logic
    .CG_GATE_REPORTER(1),        // Gate reporter logic
    .CG_GATE_TIMERS(1)           // Gate timers (if no timeouts enabled)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... connect signals same as base module
);
```

27.5.2 Example 2: Balanced Performance and Power

```
axil4_master_rd_mon_cg #(
    // Base parameters
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating - balanced approach
```

```

        .ENABLE_CLOCK_GATING(1),
        .CG_IDLE_CYCLES(16),      // Wait longer before gating (faster
wake-up)
        .CG_GATE_MONITOR(1),      // Gate monitor logic
        .CG_GATE_REPORTER(0),     // Don't gate reporter (always ready)
        .CG_GATE_TIMERS(1)        // Gate timers when not needed
    ) u_cg (
        .aclk(clk),
        .aresetn(rst_n),
        // ... connect signals same as base module
    );

```

27.5.3 Example 3: Clock Gating Disabled (Functional Verification)

```

axil4_master_rd_mon_cg #(
    // Base parameters
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating - DISABLED for verification
    .ENABLE_CLOCK_GATING(0) // Disable all gating
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... connect signals same as base module
);

```

Note: With `ENABLE_CLOCK_GATING=0`, this module is functionally identical to the base module.

27.6 Clock Gating Architecture

27.6.1 Gating Domains

The module implements independent clock gating for these functional blocks:

1. Monitor Logic Domain

- Transaction tracking and state machines
- Gated when: No active transactions for `CG_IDLE_CYCLES`
- Controlled by: `CG_GATE_MONITOR`

2. Reporter Domain

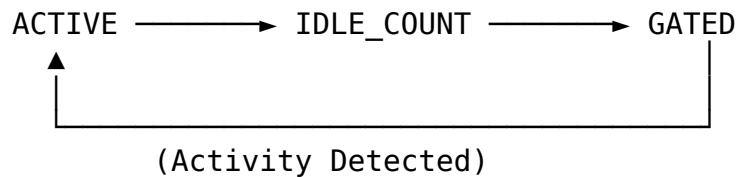
- Monitor packet generation and formatting
- Gated when: No packets to report for `CG_IDLE_CYCLES`

- Controlled by: CG_GATE_REPORTER

3. Timer Domain

- Timeout detection and performance counters
- Gated when: All timeout enables are 0
- Controlled by: CG_GATE_TIMERS

27.6.2 Gating State Machine



States:

- ACTIVE: Clocks enabled, monitoring activity
- IDLE_COUNT: Counting CG_IDLE_CYCLES before gating
- GATED: Clocks disabled, waiting for activity

27.6.3 Wake-Up Latency

Configuration	Wake-Up Time	Use Case
CG_IDLE_CYCLES=4	~4 clock cycles	Low-latency, frequent bursts
CG_IDLE_CYCLES=8	~8 clock cycles	Balanced (default)
CG_IDLE_CYCLES=16	~16 clock cycles	Maximum power savings, infrequent traffic

27.7 Power Savings Analysis

27.7.1 Typical Power Reduction

Based on representative workloads:

Traffic Pattern	Clock Gating Enabled	Power Savings
10% Utilization	Aggressive (CG_IDLE_CYCLES=4)	60-70%
25% Utilization	Balanced (CG_IDLE_CYCLES=8)	45-55%
50% Utilization	Conservative	25-35%

Traffic Pattern	Clock Gating Enabled (CG_IDLE_CYCLES=16)	Power Savings
90% Utilization	Any configuration	5-10%

Note: Actual savings depend on FPGA/ASIC technology, tool implementation, and traffic patterns.

27.7.2 Power Monitoring Signals

The module provides these status signals for power analysis:

Signal	Width	Description
cg_monitor_gated	1	Monitor domain clock is gated
cg_reporter_gated	1	Reporter domain clock is gated
cg_timers_gated	1	Timer domain clock is gated

27.8 Verification Considerations

27.8.1 Clock Gating in Simulation

Recommendation: Disable clock gating during functional verification:

```
// Testbench instantiation
axil4_master_rd_mon_cg #(
    .ENABLE_CLOCK_GATING(0) // Disable for faster simulation
) dut (
    // ... connections
);
```

Rationale: - Simpler waveforms (no clock gating events) - Faster simulation (no gating overhead) - Easier debug (no timing dependencies)

27.8.2 Power Analysis Verification

For power-specific verification:

1. **Enable clock gating** with realistic parameters
2. **Monitor gating signals** (cg_*_gated) to verify expected behavior
3. **Vary traffic patterns** to test gating effectiveness
4. **Check wake-up timing** meets system requirements

27.9 Synthesis Considerations

27.9.1 FPGA Implementations

Xilinx: - Use `ENABLE_CLOCK_GATING=1` with `BUFGCE` primitives - Tool will infer clock enables automatically - Verify with post-synthesis power analysis

Intel (Altera): - Use `ENABLE_CLOCK_GATING=1` with `ALTCLKCTRL` - May need vendor-specific clock gating primitives - Check power reports for gating effectiveness

Lattice: - Basic clock gating supported - May require manual instantiation of clock enables - Verify functionality in timing simulation

27.9.2 ASIC Implementations

- Work with foundry to select appropriate clock gating cells
 - Integrated Clock Gating (ICG) cells provide best results
 - Consider hold-time implications of clock gating
 - Verify power intent with UPF (Unified Power Format)
-

27.10 Related Modules

- [axil4_master_rd_mon](#) - Base module (non-clock-gated)
 - [axi_monitor_base](#) - Core monitoring infrastructure
 - [axi_monitor_filtered](#) - Filtering capabilities
-

27.11 See Also

- **Power Optimization Guide:** [docs/POWER_OPTIMIZATION_GUIDE.md](#)
 - **Clock Gating Best Practices:** [docs/CLOCK_GATING_GUIDE.md](#)
 - **AMBA Subsystem Overview:** [docs/markdown/RTLAmba/overview.md](#)
-

27.12 Navigation

- [← Back to Base Module](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

28 AXIL4 Master Read with Monitoring

Module: axil4_master_rd_mon.sv **Location:** rtl/amba/axil4/ **Status:** ✓ Production Ready

28.1 Overview

Combines [axil4_master_rd](#) with the core **axi_monitor_filtered** for transaction monitoring. Simplified for AXI4-Lite (single-beat, no burst, fixed ID=0).

28.1.1 Key Features

- ✓ All features of base **axil4_master_rd** module
- ✓ **Integrated Monitoring:** Uses shared axi_monitor_filtered (rtl/amba/shared/)
- ✓ **3-Level Filtering:** Packet type masks, error routing, event masking
- ✓ **Error Detection:** Protocol violations, timeouts, orphans
- ✓ **128-bit Monitor Bus:** Standardized packet format paired with 64-bit side-band timestamp
- ✓ **Reduced Complexity:** MAX_TRANSACTIONS=8 (vs 16-32 for AXI4)

28.2 Additional Parameters (Beyond Base Module)

Parameter	Type	Default	Description
UNIT_ID	int	1	4-bit unit identifier (masters typically=1)
AGENT_ID	int	10	8-bit agent identifier for this monitor
MAX_TRANSACTIONS	int	8	Max outstanding transactions (reduced for AXIL)
ENABLE_FILTERING	bit	1	Enable 3-level packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor

Parameter	Type	Default	Description
			and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
N_ADDR_RANGES	int	0	Number of address-range comparators. 0 = checker omitted (zero area). >0 = N independent [low, high] ranges; hits emit PktTypeError + AXI_ERR_ADDR_RANGE on monbus.

28.2.1 Synthesis-Cone Parameters

Each detection cone can be compiled out to save area. These are forwarded to `axi_monitor_base`; by default the classic cones are on and the debug cone is off.

Parameter	Type	Default	Effect when 0
ENABLE_ERROR_LOGIC	bit	1	Drop the error-detection cone
ENABLE_TIMEOUT_LOGIC	bit	1	Drop the timeout cone and the <code>axi_monitor_timeout</code> instance
ENABLE_COMPL_LOGIC	bit	1	Drop the completion cone
ENABLE_THRESHOLD_LOGIC	bit	1	Drop the threshold cone
ENABLE_PERF_LOGIC	bit	1	Drop the perfmon window + counters
ENABLE_DEBUG_LOGIC	bit	0	(off by default) drop the debug cone

The former `CAM_PIPELINE` / `TRANS_CAM_PIPELINE` parameters were removed — the transaction CAM is now always pipelined. Inside this wrapper `ENABLE_PERF_PACKETS` is tied to 1'b1 (perf packet path always

instantiated, gated by ENABLE_PERF_LOGIC) and ENABLE_DEBUG_MODULE is tied to 1'b0.

28.3 Performance Monitoring

When performance monitoring is enabled, the wrapper forwards a **measurement-window state machine** plus a bank of R-channel (read-data) utilization counters to `axi_monitor_base`. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows so the host can read a completed window's totals. The counters advance only while `cfg_perf_enable = 1`; `ENABLE_PERF_LOGIC = 0` drops the whole block at synthesis.

⚠ Never enable completion (`cfg_compl_enable`) and performance (`cfg_perf_enable`) packets simultaneously — see `docs/AXI_Monitor_Configuration_Guide.md`.

28.3.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**:

- `cfg_start_event_sel / cfg_end_event_sel` (3-bit) select the event source (e.g. 3'b010 selects the `cfg_perf_enable` edge).
- `cfg_start_trigger / cfg_end_trigger` pulses fire the window directly from an engine or CSR.
- `cfg_window_force_close` is a software override that closes the window immediately.

While the window is open, `window_active` is high and `window_cycles` [31:0] free-runs, counting every clock elapsed inside the window.

28.3.2 Utilization Counters (R channel)

Every in-window cycle is classified by the R-channel valid/ready into exactly one of four buckets:

Output	Width	Condition	Meaning
<code>perf_prod_cycles</code>	32	<code>rvalid && rready</code>	productive beat transferred
<code>perf_bp_cycles</code>	32	<code>rvalid && !rready</code>	back-pressure (data offered, consumer not ready)
<code>perf_starv_cyc</code>	32	<code>!rvalid && rready</code>	starvation

Output	Width	Condition	Meaning
les			(consumer ready, no valid data)
perf_idle_cycles	32	!rvalid && !rready	idle

The four buckets sum to window_cycles, so utilization = perf_prod_cycles / window_cycles.

28.3.3 Throughput Counters

Output	Width	Meaning
perf_beat_count	32	data beats transferred (= perf_prod_cycles, 1 beat/cycle)
perf_byte_count	64	bytes transferred = beats × (1 << AxSIZE); AXIL is fixed at AxSIZE=3'b010 (4 bytes)
perf_burst_count	32	AR (read-address) handshakes

AXI4-Lite note: every transaction is a single data beat (ARLEN is implicitly 0), so perf_burst_count counts AR handshakes = transactions and perf_beat_count equals the transaction count. Average burst length (perf_beat_count / perf_burst_count) is therefore always 1.

28.4 Monitor Backpressure (block_ready)

The monitor exposes a block_ready signal that goes low when its internal FIFO is saturated and cannot accept a new in-flight transaction. The wrapper ANDs block_ready into the upstream-facing fub_axil_arready so a saturated monitor stalls new transactions on the wire instead of dropping events.

- **Where the stall lands:** the upstream fub_axil_arready is forced low until the monitor drains.
- **When USE_MONITOR=0:** block_ready is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.

This replaces a previous bug where block_ready was left unconnected and a full monitor FIFO would silently lose events.

28.5 Address-Range Checker

The wrapper can be parameterized with `N_ADDR_RANGES > 0` to instantiate an N-comparator address-range checker that watches every accepted AR handshake and emits a `PktTypeError` monbus packet (event code `AXI_ERR_ADDR_RANGE = 8'h0D`) when an address falls inside any of the configured `[low, high]` inclusive ranges.

Config inputs (active only when `N_ADDR_RANGES > 0`): - `cfg_addr_check_enable` — master on/off for the checker. - `cfg_addr_range_enable[N-1:0]` — per-range enable bit. - `cfg_addr_range_low[N-1:0][AXIL_ADDR_WIDTH-1:0]` — inclusive low bound for each range. - `cfg_addr_range_high[N-1:0][AXIL_ADDR_WIDTH-1:0]` — inclusive high bound for each range.

Event encoding (within the standard 128-bit `monitor_packet_t`, `event_data` field): - `packet_type = PktTypeError (4'h0)` - `protocol = AXI (3'b000)` - `event_code = AXI_ERR_ADDR_RANGE (8'h0D)` - `event_data[63:60] = range_index` (4 bits; supports up to 16 ranges) - `event_data[59:0]` = full matched address (up to 60 bits, zero-padded if narrower)

Exact match: set `cfg_addr_range_low[i] == cfg_addr_range_high[i]`.

Filtering: the existing `cfg_axi_error_mask[13]` bit masks this event code; set it high to suppress range packets without disabling other errors. No new mask wiring needed.

Per-range coalescing: if a range hits again before its packet has been emitted, the latched address is overwritten (latest hit wins). One packet per cycle drains the pending mask via a lowest-index priority encoder. Distinct ranges never lose events; under sustained per-range bursts, only the latest address per range is reported per emission cycle.

28.6 Additional Ports (Beyond Base Module)

28.6.1 Monitor Configuration

Port	Direction	Width	Description
<code>cfg_monitor_enable</code>	Input	1	Enable monitoring
<code>cam_clear</code>	Input	1	Synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. <code>CTRL[4]</code>)
<code>cfg_error</code>	Input	1	Enable error packets

Port	Direction	Width	Description
<code>_enable</code>			
<code>cfg_timeout_enable</code>	Input	1	Enable timeout detection
<code>cfg_completion_enable</code>	Input	1	Enable transaction-completion packets
<code>cfg_threshold_enable</code>	Input	1	Enable threshold-crossed packets
<code>cfg_performance_enable</code>	Input	1	Enable performance packets
<code>cfg_debug_enable</code>	Input	1	Enable debug/trace packets (the 6th “debug cone” reporter sub-block)
<code>cfg_timeout_cycles</code>	Input	16	Timeout threshold (cycles)
<code>cfg_latency_threshold</code>	Input	32	Latency alert threshold

The debug cone is the 6th reporter sub-block (`axi_monitor_reporter_debug`), enabled by `cfg_debug_enable` and synthesized only when `ENABLE_DEBUG_LOGIC = 1`. The related `cfg_debug_level` (4b), `cfg_debug_mask` (16b), and `cfg_active_trans_threshold` (16b) inputs of `axi_monitor_base` are **not** exposed on this AXIL wrapper — they are tied to constants (4'h0, 16'h0, 16'd4) internally.

28.6.2 Performance Monitoring Ports

Port	Direction	Width	Description
<code>cfg_start_event_sel</code>	Input	3	Window start event source select
<code>cfg_end_event_sel</code>	Input	3	Window end event source select
<code>cfg_start_trigger</code>	Input	1	Pulse: open the measurement window
<code>cfg_end_trigger</code>	Input	1	Pulse: close the measurement window

Port	Direction	Width	Description
cfg_window_force_close	Input	1	Software override: force the window closed
window_active	Output	1	High while a measurement window is open
window_cycles	Output	32	Cycles elapsed in the current window
perf_valid_cycles	Output	32	valid && ready cycles
perf_bp_cycles	Output	32	valid && !ready cycles (back-pressure)
perf_starv_cycles	Output	32	!valid && ready cycles (starvation)
perf_idle_cycles	Output	32	!valid && !ready cycles
perf_beat_count	Output	32	data beats transferred
perf_byte_count	Output	64	bytes transferred
perf_burst_count	Output	32	AR (address-phase) handshakes

When perfmon is unused, the integrating block ties cfg_start_event_sel/cfg_end_event_sel to 3'b111 and the remaining cfg_* perfmon inputs to 0. When USE_MONITOR = 0 all perfmon outputs are tied to 0.

28.6.3 Filtering Masks (7 masks total)

Port	Description
cfg_axi_pkt_mask	Drop mask for packet types
cfg_axi_err_select	Error select mask
cfg_axi_timeout_mask	Timeout event mask
cfg_axi_compl_mask	Completion event mask
cfg_axi_perf_mask	Performance event mask
cfg_axi_debug_mask	Debug event mask

Port	Description
cfg_axi_full_mask	Full event mask

28.6.4 Monitor Bus Output

Port	Direction	Width	Description
monbus_pkt_valid	Output	1	Monitor packet valid
monbus_pkt_ready	Input	1	Downstream ready
monbus_pkt_data	Output	128	monitor_packet_t (128-bit format)
monbus_timestamp	Output	64	monbus_timestamp_t paired atomically with monbus_pkt_data
i_mon_time	Input	64	Free-running counter from monbus_axil_group, sampled at packet emission

28.6.5 Status

Port	Direction	Width	Description
busy	Output	1	Interface active
active_transactions	Output	8	Current outstanding count
error_count	Output	16	Cumulative error count

28.7 Usage Example

```
axil4_master_rd_mon #(
    // Base parameters
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),
```

```

    // Monitor parameters
    .UNIT_ID(1),
    .AGENT_ID(10),
    .MAX_TRANSACTIONS(8),
    .ENABLE_FILTERING(1)
) u_axil_rd_mon (
    .aclk(axi_clk),
    .aresetn(axi_resetn),

    // AXIL interfaces (same as axil4_master_rd)
    .fub_axil_araddr(cpu_araddr),
    // ... other AR/R signals ...

    // Monitor configuration
    .cfg_monitor_enable(1'b1),
    .cfg_error_enable(1'b1),
    .cfg_timeout_enable(1'b1),
    .cfg_perf_enable(1'b0),          // Avoid congestion
    .cfg_timeout_cycles(16'd1000),
    .cfg_latency_threshold(32'd500),

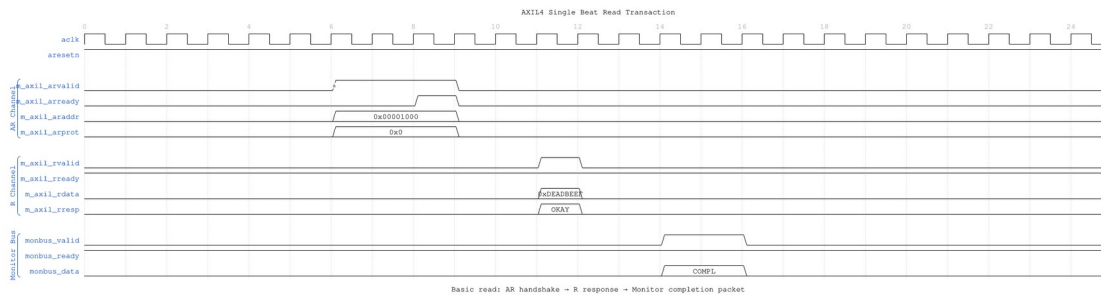
    // Filtering masks
    .cfg_axi_pkt_mask(16'b1111_1111_0000_0011),
    // ... other masks ...

    // Monitor bus
    .monbus_pkt_valid(mon_valid),
    .monbus_pkt_ready(mon_ready),
    .monbus_pkt_data(mon_data)
);

```

28.8 Timing Diagrams

28.8.1 Scenario 1: Single-Beat Read Transaction

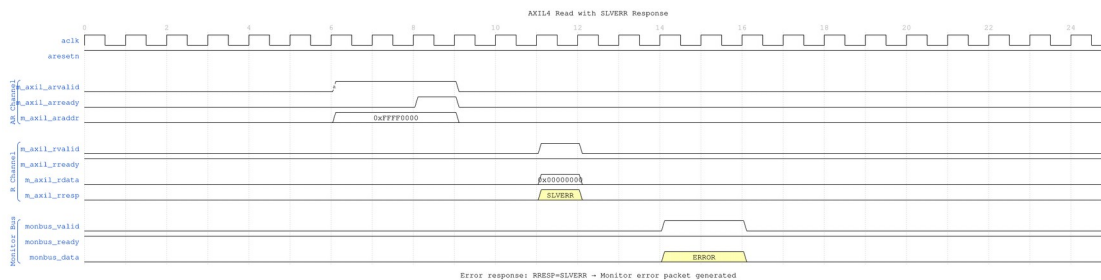


Single Beat Read

WaveJSON: [single_beat_read_001.json](#)

Key Observations: - AR channel handshake: ARVALID asserted, ARREADY responds - R channel response: Slave returns data with RRESP=OKAY - Monitor generates completion packet when R phase completes - Single-beat transaction: No burst length (implicit ARLEN=0)

28.8.2 Scenario 2: Read Error (SLVERR)

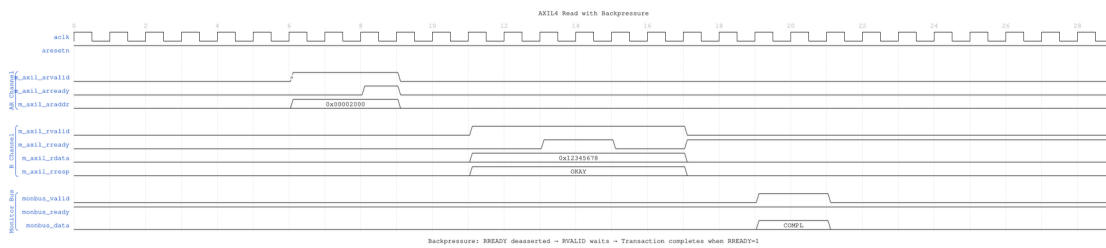


Read Error SLVERR

WaveJSON: [read_error_slvrr_001.json](#)

Key Observations: - Invalid address triggers RRESP=SLVERR - Monitor detects error response and generates ERROR packet - Transaction completes despite error (data may be undefined) - Error packet includes address and response code

28.8.3 Scenario 3: Read with Backpressure



Read Backpressure

WaveJSON: [read_backpressure_001.json](#)

Key Observations: - Master not ready: RREADY deasserted - Slave holds RVALID until RREADY=1 - Monitor tracks extended latency - Completion packet generated after handshake

28.9 AXI4-Lite Simplifications

vs Full AXI4 Monitoring: - **Fixed ID:** Always ID=0 (no out-of-order tracking) - **Single-Beat:** No burst handling (ARLEN implicitly 0) - **Reduced Tables:** MAX_TRANSACTIONS=8 (vs 16-32 for AXI4) - **Same Infrastructure:** Uses axi_monitor_filtered like AXI4

28.10 Related Documentation

28.10.1 Base Module

- [axi4_master_rd](#) - Functional module documentation

28.10.2 Monitor Infrastructure

- [AXI4 Master Read Mon](#) - Full AXI4 monitoring (detailed reference)
- [axi_monitor_filtered](#) - Core monitor engine (rtl/amba/shared/)
- [Monitor Configuration Guide](#) - Configuration strategies

28.10.3 Related Modules

- [axi4_master_wr_mon](#) - Master write with monitoring
- [axi4_slave_rd_mon](#) - Slave read with monitoring

Last Updated: 2025-10-24

28.11 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLAmba Index](#)

29 AXIL4 Master Write Monitor (Clock-Gated)

Module: axil4_master_wr_mon_cg.sv **Base Module:** [axil4_master_wr_mon](#) **Location:** rtl/amba/axil4/ **Status:** ✓ Production Ready

29.1 Quick Reference

This is the **clock-gated variant** of [axil4_master_wr_mon](#).

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ [Clock-Gated Variants Guide](#)

29.2 Summary

The axil4_master_wr_mon_cg module adds power optimization to axil4_master_wr_mon through activity-based clock gating:

- ✓ **Same Functionality:** 100% equivalent to base module
 - ✓ **Power Savings:** 25-70% depending on traffic utilization
 - ✓ **Configurable:** Idle threshold, gating domains, enable/disable
 - ✓ **Zero Overhead When Disabled:** ENABLE_CLOCK_GATING=0 → identical to base
-

29.3 Common Parameters

In addition to all [axil4_master_wr_mon](#) parameters (including USE_MONITOR and N_ADDR_RANGES):

Parameter	Default	Description
ENABLE_CLOCK_GATING	1	Master enable (0=disable, identical to base)
CG_IDLE_CYCLES	8	Cycles before clock gating activates
CG_GATE_*	1	Domain-specific gating enables
USE_MONITOR	1	Synthesis-time monitor enable (forwarded to inner monitor).

All base-module ports are forwarded unchanged, including the `cam_clear` control input (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]).

29.4 Performance Monitoring

The clock-gated wrapper forwards the base module's full performance-monitoring interface to `axi_monitor_base` **unchanged** — clock gating neither adds, removes, nor retimes any perfmon port. They behave exactly as documented for [axi4_master_wr_mon](#):

- **Config inputs:** `cfg_perf_enable`, `cfg_start_event_sel`, `cfg_end_event_sel`, `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Status / counters:** `window_active`, `window_cycles`, `perf_prod_cycles`, `perf_bp_cycles`, `perf_starv_cycles`, `perf_idle_cycles`, `perf_beat_count`, `perf_byte_count`, `perf_burst_count`

The completion/threshold/debug enables (`cfg_compl_enable`, `cfg_threshold_enable`, `cfg_debug_enable`) and the synthesis-cone parameters (`ENABLE_ERROR_LOGIC`, `ENABLE_TIMEOUT_LOGIC`, `ENABLE_COMPL_LOGIC`, `ENABLE_THRESHOLD_LOGIC`, `ENABLE_PERF_LOGIC`, `ENABLE_DEBUG_LOGIC`) are likewise forwarded unchanged. The utilization buckets watch the **W** (write-data) channel; for AXI4-Lite each transaction is a single data beat, so `perf_burst_count` counts AW handshakes = transactions.

29.5 Quick Usage

```
axil4_master_wr_mon_cg #(
    // Base module parameters (see axil4_master_wr_mon.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating (see CG guide for details)
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... all other ports same as axil4_master_wr_mon
);
```

29.6 Documentation

- **Base Module Functionality:** [axil4_master_wr_mon.md](#)
 - **Clock Gating Guide:** [clock_gated_variants.md](#)
 - **Detailed CG Examples:**
 - [axi4_master_rd_mon_cg.md](#) (AXI4 monitor)
 - [axil4_master_rd_mon_cg.md](#) (AXIL4 monitor)
 - [apb_slave_cg.md](#) (APB interface)
-

29.7 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

30 AXIL4 Master Write with Monitoring

Module: `axil4_master_wr_mon.sv` **Location:** `rtl/amba/axil4/` **Status:** ✓ Production Ready

30.1 Overview

Combines [axil4_master_wr](#) with [axi_monitor_filtered](#) for write transaction monitoring.

30.1.1 Key Features

- ✓ All features of [axil4_master_wr](#)
 - ✓ Integrated write monitoring (AW, W, B channels)
 - ✓ 3-level filtering and error detection
 - ✓ Simplified for AXI4-Lite (MAX_TRANSACTIONS=8)
-

30.2 Additional Parameters

Identical to [axil4_master_rd_mon](#) including: - UNIT_ID, AGENT_ID, MAX_TRANSACTIONS, ENABLE_FILTERING, ADD_PIPELINE_STAGE, USE_MONITOR, N_ADDR_RANGES - The synthesis-cone parameters ENABLE_ERROR_LOGIC, ENABLE_TIMEOUT_LOGIC, ENABLE_COMPL_LOGIC, ENABLE_THRESHOLD_LOGIC, ENABLE_PERF_LOGIC (all default 1) and ENABLE_DEBUG_LOGIC (default 0), each of which drops the matching detection cone when set to 0. ENABLE_PERF_PACKETS is tied 1'b1 and ENABLE_DEBUG_MODULE 1'b0 internally.

See [axil4_master_rd_mon](#) for complete parameter descriptions. The removed CAM_PIPELINE / TRANS_CAM_PIPELINE parameters no longer exist (the CAM is always pipelined).

30.3 Performance Monitoring

When performance monitoring is enabled, the wrapper forwards a **measurement-window state machine** plus a bank of W-channel (write-data) utilization counters to [axi_monitor_base](#). All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows so the host can read a completed window's totals. The counters advance only while `cfg_perf_enable = 1`; `ENABLE_PERF_LOGIC = 0` drops the whole block at synthesis.

 **Never enable completion (`cfg_compl_enable`) and performance (`cfg_perf_enable`) packets simultaneously — see [docs/AXI_Monitor_Configuration_Guide.md](#).**

30.3.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**:

- `cfg_start_event_sel / cfg_end_event_sel` (3-bit) select the event source (e.g. 3'b010 selects the `cfg_perf_enable` edge).

- `cfg_start_trigger / cfg_end_trigger` pulses fire the window directly from an engine or CSR.
- `cfg_window_force_close` is a software override that closes the window immediately.

While the window is open, `window_active` is high and `window_cycles` [31:0] free-runs, counting every clock elapsed inside the window.

30.3.2 Utilization Counters (W channel)

Every in-window cycle is classified by the W-channel valid/ready into exactly one of four buckets:

Output	Width	Condition	Meaning
<code>perf_prod_cycles</code>	32	<code>wvalid && wready</code>	productive beat transferred
<code>perf_bp_cycles</code>	32	<code>wvalid && !wready</code>	back-pressure (data offered, consumer not ready)
<code>perf_starv_cycles</code>	32	<code>!wvalid && wready</code>	starvation (consumer ready, no valid data)
<code>perf_idle_cycles</code>	32	<code>!wvalid && !wready</code>	idle

The four buckets sum to `window_cycles`, so $\text{utilization} = \text{perf_prod_cycles} / \text{window_cycles}$.

30.3.3 Throughput Counters

Output	Width	Meaning
<code>perf_beat_count</code>	32	data beats transferred (= <code>perf_prod_cycles</code> , 1 beat/cycle)
<code>perf_byte_count</code>	64	bytes transferred = beats × (1 << <code>AxSIZE</code>); AXIL is fixed at <code>AxSIZE=3'b010</code> (4 bytes)
<code>perf_burst_count</code>	32	AW (write-address) handshakes

AXI4-Lite note: every transaction is a single data beat (AWLEN is implicitly 0), so `perf_burst_count` counts AW handshakes = transactions and `perf_beat_count` equals the transaction count. Average burst length (`perf_beat_count / perf_burst_count`) is therefore always 1.

30.3.4 Perfmon & Additional Control Ports

The wrapper adds the perfmon config/status ports (`cfg_start_event_sel`, `cfg_end_event_sel`, `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`, `window_active`, `window_cycles`, `perf_prod_cycles`, `perf_bp_cycles`, `perf_starv_cycles`, `perf_idle_cycles`, `perf_beat_count`, `perf_byte_count`, `perf_burst_count`) and the completion/threshold/debug enables (`cfg_compl_enable`, `cfg_threshold_enable`, `cfg_debug_enable`) with the same directions/widths and semantics as

[**axil4_master_rd_mon**](#) — the only difference is that `perf_burst_count` counts AW handshakes and the utilization buckets watch the W channel. As on the read monitor, `cfg_debug_level`/`cfg_debug_mask`/`cfg_active_trans_threshold` are tied to constants internally and not exposed.

30.4 Monitor Backpressure (block_ready)

The monitor exposes a `block_ready` signal that goes low when its internal FIFO is saturated and cannot accept a new in-flight transaction. The wrapper ANDs `block_ready` into the upstream-facing `fub_axil_awready` so a saturated monitor stalls new transactions on the wire instead of dropping events.

- **Where the stall lands:** the upstream `fub_axil_awready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`:** `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.

This replaces a previous bug where `block_ready` was left unconnected and a full monitor FIFO would silently lose events.

30.5 Address-Range Checker

Identical to [**axil4_master_rd_mon**](#) except the checker watches AW (write address) handshakes instead of AR (read address) handshakes. The `cfg_addr_*` configuration inputs and monbus event encoding are the same. See the read monitor's Address-Range Checker section for full details.

30.6 Additional Ports

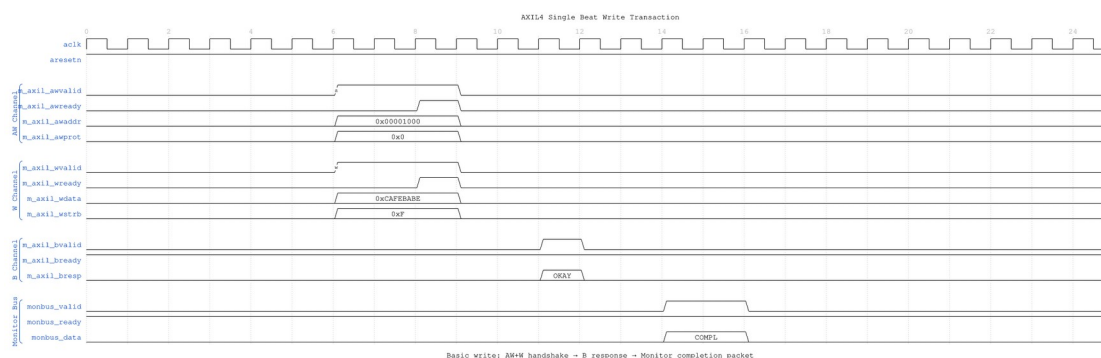
Same as [axil4_master_rd_mon](#): - Monitor configuration inputs, including `cfg_error_enable`, `cfg_timeout_enable`, `cfg_compl_enable`, `cfg_threshold_enable`, `cfg_perf_enable`, `cfg_debug_enable` - `cam_clear` control input (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]) - Filtering masks (7 masks) - The performance-monitoring config/status ports (see the [Performance Monitoring](#) section above and the [read-monitor port table](#)) - Monitor bus output (valid/ready/data) - Status outputs (busy, active_transactions, error_count)

30.7 Usage

```
axil4_master_wr_mon #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .UNIT_ID(1),
    .AGENT_ID(11), // Different from read monitor
    .MAX_TRANSACTIONS(8)
) u_axil_wr_mon (
    // AXIL write interfaces (AW, W, B)
    // Monitor configuration and bus
    // Same pattern as axil4_master_rd_mon
);
```

30.8 Timing Diagrams

30.8.1 Scenario 1: Single-Beat Write Transaction



Single Beat Write

WaveJSON: [single_beat_write_001.json](#)

30.8.2 Scenario 2: Write Error (SLVERR)



Key Observations: - Invalid address triggers BRESP=SLVERR - Monitor detects error response and generates ERROR packet - Write data sent but not committed by slave - Error packet includes address and response code

30.8.3 Scenario 3: Write with Backpressure



Key Observations: - Master not ready: BREADY deasserted - Slave holds BVALID until BREADY=1 - Monitor tracks extended write latency - Completion packet generated after B handshake

30.9 Related Modules

- Page 368 of 599

- [axil4_master_rd_mon](#) - Read monitor counterpart
 - [AXI4 Master Write Mon](#) - Full AXI4 reference
-

Last Updated: 2025-10-24

31 AXIL4 Slave Read Monitor (Clock-Gated)

Module: `axil4_slave_rd_mon_cg.sv` **Base Module:** [axil4_slave_rd_mon](#) **Location:** `rtl/amba/axil4/` **Status:** ✓ Production Ready

31.1 Quick Reference

This is the **clock-gated variant** of [axil4_slave_rd_mon](#).

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ [Clock-Gated Variants Guide](#)

31.2 Summary

The `axil4_slave_rd_mon_cg` module adds power optimization to `axil4_slave_rd_mon` through activity-based clock gating:

- ✓ **Same Functionality:** 100% equivalent to base module
 - ✓ **Power Savings:** 25-70% depending on traffic utilization
 - ✓ **Configurable:** Idle threshold, gating domains, enable/disable
 - ✓ **Zero Overhead When Disabled:** `ENABLE_CLOCK_GATING=0` → identical to base
-

31.3 Common Parameters

In addition to all [axil4_slave_rd_mon](#) parameters (including `USE_MONITOR` and `N_ADDR_RANGES`):

Parameter	Default	Description
ENABLE_CLOCK_GATING	1	Master enable (0=disable, identical to base)
CG_IDLE_CYCLES	8	Cycles before clock gating activates
CG_GATE_*	1	Domain-specific gating enables
USE_MONITOR	1	Synthesis-time monitor enable (forwarded to inner monitor).

All base-module ports are forwarded unchanged, including the `cam_clear` control input (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]).

31.4 Performance Monitoring

The clock-gated wrapper forwards the base module's full performance-monitoring interface to `axi_monitor_base` **unchanged** — clock gating neither adds, removes, nor retimes any perfmon port. They behave exactly as documented for [axil4_slave_rd_mon](#):

- **Config inputs:** `cfg_perf_enable`, `cfg_start_event_sel`, `cfg_end_event_sel`, `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Status / counters:** `window_active`, `window_cycles`, `perf_prod_cycles`, `perf_bp_cycles`, `perf_starv_cycles`, `perf_idle_cycles`, `perf_beat_count`, `perf_byte_count`, `perf_burst_count`

The completion/threshold/debug enables (`cfg_compl_enable`, `cfg_threshold_enable`, `cfg_debug_enable`) and the synthesis-cone parameters (`ENABLE_ERROR_LOGIC`, `ENABLE_TIMEOUT_LOGIC`, `ENABLE_COMPL_LOGIC`, `ENABLE_THRESHOLD_LOGIC`, `ENABLE_PERF_LOGIC`, `ENABLE_DEBUG_LOGIC`) are likewise forwarded unchanged. The utilization buckets watch the **R** (read-data) channel; for AXI4-Lite each transaction is a single data beat, so `perf_burst_count` counts AR handshakes = transactions.

31.5 Quick Usage

```
axil4_slave_rd_mon_cg #(
    // Base module parameters (see axil4_slave_rd_mon.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating (see CG guide for details)
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... all other ports same as axil4_slave_rd_mon
);
```

31.6 Documentation

- **Base Module Functionality:** [axil4_slave_rd_mon.md](#)
 - **Clock Gating Guide:** [clock_gated_variants.md](#)
 - **Detailed CG Examples:**
 - [axi4_master_rd_mon_cg.md](#) (AXI4 monitor)
 - [axil4_master_rd_mon_cg.md](#) (AXIL4 monitor)
 - [apb_slave_cg.md](#) (APB interface)
-

31.7 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

32 AXIL4 Slave Read with Monitoring

Module: `axil4_slave_rd_mon.sv` **Location:** `rtl/amba/axil4/` **Status:** ✓ Production Ready

32.1 Overview

Combines [axil4_slave_rd](#) with [axi_monitor_filtered](#) for slave-side read monitoring.

32.1.1 Key Features

- ✓ All features of [axil4_slave_rd](#)
 - ✓ Slave-side monitoring (backend latency tracking)
 - ✓ 3-level filtering and error detection
 - ✓ Simplified for AXI4-Lite (MAX_TRANSACTIONS=8)
-

32.2 Additional Parameters

Identical to [axil4_master_rd_mon](#) including N_ADDR_RANGES, but typically: - UNIT_ID = 2 (slaves use different unit ID) - AGENT_ID = 20 (slave agent IDs) - USE_MONITOR (synthesis-time monitor enable)

Also includes the synthesis-cone parameters ENABLE_ERROR_LOGIC, ENABLE_TIMEOUT_LOGIC, ENABLE_COMPL_LOGIC, ENABLE_THRESHOLD_LOGIC, ENABLE_PERF_LOGIC (all default 1) and ENABLE_DEBUG_LOGIC (default 0), each dropping its detection cone when set to 0.

ENABLE_PERF_PACKETS is tied 1'b1 and ENABLE_DEBUG_MODULE 1'b0 internally; the removed CAM_PIPELINE / TRANS_CAM_PIPELINE parameters no longer exist.

For complete parameter descriptions including N_ADDR_RANGES and the synthesis-cone parameters, see [axil4_master_rd_mon](#).

32.3 Performance Monitoring

When performance monitoring is enabled, the wrapper forwards a **measurement-window state machine** plus a bank of R-channel (read-data) utilization counters to [axi_monitor_base](#). All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows so the host can read a completed window's totals. The counters advance only while `cfg_perf_enable = 1`; `ENABLE_PERF_LOGIC = 0` drops the whole block at synthesis.

 **Never enable completion (`cfg_compl_enable`) and performance (`cfg_perf_enable`) packets simultaneously — see [docs/AXI_Monitor_Configuration_Guide.md](#).**

32.3.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**:

- `cfg_start_event_sel` / `cfg_end_event_sel` (3-bit) select the event source (e.g. 3'b010 selects the `cfg_perf_enable` edge).
- `cfg_start_trigger` / `cfg_end_trigger` pulses fire the window directly from an engine or CSR.
- `cfg_window_force_close` is a software override that closes the window immediately.

While the window is open, `window_active` is high and `window_cycles` [31:0] free-runs, counting every clock elapsed inside the window.

32.3.2 Utilization Counters (R channel)

Every in-window cycle is classified by the R-channel valid/ready into exactly one of four buckets:

Output	Width	Condition	Meaning
<code>perf_prod_cycles</code>	32	<code>rvalid && rready</code>	productive beat transferred
<code>perf_bp_cycles</code>	32	<code>rvalid && !rready</code>	back-pressure (data offered, consumer not ready)
<code>perf_starv_cycles</code>	32	<code>!rvalid && rready</code>	starvation (consumer ready, no valid data)
<code>perf_idle_cycles</code>	32	<code>!rvalid && !rready</code>	idle

The four buckets sum to `window_cycles`, so $\text{utilization} = \text{perf_prod_cycles} / \text{window_cycles}$.

32.3.3 Throughput Counters

Output	Width	Meaning
<code>perf_beat_count</code>	32	data beats transferred (= <code>perf_prod_cycles</code> , 1 beat/cycle)
<code>perf_byte_count</code>	64	bytes transferred = beats × (1 << <code>AxSIZE</code>); <code>AXIL</code> is fixed at <code>AxSIZE=3'b010</code> (4 bytes)
<code>perf_burst_count</code>	32	<code>AR</code> (read-address)

Output	Width	Meaning
		handshakes

AXI4-Lite note: every transaction is a single data beat (ARLEN is implicitly 0), so `perf_burst_count` counts AR handshakes = transactions and `perf_beat_count` equals the transaction count. Average burst length (`perf_beat_count / perf_burst_count`) is therefore always 1.

The `perfmon` config/status ports and the `cfg_compl_enable / cfg_threshold_enable / cfg_debug_enable` control inputs have the same directions/widths and semantics as the [read-monitor port table](#). As there, `cfg_debug_level/cfg_debug_mask/cfg_active_trans_threshold` are tied to constants internally and not exposed.

32.4 Monitor Backpressure (block_ready)

The monitor exposes a `block_ready` signal that goes low when its internal FIFO is saturated and cannot accept a new in-flight transaction. The wrapper ANDs `block_ready` into the upstream-facing `s_axil_arready` so a saturated monitor stalls new transactions on the wire instead of dropping events.

- **Where the stall lands:** the upstream `s_axil_arready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`:** `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.

This replaces a previous bug where `block_ready` was left unconnected and a full monitor FIFO would silently lose events.

32.5 Address-Range Checker

Identical to [axil4_master_rd_mon](#) — monitors AR (read address) handshakes and emits address-range violation packets. See the master read monitor's section for full details.

32.6 Additional Ports

Same as [axil4_master_rd_mon](#), including the `cam_clear` control input (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit,

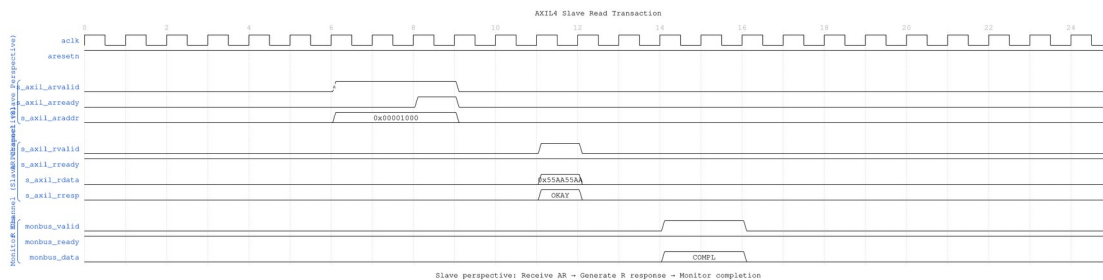
e.g. CTRL[4]), the `cfg_compl_enable` / `cfg_threshold_enable` / `cfg_debug_enable` control enables, and the performance-monitoring config/status ports (see the [Performance Monitoring](#) section above).

32.7 Usage

```
axil4_slave_rd_mon #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .UNIT_ID(2),      // Slave unit ID
    .AGENT_ID(20),    // Slave agent ID
    .MAX_TRANSACTIONS(8)
) u_axil_slave_rd_mon (
    // Slave AXIL interfaces (s_axil_*, fub_*)
    // Monitor configuration and bus
);
```

32.8 Timing Diagrams

32.8.1 Scenario 1: Slave Read Transaction

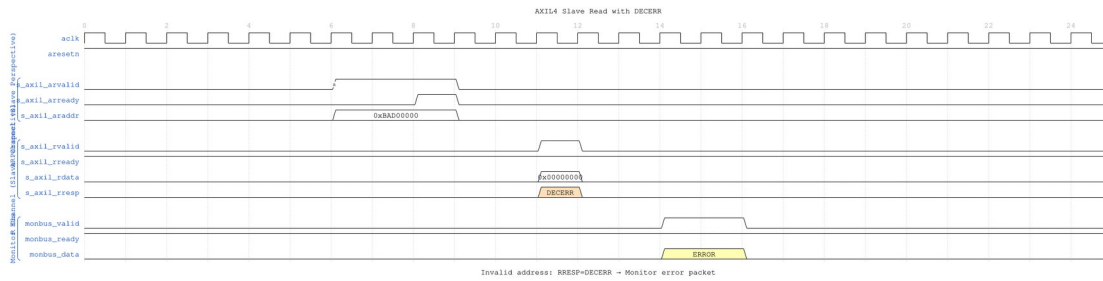


Slave Read Basic

WaveJSON: [slave_read_basic_001.json](#)

Key Observations: - Slave perspective: Receive AR request from master - Slave generates R response with data - Monitor tracks backend read latency (AR → R delay) - Completion packet indicates successful read

32.8.2 Scenario 2: Slave Read Error (DECERR)

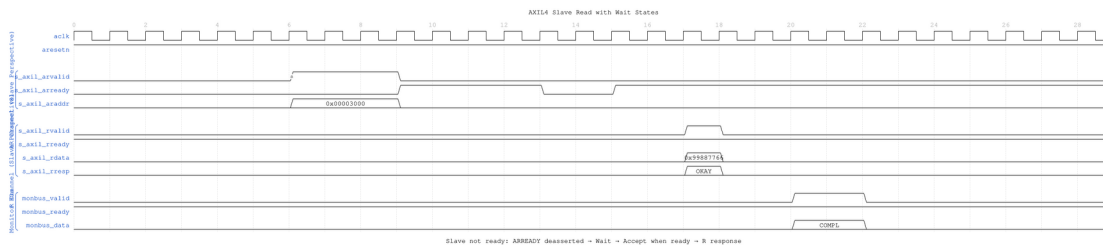


Slave Read DECERR

WaveJSON: [slave_read_decerr_001.json](#)

Key Observations: - Invalid address detected by slave address decoder - Slave returns RRESP=DECERR (decode error) - Monitor generates ERROR packet - Data value is don't-care when error occurs

32.8.3 Scenario 3: Slave Read with Wait States



Slave Read Wait

WaveJSON: [slave_read_wait_001.json](#)

Key Observations: - Slave not ready: ARREADY deasserted (wait states) - Master holds ARVALID until slave accepts - Backend read takes multiple cycles - Monitor tracks full transaction latency

32.9 Related Modules

- [axil4_slave_rd](#) - Base functional module
- [axil4_slave_wr_mon](#) - Write monitor counterpart
- [AXI4 Slave Read Mon](#) - Full AXI4 reference

Last Updated: 2025-10-24

33 AXIL4 Slave Write Monitor (Clock-Gated)

Module: axil4_slave_wr_mon_cg.sv **Base Module:** [axil4_slave_wr_mon](#) **Location:** rtl/amba/axil4/ **Status:** ✓ Production Ready

33.1 Quick Reference

This is the **clock-gated variant** of [axil4_slave_wr_mon](#).

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ [Clock-Gated Variants Guide](#)

33.2 Summary

The axil4_slave_wr_mon_cg module adds power optimization to axil4_slave_wr_mon through activity-based clock gating:

- ✓ **Same Functionality:** 100% equivalent to base module
 - ✓ **Power Savings:** 25-70% depending on traffic utilization
 - ✓ **Configurable:** Idle threshold, gating domains, enable/disable
 - ✓ **Zero Overhead When Disabled:** ENABLE_CLOCK_GATING=0 → identical to base
-

33.3 Common Parameters

In addition to all [axil4_slave_wr_mon](#) parameters (including USE_MONITOR and N_ADDR_RANGES):

Parameter	Default	Description
ENABLE_CLOCK_GATING	1	Master enable (0=disable, identical to base)
CG_IDLE_CYCLES	8	Cycles before clock gating activates
CG_GATE_*	1	Domain-specific gating

Parameter	Default	Description
USE_MONITOR	1	enables Synthesis-time monitor enable (forwarded to inner monitor).

All base-module ports are forwarded unchanged, including the `cam_clear` control input (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]).

33.4 Performance Monitoring

The clock-gated wrapper forwards the base module's full performance-monitoring interface to `axil_monitor_base` **unchanged** — clock gating neither adds, removes, nor retimes any perfmon port. They behave exactly as documented for [axil4_slave_wr_mon](#):

- **Config inputs:** `cfg_perf_enable`, `cfg_start_event_sel`, `cfg_end_event_sel`, `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Status / counters:** `window_active`, `window_cycles`, `perf_prod_cycles`, `perf_bp_cycles`, `perf_starv_cycles`, `perf_idle_cycles`, `perf_beat_count`, `perf_byte_count`, `perf_burst_count`

The completion/threshold/debug enables (`cfg_compl_enable`, `cfg_threshold_enable`, `cfg_debug_enable`) and the synthesis-cone parameters (`ENABLE_ERROR_LOGIC`, `ENABLE_TIMEOUT_LOGIC`, `ENABLE_COMPL_LOGIC`, `ENABLE_THRESHOLD_LOGIC`, `ENABLE_PERF_LOGIC`, `ENABLE_DEBUG_LOGIC`) are likewise forwarded unchanged. The utilization buckets watch the **W** (write-data) channel; for AXI4-Lite each transaction is a single data beat, so `perf_burst_count` counts AW handshakes = transactions.

33.5 Quick Usage

```
axil4_slave_wr_mon_cg #(
    // Base module parameters (see axil4_slave_wr_mon.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating (see CG guide for details)
    .ENABLE_CLOCK_GATING(1),
```

```

        .CG_IDLE_CYCLES(8)
    ) u_cg (
        .aclk(clk),
        .aresetn(rst_n),
        // ... all other ports same as axil4_slave_wr_mon
    );

```

33.6 Documentation

- **Base Module Functionality:** [axil4_slave_wr_mon.md](#)
 - **Clock Gating Guide:** [clock_gated_variants.md](#)
 - **Detailed CG Examples:**
 - [axi4_master_rd_mon_cg.md](#) (AXI4 monitor)
 - [axil4_master_rd_mon_cg.md](#) (AXIL4 monitor)
 - [apb_slave_cg.md](#) (APB interface)
-

33.7 Navigation

- [← Back to AXIL4 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

34 AXIL4 Slave Write with Monitoring

Module: `axil4_slave_wr_mon.sv` **Location:** `rtl/amba/axil4/` **Status:** ✓ Production Ready

34.1 Overview

Combines [axil4_slave_wr](#) with `axi_monitor_filtered` for slave-side write monitoring.

34.1.1 Key Features

- ✓ All features of `axil4_slave_wr`
 - ✓ Slave-side write monitoring (AW, W, B channels)
 - ✓ Backend write latency tracking
 - ✓ 3-level filtering and error detection
-

34.2 Additional Parameters

- UNIT_ID = 2 (slaves)
- AGENT_ID = 21 (slave write agent)
- USE_MONITOR (synthesis-time monitor enable)
- N_ADDR_RANGES (address-range comparator count)
- Others same as [axil4_master_rd_mon](#)

Also includes the synthesis-cone parameters ENABLE_ERROR_LOGIC, ENABLE_TIMEOUT_LOGIC, ENABLE_COMPL_LOGIC, ENABLE_THRESHOLD_LOGIC, ENABLE_PERF_LOGIC (all default 1) and ENABLE_DEBUG_LOGIC (default 0), each dropping its detection cone when set to 0. ENABLE_PERF_PACKETS is tied 1'b1 and ENABLE_DEBUG_MODULE 1'b0 internally; the removed CAM_PIPELINE / TRANS_CAM_PIPELINE parameters no longer exist.

For complete parameter descriptions, see [axil4_master_rd_mon](#).

34.3 Performance Monitoring

When performance monitoring is enabled, the wrapper forwards a **measurement-window state machine** plus a bank of W-channel (write-data) utilization counters to `axi_monitor_base`. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows so the host can read a completed window's totals. The counters advance only while `cfg_perf_enable = 1`; `ENABLE_PERF_LOGIC = 0` drops the whole block at synthesis.

⚠ Never enable completion (`cfg_compl_enable`) and performance (`cfg_perf_enable`) packets simultaneously — see [docs/AXI_Monitor_Configuration_Guide.md](#).

34.3.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**:

- `cfg_start_event_sel / cfg_end_event_sel` (3-bit) select the event source (e.g. 3'b010 selects the `cfg_perf_enable` edge).
- `cfg_start_trigger / cfg_end_trigger` pulses fire the window directly from an engine or CSR.
- `cfg_window_force_close` is a software override that closes the window immediately.

While the window is open, `window_active` is high and `window_cycles` [31:0] free-runs, counting every clock elapsed inside the window.

34.3.2 Utilization Counters (W channel)

Every in-window cycle is classified by the W-channel valid/ready into exactly one of four buckets:

Output	Width	Condition	Meaning
perf_prod_cycles	32	wvalid && wready	productive beat transferred
perf_bp_cycles	32	wvalid && !wready	back-pressure (data offered, consumer not ready)
perf_starv_cycles	32	!wvalid && wready	starvation (consumer ready, no valid data)
perf_idle_cycles	32	!wvalid && !wready	idle

The four buckets sum to window_cycles, so utilization = perf_prod_cycles / window_cycles.

34.3.3 Throughput Counters

Output	Width	Meaning
perf_beat_count	32	data beats transferred (= perf_prod_cycles, 1 beat/cycle)
perf_byte_count	64	bytes transferred = beats × (1 << AxSIZE); AXIL is fixed at AxSIZE=3 ' b010 (4 bytes)
perf_burst_count	32	AW (write-address) handshakes

AXI4-Lite note: every transaction is a single data beat (AWLEN is implicitly 0), so perf_burst_count counts AW handshakes = transactions and perf_beat_count equals the transaction count. Average burst length (perf_beat_count / perf_burst_count) is therefore always 1.

The perfmon config/status ports and the cfg_compl_enable / cfg_threshold_enable / cfg_debug_enable control inputs have the same directions/widths and semantics as the [read-monitor port table](#) (only perf_burst_count counts AW handshakes here). As there, cfg_debug_level/cfg_debug_mask/cfg_active_trans_threshold are tied to constants internally and not exposed.

34.4 Monitor Backpressure (block_ready)

The monitor exposes a `block_ready` signal that goes low when its internal FIFO is saturated and cannot accept a new in-flight transaction. The wrapper ANDs `block_ready` into the upstream-facing `s_axil_awready` so a saturated monitor stalls new transactions on the wire instead of dropping events.

- **Where the stall lands:** the upstream `s_axil_awready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`:** `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.

This replaces a previous bug where `block_ready` was left unconnected and a full monitor FIFO would silently lose events.

34.5 Address-Range Checker

Identical to [axil4_master_rd_mon](#) except the checker watches AW (write address) handshakes. The `cfg_addr_*` configuration inputs and monbus event encoding are the same. See the read monitor's Address-Range Checker section for full details.

34.6 Additional Ports

Same as [axil4_master_rd_mon](#), including the `cam_clear` control input (Input, 1) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]), the `cfg_compl_enable` / `cfg_threshold_enable` / `cfg_debug_enable` control enables, and the performance-monitoring config/status ports (see the [Performance Monitoring](#) section above).

34.7 Usage

```
axil4_slave_wr_mon #(
    .AXIL_ADDR_WIDTH(32),
    .AXIL_DATA_WIDTH(32),
    .UNIT_ID(2),
    .AGENT_ID(21),
    .MAX_TRANSACTIONS(8)
```

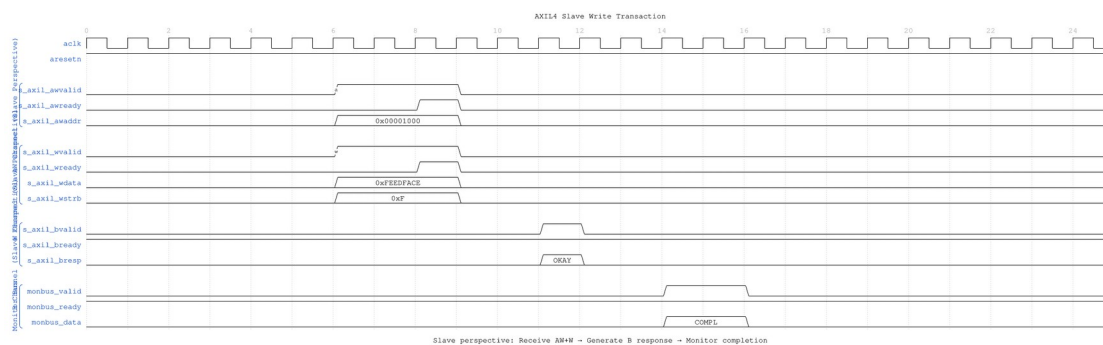
```

) u_axil_slave_wr_mon (
    // Slave AXIL write interfaces
    // Monitor configuration and bus
);

```

34.8 Timing Diagrams

34.8.1 Scenario 1: Slave Write Transaction

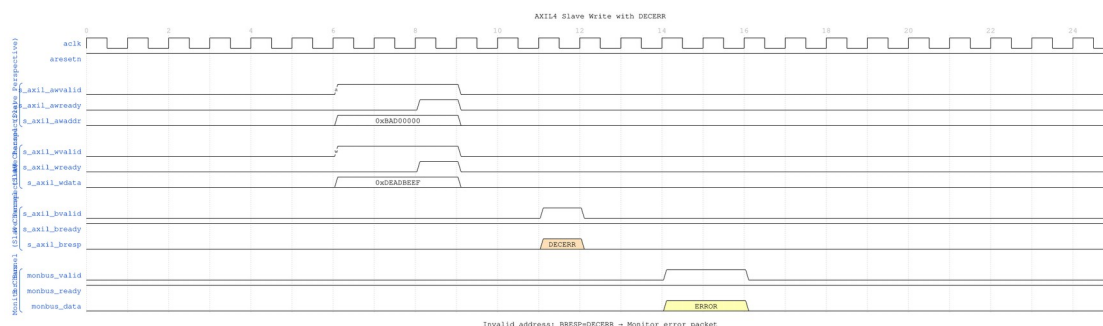


Slave Write Basic

WaveJSON: [slave_write_basic_001.json](#)

Key Observations: - Slave perspective: Receive AW+W simultaneously - Slave commits write to backend storage - Slave generates B response with BRESP=OKAY - Monitor tracks backend write latency

34.8.2 Scenario 2: Slave Write Error (DECERR)

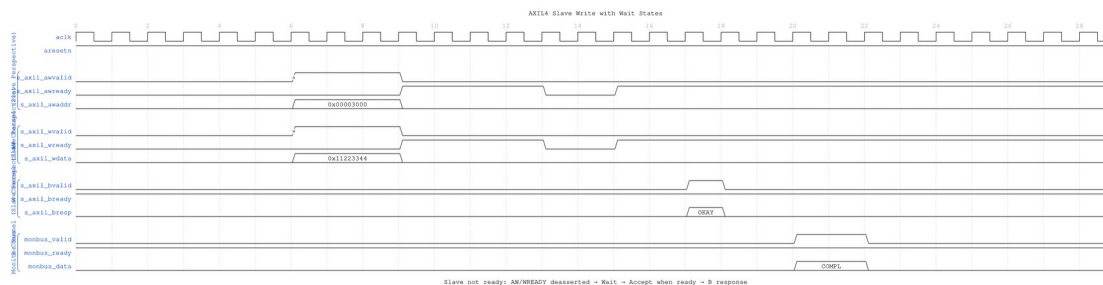


Slave Write DECERR

WaveJSON: [slave_write_decerr_001.json](#)

Key Observations: - Invalid address detected by slave - Write data received but not committed - Slave returns BRESP=DECERR - Monitor generates ERROR packet

34.8.3 Scenario 3: Slave Write with Wait States



Slave Write Wait

WaveJSON: [slave_write_wait_001.json](#)

Key Observations: - Slave not ready: AWREADY/WREADY deasserted - Master holds AW+W until slave accepts - Backend write processing delay - Monitor tracks full write latency including wait states

34.9 Related Modules

- [axi4_slave_wr](#) - Base functional module
- [axi4_slave_rd_mon](#) - Read monitor counterpart
- [AXI4 Slave Write Mon](#) - Full AXI4 reference

Last Updated: 2025-10-24

35 AXI Monitor Address-Range Checker

Module: axi_monitor_addr_check.sv **Location:** rtl/amba/shared/ **Category:** Core Infrastructure **Status:** ✓ Production Ready (Formally Verified)

35.1 Overview

The axi_monitor_addr_check module is a parallel address-range comparator instantiated within AXI monitor wrappers. It observes command-phase handshakes (AR/AW) and detects when addresses fall within user-configured inclusive ranges [low, high], emitting PktTypeError monbus packets with event code AXI_ERR_ADDR_RANGE = 8'h0D.

This is a **shared infrastructure module** used internally by AXI4/AXIL4/AXI5 monitor wrappers when parameterized with `N_ADDR_RANGES > 0`. It is not typically instantiated directly by users but is critical for address-space validation and security monitoring.

35.2 Key Features

- ✓ **Parallel Range Comparators:** N independent [low, high] inclusive range checkers
 - ✓ **Per-Range Enable:** Individual mask bit for each range
 - ✓ **Master Enable:** `cfg_addr_check_enable` gate for all comparators
 - ✓ **Zero-Area Synthesis:** When `N_ADDR_RANGES = 0`, module is completely omitted (no gates, no regs)
 - ✓ **Monbus Integration:** Standard error packet format with event code 8'h0D
 - ✓ **Per-Range Coalescing:** Latest address per range latched; one packet per cycle via priority encoder
 - ✓ **Formal Verification:** All 6 properties proven (PASS)
-

35.3 Module Purpose

The `axi_monitor_addr_check` module enables:

1. **Address-Space Validation:** Detect accesses to forbidden or suspicious memory regions
 2. **Security Monitoring:** Flag unauthorized read/write attempts to protected address ranges
 3. **Debug & Profiling:** Identify hot-spot access patterns and traffic targeting specific regions
 4. **Design Verification:** Verify address constraints in functional simulation and formal proof
-

35.4 Parameters

Parameter	Type	Default	Description
<code>N_ADDR_RANGES</code>	int	4	Number of address-range comparators ($\geq$

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	1). Max 16 (4-bit range index). Address bus width (matches AXI_ADDR_WIDTH of parent monitor). Must be <= 60 (fits the event_data address field).
ID_WIDTH	int	6	Transaction ID width (clipped to 9 bits when copied into the packet's channel_id).
UNIT_ID	logic [7:0]	8'h00	8-bit unit identifier in monitor packets.
AGENT_ID	logic [15:0]	16'h0000	16-bit agent identifier in monitor packets.
IS_READ	bit	1	Build-time flag: 1 if this monitor watches reads (AR), 0 for writes (AW). Drives direction recovery at the consumer (see Event Encoding).

35.5 Port Groups

35.5.1 Clock and Reset

Port	Direction	Width	Description
clk	Input	1	AXI clock
aresetn	Input	1	AXI active-low reset

35.5.2 Side-Band Timestamp

Port	Direction	Width	Description
i_mon_time	Input	64	Free-running counter from monbus group, broadcast to every wrapper via the shared mon_time_w net. Sampled when addr_pkt_valid asserts and driven out on addr_pkt_timestamp.

35.5.3 Command Interface (Snoop)

Port	Direction	Width	Description
cmd_valid	Input	1	Command valid (AR or AW handshake)
cmd_ready	Input	1	Command ready (slave accepting)
cmd_addr	Input	ADDR_WIDTH	Command address (araddr or awaddr)
cmd_id	Input	ID_WIDTH	Command transaction ID (arid or awid)

35.5.4 Configuration Inputs

Port	Direction	Width	Description
cfg_addr_check_enable	Input	1	Master enable for all comparators. 0 = no packets generated.
cfg_addr_range_enable[N_ADD_R_RANGES-	Input	N	Per-range enable bits. 1 = range active, 0 = range disabled.

Port	Direction	Width	Description
1:0] cfg_addr_ range_low [N_ADDR_R ANGES- 1:0] [ADDR_WID TH-1:0]	Input	N × ADDR_WIDT H	Inclusive low bound for each range.
cfg_addr_ range_hig h[N_ADDR_ RANGES- 1:0] [ADDR_WID TH-1:0]	Input	N × ADDR_WIDT H	Inclusive high bound for each range.

35.5.5 Monitor Bus Output

Port	Direction	Width	Description
addr_pkt_ valid	Output	1	Address-check error packet valid
addr_pkt_ ready	Input	1	Downstream ready to accept packet
addr_pkt_ data	Output	128	Monitor packet (monitor_packet_t, 128- bit format)
addr_pkt_ timestamp	Output	64	Sampled i_mon_time paired atomically with addr_pkt_data

35.6 Internal Architecture

The module instantiates N parallel comparators:

1. **Per-Range Comparators:** Each range *i* computes a hit signal `hit[i] = (cfg_addr_range_enable[i] && cmd_valid && cmd_ready && (cmd_addr >= cfg_addr_range_low[i]) && (cmd_addr <= cfg_addr_range_high[i]))`
2. **Per-Range Latch:** On a hit, `r_lat_addr[i]` captures the full `cmd_addr` (up to 60 bits) and `r_lat_id[i]` captures `cmd_id`; a pending bit `r_pending[i]` is set. If another hit occurs before the packet is emitted, the snapshot is overwritten (latest-win coalescing).

3. **Packet Generator:** A lowest-index priority encoder scans `r_pending[N-1:0]` each cycle. When a pending range is found, a packet is generated on the monbus carrying the latched address and 4-bit range index in `event_data`. Direction (read vs. write) is not embedded in the packet — the build-time `IS_READ` parameter determines which channel this instance watches, and consumers recover direction from the emitting monitor's (`UNIT_ID`, `AGENT_ID`). The pending bit is cleared on packet handshake.

This approach ensures: - **No event loss per range:** Distinct ranges never drop hits. - **Bounded latency:** One packet maximum per cycle per range (across all ranges). - **Memory efficient:** $O(N)$ area (N comparators + N latch pairs).

35.7 Event Encoding

Monitor bus packet format (128-bit `monitor_packet_t`):

Bits [127:124] - Packet Type:
4'h0 = PktTypeError

Bits [123:109] - Reserved (15 bits)

Bits [108:105] - Protocol:
4'h0 = `PROTOCOL_AXI`

Bits [104:97] - Event Code:
8'h0D = `AXI_ERR_ADDR_RANGE`

Bits [96:88] - Channel ID:
`cmd_id` clipped/zero-extended to 9 bits

Bits [87:72] - Agent ID:
From `AGENT_ID` parameter (16 bits)

Bits [71:64] - Unit ID:
From `UNIT_ID` parameter (8 bits)

Bits [63:60] - Range Index:
Which range (0 to $N-1$) generated this hit (max 16 ranges)

Bits [59:0] - Matched Address:
Full `cmd_addr`, zero-padded if narrower than 60 bits

is_read flag dropped: Earlier revisions reserved a bit in `event_data` for read-vs-write direction. The 128-bit layout drops it because each AXI monitor instance watches a single direction (set at

build time by IS_READ); consumers recover direction from (UNIT_ID, AGENT_ID). Note: apb_monitor_addr_check still carries is_read since a single APB monitor sees both directions on the same channel.

Side-band timestamp: addr_pkt_timestamp carries the sampled i_mon_time paired atomically with the packet through the arbiter and into monbus_group.

35.8 Formal Properties

All properties proven via formal verification (see formal/amba/axi_monitor_addr_check/formal_axi_monitor_addr_check.sv):

Property	Description	Status
P1: Hit Detection	When <code>cfg_addr_check_enable=1</code> and an address falls within enabled range, <code>pending[i]</code> is set.	PASS
P2: Range Masking	When <code>cfg_addr_check_enable=0</code> , no pending bits are set regardless of address.	PASS
P3: Packet Generation	When a pending range exists and <code>addr_pkt_ready=1</code> , a valid packet is generated with correct range index.	PASS
P4: Pending Clearance	After packet handshake, the corresponding pending bit is cleared in next cycle.	PASS
P5: Latest-Win Coalescing	If a range hits again while pending, the latched address is updated (not lost).	PASS
P6: No Cross-Range Interference	Hits in one range do not affect other ranges' pending/latched state.	PASS

35.9 Configuration Examples

35.9.1 Example 1: Single Range Checker

Monitor writes to protected register space (0x1000_0000 to 0x1000_0FFF):

```
axi4_master_wr_mon #(
    .N_ADDR_RANGES(1),
    .AXI_ADDR_WIDTH(32),
    // ... other parameters ...
) u_wr_mon (
    // ... AXI signals ...

    // Address-range checker config
    .cfg_addr_check_enable(1'b1),
    .cfg_addr_range_enable(1'b1),
    .cfg_addr_range_low(32'h1000_0000),
    .cfg_addr_range_high(32'h1000_0FFF),

    // ... monitor signals ...
);
```

35.9.2 Example 2: Multi-Range Checker

Monitor accesses to three forbidden zones:

```
axi4_master_rd_mon #(
    .N_ADDR_RANGES(3),
    .AXI_ADDR_WIDTH(32),
    // ... other parameters ...
) u_rd_mon (
    // ... AXI signals ...

    // Address-range checker config
    .cfg_addr_check_enable(1'b1),
    .cfg_addr_range_enable(3'b111),
    .cfg_addr_range_low({
        32'h2000_0000, // Range 2: System RAM (protected)
        32'h1000_0000, // Range 1: Registers
        32'h0800_0000  // Range 0: Firmware ROM (read-only)
    }),
    .cfg_addr_range_high({
        32'h2FFF_FFFF,
        32'h1000_FFFF,
        32'h08FF_FFFF
    }),
);
```

```
    // ... monitor signals ...  
);
```

35.9.3 Example 3: Exact-Match Detector

Detect accesses to a single address:

```
.cfg_addr_range_low(32'hDEAD_BEEF),  
.cfg_addr_range_high(32'hDEAD_BEEF) // low == high => exact match  
only
```

35.10 Filtering Integration

The address-range checker output is a standard error packet with event code **8'h0D** (AXI_ERR_ADDR_RANGE).

Gating via existing error mask: - Set `cfg_axi_error_mask[13]` to 1 in the parent monitor to suppress these packets. - **No new mask wiring is needed** — the existing 16-bit error mask vector already has bit 13 reserved for this event code.

Example: Disable address-range packets while keeping other errors:

```
.cfg_axi_error_mask(16'h2000) // Bit 13 high = drop range packets
```

35.11 Integration Notes

35.11.1 Instantiation Pattern

The module is instantiated **inside** AXI monitor wrappers (`axi4_master_wr_mon`, etc.) by the monitor architecture. Users do not instantiate this module directly but configure it via wrapper parameters.

35.11.2 Synthesis Behavior

- **N_ADDR_RANGES = 0:** Module is entirely synthesized away (zero area). All config inputs are tied off. Output packet valid is constant 0.
- **N_ADDR_RANGES > 0:** Full comparator logic synthesized. Area scales with N.

35.11.3 Downstream FIFO

The monbus output should be fed into a standard FIFO (e.g., `gaxi_fifo_sync`) to prevent backpressure stalls:

In practice the `addr_check` output is merged with the reporter's main packet stream by an arbiter (`monbus_arbiter`) that carries packet+timestamp atomically through a 192-bit skid. The arbiter's downstream FIFO sits at the `monbus` group boundary, sized for the aggregate of all per-wrapper streams. A standalone FIFO on the `addr_check` output is normally not needed.

35.12 Related Modules

- [axi_monitor_base](#) — Core monitor infrastructure that instantiates this module
 - [axi_monitor_filtered](#) — 3-level packet filtering (sibling to `addr_check`)
 - [axi4_master_wr_mon](#) — Wrapper that uses this module (`N_ADDR_RANGES` parameter)
-

35.13 See Also

- **Monitor Architecture:** [docs/markdown/RTLamba/overview.md](#)
 - **Monitor Configuration Guide:** [Monitor Base Configuration](#)
 - **Packet Format Specification:** [docs/markdown/RTLamba/includes/monitor_package_spec.md](#)
 - **Formal Verification:** [formal/amba/axi_monitor_addr_check/](#)
-

35.14 Navigation

- [← Back to Shared Infrastructure Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

36 AXI Monitor Base

Module: `axi_monitor_base.sv` **Location:** `rtl/amba/shared/` **Category:** Core Infrastructure
Status: ✓ Production Ready

36.1 Overview

The `axi_monitor_base` module provides Core transaction tracking and event reporting for AXI/AXIL monitors.

This is a **shared infrastructure module** used internally by AXI/AXIL monitors. It is not typically instantiated directly by users but is critical for understanding the monitor architecture.

36.2 Key Features

- ✓ **Transaction-based tracking for AXI and AXI-Lite protocols:** Transaction-based tracking for AXI and AXI-Lite protocols
 - ✓ **Out-of-order transaction handling with ID-based tracking:** Out-of-order transaction handling with ID-based tracking
 - ✓ **Data-before-address support (slave-side scenarios):** Data-before-address support (slave-side scenarios)
 - ✓ **128-bit standardized monitor bus packet output + 64-bit side-band timestamp**
 - ✓ **Configurable performance metrics tracking:** Configurable performance metrics tracking
 - ✓ **Timeout detection and threshold monitoring:** Timeout detection and threshold monitoring
 - ✓ **Debug trace support with verbosity levels:** Debug trace support with verbosity levels
-

36.3 Module Purpose

The `axi_monitor_base` module is the core building block for:

1. **Transaction Tracking:** Maintains state for all outstanding AXI/AXIL transactions
 2. **Event Detection:** Identifies protocol errors, timeouts, threshold violations
 3. **Packet Generation:** Creates standardized 128-bit `monitor_packet_t` records paired with a 64-bit side-band timestamp
 4. **Flow Control:** Manages backpressure and transaction table exhaustion
 5. **Performance Metrics:** Optional latency and throughput tracking
-

36.4 Parameters

36.4.1 Identity and Sizing

Parameter	Type	Default	Description
UNIT_ID	logic [7:0]	8'h09	8-bit unit identifier in monitor packets
AGENT_ID	logic [15:0]	16'h0063	16-bit agent identifier in monitor packets
MAX_TRANSACTIONS	int	16	Maximum outstanding transactions in the CAM
ADDR_WIDTH	int	32	Address bus width
ID_WIDTH	int	8	Transaction ID width (0 for AXIL)
ADDR_BITS_IN_PKT	int	38	Number of address LSBs carried in an error/event packet (clamped to ADDR_WIDTH)
IS_READ	bit	1	1=read monitor (R data channel, AR bursts), 0=write monitor (W data channel, AW bursts)
IS_AXI	bit	1	1=AXI protocol, 0=AXI-Lite
INTR_FIFO_DEPTH	int	8	Depth of the reporter's outgoing interrupt/event FIFO
DEBUG_FIFO_DEPTH	int	8	Depth of the debug-trace FIFO (used when the debug module is enabled)
N_ADDR_RANGES	int	0	Number of address-range comparators. 0 removes the axi_monitor_addr_check block entirely (zero

Parameter	Type	Default	Description
			area)

36.4.2 Master Switches

Parameter	Type	Default	Description
ENABLE_PERF_PACKETS	bit	0	Master switch for performance tracking. Setting it also defaults ENABLE_PERF_LOGIC on, instantiating the measurement window + counters (see Performance Monitoring)
ENABLE_DEBUG_MODULE	bit	0	Master switch for the debug-trace reporter sub-module

36.4.3 Synthesis-Cone Enables (ENABLE_*_LOGIC)

Each detection cone can be compiled out to save area. These gate the **logic**, not just packet emission — a disabled cone synthesizes away entirely. Defaults keep the classic cones on and perf/debug off.

Parameter	Type	Default	Effect when 0
ENABLE_ERROR_LOGIC	bit	1	Drop the error-detection cone (orphans, response errors)
ENABLE_TIMEOUT_LOGIC	bit	1	Drop the timeout cone and the axi_monitor_timeout instance
ENABLE_COMPL_LOGIC	bit	1	Drop the completion cone
ENABLE_THRESHOLD_LOGIC	bit	1	Drop the threshold cone (latency / active-count thresholds)
ENABLE_PERF_LOGIC	bit	= ENABLE_PERF_PACKETS	Drop the perfmon measurement window +

Parameter	Type	Default	Effect when 0
ENABLE_DEBUG_LOGIC	bit	0	counters Drop the debug cone

Removed: the former CAM_PIPELINE / TRANS_CAM_PIPELINE parameters no longer exist. The transaction CAM is now **always pipelined** (one extra cycle of active_count latency), and block_ready carries a MAX-3 margin so the monitor never accepts past MAX_TRANSACTIONS despite the pipeline delay.

36.5 Port Groups

36.5.1 Command Phase Interface (AW/AR)

Port	Direction	Width	Description
cmd_addr	Input	ADDR_WIDTH	Command address value
cmd_id	Input	ID_WIDTH	Transaction ID
cmd_len	Input	8	Burst length (AXI only)
cmd_size	Input	3	Burst size (AXI only)
cmd_burst	Input	2	Burst type (AXI only)
cmd_valid	Input	1	Command valid
cmd_ready	Input	1	Command ready

36.5.2 Data Channel Interface (R/W)

Port	Direction	Width	Description
data_id	Input	ID_WIDTH	Data transaction ID
data_last	Input	1	Last data beat indicator

Port	Direction	Width	Description
data_resp	Input	2	Response code (OKAY/EXOKA Y/SLVERR/DEC ERR)
data_valid	Input	1	Data valid
data_ready	Input	1	Data ready

36.5.3 Response Channel Interface (B)

Port	Direction	Width	Description
resp_id	Input	ID_WIDTH	Response transaction ID
resp_code	Input	2	Response code
resp_valid	Input	1	Response valid
resp_ready	Input	1	Response ready

36.5.4 Configuration Interface

Port	Direction	Width	Description
clear	Input	1	Synchronous clear — passes through to axi_monitor_trans_mgr to empty the transaction CAM and zero the active-count pipeline atomically, without a full aresetn. Pulse one cycle while the monitor is idle.
cfg_freq_sel	Input	4	Frequency selection for timeout scaling
cfg_addr_cnt	Input	4	Address phase timeout count
cfg_data_cnt	Input	4	Data phase timeout count
cfg_resp_cnt	Input	4	Response phase timeout count

Port	Direction	Width	Description
cfg_error_enable	Input	1	Enable error event packets
cfg_compl_enable	Input	1	Enable completion packets
cfg_thres_hold_enable	Input	1	Enable threshold packets
cfg_timeout_enable	Input	1	Enable timeout packets
cfg_perf_enable	Input	1	Enable performance metric packets
cfg_debug_enable	Input	1	Enable debug/trace packets (feeds the debug reporter sub-block)
cfg_active_transaction_threshold	Input	16	Active-transaction count that triggers a threshold packet
cfg_latency_threshold	Input	32	Latency value that triggers a threshold packet
cfg_debug_level	Input	4	Debug verbosity level (only used when ENABLE_DEBUG_MODULE=1)
cfg_debug_mask	Input	16	Debug event-type mask (only used when ENABLE_DEBUG_MODULE=1)

36.5.5 Address-Range Checker Interface

Active only when `N_ADDR_RANGES > 0`; otherwise these inputs are ignored and the `axi_monitor_addr_check` block is not synthesized. Address-range violation packets are the lowest-priority source on the monitor bus (reporter > debug > addr_check).

Port	Direction	Width	Description
cfg_addr_check_enable	Input	1	Master enable for the address-range checker
cfg_addr_range_enable	Input	N_ADDR_RANGES	Per-range enable bit vector

Port	Direction	Width	Description
ble			
cfg_addr_range_low	Input	N_ADDR_RANGES × ADDR_WIDTH	Per-range low (inclusive) address bounds
cfg_addr_range_high	Input	N_ADDR_RANGES × ADDR_WIDTH	Per-range high (inclusive) address bounds

36.5.6 Side-Band Timestamp Input

Port	Direction	Width	Description
i_mon_time	Input	64	Free-running counter from the monbus_group family (any wrapper), sampled at packet emission

36.5.7 Monitor Bus Output

Port	Direction	Width	Description
monbus_valid	Output	1	Monitor packet valid
monbus_ready	Input	1	Monitor packet ready (from downstream)
monbus_packet	Output	128	Standardized monitor_packet_t
monbus_timestamp	Output	64	monbus_timestamp_t paired atomically with monbus_packet
block_ready	Output	1	Flow control signal
busy	Output	1	Monitor is busy indicator
active_count	Output	8	Number of active transactions

The base module multiplexes three internal sources onto the monbus output — reporter packets (highest priority), debug packets, and addr_check error packets — driving monbus_timestamp

from the source's sampled timestamp (`i_mon_time` for reporter/debug; `addr_pkt_timestamp` from the `addr_check` submodule).

36.5.8 Performance-Window Control Inputs

These configure the measurement-window state machine. Tie the selectors to a reserved code (e.g. 3'b111) and the pulses/force-close low at instances that don't use perfmon. See [Performance Monitoring](#).

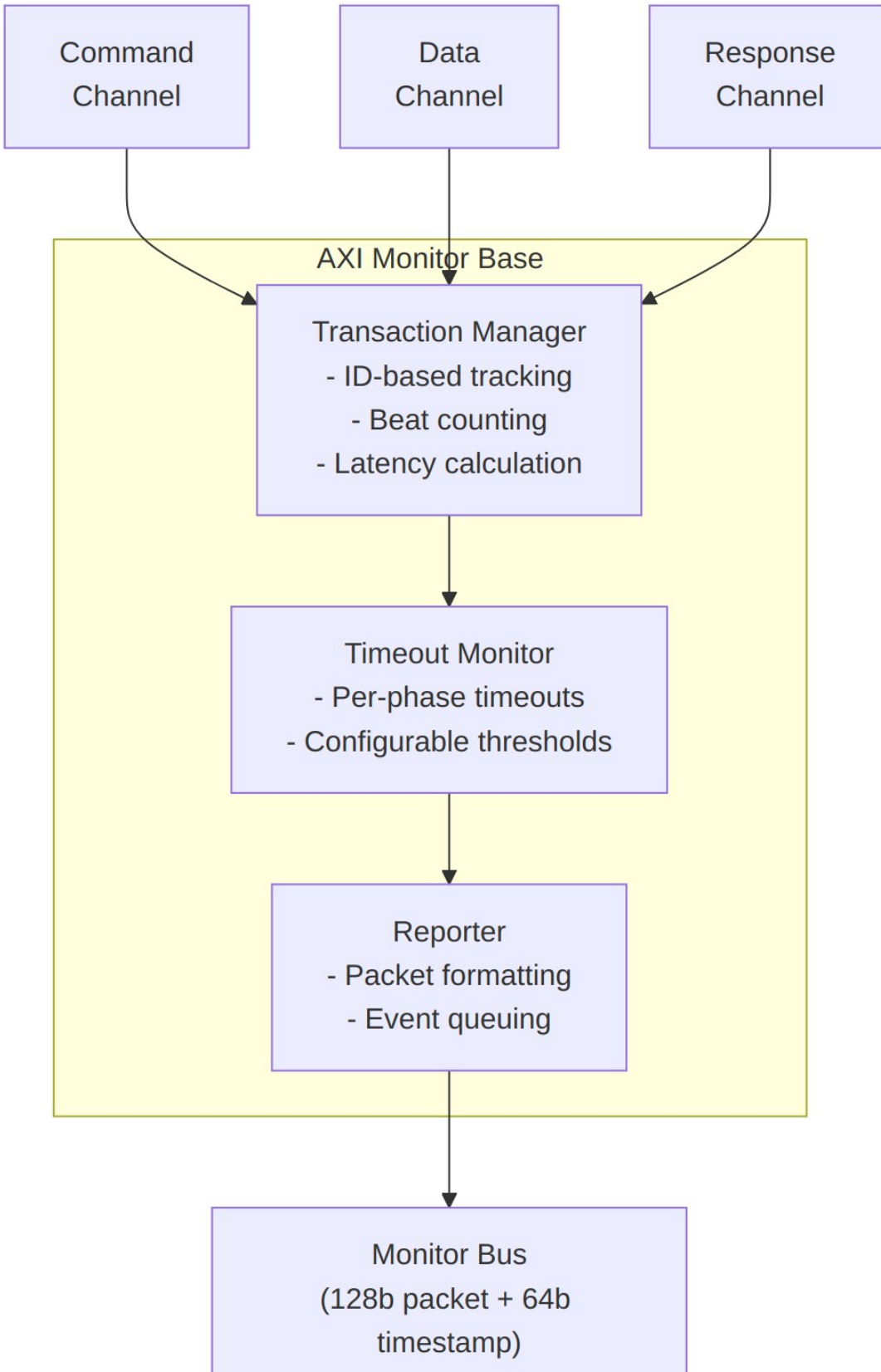
Port	Direction	Width	Description
<code>cfg_start_event_sel</code>	Input	3	Window start event source select
<code>cfg_end_event_sel</code>	Input	3	Window end event source select
<code>cfg_start_trigger</code>	Input	1	Software/engine pulse: open the measurement window
<code>cfg_end_trigger</code>	Input	1	Software/engine pulse: close the measurement window
<code>cfg_window_force_close</code>	Input	1	Software override: force the window closed immediately

36.5.9 Performance-Window Status and Counter Outputs

All counters accumulate only while `window_active=1`, reset at window-start, and hold their values through `WIN_CLOSING`/`WIN_IDLE` so the integrating block can sample a completed window's totals after `window_active` deasserts.

Port	Direction	Width	Description
<code>window_active</code>	Output	1	High while a measurement window is open
<code>window_cycles</code>	Output	32	Cycles elapsed inside the current window
<code>perf_prod_cycles</code>	Output	32	<code>data_valid</code> && <code>data_ready</code> cycles (productive beats)

Port	Direction	Width	Description
perf_bp_cycles	Output	32	data_valid && !data_ready cycles (back-pressure)
perf_starv_cycles	Output	32	!data_valid && data_ready cycles (starvation)
perf_idle_cycles	Output	32	!data_valid && !data_ready cycles (idle)
perf_beat_count	Output	32	Data beats transferred (= perf_prod_cycles, 1 beat/cycle)
perf_byte_count	Output	64	Bytes transferred = beats × (1 << latched AXSIZE)
perf_burst_count	Output	32	Address-phase handshakes inside the window (AR for reads, AW for writes)



Architecture

The module coordinates four sub-components: 1. **Transaction Manager**: Tracks transactions, manages table 2. **Timeout Monitor**: Detects stuck transactions 3. **Performance Tracker**: Optional latency/throughput metrics 4. **Reporter**: Generates standardized packets

36.6 Performance Monitoring

`axi_monitor_base` is the **canonical home** of the monitor's performance subsystem. Every AXI/AXIL wrapper (`axi4_*_mon`, `axil4_*_mon`, and the `axi_monitor_filtered` wrapper) forwards these ports straight through to this core. Because the base is generic, the data channel it measures is chosen by `IS_READ`: the **R** channel and **AR** bursts for read monitors, the **W** channel and **AW** bursts for write monitors.

The perfmon logic is instantiated when `ENABLE_PERF_LOGIC=1` (which defaults from the `ENABLE_PERF_PACKETS` master switch). It consists of a **measurement-window state machine** plus a bank of data-channel utilization and throughput counters. All counters accumulate **only while a window is open** and hold their values between windows so the host can read a completed window's totals.

36.6.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**. Event sources are chosen by the 3-bit `cfg_start_event_sel` / `cfg_end_event_sel` selectors:

Code	Start event	End event
3'b000	<code>cfg_start_trigger</code> pulse (software/CSR)	<code>cfg_end_trigger</code> pulse
3'b001	first command handshake (<code>cmd_valid</code> && <code>cmd_ready</code>)	last data (reads: <code>data_last</code> beat; writes: response handshake)
3'b010	<code>cfg_perf_enable</code> rising edge	<code>window_cycles</code> saturation
3'b011	first data handshake (<code>data_valid</code> && <code>data_ready</code>)	<code>cfg_perf_enable</code> falling edge
3'b100	<code>cfg_start_trigger</code> pulse (external-trigger convention)	<code>cfg_end_trigger</code> pulse
others	never fires (reserved)	never fires (reserved)

`cfg_window_force_close` is a software override that closes an open window immediately regardless of the end-event selector.

The state machine has three states:

- **WIN_IDLE** — waiting for a start event; `window_active=0`.
- **WIN_ACTIVE** — window open; `window_active=1`; `window_cycles` free-runs (starting at 1 so the total is inclusive of the first cycle). All counters accumulate.
- **WIN_CLOSING** — one-cycle hold before re-arming; counters are frozen and stable for sampling.

36.6.2 Utilization Buckets (data channel)

Every cycle inside the window is classified by the data channel's `valid/ready` into exactly one of four mutually-exclusive buckets. The four buckets sum to `window_cycles` by construction, so $\text{utilization} = \text{perf_prod_cycles} / \text{window_cycles}$.

Counter	Condition	Meaning
<code>perf_prod_cycles</code>	<code>valid && ready</code>	productive beat transferred
<code>perf_bp_cycles</code>	<code>valid && !ready</code>	back-pressure (data offered, sink not ready)
<code>perf_starv_cycles</code>	<code>!valid && ready</code>	starvation (sink ready, no data)
<code>perf_idle_cycles</code>	<code>!valid && !ready</code>	idle

36.6.3 Throughput Counters

Counter	Width	Meaning
<code>perf_beat_count</code>	32	data beats transferred (= <code>perf_prod_cycles</code> , 1 beat/cycle)
<code>perf_byte_count</code>	64	bytes transferred = beats × (1 << latched <code>AXSIZE</code>). <code>AXSIZE</code> is captured (<code>cmd_size</code>) at the most recent address-phase handshake and assumed constant within a burst per the AXI4 mandate. 64-bit width prevents wrap on long windows at wide

Counter	Width	Meaning
perf_burst_count	32	buses address-phase handshakes inside the window (AR for read monitors, AW for write monitors)

The integrator computes average burst length as $\text{perf_beat_count} / \text{perf_burst_count}$.

Note: the perfmon counters are exposed on the module interface for the integrating block to sample; the reporter's PerfWin packet emission is staged separately (RFC Stage B+). The four-bucket model follows DMA_UTILIZATION_MEASUREMENT.md Section 3.

36.7 Usage in Monitor System

This module is used by:

- **axi4_master_rd_mon**
- **axi4_master_wr_mon**
- **axi4_slave_rd_mon**
- **axi4_slave_wr_mon**
- **axil4_master_rd_mon**
- **axil4_master_wr_mon**
- **axil4_slave_rd_mon**
- **axil4_slave_wr_mon**

36.7.1 Integration Example

Not typically instantiated directly by users. Instead, use high-level monitors:

```
// User instantiates this:
axi4_master_rd_mon #(...) u_mon (...);
```

```
// Which internally uses:
// - axi4_master_rd (frontend)
// - axi_monitor_base (this module)
// - axi_monitor_filtered
```

36.8 Configuration Guidelines

36.8.1 Transaction Table Sizing

Design Scenario	MAX_TRANSACTIONS	Rationale
AXI4 Master (Burst)	16-32	Support multiple outstanding bursts
AXI4 Slave (Burst)	16-32	Handle out-of-order responses
AXI4-Lite Master	4-8	Single-beat, limited concurrency
AXI4-Lite Slave	4-8	Simple protocol, fewer outstanding

36.8.2 Timeout Configuration

- **cfg_addr_cnt**: Command phase timeout (AR/AW)
- **cfg_data_cnt**: Data phase timeout (R/W)
- **cfg_resp_cnt**: Response phase timeout (B)

Typical values: - Burst traffic: `cfg_*_cnt` = 4-8 (aggressive timeout) - Memory controllers: `cfg_*_cnt` = 10-15 (allow latency) - External interfaces: `cfg_*_cnt` = 15 (max tolerance)

36.9 Performance Characteristics

Metric	Value	Notes
Latency	2-3 cycles	Event detection to packet output
Throughput	1 packet/cycle	Maximum packet generation rate
Table Lookup	1 cycle	ID-based transaction lookup
Resource Usage	~500 LUTs	Depends on MAX_TRANSACTION S

36.10 Verification Considerations

36.10.1 Key Test Scenarios

1. **Transaction Table Management:**
 - Fill table to MAX_TRANSACTIONS
 - Verify table exhaustion handling
 - Test ID reuse after completion
 2. **Out-of-Order Transactions:**
 - Issue multiple transactions with different IDs
 - Complete in random order
 - Verify correct ID matching
 3. **Timeout Detection:**
 - Initiate transaction without completion
 - Verify timeout event at configured threshold
 - Check timeout packet contains correct ID
 4. **Performance Metrics:**
 - Enable ENABLE_PERF_PACKETS
 - Verify latency calculations
 - Check throughput tracking
-

36.11 Related Modules

- [axi_monitor_filtered](#)
 - [axi_monitor_trans_mgr](#)
 - [axi_monitor_reporter](#)
 - [axi_monitor_timeout](#)
-

36.12 See Also

- **Monitor Architecture:** docs/markdown/RTLamba/overview.md
 - **Monitor Configuration Guide:** [Monitor Base Configuration](#)
 - **Packet Format Specification:**
docs/markdown/RTLamba/includes/monitor_package_spec.md
-

36.13 Navigation

- [← Back to Shared Infrastructure Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

37 AXI Monitor Filtered

Module: `axi_monitor_filtered.sv` **Location:** `rtl/amba/shared/` **Category:** Core Infrastructure **Status:** ✓ Production Ready

37.1 Overview

`axi_monitor_filtered` is a **filtering wrapper around `axi_monitor_base`**. It instantiates one `axi_monitor_base` internally, taps the same AXI/AXIL command, data, and response channels, and then applies a configurable drop filter to the 128-bit monitor packets the base emits before forwarding them downstream. It does **not** take a monitor-bus input stream — it produces the monitor bus from the channels it observes.

This is a **shared infrastructure module** used internally by AXI/AXIL monitors. It is not typically instantiated directly by users but is critical for understanding the monitor architecture.

37.2 Key Features

- ✓ **Wraps `axi_monitor_base`:** all transaction tracking, timeout, threshold, and perfmon logic lives in the base; this module only filters its output stream.
- ✓ **Level 1 — packet-type drop mask** (`cfg_axi_pkt_mask`): drop entire packet types by index.
- ✓ **Level 2 — error select** (`cfg_axi_err_select`): reserved for cross-routing; in the AXI wrapper it is used only for **configuration-conflict validation** (see `cfg_conflict_error`), not applied to the stream.
- ✓ **Level 3 — per-event-code masking:** one 16-bit drop mask per packet type (`cfg_axi_error_mask`, `cfg_axi_timeout_mask`, `cfg_axi_compl_mask`, `cfg_axi_thresh_mask`, `cfg_axi_perf_mask`, `cfg_axi_addr_mask`, `cfg_axi_debug_mask`) indexed by the packet's event code.

- ✓ **Configuration conflict detection:** `cfg_conflict_error` flags overlapping `cfg_axi_pkt_mask` / `cfg_axi_err_select` bits.
 - ✓ **Bypass mode:** `ENABLE_FILTERING=0` passes every packet straight through.
 - ✓ **Optional pipeline stage:** `ADD_PIPELINE_STAGE=1` registers the filtered output for timing closure.
-

37.3 Module Purpose

The `axi_monitor_filtered` module is the core building block for:

1. **Traffic Management:** Reduces monitor bus congestion by dropping unwanted packet types and event codes at the source.
 2. **Granular Control:** Supports per-packet-type (Level 1) and per-event-code (Level 3) drop masking.
 3. **Configuration Validation:** Flags conflicting mask settings via `cfg_conflict_error`.
 4. **Protocol Isolation:** Drops any non-AXI-protocol packet (protocol field $\neq$ 4'h0), which should never occur inside an AXI monitor.
-

37.4 Parameters

Parameter	Type	Default	Description
<code>UNIT_ID</code> / <code>AGENT_ID</code>	logic	8'h01 / 16'h000A	Identity bits stamped into emitted monitor packets.
<code>MAX_TRANSACTIONS</code>	int	16	Outstanding transaction table depth (passed to <code>axi_monitor_base</code> → <code>axi_monitor_trans_mgr</code>)
<code>ADDR_WIDTH</code> / <code>ID_WIDTH</code>	int	32 / 8	Address and AXI ID widths.
<code>IS_READ</code> / <code>IS_AXI</code>	bit	1 / 1	Direction (read vs write) and protocol family (AXI4 vs AXIL).
<code>ENABLE_PERF_PACKET</code>	bit	1	Emit perf packets onto

Parameter	Type	Default	Description
S			the monitor bus.
ENABLE_DEBUG_MODULE	bit	0	Instantiate the debug reporter sub-block.
Reporter sub-block enables			Each gates one of the six reporter sub-blocks (axi_monitor_reporter_*) at elaboration time.
			Pass-through to axi_monitor_base.
ENABLE_ERROR_LOGIC	bit	1	Error reporter (orphans, resp errors).
ENABLE_TIMEOUT_LOGIC	bit	1	Timeout reporter (addr/data/resp timers).
ENABLE_COMPL_LOGIC	bit	1	Completion reporter.
ENABLE_THRESHOLD_LOGIC	bit	1	Threshold reporter (latency / active-count thresholds).
ENABLE_PERF_LOGIC	bit	ENABLE_PERF_PACKETS	Perf reporter (the Stage B counters below).
ENABLE_DEBUG_LOGIC	bit	0	Debug reporter.
ENABLE_FILTERING	bit	1	Master enable for filtering.
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing.
N_ADDR_RANGES	int	0	Number of address-range comparators in the axi_monitor_addr_check sub-block (0 = the comparator block is not synthesised at all).

37.5 Port Groups

37.5.1 Observed AXI/AXIL Channels (inputs to the wrapped base)

This module has **no monitor-bus input stream**. It taps the same command, data, and response channels as `axi_monitor_base` and passes them through unchanged. See `axi_monitor_base` for full descriptions.

Group	Ports
Command (AW/AR)	<code>cmd_addr</code> , <code>cmd_id</code> , <code>cmd_len</code> , <code>cmd_size</code> , <code>cmd_burst</code> , <code>cmd_valid</code> , <code>cmd_ready</code>
Data (W/R)	<code>data_id</code> , <code>data_last</code> , <code>data_resp</code> , <code>data_valid</code> , <code>data_ready</code>
Response (B)	<code>resp_id</code> , <code>resp_code</code> , <code>resp_valid</code> , <code>resp_ready</code>
Timer / enables / thresholds	<code>cfg_freq_sel</code> , <code>cfg_addr_cnt</code> , <code>cfg_data_cnt</code> , <code>cfg_resp_cnt</code> , <code>cfg_error_enable</code> , <code>cfg_compl_enable</code> , <code>cfg_threshold_enable</code> , <code>cfg_timeout_enable</code> , <code>cfg_perf_enable</code> , <code>cfg_debug_enable</code> , <code>cfg_debug_level</code> , <code>cfg_debug_mask</code> , <code>cfg_active_trans_threshold</code> , <code>cfg_latency_threshold</code>
Address-range checker (when <code>N_ADDR_RANGES > 0</code>)	<code>cfg_addr_check_enable</code> , <code>cfg_addr_range_enable</code> , <code>cfg_addr_range_low</code> , <code>cfg_addr_range_high</code>
Side-band time	<code>i_mon_time</code> (<code>monbus_timestamp_t</code>)

All of the above are passed through to the internal `axi_monitor_base` verbatim.

37.5.2 Monitor Bus Output (filtered)

Port	Direction	Width	Description
<code>monbus_valid</code>	Output	1	Filtered packet valid (base valid AND not dropped)
<code>monbus_ready</code>	Input	1	Downstream ready
<code>monbus_packet</code>	Output	128	Filtered

Port	Direction	Width	Description
monbus_timestamp	Output	64	monitor_packet_t (unmodified; only passed or dropped) monbus_timestamp_t paired atomically with monbus_packet

37.5.3 Reset and Synchronous Control

Port	Direction	Width	Description
aclk / aresetn	Input	1 / 1	Standard clock and active-low async reset.
clear	Input	1	Synchronous clear — passes through to axi_monitor_base → axi_monitor_trans_mgr to empty the transaction CAM and zero the active-count pipeline without aresetn. Pulse one cycle while idle.

37.5.4 Filter Configuration

All filter masks use **drop semantics**: a set bit drops the corresponding packet type or event code. All masks are 16-bit.

Port	Direction	Width	Description
cfg_axi_pkt_mask	Input	16	Level 1 packet-type drop mask. Bit pkt_type set → drop all packets of that type
cfg_axi_err_select	Input	16	Level 2 error-select. Reserved for cross-routing; in the AXI wrapper it is consumed only by the conflict check, not applied to the stream

Port	Direction	Width	Description
cfg_axi_error_mask	Input	16	Level 3 per-event-code drop mask for Error packets
cfg_axi_timeout_mask	Input	16	Level 3 drop mask for Timeout packets
cfg_axi_completion_mask	Input	16	Level 3 drop mask for Completion packets
cfg_axi_threshold_mask	Input	16	Level 3 drop mask for Threshold packets
cfg_axi_performance_mask	Input	16	Level 3 drop mask for Performance packets
cfg_axi_address_match_mask	Input	16	Level 3 drop mask for Address-Match packets
cfg_axi_debug_mask	Input	16	Level 3 drop mask for Debug packets

The Level 3 masks are indexed by the packet's event code (low nibble). For each packet the wrapper selects the mask matching its packet type, then drops the packet if `mask[event_code[3:0]]` is set.

37.5.5 Configuration Status Output

Port	Direction	Width	Description
cfg_conflict_error	Output	1	Asserted when <code>cfg_axi_pkt_mask</code> & <code>cfg_axi_err_select</code> is non-zero (overlapping type is both dropped and error-selected)

37.5.6 Performance Window Control (Stage A of perfmon RFC)

Port	Direction	Width	Description
cfg_start_event_sel	Input	3	Event selector that opens the perf window (e.g. command-handshake, first beat, software pulse).

Port	Direction	Width	Description
cfg_end_event_sel	Input	3	Event selector that closes the perf window.
cfg_start_trigger	Input	1	Software-driven open pulse (combines with cfg_start_event_sel).
cfg_end_trigger	Input	1	Software-driven close pulse.
cfg_window_force_close	Input	1	Asynchronous emergency close (drops any in-flight perf totals).
window_active	Output	1	The perf window is currently open.
window_cycles	Output	32	Number of cycles the current window has been open.

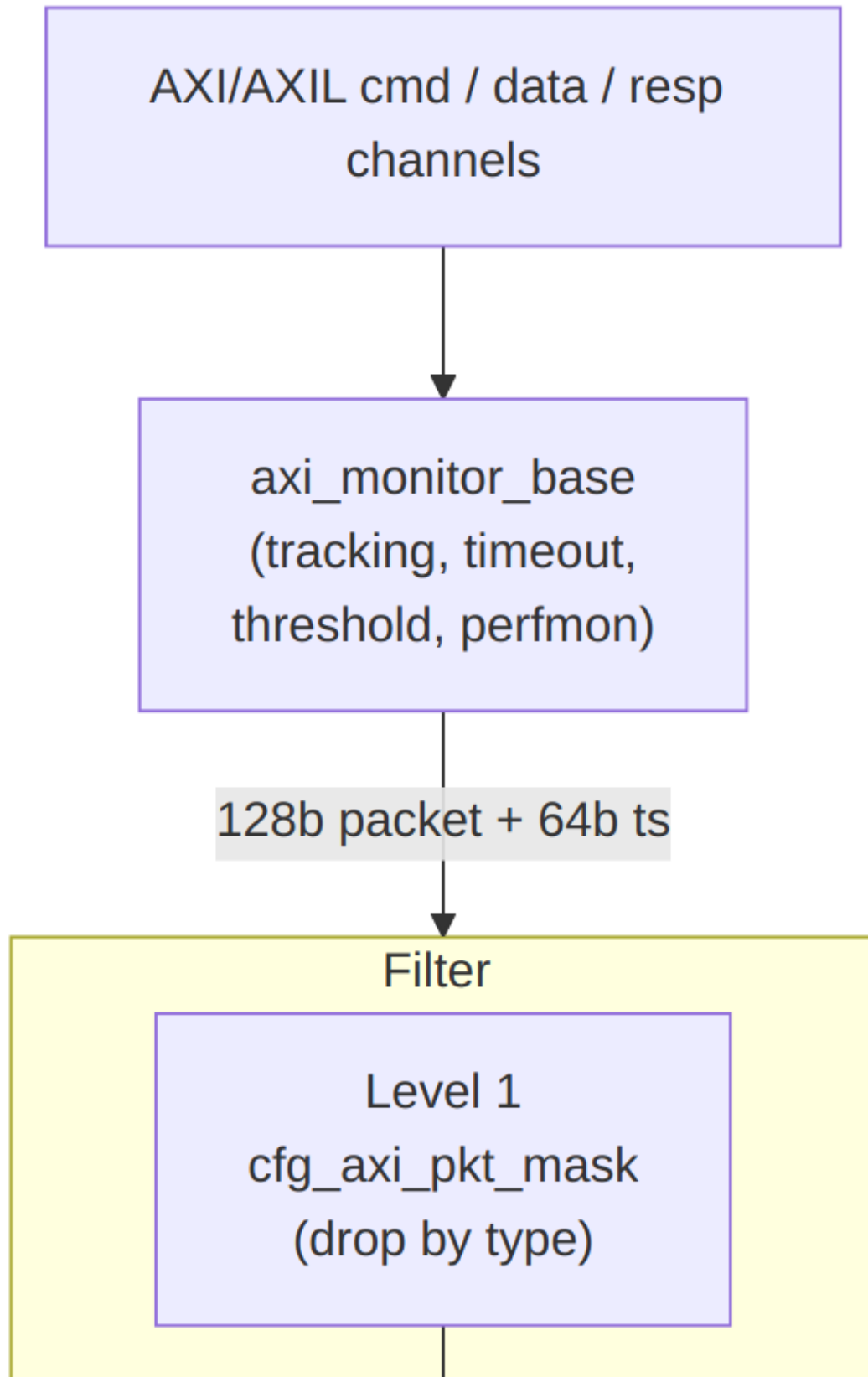
Tie cfg_start_event_sel/cfg_end_event_sel to 3'b111 and the triggers + cfg_window_force_close to 1'b0 at instances that don't use perfmon.

37.5.7 Performance Counters (Stage B of perfmon RFC)

Port	Direction	Width	Description
perf_prod_cycles	Output	32	Productive cycles (valid + ready both high).
perf_bp_cycles	Output	32	Back-pressure cycles (valid high, ready low).
perf_starv_cycles	Output	32	Starvation cycles (valid low, ready high).
perf_idle_cycles	Output	32	Idle cycles (both low).
perf_beat_count	Output	32	Beat handshakes accumulated this window.
perf_byte_count	Output	64	Bytes transferred this window (derived from cmd_size/cmd_len).
perf_burst_count	Output	32	Bursts (command

Port	Direction	Width	Description
			handshakes) this window.

These counters latch on each `window_active` open and freeze on close; software reads them after the close edge.



Architecture

The base monitor generates every packet; the filter then drops unwanted ones. A dropped packet is acknowledged back to the base (`base_monbus_ready`) so the base never stalls on packets that will be discarded. Level 2 (`cfg_axi_err_select`) is a validation-only input in this wrapper and is not shown in the drop path.

37.6 Usage in Monitor System

This module is used by:

- **axi4_master_rd_mon**
- **axi4_master_wr_mon**
- **axi4_slave_rd_mon**
- **axi4_slave_wr_mon**

37.6.1 Internal Integration

This module is instantiated automatically within higher-level monitor modules. Users configure behavior through top-level monitor parameters.

37.7 Configuration Guidelines

37.7.1 Filter Strategy

`cfg_axi_pkt_mask` uses **drop semantics** — a bit set at index `pkt_type` drops that type. Leave a bit 0 to keep the type. Indices follow `monitor_common_pkg` (`PktTypeError`, `PktTypeCompletion`, `PktTypeThreshold`, `PktTypeTimeout`, `PktTypePerf`, `PktTypeAddrMatch`, `PktTypeDebug`). Tie the per-event Level 3 masks to `16'h0000` unless you need to suppress specific event codes.

Pass everything (no filtering):

```
.cfg_axi_pkt_mask    (16'h0000), // drop nothing
// or set ENABLE_FILTERING = 0 at elaboration for full bypass
```

Suppress performance packets (typical functional run):

```
.cfg_axi_pkt_mask    (16'h0000 | (16'h1 << PktTypePerf)), // drop only
Perf
```

Performance-only capture (drop completions to cut traffic):

```
.cfg_axi_pkt_mask    ((16'h1 <= PktTypeCompletion) |
                    (16'h1 <= PktTypeTimeout)),           // keep
Error + Perf
```

Suppress one Error event code (Level 3):

```
.cfg_axi_pkt_mask    (16'h0000),                          // keep all types
.cfg_axi_error_mask (16'h1 <= ERR_EVENT_CODE),           // drop just that
error event
```

37.8 Performance Characteristics

Metric	Value	Notes
Filtering Latency	0-1 cycles	Combinatorial (0) or registered (1)
Throughput	1 packet/cycle	No backpressure introduced
Resource Usage	~100 LUTs	Minimal overhead

37.9 Verification Considerations

37.9.1 Key Test Scenarios

1. Level 1 masking:

- Generate all packet types from the base
- Set individual `cfg_axi_pkt_mask` bits and verify only the masked types are dropped

2. Level 3 masking:

- Set a per-type event mask (e.g. `cfg_axi_error_mask`) and verify only packets whose event code matches the set bit are dropped

3. Configuration conflict:

- Set the same bit in both `cfg_axi_pkt_mask` and `cfg_axi_err_select` and verify `cfg_conflict_error` asserts

4. Packet integrity:

- Verify passed packets are bit-identical to the base output (filter only drops, never mutates)
- Verify a dropped packet is acked to the base so it never stalls

- Exercise ADD_PIPELINE_STAGE=1 and confirm one-cycle latency with correct backpressure
-

37.10 Related Modules

- [axi_monitor_base](#)
-

37.11 See Also

- **Monitor Architecture:** docs/markdown/RTLAmba/overview.md
 - **Monitor Configuration Guide:** [Monitor Base Configuration](#)
 - **Packet Format Specification:**
docs/markdown/RTLAmba/includes/monitor_package_spec.md
-

37.12 Navigation

- [← Back to Shared Infrastructure Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

38 AXI Monitor Reporter — Completion Cone

Module: axi_monitor_reporter_compl.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

38.1 Overview

The AXI Monitor Reporter Completion Cone is the completion-packet detection sub-block of axi_monitor_reporter. It scans the outstanding-transaction table for slots that have reached the TRANS_COMPLETE state but have not yet been reported, priority-encodes the first such slot, and emits the fields for a PktTypeCompletion MonBus packet. It was split out of the monolithic reporter so integrators who do not need completion packets can drop it with ENABLE_COMPL_LOGIC=0 and pay zero LUT cost for the per-slot scan and priority encoder. The block is purely combinational.

38.1.1 Key Features

- Scans the shared transaction table for unreported TRANS_COMPLETE slots

- First-match priority encoder selects one slot per cycle
 - Emits PktTypeCompletion packet fields with event code EVT_TRANS_COMPLETE
 - Runtime-gated by cfg_compl_enable; compile-time removable via the parent's ENABLE_COMPL_LOGIC
 - Reports the selected slot index (sel_idx) back to the top reporter for event_reported feedback
 - Pure combinational — all state (table, event_reported) lives in the top reporter
-

38.2 Module Purpose

The transaction manager tracks every outstanding AXI transaction and marks a slot TRANS_COMPLETE when it finishes cleanly. Someone must turn those completions into MonBus packets exactly once each. This cone does the detect-and-select half of that job: it finds the completed-but-unreported slots, picks the first, and hands the packet fields plus the chosen slot index up to the top reporter, which pushes the packet into the shared MonBus FIFO and sets the slot's event_reported bit so it is not emitted again. Factoring it into its own module means the (non-trivial) MAX_TRANSACTIONS-wide scan and encoder synthesizes only when completion reporting is actually wanted.

Use Cases: - Emitting one completion packet per successfully-finished AXI transaction - Completion-tracking test/verification configurations - Transaction-throughput accounting on the MonBus

Key Benefit: Isolates the completion scan/encoder so it can be compiled out (ENABLE_COMPL_LOGIC=0) for zero area when completion packets are not needed.

38.3 Parameters

Parameter	Type	Default	Description
MAX_TRANSACTION S	int	16	Depth of the shared transaction table scanned
IDX_W	int	$\lceil \log_2(\text{MAX_TRANSACTIONS}) \rceil$	Width of the selected-slot index

38.4 Port Groups

38.4.1 Inputs (shared monitor state)

Port	Direction	Width	Description
trans_table	input	bus_transaction_t × MAX_TRANS ACTIONS	The outstanding-transaction table
event_reported	input	MAX_TRANS ACTIONS	Per-slot “already reported” mask (from the top reporter)
cfg_completion_enable	input	1	Runtime enable for completion reporting

38.4.2 Outputs (packet fields to the top reporter)

Port	Direction	Width	Description
pkt_valid	output	1	A completed-unreported slot was found this cycle
pkt_type	output	4	Packet type = PktTypeCompletion
pkt_event_code	output	8	Event code = EVT_TRANS_COMPLETE
pkt_channel	output	9	Channel id {3'b0, slot.channel[5:0]}
pkt_data	output	64	Slot address, zero-padded to 64 bits
sel_idx	output	IDX_W	Index of the selected slot (for event_reported feedback)

38.5 Functional Description

38.5.1 Completion Scan

For every slot in `trans_table`, the slot qualifies as a completion event when it is `valid`, its `event_reported` bit is clear, its `state == TRANS_COMPLETE`, and `cfg_compl_enable` is set. All qualifying slots are collected into a one-hot-per-bit `w_events` vector.

38.5.2 First-Match Selection

A second loop scans `w_events` from index 0 upward and latches the first set bit into `w_sel`, asserting `w_has_event`. The `WIDTHTRUNC` lint is locally waived where the loop index is narrowed to `IDX_W`. This gives strict low-index-first priority so exactly one completion is emitted per cycle even when several slots complete together; the rest are picked up on subsequent cycles once the winner is marked reported.

38.5.3 Emitted Fields

- `pkt_valid = w_has_event`
- `sel_idx = w_sel` (tells the top reporter which slot to mark reported)
- `pkt_type = PktTypeCompletion`
- `pkt_event_code = EVT_TRANS_COMPLETE`
- `pkt_channel = {3'b0, trans_table[w_sel].channel[5:0]}`
- `pkt_data = pad_address(trans_table[w_sel].addr)` — the 32-bit slot address zero-extended to 64 bits

38.5.4 Ownership of State

The block holds no state of its own. The transaction table and the `event_reported` feedback mask both live in the top reporter (`axi_monitor_reporter`), which consumes `sel_idx` to set the reported bit after the packet is accepted. Keeping this cone combinational lets several such cones (completion, error, timeout, ...) share the same table snapshot each cycle.

38.6 Usage Example

```
// Instantiated inside axi_monitor_reporter, gated by
ENABLE_COMPL_LOGIC.
axi_monitor_reporter_compl #(
    .MAX_TRANSACTIONS (MAX_TRANSACTIONS)
) u_compl (
    .trans_table      (trans_table),
    .event_reported   (event_reported),
    .cfg_compl_enable(cfg_compl_enable),
```

```

        .pkt_valid      (compl_valid),
        .pkt_type       (compl_type),
        .pkt_event_code (compl_event_code),
        .pkt_channel    (compl_channel),
        .pkt_data       (compl_data),
        .sel_idx        (compl_sel_idx)    // top reporter sets
event_reported[compl_sel_idx]
);

```

38.7 Design Notes

38.7.1 Compile-Out Path

The parent instantiates this cone under `ENABLE_COMPL_LOGIC`. Setting it to 0 removes the `MAX_TRANSACTIONS`-wide qualifier scan and priority encoder entirely, so a monitor build that never reports completions pays no LUT cost for them. `cfg_compl_enable` is the softer runtime mask when the logic is present.

38.7.2 Combinational by Design

All state lives upstream; this block is a pure combinational detect-and-select cone. Multiple packet cones can therefore examine the same table each cycle without duplicating storage, and the top reporter arbitrates which one wins the MonBus this cycle.

38.7.3 One Packet Per Cycle

The strict first-match encoder emits at most one completion per cycle. Simultaneous completions are serialized across cycles as each winner is marked reported.

38.8 Related Modules

38.8.1 Used By

- **axi_monitor_reporter.sv** - Top reporter that pushes the packet into the MonBus FIFO and drives `event_reported`

38.8.2 Uses

- **monitor_common_pkg** - `PktTypeCompletion`, `TRANS_COMPLETE`, `bus_transaction_t`
- **monitor_amba4_pkg** - `EVT_TRANS_COMPLETE` event code

38.8.3 See Also

- **axi_monitor_reporter_error.sv** - Error/orphan detection cone (sibling sub-block)
 - **axi_monitor_reporter_debug.sv** - State-change debug emitter (sibling sub-block)
 - **axi_monitor_base.sv** - The monitor scaffold that houses trans_mgr + reporter
-

38.9 References

38.9.1 Source Code

- RTL: rtl/amba/monitor/axi_monitor_reporter_compl.sv

38.9.2 Documentation

- Packet format: docs/markdown/RTLAmba/includes/monitor_package_spec.md
 - Monitor base: docs/markdown/RTLAmba/axi_monitor_base.md
 - Known issues: rtl/amba/KNOWN_ISSUES/axi_monitor_reporter.md
-

Last Updated: 2026-07-15

38.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

39 AXI Monitor Reporter — Debug State-Change Emitter

Module: axi_monitor_reporter_debug.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

39.1 Overview

The AXI Monitor Reporter Debug Emitter is the state-change (trace) sub-block of `axi_monitor_reporter` — the sixth cone alongside error, timeout, completion, threshold, and perf. It gives integrators (and the compression-analysis flow) a packet stream that mirrors the live transaction-table FSM: one `PktTypeDebug` packet per (slot, state-change) event. It holds a per-slot previous-state vector, compares it against the live state each cycle, priority-encodes the first slot that transitioned, and emits a packet carrying that slot's address plus the (`prev_state`, `new_state`) tuple. Unlike the combinational completion/error cones, this block is sequential (it flops the previous state).

39.1.1 Key Features

- One `PktTypeDebug` packet per per-slot state transition, mirroring the `trans_table` FSM
 - Per-slot 3-bit `prev_state` vector, `MAX_TRANSACTIONS` deep, flopped every cycle
 - First-match priority encoder selects one changed slot per cycle
 - Packs (`prev_state`, `new_state`) in the high bits of the 64-bit data field, address in the low bits
 - Direct-inject path with an `output_busy` gate (bypasses the FIFO, like threshold/perf)
 - Runtime-gated by `cfg_debug_enable`; compile-time removable via the parent's `ENABLE_DEBUG_LOGIC`
-

39.2 Module Purpose

Deep debugging and compression research both benefit from a packet stream that reflects the internal monitor FSM rather than just terminal events. This block produces exactly that: whenever any transaction slot changes state, it emits a debug packet naming the slot's address and the state edge it took. Slot-free transitions are handled cleanly — when a slot is freed, its `prev_state` resets to `TRANS_IDLE` so the next allocation produces a fresh `IDLE → ADDR_PHASE` edge rather than a spurious `IDLE → IDLE`. The compression analysis can use either the state tuple or the address as its dictionary key, whichever carries more entropy.

Use Cases: - FSM-level trace of the transaction table for deep debugging - Feeding the MonBus compression-analysis flow with a rich event stream - Correlating state transitions with observed protocol behavior during bring-up

Key Benefit: Surfaces the live per-slot FSM as MonBus packets, giving debuggers and the compressor a per-transition trace that terminal-event cones cannot provide.

39.3 Parameters

Parameter	Type	Default	Description
MAX_TRANSACTION_S	int	16	Depth of the transaction table monitored
IDX_W	int	$\lceil \log_2(\text{MAX_TRANSACTIONS}) \rceil$	Width of the internal selected-slot index

39.4 Port Groups

39.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

39.4.2 Inputs (shared monitor state)

Port	Direction	Width	Description
trans_table	input	$\text{bus_transaction_t} \times \text{MAX_TRANSACTIONS}$	The outstanding-transaction table
cfg_debug_enable	input	1	Runtime enable / mask for debug packet emission

39.4.3 Direct-Inject Handshake

Port	Direction	Width	Description
output_busy	input	1	High when the shared MonBus output is occupied; gates pkt_valid
pkt_taken	input	1	Pulsed when this block's packet is accepted (currently unused — see

Port	Direction	Width	Description
Design Notes)			

39.4.4 Outputs (packet fields)

Port	Direction	Width	Description
pkt_valid	output	1	A state change was found and the output bus is free
pkt_type	output	4	Packet type = PktTypeDebug
pkt_event_code	output	8	Event code = AXI_DEBUG_STATE_CHANGE
pkt_chann_el	output	9	Channel id {3'b0, slot.channel[5:0]}
pkt_data	output	64	{prev_state[2:0], new_state[2:0], 26'h0, addr[31:0]}

39.5 Functional Description

39.5.1 Change Detection

When `cfg_debug_enable` is set, the block scans every slot: a slot is flagged changed when it is valid and its live state differs from the captured `r_prev_state[idx]`. All changed slots collect into `w_changed`, and a first-match priority encoder selects the lowest-index changed slot into `w_sel` (asserting `w_has_event`). When `cfg_debug_enable` is low, no slot is flagged, so no packets are produced.

39.5.2 Previous-State Flop

Each cycle `r_prev_state[idx]` snapshots the slot's live state — **except** when the slot is not valid, in which case it resets to `TRANS_IDLE`. This is the mechanism that makes freed slots behave: an emptied slot returns to `IDLE`, so its next allocation registers as a real `IDLE → ADDR_PHASE` transition rather than an aliased `IDLE → IDLE` non-event.

39.5.3 Packet Assembly

When a change is found and the output bus is free (`w_has_event && !output_busy`):

- `pkt_valid = 1`
- `pkt_type = PktTypeDebug`
- `pkt_event_code = AXI_DEBUG_STATE_CHANGE`

- `pkt_channel = {3'b0, trans_table[w_sel].channel[5:0]}`
- `pkt_data = {3'(r_prev_state[w_sel]), 3'(trans_table[w_sel].state), 26'h0, trans_table[w_sel].addr}`

The (prev, new) state tuple sits in the top six bits and the 32-bit address in the low bits, leaving a 26-bit zero gap between them. The compression analysis can key on either field.

39.5.4 Direct-Inject and output_busy

Like the threshold and perf cones, debug packets bypass the shared FIFO and inject directly onto the MonBus, so the block must observe output_busy to avoid overwriting an in-flight packet — pkt_valid is gated by !output_busy.

39.6 Usage Example

```
// Instantiated inside axi_monitor_reporter, gated by
ENABLE_DEBUG_LOGIC.
axi_monitor_reporter_debug #(
    .MAX_TRANSACTIONS (MAX_TRANSACTIONS)
) u_debug (
    .aclk             (aclk),
    .aresetn          (aresetn),

    .trans_table      (trans_table),
    .cfg_debug_enable (cfg_debug_enable),

    // Direct-inject arbitration with the top reporter
    .output_busy      (monbus_output_busy),
    .pkt_taken        (debug_pkt_taken),    // currently unused inside
the block

    .pkt_valid        (dbg_valid),
    .pkt_type         (dbg_type),
    .pkt_event_code   (dbg_event_code),
    .pkt_channel      (dbg_channel),
    .pkt_data         (dbg_data)
);
```

Note: enabling debug packets alongside completions/perf can congest the MonBus. See the AXI Monitor Configuration Guide before turning `cfg_debug_enable` on in a real run.

39.7 Design Notes

39.7.1 pkt_taken Is Intentionally Unused

The `r_prev_state` flop advances every cycle regardless of whether the emitted packet was accepted, so a missed transition simply gets aliased into the next change. `pkt_taken` is therefore not consumed today — it is kept on the port list (with an explicit `UNUSED` lint waiver) for symmetry with the threshold/perf cones and for future hooks such as backpressure on debug bursts.

39.7.2 Sequential, Not Combinational

Unlike the completion and error cones (pure combinational), this block flops `r_prev_state`, which is inherently required to detect edges. It therefore takes `aclk/aresetn`.

39.7.3 Compile-Out Path

Instantiated under `ENABLE_DEBUG_LOGIC` in the parent; setting it to 0 drops the block entirely. `cfg_debug_enable` is the runtime mask when the logic is present — emissions are gated but the logic still synthesizes.

39.7.4 State Tuple Encoding

The 3-bit state codes follow `transaction_state_t` (`TRANS_IDLE=0`, `ADDR_PHASE=1`, `DATA_PHASE=2`, `COMPLETE=3`, `ERROR=4`, `ORPHANED=5`), packed as `{prev, new}` in `pkt_data[63:58]`.

39.8 Related Modules

39.8.1 Used By

- **axi_monitor_reporter.sv** - Top reporter that arbitrates the direct-inject MonBus path

39.8.2 Uses

- **monitor_common_pkg** - `PktTypeDebug`, `transaction_state_t`, `TRANS_IDLE`, `bus_transaction_t`
- **monitor_amba4_pkg** - `AXI_DEBUG_STATE_CHANGE` event code
- **reset_defs.svh** - `ALWAYS_FF_RST` / `RST_ASSERTED` reset macros

39.8.3 See Also

- **axi_monitor_reporter_compl.sv** - Completion detection cone (sibling sub-block)

- **axi_monitor_reporter_error.sv** - Error/orphan detection cone (sibling sub-block)
 - **axi_monitor_base.sv** - The monitor scaffold that houses trans_mgr + reporter
-

39.9 References

39.9.1 Source Code

- RTL: rtl/amba/monitor/axi_monitor_reporter_debug.sv

39.9.2 Documentation

- Packet format: docs/markdown/RTLAmba/includes/monitor_package_spec.md
 - Monitor base: docs/markdown/RTLAmba/axi_monitor_base.md
 - Configuration: docs/AXI_Monitor_Configuration_Guide.md
-

Last Updated: 2026-07-15

39.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

40 AXI Monitor Reporter — Error Cone

Module: axi_monitor_reporter_error.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

40.1 Overview

The AXI Monitor Reporter Error Cone is the error-packet detection sub-block of axi_monitor_reporter. It scans the outstanding-transaction table for unreported transactions in the TRANS_ERROR (genuine error, not a timeout) or TRANS_ORPHANED state, priority-encodes the first match, and emits the fields for a PktTypeError MonBus packet. It was split out of the

monolithic reporter so integrators can drop it with `ENABLE_ERROR_LOGIC=0` and pay zero LUT cost for the per-slot scan and priority encoder. The block is purely combinational.

40.1.1 Key Features

- Scans for unreported `TRANS_ERROR` (non-timeout) and `TRANS_ORPHANED` slots
 - Uses `timeout_detected` to keep genuine errors distinct from timeouts (which the timeout cone claims)
 - First-match priority encoder selects one slot per cycle
 - Emits `PktTypeError` with the slot's own `event_code.raw_code`
 - Runtime-gated by `cfg_error_enable`; compile-time removable via the parent's `ENABLE_ERROR_LOGIC`
 - Pure combinational — `table`, `event_reported`, and threshold state stay in the top reporter
-

40.2 Module Purpose

The transaction manager marks a slot `TRANS_ERROR` when a transaction returns `SLVERR/DECERR` or violates protocol, and `TRANS_ORPHANED` when data or a response arrives with no matching command. These must become MonBus error packets exactly once each. This cone performs the detect-and-select half: it finds the error/orphan slots that have not yet been reported, excludes any slot already flagged as a timeout so the timeout cone can own those without aliasing, picks the first match, and hands the packet fields plus the chosen slot index to the top reporter for FIFO push and `event_reported` feedback. Splitting it out lets the wide scan and encoder be compiled away when error reporting is not required.

Use Cases: - Emitting `SLVERR / DECERR / protocol-violation` error packets - Reporting orphaned data or response beats (no matching command) - Functional-verification configurations that must catch bus errors

Key Benefit: Isolates the error/orphan scan/encoder for `ENABLE_ERROR_LOGIC=0` compile-out, and cleanly separates genuine errors from timeouts via the `timeout_detected` mask.

40.3 Parameters

Parameter	Type	Default	Description
<code>MAX_TRANSACTION_S</code>	int	16	Depth of the shared transaction table scanned
<code>IDX_W</code>	int	<code>\$clog2(MAX_TRA</code>	Width of the selected-slot

Parameter	Type	Default	Description
		NSACTIONS)	index

40.4 Port Groups

40.4.1 Inputs (shared monitor state)

Port	Direction	Width	Description
trans_table	input	bus_transaction_t × MAX_TRANSACTIONS	The outstanding-transaction table
event_reported	input	MAX_TRANSACTIONS	Per-slot “already reported” mask (from the top reporter)
timeout_detected	input	MAX_TRANSACTIONS	Per-slot timeout mask; excludes timeout slots from the error scan
cfg_error_enable	input	1	Runtime enable for error reporting

40.4.2 Outputs (packet fields to the top reporter)

Port	Direction	Width	Description
pkt_valid	output	1	An unreported error/orphan slot was found this cycle
pkt_type	output	4	Packet type = PktTypeError
pkt_event_code	output	8	Event code = selected slot's event_code.raw_code
pkt_channel	output	9	Channel id {3'b0, slot.channel[5:0]}
pkt_data	output	64	Slot address, zero-padded to 64 bits
sel_idx	output	IDX_W	Index of the selected slot (for event_reported)

Port	Direction	Width	Description
			feedback)

40.5 Functional Description

40.5.1 Error / Orphan Scan

For every slot, the slot qualifies as an error event when it is valid, its event_reported bit is clear, cfg_error_enable is set, and **either**:

- state == TRANS_ERROR **and** its timeout_detected bit is clear (a genuine error, not a timeout), **or**
- state == TRANS_ORPHANED (data/response with no matching command).

Qualifying slots collect into w_events. Consulting timeout_detected is what keeps this cone from aliasing timeouts: a transaction that errored because it timed out is left for the timeout sub-block to report, so the same slot is never emitted twice under two packet types.

40.5.2 First-Match Selection

A second loop takes the first set bit of w_events into w_sel and asserts w_has_event (with the local WIDTHTRUNC lint waiver on the index narrowing). Strict low-index-first priority means exactly one error is emitted per cycle; additional errors are serialized on later cycles as each winner is marked reported upstream.

40.5.3 Emitted Fields

- pkt_valid = w_has_event
- sel_idx = w_sel
- pkt_type = PktTypeError
- pkt_event_code = trans_table[w_sel].event_code.raw_code — the specific AXI error code captured by the trans manager (e.g. AXI_ERR_RESP_SLVERR, AXI_ERR_RESP_DECERR, AXI_ERR_DATA_ORPHAN, AXI_ERR_RESP_ORPHAN)
- pkt_channel = {3'b0, trans_table[w_sel].channel[5:0]}
- pkt_data = pad_address(trans_table[w_sel].addr)

Unlike the completion and debug cones, the error cone forwards the transaction's own captured raw error code rather than a fixed constant, so the packet carries the precise error kind.

40.5.4 Ownership of State

The block is stateless. The transaction table, the event_reported feedback, the timeout_detected mask, and the threshold flags all live in the top reporter, which consumes sel_idx to set the reported bit after acceptance. Keeping the cone combinational lets it share the table snapshot with the sibling cones each cycle.

40.6 Usage Example

```
// Instantiated inside axi_monitor_reporter, gated by
ENABLE_ERROR_LOGIC.
axi_monitor_reporter_error #(
    .MAX_TRANSACTIONS (MAX_TRANSACTIONS)
) u_error (
    .trans_table      (trans_table),
    .event_reported   (event_reported),
    .timeout_detected (timeout_detected), // keeps timeouts out of
the error scan
    .cfg_error_enable (cfg_error_enable),

    .pkt_valid      (err_valid),
    .pkt_type       (err_type),
    .pkt_event_code  (err_event_code),
    .pkt_channel     (err_channel),
    .pkt_data        (err_data),
    .sel_idx         (err_sel_idx)       // top reporter sets
event_reported[err_sel_idx]
);
```

40.7 Design Notes

40.7.1 Timeout De-Aliasing

The timeout_detected input is the mechanism that prevents a timed-out transaction (which the trans manager may also mark TRANS_ERROR) from being reported by both this cone and the timeout cone. Only errors with a clear timeout bit are claimed here.

40.7.2 Compile-Out Path

Instantiated under ENABLE_ERROR_LOGIC in the parent; setting it to 0 removes the wide scan and encoder for zero LUT cost. cfg_error_enable is the runtime mask when the logic is present.

40.7.3 Raw Event Code Forwarding

Because `pkt_event_code` comes from the slot's `event_code.raw_code`, this cone reports the exact error type (SLVERR, DECERR, orphan variants, protocol errors, ...) rather than a single generic error code.

40.8 Related Modules

40.8.1 Used By

- **axi_monitor_reporter.sv** - Top reporter that pushes the packet into the MonBus FIFO and drives `event_reported`

40.8.2 Uses

- **monitor_common_pkg** - `PktTypeError`, `TRANS_ERROR`, `TRANS_ORPHANED`, `bus_transaction_t`
- **monitor_amba4_pkg** - AXI error event codes (via the slot's `event_code`)

40.8.3 See Also

- **axi_monitor_reporter_compl.sv** - Completion detection cone (sibling sub-block)
 - **axi_monitor_reporter_debug.sv** - State-change debug emitter (sibling sub-block)
 - **axi_monitor_base.sv** - The monitor scaffold that houses `trans_mgr` + reporter
-

40.9 References

40.9.1 Source Code

- RTL: `rtl/amba/monitor/axi_monitor_reporter_error.sv`

40.9.2 Documentation

- Packet format: `docs/markdown/RTLAmba/includes/monitor_package_spec.md`
 - Monitor base: `docs/markdown/RTLAmba/axi_monitor_base.md`
 - Known issues: `rtl/amba/KNOWN_ISSUES/axi_monitor_reporter.md`
-

40.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

41 AXI Monitor Reporter (Dispatcher + 6 Sub-blocks)

Modules: - `axi_monitor_reporter.sv` — thin top-level dispatcher -
`axi_monitor_reporter_error.sv` — error-packet detection (combinational) -
`axi_monitor_reporter_timeout.sv` — timeout-packet detection (combinational) -
`axi_monitor_reporter_compl.sv` — completion-packet detection (combinational) -
`axi_monitor_reporter_threshold.sv` — threshold-packet detection (16 latency flops + edge flags) -
`axi_monitor_reporter_perf.sv` — performance-packet generation (counters + 5-state FSM) -
`axi_monitor_reporter_debug.sv` — debug-packet generation

Location: `rtl/amba/shared/` **Category:** Core Infrastructure **Status:** Production Ready
(refactored to sub-blocks 2026-06-06)

41.1 Overview

The `axi_monitor_reporter` family generates Monitor-bus packets. The top-level `axi_monitor_reporter.sv` is a **thin dispatcher** that multiplexes one packet stream out of (up to) six packet-type-specific sub-blocks into the shared monbus FIFO and 128-bit output register. It also reports each emitted event back to `axi_monitor_trans_mgr` so the transaction table can release entries.

Per-packet-type detection lives in the six sub-blocks. The dispatcher gates each sub-block via an `ENABLE_*_LOGIC` parameter, so integrators can drop any combination at elaboration time and pay zero LUT/FF cost for the unused detection cones.

The bridge case (`ENABLE_ERROR_LOGIC=1`, all others 0) drops roughly 70% of the reporter's LUT/FF.

41.2 Key Features

- 128-bit standardized `monitor_packet_t` formatting (via package helpers)

- Packet type multiplexing across six sub-blocks (error / timeout / compl / threshold / perf / debug) with per-type elaboration gates
- Protocol identification (AXI4, AXI5, APB, AXIS, CORE)
- Event code and data field population from the active sub-block
- Unit ID and Agent ID insertion (caller-configured constants)
- Packet valid/ready handshaking
- Internal monbus FIFO for packet queuing
- Event-reported feedback to axi_monitor_trans_mgr (closes the transaction-table loop documented in FIX-001)

41.3 Sub-blocks

Sub-block	Gate parameter	Generates pkt_type	Logic shape
axi_monitor_reporter_error	ENABLE_ERROR_LOG IC	PktTypeError	combinational
axi_monitor_reporter_timeout	ENABLE_TIMEOUT_LOGIC	PktTypeTimeout	combinational
axi_monitor_reporter_compl	ENABLE_COMPL_LOG IC	PktTypeCompletion	combinational
axi_monitor_reporter_threshold	ENABLE_THRESHOLD_LOGIC	PktTypeThreshold	16 latency flops + edge detect
axi_monitor_reporter_perf	ENABLE_PERF_LOGIC (alias ENABLE_PERF_PACKETS)	PktTypePerf	window counters + 5-state FSM
axi_monitor_reporter_debug	ENABLE_DEBUG_LOG IC (default 0)	PktTypeDebug	event-encoded debug points

Each sub-block presents the same “raise a request with packet payload” contract to the dispatcher, which arbitrates and pipes the winner into the FIFO. None of them are intended to be instantiated directly by integrators — they are private to the reporter family.

41.4 Module Purpose

The reporter dispatcher provides:

1. **Per-type detection gating** — only the enabled sub-blocks consume LUT/FF, so a single-purpose deployment (e.g. error-only on the bridge) is lean.
 2. **Packet formatting** — pulls type / protocol / event code / event data / channel id from the active sub-block and packs into the 128-bit `monitor_packet_t`.
 3. **Routing IDs** — inserts the static `UNIT_ID` / `AGENT_ID` so the downstream arbiter and the host can route packets back to their source.
 4. **Queuing** — buffers up to `INTR_FIFO_DEPTH` packets when the downstream monbus is back-pressured.
 5. **Event acknowledge** — drives `event_reported_*` back to `axi_monitor_trans_mgr` so the transaction table can release its entry once the packet is in the FIFO.
-

41.5 Parameters

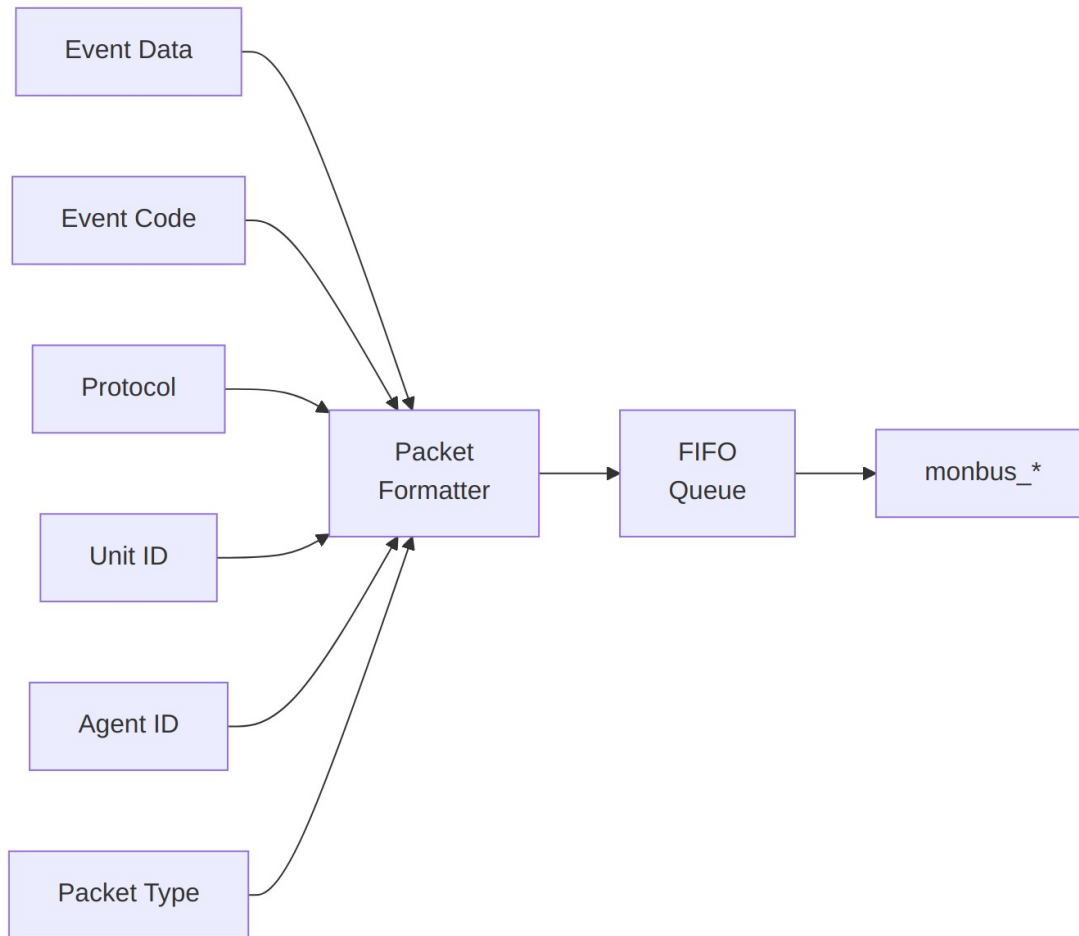
Parameter	Type	Default	Description
<code>MAX_TRANSACTIONS</code>	int	16	Transaction-table size shared with <code>axi_monitor_trans_mgr</code>
<code>ADDR_WIDTH</code>	int	32	Address width carried in <code>event_data</code>
<code>UNIT_ID</code>	logic [7:0]	8'h09	8-bit unit identifier (static)
<code>AGENT_ID</code>	logic [15:0]	16'h0063	16-bit agent identifier (static)
<code>IS_READ</code>	bit	1	1 = read-channel monitor, 0 = write-channel
<code>INTR_FIFO_DEPTH</code>	int	8	Reporter packet FIFO depth
<code>ENABLE_ERROR_LOGIC</code>	bit	1	Instantiate the error detection sub-block
<code>ENABLE_TIMEOUT_LOGIC</code>	bit	1	Instantiate the timeout detection sub-block
<code>ENABLE_COMPL_LOGIC</code>	bit	1	Instantiate the

Parameter	Type	Default	Description
			completion detection sub-block
ENABLE_THRESHOLD_LOGIC	bit	1	Instantiate the threshold detection sub-block
ENABLE_PERF_LOGIC	bit	(alias)	Instantiate the perf sub-block; defaults to ENABLE_PERF_PACKETS for legacy compat
ENABLE_DEBUG_LOGIC	bit	0	Instantiate the debug sub-block

41.6 Port Groups

See RTL source: `rtl/amba/monitor/axi_monitor_reporter.sv` for complete port listing.

Key interface groups: - Clock and reset - Input signals from monitored interface - Configuration signals - Output signals to downstream logic



Architecture

Packet Format (128-bit `monitor_packet_t`): | Bits | Width | Field | |——|——|——| |
 [127:124] | 4 | Packet Type (error / completion / timeout / perf / etc.) | | [123:109] | 15 | Reserved
 (forward-compat slack) | | [108:105] | 4 | Protocol (AXI / AXIS / APB / ARB / CORE) | | [104:97] | 8
 | Event Code (protocol-specific) | | [96:88] | 9 | Channel ID (AXI ID or channel index) | | [87:72] |
 16 | Agent ID | | [71:64] | 8 | Unit ID | | [63:0] | 64 | Event Data (full address, latency, counter
 value, etc.) |

The reporter drives `monbus_packet` (128b) and `monbus_timestamp` (64b) together so the side-band timestamp travels paired with each packet through the arbiter and into the [monbus_group family](#).

41.7 Usage in Monitor System

This module is used by:

- **axi_monitor_base**

41.7.1 Internal Integration

This module is instantiated automatically within higher-level monitor modules. Users configure behavior through top-level monitor parameters.

41.8 Configuration Guidelines

See individual monitor documentation for configuration examples.

Configuration is typically handled at the top-level monitor instantiation.

41.9 Performance Characteristics

Metric	Value	Notes
Latency	1-2 cycles	Typical processing delay
Throughput	1 operation/cycle	Maximum rate
Resource Usage	Varies	Depends on configuration

41.10 Verification Considerations

41.10.1 Test Coverage

- Functional correctness of core logic
- Boundary conditions (min/max values)
- Error handling and recovery
- Interface protocol compliance

See: `val/amba/test_axi_monitor_reporter.py` for verification tests

41.11 Related Modules

- **axi_monitor_base**

- [arbiter_monbus_common](#)
-

41.12 See Also

- **Monitor Architecture:** docs/markdown/RTLAmba/overview.md
 - **Monitor Configuration Guide:** [Monitor Base Configuration](#)
 - **Packet Format Specification:**
docs/markdown/RTLAmba/includes/monitor_package_spec.md
-

41.13 Navigation

- [← Back to Shared Infrastructure Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

42 AXI Monitor Reporter — Performance Packets

Module: axi_monitor_reporter_perf.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

42.1 Overview

The axi_monitor_reporter_perf module is the performance-packet emitter for the AXI/AXIL monitor family. It is one of the per-packet-type sub-blocks that the top-level axi_monitor_reporter dispatches to. It owns two lifetime counters — completed transactions and error transactions — and a small cycle FSM that periodically publishes count-rollup packets of type PktTypePerf onto the monitor bus (MonBus).

This block was split out of the original monolithic reporter so integrators who do not need performance counters can drop it (compile it out with ENABLE_PERF_LOGIC=0 at the reporter level) and reclaim the counter area.

42.1.1 Key Features

- Lifetime completion and error transaction counters (16-bit each)
- Counters driven from mark-reported masks supplied by the top reporter
- 5-state cycle FSM that paces packet publication
- Emits one packet at a time via a simple pkt_valid / pkt_taken handshake

- Backpressure aware: only advances when the output bus is free (output_busy low)
- Emits AXI_PERF_COMPLETED_COUNT and AXI_PERF_ERROR_COUNT event codes
- Counters exposed as ports for status / debug read-back

42.2 Module Purpose

Performance analysis of an AXI interface needs a periodic summary of how many transactions have completed and how many ended in error over the life of the monitor. This block accumulates those two counts and, when the output path is idle, walks a fixed FSM that emits a completion-count packet followed by an error-count packet.

Use Cases: - Long-run throughput and error-rate characterization of an AXI/AXIL master or slave
 - Coarse “health” telemetry streamed to a host over the MonBus - Regression sanity: confirming the number of completions matches the stimulus - Cross-checking the window-bucket perfmon path (Stage A/B) against a lifetime rollup

Key Benefit: Real area savings when disabled (the counters and FSM are removed), while providing a lightweight lifetime performance rollup that runs in parallel with the window-bucket perfmon counters in axi_monitor_base.

42.3 Parameters

Parameter	Type	Default	Description
MAX_TRANSACTIONS	int	16	Number of transaction slots; sets the width of the error_marked_mask / compl_marked_mask inputs

42.4 Port Groups

42.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Monitor clock
aresetn	input	1	Active-low asynchronous

Port	Direction	Width	Description
			reset

42.4.2 Control / Status Inputs

Port	Direction	Width	Description
cfg_perf_enable	input	1	Enable performance packet generation (also gates the FSM)
output_busy	input	1	Output path busy (FIFO has data or monbus_valid asserted); FSM stalls while high
pkt_taken	input	1	Strobed by the top reporter when this block's packet is accepted (currently observational — see Design Notes)
error_marked_mask	input	MAX_TRANS ACTIONS	Per-slot bit set the cycle an error event is marked-reported into the FIFO
compl_marked_mask	input	MAX_TRANS ACTIONS	Per-slot bit set the cycle a completion event is marked-reported into the FIFO

42.4.3 Packet Output

Port	Direction	Width	Description
pkt_valid	output	1	A performance packet is available this cycle
pkt_type	output	4	Packet type — constant PktTypePerf (4'h4)
pkt_event_code	output	8	Event code: AXI_PERF_COMPLETED_COUNT (8'h7) or AXI_PERF_ERROR_COUNT (8'h8)

Port	Direction	Width	Description
pkt_channel	output	9	Channel field (unused here — driven to 0)
pkt_data	output	64	Zero-extended count value being reported

42.4.4 Lifetime Counters (Status / Debug)

Port	Direction	Width	Description
perf_completed_count	output	16	Running total of completed transactions
perf_error_count	output	16	Running total of error transactions

42.5 Functional Description

42.5.1 Lifetime Counters

Two 16-bit registers, `r_completed_count` and `r_error_count`, accumulate over the life of the monitor. On every clock, the block scans the `error_marked_mask` and `compl_marked_mask` inputs across all `MAX_TRANSACTIONS` slots and increments the relevant counter for each asserted bit. Because the counters advance directly from the mark masks (not from `pkt_taken`), they track every event the top reporter accepted regardless of whether a rollup packet is emitted. Both counters are exposed on the port list for status read-back.

42.5.2 Cycle FSM

A 5-state counter (`r_state`) paces packet publication and matches the original monolithic reporter's behavior one-for-one. The FSM only advances while `cfg_perf_enable` is asserted and `output_busy` is low:

State	Name	Action
3'h0	ADDR_LATENCY	No packet (placeholder state)
3'h1	DATA_LATENCY	No packet (placeholder state)
3'h2	TOTAL_LATENCY	No packet (placeholder state)
3'h3	COMPLETED_COUNT	Assert <code>w_gen_completed</code> if <code>r_completed_count > 0</code>
3'h4	ERROR_COUNT	Assert <code>w_gen_errors</code> if <code>r_error_count > 0</code> , then

State	Name	Action
		wrap to 3'h0

The three latency states are placeholders retained for behavioral compatibility; they emit no packet. Only the completed-count and error-count states can produce output.

42.5.3 Output Multiplexer

The output mux prioritizes the completion-count packet over the error-count packet. When `w_gen_completed` is set, `pkt_valid` asserts with event code `AXI_PERF_COMPLETED_COUNT` and `pkt_data` carrying the zero-extended completed count. Otherwise, when `w_gen_errors` is set, `pkt_valid` asserts with `AXI_PERF_ERROR_COUNT` and the error count. `pkt_type` is always `PktTypePerf` and `pkt_channel` is always 0.

42.5.4 Handshake

The block presents one packet at a time. The top reporter samples `pkt_valid`, forwards the packet fields onto the MonBus, and strobes `pkt_taken` when the packet is accepted. Publication rate is naturally throttled by the FSM (which only steps when the output is not busy), so the emitter cannot flood the bus.

42.6 Usage Example

This block is not instantiated directly by users; it is instantiated inside `axi_monitor_reporter`. The pattern is:

```
axi_monitor_reporter_perf #(
    .MAX_TRANSACTIONS (MAX_TRANSACTIONS)
) u_reporter_perf (
    .aclk                (aclk),
    .aresetn             (aresetn),

    .cfg_perf_enable     (cfg_perf_enable),
    .output_busy         (w_output_busy),      // FIFO data or
monbus_valid
    .pkt_taken           (w_perf_pkt_taken),    // top reporter
accepted our packet
    .error_marked_mask   (w_error_marked_mask),
    .compl_marked_mask   (w_compl_marked_mask),

    .pkt_valid           (w_perf_pkt_valid),
    .pkt_type            (w_perf_pkt_type),
    .pkt_event_code      (w_perf_pkt_event_code),
    .pkt_channel         (w_perf_pkt_channel),
    .pkt_data            (w_perf_pkt_data),
```

```
    .perf_completed_count (perf_completed_count),  
    .perf_error_count     (perf_error_count)  
);
```

42.7 Design Notes

42.7.1 Never Enable Completion + Performance Packets Together

This is the single most important integration caveat for the monitor family. Enabling both completion packets (`cfg_compl_enable`) and performance packets (`cfg_perf_enable`) simultaneously overwhelms the monitor bus and causes packet congestion. Use separate monitor configurations: a functional-debug config (error + completion + timeout) and a performance config (error + perf, with completions disabled). See `docs/AXI_Monitor_Configuration_Guide.md`.

42.7.2 `pkt_taken` Is Currently Observational

`pkt_taken` is on the port list but does not gate the counters today — they update from the mark masks unconditionally. The port is retained for future hooks such as back-pressure on packet bursts. The RTL ties it to an unused net to keep lint clean.

42.7.3 Relationship to the Window-Bucket Perfmon

This block emits the legacy `PktTypePerf` count-rollup packets that summarize completion/error counts over the monitor's lifetime. It runs in parallel with the Stage A/B window-bucket perfmon counters in `axi_monitor_base`, which emit `PktTypePerfWin` / `PktTypePerfHist` window-aggregate packets. The two mechanisms are complementary, not redundant.

42.7.4 Counter Wrap

Both counters are 16-bit and wrap on overflow. For very long runs the host should account for wrap or read the counters periodically via the status ports.

42.8 Related Modules

42.8.1 Used By

- **`axi_monitor_reporter.sv`** — instantiates this block as its performance-packet sub-emitter
- **`axi_monitor_base.sv`** — top-level monitor scaffold that wires the reporter into the MonBus

42.8.2 Uses

- **monitor_common_pkg** — PktTypePerf and shared packet definitions
- **monitor_amba4_pkg** — AXI_PERF_COMPLETED_COUNT / AXI_PERF_ERROR_COUNT event codes
- **reset_defs.svh** — reset macros (ALWAYS_FF_RST, RST_ASSERTED)

42.8.3 See Also

- **axi_monitor_reporter_threshold.sv** — threshold-crossing packet emitter (sibling)
 - **axi_monitor_reporter_timeout.sv** — timeout packet emitter (sibling)
-

42.9 References

42.9.1 Source Code

- RTL: rtl/amba/monitor/axi_monitor_reporter_perf.sv
- Parent: rtl/amba/monitor/axi_monitor_reporter.sv
- Packages: rtl/amba/includes/monitor_common_pkg.sv, rtl/amba/includes/monitor_amba4_pkg.sv

42.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - Monitor Base: docs/markdown/RTLAmba/axi_monitor_base.md
 - Configuration: docs/AXI_Monitor_Configuration_Guide.md
 - Packet Format: docs/markdown/RTLAmba/includes/monitor_package_spec.md
-

Last Updated: 2026-07-15

42.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

43 AXI Monitor Reporter — Threshold Packets

Module: `axi_monitor_reporter_threshold.sv` **Location:** `rtl/amba/shared/` **Status:**
Production Ready

43.1 Overview

The `axi_monitor_reporter_threshold` module is the threshold-crossing packet emitter for the AXI/AXIL monitor family. It is one of the per-packet-type sub-blocks dispatched by the top-level `axi_monitor_reporter`. It watches two conditions — the number of currently active transactions crossing a configured limit, and any transaction's measured latency exceeding a configured limit — and emits `PktTypeThreshold` packets when either crossing occurs.

The block was split out of the original monolithic reporter so integrators can drop it (`ENABLE_THRESHOLD_LOGIC=0`) and recover real area, because it owns the per-slot latency pipeline (16×32 -bit latency flops plus 16 threshold flags).

43.1.1 Key Features

- Active-transaction-count threshold detection (instantaneous condition)
 - Per-slot latency threshold detection with a registered latency pipeline
 - Edge-sticky flags to fire once per crossing rather than every cycle
 - Read/write latency measurement selectable via `IS_READ`
 - Internal arbitration: active-count wins over latency events
 - One packet at a time via `pkt_valid` / `pkt_taken` handshake
 - Emits `AXI_THRESH_ACTIVE_COUNT` and `AXI_THRESH_LATENCY` event codes
-

43.2 Module Purpose

Monitoring a bus for congestion and latency outliers requires two distinct checks: how many transactions are in flight right now, and whether any individual transaction took too long. This block performs both. The active count is computed combinationally by scanning the transaction table; latency is computed per slot in a registered pipeline (splitting the wide subtract-and-compare across a cycle to close timing) and compared against the configured latency threshold.

Use Cases: - Detecting bus congestion when outstanding-transaction count exceeds a budget - Flagging latency-outlier transactions that exceed a service-level target - Feeding threshold-crossing telemetry to a host for QoS monitoring - Regression checks that latency stays within expected bounds

Key Benefit: Real area savings when disabled (the 16-deep latency pipeline is removed), plus a timing-friendly split of the latency compare so the wide carry chain does not sit on the packet-generation critical path.

43.3 Parameters

Parameter	Type	Default	Description
MAX_TRANSACTIONS	int	16	Number of transaction slots scanned for active count and latency
IS_READ	bit	1'b1	1 = measure read latency (data – addr timestamp); 0 = measure write latency (resp – addr timestamp)
IDX_W	int	$\$clog2(MAX_TRANSACTIONS)$	Derived width of the selected-slot index

43.4 Port Groups

43.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Monitor clock
aresetn	input	1	Active-low asynchronous reset

43.4.2 Transaction Table and Configuration

Port	Direction	Width	Description
trans_table	input	bus_transaction_t [MAX_TRANSACTIONS]	Live outstanding-transaction table (valid, state, timestamps, channel)
cfg_thres_hold_enab	input	1	Enable threshold detection

Port	Direction	Width	Description
le			
active_transactions_threshold	input	16	Active-transaction-count limit; crossing above it fires a packet
latency_threshold	input	32	Per-transaction latency limit (in timestamp ticks)

43.4.3 Handshake / Backpressure

Port	Direction	Width	Description
output_busy	input	1	Output bus busy (FIFO read or monbus_valid); threshold packets inject directly, so this prevents overwriting an in-flight one
pkt_taken	input	1	Pulsed by the top reporter when this block's packet was accepted; clears the edge flag and arms the next detection

43.4.4 Packet Output

Port	Direction	Width	Description
pkt_valid	output	1	A threshold packet is available this cycle
pkt_type	output	4	Packet type — constant PktTypeThreshold (4'h2)
pkt_event_code	output	8	Event code: AXI_THRESH_ACTIVE_COUNT (8'h0) or AXI_THRESH_LATENCY (8'h1)
pkt_channel	output	9	Channel of the selected transaction (0 for active-count events)

Port	Direction	Width	Description
pkt_data	output	64	Zero-extended active count, or the zero-extended latency value

43.5 Functional Description

43.5.1 Active-Count Detection

A combinational loop scans `trans_table` and counts slots that are `valid` and in a state other than `TRANS_COMPLETE` or `TRANS_ERROR` — that is, transactions genuinely in flight.

`w_active_detect` asserts when threshold detection is enabled, the count strictly exceeds `active_trans_threshold`, the edge flag `r_active_crossed` is not already set, and the output is not busy. Because this reflects an instantaneous condition, it takes priority in the output mux.

43.5.2 Per-Slot Latency Pipeline

For each slot, a registered pipeline computes latency from the live transaction table:

`data_timestamp - addr_timestamp` for reads (`IS_READ=1`) or `resp_timestamp - addr_timestamp` for writes. The result is stored in `r_latency[idx]`, and a companion flag `r_latency_over_thresh[idx]` is set when the slot is valid, in `TRANS_COMPLETE` state, its latency exceeds `latency_threshold`, and the latency edge flag is not already set. Registering the subtract and compare splits what would otherwise be a 16-wide carry chain plus output mux across a clock cycle.

43.5.3 Latency Event Selection

A combinational priority encoder (`w_lat_sel / w_has_lat`) picks the first slot with `r_latency_over_thresh` asserted, off the registered pipeline so it does not recreate the wide combinational path.

43.5.4 Edge-Sticky Flags

Two flags prevent re-firing on the same crossing every cycle:

- `r_active_crossed` — set when an active-count packet is accepted (`pkt_taken` with matching type/event code); cleared automatically when the active count drops back to or below the threshold.
- `r_latency_crossed` — set when a latency packet is accepted; gates further latency detections until re-armed.

43.5.5 Output Multiplexer

Active-count beats latency. When `w_active_detect` is asserted, `pkt_valid` fires with `AXI_THRESH_ACTIVE_COUNT`, `pkt_data` = zero-extended active count, and `pkt_channel` = 0. Otherwise, when a latency event is pending and the output is free (`w_has_lat && ! output_busy`), `pkt_valid` fires with `AXI_THRESH_LATENCY`, `pkt_data` = the selected slot's latency, and `pkt_channel` = that slot's channel (low 6 bits, zero-extended).

43.6 Usage Example

This block is instantiated inside `axi_monitor_reporter`, not by users directly:

```
axi_monitor_reporter_threshold #(
    .MAX_TRANSACTIONS (MAX_TRANSACTIONS),
    .IS_READ          (IS_READ)
) u_reporter_threshold (
    .aclk              (aclk),
    .aresetn           (aresetn),

    .trans_table       (w_trans_table),
    .cfg_threshold_enable (cfg_threshold_enable),
    .active_trans_threshold (cfg_active_trans_threshold),
    .latency_threshold  (cfg_latency_threshold),

    .output_busy       (w_output_busy),
    .pkt_taken         (w_thresh_pkt_taken),

    .pkt_valid         (w_thresh_pkt_valid),
    .pkt_type          (w_thresh_pkt_type),
    .pkt_event_code    (w_thresh_pkt_event_code),
    .pkt_channel       (w_thresh_pkt_channel),
    .pkt_data          (w_thresh_pkt_data)
);
```

43.7 Design Notes

43.7.1 Threshold Packets Bypass the FIFO

Unlike error/completion packets, threshold packets inject directly into the output rather than routing through the reporter FIFO. That is why the block needs `output_busy` — to avoid overwriting a packet already in flight on the MonBus.

43.7.2 Active Count vs Latency Priority

Active count is an instantaneous, whole-table condition; latency is a per-slot event that has already been captured in the pipeline flag. The mux deliberately favors the active-count crossing so the more time-sensitive congestion signal is never starved by a queue of latency reports.

43.7.3 Latency Pipeline Is the Area Cost

The 16×32 -bit latency registers plus 16 threshold flags are the reason disabling this block (ENABLE_THRESHOLD_LOGIC=0) yields a meaningful area reduction. When the feature is not needed, dropping the block removes the entire pipeline.

43.7.4 Read vs Write Latency

IS_READ selects which timestamp pair defines latency. Instantiate a read-monitor threshold block with IS_READ=1 and a write-monitor block with IS_READ=0 so the measured interval matches the protocol phase of interest.

43.8 Related Modules

43.8.1 Used By

- **axi_monitor_reporter.sv** — instantiates this block as its threshold-packet sub-emitter
- **axi_monitor_base.sv** — top-level monitor scaffold

43.8.2 Uses

- **monitor_common_pkg** — PktTypeThreshold, bus_transaction_t, transaction states
- **monitor_amba4_pkg** — AXI_THRESH_ACTIVE_COUNT / AXI_THRESH_LATENCY event codes
- **reset_defs.svh** — reset macros

43.8.3 See Also

- **axi_monitor_reporter_perf.sv** — performance packet emitter (sibling)
 - **axi_monitor_reporter_timeout.sv** — timeout packet emitter (sibling)
 - **axi_monitor_trans_mgr.sv** — maintains the trans_table consumed here
-

43.9 References

43.9.1 Source Code

- RTL: rtl/amba/monitor/axi_monitor_reporter_threshold.sv
- Parent: rtl/amba/monitor/axi_monitor_reporter.sv
- Packages: rtl/amba/includes/monitor_common_pkg.sv, rtl/amba/includes/monitor_amba4_pkg.sv

43.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - Monitor Base: docs/markdown/RTLAmba/axi_monitor_base.md
 - Packet Format: docs/markdown/RTLAmba/includes/monitor_package_spec.md
-

Last Updated: 2026-07-15

43.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

44 AXI Monitor Reporter — Timeout Packets

Module: axi_monitor_reporter_timeout.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

44.1 Overview

The axi_monitor_reporter_timeout module is the timeout-packet emitter for the AXI/AXIL monitor family. It is one of the per-packet-type sub-blocks dispatched by the top-level axi_monitor_reporter. It is a pure combinational cone that scans the transaction table for unreported error slots the timeout detector has flagged, priority-encodes the first match, and drives the corresponding PktTypeTimeout packet fields.

The block was split out of the original monolithic reporter so integrators can drop it (ENABLE_TIMEOUT_LOGIC=0).

44.1.1 Key Features

- Pure combinational detection (no internal state)
 - Scans for valid, unreported, error-state slots flagged as timed out
 - Priority-encodes the first matching slot
 - Emits the packet's event code, channel, and address directly from the table
 - Exposes the selected slot index (`sel_idx`) for the top reporter's mark-reported feedback
 - Shares the `timeout_detected` vector with the error sub-block to avoid double-reporting
-

44.2 Module Purpose

When a transaction stalls, the `axi_monitor_timeout` block asserts a per-slot `timeout_detected` bit and the transaction manager moves that slot into the `TRANS_ERROR` state. This reporter block turns that condition into a MonBus packet: it finds the first slot that is valid, in error, timed out, and not yet reported, and emits a timeout packet describing it (event code, channel, and address).

Use Cases: - Reporting stuck AXI transactions (missing R/B response, hung handshake) - Surfacing protocol hangs to a host IRQ handler for recovery - Regression detection of deadlock or backpressure faults - Correlating a timeout with the offending address and channel

Key Benefit: Zero-state, low-cost detection that shares its `timeout_detected` and `event_reported` vectors with the error emitter, guaranteeing each stuck transaction is reported exactly once.

44.3 Parameters

Parameter	Type	Default	Description
<code>MAX_TRANSACTIONS</code>	<code>int</code>	16	Number of transaction slots scanned
<code>IDX_W</code>	<code>int</code>	<code>\$clog2(MAX_TRANSACTIONS)</code>	Derived width of the selected-slot index

44.4 Port Groups

44.4.1 Inputs

Port	Direction	Width	Description
trans_table	input	bus_transaction_t [MAX_TRANSACTIONS]	Live outstanding-transaction table (valid, state, event_code, channel, addr)
event_reported	input	MAX_TRANSACTIONS	Per-slot bit indicating the slot's event was already emitted (suppresses re-reporting)
timeout_detected	input	MAX_TRANSACTIONS	Per-slot timeout flags from axi_monitor_timeout
cfg_timeout_enable	input	1	Enable timeout packet generation

44.4.2 Packet Output

Port	Direction	Width	Description
pkt_valid	output	1	A timeout packet is available (a matching slot was found)
pkt_type	output	4	Packet type — constant PktTypeTimeout (4'h3)
pkt_event_code	output	8	Event code taken from the selected slot's event_code.raw_code
pkt_channel	output	9	Channel of the selected slot (low 6 bits, zero-extended)
pkt_data	output	64	Zero-extended address of the selected slot
sel_idx	output	IDX_W	Index of the selected slot, returned to the top reporter for mark-reported feedback

44.5 Functional Description

44.5.1 Candidate Detection

A combinational loop builds a one-hot-eligible mask `w_events`: a slot qualifies when it is `valid`, its `event_reported` bit is clear, its state is `TRANS_ERROR`, timeout reporting is enabled (`cfg_timeout_enable`), and the slot's `timeout_detected` bit is set. This intersection ensures only genuinely-timed-out, not-yet-reported error slots are candidates.

44.5.2 Priority Encoding

A second loop scans `w_events` from index 0 upward and latches the first asserted slot into `w_sel`, asserting `w_has_event`. This gives a deterministic, lowest-index-first selection.

44.5.3 Output Assignment

The outputs are pure continuous assignments off the selected slot:

- `pkt_valid = w_has_event`
- `sel_idx = w_sel`
- `pkt_type = PktTypeTimeout`
- `pkt_event_code = trans_table[w_sel].event_code.raw_code`
- `pkt_channel = {3'b0, trans_table[w_sel].channel[5:0]}`
- `pkt_data = pad_address(trans_table[w_sel].addr)` (zero-extended to 64 bits)

44.5.4 No Double-Reporting

The error sub-block masks the same `timeout_detected` slots, so a stuck transaction that is reported as a timeout here is not also reported as a plain error. The top reporter uses `sel_idx` to set the slot's `event_reported` bit, retiring the candidate.

44.6 Usage Example

This block is instantiated inside `axi_monitor_reporter`, not by users directly:

```
axi_monitor_reporter_timeout #(
    .MAX_TRANSACTIONS (MAX_TRANSACTIONS)
) u_reporter_timeout (
    .trans_table      (w_trans_table),
    .event_reported   (w_event_reported),
    .timeout_detected (w_timeout_detected),
    .cfg_timeout_enable (cfg_timeout_enable),
```

```

        .pkt_valid          (w_timeout_pkt_valid),
        .pkt_type          (w_timeout_pkt_type),
        .pkt_event_code    (w_timeout_pkt_event_code),
        .pkt_channel       (w_timeout_pkt_channel),
        .pkt_data          (w_timeout_pkt_data),
        .sel_idx           (w_timeout_sel_idx)
    );

```

44.7 Design Notes

44.7.1 Pure Combinational

The block holds no state; all edge-tracking and mark-reported bookkeeping lives in the top reporter. This keeps the timeout cone cheap and lets the top reporter arbitrate timeout packets against the other sub-emitters within one cycle.

44.7.2 Event Code Comes From the Slot

`pkt_event_code` is not a fixed constant — it is the slot's captured `event_code.raw_code`, so the packet carries the specific timeout reason recorded when the transaction manager moved the slot to `TRANS_ERROR`.

44.7.3 Shared Vectors Are Load-Bearing

Correct single-reporting depends on the top reporter driving `event_reported` and the timeout detector driving `timeout_detected` consistently. Historically, a missing `event_reported` feedback path caused transaction-table exhaustion; that feedback is now in place (see [rtl/amba/KNOWN_ISSUES/axi_monitor_reporter.md](#)).

44.8 Related Modules

44.8.1 Used By

- **axi_monitor_reporter.sv** — instantiates this block as its timeout-packet sub-emitter
- **axi_monitor_base.sv** — top-level monitor scaffold

44.8.2 Uses

- **monitor_common_pkg** — `PktTypeTimeout`, `bus_transaction_t`, `TRANS_ERROR`

- **monitor_amba4_pkg** — AMBA event-code definitions
- (No sequential logic — no reset macros required)

44.8.3 See Also

- **axi_monitor_timeout.sv** — produces the timeout_detected vector consumed here
 - **axi_monitor_reporter_perf.sv** — performance packet emitter (sibling)
 - **axi_monitor_reporter_threshold.sv** — threshold packet emitter (sibling)
-

44.9 References

44.9.1 Source Code

- RTL: rtl/amba/monitor/axi_monitor_reporter_timeout.sv
- Parent: rtl/amba/monitor/axi_monitor_reporter.sv
- Packages: rtl/amba/includes/monitor_common_pkg.sv, rtl/amba/includes/monitor_amba4_pkg.sv

44.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - Monitor Base: docs/markdown/RTLAmba/axi_monitor_base.md
 - Known Issues: rtl/amba/KNOWN_ISSUES/axi_monitor_reporter.md
 - Packet Format: docs/markdown/RTLAmba/includes/monitor_package_spec.md
-

Last Updated: 2026-07-15

44.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

45 AXI Monitor Timeout

Module: axi_monitor_timeout.sv **Location:** rtl/amba/shared/ **Category:** Core Infrastructure **Status:** ✓ Production Ready

45.1 Overview

The `axi_monitor_timeout` module provides Configurable timeout detection for stuck transactions.

This is a **shared infrastructure module** used internally by AXI/AXIL monitors. It is not typically instantiated directly by users but is critical for understanding the monitor architecture.

45.2 Key Features

- ✓ **Per-phase timeout detection (AR/AW, R/W, B):** Per-phase timeout detection (AR/AW, R/W, B)
 - ✓ **Configurable timeout thresholds:** Configurable timeout thresholds
 - ✓ **Frequency scaling for timeout counts:** Frequency scaling for timeout counts
 - ✓ **Timeout event reporting with transaction ID:** Timeout event reporting with transaction ID
 - ✓ **Active transaction tracking:** Active transaction tracking
 - ✓ **Timeout clear on transaction completion:** Timeout clear on transaction completion
-

45.3 Module Purpose

The `axi_monitor_timeout` module is the core building block for:

1. **Per-Phase Monitoring:** Separate timeouts for AR/AW, R/W, B channels
 2. **Configurable Thresholds:** Adjustable timeout values per phase
 3. **Event Reporting:** Generates timeout packets with transaction details
 4. **Active Tracking:** Monitors all outstanding transactions
 5. **Frequency Scaling:** Adapts timeout counts to system clock frequency
-

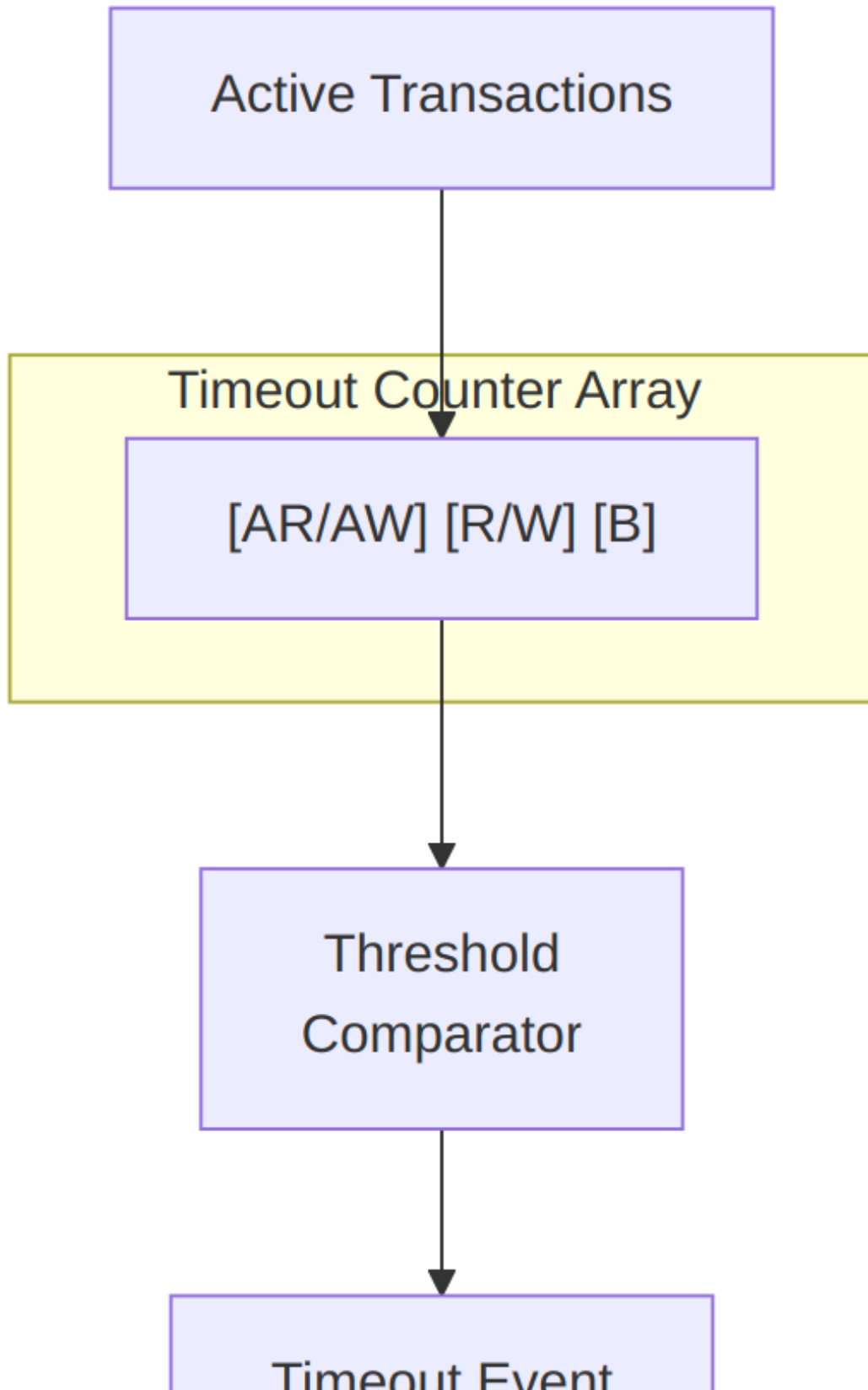
45.4 Parameters

Parameter	Type	Default	Description
MAX_TRANSACTION	int	16	Number of transactions to monitor
ADDR_WIDTH	int	32	Address width for reporting
IS_READ	bit	1	1 = read channel, 0 = write channel

45.5 Port Groups

See RTL source: `rtl/amba/monitor/axi_monitor_timeout.sv` for complete port listing.

Key interface groups: - Clock and reset - Input signals from monitored interface - Configuration signals - Output signals to downstream logic



Architecture

Each transaction has 3 independent timeout counters for different phases.

45.6 Usage in Monitor System

This module is used by:

- **axi_monitor_base**

45.6.1 Internal Integration

This module is instantiated automatically within higher-level monitor modules. Users configure behavior through top-level monitor parameters.

45.7 Configuration Guidelines

See individual monitor documentation for configuration examples.

Configuration is typically handled at the top-level monitor instantiation.

45.8 Performance Characteristics

Metric	Value	Notes
Latency	1-2 cycles	Typical processing delay
Throughput	1 operation/cycle	Maximum rate
Resource Usage	Varies	Depends on configuration

45.9 Verification Considerations

45.9.1 Test Coverage

- Functional correctness of core logic
- Boundary conditions (min/max values)
- Error handling and recovery

- Interface protocol compliance

See: `val/amba/test_axi_monitor_timeout.py` for verification tests

45.10 Related Modules

- [axi_monitor_base](#)
-

45.11 See Also

- **Monitor Architecture:** `docs/markdown/RTLAmba/overview.md`
 - **Monitor Configuration Guide:** [Monitor Base Configuration](#)
 - **Packet Format Specification:**
`docs/markdown/RTLAmba/includes/monitor_package_spec.md`
-

45.12 Navigation

- [← Back to Shared Infrastructure Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

46 AXI Monitor Timer

Module: `axi_monitor_timer.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

46.1 Overview

The AXI Monitor Timer provides frequency-invariant timing for the AXI monitor infrastructure by generating consistent timer ticks regardless of the system clock frequency. It combines a cycle-accurate global timestamp counter with a frequency-invariant timer tick generator to support both precise latency measurement and consistent timeout detection across different clock frequencies.

46.1.1 Key Features

- Frequency-invariant timer tick generation
- Configurable tick frequency via 4-bit selection

- Global timestamp counter (32-bit cycle counter)
 - Uses counter_freq_invariant for portable timing
 - Zero-latency timestamp (combinational output)
 - Consistent timeout behavior across frequencies
 - Single clock domain operation
 - Minimal resource utilization
-

46.2 Module Purpose

The Timer module solves a critical portability problem in monitor architectures: timeout thresholds specified in cycles become frequency-dependent, requiring reconfiguration when clock frequency changes. By using frequency-invariant timer ticks, timeout thresholds can be specified in abstract “ticks” that represent consistent time periods regardless of the underlying clock frequency.

The global timestamp counter provides cycle-accurate timing for latency measurement and performance analysis, while the timer tick output enables timeout detection with configurable resolution.

46.3 Parameters

Runtime frequency selection is via `cfg_freq_sel`. The module also exposes elaboration parameters that size the clock-frequency-invariant (CFI) counter table:

Parameter	Type	Default	Description
CFI_MIN_FREQ_MHZ	int	5	Minimum frequency (MHz) in the CFI table
CFI_MAX_FREQ_MHZ	int	220	Maximum frequency (MHz) in the CFI table
CFI_NUM_FREQ_ENTRIES	int	16	Number of frequency entries
CFI_FREQ_STRATEGY	int	0	Frequency-spacing strategy selector

SEL_WIDTH is derived from CFI_NUM_FREQ_ENTRIES and sizes `cfg_freq_sel`.

46.4 Port Groups

46.4.1 Global Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock signal
aresetn	input	1	Active-low asynchronous reset

46.4.2 Timer Configuration

Port	Direction	Width	Description
cfg_freq_sel	input	4	Frequency selection for timer tick (0-15)

46.4.3 Timer Outputs

Port	Direction	Width	Description
timer_tick	output	1	Timer tick signal (frequency-invariant)
timestamp	output	32	Global timestamp counter (cycle count)

46.5 Functional Description

The Timer module implements two independent timing functions that work together to support the monitor infrastructure.

46.5.1 Global Timestamp Counter

The timestamp counter provides cycle-accurate time reference:

Implementation: - 32-bit up-counter - Increments every clock cycle - Resets to zero on aresetn assertion - Registered output (r_timestamp)

Purpose: - Transaction timestamp marking (addr_timestamp, data_timestamp, resp_timestamp) - Latency calculation (timestamp differences) - Performance metric computation - Event correlation across transactions

Range and Wraparound: - Maximum count: $2^{32} - 1 = 4,294,967,295$ cycles - At 1 GHz: ~4.3 seconds before wraparound - At 100 MHz: ~43 seconds before wraparound - Wraparound handled naturally in latency calculations (32-bit arithmetic)

Usage Example:

```
// Transaction manager records timestamp  
addr_timestamp <= timestamp; // When address phase completes  
  
// Later, calculate latency  
latency = data_timestamp - addr_timestamp; // Handles wraparound
```

46.5.2 Frequency-Invariant Timer Tick

The timer tick provides consistent periodic signal across frequencies:

Implementation: - Uses counter_freq_invariant module from rtl/common/ - Configured via cfg_freq_sel (4-bit selection: 0-15) - Generates single-cycle pulse (timer_tick) at configured interval - Combinational output (w_timer_tick)

Frequency Selection: The cfg_freq_sel value selects from predefined frequency ratios:

cfg_freq_sel	Tick Period (cycles)	100 MHz Period	1 GHz Period
0	1	10 ns	1 ns
1	2	20 ns	2 ns
2	4	40 ns	4 ns
3	8	80 ns	8 ns
4	16	160 ns	16 ns
5	32	320 ns	32 ns
6	64	640 ns	64 ns
7	128	1.28 us	128 ns
8	256	2.56 us	256 ns
9	512	5.12 us	512 ns
10	1024	10.24 us	1.024 us
11	2048	20.48 us	2.048 us
12	4096	40.96 us	4.096 us

cfg_freq_sel	Tick Period (cycles)	100 MHz Period	1 GHz Period
13	8192	81.92 us	8.192 us
14	16384	163.84 us	16.384 us
15	32768	327.68 us	32.768 us

Purpose: - Timeout detection timing base - Consistent timeout duration across frequencies - Periodic performance metric reporting - Threshold comparison timing

Tick Characteristics: - Single cycle pulse (asserts high for 1 cycle) - Precise period (no jitter or drift) - Synchronous to aclk - Immediately reflects cfg_freq_sel changes

46.5.3 Counter Freq Invariant Integration

The module instantiates counter_freq_invariant with specific configuration:

Instance Configuration:

```
counter_freq_invariant #(
    .COUNTER_WIDTH (1),           // Only need 1-bit counter (tick pulse)
    .PRESCALER_MAX (65536)       // Maximum prescaler value
) timer_counter (
    .clk            (aclk),
    .rst_n          (aresetn),
    .sync_reset_n(1'b1),         // No sync reset
    .freq_sel       (cfg_freq_sel),
    .tick           (w_timer_tick),
    .counter        ()           // Counter output unused
);
```

Design Rationale: - COUNTER_WIDTH=1: Only tick pulse needed, not counter value - PRESCALER_MAX=65536: Supports full frequency selection range - sync_reset_n tied high: No need for synchronous reset - counter output unused: Only tick signal required

counter_freq_invariant Module: - Located in rtl/common/counter_freq_invariant.sv - Implements programmable frequency divider - Uses binary counter with configurable threshold - Generates precise periodic pulses - See rtl/common/ documentation for detailed behavior

46.6 Integration with Monitor Architecture

The Timer module provides fundamental timing services to the entire monitor infrastructure.

46.6.1 Output Integration

To Transaction Manager (axi_monitor_trans_mgr): - timestamp output provides time reference
- Used to mark transaction phase completion times - Enables latency calculation in reporter

To Timeout Module (axi_monitor_timeout): - timer_tick output drives timeout detection -
Timeout module increments counters on each tick - Enables frequency-invariant timeout thresholds

To Reporter Module (axi_monitor_reporter): - Indirect use via transaction table timestamps -
Reporter calculates latencies from timestamp differences - Performance metrics derived from timestamp data

46.6.2 Configuration Strategy

High-Frequency Systems (>500 MHz):

```
cfg_freq_sel = 4'd8; // 256 cycles (~256ns @ 1GHz)
```

Provides fine timeout granularity without excessive tick rate.

Medium-Frequency Systems (100-500 MHz):

```
cfg_freq_sel = 4'd6; // 64 cycles (~640ns @ 100MHz)
```

Balanced granularity and overhead.

Low-Frequency Systems (<100 MHz):

```
cfg_freq_sel = 4'd4; // 16 cycles (~160ns @ 100MHz)
```

Finer granularity needed due to lower clock frequency.

General Guideline: Select `cfg_freq_sel` such that:

Tick Period = Target Timeout / (Max Threshold Value)

For example, 10us timeout with max threshold=10:

Tick Period = 10us / 10 = 1us

@ 100MHz: 1us = 100 cycles -> `cfg_freq_sel = 10` (1024 cycles ~ 10us)

46.7 Usage Example

```
// Timer module instance
axi_monitor_timer u_timer (
    .aclk          (axi_clk),
    .aresetn       (axi_rst_n),
```

```

    // Frequency selection
    .cfg_freq_sel (4'd6), // 64 cycles per tick

    // Timer outputs
    .timer_tick    (timer_tick),
    .timestamp      (global_timestamp)
);

// Transaction manager uses timestamp
axi_monitor_trans_mgr #(
    .MAX_TRANSACTIONS (16),
    // ...
) u_trans_mgr (
    .aclk          (axi_clk),
    .aresetn        (axi_rst_n),

    // Timestamp input
    .timestamp      (global_timestamp),

    // ... other connections
);

// Timeout module uses timer tick
axi_monitor_timeout #(
    .MAX_TRANSACTIONS (16),
    // ...
) u_timeout (
    .aclk          (axi_clk),
    .aresetn        (axi_rst_n),

    // Timer tick input
    .timer_tick    (timer_tick),

    // Timeout thresholds (in ticks)
    .cfg_addr_cnt  (4'd8), // 8 ticks
    .cfg_data_cnt  (4'd12), // 12 ticks
    .cfg_resp_cnt  (4'd8), // 8 ticks

    // ... other connections
);

// Timeout duration calculation
// At cfg_freq_sel=6 (64 cycles/tick):
// addr timeout = 8 ticks * 64 cycles = 512 cycles
// data timeout = 12 ticks * 64 cycles = 768 cycles
// resp timeout = 8 ticks * 64 cycles = 512 cycles

```

46.8 Design Notes

46.8.1 Timestamp Wraparound Handling

The 32-bit timestamp counter wraps to zero after ~4.3 seconds at 1 GHz:

Latency Calculation: Latency calculations naturally handle wraparound through unsigned arithmetic:

```
latency = end_timestamp - start_timestamp; // Works across wraparound
```

Example:

```
start_timestamp = 0xFFFF_FFFF (max value)
end_timestamp   = 0x0000_0005 (after wraparound)
latency = 0x0000_0005 - 0xFFFF_FFFF = 0x0000_0006 (6 cycles)
```

Limitations: - Maximum measurable latency: 2^{31} cycles (half of counter range) - Exceeding this causes incorrect results - In practice, transaction latencies $\ll 2^{31}$ cycles

Mitigation: If longer-term timestamps needed, extend timestamp to 64 bits (requires parameter addition).

46.8.2 Frequency Selection Trade-offs

Choosing `cfg_freq_sel` involves trade-offs:

Fine Granularity (Low `cfg_freq_sel`): - Advantages: Precise timeout detection, responsive system
- Disadvantages: High tick rate, more dynamic power - Use when: Fast timeout response required

Coarse Granularity (High `cfg_freq_sel`): - Advantages: Lower tick rate, reduced power -
Disadvantages: Coarse timeout resolution, slower detection - Use when: Power-sensitive or timeout precision less critical

Recommended Range: `cfg_freq_sel` = 4-8 for most applications (16-256 cycles per tick)

46.8.3 Dynamic Frequency Scaling

When system clock frequency changes (DFS), consider:

Timestamp Counter: - Continues counting cycles - Latency measurements in cycles, not absolute time - Latency will vary with frequency changes

Timer Tick: - Tick period in cycles remains constant - Tick period in absolute time changes - May need to adjust `cfg_freq_sel` to maintain desired timeout duration

Recommendation: If DFS used, adjust `cfg_freq_sel` proportionally to maintain consistent timeout durations in absolute time.

46.8.4 Zero Timestamp on Reset

The timestamp counter resets to zero on aresetn assertion:

Implications: - First transaction after reset has timestamp near zero - Timestamps from before reset invalid after reset - No timestamp preservation across resets

Best Practice: Reset all monitors simultaneously to ensure consistent timestamp domain across the system.

46.8.5 Timer Tick Duty Cycle

The timer_tick output is a single-cycle pulse:

Characteristics: - High for exactly 1 cycle - Low for (tick_period - 1) cycles - Duty cycle = $1 / \text{tick_period}$

Usage: Modules should use timer_tick as an enable/trigger signal, not as a clock. It's synchronous to aclk but much slower, making it unsuitable as a clock source.

46.8.6 Resource Utilization

The timer module has minimal resource cost:

Registers: - 32-bit timestamp counter - 1-bit counter in counter_freq_invariant - Prescaler state in counter_freq_invariant

Combinational Logic: - Increment logic for timestamp - Frequency divider in counter_freq_invariant

Estimated Area: - ~50 FFs (flip-flops) - ~100 LUTs (look-up tables) - Negligible relative to full monitor

46.8.7 Multiple Timer Instances

Each monitor instance can have its own timer if needed:

Advantages of Separate Timers: - Independent frequency selection per monitor - No timing constraints between monitors - Simpler physical design (local timing)

Advantages of Shared Timer: - Single timestamp domain across monitors - Easier event correlation - Reduced resource utilization

Recommendation: Use shared timer unless monitors operate at different frequencies or in different clock domains.

46.9 Related Modules

46.9.1 Used By

- `axi_monitor_base.sv` (primary user)
- `axi_monitor_filtered.sv` (via `axi_monitor_base`)
- All AXI/AXIL monitor variants

46.9.2 Uses

- `counter_freq_invariant.sv` (from `rtl/common/`)
- Provides frequency-invariant counting capability

46.9.3 Critical Interfaces

- **To `axi_monitor_trans_mgr`:**
 - timestamp output (32-bit time reference)
- **To `axi_monitor_timeout`:**
 - timer_tick output (timeout trigger)

46.9.4 Related Infrastructure

- `axi_monitor_trans_mgr.sv` (timestamp consumer)
 - `axi_monitor_timeout.sv` (timer tick consumer)
 - `axi_monitor_reporter.sv` (indirect via timestamps)
-

46.10 References

46.10.1 Specifications

- Base Module: [axi_monitor_base.md](#)
- Timeout Module: [axi_monitor_timeout.md](#)
- Transaction Manager: [axi_monitor_trans_mgr.md](#)

46.10.2 Source Code

- RTL: `rtl/amba/monitor/axi_monitor_timer.sv`
- Counter: `rtl/common/counter_freq_invariant.sv`
- Documentation: `docs/markdown/RTLCommon/counter_freq_invariant.md`
- Tests: `val/amba/test_axi4_master_rd_mon.py` (integration)

46.10.3 Configuration Guides

- [Monitor Base Configuration](#) - Timeout configuration
 - `rtl/amba/PRD.md` - Subsystem requirements
-

Last Updated: 2025-10-24

46.11 Navigation

- [Back to axi_monitor_base](#)
- [Previous: axi_monitor_timeout](#)
- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

47 AXI Monitor Transaction Manager

Module: `axi_monitor_trans_mgr.sv` **Location:** `rtl/amba/shared/` **Category:** Core Infrastructure **Status:** Production Ready (CAM-backed revision, 2026-06-08)

47.1 Overview

`axi_monitor_trans_mgr` tracks outstanding AXI transactions through their `addr` → `data` → `resp` lifecycle. It exposes a registered table of in-flight transactions (`trans_table[N]`) for downstream consumers (reporter, debug, timeout) and feeds the monitor bus with state-change events.

This is a **shared infrastructure module** used internally by every AXI4 / AXI5 / AXI-Lite monitor. Users don't instantiate it directly; they configure behaviour through the top-level monitor wrapper.

The current revision delegates per-transaction keying + storage to the shared `monitor_trans_cam` module. A previous in-place revision (2026-04-23) is parked at `rtl/amba/shared/mon_temp/axi_monitor_trans_mgr.sv` and remains available for rollback or timing comparison.

47.2 Key Features

- Tracks up to `MAX_TRANSACTIONS` outstanding (default 16)

- 3 independent ID lookups per cycle (addr / data / resp) via CAM
 - Out-of-order transaction support
 - Burst beat counting
 - Per-phase timestamps for latency reporting
 - Orphan-data / orphan-resp detection (per AXI4 / AXI-Lite rules)
 - State-change events for downstream packet generation
 - Active-transaction counter
 - Cleanup-when-event-reported handshake with the reporter
 - AXI4 / AXI4-Lite / read / write variants via parameters
-

47.3 Architecture



axi_monitor_trans_mgr block diagram

Source: axi_monitor_trans_mgr.mmd

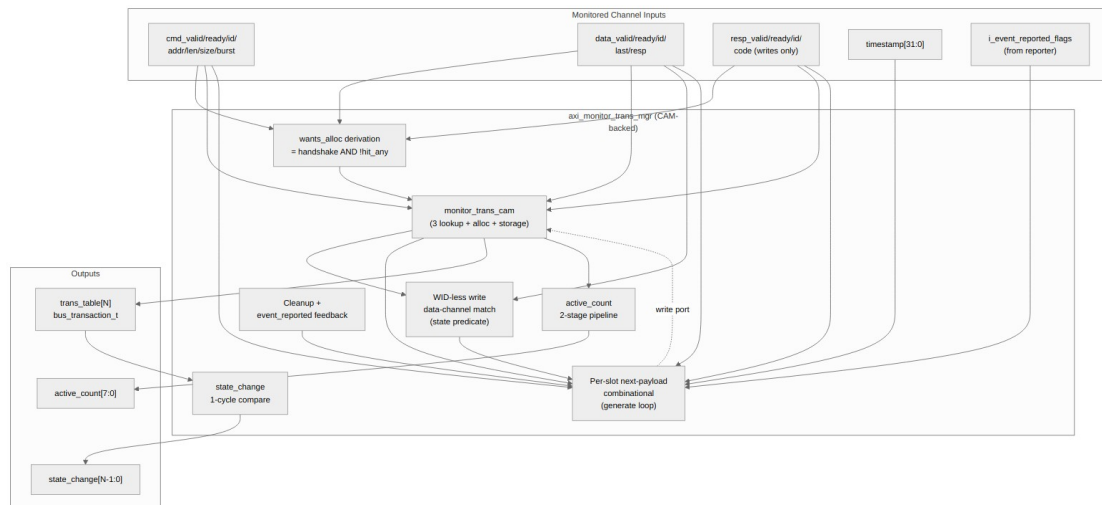


Diagram 21

The trans_mgr owns: - The **WID-less write data-channel match** (state predicate over the payload, not an id match — the CAM only sees ids). - The per-slot **next-payload computation** (the per-phase if/else chain that says “what should slot i’s bus_transaction_t look like next cycle”). - The **wants_alloc** derivation (= input handshake AND not-hit-any). - Cleanup eligibility, event_reported feedback, active_count pipeline, state_change detection.

The CAM owns: - Per-slot (valid, id, payload) storage. - The 3 parallel ID lookups (addr_match_oh, data_match_oh, resp_match_oh). - The free-slot vector and 3-way priority-encoded alloc one-hots.

This split makes the parallel-match shape that closes 100 MHz on the xc7a100t-1 ((* keep = "true" *) per-bit match vectors, per-slot generate-loop storage) explicit and reusable.

47.4 Parameters

Parameter	Type	Default	Description
MAX_TRANSACTIONS	int	16	Transaction table depth
ADDR_WIDTH	int	32	Width of address bus tracked
ID_WIDTH	int	8	Width of AXI ID
IS_READ	bit	1	1 for read monitors, 0 for write

Parameter	Type	Default	Description
IS_AXI	bit	1	1 for AXI4, 0 for AXI-Lite
ENABLE_PERF_PACKETS	bit	0	Reserved — perf packet generation hook

The AW and IW short-alias parameters are retained for API stability with prior revisions; they default to ADDR_WIDTH and ID_WIDTH.

47.5 Module Interface

The module exports a registered trans_table of bus_transaction_t entries (see monitor_amba4_pkg.sv) plus aggregate status:

```

module axi_monitor_trans_mgr
    import monitor_common_pkg::*;
    import monitor_amba4_pkg::*;
#(
    parameter int MAX_TRANSACTIONS = 16,
    parameter int ADDR_WIDTH      = 32,
    parameter int ID_WIDTH        = 8,
    parameter bit IS_READ         = 1'b1,
    parameter bit IS_AXI         = 1'b1,
    ...
) (
    input   logic          aclk,
    input   logic          aresetn,

    // Synchronous clear: empty the transaction CAM and zero the
    // active-count pipeline on the next edge (no full reset needed).
    // Pulse one cycle while idle.
    input   logic          clear,

    // Address channel
    input   logic          cmd_valid,
    input   logic          cmd_ready,
    input   logic [IW-1:0] cmd_id,
    input   logic [AW-1:0] cmd_addr,
    input   logic [7:0]    cmd_len,
    input   logic [2:0]    cmd_size,
    input   logic [1:0]    cmd_burst,

```

```

// Data channel
input  logic                                data_valid,
input  logic                                data_ready,
input  logic [IW-1:0]                      data_id,
input  logic                                data_last,
input  logic [1:0]                        data_resp,

// Response channel (write only)
input  logic                                resp_valid,
input  logic                                resp_ready,
input  logic [IW-1:0]                      resp_id,
input  logic [1:0]                        resp_code,

input  logic [31:0]                        timestamp,
input  logic [MAX_TRANSACTIONS-1:0]        i_event_reported_flags,

output bus_transaction_t
trans_table[MAX_TRANSACTIONS],
output logic [7:0]                        active_count,
output logic [MAX_TRANSACTIONS-1:0]        state_change
);

```

The `bus_transaction_t` struct (defined in `monitor_amba4_pkg.sv`) contains: - State flags: `valid`, `cmd_received`, `data_started`, `data_completed`, `resp_received`, `event_reported`, `eos_seen` - FSM state: `state` (`TRANS_IDLE` / `ADDR_PHASE` / `DATA_PHASE` / `COMPLETE` / `ERROR` / `ORPHANED`) - Captured fields: `addr`, `id`, `len`, `size`, `burst`, `channel` - Timers and timestamps: `addr_timer`, `data_timer`, `resp_timer`, `addr_timestamp`, `data_timestamp`, `resp_timestamp` - Beat tracking: `expected_beats`, `data_beat_count` - `event_code` union (`axi_error` / `axi_timeout` / etc.)

Total: 285 bits per entry. With `MAX_TRANSACTIONS=16`, the `trans_table` is ~4.5 Kb of registered state.

47.6 Transaction Lifecycle

A typical AXI4 read transaction:

```

cmd_valid handshake →
(cmd_id, cmd_addr, ...)

addr_alloc fires (free CAM slot picked)
valid          = 1
state          = TRANS_ADDR_PHASE
id             = cmd_id
cmd_received= cmd_ready
expected_beats = cmd_len + 1
addr_timestamp = timestamp

```

cmd_handshake → on same id again	addr_update fires (slot already exists) cmd_received <= 1 addr_timer <= 0 addr_timestamp <= timestamp
data_valid handshake → (data_id == cmd_id, data_last, data_resp)	data_update fires (id match) data_started <= 1 data_beat_count++ state <= TRANS_DATA_PHASE if data_last: data_completed <= 1 state <= TRANS_COMPLETE if data_resp[1]: # RESP error state <= TRANS_ERROR event_code <= EVT_RESP_*
event_reported flag → pool from reporter cycle	(later, reporter handles the event) cleanup fires valid <= 0 # slot returned to free # CAM sees the entry as free on next cycle

For AXI4 writes the data phase uses a state predicate match (not id), since AXI4 W has no WID. For AXI-Lite writes, the data channel CAN allocate an orphan slot if data arrives before the AW handshake.

47.7 Synthesis Notes

The CAM-backed revision preserves the 2026-04-23 WNS fix:

Construct	Rationale
Per-slot always_comb for next-payload (in a generate loop)	N independent small cones; synth cannot fuse them across slots
Per-slot CAM storage via monitor_trans_cam (generate-loop always_ff)	Same property at the registered storage layer
(* keep = "true" *) on CAM match vectors	Prevents Vivado from fusing match- result usage into the update cones, which would re-introduce the 12-LUT- level WNS issue
Registered alloc/cleanup one-hot	The popcount-from-OR-of-three-

Construct	Rationale
vectors (q_addr_alloc_oh, q_data_alloc_oh, q_resp_alloc_oh, q_cleanup_vec) feeding the active_count popcount tree	onehots adder is now one cycle behind the events that drove it. Both alloc <i>and</i> cleanup are registered by the same amount, so the accounting balances — it just lags 1 cycle. Status-counter consumers don't care about the lag, and the match → alloc → popcount → flop critical path drops to within budget at 100 MHz (bee67f51, f909f01f).
Pipelined state_change (1 cycle of r_trans_table_prev)	Cheap comparison against last cycle's table; output lag is 1 cycle
Synchronous clear clears the active-count pipeline alongside the CAM	Without this, asserting clear would invalidate the CAM but leave stale r_alloc_cnt / r_cleanup_cnt, briefly publishing a nonzero active_count against an empty table. The clear handling zeroes both pipeline stages atomically.

The combined effect is that no signal in the trans_mgr has more than ~6 LUT levels between flops at typical configurations — closes 100 MHz on xc7a100t-1 with margin.

47.8 Equivalence Reference

Bit-exact equivalence between the CAM-backed version and the legacy in-place revision is proven by val/amba/test_axi_monitor_trans_mgr_equiv.py:

- 500 cycles of deterministic random stimulus (fixed seed=42)
- Cycle-by-cycle capture of trans_table[N], active_count, state_change
- Bit-exact comparison across the 285-bit bus_transaction_t × N slots, plus the two aggregate outputs

FULL parameter sweep: 6 configs covering AXI4 / AXI-Lite × read / write × small / large
MAX_TRANSACTIONS — 12 tests (6 configs × prod + stage). All 12 pass.

If you make changes to either trans_mgr (production at rtl/amba/shared/ or legacy at rtl/amba/shared/mon_temp/), running the equiv test quickly catches any introduced divergence.

47.9 TRANS_MGR_VARIANT Knob

The full *_mon regression in test_axi4_master_rd_mon.py accepts a TRANS_MGR_VARIANT env var:

Default – current production (CAM-backed)

```
pytest val/amba/test_axi4_master_rd_mon.py -v
```

Roll back to the pre-CAM in-place revision (parked in mon_temp/)

```
TRANS_MGR_VARIANT=legacy pytest val/amba/test_axi4_master_rd_mon.py -v
```

Useful for timing comparisons on real silicon, or as a hot-swap path if a regression sneaks past the equiv test.

47.10 Performance Characteristics

Metric	Value	Notes
Throughput	1 transaction-event/cycle	Per-phase handshake; up to 3 phases (alloc / update / cleanup) can fire per cycle
trans_table latency	1 cycle	Output is registered
active_count latency	2 cycles	Pipelined adder
state_change latency	1 cycle	Compared against prev cycle
Resource	~5 Kb storage	16 × 285-bit struct, in the CAM

47.11 Verification

Test	Coverage
val/amba/ test_axi_monitor_trans_mgr_equiv.	Bit-exact CAM-backed vs legacy equivalence, 500 cycles × 6 param

Test	Coverage
py	configs
val/amba/ test_monitor_trans_cam.py	The CAM sub-module in isolation
val/amba/ test_axi4_master_rd_mon.py and all *_mon* tests	End-to-end through the full monitor stack

Run the equivalence test to validate trans_mgr changes:

```
pytest val/amba/test_axi_monitor_trans_mgr_equiv.py -v
```

47.12 Related Modules

Module	Role
monitor_trans_cam	Per-slot keying + storage with 3 lookup ports and alloc priority encoder
axi_monitor_base	Top-level monitor wrapper that instantiates trans_mgr + timer + reporter
axi_monitor_reporter	Consumes trans_table + state_change to generate monbus packets
axi_monitor_timeout	Watches the per-phase timers in trans_table for timeout events

47.13 See Also

- **Monitor Architecture:** docs/markdown/RTLAmba/overview.md
- **Monitor Configuration Guide:** axi_monitor_base.md
- **Packet Format Specification:** monitor_package_spec.md
- **Legacy parking area:** rtl/amba/shared/mon_temp/ (pre-CAM in-place revision)

47.14 Navigation

- [← Back to Shared Infrastructure Index](#)
- [← Back to RTLAmba Index](#)

48 Monitor Bus Round-Robin Arbiter

Module: `monbus_arbiter.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

48.1 Overview

The Monitor Bus Round-Robin Arbiter aggregates monitor bus packet streams from multiple clients into a single output stream using fair round-robin arbitration with ACK protocol. Optional skid buffers on inputs and output improve timing closure and provide elasticity for backpressure scenarios.

48.1.1 Key Features

- Round-robin arbitration for monitor bus packet streams
 - ACK mode operation (grants held until acknowledged)
 - Optional input skid buffers per client (2, 4, 6, or 8 entry depth)
 - Optional output skid buffer (2, 4, 6, or 8 entry depth)
 - 128-bit packet + 64-bit side-band timestamp, carried atomically through a 192-bit skid
 - Parameterizable client count (1-64)
 - Zero-latency pass-through when skid buffers disabled
-

48.2 Module Purpose

This module serves as an aggregation point for monitor bus streams from multiple monitoring sources (AXI monitors, APB monitors, arbiter monitors, etc.). It ensures fair access to the consolidated monitor bus output while preventing packet loss through integrated buffering and proper backpressure handling.

The ACK protocol ensures that granted clients can complete their packet transmission before the arbiter moves to the next request, preventing fragmentation of multi-cycle transfers.

48.3 Parameters

Parameter	Type	Default	Description
CLIENTS	int	4	Number of monitor bus clients (1-64)
INPUT_SKID_ENABLER	int	1	Enable skid buffers on input interfaces
OUTPUT_SKID_ENABLE	int	1	Enable skid buffer on output interface
INPUT_SKID_DEPTH	int	2	Depth of input skid buffers (2, 4, 6, or 8)
OUTPUT_SKID_DEPTH	int	2	Depth of output skid buffer (2, 4, 6, or 8)

48.4 Port Groups

48.4.1 Clock and Reset

Port	Direction	Width	Description
axi_aclk	input	1	Clock signal
axi_aresetn	input	1	Active-low asynchronous reset

48.4.2 Block Arbitration

Port	Direction	Width	Description
block_arb	input	1	Block arbitration when asserted

48.4.3 Monitor Bus Inputs (Array of CLIENTS)

Port	Direction	Width	Description
monbus_valid_in[CLIENTS]	input	CLIENTS	Per-client packet valid signals
monbus_ready	output	CLIENTS	Per-client ready signals

Port	Direction	Width	Description
ady_in[CLIENTS]			
monbus_packet_in[CLIENTS]	input	CLIENTS × 128	Per-client monitor_packet_t packets
monbus_timestamp_in[CLIENTS]	input	CLIENTS × 64	Per-client monbus_timestamp_t, sampled at each client's emission time

48.4.4 Monitor Bus Output (Aggregated)

Port	Direction	Width	Description
monbus_valid	output	1	Aggregated packet valid
monbus_ready	input	1	Aggregated ready from downstream
monbus_packet	output	128	Aggregated monitor_packet_t
monbus_timestamp	output	64	Aggregated monbus_timestamp_t, paired atomically with monbus_packet

48.4.5 Debug/Status Outputs

Port	Direction	Width	Description
grant_valid	output	1	Grant is active this cycle
grant	output	CLIENTS	One-hot grant vector
grant_id	output	$\$clog2(CLIENTS)$	Binary encoded grant ID
last_grant	output	CLIENTS	Previous grant (for round-robin rotation)

48.5 Functional Description

48.5.1 Arbiter Request and Grant ACK Logic

The module maps monitor bus protocol to arbiter protocol:

Request Mapping: Each client's `monbus_valid_in[i]` becomes `arbiter request[i]`

Grant ACK Logic: ACK occurs when both grant is asserted AND client has valid data:

```
grant_ack[i] = grant[i] && monbus_valid_in[i]
```

This implements the “stick on grant until both request and grant ack are high” requirement, ensuring clients can complete their packet transmission.

48.5.2 Round-Robin Arbiter Instance

Uses `arbiter_round_robin` with `WAIT_GNT_ACK=1`: - Fair rotation through active requests - Grant held until acknowledged - `Block_arb` input for external control

48.5.3 Client Ready Signal Generation

Each client's ready signal asserted when: 1. That client is currently granted AND 2. Downstream (internal output) is ready to accept data

```
monbus_ready_in[i] = grant[i] && int_monbus_ready
```

This ensures only the granted client can transfer data, preventing collisions.

48.5.4 Output Multiplexer

Selects data from the granted client:

```
if (grant_valid)
    int_monbus_packet = monbus_packet_in[grant_id];
```

48.5.5 Optional Skid Buffers

All skid buffers in this arbiter carry the **packet and timestamp atomically** in a 192-bit payload (`MONBUS_PKT_WIDTH + MONBUS_TS_WIDTH = 128 + 64`). The arbiter never separates a packet from its sampled timestamp, so consumers downstream of the [monbus_group family](#) see a coherent (pkt, ts) tuple even after multiple levels of skid.

Input Skid Buffers (per client): - Uses `gaxi_skid_buffer` instances configured for 192-bit data - Provides elasticity for clients with bursty traffic - Improves timing closure by breaking long paths - Depth configurable: 2, 4, 6, or 8 entries

Output Skid Buffer: - Buffers aggregated stream before output (also 192-bit) - Prevents backpressure propagation to arbiter - Same depth options as input buffers

When to Enable Skid Buffers: - Enable INPUT_SKID if clients have high-latency paths or bursty traffic - Enable OUTPUT_SKID if downstream consumer has variable latency - Disable both for minimum latency (zero-overhead pass-through)

48.6 Usage Example

```
// Aggregate monitor bus streams from 4 clients
monbus_arbiter #(
    .CLIENTS                (4),
    .INPUT_SKID_ENABLE       (1),    // Enable input buffers
    .OUTPUT_SKID_ENABLE      (1),    // Enable output buffer
    .INPUT_SKID_DEPTH        (4),    // 4-entry input buffers
    .OUTPUT_SKID_DEPTH       (4)     // 4-entry output buffer
) u_monbus_arb (
    .axi_aclk                (clk),
    .axi_aresetn              (rst_n),
    .block_arb                (1'b0), // Not blocked

    // Connect 4 client monitor bus streams (packet + timestamp per
    // client)
    .monbus_valid_in          ('{mon0_valid, mon1_valid, mon2_valid,
mon3_valid}),
    .monbus_ready_in          ('{mon0_ready, mon1_ready, mon2_ready,
mon3_ready}),
    .monbus_packet_in         ('{mon0_packet, mon1_packet, mon2_packet,
mon3_packet}),
    .monbus_timestamp_in      ('{mon0_ts,      mon1_ts,      mon2_ts,
mon3_ts}),

    // Aggregated output stream (packet + timestamp paired atomically)
    .monbus_valid              (agg_valid),
    .monbus_ready              (agg_ready),
    .monbus_packet              (agg_packet),
    .monbus_timestamp           (agg_ts),

    // Debug outputs
    .grant_valid               (arb_grant_valid),
    .grant                     (arb_grant_onehot),
    .grant_id                   (arb_grant_id),
    .last_grant                 (arb_last_grant)
);

// Downstream consumer is typically the monbus_group family, which
// expects the
// 128-bit packet on monbus_packet and the 64-bit timestamp on
```

```
// monbus_timestamp. No additional FIFO is needed at this boundary –  
// the group has its own ingress skid.
```

48.7 Design Notes

48.7.1 ACK Mode Operation

The ACK mode ensures proper packet transfer:

Grant Assertion: When arbiter selects a client, grant[i] asserts **Client Response:** Client's monbus_valid_in[i] must be high **Grant ACK:** grant_ack[i] = grant[i] && monbus_valid_in[i] **Hold Grant:** Arbiter holds grant[i] until grant_ack[i] asserts **Next Client:** After ACK, arbiter moves to next requesting client

This prevents: - Packet fragmentation (grant switching mid-transfer) - Lost packets (grant removed before client ready) - Unfair arbitration (clients getting multiple back-to-back grants)

48.7.2 Skid Buffer Depth Selection

Choose depth based on system characteristics:

2 Entries (Minimum): - Minimal buffering for timing closure only - Use when clients have low, consistent latency

4 Entries (Recommended): - Good balance of buffering and area - Handles typical backpressure scenarios - Recommended for most systems

6-8 Entries: - High buffering for very bursty traffic - Use when downstream has high variable latency - Higher area cost

48.7.3 Zero-Latency Configuration

For minimum latency:

```
.INPUT_SKID_ENABLE    (0),  
.OUTPUT_SKID_ENABLE  (0)
```

This provides direct combinational paths: - monbus_valid_in[grant_id] -> monbus_valid (combinational) - monbus_packet_in[grant_id] -> monbus_packet (combinational) - monbus_ready -> monbus_ready_in[grant_id] (combinational)

Use when: - Timing is not critical - Clients and downstream have matched latencies - Minimum latency required for event ordering

48.7.4 Assertions

The module includes comprehensive assertions:

Grant One-Hot: Verifies grant vector is one-hot when valid **Grant ID Consistency:** Verifies grant_id corresponds to asserted grant bit **Ready Exclusivity:** Verifies only granted client receives ready signal

48.8 Related Modules

48.8.1 Used By

- System-level monitor bus aggregation hierarchies
- Multi-subsystem monitoring infrastructures

48.8.2 Uses

- arbiter_round_robin.sv (ACK-mode arbiter)
- gaxi_skid_buffer.sv (optional input/output buffering)

48.8.3 Related Modules

- arbiter_monbus_common.sv (generates monitor bus packets)
 - arbiter_rr_pwm_monbus.sv (arbiter with integrated monitoring)
-

48.9 References

48.9.1 Specifications

- Internal: rtl/amba/PRD.md (AMBA subsystem requirements)
- Internal: docs/markdown/RTLAmba/includes/monitor_package_spec.md (monitor bus protocol)

48.9.2 Source Code

- RTL: rtl/amba/monitor/monbus_arbiter.sv
 - Tests: val/amba/test_monbus_arbiter.py (if exists)
-

Last Updated: 2025-10-24

48.10 Navigation

- [Back to Shared Infrastructure Index](#)

- [Back to RTLamba Index](#)

49 MonBus Group — AXI4 / AXI4

Module: monbus_axi4_axi4_group.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

49.1 Overview

The monbus_axi4_axi4_group module is the AXI4-slave-read + AXI4-master-write wrapper around monbus_group_core. It pairs a full AXI4 slave-read leaf (the error/interrupt drain port, supporting burst reads) with a full AXI4 master-write leaf (the bulk-capture port, emitting large write bursts). Both leaves are true AXI4, so their FUB interfaces pass straight through to the core's AXI4-shaped FUB ports; the wrapper only supplies safe constant defaults for the AXI4 sideband fields the core does not produce.

49.1.1 Key Features

- Full AXI4 on both the error-drain (slave read) and bulk-capture (master write) sides
 - Burst AR support on the slave-read drain (walk multiple error records per burst)
 - Large write bursts on the master-write side (MAX_BURST_BEATS up to 256)
 - Thin structural adapter — all logic lives in monbus_group_core
 - Configurable skid depths on all five AXI channels
 - Optional runtime compression (cfg_compress_en) with USE_COMPRESSION
 - Full per-protocol filter mask set (AXI / AXIS / CORE) passed to the core
-

49.2 Module Purpose

An SoC monitoring subsystem needs to expose captured monitor packets to two AXI4 consumers: a CPU that reads error records over a slave port, and a DMA-style write path that dumps a bulk trace to memory. This wrapper wires the two AXI4 leaf protocol adapters (axi4_slave_rd, axi4_master_wr) to the shared capture core, giving a drop-in AXI4/AXI4 capture block.

Use Cases: - AXI4-connected monitoring block where both the error-read and trace-write masters are AXI4 - High-throughput trace capture that benefits from long master-write bursts - CPU IRQ handler reading error records via AXI4 burst reads

Key Benefit: True AXI4 on both sides with zero protocol translation loss — the leaves pass through to the core, so burst geometry, IDs, and full-size bursts are available end to end.

49.3 Parameters

Parameter	Type	Default	Description
FIFO_DEPTH_ERR	int	64	Error FIFO depth in 192-bit records
FIFO_DEPTH_WRITE	int	96	Write FIFO depth in 64-bit beats
ADDR_WIDTH	int	32	Address width for both AXI4 ports
AXI_ID_WIDTH_S	int	8	Slave-read ID width
AXI_ID_WIDTH_M	int	8	Master-write ID width
AXI_USER_WIDTH	int	1	AXI user-signal width
MAX_BURST_BEATS	int	64	Max beats per master-write sub-burst
FLUSH_TIMEOUT_CYCLES	int	1024	Cycles before a timeout-triggered flush
NUM_PROTOCOLS	int	3	Informational only
USE_COMPRESSION	int	0	0 = raw 3-beat records; 1 = elaborate the compressor
SKID_DEPTH_AR	int	2	Slave-read AR channel skid depth
SKID_DEPTH_R	int	4	Slave-read R channel skid depth
SKID_DEPTH_AW	int	2	Master-write AW channel skid depth
SKID_DEPTH_W	int	4	Master-write W channel skid depth
SKID_DEPTH_B	int	2	Master-write B channel skid depth

49.4 Port Groups

49.4.1 Clock, Reset, Clear

Port	Direction	Width	Description
axi_aclk	input	1	Clock
axi_aresetn	input	1	Active-low asynchronous reset
cam_clear	input	1	Synchronous compressor CAM + stats clear

49.4.2 MonBus Ingress and Timestamp

Port	Direction	Width	Description
monbus_valid	input	1	Incoming packet valid
monbus_ready	output	1	Ready to accept
monbus_packet	input	monitor_packet_t (128)	Monitor packet
monbus_timestamp	input	monbus_timestamp_t (64)	Side-band timestamp
mon_time_out	output	monbus_timestamp_t (64)	Free-running counter for wrappers' i_mon_time

49.4.3 S-Side — AXI4 Slave Read (error record drain)

Port	Direction	Width	Description
s_axi_arid	input	AXI_ID_WIDTH_S	Read address ID
s_axi_araddr	input	ADDR_WIDTH	Read address
s_axi_arlen	input	8	Burst length (beats – 1)
s_axi_arsize	input	3	Burst size
s_axi_arburst	input	2	Burst type
s_axi_arlock	input	1	Lock

Port	Direction	Width	Description
s_axi_arcache	input	4	Cache attributes
s_axi_arprot	input	3	Protection
s_axi_arqos	input	4	QoS
s_axi_arregion	input	4	Region
s_axi_aruser	input	AXI_USER_WIDTH	User
s_axi_arvalid	input	1	AR valid
s_axi_arready	output	1	AR ready
s_axi_rid	output	AXI_ID_WIDTH	Read data ID
s_axi_rdata	output	64	Read data (sliced error record)
s_axi_rresp	output	2	Read response
s_axi_rlast	output	1	Read last
s_axi_ruser	output	AXI_USER_WIDTH	User (tied to 0)
s_axi_rvalid	output	1	R valid
s_axi_rready	input	1	R ready

49.4.4 M-Side — AXI4 Master Write (bulk trace capture)

Port	Direction	Width	Description
m_axi_awid	output	AXI_ID_WIDTH	Write address ID
m_axi_awaddr	output	ADDR_WIDTH	Write address
m_axi_awlen	output	8	Burst length (beats – 1)
m_axi_awsz	output	3	Burst size
m_axi_awburst	output	2	Burst type
m_axi_awlock	output	1	Lock
m_axi_awcache	output	4	Cache

Port	Direction	Width	Description
			attributes
m_axi_awprot	output	3	Protection
m_axi_awqos	output	4	QoS
m_axi_awregion	output	4	Region
m_axi_awuser	output	AXI_USER_WIDTH	User
m_axi_awvalid	output	1	AW valid
m_axi_awready	input	1	AW ready
m_axi_wdata	output	64	Write data
m_axi_wstrb	output	8	Write strobes
m_axi_wlast	output	1	Write last
m_axi_wuser	output	AXI_USER_WIDTH	User
m_axi_wvalid	output	1	W valid
m_axi_wready	input	1	W ready
m_axi_bid	input	AXI_ID_WIDTH_M	Write response ID
m_axi_bresp	input	2	Write response
m_axi_buser	input	AXI_USER_WIDTH	User (ignored)
m_axi_bvalid	input	1	B valid
m_axi_bready	output	1	B ready

49.4.5 Status and Egress Configuration

Port	Direction	Width	Description
irq_out	output	1	Error FIFO non-empty
cfg_base_addr	input	ADDR_WIDTH_H	Capture window base
cfg_limit_addr	input	ADDR_WIDTH_H	Capture window limit
cfg_flush_watermark	input	16	Flush-trigger beat count

Port	Direction	Width	Description
k			
cfg_compress_en	input	1	Runtime compression enable
cfg_axi_* / cfg_axis_* / cfg_core_*	input	16 each	Per-protocol filter masks (drop / err-select / per-event-code); see monbus_group_core
err_fifo_full	output	1	Error FIFO full
write_fifo_full	output	1	Write FIFO full
err_fifo_count	output	16	Error FIFO occupancy (records)
write_fifo_count	output	16	Write FIFO occupancy (beats)
mon_compressor_stats_*	output	32 each	Compressor statistics (0 in raw mode)

49.5 Functional Description

49.5.1 S-Side Error Drain (AXI4 slave read)

The `axi4_slave_rd` leaf terminates the incoming AXI4 read channel and presents a clean FUB read interface (`fub_axi_ar*` / `fub_axi_r*`) to the core. The core's slave-read slicer serves each 192-bit error record as three 64-bit beats (timestamp, packet-hi, packet-lo). Because the leaf is full AXI4, a single AR burst can request several records at once. The AXI4-only AR sideband fields (`arlock`/`arcache`/`arqos`/`arregion`/`aruser`) are consumed by the leaf and not forwarded to the core; `s_axi_ruser` is tied to 0.

49.5.2 M-Side Bulk Capture (AXI4 master write)

The core's master-write FUB drives the `axi4_master_wr` leaf, which emits the full AXI4 write channel. The wrapper hard-wires the AXI4 sideband fields the core does not generate to safe defaults at the leaf's FUB inputs: `awlock=0`, `awcache=4'b0010` (normal non-cacheable bufferable), `awprot=0`, `awqos=0`, `awregion=0`, `awuser=0`, `wuser=0`. `MAX_BURST_BEATS` defaults to 64, so a flush typically drains as one large sub-burst.

49.5.3 Shared Egress Configuration

All filtering, FIFO sizing, address-window, watermark, timeout, and compression behavior is set by the config ports and handed straight to `monbus_group_core`. The three per-protocol mask groups (`cfg_axi_*`, `cfg_axis_*`, `cfg_core_*`) select which packet types drop, which route to the error FIFO, and which event codes are masked — identical semantics across all four group wrappers.

49.6 Usage Example

```
monbus_axi4_axi4_group #(
    .ADDR_WIDTH      (32),
    .AXI_ID_WIDTH_S  (8),
    .AXI_ID_WIDTH_M  (8),
    .MAX_BURST_BEATS (64),
    .USE_COMPRESSION (0)
) u_mon_group (
    .axi_aclk      (aclk),
    .axi_aresetn   (aresetn),
    .cam_clear     (1'b0),

    .monbus_valid   (agg_valid),    // from monbus_arbiter
    .monbus_ready   (agg_ready),
    .monbus_packet   (agg_packet),
    .monbus_timestamp (agg_timestamp),
    .mon_time_out    (mon_time),

    // AXI4 slave-read drain -> CPU
    .s_axi_arid
    (cpu_arid), .s_axi_araddr(cpu_araddr), .s_axi_arlen(cpu_arlen),
    .s_axi_arvalid(cpu_arvalid), .s_axi_arready(cpu_arready),
    .s_axi_rid
    (cpu_rid), .s_axi_rdata(cpu_rdata), .s_axi_rlast(cpu_rlast),
    .s_axi_rvalid(cpu_rvalid), .s_axi_rready(cpu_rready),
    // ... AR sideband ...

    // AXI4 master-write bulk capture -> memory
    .m_axi_awaddr(cap_awaddr), .m_axi_awlen(cap_awlen), .m_axi_awvalid
    (cap_awvalid),
    .m_axi_awready(cap_awready), .m_axi_wdata(cap_wdata), .m_axi_wlast
    (cap_wlast),
    .m_axi_wvalid(cap_wvalid), .m_axi_wready(cap_wready),
    .m_axi_bvalid(cap_bvalid), .m_axi_bready(cap_bready),
    // ... AW sideband ...
```

```
.irq_out          (mon_irq),
.cfg_base_addr    (cap_base),
.cfg_limit_addr   (cap_limit),
.cfg_flush_watermark (16'd48),
.cfg_compress_en  (1'b0)
// ... per-protocol masks ...
);
```

49.7 Design Notes

49.7.1 Both Sides Are Full AXI4

Unlike the AXIL variants, neither leaf forces MAX_BURST_BEATS=1 and neither injects single-beat defaults into the core — the AXI4 IDs and burst lengths flow through. This is the highest-throughput member of the group family.

49.7.2 AW Sideband Defaults Live in the Wrapper

The core deliberately does not model awlock/awcache/awqos/awregion/awuser. The wrapper supplies conservative constants at the master-write leaf so downstream interconnect sees a well-formed AXI4 burst.

49.7.3 Skid Depths Are Tunable

The five SKID_DEPTH_* parameters size the per-channel skid buffers in the leaves; increase them to absorb more backpressure on congested interconnect.

49.8 Related Modules

49.8.1 Used By

- SoC monitoring subsystems requiring an AXI4/AXI4 MonBus capture block

49.8.2 Uses

- **monbus_group_core.sv** — protocol-agnostic capture core (all logic)
- **axi4_slave_rd.sv** — AXI4 slave-read leaf (error drain)
- **axi4_master_wr.sv** — AXI4 master-write leaf (bulk capture)
- **monitor_common_pkg** — packet and timestamp types

49.8.3 See Also

- **monbus_axi4_axil_group.sv** — AXI4 read + AXIL write variant

- **monbus_axil_axi4_group.sv** — AXIL read + AXI4 write variant
 - **monbus_axil_axil_group.sv** — AXIL read + AXIL write variant
 - **monbus_arbiter.sv** — upstream aggregation
-

49.9 References

49.9.1 Source Code

- RTL: rtl/amba/monitor/monbus_axi4_axi4_group.sv
- Core: rtl/amba/monitor/monbus_group_core.sv

49.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - Group Core: docs/markdown/RTLAmba/monbus_group_core.md
 - Packet Format:
docs/markdown/RTLAmba/includes/monitor_package_spec.md
-

Last Updated: 2026-07-15

49.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

50 MonBus Group — AXI4 / AXIL

Module: monbus_axi4_axil_group.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

50.1 Overview

The `monbus_axi4_axil_group` module is the AXI4-slave-read + AXIL-master-write wrapper around `monbus_group_core`. It pairs a full AXI4 slave-read leaf (the burst-capable error/interrupt drain port) with a single-beat AXI4-Lite master-write leaf (the bulk-capture port). The slave-read FUB passes through to the core's AXI4-shaped read FUB, while the master-write side forces `MAX_BURST_BEATS=1` so the core emits one beat per AW — matching AXIL's single-beat semantics.

50.1.1 Key Features

- AXI4 burst reads on the error-drain side (walk multiple records per burst)
 - AXIL single-beat writes on the bulk-capture side (MAX_BURST_BEATS=1)
 - Thin structural adapter — all logic lives in monbus_group_core
 - AXI4-only AR sideband fields dropped at the wrapper boundary
 - Master-write ID forced to width 1 (AXIL has no ID)
 - Optional runtime compression (cfg_compress_en) with USE_COMPRESSION
 - Full per-protocol filter mask set (AXI / AXIS / CORE) passed to the core
-

50.2 Module Purpose

Some systems read error records over a high-throughput AXI4 port but write the bulk trace to a simple AXI4-Lite peripheral bus (a register-file-style capture sink, a low-cost bridge, or a control-plane fabric). This wrapper mixes the two: an axi4_slave_rd leaf for burst error reads and an axil4_master_wr leaf for single-beat trace writes, both driven by the shared capture core.

Use Cases: - Trace capture to an AXIL-only memory-mapped sink while reading errors over AXI4 - Control-plane monitoring where the write path is a lightweight AXIL bus - Mixed-fabric SoCs pairing an AXI4 CPU read port with an AXIL capture port

Key Benefit: Reuses the exact same capture core as the all-AXI4 variant — only MAX_BURST_BEATS and the master-write leaf differ — so behavior and configuration stay identical across the family, with the write side degrading gracefully to single beats.

50.3 Parameters

Parameter	Type	Default	Description
FIFO_DEPTH_ERR	int	64	Error FIFO depth in 192-bit records
FIFO_DEPTH_WRITE	int	96	Write FIFO depth in 64-bit beats
ADDR_WIDTH	int	32	Address width for both ports
AXI_ID_WIDTH	int	8	Slave-read ID width
AXI_USER_WIDTH	int	1	AXI user-signal width (slave-read side)

Parameter	Type	Default	Description
FLUSH_TIMEOUT_CYCLES	int	1024	Cycles before a timeout-triggered flush
NUM_PROTOCOLS	int	3	Informational only
USE_COMPRESSION	int	0	0 = raw 3-beat records; 1 = elaborate the compressor
SKID_DEPTH_AR	int	2	Slave-read AR channel skid depth
SKID_DEPTH_R	int	4	Slave-read R channel skid depth
SKID_DEPTH_AW	int	2	AXIL master-write AW channel skid depth
SKID_DEPTH_W	int	2	AXIL master-write W channel skid depth
SKID_DEPTH_B	int	2	AXIL master-write B channel skid depth

Internally, the core is instantiated with AXI_ID_WIDTH_M=1, AXI_ID_WIDTH_S=AXI_ID_WIDTH, and MAX_BURST_BEATS=1.

50.4 Port Groups

50.4.1 Clock, Reset, Clear

Port	Direction	Width	Description
axi_aclk	input	1	Clock
axi_aresetn	input	1	Active-low asynchronous reset
cam_clear	input	1	Synchronous compressor CAM + stats clear

50.4.2 MonBus Ingress and Timestamp

Port	Direction	Width	Description
monbus_valid	input	1	Incoming packet valid
monbus_ready	output	1	Ready to accept
monbus_packet	input	monitor_packet_t (128)	Monitor packet
monbus_timestamp	input	monbus_timestamp_t (64)	Side-band timestamp
mon_time_out	output	monbus_timestamp_t (64)	Free-running counter for wrappers' i_mon_time

50.4.3 S-Side — AXI4 Slave Read (error record drain)

Port	Direction	Width	Description
s_axi_arid	input	AXI_ID_WIDTH	Read address ID
s_axi_araddr	input	ADDR_WIDTH	Read address
s_axi_arlen	input	8	Burst length (beats – 1)
s_axi_arsize	input	3	Burst size
s_axi_arburst	input	2	Burst type
s_axi_arlock	input	1	Lock
s_axi_arcache	input	4	Cache attributes
s_axi_arprot	input	3	Protection
s_axi_arqos	input	4	QoS
s_axi_arregion	input	4	Region
s_axi_aruser	input	AXI_USER_WIDTH	User
s_axi_arvalid	input	1	AR valid
s_axi_arready	output	1	AR ready
s_axi_rid	output	AXI_ID_WIDTH	Read data ID
s_axi_rdata	output	64	Read data

Port	Direction	Width	Description
			(sliced error record)
s_axi_rresp	output	2	Read response
s_axi_rlast	output	1	Read last
s_axi_ruser	output	AXI_USER_WID TH	User (tied to 0)
s_axi_rvalid	output	1	R valid
s_axi_rready	input	1	R ready

50.4.4 M-Side — AXIL Master Write (bulk trace capture)

Port	Direction	Width	Description
m_axil_awvalid	output	1	AW valid
m_axil_awready	input	1	AW ready
m_axil_awaddr	output	ADDR_WIDTH	Write address
m_axil_awprot	output	3	Protection
m_axil_wvalid	output	1	W valid
m_axil_wready	input	1	W ready
m_axil_wdata	output	64	Write data
m_axil_wstrb	output	8	Write strobes
m_axil_bvalid	input	1	B valid
m_axil_bready	output	1	B ready
m_axil_bresp	input	2	Write response

50.4.5 Status and Egress Configuration

Port	Direction	Width	Description
irq_out	output	1	Error FIFO non-empty
cfg_base_addr	input	ADDR_WIDT H	Capture window base
cfg_limit_addr	input	ADDR_WIDT H	Capture window limit
cfg_flush_watermar	input	16	Flush-trigger beat count

Port	Direction	Width	Description
k			
cfg_compress_en	input	1	Runtime compression enable
cfg_axi_* / cfg_axis_* / cfg_core_*	input	16 each	Per-protocol filter masks (drop / err-select / per-event-code); see monbus_group_core
err_fifo_full	output	1	Error FIFO full
write_fifo_full	output	1	Write FIFO full
err_fifo_count	output	16	Error FIFO occupancy (records)
write_fifo_count	output	16	Write FIFO occupancy (beats)
mon_compressor_stats_*	output	32 each	Compressor statistics (0 in raw mode)

50.5 Functional Description

50.5.1 S-Side Error Drain (AXI4 slave read)

Identical to the AXI4/AXI4 variant on the read side: the `axi4_slave_rd` leaf terminates the AXI4 read channel and hands a clean read FUB to the core, whose slicer serves each 192-bit error record as three 64-bit beats. Full AXI4 bursts can request multiple records at once. The AXI4-only AR sideband fields (`arlock/arcache/arqos/arregion/aruser`) are dropped at the wrapper boundary; `s_axi_ruser` is tied to 0.

50.5.2 M-Side Bulk Capture (AXIL master write)

The core is parameterized with `MAX_BURST_BEATS=1`, so its master-write burst writer emits one AW per beat — the single-beat pattern AXIL requires. The core's AXI4-shaped master-write FUB is bridged to the `axil4_master_wr` leaf: `awaddr, awvalid/awready, wdata, wstrb, wvalid/wready, and bresp/bvalid/bready` connect through, while the AXI4-only FUB outputs the core still drives (`awid, awlen, awsize, awburst, wlast`) are captured in unused nets and discarded. The AXIL leaf drives `awprot=0`, and the core's `bid` input is tied to 0.

50.5.3 Shared Egress Configuration

Filtering, FIFO sizing, address-window, watermark, timeout, and compression are all handled by `monbus_group_core` via the same config ports as every other group wrapper. The three per-protocol mask groups (`cfg_axi_*`, `cfg_axis_*`, `cfg_core_*`) behave identically here.

50.6 Usage Example

```
monbus_axi4_axil_group #(
    .ADDR_WIDTH      (32),
    .AXI_ID_WIDTH    (8),
    .USE_COMPRESSION (0)
) u_mon_group (
    .axi_aclk      (aclk),
    .axi_aresetn   (aresetn),
    .cam_clear     (1'b0),

    .monbus_valid   (agg_valid),
    .monbus_ready   (agg_ready),
    .monbus_packet   (agg_packet),
    .monbus_timestamp (agg_timestamp),
    .mon_time_out    (mon_time),

    // AXI4 slave-read drain -> CPU
    .s_axi_arid(cpu_arid), .s_axi_araddr(cpu_araddr), .s_axi_arlen(cpu
_arlen),
    .s_axi_arvalid(cpu_arvalid), .s_axi_arready(cpu_arready),
    .s_axi_rdata(cpu_rdata), .s_axi_rlast(cpu_rlast),
    .s_axi_rvalid(cpu_rvalid), .s_axi_rready(cpu_rready),
    // ... AR sideband ...

    // AXIL master-write bulk capture -> peripheral sink
    .m_axil_awaddr(cap_awaddr), .m_axil_awvalid(cap_awvalid), .m_axil_
awready(cap_awready),
    .m_axil_wdata(cap_wdata), .m_axil_wstrb(cap_wstrb),
    .m_axil_wvalid(cap_wvalid), .m_axil_wready(cap_wready),
    .m_axil_bvalid(cap_bvalid), .m_axil_bready(cap_bready), .m_axil_br
esp(cap_bresp),

    .irq_out          (mon_irq),
    .cfg_base_addr     (cap_base),
    .cfg_limit_addr    (cap_limit),
    .cfg_flush_watermark (16'd48),
    .cfg_compress_en    (1'b0)
```

```
// ... per-protocol masks ...  
);
```

50.7 Design Notes

50.7.1 AXIL Forces Single-Beat Writes

The only functional difference from the AXI4/AXI4 variant is MAX_BURST_BEATS=1 and the AXIL master-write leaf. Every flush emits one beat per AW handshake, so throughput on the capture side is lower than the AXI4/AXI4 group — appropriate for a lightweight AXIL sink.

50.7.2 Core Still Drives AXI4 FUB Fields

Because the core is AXI4-shaped internally, it always drives awlen/awsize/awburst/awid/wlast. On the AXIL side these are simply not routed to the leaf (captured in _unused nets). This is intentional and keeps the core single-source.

50.7.3 Read Side Is Unchanged

The error-drain path is byte-for-byte the same as the AXI4/AXI4 group — the record slicer, burst AR handling, and IRQ behavior are all in the shared core.

50.8 Related Modules

50.8.1 Used By

- SoC monitoring subsystems reading errors over AXI4 but capturing the trace over AXIL

50.8.2 Uses

- **monbus_group_core.sv** — protocol-agnostic capture core (all logic), MAX_BURST_BEATS=1
- **axi4_slave_rd.sv** — AXI4 slave-read leaf (error drain)
- **axil4_master_wr.sv** — AXIL master-write leaf (bulk capture)
- **monitor_common_pkg** — packet and timestamp types

50.8.3 See Also

- **monbus_axi4_axi4_group.sv** — AXI4 read + AXI4 write variant
- **monbus_axil_axi4_group.sv** — AXIL read + AXI4 write variant
- **monbus_axil_axil_group.sv** — AXIL read + AXIL write variant

- **monbus_arbiter.sv** — upstream aggregation
-

50.9 References

50.9.1 Source Code

- RTL: rtl/amba/monitor/monbus_axi4_axil_group.sv
- Core: rtl/amba/monitor/monbus_group_core.sv

50.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - Group Core: docs/markdown/RTLAmba/monbus_group_core.md
 - Packet Format:
docs/markdown/RTLAmba/includes/monitor_package_spec.md
-

Last Updated: 2026-07-15

50.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

51 Monitor Bus Group — AXIL Slave-Read / AXI4 Master-Write

Module: monbus_axil_axi4_group.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

51.1 Overview

monbus_axil_axi4_group is the mixed-fabric wrapper of the monitor-bus delivery family. It wraps the protocol-agnostic monbus_group_core with an **AXIL slave-read err-drain port** (CPU IRQ handler pops error records) and a **full AXI4 burst master-write port** (streams captured records into a memory ring, bunching multiple records into multi-beat bursts to amortize address-channel overhead). It is the delivery block stream_char / rapids_char instantiate for MonBus egress when the drain fabric is AXIL but the memory ring lives behind an AXI4 fabric.

51.1.1 Key Features

- AXIL slave-read err-FIFO drain (CPU-facing IRQ path)
 - AXI4 burst master-write bulk-capture (MAX_BURST_BEATS up to 256, default 64)
 - Selectable err-drain data width: 64-bit (one beat per record slice) or 32-bit (2:1 read serializer for a 32-bit host crossbar)
 - Shared per-protocol egress packet filter (AXI / AXIS / CORE)
 - `irq_out` asserted whenever the err FIFO is non-empty
 - Optional runtime compression (USE_COMPRESSION + `cfg_compress_en`)
 - Compressor statistics and FIFO status/count outputs
-

51.2 Module Purpose

Like the AXIL/AXIL variant, this wrapper splits the arbitrated monitor-bus stream into a CPU-facing error drain and a memory-ring bulk dump. The difference is the dump side: a full AXI4 burst master lets the burst writer emit many records in a single $AW + N \times W + B$, which is throughput-optimal when the ring sits behind an AXI4 fabric.

The core's FUBs are AXI4-shaped internally. On the read side the wrapper bridges the AXIL leaf into the core's AXI4-shaped read FUB (`arid=0` / `arlen=0` / `arsize=3` / `arburst=INCR`). On the write side the core drives most AXI4 fields directly; the fields the core does not produce (`awlock` / `awcache` / `awqos` / `awregion` / `awuser` / `wuser`) are tied to safe defaults at the AXI4 leaf boundary.

Use Cases: - MonBus egress for `stream_char` / `rapids_char` with an AXIL drain and an AXI4 ring fabric - CPU IRQ error drain plus high-throughput burst dump into memory - Trace capture where amortizing AXI4 address-channel overhead matters

Key Benefit: A lightweight AXIL CPU drain paired with a full AXI4 burst dump, so trace records land in memory with minimal address-channel overhead.

51.3 Parameters

Parameter	Type	Default	Description
FIFO_DEPTH_ERR	int	64	Error FIFO depth, in records
FIFO_DEPTH_WRITE	int	96	Write FIFO depth, in beats (96 beats = 32 raw

Parameter	Type	Default	Description
			records)
ADDR_WIDTH	int	32	Address width on the slave-read and master-write ports
S_AXIL_DATA_WIDTH	int	64	Err-drain read data width. 64 = one beat per 64-bit record slice; 32 = a 2:1 read serializer presents each slice as a low then high beat (6 beats/record) for a 32-bit host crossbar
AXI_ID_WIDTH	int	8	Master-write AXI4 ID width
AXI_USER_WIDTH	int	1	Master-write AXI4 USER width
MAX_BURST_BEATS	int	64	Maximum beats per master-write AW (AXI4 protocol max is 256)
FLUSH_TIMEOUT_CYCLES	int	1024	Cycles since last accepted W handshake before a timeout-driven flush
NUM_PROTOCOLS	int	3	Informational — AXI / AXIS / CORE filter configs are unconditional
USE_COMPRESSION	int	0	Elaboration: 0 omits the compressor (raw-only); 1 elaborates it for runtime selection via <code>cfg_compress_en</code>
SKID_DEPTH_AR	int	2	Slave-read AR skid depth
SKID_DEPTH_R	int	4	Slave-read R skid depth
SKID_DEPTH_AW	int	2	Master-write AW skid depth

Parameter	Type	Default	Description
SKID_DEPTH_W	int	4	Master-write W skid depth (deeper to absorb burst W)
SKID_DEPTH_B	int	2	Master-write B skid depth

51.4 Port Groups

51.4.1 Clock, Reset, and CAM Clear

Port	Direction	Width	Description
axi_aclk	input	1	Clock
axi_aresetn	input	1	Active-low asynchronous reset
cam_clear	input	1	Synchronous clear of the compressor CAM + stat counters (tied off in raw-only builds)

51.4.2 Monitor-Bus Input + Timestamp

Port	Direction	Width	Description
monbus_valid	input	1	Monitor-bus packet valid
monbus_ready	output	1	Monitor-bus ready (backpressure to the arbiter)
monbus_packet	input	monitor_packet_t	128-bit monitor packet
monbus_timestamp	input	monbus_timestamp_t	64-bit source timestamp
mon_time_out	output	monbus_timestamp_t	Free-running 64-bit timestamp counter

51.4.3 AXIL Slave Read — Err-FIFO Drain

Port	Direction	Width	Description
s_axil_arvalid	input	1	Read-address valid
s_axil_arready	output	1	Read-address ready
s_axil_araddr	input	ADDR_WIDTH	Read address
s_axil_arprot	input	3	Read protection attributes
s_axil_rvalid	output	1	Read-data valid
s_axil_rready	input	1	Read-data ready
s_axil_rdata	output	S_AXIL_DATA_WIDTH	Read data (record slice, or half-slice in 32-bit mode)
s_axil_rresp	output	2	Read response

51.4.4 AXI4 Master Write — Burst Bulk Capture

Port	Direction	Width	Description
m_axi_awid	output	AXI_ID_WIDTH	Write-address ID
m_axi_awaddr	output	ADDR_WIDTH	Write burst base address (within the ring window)
m_axi_awlen	output	8	Burst length minus 1 (sub_burst_beats - 1)
m_axi_awsiz	output	3	Beat size (fixed at 3 = 8 bytes)
m_axi_awburst	output	2	Burst type (fixed INCR)
m_axi_awlock	output	1	Lock (tied 0 at the leaf)
m_axi_awcache	output	4	Cache (tied 4'b0010)

Port	Direction	Width	Description
ache			Normal Non-cacheable Bufferable)
m_axi_awp rot	output	3	Protection (tied 0)
m_axi_awq os	output	4	QoS (tied 0)
m_axi_awr egion	output	4	Region (tied 0)
m_axi_awu ser	output	AXI_USER_W IDTH	User (tied 0)
m_axi_awv alid	output	1	Write-address valid
m_axi_awr eady	input	1	Write-address ready
m_axi_wda ta	output	64	Write data (one 64-bit beat)
m_axi_wst rb	output	8	Write strobe
m_axi_wla st	output	1	Last beat of the sub-burst
m_axi_wus er	output	AXI_USER_W IDTH	Write-data user (tied 0)
m_axi_wva lid	output	1	Write-data valid
m_axi_wre ady	input	1	Write-data ready
m_axi_bid	input	AXI_ID_WID TH	Write-response ID
m_axi_bres p	input	2	Write response
m_axi_bus er	input	AXI_USER_W IDTH	Write-response user (unused)
m_axi_bval	input	1	Write-response valid

Port	Direction	Width	Description
id			
m_axi_bready	output	1	Write-response ready

51.4.5 Interrupt

Port	Direction	Width	Description
irq_out	output	1	Asserted whenever the err FIFO is non-empty

51.4.6 Configuration

Port	Direction	Width	Description
cfg_base_addr	input	ADDR_WIDT H	Master-write ring base address
cfg_limit_addr	input	ADDR_WIDT H	Master-write ring limit address (writer wraps, does not saturate)
cfg_flush_watermark	input	16	Write-FIFO depth (beats) at which a flush fires
cfg_compression_en	input	1	Runtime compression enable (effective only when USE_COMPRESSION=1)

51.4.7 Per-Protocol Filter Masks

Three protocols (AXI, AXIS, CORE), each with a packet-type mask, an err-select mask, and per-event-category masks. All are 16-bit inputs — same shape as the AXIL/AXIL variant.

Port	Protocol	Role
cfg_axi_pkt_mask	AXI	Drop by packet type
cfg_axi_err_select	AXI	Route to err FIFO by packet type
cfg_axi_error_mask / cfg_axi_timeout_ma	AXI	Per-event-code drop masks

Port	Protocol	Role
sk / cfg_axi_compl_mask / cfg_axi_thresh_mask / cfg_axi_perf_mask / cfg_axi_addr_mask / cfg_axi_debug_mask		
cfg_axis_pkt_mask	AXIS	Drop by packet type
cfg_axis_err_select	AXIS	Route to err FIFO by packet type
cfg_axis_error_mask / cfg_axis_timeout_mask / cfg_axis_compl_mask / cfg_axis_credit_mask / cfg_axis_channel_mask / cfg_axis_stream_mask	AXIS	Per-event-code drop masks
cfg_core_pkt_mask	CORE	Drop by packet type
cfg_core_err_select	CORE	Route to err FIFO by packet type
cfg_core_error_mask / cfg_core_timeout_mask / cfg_core_compl_mask / cfg_core_thresh_mask / cfg_core_perf_mask	CORE	Per-event-code drop masks

Port	Protocol	Role
/ cfg_core_debug_ma sk		

51.4.8 Status / Debug

Port	Direction	Width	Description
err_fifo_full	output	1	Err FIFO write port not ready
write_fifo_full	output	1	Write FIFO write port not ready
err_fifo_count	output	16	Err FIFO entry count (records)
write_fifo_count	output	16	Write FIFO entry count (beats)
mon_compressor_stats_tier1_a / _tier1_b / _tier1_c / _tier0 / _cam_miss / _delta_ts_ovf / _event_data_ovf / _ed_delta_ovf	output	32 each	Compressor statistics (live only when USE_COMPRESSION=1)

51.5 Functional Description

51.5.1 S-Side Err-Drain Read Protocol

Each monbus err record is three 64-bit slices — {tag, source_ts}, packet[127:64], packet[63:0]. A 64-bit axil4_slave_rd leaf bridges the external AXIL read onto the core's 64-

bit read FUB, and the core's slice counter returns the three slices in sequence; the err-FIFO record is popped only after the packet-low slice is read.

- **64-bit drain (`S_AXIL_DATA_WIDTH == 64`):** the leaf is wired straight through — 3 beats/record.
- **32-bit drain (`S_AXIL_DATA_WIDTH == 32`):** a phase bit splits each 64-bit leaf beat into a low then high 32-bit external read — 6 beats/record. The AR is forwarded to the leaf only on the low phase (one core read per pair, no prefetch); the low read consumes the beat and latches its high half; the high read replays the latch, gating R on its own accepted AR (`r_hi_ar`) to honor AXIL AR-before-R ordering. This is the identical mechanism used in `monbus_axil_axil_group`.

51.5.2 M-Side Bulk-Capture Protocol (AXI4 Burst)

Surviving (non-err) packets go to the write FIFO and are streamed out the AXI4 master-write port by the core's burst writer. Because this is a full AXI4 master, one drain cycle typically becomes one large sub-burst — e.g. 24 beats = 8 raw records in a single $AW + 24 \times W + B$ — capped per AW by `MAX_BURST_BEATS`. `awsize` is fixed at 3 (8 bytes/beat), `awburst` at `INCR`, `awlen` = `sub_burst_beats` - 1, and `wlast` asserts on the final beat of each sub-burst. The address is **circular** within [`cfg_base_addr`, `cfg_limit_addr`] (wraps, does not saturate). The writer fires on watermark or timeout. See `monbus_group` for the full burst-writer geometry (3-stage pipeline, mod-3 rounding, 4KB-boundary respect, rewind-snap).

The core drives `awid` / `awaddr` / `awlen` / `awsize` / `awburst` and `wdata` / `wstrb` / `wlast` / `wvalid`. The AXI4 fields the core does not produce are tied to safe defaults at the leaf: `awlock=0`, `awcache=4'b0010` (Normal Non-cacheable Bufferable), `awprot=0`, `awqos=0`, `awregion=0`, `awuser=0`, `wuser=0`.

51.5.3 Interrupt

`irq_out` is asserted by the core whenever the err FIFO is non-empty.

51.5.4 Shared Egress Packet Filter

The per-protocol filter (AXI, AXIS, CORE) decides for each packet: drop, route to the err FIFO (`err_select`), or route to the write FIFO (default). Protocols not in the supported set (APB, ARB) are always dropped. The filter config is identical across all four family wrappers — see `monbus_group` for the per-event category tables and masking rules.

51.5.5 Compression

When `USE_COMPRESSION=1`, `monbus_compressor` is selectable at runtime via `cfg_compress_en`, emitting one 64-bit self-tagged slot per record instead of three raw beats; `cam_clear` synchronously empties the template CAM and zeroes stats between runs. In raw-only builds these fold away. (Unlike the AXIL/AXIL variant, this wrapper does not expose `HALF_BEAT_EN`.)

51.6 Usage Example

```
monbus_axil_axi4_group #(
    .FIFO_DEPTH_ERR      (64),
    .FIFO_DEPTH_WRITE    (96),    // beats (3x for raw mode)
    .ADDR_WIDTH          (32),
    .S_AXIL_DATA_WIDTH   (64),    // or 32 for a narrow host crossbar
    .AXI_ID_WIDTH        (8),
    .MAX_BURST_BEATS     (64),
    .FLUSH_TIMEOUT_CYCLES (1024),
    .USE_COMPRESSION     (0)
) u_monbus_egress (
    .axi_aclk      (aclk),
    .axi_aresetn   (aresetn),
    .cam_clear     (1'b0),

    // Arbitrated monitor-bus input
    .monbus_valid   (mon_valid),
    .monbus_ready   (mon_ready),
    .monbus_packet  (mon_packet),
    .monbus_timestamp (mon_timestamp),
    .mon_time_out   (mon_time),

    // CPU-facing AXIL err drain
    .s_axil_arvalid (cpu_arvalid),
    .s_axil_arready (cpu_arready),
    .s_axil_araddr  (cpu_araddr),
    .s_axil_arprot  (cpu_arprot),
    .s_axil_rvalid  (cpu_rvalid),
    .s_axil_rready  (cpu_rready),
    .s_axil_rdata   (cpu_rdata),
    .s_axil_rresp   (cpu_rresp),

    // AXI4 burst memory-ring dump
    .m_axi_awid     (ring_awid),
    .m_axi_awaddr   (ring_awaddr),
    .m_axi_awlen    (ring_awlen),
    /* ... remaining m_axi_aw*, m_axi_w*, m_axi_b* ... */

    .irq_out        (monbus_irq),
    .cfg_base_addr  (RING_BASE),
    .cfg_limit_addr (RING_LIMIT),
    .cfg_flush_watermark (16'd24),
    .cfg_compress_en (1'b0),
```

```

// Per-protocol filter masks (AXI / AXIS / CORE) ...
.cfg_axi_pkt_mask      (axi_pkt_mask),
.cfg_axi_err_select    (axi_err_select),
/* ... remaining masks ... */

.err_fifo_count        (err_count),
.write_fifo_count      (wr_count)
/* ... compressor stats ... */
);

```

51.7 Design Notes

51.7.1 When to Pick This Over AXIL/AXIL

Choose this wrapper when the memory ring lives behind an AXI4 fabric and you want to bunch records into multi-beat bursts to amortize address-channel overhead. The memory image is identical to the AXIL-master case; only the address-channel handshake count differs. Pick MAX_BURST_BEATS to taste (up to 256).

51.7.2 AXI4 Leaf Default Tie-Offs

The core only produces the AXI4 fields it needs. The remaining AXI4 qualifier fields (awlock / awcache / awqos / awregion / awuser / wuser) are tied at the axi4_master_wr leaf boundary, with awcache set to Normal Non-cacheable Bufferable — appropriate for a trace-ring write that must reach memory.

51.7.3 Why One Core + Four Wrappers

The filter, FIFOs, burst writer, and drain live once in monbus_group_core; the AXIL read leaf, the AXI4 write leaf, and the two FUB bridges live here. Each family wrapper carries only the exact port shape its fabric demands.

51.8 Related Modules

51.8.1 Used By

- stream_char / rapids_char MonBus egress (AXIL drain + AXI4 ring)
- Trace-capture topologies needing high-throughput burst dump into AXI4 memory

51.8.2 Uses

- **monbus_group_core.sv** — Protocol-agnostic filter + FIFO + burst-writer + drain
- **axil4_slave_rd.sv** — Slave-read (err-drain) skid leaf
- **axi4_master_wr.sv** — Master-write (burst bulk-capture) skid leaf
- **monbus_compressor.sv** — Optional runtime compressor (USE_COMPRESSION=1)
- **monitor_common_pkg** — monitor_packet_t / monbus_timestamp_t types
- **reset_defs.svh** — Reset macros

51.8.3 See Also

- **monbus_axil_axil_group.sv** — Same drain, AXIL single-beat master-write
 - **monbus_axi4_axil_group.sv** / **monbus_axi4_axi4_group.sv** — AXI4-burst slave-read variants
 - **monbus_arbiter.sv** — Upstream multi-source merge (instantiate before this wrapper for N>1 sources)
 - **axi4_dma_observer.sv** — Instantiates this wrapper as its central filter + err FIFO + dump writer
-

51.9 References

51.9.1 Source Code

- RTL: rtl/amba/monitor/monbus_axil_axi4_group.sv
- Core: rtl/amba/monitor/monbus_group_core.sv
- Tests: val/amba/test_monbus_axil_axi4_group.py

51.9.2 Documentation

- Family spec: docs/markdown/RTLamba/monbus_group.md
 - Architecture: docs/markdown/RTLamba/shared/README.md
 - Packet format:
docs/markdown/RTLamba/includes/monitor_package_spec.md
-

Last Updated: 2026-07-15

51.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

52 Monitor Bus Group — AXIL Slave-Read / AXIL Master-Write

Module: `monbus_axil_axil_group.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

52.1 Overview

`monbus_axil_axil_group` is the all-AXI4-Lite wrapper of the monitor-bus delivery family. It wraps the protocol-agnostic `monbus_group_core` with an **AXIL slave-read err-drain port** (for the CPU IRQ handler to pop error records) and an **AXIL single-beat master-write port** (to stream captured records into a memory ring). It is the delivery block that `stream_char / rapids_char` instantiate for MonBus egress when both the drain and dump fabrics are AXIL.

This wrapper replaces the legacy `monbus_axil_group.sv`, which fused the core and the AXIL skids into one module.

52.1.1 Key Features

- AXIL slave-read err-FIFO drain (CPU-facing IRQ path)
 - AXIL single-beat master-write bulk-capture (memory-ring egress)
 - Selectable err-drain data width: 64-bit (one beat per record slice) or 32-bit (2:1 read serializer for a 32-bit host crossbar)
 - Shared per-protocol egress packet filter (AXI / AXIS / CORE)
 - `irq_out` asserted whenever the err FIFO is non-empty
 - Optional runtime compression (`USE_COMPRESSION + cfg_compress_en`) with half-beat packing (`HALF_BEAT_EN`)
 - Compressor statistics and FIFO status/count outputs
-

52.2 Module Purpose

The monitor bus produces a single arbitrated stream of 128-bit packets plus a 64-bit timestamp. That stream has to be split into two destinations: error/interrupt records the CPU polls, and bulk trace records written into a memory ring for later offline analysis. `monbus_group_core` does the filtering, FIFOing, watermark/timeout burst-writing, and slave-read drain. This wrapper gives the

core an exact AXIL port shape on both the CPU-facing (slave-read) and memory-facing (master-write) sides, so the caller ties off no spurious AXI4-only fields.

The core's FUBs are AXI4-shaped internally; the wrapper bridges the AXIL leaves to those FUBs (supplying `arid=0` / `arlen=0` / `arsize=3` / `arburst=INCR` on the read side and forcing `MAX_BURST_BEATS=1` on the write side).

Use Cases: - MonBus egress for `stream_char` / `rapids_char` on an AXIL fabric - CPU IRQ-driven error drain with a memory-mapped ring dump - Minimal-footprint monitor delivery where both fabrics are AXIL - 32-bit host crossbar drain via the built-in 2:1 read serializer

Key Benefit: Exact AXIL port shape on both sides with no fake AXI4 fields, plus a built-in 32-bit err-drain serializer for narrow host buses.

52.3 Parameters

Parameter	Type	Default	Description
FIFO_DEPTH_ERR	int	64	Error FIFO depth, in records
FIFO_DEPTH_WRITE	int	96	Write FIFO depth, in beats (96 beats = 32 raw records)
ADDR_WIDTH	int	32	Address width on the slave-read and master-write ports
S_AXIL_DATA_WIDTH	int	64	Err-drain read data width. 64 = one beat per 64-bit record slice (3 beats/record); 32 = a 2:1 read serializer presents each slice as a low then high beat (6 beats/record) for a 32-bit host crossbar
FLUSH_TIMEOUT_CYCLES	int	1024	Cycles since last accepted W handshake before a timeout-driven flush
NUM_PROTOCOLS	int	3	Informational — AXI / AXIS / CORE filter configs

Parameter	Type	Default	Description
USE_COMPRESSION	int	0	are unconditional Elaboration: 0 omits the compressor (raw-only); 1 elaborates it for runtime selection via <code>cfg_compress_en</code>
HALF_BEAT_EN	int	0	Elaboration: pack two 30-bit half-slots per 64-bit beat downstream of the compressor (requires <code>USE_COMPRESSION=1</code>)
SKID_DEPTH_AR	int	2	Slave-read AR skid depth
SKID_DEPTH_R	int	4	Slave-read R skid depth
SKID_DEPTH_AW	int	2	Master-write AW skid depth
SKID_DEPTH_W	int	2	Master-write W skid depth
SKID_DEPTH_B	int	2	Master-write B skid depth

The AXIL master forces `MAX_BURST_BEATS=1` in the core, so this wrapper has no `MAX_BURST_BEATS` parameter (unlike the AXI4-master variant).

52.4 Port Groups

52.4.1 Clock, Reset, and CAM Clear

Port	Direction	Width	Description
<code>axi_aclk</code>	input	1	Clock
<code>axi_areset_n</code>	input	1	Active-low asynchronous reset
<code>cam_clear</code>	input	1	Synchronous clear of the compressor CAM + stat counters (tied off in raw-only builds)

52.4.2 Monitor-Bus Input + Timestamp

Port	Direction	Width	Description
monbus_valid	input	1	Monitor-bus packet valid
monbus_ready	output	1	Monitor-bus ready (backpressure to the arbiter)
monbus_packet	input	monitor_packet_t	128-bit monitor packet
monbus_timestamp	input	monbus_timestamp_t	64-bit source timestamp
mon_time_out	output	monbus_timestamp_t	Free-running 64-bit timestamp counter

52.4.3 AXIL Slave Read — Err-FIFO Drain

Port	Direction	Width	Description
s_axil_arvalid	input	1	Read-address valid
s_axil_arready	output	1	Read-address ready
s_axil_araddr	input	ADDR_WIDTH	Read address (record-slice index; address value not memory-mapped by the core)
s_axil_arprot	input	3	Read protection attributes
s_axil_rvalid	output	1	Read-data valid
s_axil_rready	input	1	Read-data ready
s_axil_rdata	output	S_AXIL_DATA_WIDTH	Read data (record slice, or half-slice in 32-bit mode)
s_axil_rresp	output	2	Read response

52.4.4 AXIL Master Write — Bulk Capture

Port	Direction	Width	Description
m_axil_aw valid	output	1	Write-address valid
m_axil_aw ready	input	1	Write-address ready
m_axil_aw addr	output	ADDR_WIDT H	Write address (within [cfg_base_addr, cfg_limit_addr])
m_axil_aw prot	output	3	Write protection attributes
m_axil_wv alid	output	1	Write-data valid
m_axil_wr eady	input	1	Write-data ready
m_axil_wd ata	output	64	Write data (one 64-bit beat)
m_axil_wst rb	output	8	Write strobe
m_axil_bva lid	input	1	Write-response valid
m_axil_bre ady	output	1	Write-response ready
m_axil_bre sp	input	2	Write response

52.4.5 Interrupt

Port	Direction	Width	Description
irq_out	output	1	Asserted whenever the err FIFO is non-empty

52.4.6 Configuration

Port	Direction	Width	Description
cfg_base_a ddr	input	ADDR_WIDT H	Master-write ring base address
cfg_limit_a ddr	input	ADDR_WIDT H	Master-write ring limit address (writer wraps, does not saturate)
cfg_flush_ watermark	input	16	Write-FIFO depth (beats) at which a flush fires
cfg_compr ess_en	input	1	Runtime compression enable (effective only when USE_COMPRESSION=1)

52.4.7 Per-Protocol Filter Masks

Three protocols (AXI, AXIS, CORE), each with a packet-type mask, an err-select mask, and per-event-category masks. All are 16-bit inputs.

Port	Protocol	Role
cfg_axi_pkt_mask	AXI	Drop by packet type
cfg_axi_err_select	AXI	Route to err FIFO by packet type
cfg_axi_error_mask / cfg_axi_timeout_ma sk / cfg_axi_compl_mask / cfg_axi_thresh_mas k / cfg_axi_perf_mask / cfg_axi_addr_mask / cfg_axi_debug_mask	AXI	Per-event-code drop masks
cfg_axis_pkt_mask	AXIS	Drop by packet type
cfg_axis_err_select	AXIS	Route to err FIFO by packet type
cfg_axis_error_mas	AXIS	Per-event-code drop

Port	Protocol	Role
k / cfg_axis_timeout_mask / cfg_axis_compl_mask / k / cfg_axis_credit_mask / k / cfg_axis_channel_mask / cfg_axis_stream_mask		masks
cfg_core_pkt_mask	CORE	Drop by packet type
cfg_core_err_select	CORE	Route to err FIFO by packet type
cfg_core_error_mask / k / cfg_core_timeout_mask / ask / cfg_core_compl_mask / sk / cfg_core_thresh_mask / sk / cfg_core_perf_mask / / cfg_core_debug_mask	CORE	Per-event-code drop masks

52.4.8 Status / Debug

Port	Direction	Width	Description
err_fifo_full	output	1	Err FIFO write port not ready
write_fifo_full	output	1	Write FIFO write port not ready
err_fifo_count	output	16	Err FIFO entry count (records)

Port	Direction	Width	Description
write_fifo_count	output	16	Write FIFO entry count (beats)
mon_compressor_stat	output	32 each	Compressor statistics (live only when USE_COMPRESSION=1)
_tier1_a / _tier1_b / _tier1_c / _tier0 / _cam_miss / _delta_ts_o vf / _event_data_ovf / _ed_delta_ovf			

52.5 Functional Description

52.5.1 S-Side Err-Drain Read Protocol

Each monbus err record is three 64-bit slices — {tag, source_ts}, packet[127:64], packet[63:0]. A 64-bit axil4_slave_rd leaf bridges the external AXIL read onto the core's 64-bit read FUB, and the core's slice counter returns the three slices in sequence; the err-FIFO record is popped only after slice 2 (packet low half) is read.

- **64-bit drain (S_AXIL_DATA_WIDTH == 64, g_drain_direct):** the leaf is wired straight through, one beat per slice — 3 beats/record.
- **32-bit drain (S_AXIL_DATA_WIDTH == 32, g_drain_2to1):** a phase bit splits each 64-bit leaf beat into a low then high 32-bit external read — 6 beats/record. The external AR is forwarded to the leaf only on the low phase, so the leaf issues exactly one core read per pair and never prefetches. The low read streams and consumes the beat while latching its high half; the high read replays the latch and gates R on its own accepted AR (r_hi_ar) to honor AXIL AR-before-R ordering (a master still holding r_ready from the low beat cannot consume the high beat early, which would desync the pair and double the core reads).

52.5.2 M-Side Bulk-Capture Protocol

Surviving (non-err) packets go to the write FIFO and are streamed out the AXIL master-write port by the core's burst writer. Because this is an AXIL master, MAX_BURST_BEATS=1 — each record is emitted as single-beat AW/W/B handshakes at consecutive addresses (three per raw record). The writer fires on watermark (write_fifo_count >= cfg_flush_watermark) or timeout (FLUSH_TIMEOUT_CYCLES), and the address is **circular** within [cfg_base_addr, cfg_limit_addr] — it wraps rather than saturating. Host-side ring pointers must match this wrap behavior. See monbus_group for the full burst-writer geometry.

52.5.3 Interrupt

irq_out is asserted by the core whenever the err FIFO is non-empty, so a CPU can take an interrupt and drain records over the slave-read port.

52.5.4 Shared Egress Packet Filter

The per-protocol filter (AXI, AXIS, CORE) decides for each packet: drop (via pkt_mask or a per-event mask), route to the err FIFO (via err_select), or route to the write FIFO (default). Protocols not in the supported set (APB, ARB) are always dropped. The filter config is identical across all four family wrappers — see monbus_group for the per-event category tables and masking rules.

52.5.5 Compression

When USE_COMPRESSION=1, monbus_compressor can be selected at runtime via cfg_compress_en, emitting one 64-bit self-tagged slot per record into the write FIFO instead of three raw beats; HALF_BEAT_EN=1 further packs two 30-bit half-slots per beat. cam_clear synchronously empties the compressor template CAM and zeroes its stat counters between runs. In raw-only builds these fold away.

52.6 Usage Example

```
monbus_axil_axil_group #(
    .FIFO_DEPTH_ERR      (64),
    .FIFO_DEPTH_WRITE    (96),    // beats (3x for raw mode)
    .ADDR_WIDTH          (32),
    .S_AXIL_DATA_WIDTH   (64),    // or 32 for a narrow host crossbar
    .FLUSH_TIMEOUT_CYCLES (1024),
    .USE_COMPRESSION      (0)
) u_monbus_egress (
    .axi_aclk      (aclk),
    .axi_aresetn   (aresetn),
    .cam_clear     (1'b0),

    // Arbitrated monitor-bus input
    .monbus_valid  (mon_valid),
```

```

.monbus_ready      (mon_ready),
.monbus_packet     (mon_packet),
.monbus_timestamp  (mon_timestamp),
.mon_time_out      (mon_time),

// CPU-facing err drain
.s_axil_arvalid    (cpu_arvalid),
.s_axil_arready    (cpu_arready),
.s_axil_araddr     (cpu_araddr),
.s_axil_arprot     (cpu_arprot),
.s_axil_rvalid     (cpu_rvalid),
.s_axil_rready     (cpu_rready),
.s_axil_rdata      (cpu_rdata),
.s_axil_rresp      (cpu_rresp),

// Memory-ring dump
.m_axil_awvalid    (ring_awvalid),
.m_axil_awready    (ring_awready),
.m_axil_awaddr     (ring_awaddr),
/* ... m_axil_w*, m_axil_b* ... */

.irq_out           (monbus_irq),
.cfg_base_addr     (RING_BASE),
.cfg_limit_addr    (RING_LIMIT),
.cfg_flush_watermark (16'd24),
.cfg_compress_en   (1'b0),

// Per-protocol filter masks (AXI / AXIS / CORE) ...
.cfg_axi_pkt_mask  (axi_pkt_mask),
.cfg_axi_err_select (axi_err_select),
/* ... remaining masks ... */

.err_fifo_count    (err_count),
.write_fifo_count  (wr_count)
/* ... compressor stats ... */
);

```

52.7 Design Notes

52.7.1 Legacy Replacement

This wrapper (plus its three siblings) replaces the fused `monbus_axil_group.sv`. Key surface changes from the legacy module: `cfg_flush_watermark` is a new required input,

`err_fifo_count / write_fifo_count` are 16 bits, and `FIFO_DEPTH_WRITE` is in beats (not records). See the migration section in `monbus_group`.

52.7.2 Why One Core + Four Wrappers

SystemVerilog cannot conditionally include ports in a single module's port list, so each protocol combination gets its own thin wrapper with the exact port shape. The filter, FIFOs, burst writer, and drain live once in `monbus_group_core`; the AXIL leaves and the FUB bridge live here.

52.7.3 32-Bit Drain Serializer

The `g_drain_2to1` path exists for 32-bit host crossbars. It is a careful 2:1 serializer: consuming the leaf beat on the low read (rather than holding it across both) makes `s_axil_rvalid` pulse once per external beat, so a master holding `rready` cannot mis-toggle the phase. See the shared mechanism note in the AXIL/AXI4 wrapper — the two wrappers use identical code.

52.8 Related Modules

52.8.1 Used By

- `stream_char / rapids_char` MonBus egress (AXIL fabric)
- Any monitor-delivery topology with AXIL drain and AXIL dump fabrics

52.8.2 Uses

- **`monbus_group_core.sv`** — Protocol-agnostic filter + FIFO + burst-writer + drain
- **`axil4_slave_rd.sv`** — Slave-read (err-drain) skid leaf
- **`axil4_master_wr.sv`** — Master-write (bulk-capture) skid leaf
- **`monbus_compressor.sv`** — Optional runtime compressor (`USE_COMPRESSION=1`)
- **`monitor_common_pkg`** — `monitor_packet_t / monbus_timestamp_t` types
- **`reset_defs.svh`** — Reset macros

52.8.3 See Also

- **`monbus_axil_axi4_group.sv`** — Same drain, AXI4-burst master-write
- **`monbus_axi4_axil_group.sv / monbus_axi4_axi4_group.sv`** — AXI4-burst slave-read variants
- **`monbus_arbiter.sv`** — Upstream multi-source merge (instantiate before this wrapper for $N > 1$ sources)

- `sdpram_slave_axil_axil.sv` — Canonical memory-ring backend for the master-write port
-

52.9 References

52.9.1 Source Code

- RTL: `rtl/amba/monitor/monbus_axil_axil_group.sv`
- Core: `rtl/amba/monitor/monbus_group_core.sv`
- Tests: `val/amba/test_monbus_axil_axil_group.py`,
`test_monbus_axil_axil_group_compressed.py`,
`test_monbus_axil_axil_group_master_write.py`

52.9.2 Documentation

- Family spec: `docs/markdown/RTLAmba/monbus_group.md`
 - Architecture: `docs/markdown/RTLAmba/shared/README.md`
 - Packet format:
`docs/markdown/RTLAmba/includes/monitor_package_spec.md`
-

Last Updated: 2026-07-15

52.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

53 Monitor Bus LRU CAM

Module: `monbus_cam.sv` **Location:** `rtl/amba/shared/` **Category:** Bulk-Trace Compression Infrastructure **Status:** Reference design — superseded in-production by `monbus_cam_pipe`

 **Deprecation note.** `monbus_cam.sv` is the single-cycle reference design. The in-production CAM inside `monbus_compressor` is now `monbus_cam_pipe.sv`, a 2-cycle pipelined variant that splits the 49-bit compare → priority-encode → move-to-front chain into two stages so the

path closes at 100 MHz on Nexys A7 (f909f01f / 0d1c0a1a). The two CAMs are functionally identical (true-LRU, same action set, same per-entry storage); the pipelined version adds a 1-cycle latency and a credit-gated result skid but holds throughput at 1 record/cycle. Both files are kept in tree: this single-cycle module serves as the executable spec for the LRU semantics and the compressor's CAM behavior, and is what the algorithmic tests (`test_monbus_cam.py`) target. New code should instantiate `monbus_cam_pipe`.

53.1 Overview

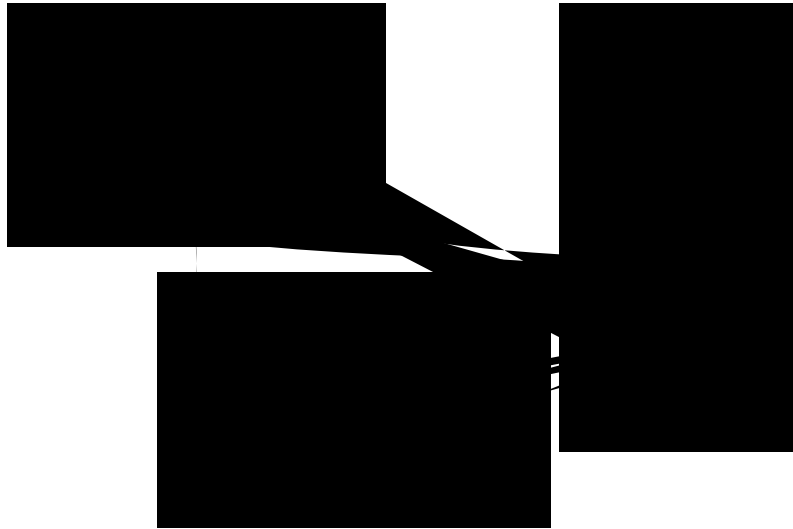
`monbus_cam` is a **true-LRU caching content-addressable memory** used by the `monbus_compressor` to assign 5-bit template indices (`tmpl_idx`) to the 49-bit template keys it extracts from monitor packets. “True LRU” means the eviction victim is the least recently *accessed* entry (matched, touched, or installed) — not the least recently *inserted*. This matters because the bulk-trace compression format relies on both encoder and decoder maintaining identical CAM state from the slot stream alone; if the two sides diverge on eviction order, the decoder produces garbage.

The module is a **bit-exact mirror** of the Python `Cam` class in `bin/TBClasses/monbus/monbus_compressor.py`. Any divergence between the two implementations is a regression.

53.2 Key Features

- 32-entry capacity (locked by the bulk-trace format spec)
 - 49-bit key, 64-bit payload (both parameterizable)
 - **Per-entry timestamp storage** (`TS_WIDTH=24`) for per-template `delta_ts` — see the dedicated section below
 - Single combinational access port: lookup + commit in one cycle
 - True LRU eviction via **position-indexed storage** — the slot index IS the recency rank (slot 0 = MRU, slot `DEPTH-1` = LRU)
 - 3 caller-driven actions: `NONE`, `TOUCH`, `INSTALL`
 - Eviction pulse output for stats / instrumentation
 - Simulation-only protocol assertions on the caller
-

53.3 Architecture



monbus_cam block diagram

Source: monbus_cam.mmd

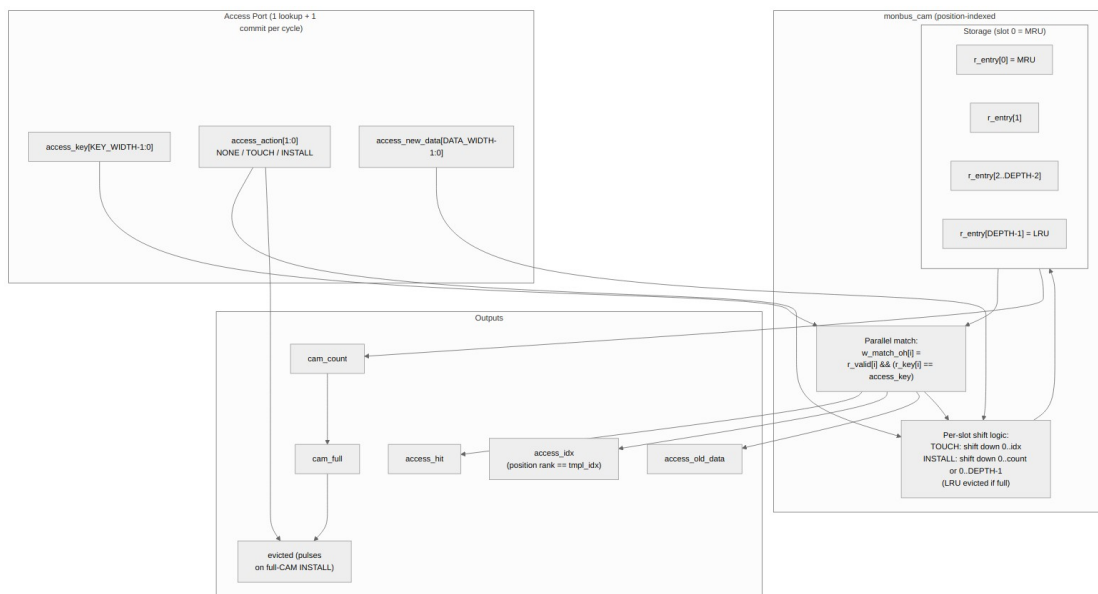


Diagram 22

53.4 Top-level Interface

```
module monbus_cam #(
    parameter int KEY_WIDTH = 49,
```

```

    parameter int DATA_WIDTH = 64,
    parameter int TS_WIDTH    = 24,    // per-entry last_ts width (per-
template delta_ts)
    parameter int DEPTH      = 32,
    parameter int IDX_WIDTH  = (DEPTH > 1) ? $clog2(DEPTH) : 1,
    parameter int CNT_WIDTH  = $clog2(DEPTH + 1)
) (
    input logic          clk,
    input logic          rst_n,

    // Access port (one combinational lookup + one commit per cycle)
    input logic [KEY_WIDTH-1:0] access_key,
    output logic               access_hit,
    output logic [IDX_WIDTH-1:0] access_idx,    // position rank
    (only valid on hit)
    output logic [DATA_WIDTH-1:0] access_old_data, // pre-commit
payload at access_idx
    output logic [TS_WIDTH-1:0] access_old_ts,    // pre-commit
timestamp at access_idx

    input logic [1:0]          access_action,
    input logic [DATA_WIDTH-1:0] access_new_data,
    input logic [TS_WIDTH-1:0] access_new_ts,    // timestamp to
write on TOUCH / INSTALL

    // Status
    output logic               cam_full,
    output logic [CNT_WIDTH-1:0] cam_count,
    output logic               evicted           // pulses on full-
CAM INSTALL
);

```

53.5 Storage Model: Position-Indexed LRU

The fundamental design choice is that **the slot index IS the position rank**:

```

r_entry[0]           = most-recently-used (MRU)
r_entry[1] .. r_entry[count-1] = ordered, newer first
r_entry[count..DEPTH-1] = invalid (empty slots)

```

This means: - The `access_idx` output on a hit is the entry's current position rank, which is exactly what `tmpl_idx` needs to be in the compressed slot stream. - On `TOUCH` or `INSTALL`, the matched/new entry moves to slot 0 and older entries shift down by one position. This is one structural operation that updates *both* the storage AND the rank simultaneously. - The LRU victim

is always whoever sits at slot DEPTH-1. No tag pointers, no doubly-linked list, no per-entry counter — pure structural.

The trade-off: every TOUCH/INSTALL performs a shift of up to DEPTH-1 entries on a single clock edge. For DEPTH=32 this is well within timing budget (parallel per-slot updates in a generate loop) but it's the reason the format spec locks the size at 32 and not 64 or 128.

53.6 Actions

The caller drives exactly one action per cycle on the `access_action` port:

Action	Encoding	Caller protocol	State change on commit
ACTION_NONE	2'b00	Pure lookup — anytime	None. Just samples hit/idx/old_data.
ACTION_TOUCH	2'b01	Must coincide with a hit	Matched entry moves to slot 0, payload updated with access_new_data.
ACTION_INSTALL	2'b10	Must coincide with a miss	New entry installed at slot 0. If cam_full, the entry at slot DEPTH-1 is evicted and evicted pulses high that cycle.
2'b11	reserved	—	Treated as NONE (no state change).

Caller protocol enforcement (simulation-only, via \$error): - TOUCH without access_hit is illegal — the caller saw a miss but is claiming to be touching an existing entry. - INSTALL while access_hit is illegal — the key is already present and reinstalling would create a duplicate. - The internal match vector must be at most one-hot.

The natural compressor pattern is TOUCH-on-hit / INSTALL-on-miss, which satisfies all three constraints by construction.

53.7 Per-Slot Update Mechanics

On every clock edge:

Action	What happens to slot <i>i</i>
NONE or reserved	Unchanged.
TOUCH matching slot <i>P</i>	Slots 0 . . <i>P</i> - 1 shift down by 1, slot 0 becomes the (matched key, new_data), slot <i>P</i> and below are unchanged from their previous positions but written one slot earlier.
INSTALL when !cam_full	Insertion position is cam_count. Slots 1 . . cam_count shift down from 0 . . cam_count - 1. Slot 0 becomes the new entry. cam_count++.
INSTALL when cam_full	Insertion position is DEPTH - 1 (overwriting the LRU). Slots 1 . . DEPTH - 1 shift down from 0 . . DEPTH - 2. Slot 0 becomes the new entry. evicted pulses high. cam_count stays at DEPTH.

A per-slot generate loop generates DEPTH independent always_ff updates, each gated by `do_shift && (CNT_WIDTH'(i) <= shift_to)`. This compiles to ~one LUT level per slot on the per-bit datapath — Vivado synthesises the whole shift as DEPTH parallel small update cones.

53.8 Per-Entry Timestamp Storage

Earlier revisions of the CAM stored only (key, data) per entry; the compressor measured `delta_ts` against a single global `r_last_ts`. That worked for single-source streams but collapsed compression to raw whenever multiple sources interleaved templates with non-monotonic absolute timestamps (the 4-channel STREAM characterization case).

The current CAM adds a **per-entry** `r_ts[TS_WIDTH=24]` array that shifts in lockstep with the key and data on every TOUCH / INSTALL:

`r_key[i], r_data[i], r_ts[i]` shift together when slot *i* moves

`access_old_ts` outputs the *pre-commit* timestamp at the matched slot (valid only when `access_hit`) — i.e. the timestamp of the previous record that used this template. The compressor uses it to compute `delta_ts = src_ts_lo - cam_access_old_ts`, then writes the current record's `source_ts[23:0]` back into the slot via `access_new_ts` in the same cycle.

TS_WIDTH = 24 bits. Format-B (the 23-bit-delta Tier-1 format) needs 24 bits to *detect* its delta overflow. 16 bits silently aliases large gaps to wrong encodes.

The CAM is therefore no longer pure opaque-payload — its caller needs to drive `access_new_ts` along with `access_new_data` on every TOUCH / INSTALL. The compressor wires this directly from the incoming record's low 24 timestamp bits.

53.9 Match Logic

The match vector is a parallel one-hot:

```
always_comb begin
    for (int i = 0; i < DEPTH; i++) begin
        w_match_oh[i] = r_valid[i] && (r_key[i] == access_key);
    end
end
```

That's 1 LUT per bit × 32 bits × 49-bit equality. Vivado fuses each equality into ~3 LUT levels; the whole match is independent of the rest of the cycle's logic (no chained dependencies on shift/count signals). The priority encoder feeding `access_idx` is then DEPTH-1-input.

53.10 Configuration

The default values (KEY_WIDTH=49, DATA_WIDTH=64, DEPTH=32) are what the compressor instantiates and what the locked format spec mandates. The parameters exist so the module can be reused in other contexts (e.g. a smaller on-chip event cache) but `monbus_compressor` itself doesn't override them.

If you change DEPTH, the corresponding change in the Python golden's `DEFAULT_CAM_SIZE` constant must happen in lockstep — otherwise the encoded slot stream will diverge.

53.11 Test

`val/amba/test_monbus_cam.py` runs 10 sub-tests covering:

1. Reset state (all empty, `cam_full=0`, no matches)
2. Install + lookup basic round-trip
3. Fill to DEPTH-1 (no overflow path exercised)
4. TOUCH updates payload, `idx` becomes 0
5. `cam_full` asserts on the Nth install
6. LRU eviction on INSTALL when full
7. evicted pulses **only** on full-CAM install (not on lookup, not on touch)
8. Miss on absent key (no state change)
9. TOUCH moves the entry to MRU (the LRU-specific invariant)
10. Random stress (~500 ops at FUNC, 5000 at FULL) cross-checked against a Python LRU model

The Python model in the test is a bit-exact mirror of the RTL — same storage semantics, same shift rules. Random stress is constrained to the caller protocol (no INSTALL on hit, no TOUCH on miss) so the protocol assertions never fire spuriously.

```
pytest val/amba/test_monbus_cam.py -v
```

REG_LEVEL parameter sweep (gate / func / full): - **GATE**: 1 config (default 49/64/32) - **FUNC**: 2 configs (+ small DEPTH=8) - **FULL**: 6 configs (DEPTH 4/8/16/32, key 16/32/49/64, data 16/32/64)

53.12 Related Modules

Module	Role
<code>monbus_compressor</code>	Sole consumer in the production design
<code>bin/TBClasses/monbus/monbus_compressor.py</code> (Cam class)	Python golden mirror
<code>monitor_trans_cam</code>	Sister CAM, different use case (AXI ID matching, multi-port, no LRU)

54 Monitor Bus LRU CAM (Pipelined)

Module: `monbus_cam_pipe.sv` **Location:** `rtl/amba/shared/` **Category:** Bulk-Trace Compression Infrastructure **Status:** Production — replaces `monbus_cam` inside the compressor

54.1 Overview

monbus_cam_pipe is a **2-cycle pipelined variant** of monbus_cam. It implements the same true-LRU CAM semantics — same key/data/timestamp storage, same MRU-at-slot-0 move-to-front, same hit/idx/old_* result — but splits the combinational match → priority-encode → commit chain into two registered stages so the path closes at 100 MHz on Nexys A7.

It is the in-production CAM inside monbus_compressor. The single-cycle monbus_cam.sv is retained as a reference design and as the algorithmic oracle for unit tests.

```
single-cycle monbus_cam : one cycle: compare → encode → commit +  
result  
pipelined   monbus_cam_pipe : cycle T   : 32-way compare  
                                cycle T+1 : priority-encode + commit +  
result reg
```

The latency cost is **+1 cycle**. Throughput is unchanged: a forwarding network corrects each new lookup for the in-flight commit, so back-to-back accesses see bit-exact (hit, idx, old_data, old_ts) sequences identical to monbus_cam. Validated by val/amba/test_monbus_cam_pipe.py.

54.2 Top-level Interface

```
module monbus_cam_pipe #(
    parameter int KEY_WIDTH  = 49,
    parameter int DATA_WIDTH = 64,
    parameter int TS_WIDTH   = 24,
    parameter int DEPTH      = 32
) (
    input logic          clk,
    input logic          rst_n,

    // Synchronous clear: invalidate all entries and flush the
    // lookup pipeline on the next edge. Lets the host rebase the
    // template table without asserting rst_n. Takes priority over
    // access_en.
    input logic          clear,

    // Access presented this cycle (one per cycle when access_en=1).
    input logic          access_en,
    input logic [KEY_WIDTH-1:0] access_key,
    input logic [DATA_WIDTH-1:0] access_new_data,
    input logic [TS_WIDTH-1:0]  access_new_ts,

    // Result of the access presented LAST cycle.
```



```

output logic                                result_valid,
output logic                                result_hit,
output logic [IDX_WIDTH-1:0]               result_idx,
output logic [DATA_WIDTH-1:0]             result_old_data,
output logic [TS_WIDTH-1:0]               result_old_ts,

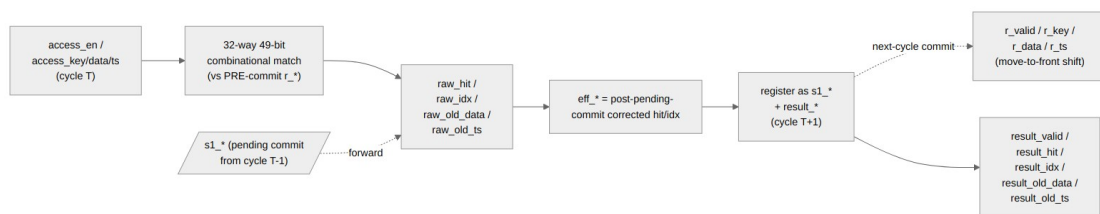
output logic                                cam_full,
output logic [CNT_WIDTH-1:0]             cam_count
);

```

Differences from monbus_cam:

Aspect	monbus_cam	monbus_cam_pipe
Latency	0 (combinational lookup + commit)	1 cycle
Throughput	1 access/cycle	1 access/cycle
Action selection	Caller drives action[1:0]	Self-derived from forwarded hit (TOUCH on hit, INSTALL on miss)
clear input	(added in same commit)	yes — sync invalidate + pipe flush

The self-derived action drop matters: a pipelined caller cannot know the hit at present-cycle time (exposing a combinational hit would defeat the pipeline). The compressor always TOUCHes on hit and INSTALLs on miss, so the CAM derives this internally and removes the two action bits from its own port surface.



Pipeline Diagram

The two registered stages are:

Stage	Cycle	Work
1 (compare)	T	32-way 49-bit key compare against the pre-commit register array.

Stage	Cycle	Work
		Combinational priority encode picks the highest-numbered match (LRU side of the array, to keep the encoder ordering identical to the single-cycle module).
2 (commit)	T+1	Apply the <i>pending</i> commit from the previous access (move-to-front shift). Register the forward-corrected result of cycle T's access. Capture cycle T+1's access as the next pending commit.

The cycle T+1 commit and the cycle T+1 result registration both reference the same r_* snapshot, so the result reflects the array state *after* the in-flight move-to-front.

54.3 Forwarding (depth-1)

When access **A** is compared at cycle T+1, the previous access **P** (captured at cycle T) is committing this same cycle. **A**'s combinational match sees the *pre-P* array, so the forward-correct adjustment is needed:

Case	Effect on A
A.key == P.key	P just wrote slot 0 with its new data/ts → A hits with idx=0, old_data = P.new_data, old_ts = P.new_ts
A matched the LRU entry that P evicted (full-CAM install)	A is now a miss (the evicted key is gone)
A.raw_idx < P.shift_to	A's entry shifted one slot down → eff_idx = raw_idx + 1
Otherwise	A's match position is unchanged

P.`shift_to` and the actual commit shift both read the **same** pre-commit `r_*` this cycle, so the forwarding and the commit always agree. Bubbles (`access_en=0`) still let P commit and clear the pending slot, so a later access sees a fully-committed array and no forwarding is needed.

The end result: the (`hit`, `idx`, `old_data`, `old_ts`) output sequence for any access trace is **byte-exact** with the single-cycle `monbus_cam` for the same input trace (modulo the +1-cycle latency). This is the property the equivalence test guards.

54.4 Self-Derived Action

The two-bit `access_action` port from `monbus_cam` is gone. Instead:

```
s1_action = eff_hit ? ACTION_TOUCH : ACTION_INSTALL (registered)
```

On a hit, the matched entry moves to slot 0 with the new data/ts written through (TOUCH). On a miss, a new entry is installed at slot 0 (INSTALL), evicting the LRU if the CAM is full. The caller cannot suppress the commit — every `access_en=1` cycle is a commit.

For the compressor this is exactly the desired behavior: every record TOUCHes on hit and INSTALLs on a CAM miss; the previous CAM's `ACTION_NONE` lookups were not used in production. If a future caller needs lookups without commits, it would need a separate input port gating the commit (not present today).

54.5 Synchronous Clear

`clear` is a single-cycle synchronous reset:

- All `r_valid[i]` cleared (so the CAM appears empty next cycle).
- `r_count` zeroed.
- `s1_valid` cleared (any pending commit is dropped).
- `result_valid` cleared (any in-flight result is dropped).
- Storage (`r_key`, `r_data`, `r_ts`) is **not** cleared — it's ignored while `r_valid[i]=0` so leaving it stale is harmless.

`clear` takes priority over `access_en`: an access pulse arriving in the same cycle as a clear is dropped. The host should hold `clear` high for one cycle while the upstream compression source is idle, which is the contract for the Stream CTRL [4] bit driving this through the monbus group wrapper.

54.6 Verification

```
pytest val/amba/test_monbus_cam_pipe.py -v
```

The test drives random access traces through both `monbus_cam` and `monbus_cam_pipe` in parallel and asserts the (`hit`, `idx`, `old_data`, `old_ts`) sequence matches cycle-for-cycle (modulo the +1-cycle latency on the pipelined side). Both modules are fed from the same random key/data/ts stream so any divergence shows up immediately.

Additional coverage: - `test_monbus_compressor.py` — uses `monbus_cam_pipe` end-to-end inside the compressor and cross-checks the slot stream against the Python golden encoder. - `test_monbus_axil_axil_group_compressed.py` — same coverage at the group-wrapper level, including window wrap.

54.7 Related Modules

Module	Role
<code>monbus_cam</code>	Single-cycle reference design — same LRU semantics, used by the unit-test oracle
<code>monbus_compressor</code>	Production caller of this CAM
monbus_group family	Host of the compressor + master writer

55 Monitor Bus Compressor

Module: `monbus_compressor.sv` **Location:** `rtl/amba/shared/` **Category:** Bulk-Trace Compression **Status:** Production Ready

55.1 Overview

`monbus_compressor` is a **hardware bulk-trace encoder** for the monitor bus. It consumes streamed (`monitor_packet_t`, `monbus_timestamp_t`) records, exploits the heavy templating that real workloads exhibit, and emits a 64-bit slot stream that compresses by roughly 2.6× with **byte-identical** equivalence to a Python golden encoder.

The compressor sits in front of the master writer inside the [monbus_group family](#) when a wrapper opts in via the `USE_COMPRESSION=1` parameter. The slot stream is what eventually lands in the SRAM dump ring on the FPGA, so smaller slots → longer recording windows in the same memory budget.

The format was **locked** at commit 5bbb83d1 after a sweep across CAM sizes and a real-silicon validation run on a Nexys A7. Once locked, the slot stream is *the* wire format — any RTL change here that diverges from the Python golden constitutes a regression.

55.2 Why Compress Monitor Traces?

A 128-bit monitor packet plus a 64-bit timestamp is 24 bytes per event. At even modest event rates (a few million events/sec), an AXI4 monitor running all packet types enabled can saturate a 32-bit AXIL writer’s bandwidth and fill a typical SRAM dump region in milliseconds.

The key observation behind this compressor: real workloads are *extremely* repetitive at the monitor-packet level. In the locked validation dataset (desc_axi_16desc_8ch_1MB, 682 records of descriptor-fetch traffic):

Metric	Value
Distinct (pkt_type, protocol, event_code, channel_id, agent_id, unit_id) tuples	32 (4.7 % unique)
Records emitted	682
Compression ratio	2.66×
Tier-1 hit rate	93.5 %
Tier-0 escapes	6.5 % (44 records)

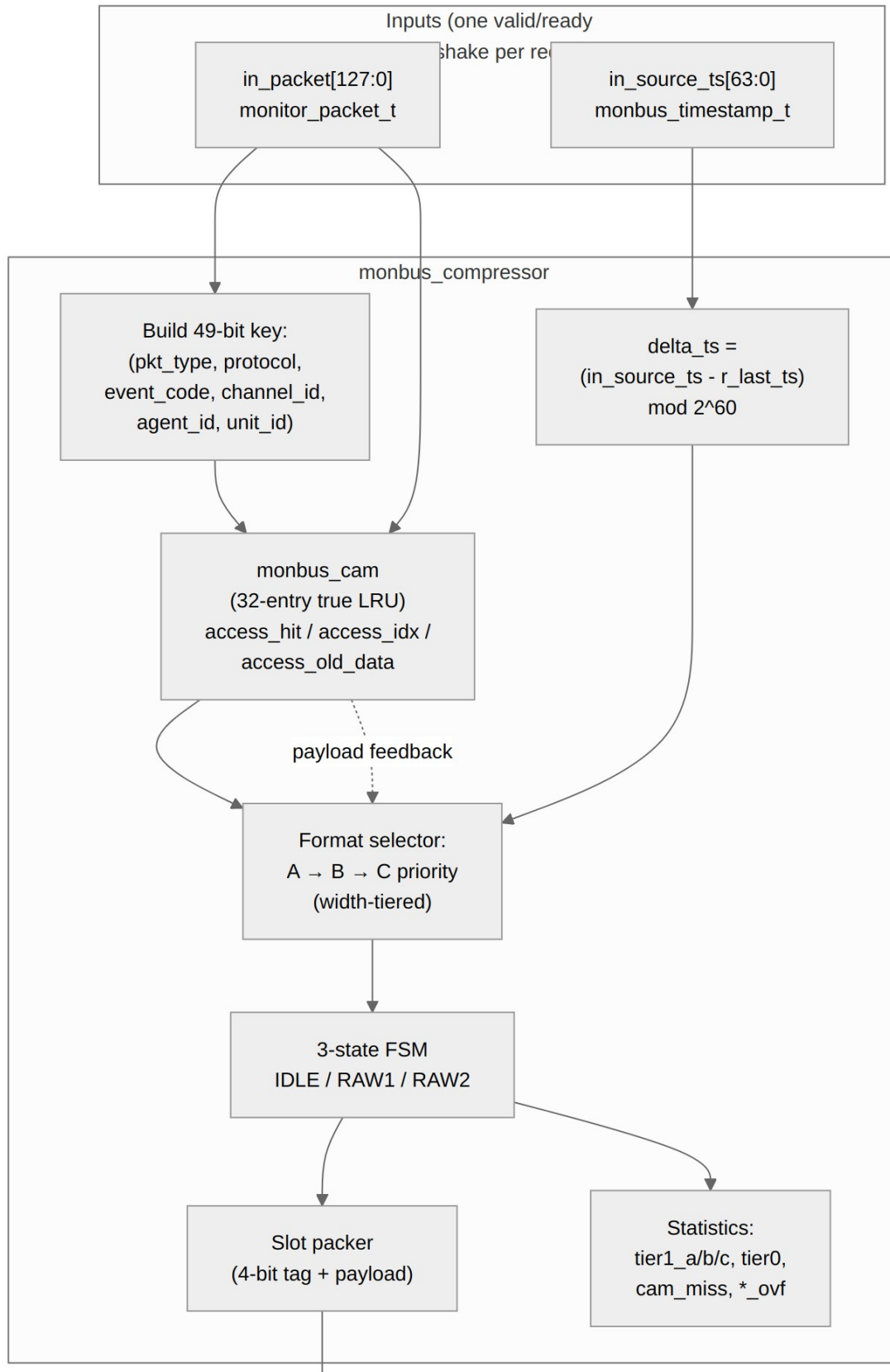
The 32 distinct “templates” fit *exactly* in a 32-entry CAM. Going wider buys zero incremental hits on this workload. That’s the design point.

55.3 Architecture



monbus_compressor block diagram

Source: `monbus_compressor.mmd`



55.4 Top-level Interface

```

module monbus_compressor
    import monitor_common_pkg::*;
#(
    // 0 = default; 64-bit-slot codec only (the committed, timing-
    // closed path).
    // 1 = also expose a 30-bit half-slot sideband (out_half_*) for
    the
    //      downstream `monbus_halfbeat_packer`. Folds away when 0.
    parameter int HALF_BEAT_EN = 0
) (
    input   logic          clk,
    input   logic          rst_n,
    // Synchronous clear: empty the template CAM (and zero the stat
    // counters)
    // without asserting rst_n. Pulse one cycle while idle between
    runs.
    input   logic          clear,

    // Records in (one valid/ready handshake)
    input   logic          in_valid,
    output  logic          in_ready,
    input   monitor_packet_t in_packet,    // 128 bits
    input   monbus_timestamp_t in_source_ts, // 64 bits

    // Slots out (one valid/ready handshake)
    output  logic          out_valid,
    input   logic          out_ready,
    output  logic [63:0]    out_slot,

    // Half-slot sideband (valid alongside out_valid on a tier-1 beat
    that
    // also fits a 30-bit half-slot). Used by monbus_halfbeat_packer
    when
    // HALF_BEAT_EN=1; tied 0 when HALF_BEAT_EN=0.
    output  logic          out_half_valid,
    output  logic [29:0]    out_half_slot,

    // Statistics counters (combinational from registers)
    output  logic [31:0]    stat_tier1_a,
    output  logic [31:0]    stat_tier1_b,
    output  logic [31:0]    stat_tier1_c,

```



```

    output logic [31:0]      stat_tier0,
    output logic [31:0]      stat_cam_miss,
    output logic [31:0]      stat_delta_ts_ovf,
    output logic [31:0]      stat_event_data_ovf,
    output logic [31:0]      stat_ed_delta_ovf
);

```

The 64-bit slot format and CAM size (32 entries × 49-bit key) are locked to match the Python golden. Only HALF_BEAT_EN is selectable: it gates an *additional* output port pair that feeds an optional downstream packer; the main out_slot stream is unchanged.

55.4.1 Synchronous CAM clear

clear is a synchronous, level-sensitive input. Holding it for one cycle (while idle) invalidates every CAM entry — r_valid, last_event_data, and last_ts — *and* zeroes the stat counters. This is the path the Stream characterization block uses (CTRL[4]): the host can rebase compression statistics between runs without toggling rst_n (which would also reset the surrounding monbus pipeline, FIFOs, and AXI skids). Drive it low at all other times.

55.5 Compression Techniques

The compressor combines four ideas, applied in priority order per record:

1. **Template extraction with CAM-backed indexing**
2. **Delta-encoding of the timestamp**
3. **Width-tiered payload encoding** (Tier-1 formats A/B/C)
4. **Differential encoding of the payload** (Tier-1 format C for monotonic counters / sequential addresses)
5. **Tier-0 RAW escape** when none of the above apply

Each successful Tier-1 encoding squeezes 24 bytes (raw record) into 8 bytes (one slot) — that’s the 3× upper bound. The 2.66× achieved ratio reflects the 6.5 % escape rate, plus the framing overhead in Tier-0.

55.5.1 1. Template extraction

The 128-bit monitor packet has six “low-entropy” fields that repeat heavily across events generated by the same logical agent:

Field	Bits	Description
packet_type	4	error / timeout / completion / threshold / perf / debug

Field	Bits	Description
protocol	4	AXI / AXIS / APB / CORE
event_code	8	sub-category within packet type
channel_id	9	per-engine channel identifier
agent_id	16	per-IP agent identifier
unit_id	8	sub-unit within an IP
template key total	49	

Two events from the same agent reporting the same condition share all 49 bits. The only differences cycle-to-cycle are the 64-bit event_data field (addresses, counters, byte counts) and the timestamp. The compressor identifies each template once, assigns it a **5-bit tmpl_idx** (0..31), and from then on sends only (tmpl_idx, event_data, delta_timestamp) instead of the full 49-bit key.

The mapping (key) → tmpl_idx lives in a 32-entry **caching CAM** (monbus_cam). Both the encoder and the decoder maintain the same CAM state, driven by identical (action, key, data) sequences derived from the slot stream. No out-of-band table sync is required — the slot stream is self-describing.

55.5.22. Delta-encoded timestamp

Absolute timestamps need 60 bits to cover practical recording windows (2^{60} cycles $\approx$ 117 days at 100 MHz). Deltas between consecutive events on the same monitor bus rarely exceed a few thousand cycles in the steady state, so the delta requires only 15 to 23 bits.

$\text{delta_ts} = (\text{current_source_ts} - \text{r_last_ts}) \& ((1 \ll 60) - 1)$

The 60-bit modulo handles natural wrap. r_last_ts is updated on **every** record encoded (Tier-0 and Tier-1 alike), so the decoder can rebuild the absolute timestamp from a chain of deltas without losing precision.

55.5.33. Width-tiered Tier-1 formats

Three Tier-1 slot formats each cover a different (delta_ts_width, event_data_width) trade-off. The encoder picks the **first one that fits** in this priority order:

Format	Tag	tmpl_idx	delta_ts	event_data	Notes
A	0x1	5b @ [59:55]	15b @ [54:40]	40b @ [39:0]	Common case
B	0x2	5b @ [59:55]	23b @ [54:32]	32b @ [31:0]	Big delta_ts

Format	Tag	tmpl_idx	delta_ts	event_data	Notes
C	0x3	5b @ [59:55]	15b @ [54:40]	40b @ [39:0] signed delta	Monotonic counters

The choice is governed by:

```

delta_ts ≤ 215−1 AND event_data ≤ 240−1 → Format A (most common)
delta_ts ≤ 223−1 AND event_data ≤ 232−1 → Format B
delta_ts ≤ 215−1 AND ed_delta in [−239, 239−1] → Format C
otherwise → Tier-0 escape

```

Format A assumes the agent has been active recently ($\text{delta_ts} < \sim 328 \mu\text{s}$ @ 100 MHz) and reports an `event_data` value that fits in 40 bits. This covers most steady-state traffic where the same template keeps firing.

Format B handles longer idle periods — up to ~ 84 ms between events on the same template. The `event_data` budget shrinks from 40 to 32 bits, which still covers most addresses on 32-bit address spaces.

Format C is the differential payload encoder, described next.

55.5.44. Differential payload encoding (Format C)

For monotonically increasing counters (transaction counts, accumulated byte totals, sequential addresses), absolute `event_data` may exceed 40 bits but the *delta* between consecutive events on the same template fits in 40 signed bits. Format C exploits this:

```

ed_delta = event_data - CAM[tmpl_idx].last_event_data      # signed
if (−239) ≤ ed_delta < 239:
    emit Format C slot containing (tmpl_idx, delta_ts, ed_delta)
    update CAM[tmpl_idx].last_event_data = event_data

```

The encoder records the previous `event_data` per template (stored as the CAM's **payload** field — 64 bits per slot). The decoder maintains the same CAM, so when it sees a Format C slot it reconstructs:

```
event_data = CAM[tmpl_idx].last_event_data + ed_delta
```

The locked validation dataset has 0 Format C hits — descriptor-fetch traffic isn't monotonic enough. But for stream-data or descriptor-write traces, this format kicks in heavily.

55.5.55. Tier-0 RAW escape

If none of the Tier-1 formats fit (CAM miss, or $\text{delta_ts} > 8\text{M}$ cycles, or $\text{event_data} > 2^{40}$), the compressor falls back to a 3-beat RAW record:

```

beat 0 (tag = 0x0): [63:60] = 0x0 | [59:0] = source_ts[59:0]
beat 1              : packet[127:64]
beat 2              : packet[63:0]

```

On a Tier-0 escape **caused by a CAM miss**, the encoder also **installs** the new template at the MRU position of the CAM (evicting the LRU if the CAM is full). The decoder does the identical install on its side, so subsequent Tier-1 references to the new `tmpl_idx` resolve correctly.

On a Tier-0 escape **caused by an overflow** (`delta_ts` or `event_data` too big), the encoder does NOT re-install — the key was already in the CAM with the right `tmpl_idx`, and the escape is just a 3-beat fallback for the one unusually-large record. Statistics counters record which overflow reason caused each escape, so the host can tell the difference.

55.6 Slot Bit Layouts

The 64-bit slot is always self-describing through the 4-bit tag in [63:60]:

Tag 0x0 – RAW (Tier-0 escape, 3 beats total)

```

beat 0: [63:60] tag=0x0
         [59:0] source_ts[59:0]
beat 1: packet[127:64]
beat 2: packet[63:0]

```

Tag 0x1 – Tier-1 Format A (1 beat)

```

beat 0: [63:60] tag=0x1
         [59:55] tmpl_idx[4:0]
         [54:40] delta_ts[14:0]
         [39:0]  event_data[39:0]

```

Tag 0x2 – Tier-1 Format B (1 beat)

```

beat 0: [63:60] tag=0x2
         [59:55] tmpl_idx[4:0]
         [54:32] delta_ts[22:0]
         [31:0]  event_data[31:0]

```

Tag 0x3 – Tier-1 Format C (1 beat)

```

beat 0: [63:60] tag=0x3
         [59:55] tmpl_idx[4:0]
         [54:40] delta_ts[14:0]
         [39:0]  ed_delta[39:0] (signed)

```

Tags 0x4-0xF – Reserved (decoder must error on these)

The decoder reads one 64-bit slot, inspects bits [63:60], and knows immediately whether to read 0 more beats (Tier-1) or 2 more beats (Tier-0 RAW). No lookahead is required.

55.7 CAM Design

The 32-entry caching CAM lives in `rtl/amba/monitor/monbus_cam_pipe.sv`. It is a **true LRU** CAM with position-indexed storage — the position rank IS the `tmpl_idx`. This matters because both encoder and decoder must agree on the rank assignment when they install or touch a template, and position-indexed storage makes the rank update trivially atomic with the storage update.

Production CAM is pipelined. Earlier builds used the single-cycle `monbus_cam` directly in the encoder's combinational critical path. To close 100 MHz on Nexys A7, the in-production CAM is now `monbus_cam_pipe` — a 2-cycle variant that registers the 32-way compare → priority-encode → move-to-front sequence into two stages, and feeds the encoder through a credit-gated result skid (`SKID_DEPTH=2`). A `r_credit` counter tracks records in flight + skid occupancy so `cam_en` only fires when there is a slot to land the result, even under back-to-back compression bursts. Throughput stays at **1 record/cycle** despite the +1 cycle of CAM latency; the single-cycle `monbus_cam.sv` is kept in-tree as the reference design (see its doc for the deprecation note).

Per-entry size:

Field	Bits
valid	1
key (concatenated 6-tuple)	49
last_event_data	64
position rank	5
total per entry	119

Total CAM storage: 32 entries × 119 bits ≈ **3.8 Kb (~480 bytes)**.

The CAM exposes three actions on its single access port:

Action	Encoding	Effect
<code>ACTION_NONE</code>	<code>2'b00</code>	Lookup only, no state change
<code>ACTION_TOUCH</code>	<code>2'b01</code>	Matched entry moves to MRU, <code>last_event_data</code> updated
<code>ACTION_INSTALL</code>	<code>2'b10</code>	New entry installed at MRU (evicts LRU if full)

The compressor's FSM issues TOUCH after Tier-1 hits, INSTALL after Tier-0 escapes caused by a CAM miss, and NONE during the slot 1 / slot 2 of a Tier-0 RAW expansion.

55.8 Encoder Decision Tree

Per input record (one cycle):

1. Build the 49-bit template key from `in_packet`.
Compute `event_data = in_packet[63:0]` (the lower 64 bits).
Compute `delta_ts = (in_source_ts - r_last_ts) & ((1 << 60) - 1)`.
2. CAM lookup (combinational):
 - MISS → Tier-0 escape
Issue ACTION_INSTALL on the CAM (key, event_data).
FSM enters S_RAW1 → S_RAW2; emits 3 slots over 3 cycles.
Bump `stat_cam_miss + stat_tier0`.
 - HIT(idx) → continue to step 3.
3. Try Tier-1 formats A → B → C in order:
 - A. if `delta_ts < 215` AND `event_data < 240`:
emit Format A slot, ACTION_TOUCH with new event_data.
Bump `stat_tier1_a`.
`r_last_ts ← in_source_ts`. Done.
 - B. else if `delta_ts < 223` AND `event_data < 232`:
emit Format B slot, ACTION_TOUCH.
Bump `stat_tier1_b`.
`r_last_ts ← in_source_ts`. Done.
 - C. else if `delta_ts < 215`:
`ed_delta = event_data - CAM[idx].last_event_data` (signed)
if `-239 ≤ ed_delta < 239`:
emit Format C slot, ACTION_TOUCH.
Bump `stat_tier1_c`.
`r_last_ts ← in_source_ts`. Done.
- else:
Tier-0 escape (CAM hit but record didn't fit any Tier-1).
Do NOT install (key already in CAM).
Bump `stat_delta_ts_ovf / stat_event_data_ovf / stat_ed_delta_ovf`
depending on which overflow was the trigger, + `stat_tier0`.

The format-selector logic is combinational on cycle 0; the slot emission and CAM state update happen on the same clock edge.

55.9 Pipeline and Timing

The encoder is split into **2 registered stages** (1 in, 1 register in the middle, 1 out — net latency 2 cycles). Throughput is unchanged from the original single-cycle design.

Stage	Logic
1 — lookup / commit	key build → CAM lookup → CAM commit (including per-template <code>last_ts</code> update via <code>access_new_ts</code>)
2a — encode register (<code>q_*</code>)	<code>delta_ts</code> (per-template) / <code>fits</code> / format-select / slot pack — REGISTERED
2b — output	drive the slot(s); RAW (tier-0) 3-beat expansion

Path	Latency	Throughput
Tier-1 record (CAM hit, Tier-1 fits)	1 record → 1 slot, 2 cycles in flight	1 record / cycle
Tier-0 record (CAM miss, or all Tier-1 overflow)	1 record → 3 slots, 2 cycles + 2 RAW beats	1 record / 3 cycles

There is **no CAM read-after-write hazard**: the commit action depends only on hit/miss (the stage-1 lookup), not on the stage-2 format. The next record's lookup one cycle later sees the committed state — exactly as in the single-cycle design. Bit-exact slot stream against the Python golden either way.

55.9.1 Per-template `delta_ts`

Earlier revisions measured `delta_ts` against a single global `r_last_ts`. With interleaved templates from multiple sources (the 4-channel STREAM characterization case), that global delta could go apparently-negative between two interleaved templates and escape to raw — collapsing compression.

The current encoder measures `delta_ts` against **the matched CAM entry's stored `last_ts`** (`cam_access_old_ts`), and writes back the current record's `source_ts[23:0]` to that slot via `access_new_ts` in the same cycle. The CAM's per-entry `r_ts[TS_WIDTH=24]` shifts in lockstep with the LRU move-to-front. The Python encoder/decoder were updated in lockstep — 24 / 24 golden + 1 / 1 RTL compressor + 3 / 3 group cosim tests pass, bit-exact.

TS_STORE_BITS = 24. Format-B (the 23-bit-delta format) needs 24 to *detect* its overflow — 16 silently aliases large gaps to wrong encodes.

55.9.2 Input skid

A 2-deep gaxi_skid_buffer registers the (source_ts, packet) feed into the compressor. The monbus aggregator's output skid sits far from this compressor's CAM in the Nexys A7 floorplan, and aggregator → in_key → 32-way CAM match/commit was the route-dominated 100 MHz worst path (WNS swung ±0.25 ns on placement alone). The input skid ends that long hop at a local flop.

55.9.3 pblock (Nexys A7 build only)

The expanded per-template CAM (32 entries × TS_STORE_BITS=24) crowded the descriptor monitor's column on Nexys A7 and pushed its internal trans_mgr/u_cam → r_alloc_cnt path route-bound. The characterization build adds a pblock_monbus constraint pinning the monbus group to CLOCKREGION_X0Y2:X0Y3 (out of column X1), so the monitor can recompact. The monitor → group crossing is registered through the input skid above, so the longer inter-region hop is fine. 100 MHz closes (WNS +0.012 ns, 0 failing endpoints). See projects/NexysA7/stream_characterization/ for the constraint file and floorplan details.

55.10 Verification Methodology

The acceptance criterion is **byte-identical equivalence** between the RTL slot stream and the Python golden encoder (bin/TBClasses/monbus/monbus_compressor.py) for the same input record sequence. There is no other golden — the Python class IS the spec.

Two layers of validation:

55.10.1 1. Unit-level (the compressor in isolation)

```
pytest val/amba/test_monbus_compressor.py -v
```

Two phases: - **Phase 1**: small synthesized stream that exercises all 4 slot tags and CAM eviction. ~9 slots, easy to debug. - **Phase 2**: the real-silicon dataset desc_axi_16desc_8ch_1MB.json (682 records → 770 golden slots). Cross-checks each slot bit-exact, plus asserts the per-tier statistics counters match the Python encoder's per-tier counts exactly.

```
$ pytest val/amba/test_monbus_compressor.py -v
```

```
...
INFO records=682, golden_slots=770
INFO rtl_a=628, rtl_b=10, rtl_c=0, tier0=44
INFO === Phase 2: PASS ===
INFO === ALL PHASES PASSED ===
```

55.10.2 2. Integration-level (compressor + AXIL writer + base/limit wrap)

```
pytest val/amba/test_monbus_axil_axil_group_compressed.py -v
```


Three phases: - Generous window (no wrap): 9 slots from the synthesized stream - Tight 8-slot window: forces a mid-stream wrap back to `cfg_base_addr` - Real-silicon dataset: 770 slots through the full `monbus_axil_axil_group` with `USE_COMPRESSION=1`

Both layers run as part of the standard amba regression and must pass before any compressor change merges.

55.11 Statistics Counters

Each output stat is a 32-bit registered counter. They saturate (do not wrap) at `0xFFFF_FFFF` so a long-running capture never silently rolls back to 0.

Counter	Increments when
<code>stat_tier1_a</code>	Format A slot emitted
<code>stat_tier1_b</code>	Format B slot emitted
<code>stat_tier1_c</code>	Format C slot emitted
<code>stat_tier0</code>	Tier-0 RAW escape emitted (3 slots)
<code>stat_cam_miss</code>	Tier-0 escape caused by CAM miss
<code>stat_delta_ts_ovf</code>	Tier-0 escape caused by <code>delta_ts > 2²³</code>
<code>stat_event_data_ovf</code>	Tier-0 escape caused by <code>event_data > 2⁴⁰</code> (<code>delta_ts</code> fit)
<code>stat_ed_delta_ovf</code>	Tier-0 escape caused by <code>ed_delta</code> out of $\pm 2^{39}$

The host firmware reads these via a configurable register block at the end of a capture run to characterize compression effectiveness per workload. The ratio $(\text{stat_tier1_a} + \text{stat_tier1_b} + \text{stat_tier1_c}) / \text{total_records}$ is the hit rate; on the validation dataset it's 93.5 %.

55.12 Validation Numbers (locked dataset)

Workload: `desc_axi_16desc_8ch_1MB` — 8 channels × 16 descriptors × 1 MB DMA, descriptor-fetch monitor on a Nexys A7.

Metric	Value
Input records	682
Output slots	770

Metric	Value
Compression ratio	2.66×
Tier-1 A hits	628 (92.1 %)
Tier-1 B hits	10 (1.5 %)
Tier-1 C hits	0 (0 %)
Tier-0 escapes	44 (6.5 %)
└ caused by CAM miss	32
└ caused by delta_ts overflow	12
└ caused by event_data overflow	0
└ caused by ed_delta overflow	0
Distinct templates seen	32 (perfectly fits CAM=32)

The dataset and acceptance recipe live at [projects/NexysA7/stream_characterization/reports/compression_dataset/](https://projects.nexysa7.com/stream_characterization/reports/compression_dataset/).

55.13 Related Modules

Module	Role
monbus_cam_pipe	Pipelined 32-entry LRU CAM — production CAM in the compressor
monbus_cam	Single-cycle reference CAM (superseded; kept for comparison)
monbus_halfbeat_packer.sv	Pairs two 30-bit half-slots into one 64-bit beat downstream of the compressor (enabled by HALF_BEAT_EN=1)
monbus_group family	Host of the compressor, plus the master writer that drains slots to memory
sdpram_slave_axil_axil	Typical SRAM-ring backend for the compressed slot stream
bin/TBClasses/monbus/monbus_compressor.py	Python golden encoder/decoder — the format spec

55.14 Test

- **Acceptance (byte-identical vs golden):**
val/amba/test_monbus_compressor.py
- **End-to-end (with AXIL writer + wrap):**
val/amba/test_monbus_axil_axil_group_compressed.py
- **CAM sub-module:** val/amba/test_monbus_cam.py

Run all three:

```
pytest val/amba/test_monbus_cam.py \
       val/amba/test_monbus_compressor.py \
       val/amba/test_monbus_axil_axil_group_compressed.py -v
```

56 MonBus Group Core

Module: monbus_group_core.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

56.1 Overview

The monbus_group_core module is the protocol-agnostic heart of the monbus_*_*_group family. It receives a single monitor-bus (MonBus) stream plus a side-band timestamp, applies per-protocol filter masks, and routes accepted packets into one of two destinations: a record-granular error/interrupt FIFO drained over an AXI4-shaped slave-read port, or a beat-granular write FIFO drained over an AXI4-shaped master-write port with watermark and timeout flushing.

This module is the single source of truth for filtering, FIFO management, optional compression, and the burst writer/slicer state machines. The monbus_<p1>_<p2>_group.sv wrappers are pure structural adapters that bridge its two AXI4-shaped FUB ports into protocol-specific leaf skids.

56.1.1 Key Features

- Single MonBus ingress with per-protocol filter masks (AXI, AXIS, CORE)
- Free-running timestamp generator distributed to every monitor (mon_time_out)
- Error/interrupt FIFO: record-granular (192-bit records = 128-bit packet + 64-bit timestamp)
- Write FIFO: beat-granular (one entry = one 64-bit beat)
- AXI4-shaped slave-read drain with burst AR support and a 3-slice record slicer
- AXI4-shaped master-write burst writer with address-window, 4KB, and watermark controls

- Runtime-selectable compression (`cfg_compress_en`) with optional half-beat packing
 - Raw mode emits complete 24-byte (3-beat) records; compressed mode emits self-tagged 8-byte slots
 - Timing-closed 3-stage geometry pipeline for burst planning at 100 MHz
-

56.2 Module Purpose

Aggregated monitor packets must be delivered to two consumers with different needs: a CPU interrupt handler that walks recent error records, and a bulk-capture memory buffer that stores a long trace. This core splits the filtered stream between an error FIFO (read on demand by an IRQ handler) and a write FIFO (flushed to memory as AXI bursts), and handles all the address arithmetic, record framing, and optional compression in one place.

Use Cases: - Central MonBus capture block behind `monbus_arbiter` in an SoC monitoring subsystem - Streaming error records to a CPU IRQ handler while bulk-capturing a full trace to DRAM - Compressed trace capture to fit more monitor history in a fixed buffer - Building protocol-specific group wrappers (AXI4/AXI4, AXI4/AXIL, AXIL/AXI4, AXIL/AXIL) on a shared core

Key Benefit: All the hard logic — filtering, dual-FIFO routing, burst geometry, compression — lives once in this core; the four protocol-pair wrappers reduce to thin structural adapters, so a bug fix or feature lands in one file for every group.

56.3 Parameters

Parameter	Type	Default	Description
<code>FIFO_DEPTH_ERR</code>	int	64	Error FIFO depth in 192-bit records
<code>FIFO_DEPTH_WRITE</code>	int	96	Write FIFO depth in 64-bit beats (beat-granular)
<code>ADDR_WIDTH</code>	int	32	Address width for both FUB ports
<code>AXI_ID_WIDTH_M</code>	int	1	Master-write ID width (1 in AXIL builds)
<code>AXI_ID_WIDTH_S</code>	int	1	Slave-read ID width (1 in AXIL builds)
<code>MAX_BURST_BEATS</code>	int	1	Max beats per master-

Parameter	Type	Default	Description
FLUSH_TIMEOUT_CYCLES	int	1024	write sub-burst (1 for AXIL, up to 256 for AXI4) Cycles since last accepted W beat before forcing a flush
NUM_PROTOCOLS	int	3	Informational only
USE_COMPRESSION	int	0	0 = raw 3-beat records only; 1 = elaborate the compressor hardware
HALF_BEAT_EN	int	0	1 = pack two 30-bit half-slots per beat (requires USE_COMPRESSION==1)

56.4 Port Groups

56.4.1 Clock, Reset, and Clear

Port	Direction	Width	Description
axi_aclk	input	1	Core clock
axi_aren	input	1	Active-low asynchronous reset
cam_clear	input	1	Synchronous clear of the compressor template CAM + stats (no effect when USE_COMPRESSION==0); pulse when idle

56.4.2 MonBus Ingress and Timestamp

Port	Direction	Width	Description
monbus_valid	input	1	Incoming packet valid
monbus_ready	output	1	Core ready to accept the packet
monbus_packet	input	monitor_packet_t (128)	Monitor packet

Port	Direction	Width	Description
monbus_timestamp	input	monbus_timestamp_t (64)	Side-band timestamp for the packet
mon_time_out	output	monbus_timestamp_t (64)	Free-running counter; drive to every wrapper's i_mon_time

56.4.3 Status / IRQ / Debug

Port	Direction	Width	Description
irq_out	output	1	Asserted while the error FIFO is non-empty
err_fifo_full	output	1	Error FIFO cannot accept a write
write_fifo_full	output	1	Write FIFO cannot accept a beat
err_fifo_count	output	16	Error FIFO occupancy (records)
write_fifo_count	output	16	Write FIFO occupancy (beats)

56.4.4 Address Window and Flush Configuration

Port	Direction	Width	Description
cfg_base_addr	input	ADDR_WIDTH_H	Base address of the master-write capture window
cfg_limit_addr	input	ADDR_WIDTH_H	Inclusive limit address of the capture window
cfg_flush_watermark	input	16	Beat count in the write FIFO that triggers a flush burst
cfg_compr	input	1	Runtime compression

Port	Direction	Width	Description
ess_en			select (meaningful only when USE_COMPRESSION==1); hold stable while the write path is active

56.4.5 Per-Protocol Filter Masks

Three protocol groups (AXI = protocol 0, AXIS = protocol 1, CORE = protocol 4), each with a packet-drop mask, an error-select mask, and per-packet-type event masks. All are 16 bits.

Port	Direction	Width	Description
cfg_axi_pkt_mask	input	16	AXI: per-packet-type drop mask (bit = packet type)
cfg_axi_err_select	input	16	AXI: per-packet-type route-to-error-FIFO select
cfg_axi_err_mask	input	16	AXI: per-event-code mask for Error packets
cfg_axi_timeout_mask	input	16	AXI: per-event-code mask for Timeout packets
cfg_axi_completion_mask	input	16	AXI: per-event-code mask for Completion packets
cfg_axi_threshold_mask	input	16	AXI: per-event-code mask for Threshold packets
cfg_axi_perf_mask	input	16	AXI: per-event-code mask for Perf packets
cfg_axi_addr_match_mask	input	16	AXI: per-event-code mask for AddrMatch packets
cfg_axi_debug_mask	input	16	AXI: per-event-code mask for Debug packets
cfg_axis_pkt_mask	input	16	AXIS: per-packet-type drop mask
cfg_axis_err_select	input	16	AXIS: per-packet-type route-to-error-FIFO select

Port	Direction	Width	Description
cfg_axis_error_mask	input	16	AXIS: per-event-code mask for Error packets
cfg_axis_timeout_mask	input	16	AXIS: per-event-code mask for Timeout packets
cfg_axis_compl_mask	input	16	AXIS: per-event-code mask for Completion packets
cfg_axis_credit_mask	input	16	AXIS: per-event-code mask for Credit packets
cfg_axis_channel_mask	input	16	AXIS: per-event-code mask for Channel packets
cfg_axis_stream_mask	input	16	AXIS: per-event-code mask for Stream packets
cfg_core_pkt_mask	input	16	CORE: per-packet-type drop mask
cfg_core_err_select	input	16	CORE: per-packet-type route-to-error-FIFO select
cfg_core_error_mask	input	16	CORE: per-event-code mask for Error packets
cfg_core_timeout_mask	input	16	CORE: per-event-code mask for Timeout packets
cfg_core_compl_mask	input	16	CORE: per-event-code mask for Completion packets
cfg_core_thresh_mask	input	16	CORE: per-event-code mask for Threshold packets
cfg_core_perf_mask	input	16	CORE: per-event-code mask for Perf packets
cfg_core_debug_mask	input	16	CORE: per-event-code mask for Debug packets

56.4.6 Compressor Statistics

Port	Direction	Width	Description
mon_compressor_stat_tier1_a	output	32	Tier-1 compression counter A (0 in raw mode)
mon_compressor_stat_tier1_b	output	32	Tier-1 compression counter B
mon_compressor_stat_tier1_c	output	32	Tier-1 compression counter C
mon_compressor_stat_tier0	output	32	Tier-0 (raw escape) count
mon_compressor_stat_cam_miss	output	32	Template CAM miss count
mon_compressor_stat_delta_ts_ovf	output	32	Delta-timestamp overflow count
mon_compressor_stat_event_data_ovf	output	32	Event-data overflow count
mon_compressor_stat_ed_delta_ovf	output	32	Event-data-delta overflow count

56.4.7 AXI4-Shaped Master-Write FUB (bulk capture)

Port	Direction	Width	Description
fub_m_awid	output	AXI_ID_WIDTH_M	Write address ID (driven to 0)
fub_m_awaddr	output	ADDR_WIDTH_H	Write burst start address
fub_m_awlen	output	8	Sub-burst length (beats – 1)
fub_m_awsiz	output	3	Fixed 3 (8 bytes/beat)

Port	Direction	Width	Description
fub_m_awburst	output	2	Fixed INCR (2'b01)
fub_m_awvalid	output	1	AW valid
fub_m_awready	input	1	AW ready
fub_m_wdata	output	64	Write beat data (from write FIFO)
fub_m_wstrobe	output	8	Write strobes (fixed 8'hFF)
fub_m_wlast	output	1	Last beat of the sub-burst
fub_m_wvalid	output	1	W valid
fub_m_wready	input	1	W ready
fub_m_bid	input	AXI_ID_WIDTH_M	Write response ID (ignored)
fub_m_bresp	input	2	Write response (ignored)
fub_m_bvalid	input	1	B valid
fub_m_bready	output	1	B ready

56.4.8 AXI4-Shaped Slave-Read FUB (error drain)

Port	Direction	Width	Description
fub_s_arid	input	AXI_ID_WIDTH_S	Read address ID
fub_s_arddr	input	ADDR_WIDTH_H	Read address (unused; drain is a stream)
fub_s_arlen	input	8	Burst length (beats – 1)
fub_s_arsize	input	3	Burst size (unused; always 64-bit)
fub_s_arburst	input	2	Burst type (unused; always INCR)
fub_s_arv	input	1	AR valid

Port	Direction	Width	Description
alid			
fub_s_arry eady	output	1	AR ready (accepts only when idle)
fub_s_rid	output	AXI_ID_WIDTH	Read data ID (echoes burst ID)
fub_s_rdata	output	64	Sliced record beat
fub_s_response	output	2	Read response (fixed OKAY)
fub_s_rlast	output	1	Last beat of the burst
fub_s_rvalid	output	1	R valid
fub_s_rready	input	1	R ready

56.5 Functional Description

56.5.1 Free-Running Timestamp

A counter (`r_ts_counter`) increments every clock and is exposed as `mon_time_out`. Every monitor wrapper drives its `i_mon_time` from this output so all packets across the group share one time base.

56.5.2 Packet Filtering

Each incoming packet is decomposed into `pkt_type`, `pkt_protocol` (bits [108:105]), and `pkt_event_code`. Filtering runs in three tiers per protocol:

1. **Drop mask** (`cfg_*_pkt_mask[pkt_type]`) — if set, the packet is dropped outright.
2. **Error-select** (`cfg_*_err_select[pkt_type]`) — if set (and not dropped), the packet routes to the error FIFO; otherwise it routes to the write path.
3. **Event mask** — when the event code's high nibble is 0 (`ec_in_mask_range`), the low nibble indexes a per-packet-type event mask; a set bit forces a drop and clears the error-FIFO routing.

An unrecognized protocol drops the packet. The final routing decision is `pkt_to_write_path = !pkt_drop && !pkt_to_err_fifo`.

56.5.3 Error FIFO and Slave-Read Drain

Accepted error packets are stored as 192-bit records {timestamp, packet} in `u_err_fifo`. `irq_out` asserts whenever the FIFO is non-empty. The slave-read drain presents each record as three 64-bit slices over a burst read:

- slice 0 = {tag=4'h0, source_ts[59:0]}
- slice 1 = packet[127:64]
- slice 2 = packet[63:0] (record is popped here)

AR is accepted only at slice 0 with at least one record buffered; `rvalid` drops mid-burst if the FIFO underruns and resumes when a new record arrives; `rlast` asserts on the (arlen+1)-th beat. The CPU should size `arlen` as a multiple-of-3 minus one to land cleanly on record boundaries.

56.5.4 Write Path — Raw Expander vs Compressor

Two producers feed the beat-granular write FIFO, selected at runtime by `w_use_comp = (USE_COMPRESSION != 0) && cfg_compress_en`:

- **Raw expander** (always elaborated): a 3-state FSM (EXP_TS/EXP_HI/EXP_LO) pushes {ts, pkt_hi, pkt_lo} beats atomically — a record is never split across backpressure. Per-beat `wr_ready` is used rather than a “3 slots free” precheck to avoid a count → valid → count combinational loop.
- **Compressor** (elaborated only when `USE_COMPRESSION==1`): `monbus_compressor` sits between the input and the FIFO, emitting one self-tagged 64-bit slot per record. A 2-deep input skid registers (source_ts, packet) right at the compressor boundary so the route-dominated aggregator → CAM path ends at a local flop (+1 cycle latency, throughput preserved, slot stream bit-exact). When `HALF_BEAT_EN==1`, `monbus_halfbeat_packer` packs two 30-bit half-slots per beat downstream of the compressor.

Only one path is active at a time; the two FIFO-write outputs are muxed by `w_use_comp`. `monbus_ready` is driven by whichever path accepted the packet (drop, error-FIFO write ready, or write-path term).

56.5.5 Master-Write Burst Writer

The write FIFO is flushed to memory as AXI4 bursts. A flush is triggered when either the FIFO occupancy reaches `cfg_flush_watermark` or `FLUSH_TIMEOUT_CYCLES` elapse since the last accepted W beat — and at least one whole record (`BEATS_PER_UNIT`, 3 in raw mode, 1 compressed) is available.

Burst length is bounded by the minimum of FIFO occupancy, MAX_BURST_BEATS, distance to `cfg_limit_addr`, and distance to the next 4KB boundary, then rounded down to whole records. The address arithmetic is pipelined over 3 registered stages (the plan trails the stable `r_wr_addr`, which only moves in `WR_W`), keeping the wide window/4KB/round-to-record chain off the 100 MHz critical path. The fresh FIFO-occupancy cap is applied combinationally at commit so a stale count cannot short the burst. The mod-3 whole-record rounding uses `mod_3_compress` carry-save instances rather than a wide divider.

The writer FSM (`WR_IDLE` → `WR_AW` → `WR_W` → `WR_B`) emits as many `AW` + `N×W` + `B` sub-bursts as needed to drain the planned beat count, advancing the address one 8-byte beat per accepted `W`. When the current window/4KB region cannot fit a whole record, the writer rewinds `r_wr_addr` to `cfg_base_addr` and re-settles the geometry pipeline.

56.6 Usage Example

`monbus_group_core` is instantiated by the group wrappers, not directly. A wrapper connects the two AXI4-shaped FUBs to protocol leaves:

```
monbus_group_core #(
    .FIFO_DEPTH_ERR          (64),
    .FIFO_DEPTH_WRITE        (96),
    .ADDR_WIDTH               (32),
    .AXI_ID_WIDTH_M           (8),
    .AXI_ID_WIDTH_S           (8),
    .MAX_BURST_BEATS          (64),           // AXI4 build; use 1 for AXIL
    master_write
    .FLUSH_TIMEOUT_CYCLES     (1024),
    .USE_COMPRESSION          (0)
) u_core (
    .axi_aclk      (axi_aclk),
    .axi_aresetn   (axi_aresetn),
    .cam_clear     (cam_clear),

    .monbus_valid   (monbus_valid),
    .monbus_ready   (monbus_ready),
    .monbus_packet   (monbus_packet),
    .monbus_timestamp (monbus_timestamp),
    .mon_time_out    (mon_time_out),

    .cfg_base_addr   (cfg_base_addr),
    .cfg_limit_addr  (cfg_limit_addr),
    .cfg_flush_watermark (cfg_flush_watermark),
    .cfg_compress_en (cfg_compress_en),
    // ... per-protocol masks ...
```

```

// AXI4-shaped master-write FUB -> axi4_master_wr leaf
.fub_m_awaddr (wr_fub_awaddr), .fub_m_awlen (wr_fub_awlen),
.fub_m_awvalid(wr_fub_awvalid), .fub_m_awready(wr_fub_awready),
.fub_m_wdata  (wr_fub_wdata),  .fub_m_wlast (wr_fub_wlast),
// ...

// AXI4-shaped slave-read FUB <- axi4_slave_rd leaf
.fub_s_arlen  (rd_fub_arlen),  .fub_s_arvalid(rd_fub_arvalid),
.fub_s_rdata  (rd_fub_rdata),  .fub_s_rlast (rd_fub_rlast)
// ...
);

```

56.7 Design Notes

56.7.1 Hold `cfg_compress_en` Stable

`cfg_compress_en` changes both the record framing (raw 3-beat vs compressed 1-beat) and `BEATS_PER_UNIT`. Switching it mid-stream would mix formats in the write FIFO and corrupt burst sizing. Program it once before monitoring starts.

56.7.2 AXIL Builds Use the Same Core

AXIL group wrappers instantiate this same AXI4-shaped core with `MAX_BURST_BEATS=1` and ID widths of 1, supplying single-beat defaults (`len=0`, `size=$clog2(8)`, `INCR`, `id=0`) at the leaf. The core does not need an AXIL-specific variant.

56.7.3 FIFO Granularity Difference

The error FIFO is record-granular (192-bit); the write FIFO is beat-granular (64-bit). `err_fifo_count` reports records while `write_fifo_count` reports beats — do not compare them directly.

56.7.4 Timing-Motivated Structure

Several structural choices exist purely to close 100 MHz timing: the 3-stage geometry pipeline, local registering of `cfg_base_addr/cfg_limit_addr` with a `max_fanout` cap, the registered raw FIFO count shared by both the flush trigger and the burst cap, and the compressor input skid. A prior split (fresh trigger against a lagged cap) shorted bursts to 21/24 beats — both now derive from the same registered count.

56.7.5 4KB and Window Compliance

The burst is always sized so its last byte stays at or below `cfg_limit_addr` and never crosses a 4KB boundary, satisfying AXI4 addressing rules without mid-burst wrapping.

56.8 Related Modules

56.8.1 Used By

- **monbus_axi4_axi4_group.sv** — AXI4 slave-read + AXI4 master-write wrapper
- **monbus_axi4_axil_group.sv** — AXI4 slave-read + AXIL master-write wrapper
- **monbus_axil_axi4_group.sv** — AXIL slave-read + AXI4 master-write wrapper
- **monbus_axil_axil_group.sv** — AXIL slave-read + AXIL master-write wrapper

56.8.2 Uses

- **gaxi_fifo_sync.sv** — error FIFO and write FIFO
- **gaxi_skid_buffer.sv** — compressor input skid
- **monbus_compressor.sv** — optional packet compressor (present when `USE_COMPRESSION==1`)
- **monbus_halfbeat_packer.sv** — optional half-beat packer (`HALF_BEAT_EN==1`)
- **mod_3_compress.sv** — carry-save mod-3 for whole-record rounding
- **monitor_common_pkg** — packet types, protocols, `monitor_packet_t`, `monbus_timestamp_t`

56.8.3 See Also

- **monbus_arbiter.sv** — aggregates multiple monitor streams upstream of this core
 - **monbus_group.md** — group-level overview
-

56.9 References

56.9.1 Source Code

- RTL: `rtl/amba/monitor/monbus_group_core.sv`
- Package: `rtl/amba/includes/monitor_common_pkg.sv`

56.9.2 Documentation

- Architecture: `docs/markdown/RTLAmba/shared/README.md`

- Compressor: docs/markdown/RTLAmba/monbus_compressor.md
 - Packet Format:
docs/markdown/RTLAmba/includes/monitor_package_spec.md
-

Last Updated: 2026-07-15

56.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

57 Monitor Bus Group Family

Modules: - monbus_group_core.sv — protocol-agnostic backend (filter, FIFOs, watermark/timeout burst writer, slice-counter slave-read drain) - monbus_axil_axil_group.sv — wrapper: AXIL slave-read + AXIL master-write - monbus_axil_axi4_group.sv — wrapper: AXIL slave-read + AXI4-burst master-write - monbus_axi4_axil_group.sv — wrapper: AXI4-burst slave-read + AXIL master-write - monbus_axi4_axi4_group.sv — wrapper: AXI4-burst slave-read + AXI4-burst master-write

Location: rtl/amba/shared/ **Category:** Monitor Bus → AXI / AXIL Aggregation **Status:** Production Ready

57.1 Overview

The monbus_group family is the **delivery layer** for the monitor bus. It takes a single monbus stream — already arbitrated upstream if multiple sources need to be merged — and:

1. **Filters** packets per protocol (drop / route-to-err-FIFO / route-to-write-FIFO)
2. **Drains** the error FIFO over a slave-read port for CPU IRQ handlers
3. **Streams** the write FIFO over a master-write port into a memory ring
4. **Optionally compresses** the write-path traffic before it lands in memory (gated by USE_COMPRESSION=1)

The family replaces the previous single monbus_axil_group.sv module. The new design exposes the same behavior under four protocol-permutation wrappers, so callers can pick the exact slave/master shape their fabric demands without tying off “fake AXI4” fields on AXIL sides.

57.2 Architecture

The diagram below is for the AXIL/AXIL variant; the other three wrappers are structurally identical except for the leaf skids and the FUB-to-leaf bridge (e.g., `axi4_slave_rd` instead of `axil4_slave_rd`).

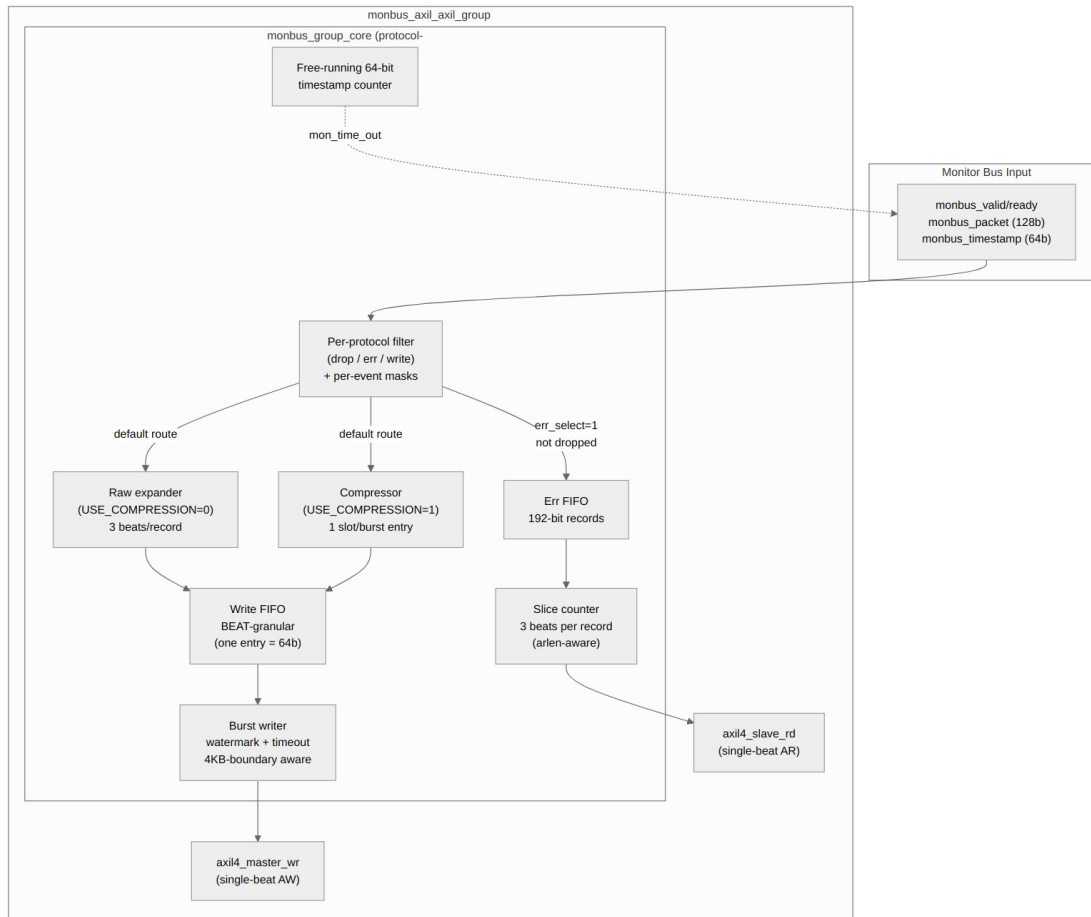


Diagram 25

The module has **no built-in arbitration** — it is a single-input block. Callers with N upstream sources (e.g., RAPIDS source+sink) instantiate a separate `monbus_arbiter` upstream to merge their streams before feeding the group. Keeping arbitration orthogonal to capture means the family never has to duplicate the arbiter for each consumer's input count.

57.3 Why one core + four wrappers?

SystemVerilog cannot conditionally include or exclude ports from a single module's port list. The port list is a static syntactic construct, fixed at module declaration. So to give each protocol

combination an exact port shape — no spurious AXI4-only fields for the caller to tie off — the family is split:

- **One common backend** (`monbus_group_core.sv`) carries the filter, FIFOs, watermark/timeout burst-writer FSM, slice-counter drain, and optional compressor. Its FUBs are AXI4-shaped on both sides (`id` / `awlen` / `awsize` / `awburst` / `wlast` / `arlen` / `rlast` / etc.). AXIL wrappers feed single-beat defaults (`len=0`, `size=3`, `burst=INCR`, `id=0`) and force `MAX_BURST_BEATS=1`.
- **Four thin wrappers**, each with the right protocol-shaped port list, that directly instantiate the matching leaf skids (`axi4_slave_rd` / `axil4_slave_rd` and `axi4_master_wr` / `axil4_master_wr`) and bridge their FUBs to the core's FUB.

Wrapper	Slave-read shape	Master-write shape	MAX_BURST_BEATS (default)
<code>monbus_axil_axil_group</code>	AXIL <code>s_axil_ar*/r*</code>	AXIL <code>m_axil_aw*/w*/b*</code>	1
<code>monbus_axil_axi4_group</code>	AXIL <code>s_axil_ar*/r*</code>	AXI4 <code>m_axi_aw*/w*/b*</code> (full)	64
<code>monbus_axi4_axil_group</code>	AXI4 <code>s_axi_ar*/r*</code> (full)	AXIL <code>m_axil_aw*/w*/b*</code>	1
<code>monbus_axi4_axi4_group</code>	AXI4 <code>s_axi_ar*/r*</code> (full)	AXI4 <code>m_axi_aw*/w*/b*</code> (full)	64

Callers pick the wrapper module name matching their fabric. The bare `monbus_group_core` is not intended to be instantiated directly — its AXI4-shaped FUBs are an internal contract.

57.4 Common Parameters

All wrappers expose the same set of behavioral knobs (per-wrapper parameters like `AXI_ID_WIDTH` / `AXI_USER_WIDTH` are only present where at least one side is AXI4):

Parameter	Default	Notes
<code>FIFO_DEPTH_ERR</code>	64	Error FIFO depth, in records (192-bit entries).
<code>FIFO_DEPTH_WRITE</code>	96	Write FIFO depth, in

Parameter	Default	Notes
ADDR_WIDTH	32	beats (64-bit entries). 96 beats = 32 raw records. Address width on both slave and master ports.
FLUSH_TIMEOUT_CYCLES	1024	Cycles since the last accepted W handshake before the writer fires a timeout-driven flush.
MAX_BURST_BEATS	1 (AXIL master) / 64 (AXI4 master)	Maximum beats per master-write burst. AXI4 protocol max is 256.
NUM_PROTOCOLS	3	Informational — AXI, AXIS, CORE filter configs are unconditional.
USE_COMPRESSION	0	Elaboration: 0 omits the compressor entirely (raw-only build, minimum area); 1 elaborates the compressor so it can be selected at runtime via <code>cfg_compress_en</code> . The raw 3-beat-per-record path is always elaborated.
HALF_BEAT_EN	0	Elaboration: pack two 30-bit half-slots per 64-bit beat downstream of the compressor (<code>monbus_halfbeat_packer</code>), pushing tier-1 bandwidth from 1 beat/record toward 0.5 beat/record. Requires <code>USE_COMPRESSION=1</code> ;

Parameter	Default	Notes
SKID_DEPTH_AR/R/AW/W/B	2/4/2/2/2	folds away when 0. Skid buffer depths for each channel. AXI4 wrappers default SKID_DEPTH_W=4 to absorb burst W bursts.

57.4.1 Runtime config

In addition to the elaboration parameters, the group exposes runtime inputs that pick mode and shape on the fly:

Input	Width	Notes
cfg_compress_en	1	Runtime compression enable. Only effective when USE_COMPRESSION=1 (otherwise the compressor isn't elaborated and the bit folds away). Must be stable while the write path is active.
cam_clear	1	Synchronous CAM clear (level-sensitive, sample one cycle while idle). Empties the compressor template CAM and zeroes the compressor stat counters so the next run starts with a fresh hit/miss profile — without asserting axi_aresetn. Tied off in raw-only builds (USE_COMPRESSION=0) where there is no CAM.

Input	Width	Notes
		Stream uses this on CTRL[4].
cfg_base_addr / cfg_limit_addr	ADDR_WIDTH	Master-write address window.
cfg_flush_watermark	16	Master-write FIFO depth (in beats) at which a flush is triggered.
cfg_<proto>*_mask / cfg_<proto>_err_select	16 each	Per-protocol filter and err-FIFO routing masks; see the dedicated section below.

57.5 Beat Layout

The write-FIFO and slave-read drain are **beat-granular** (one queue entry = one 64-bit beat). Two emit modes:

57.5.1 Raw mode (cfg_compress_en = 0, or USE_COMPRESSION = 0 build)

Each accepted monbus record becomes three sequential 64-bit beats in the write FIFO:

```
beat 0 = {tag[3:0]=4'h0, source_ts[59:0]}
beat 1 = packet[127:64]
beat 2 = packet[63:0]
```

The expander is **atomic per record**: once it commits to driving the ts beat, it drives beats 1 and 2 before accepting a new record. This means partial records never appear in the FIFO, and the master-write burst writer can safely round burst lengths down to a multiple of 3.

The tag nibble in beat 0 is reserved for future on-the-wire format variants. The raw writer hardwires it to 4'h0.

57.5.2 Compressed mode (cfg_compress_en = 1, requires USE_COMPRESSION = 1 build)

monbus_compressor consumes (packet, source_ts) records via its in-handshake and emits 64-bit self-tagged slots via its out-handshake. Each slot becomes one beat directly in the write FIFO. Slot tags (bits [63:60]):

```
4'h0 = raw expansion beat (Tier-0 fallback)
4'h1 = Tier-1 format A
4'h2 = Tier-1 format B
```

```
4'h3 = Tier-1 format C
4'h4..4'hF = reserved
```

See `monbus_compressor.md` for the encoder's internal state and the byte-exact Python golden.

The raw expander is **always elaborated**; the compressor is elaborated only when `USE_COMPRESSION=1`. The two paths' FIFO-write outputs and `monbus_ready` are muxed on `w_use_comp = (USE_COMPRESSION != 0) && cfg_compress_en`. Only one path is ever active; the inactive one is gated off at its input. `BEATS_PER_UNIT` (3 in raw, 1 in compressed) is therefore a runtime quantity (`w_beats_per_unit`), feeding the burst-writer geometry described below.

Note on switching modes: `cfg_compress_en` must be stable while there is unflushed data in the write FIFO. Toggling it mid-stream would mix raw 3-beat records with compressed 1-beat slots at the same memory addresses — the host decoder couldn't reassemble them. Software should drain the FIFO (poll `write_fifo_count` to zero) before flipping the bit.

57.5.3 Slave-read drain (both modes)

The slave-read port returns the same 3-beat layout per err-FIFO record, sliced by an internal counter. In AXI4 mode, `arlen+1` beats are emitted across the burst with `r_last` asserting on the final beat; the slicer advances through records, and the FIFO entry is popped only when slice 2 (`SLICE_PKT_LO`) fires. In AXIL mode each AR returns one beat.

57.6 Master-Write Behavior

The burst writer fires when **either**:

- `flush_trigger_watermark: r_fifo_beats >= cfg_flush_watermark`
- `flush_trigger_timeout: r_timeout_cnt >= FLUSH_TIMEOUT_CYCLES`

...and at least `BEATS_PER_UNIT` beats are available (3 in raw mode, 1 in compressed mode).

57.6.1 Burst geometry: 3-stage pipeline + fresh-FIFO cap at commit

The drain-plan math was originally a single combinational chain off `r_wr_addr` feeding straight back into `r_wr_addr` — the 100 MHz critical path on Nexys A7 (-1) (WNS -7.06 ns post-route). Since `r_wr_addr` is stable while the writer sits in `WR_IDLE` (only `WR_W` advances it) and the write FIFO only grows there, the **address geometry** is now a **3-stage registered pipeline**:

```
stage 1 (caps):          bytes_to_limit / bytes_to_4kb from
r_wr_addr,              plus the rewind decision (geom_addr).
                         min-cap tree + whole-record /3 rounding
stage 2 (planned):       via u_mod3_geo (see "mod-3 rounding" below).
```

```
stage 3 (rounded + addr): r_plan_geo_units (address-feasible whole-
                           record count) + r_plan_addr (eff_addr).
```

A `geom_valid` settle counter holds the plan invalid for the first few cycles after `r_wr_addr` moves so the pipeline can reflect the settled address before the FSM commits.

57.6.1.1 *Rewind-snap: in-window with no record-room left*

If a flush fires while `r_wr_addr` is in-window but the remaining bytes to the next 4KB boundary cannot fit a whole record, the pipeline produces `r_plan_ok=false` (units rounded down to zero) and the writer would otherwise wedge in `WR_IDLE`. The FSM detects this case (`do_flush && geom_valid && !r_plan_ok && r_wr_addr != cfg_base_addr`) and **snaps `r_wr_addr` to `cfg_base_addr`**, staying in `WR_IDLE` so the pipeline re-settles with fresh geometry. The next pipeline output (after the settle counter expires) has `r_plan_ok=true` provided `cfg_base_addr` itself has at least one whole record's worth of room before the next 4KB line — the host's responsibility.

This case is hit by the AXIL/AXIL master_write stress in Phase 5, where `cfg_base_addr` is intentionally placed only a few beats below a 4KB boundary so the very first drain has to wrap-snap before it can emit anything.

The **FRESH FIFO-occupancy cap** is applied combinationally at the `WR_IDLE` commit (not inside the pipeline):

```
// Pipeline produces r_plan_geo_units = address-feasible whole-record
beats.
// At commit time, cap by the CURRENT FIFO contents:
units_commit = min(r_plan_geo_units, w_fifo_units)
```

where `w_fifo_units = r_fifo_beats - (r_fifo_beats mod 3)` (or `r_fifo_beats` itself in compressed mode where the unit is 1 beat). This matters because the FIFO keeps filling while the writer sits in `WR_IDLE`; a stale (3-cycle-old) FIFO count would short the burst — for example, capping a watermark-triggered 24-beat plan to 21. The fresh path starts from a single counter (no address subtract/shift), so it doesn't recreate the critical path.

Per-AW sub-burst sizing happens **inside the FSM**, capping each AW by `MAX_BURST_BEATS`:

```
sub_burst_beats = min(remaining_in_cycle, MAX_BURST_BEATS)
awlen           = sub_burst_beats - 1
```

The split between drain-cycle plan (whole records) and per-AW sub-burst (`MAX_BURST_BEATS`) is what makes the family work for both AXIL (`MAX_BURST_BEATS = 1`) and AXI4 masters in raw mode (`BEATS_PER_UNIT = 3`):

- **AXI4 master, raw mode:** one drain cycle $\approx$ one large sub-burst. e.g. 24 beats = 8 records in a single AW + $24 \times W + B$. Throughput- optimal.
- **AXIL master, raw mode:** one drain cycle = N single-beat sub-bursts. Each record requires three AW + $1 \times W + B$ handshakes at consecutive addresses.

The memory image is identical to the AXI4 case; only the address-channel handshake count differs.

57.6.2 Address-window behavior: wrap, not saturate

The master-write address is **circular** within `[cfg_base_addr, cfg_limit_addr]`. When the writer rolls past the window's end (or starts out of window, e.g. `r_wr_addr = 0` immediately after reset), the geometry pipeline finds `r_wr_addr` outside `[cfg_base_addr, cfg_limit_addr]` and computes against a pre-rewound `geom_addr = cfg_base_addr`. The final `r_plan_addr` reflects that rewind — so the next burst starts at `cfg_base_addr` and the dump ring keeps moving forever.

This is **not** a saturation behavior. The writer never stalls at `cfg_limit_addr`; it always wraps. Any host-side bookkeeping (SRAM-dump write pointer, ring-buffer position cursor, etc.) **must match this wrap behavior** — saturating a host-visible pointer at the SRAM cap will desynchronize it from the actual ring state. The `stream_char_harness.sv` debug pointer was fixed for exactly this reason on 2026-06-14 (d82b1e04); see that commit message for the hardware-confirmed symptom.

57.6.3 mod-3 rounding (no /3 operator)

Rounding a beat count down to a whole record ($X - (X \bmod 3)$) is done by two `mod_3_compress` instances (`u_mod3_geo`, `u_mod3_fifo`) — a carry-save-compressor implementation of $X \bmod 3$ via base-4 digit sum (each 2-bit group has weight $4^k \equiv 1 \bmod 3$). No `*` or `/` operators, so Vivado neither infers a DSP48 (a combinational path through a far-placed DSP was catastrophic) nor a CARRY4 iterative divider — both used to blow timing. Exhaustively verified for all 65 536 inputs (`val/common/test_mod_3_compress.py`).

57.6.4 Final port behavior

`awsize` is fixed at 3 (8 bytes per beat) and `awburst` at INCR (2'b01). `awlen` is `sub_burst_beats - 1`. `wlast` asserts on the final beat of each sub-burst.

57.7 Per-Protocol Filter Configuration

The filter is configured per protocol (AXI, AXIS, CORE). For each protocol the caller provides:

- `cfg_<proto>_pkt_mask[15:0]` — drop the packet if the bit at `pkt_type` is set.
- `cfg_<proto>_err_select[15:0]` — route to the err FIFO if the bit at `pkt_type` is set (and not dropped). All other surviving packets go to the write FIFO.

- `cfg_<proto>_<event_type>_mask[15:0]` — per-event-code mask. If the bit at `event_code[3:0]` is set, the packet is dropped regardless of the above. Per-protocol event categories differ:
 - **AXI:** error, timeout, compl, thresh, perf, addr, debug
 - **AXIS:** error, timeout, compl, credit, channel, stream
 - **CORE:** error, timeout, compl, thresh, perf, debug

Per-event masks are 16 bits but only `event_code[3:0]` indexes; codes ≥ 16 are treated as “not in mask range” (no masking applied). Protocols not in the supported set (APB=2, ARB=3) are always dropped.

57.8 Sub-modules Instantiated

Sub-module	Role
<code>axi4_slave_rd / axil4_slave_rd</code>	Slave-read skid (one per wrapper)
<code>axi4_master_wr / axil4_master_wr</code>	Master-write skid (one per wrapper)
<code>gaxi_fifo_sync × 2</code>	Err FIFO + write FIFO inside the core
<code>monbus_compressor</code> (conditional)	Compressed-mode encoder; only when <code>USE_COMPRESSION=1</code>
<code>monbus_halfbeat_packer</code> (conditional)	Pairs two 30-bit half-slots into one 64-bit beat downstream of the compressor; only when <code>HALF_BEAT_EN=1</code> (which itself requires <code>USE_COMPRESSION=1</code>)
<code>monbus_cam_pipe</code> (transitive)	Pipelined 49-bit-key LRU CAM with per-template <code>last_ts/last_data</code> ; replaces the unpipelined <code>monbus_cam</code> as the in-production CAM inside the compressor. See <code>monbus_cam</code> for the deprecation note.
<code>mod_3_compress × 2</code> (rtl/common/)	Geometry / FIFO whole-record rounding ($X - (X \bmod 3)$)
<code>math_adder_carry_save_nbit</code> (transitive)	3:2 compressor primitive used by <code>mod_3_compress</code>
<code>gaxi_skid_buffer</code> (inside compressor)	Input skid on the (<code>source_ts</code> , <code>packet</code>) feed; breaks the aggregator →

Sub-module	Role
	CAM long route

57.8.1 Canonical filelist

A single canonical filelist enumerates the group core's dependency tree so a new core dep lands in **one place**:

```
rtl/amba/filelists/monbus_group.f
```

It lists `math_adder_carry_save_nbit + mod_3_compress + monbus_cam + monbus_cam_pipe + monbus_compressor + monbus_halfbeat_packer + monbus_group_core`. All consumers (val/amba tests, RAPIDS / STREAM macro filelists) - f-include it rather than listing those sources inline.

57.9 Status / Debug Outputs

Common to all wrappers:

Output	Width	Meaning
irq_out	1	High whenever the err FIFO is non-empty
err_fifo_full	1	Err FIFO write port not ready
write_fifo_full	1	Write FIFO write port not ready
err_fifo_count	16	Err FIFO entry count (records)
write_fifo_count	16	Write FIFO entry count (beats)
mon_time_out	64	Free-running timestamp counter
mon_compressor_stat_*	32 each	Compressor statistics, only live when <code>USE_COMPRESSION=1</code> ; tied to 0 in raw mode

The compressor stat ports stay in the port list regardless of elaboration mode so wrapper layers see a consistent port surface; only their semantics differ.

57.10 Test

Verification lives in `val/amba/`:

- `test_monbus_axil_axil_group.py` — basic flow + err-FIFO drain on the AXIL/AXIL wrapper (slave-read coverage; master-write side is driven into a synthetic sink in this test).
- `test_monbus_axil_axil_group_compressed.py` — byte-exact compressed slot stream comparison against the Python Encoder golden across three phases (small synth stream, real-silicon 682-record dataset, wrap-window).
- `test_monbus_axil_axil_group_master_write.py` — raw-mode master-write coverage on the AXIL/AXIL wrapper. Three phases: watermark-driven flush (asserts $3*N$ beats at 8-byte stride starting at `cfg_base_addr`), timeout-driven flush, window wrap-back. The test that would have caught the AXIL-master raw-mode flush deadlock fixed on 2026-06-11.
- `test_monbus_axil_axi4_group.py` — AXI4 burst master-write coverage on the AXIL/AXI4 wrapper. Three phases: watermark-driven multi-beat burst, timeout-triggered flush, 4KB-boundary respect.
- `test_monbus_axi4_axil_group.py` — dedicated AXI4-slave + AXIL-master coverage. Phase 1 covers the master-write raw-mode flush; Phase 2 covers the AXI4 burst slave-read drain (asserts `rlast` timing across a 6-beat AR with custom `arid`).

```
pytest val/amba/test_monbus_axil_axil_group.py \  
       val/amba/test_monbus_axil_axil_group_compressed.py \  
       val/amba/test_monbus_axil_axil_group_master_write.py \  
       val/amba/test_monbus_axil_axi4_group.py \  
       val/amba/test_monbus_axi4_axil_group.py -v
```

`monbus_axi4_axi4_group` does not yet have a dedicated test; its master-write path is covered by `test_monbus_axil_axi4_group.py` (same master leaf) and its AXI4-burst slave-read drain is covered by `test_monbus_axi4_axil_group.py` Phase 2 (same slave leaf). Adding a direct test for the pure-AXI4 wrapper is a future-work item.

57.11 Migration from `monbus_axil_group`

The legacy `monbus_axil_group.sv` module is gone. Callers should migrate to the wrapper matching their fabric. Key port-surface changes:

1. **M_AXIL_DATA_WIDTH dropped; S_AXIL_DATA_WIDTH retained** — the master-write data path is locked at 64 bits, so M_AXIL_DATA_WIDTH is gone. S_AXIL_DATA_WIDTH survives on the two AXIL-slave-read wrappers (monbus_axil_axil_group, monbus_axil_axi4_group) with a default of 64 (one beat per record slice); set it to 32 to enable the built-in 2:1 read serializer for a 32-bit host crossbar (6 beats per record). The AXI4-slave-read wrappers have no such parameter.
2. **cfg_flush_watermark[15:0] is a new required input**. Set to 1 for the legacy “fire immediately” behavior; set higher to batch.
3. **err_fifo_count and write_fifo_count are now 16 bits** (were 8).
4. **FIFO_DEPTH_WRITE is in beats**, not records. 32 records of the legacy module = 96 beats in raw mode.

Example (AXIL/AXIL — the closest analog to the legacy module):

```
// Old
monbus_axil_group #(
    .FIFO_DEPTH_ERR      (64),
    .FIFO_DEPTH_WRITE    (32),    // records
    .S_AXIL_DATA_WIDTH   (64),
    .M_AXIL_DATA_WIDTH   (64),
    .USE_COMPRESSION      (0)
) u_group (...);

// New
monbus_axil_axil_group #(
    .FIFO_DEPTH_ERR      (64),
    .FIFO_DEPTH_WRITE    (96),    // beats (3x for raw mode)
    .FLUSH_TIMEOUT_CYCLES (1024),
    .USE_COMPRESSION      (0)
) u_group (
    .cfg_flush_watermark (16'd24), // 8 records worth, eager flush
    ...
);
```

For AXI4 burst capture (e.g., when the memory ring lives behind an AXI4 fabric and you want to bunch records into multi-beat bursts to amortize address-channel overhead), switch the wrapper to monbus_axil_axi4_group and pick MAX_BURST_BEATS to taste.

57.12 Related Modules

Module	Role
monbus_compressor	Bulk-trace encoder used when USE_COMPRESSION=1
monbus_cam_pipe	Pipelined LRU CAM backing the compressor (production)
monbus_cam	Unpipelined reference CAM (superseded by monbus_cam_pipe)
monbus_arbiter	Upstream multi-source merge (instantiate before this family if you have N>1 sources)
sdpram_slave_axil_axil	Canonical memory-ring backend for the master-write port (AXIL/AXIL variant pairs with sdpram_slave_axil_axil)
axi_monitor_base	Source of the monitor packets this family captures
axi_monitor_reporter	Per-protocol packet-emission frontend

58 MonBus Half-Beat Packer

Module: monbus_halfbeat_packer.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

58.1 Overview

The monbus_halfbeat_packer module packs two 30-bit half-slots into a single 64-bit beat, sitting downstream of monbus_compressor. The compressor emits one 64-bit beat per tier-1 record (a 66.7% reduction ceiling — 1 beat per 3-beat raw record); by pairing two compatible records into one beat, this packer pushes the reduction to as much as 83.3% (0.5 beat per record). It is bit-exact to the Python golden model Encoder(half_beat=True).

58.1.1 Key Features

- Pairs two 30-bit half-slots into one TAG_HALF_PAIR (0x4) beat
- Buffers one half-slot until a partner arrives
- Preserves record order: flushes a buffered half before forwarding a non-half beat

- Flushes a lone trailing half when the input goes idle (last record never stranded)
 - Forwards full 64-bit slots and raw-escape beats verbatim
 - Bit-exact to the compressor Python golden model for gap-free record streams
 - Simple valid/ready handshake on both ports
-

58.2 Module Purpose

Trace-capture bandwidth is precious. The compressor already reduces most records to a single 64-bit beat, but records that also fit in 30 bits can be paired two-to-a-beat. This packer performs that pairing with a one-slot holding buffer, emitting a combined beat when a second eligible half arrives while preserving strict record order for everything that cannot be paired.

Use Cases: - Maximizing trace history stored in a fixed capture buffer - Reducing MonBus write bandwidth to memory in compressed-capture builds - Matching the software decoder's half-beat encoding exactly (cosim parity)

Key Benefit: Breaks the compressor's 1-beat-per-record floor without changing record semantics — pairing is opportunistic and always order-preserving, so it is never wrong, only occasionally less-packed under input bubbles.

58.3 Parameters

This module has no parameters.

58.4 Port Groups

58.4.1 Clock and Reset

Port	Direction	Width	Description
clk	input	1	Clock
rst_n	input	1	Active-low asynchronous reset

58.4.2 Input — Beats from the Compressor

Port	Direction	Width	Description
in_valid	input	1	Input beat valid
in_ready	output	1	Packer ready to accept the input beat
in_slot	input	64	Full 64-bit slot (tier-1 slot or one of a raw record's 3 beats)
in_half_valid	input	1	Set when in_slot's record also fits a 30-bit half-slot
in_half_slot	input	30	The 30-bit half-slot for the current record

58.4.3 Output — Packed Beats to the Write FIFO

Port	Direction	Width	Description
out_valid	output	1	Output beat valid
out_ready	input	1	Downstream ready
out_slot	output	64	Packed beat (paired half-slots, forwarded full slot, or half + NOP flush)

58.5 Functional Description

58.5.1 Beat Layout

The paired beat matches `monbus_compressor.py`:

```
TAG_HALF_PAIR beat = {tag[63:60]=0x4, slotA[59:30], slotB[29:0]}
half-slot          = {sub[29:28], idx[27:23], delta_ts[22:13],
data[12:0]}
NOP slot           = 30'd0    (sub == HSUB_NOP)
```

58.5.2 Case Decode

Five mutually-considered cases drive the packer combinatorially, based on `in_valid`, `in_half_valid`, and whether a half is buffered (`r_pend_valid`):

Signal	Condition	Action
pair_now	half arrives, one already buffered	Emit the pair {TAG, pend, in_half}, consume input on accept
buffer_now	half arrives, none buffered	Latch it, emit nothing, always consume input
fwd_now	non-half, none buffered	Forward in_slot verbatim, consume input on accept
flush_fwd	non-half, one buffered	Emit the lone buffered half (paired with NOP) first, hold the input
idle_flush	input idle, one buffered	Emit the lone trailing half (paired with NOP)

out_valid asserts for any of pair_now, fwd_now, flush_fwd, or idle_flush.

58.5.3 Handshake and Ordering

in_ready is asserted unconditionally for buffer_now (no output is produced that cycle), and follows out_ready for pair_now/fwd_now. For flush_fwd, in_ready is held low: the buffered half must be flushed first, so the non-half input is consumed the next cycle as fwd_now. This is what preserves record order — a full slot can never jump ahead of a still-buffered earlier half.

58.5.4 Pending-Slot Register

r_pend_valid / r_pend_slot hold the first-of-pair half. The register sets on buffer_now and clears when the buffered half is consumed — either paired (pair_now) or flushed (flush_fwd / idle_flush) — gated by out_ready.

58.5.5 Bit-Exactness

The golden model flushes a lone trailing half exactly once at end-of-stream. This packer flushes whenever its input is idle with a half buffered, which is identical for a gap-free record stream (the cosim drives records back-to-back, then idles once). A mid-stream input bubble would flush early — harmless (slightly less packing), never wrong.

58.6 Usage Example

The packer is instantiated inside `monbus_group_core` under the `HALF_BEAT_EN==1` generate branch:

```
monbus_halfbeat_packer u_packer (  
    .clk            (axi_aclk),  
    .rst_n          (axi_aresetn),  
    .in_valid        (comp_out_valid),  
    .in_ready        (comp_out_ready),  
    .in_slot         (comp_out_slot),  
    .in_half_valid   (comp_out_half_valid),  
    .in_half_slot    (comp_out_half_slot),  
    .out_valid        (comp_wr_valid),  
    .out_ready        (write_fifo_wr_ready),  
    .out_slot         (comp_wr_data)  
);
```

When `HALF_BEAT_EN==0`, the compressor drives the write FIFO directly (the committed, timing-closed path) and this packer is not elaborated.

58.7 Design Notes

58.7.1 Requires the Compressor

Half-beat packing only makes sense downstream of the compressor, so `HALF_BEAT_EN==1` requires `USE_COMPRESSION==1`. In raw-only builds neither block exists.

58.7.2 Order Preservation Is Non-Negotiable

The `flush_fwd` hold is the crux of correctness: without it, a full slot could be emitted while an earlier half still sits buffered, reordering the record stream. The single-cycle hold guarantees the buffered half is written first.

58.7.3 One-Slot Buffer Only

The packer holds at most one pending half. It never accumulates more than two records' worth of state, keeping it cheap and its latency bounded to a single beat.

58.8 Related Modules

58.8.1 Used By

- **monbus_group_core.sv** — instantiates the packer in the HALF_BEAT_EN==1 branch

58.8.2 Uses

- **reset_defs.svh** — reset macros (ALWAYS_FF_RST, RST_ASSERTED)

58.8.3 See Also

- **monbus_compressor.sv** — upstream compressor supplying the half sideband
 - **monbus_group_core.sv** — capture core that owns the compressed write path
-

58.9 References

58.9.1 Source Code

- RTL: rtl/amba/monitor/monbus_halfbeat_packer.sv
- Golden model: monbus_compressor.py (Encoder(half_beat=True))

58.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - Compressor: docs/markdown/RTLAmba/monbus_compressor.md
 - Group Core: docs/markdown/RTLAmba/monbus_group_core.md
-

Last Updated: 2026-07-15

58.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

59 Monitor Transaction CAM

Module: monitor_trans_cam.sv **Location:** rtl/amba/shared/ **Category:** AXI Monitor Infrastructure **Status:** Production Ready

59.1 Overview

monitor_trans_cam is a **multi-port ID CAM with opaque payload storage** used by the axi_monitor_trans_mgr to track outstanding AXI transactions by ID. It replaces the inline parallel-match logic that previously lived in axi_monitor_trans_mgr (2026-04-23 revision), moving the keying state and payload storage into a reusable shared module without changing the synthesis shape that closes 100 MHz on the xc7a100t-1 Artix-7.

The module is intentionally distinct from monbus_cam:

Feature	monbus_cam	monitor_trans_cam
Use case	Caching (compressor template index)	Associative storage (AXI ID tracking)
Eviction	True LRU	None (explicit alloc / free)
Ports	1 lookup	3 lookups + alloc priority encoder
Payload	64 b	Parameterizable (typically \$bits(bus_transaction_t))
Default depth	32	16
Match output	hit + idx + old_data	Three one-hot vectors

59.2 Key Features

- 3 independent ID lookup ports per cycle (addr / data / resp)
- One-hot match vectors (*_match_oh) plus a lowest-index “first” variant for AXI4-write WID-less matching
- Free-slot vector (free_oh) and priority-encoded **3-way mutex alloc** (addr_alloc_oh / data_alloc_oh / resp_alloc_oh)
- Per-slot write port (one-hot enable, separate next-state inputs per slot)

- Per-slot read port (registered current state) exposed for the caller
- (* keep = "true" *) attributes on match/free vectors so synth can't fuse them into downstream update cones (preserves the trans_mgr WNS fix)
- Simulation-only protocol assertions on the caller

59.3 Architecture



monitor_trans_cam block diagram

Source: monitor_trans_cam.mmd

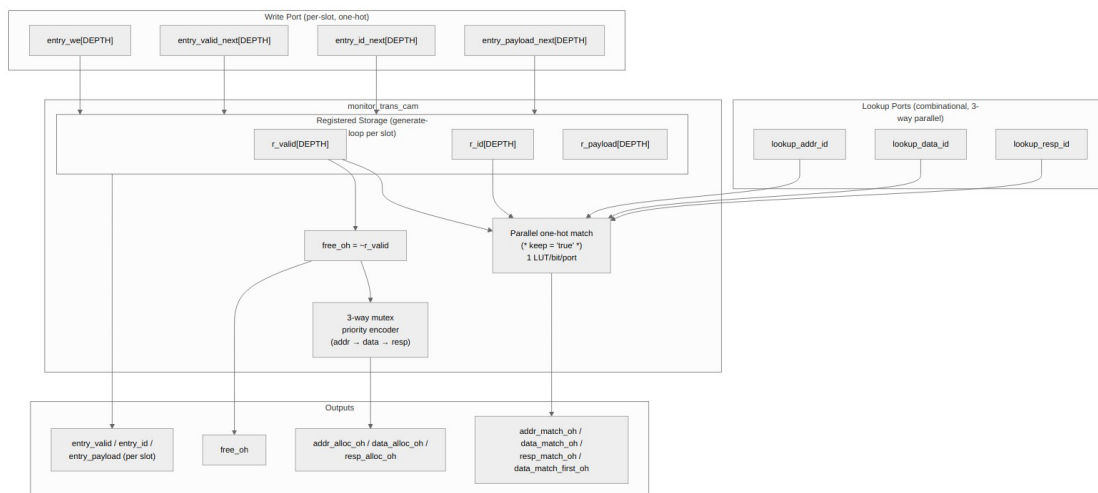


Diagram 26

59.4 Top-level Interface

```
module monitor_trans_cam #(
    parameter int DEPTH          = 16,
    parameter int ID_WIDTH       = 8,
    parameter int PAYLOAD_WIDTH = 128
) (
    input logic clk,
    input logic rst_n,

    // Synchronous clear: on the next edge, invalidate every entry
    // (empty the CAM) so the host can rebase the transaction table
    // without a full rst_n. Takes priority over per-slot entry_we.
    input logic clear,

    // ---- Lookup ports (combinational) ----
    input logic [ID_WIDTH-1:0] lookup_addr_id,
    input logic [ID_WIDTH-1:0] lookup_data_id,
    input logic [ID_WIDTH-1:0] lookup_resp_id,
    output logic [DEPTH-1:0] addr_match_oh,
    output logic [DEPTH-1:0] data_match_oh,
    output logic [DEPTH-1:0] resp_match_oh,
    output logic [DEPTH-1:0] data_match_first_oh,
    // lowest-index match for

data port

    // ---- Free + 3-way alloc priority encoder ----
    output logic [DEPTH-1:0] free_oh,
    input logic addr_wants_alloc,
    input logic data_wants_alloc,
    input logic resp_wants_alloc,
    output logic [DEPTH-1:0] addr_alloc_oh,
    output logic [DEPTH-1:0] data_alloc_oh,
    output logic [DEPTH-1:0] resp_alloc_oh,

    // ---- Per-slot write port ----
    input logic [DEPTH-1:0] entry_we,
    input logic [DEPTH-1:0] entry_valid_next,
    input logic [ID_WIDTH-1:0] entry_id_next
[DEPTH],
    input logic [PAYLOAD_WIDTH-1:0] entry_payload_next
[DEPTH],

    // ---- Per-slot read port (registered) ----
    output logic [DEPTH-1:0] entry_valid,
    output logic [ID_WIDTH-1:0] entry_id
[DEPTH],
```

```

        output logic [PAYLOAD_WIDTH-1:0]          entry_payload
[DEPTH]
);

```

59.5 Why Three Lookup Ports?

The AXI monitor needs to match an arriving transaction beat against the outstanding transaction table along three independent channels in the same cycle:

Lookup port	Purpose
lookup_addr_id	Matches an arriving AW/AR beat to an in-flight transaction (or detects “this is a brand-new transaction”)
lookup_data_id	Matches an arriving R beat (read) or detects orphan-data-without-AW (write, AXI-Lite only)
lookup_resp_id	Matches an arriving B beat (write) to an in-flight transaction

All three lookups go in parallel. The CAM’s internal storage feeds 3 one-hot match vectors, each a 1-LUT-per-bit `valid && (id == lookup_id)` test. With DEPTH=16 that’s 16 small independent cones per port, no chained dependencies.

The `(* keep = "true" *)` attribute on the match vectors prevents Vivado from fusing them into the downstream `axi_monitor_trans_mgr` update logic — without that, the 16 entries’ update cones would fold together into one shared wide function, producing ~12 LUT levels and failing 100 MHz on the -1 part. With the attribute, each entry’s update stays an independent small cone.

59.6 The “First-Match” Variant

For **AXI4 writes**, the data channel has no WID. The `trans_mgr` matches an arriving W beat to the first outstanding write transaction by state predicate (`valid && state ∈ {ADDR_PHASE, DATA_PHASE}` && `cmd_received && !data_completed`), NOT by id. That state predicate is computed *outside* the CAM (it requires fields the CAM doesn’t see, like `state` and `cmd_received`).

The CAM still helps by providing `data_match_first_oh`, which is the lowest-index bit of `data_match_oh` (the id-based match). For AXI4 reads (WID present, id match works), the `trans_mgr` uses `data_match_oh` directly; for AXI4 writes, the `trans_mgr` computes its own first-

match-by-state vector locally and ignores the CAM's. The two paths share the rest of the CAM machinery (free, alloc, write port) so the cost is minimal.

59.7 3-Way Mutex Alloc Priority Encoder

When a new transaction arrives that doesn't match an existing entry, the `trans_mgr` needs to allocate a free slot for it. Up to three independent “want to alloc” requests can fire in the same cycle (`addr`, `data`, `resp`), and each needs a *different* slot — the priority encoder ensures no two phases ever claim the same free slot.

The encoder is purely combinational:

```
remaining_free = free_oh

if addr_wants_alloc:
    addr_alloc_oh = lowest-set-bit(remaining_free)
    remaining_free &= ~addr_alloc_oh

if data_wants_alloc:
    data_alloc_oh = lowest-set-bit(remaining_free)
    remaining_free &= ~data_alloc_oh

if resp_wants_alloc:
    resp_alloc_oh = lowest-set-bit(remaining_free)
    remaining_free &= ~resp_alloc_oh
```

The three outputs are guaranteed to be disjoint (mutex assertion enforces this in simulation). If `wants_alloc` is asserted with no free slot, the corresponding `*_alloc_oh` is '0 — the caller's `wants_alloc` signals are typically gated by free-slot availability anyway, but the CAM doesn't require that.

Priority order is `addr` → `data` → `resp`. AXI4 writes can have all three firing in the same cycle on a tight pipeline, and the order matches the phase order so the slot indices in the table grow in a reasonable chronological order.

59.8 Per-Slot Storage and Write Port

The CAM owns the registered storage:

```
logic                                r_valid    [DEPTH];
logic [ID_WIDTH-1:0]                 r_id       [DEPTH];
logic [PAYLOAD_WIDTH-1:0]             r_payload  [DEPTH];
```

The write port is one-hot per slot:

```
// Per slot i:  if (entry_we[i]) then on next clock edge,
//              r_valid[i]   <= entry_valid_next[i];
//              r_id[i]      <= entry_id_next[i];
//              r_payload[i] <= entry_payload_next[i];
```

To clear a slot, drive `entry_we[i]=1` with `entry_valid_next[i]=0`. The payload and id get written too (with whatever the caller drives) but the valid bit determines whether subsequent lookups see the entry.

The per-slot updates live in a generate loop of independent `always_ff` blocks, one per slot. Each block depends only on local one-hot bits and shared inputs — synth cannot fuse them across slots. The result is DEPTH parallel small update cones rather than one giant shared one.

59.9 Caller Pattern (`axi_monitor_trans_mgr`)

The `trans_mgr` uses the CAM's outputs to compute per-slot next-state in a generate-loop `always_comb`, then drives the write port:

```
For each slot i (combinational):
    next_payload = current_payload  (default: hold)
    next_we      = 0

    if addr_alloc_oh[i]:
        # New transaction starting in addr phase
        next_payload = initialize from cmd_*
        next_we = 1
        next_id = cmd_id

    elif addr_update_oh[i] && cmd_handshake:
        # Existing entry getting addr fields updated
        next_payload.cmd_received = 1
        next_payload.addr_timestamp = timestamp
        next_we = 1

    ... data and resp phases similar ...

    if can_cleanup[i]:
        next_payload.valid = 0
        next_we = 1

    if event_reported_flag[i] && !current_payload.event_reported:
        next_payload.event_reported = 1
        next_we = 1
```


Drive `entry_we[i]`, `entry_valid_next[i]`, `entry_payload_next[i]` for the CAM.

The CAM's `entry_valid` output mirrors the payload's `.valid` field — the `trans_mgr` writes them in sync atomically. Lookups use `entry_valid` (the CAM's view) so the match logic stays tight.

59.10 Configuration

Parameter	Default	Notes
DEPTH	16	Maximum outstanding transactions. Asserts in TBs require ≥ 4 .
ID_WIDTH	8	AXI ID width. Typically 4–8 for AXI4.
PAYLOAD_WIDTH	128	Caller's opaque payload. <code>trans_mgr</code> uses <code>\$bits(bus_transaction_t)</code> (~285 bits).

The `trans_mgr` instantiates with `PAYLOAD_WIDTH = $bits(bus_transaction_t)`, storing the full struct (including the redundant `id` field — the CAM's `entry_id` and `payload.id` are written atomically so they always agree).

59.11 Synthesis Notes

Key design choices to preserve the `trans_mgr`'s 2026-04-23 WNS fix:

Construct	Rationale
<code>(* keep = "true" *) on *_match_oh, free_oh</code>	Anchors the match vectors so Vivado doesn't fuse them into downstream update cones
Per-slot generate-loop <code>always_ff</code>	DEPTH independent small update cones instead of one shared cone
Combinational lookup + commit-in-one-cycle write	Caller can drive lookup → update without an extra pipeline stage
Position-IS-rank NOT used	This CAM has no LRU — slots are

Construct	Rationale
	positionally assigned by the caller; the index has no recency semantic

The combinational depth from input port to CAM output is:

```
lookup_id → match_oh → hit_any (one OR-tree)
                        ↘
                        first_oh (priority encoder)
```

That's ~3–4 LUT levels for DEPTH=16 — comfortably under the 100 MHz budget on the -1 part.

59.12 Test

val/amba/test_monitor_trans_cam.py runs 17 sub-tests covering:

#	Sub-test
1	Reset state — all entries invalid, free_oh = all-1s
2	Single write + lookup — write slot 0 with id=K, lookup hits
3	Independent writes — each slot writable in isolation
4	Three-port lookup — distinct addr/data/resp ids resolve independently
5	data_match_first_oh — duplicate ids: lowest-index wins
6	free_oh tracks valid bit per slot
7	Clear via we=1 + valid_next=0
8	Alloc one phase — addr_wants_alloc → lowest-index free slot
9	Alloc mutex — all three phases want → three distinct slots
10	Alloc partial wants — only some phases want, others zero
11	Alloc when full — no free slots → all

#	Sub-test
	*_alloc_oh = 0
12	Payload preserved across idle cycles
13	Concurrent writes — entry_we to multiple slots in one cycle
14	Lookup miss — id absent from CAM → match_oh = 0
15	Alloc priority cascade — pre-occupy slots, watch alloc skip them
16	ID collision tolerance — two valid slots with same id
17	Random stress — random write/lookup/alloc mix vs Python model

The Python golden model mirrors the RTL state and combinational outputs exactly. At FULL REG_LEVEL the random stress runs 5000 ops per config, and every cycle cross-checks all 9 combinational outputs (3 *_match_oh, first_oh, free_oh, 3 *_alloc_oh, entry_valid) against the model.

```
pytest val/amba/test_monitor_trans_cam.py -v
```

REG_LEVEL parameter sweep: - **GATE**: 1 config (8/64/16) - **FUNC**: 2 configs (+ 4/32/8) - **FULL**: 7 configs (depth 4/8/16/32, id 4/6/8, payload 32/64/128/256)

59.13 Related Modules

Module	Role
axi_monitor_trans_mgr	Sole consumer — uses the CAM for ID tracking and per-slot storage
monbus_cam	Sister CAM for the bulk-trace compressor (LRU, single port, different design intent)
bin/TBClasses/shared/	Test framework utilities